

My Project

1923

Generated by Doxygen 1.14.0

1 Topic Index	1
1.1 Topics	1
2 Data Structure Index	3
2.1 Data Structures	3
3 File Index	5
3.1 File List	5
4 Topic Documentation	9
4.1 API	9
4.1.1 Detailed Description	10
4.1.2 Enumeration Type Documentation	10
4.1.2.1 MQTTErrors	10
4.1.3 Function Documentation	11
4.1.3.1 mqtt_connect()	11
4.1.3.2 mqtt_disconnect()	12
4.1.3.3 mqtt_error_str()	12
4.1.3.4 mqtt_init()	13
4.1.3.5 mqtt_init_reconnect()	15
4.1.3.6 mqtt_ping()	16
4.1.3.7 mqtt_publish()	16
4.1.3.8 mqtt_reconnect()	17
4.1.3.9 mqtt_reinit()	18
4.1.3.10 mqtt_subscribe()	18
4.1.3.11 mqtt_sync()	19
4.1.3.12 mqtt_unsubscribe()	20
4.2 Control Packet Serialization	20
4.2.1 Detailed Description	21
4.2.2 Macro Definition Documentation	21
4.2.2.1 MQTT_PROTOCOL_LEVEL	21
4.2.2.2 MQTT_SUBSCRIBE_REQUEST_MAX_NUM_TOPICS	21
4.2.2.3 MQTT_UNSUBSCRIBE_REQUEST_MAX_NUM_TOPICS	22
4.2.3 Enumeration Type Documentation	22
4.2.3.1 MQTTConnectFlags	22
4.2.3.2 MQTTPublishFlags	22
4.2.4 Function Documentation	23
4.2.4.1 mqtt_pack_connection_request()	23
4.2.4.2 mqtt_pack_disconnect()	24
4.2.4.3 mqtt_pack_fixed_header()	24
4.2.4.4 mqtt_pack_ping_request()	24
4.2.4.5 mqtt_pack_publish_request()	25
4.2.4.6 mqtt_pack_pubxxx_request()	26

4.2.4.7 mqtt_pack_subscribe_request()	26
4.2.4.8 mqtt_pack_unsubscribe_request()	27
4.3 Control Packet Deserialization	27
4.3.1 Detailed Description	29
4.3.2 Enumeration Type Documentation	29
4.3.2.1 MQTTConnackReturnCode	29
4.3.2.2 MQTTControlPacketType	29
4.3.2.3 MQTTSubackReturnCodes	30
4.3.3 Function Documentation	30
4.3.3.1 mqtt_unpack_connack_response()	30
4.3.3.2 mqtt_unpack_fixed_header()	31
4.3.3.3 mqtt_unpack_publish_response()	31
4.3.3.4 mqtt_unpack_pubxxx_response()	32
4.3.3.5 mqtt_unpack_response()	32
4.4 Utilities	33
4.4.1 Detailed Description	34
4.4.2 Macro Definition Documentation	34
4.4.2.1 mqtt_mq_currsz	34
4.4.2.2 mqtt_mq_get	34
4.4.2.3 mqtt_mq_length	34
4.4.3 Enumeration Type Documentation	34
4.4.3.1 MQTTQueuedMessageState	34
4.4.4 Function Documentation	35
4.4.4.1 __mqtt_next_pid()	35
4.4.4.2 __mqtt_puback()	35
4.4.4.3 __mqtt_pubcomp()	35
4.4.4.4 __mqtt_pubrec()	36
4.4.4.5 __mqtt_pubrel()	36
4.4.4.6 __mqtt_rcv()	36
4.4.4.7 __mqtt_send()	37
4.4.4.8 mqtt_mq_clean()	37
4.4.4.9 mqtt_mq_find()	37
4.4.4.10 mqtt_mq_init()	38
4.4.4.11 mqtt_mq_register()	38
4.5 Platform abstraction layer	39
4.5.1 Detailed Description	39
4.5.2 Function Documentation	40
4.5.2.1 mqtt_pal_rcvall()	40
4.5.2.2 mqtt_pal_sendall()	41
4.6 ADC3: Analog-to-Digital Converter with Context	42
4.6.1 Detailed Description	44
4.6.2 Typedef Documentation	44

4.6.2.1 adc_result_t	44
4.6.3 Enumeration Type Documentation	45
4.6.3.1 ADC_channel_t	45
4.6.3.2 ADC_context_t	45
4.6.4 Function Documentation	45
4.6.4.1 ADC_ClearAccumulator()	45
4.6.4.2 ADC_DefineSetPoint()	46
4.6.4.3 ADC_DisableChannelScan()	46
4.6.4.4 ADC_DisableChannelSequencer()	46
4.6.4.5 ADC_DisableChargePump()	47
4.6.4.6 ADC_DisableContinuousConversion()	47
4.6.4.7 ADC_DischargeSampleCapacitor()	48
4.6.4.8 ADC_EnableChannelScan()	48
4.6.4.9 ADC_EnableChannelSequencer()	49
4.6.4.10 ADC_EnableChargePump()	49
4.6.4.11 ADC_EnableContinuousConversion()	49
4.6.4.12 ADC_EnableDoubleSampling()	50
4.6.4.13 ADC_GetAccumulatorValue()	50
4.6.4.14 ADC_GetConversionResult()	51
4.6.4.15 ADC_GetConversionStageStatus()	51
4.6.4.16 ADC_GetCurrentCountofConversions()	52
4.6.4.17 ADC_GetErrorCalculation()	52
4.6.4.18 ADC_GetFilterValue()	53
4.6.4.19 ADC_GetPreviousResult()	53
4.6.4.20 ADC_GetSingleConversion()	53
4.6.4.21 ADC_HasAccumulatorOverflowed()	54
4.6.4.22 ADC_HasErrorCrossedLowerThreshold()	54
4.6.4.23 ADC_HasErrorCrossedUpperThreshold()	55
4.6.4.24 ADC_Initialize()	55
4.6.4.25 ADC_IsConversionDone()	56
4.6.4.26 ADC_LoadAcquisitionRegister()	57
4.6.4.27 ADC_SelectContext()	57
4.6.4.28 ADC_SetActiveClockTuningInterruptHandler()	58
4.6.4.29 ADC_SetADIIInterruptHandler()	58
4.6.4.30 ADC_SetContext1ThresholdInterruptHandler()	59
4.6.4.31 ADC_SetContext2ThresholdInterruptHandler()	60
4.6.4.32 ADC_SetContext3ThresholdInterruptHandler()	60
4.6.4.33 ADC_SetContext4ThresholdInterruptHandler()	61
4.6.4.34 ADC_SetLowerThreshold()	61
4.6.4.35 ADC_SetPrechargeTime()	62
4.6.4.36 ADC_SetRepeatCount()	62
4.6.4.37 ADC_SetStopOnInterrupt()	63

4.6.4.38 ADC_SetUpperThreshold()	63
4.6.4.39 ADC_StartChannelSequencer()	63
4.6.4.40 ADC_StartConversion()	64
4.6.4.41 ADC_StopConversion()	64
5 Data Structure Documentation	67
5.1 B_type Struct Reference	67
5.1.1 Field Documentation	68
5.1.1.1 a_pin	68
5.1.1.2 a_switch	68
5.1.1.3 a_trigger	68
5.1.1.4 a_type	68
5.1.1.5 alt_display	68
5.1.1.6 canbus_online	68
5.1.1.7 day_check	69
5.1.1.8 dim_delay	69
5.1.1.9 display_dim	69
5.1.1.10 display_on	69
5.1.1.11 display_update	69
5.1.1.12 flush	69
5.1.1.13 FM80_charged	69
5.1.1.14 FM80_io	69
5.1.1.15 FM80_online	69
5.1.1.16 fwrev	69
5.1.1.17 LOG	70
5.1.1.18 log	70
5.1.1.19 modbus_online	70
5.1.1.20 mui	70
5.1.1.21 net_balance	70
5.1.1.22 node_id	70
5.1.1.23 once	70
5.1.1.24 one_sec_flag	70
5.1.1.25 pacing	70
5.1.1.26 pv_high	70
5.1.1.27 pv_prev	71
5.1.1.28 pv_update	71
5.1.1.29 run_time	71
5.1.1.30 rx_count	71
5.1.1.31 ten_sec_flag	71
5.2 blob_log Union Reference	71
5.2.1 Field Documentation	72
5.2.1.1 blob	72

5.2.1.2 log	72
5.3 blob_type Struct Reference	72
5.3.1 Field Documentation	72
5.3.1.1 blob	72
5.3.1.2 rx_flag	72
5.3.1.3 tx_flag	72
5.4 C_data Struct Reference	73
5.4.1 Field Documentation	73
5.4.1.1 config_ok	73
5.4.1.2 cstate	73
5.4.1.3 data_count	74
5.4.1.4 data_ok	74
5.4.1.5 data_prev	74
5.4.1.6 id_ok	74
5.4.1.7 light_ok	74
5.4.1.8 M	74
5.4.1.9 mcmd	74
5.4.1.10 modbus_command	74
5.4.1.11 passwd_ok	74
5.4.1.12 req_length	74
5.4.1.13 serial_ok	75
5.4.1.14 tm_ok	75
5.4.1.15 trace	75
5.4.1.16 version_ok	75
5.5 CAN1_RX_FIFO Struct Reference	75
5.5.1 Field Documentation	75
5.5.1.1 channel	75
5.5.1.2 fifoHead	75
5.6 CAN_FIFOREG Struct Reference	76
5.6.1 Field Documentation	76
5.6.1.1 CONH	76
5.6.1.2 CONL	76
5.6.1.3 CONT	76
5.6.1.4 CONU	76
5.6.1.5 STAH	76
5.6.1.6 STAL	76
5.6.1.7 STAT	76
5.6.1.8 STAU	77
5.6.1.9 UA	77
5.7 can_frames_t Struct Reference	77
5.7.1 Field Documentation	77
5.7.1.1 rec_count	77

5.7.1.2 tx_count	77
5.8 CAN_MSG_FIELD Union Reference	77
5.8.1 Detailed Description	78
5.8.2 Field Documentation	78
5.8.2.1 [struct]	78
5.8.2.2 brs	78
5.8.2.3 dlc	78
5.8.2.4 formatType	78
5.8.2.5 frameType	79
5.8.2.6 idType	79
5.8.2.7 msgfields	79
5.9 CAN_MSG_OBJ Struct Reference	79
5.9.1 Field Documentation	79
5.9.1.1 data	79
5.9.1.2 field	80
5.9.1.3 msgId	80
5.10 can_rec_count_t Struct Reference	80
5.10.1 Field Documentation	80
5.10.1.1 rec_count	80
5.10.1.2 rec_flag	80
5.11 EB_data Struct Reference	80
5.11.1 Field Documentation	81
5.11.1.1 alt_display	81
5.11.1.2 bat_amp_whole	81
5.11.1.3 bat_cycles	81
5.11.1.4 bat_energy	81
5.11.1.5 bat_mode	81
5.11.1.6 bat_time	82
5.11.1.7 cc_mode	82
5.11.1.8 checkmark	82
5.11.1.9 crc	82
5.11.1.10 date	82
5.11.1.11 ENac	82
5.11.1.12 ENva	82
5.11.1.13 ENvar	82
5.11.1.14 ENw	82
5.11.1.15 fm80_time	82
5.11.1.16 FMa	83
5.11.1.17 FMbv	83
5.11.1.18 FMpv	83
5.11.1.19 FMw	83
5.11.1.20 loaded	83

5.11.1.21 q84_sequence	83
5.11.1.22 time	83
5.11.1.23 version	83
5.11.1.24 volt_whole	83
5.12 EM_data1 Struct Reference	84
5.12.1 Field Documentation	84
5.12.1.1 al1	84
5.12.1.2 al2	84
5.12.1.3 al3	84
5.12.1.4 hz	85
5.12.1.5 pfl1	85
5.12.1.6 pfl2	85
5.12.1.7 pfl3	85
5.12.1.8 pfsys	85
5.12.1.9 phaseseq	85
5.12.1.10 val1	85
5.12.1.11 val2	85
5.12.1.12 val3	85
5.12.1.13 varl1	85
5.12.1.14 varl2	86
5.12.1.15 varl3	86
5.12.1.16 varsys	86
5.12.1.17 vasys	86
5.12.1.18 vl1l2	86
5.12.1.19 vl1n	86
5.12.1.20 vl2l3	86
5.12.1.21 vl2n	86
5.12.1.22 vl3l1	86
5.12.1.23 vl3n	86
5.12.1.24 vllsys	87
5.12.1.25 vlnsys	87
5.12.1.26 wl1	87
5.12.1.27 wl2	87
5.12.1.28 wl3	87
5.12.1.29 wsys	87
5.13 EM_data2 Struct Reference	87
5.13.1 Field Documentation	88
5.13.1.1 hz	88
5.13.1.2 kvahp	88
5.13.1.3 kvaht	88
5.13.1.4 kvarhnp	88
5.13.1.5 kvarhnt	88

5.13.1.6 kvarhpp	88
5.13.1.7 kvarhpt	88
5.13.1.8 kwhnp	88
5.13.1.9 kwhnt	88
5.13.1.10 kwhpl1	88
5.13.1.11 kwhpl2	89
5.13.1.12 kwhpl3	89
5.13.1.13 kwhpp	89
5.13.1.14 kwhpt	89
5.13.1.15 rhlc	89
5.13.1.16 rhm	89
5.13.1.17 rhmk	89
5.13.1.18 rhmkp	89
5.13.1.19 rhmp	89
5.14 EM_serial Struct Reference	90
5.14.1 Field Documentation	90
5.14.1.1 serial	90
5.14.1.2 year	90
5.15 EM_version Struct Reference	90
5.15.1 Field Documentation	90
5.15.1.1 firmware	90
5.16 M_data Struct Reference	91
5.16.1 Field Documentation	91
5.16.1.1 blink_lock	91
5.16.1.2 boot_code	91
5.16.1.3 config	91
5.16.1.4 crc_calc	91
5.16.1.5 crc_data	91
5.16.1.6 crc_error	92
5.16.1.7 error	92
5.16.1.8 power_on	92
5.16.1.9 pwm_volts	92
5.16.1.10 recv_count	92
5.16.1.11 rx	92
5.16.1.12 send_count	92
5.16.1.13 sends	92
5.16.1.14 stable	92
5.16.1.15 to_error	92
5.17 M_time_data Struct Reference	93
5.17.1 Field Documentation	93
5.17.1.1 clock_2hz	93
5.17.1.2 clock_500ahz	93

5.17.1.3 clock_500hz	93
5.18 mbedtls_context Struct Reference	93
5.18.1 Field Documentation	94
5.18.1.1 ca_crt	94
5.18.1.2 ctr_drbg	94
5.18.1.3 entropy	94
5.18.1.4 net_ctx	94
5.18.1.5 ssl_conf	94
5.18.1.6 ssl_ctx	94
5.19 mqtt_client Struct Reference	95
5.19.1 Detailed Description	96
5.19.2 Field Documentation	96
5.19.2.1 curr	96
5.19.2.2 curr_sz	96
5.19.2.3 error	97
5.19.2.4 inspector_callback	97
5.19.2.5 keep_alive	97
5.19.2.6 mem_size	97
5.19.2.7 mem_start	97
5.19.2.8 mq	97
5.19.2.9 mutex	98
5.19.2.10 number_of_keep_alives	98
5.19.2.11 number_of_timeouts	98
5.19.2.12 pid_lfsr	98
5.19.2.13 publish_response_callback	98
5.19.2.14 publish_response_callback_state	99
5.19.2.15 reconnect_callback	99
5.19.2.16 reconnect_state	99
5.19.2.17 [struct]	99
5.19.2.18 response_timeout	99
5.19.2.19 send_offset	100
5.19.2.20 sockfd	100
5.19.2.21 time_of_last_send	100
5.19.2.22 typical_response_time	100
5.20 mqtt_fixed_header Struct Reference	100
5.20.1 Detailed Description	101
5.20.2 Field Documentation	101
5.20.2.1 control_flags	101
5.20.2.2 control_type	101
5.20.2.3 remaining_length	101
5.21 mqtt_message_queue Struct Reference	102
5.21.1 Detailed Description	103

5.21.2 Field Documentation	103
5.21.2.1 curr	103
5.21.2.2 curr_sz	103
5.21.2.3 mem_end	103
5.21.2.4 mem_start	103
5.21.2.5 queue_tail	104
5.22 mqtt_queued_message Struct Reference	104
5.22.1 Detailed Description	104
5.22.2 Field Documentation	104
5.22.2.1 control_type	104
5.22.2.2 packet_id	105
5.22.2.3 size	105
5.22.2.4 start	105
5.22.2.5 state	105
5.22.2.6 time_sent	105
5.23 mqtt_response Struct Reference	106
5.23.1 Detailed Description	107
5.23.2 Field Documentation	107
5.23.2.1 connack	107
5.23.2.2 [union]	107
5.23.2.3 fixed_header	107
5.23.2.4 pingresp	107
5.23.2.5 puback	107
5.23.2.6 pubcomp	108
5.23.2.7 publish	108
5.23.2.8 pubrec	108
5.23.2.9 pubrel	108
5.23.2.10 suback	108
5.23.2.11 unsuback	108
5.24 mqtt_response_connack Struct Reference	108
5.24.1 Detailed Description	109
5.24.2 Field Documentation	109
5.24.2.1 return_code	109
5.24.2.2 session_present_flag	109
5.25 mqtt_response_pingresp Struct Reference	109
5.25.1 Detailed Description	110
5.25.2 Field Documentation	110
5.25.2.1 dummy	110
5.26 mqtt_response_puback Struct Reference	110
5.26.1 Detailed Description	111
5.26.2 Field Documentation	111
5.26.2.1 packet_id	111

5.27 mqtt_response_pubcomp Struct Reference	111
5.27.1 Detailed Description	111
5.27.2 Field Documentation	111
5.27.2.1 packet_id	111
5.28 mqtt_response_publish Struct Reference	112
5.28.1 Detailed Description	112
5.28.2 Field Documentation	112
5.28.2.1 application_message	112
5.28.2.2 application_message_size	113
5.28.2.3 dup_flag	113
5.28.2.4 packet_id	113
5.28.2.5 qos_level	113
5.28.2.6 retain_flag	113
5.28.2.7 topic_name	113
5.28.2.8 topic_name_size	114
5.29 mqtt_response_pubrec Struct Reference	114
5.29.1 Detailed Description	114
5.29.2 Field Documentation	114
5.29.2.1 packet_id	114
5.30 mqtt_response_pubrel Struct Reference	114
5.30.1 Detailed Description	115
5.30.2 Field Documentation	115
5.30.2.1 packet_id	115
5.31 mqtt_response_suback Struct Reference	115
5.31.1 Detailed Description	116
5.31.2 Friends And Related Symbol Documentation	116
5.31.2.1 mqtt_unpack_suback_response()	116
5.31.3 Field Documentation	116
5.31.3.1 num_return_codes	116
5.31.3.2 packet_id	116
5.31.3.3 return_codes	117
5.32 mqtt_response_unsuback Struct Reference	117
5.32.1 Detailed Description	117
5.32.2 Friends And Related Symbol Documentation	117
5.32.2.1 mqtt_unpack_unsuback_response()	117
5.32.3 Field Documentation	118
5.32.3.1 packet_id	118
5.33 MREG Union Reference	118
5.33.1 Field Documentation	118
5.33.1.1 bytes	118
5.33.1.2 value	118
5.34 MREG32 Union Reference	119

5.34.1 Field Documentation	119
5.34.1.1 bytes	119
5.34.1.2 value	119
5.35 MREG64 Union Reference	119
5.35.1 Field Documentation	119
5.35.1.1 bytes	119
5.35.1.2 value	119
5.36 mx_log_packed_t Struct Reference	120
5.36.1 Field Documentation	120
5.36.1.1 a	120
5.37 mx_logpage_t Struct Reference	120
5.37.1 Field Documentation	120
5.37.1.1 absorb_time	120
5.37.1.2 amp_hours	121
5.37.1.3 amps_peak	121
5.37.1.4 bat_max	121
5.37.1.5 bat_min	121
5.37.1.6 day	121
5.37.1.7 float_time	121
5.37.1.8 kilowatt_hours	121
5.37.1.9 kilowatts_peak	121
5.37.1.10 select	121
5.37.1.11 type	121
5.37.1.12 volts_peak	122
5.38 mx_status_packed_t Struct Reference	122
5.38.1 Field Documentation	122
5.38.1.1 a	122
5.39 PWMDC Union Reference	122
5.39.1 Field Documentation	122
5.39.1.1 bpwm	122
5.39.1.2 lpwm	123
5.40 ringBufS_t Struct Reference	123
5.40.1 Field Documentation	123
5.40.1.1 buf	123
5.40.1.2 count	123
5.40.1.3 head	123
5.40.1.4 tail	123
5.41 spi1_configuration_t Struct Reference	124
5.41.1 Detailed Description	124
5.41.2 Field Documentation	124
5.41.2.1 baud	124
5.41.2.2 con0	124

5.41.2.3 con1	124
5.41.2.4 con2	124
5.41.2.5 operation	125
5.42 spi_link_type Struct Reference	125
5.42.1 Field Documentation	125
5.42.1.1 config	125
5.42.1.2 delay	125
5.42.1.3 int_count	125
5.42.1.4 LCD_DATA	125
5.42.1.5 LCD_TIMER	126
5.42.1.6 SPI_AUX	126
5.42.1.7 SPI_LCD	126
5.42.1.8 txbuf	126
5.43 uart1_status_t Union Reference	126
5.43.1 Detailed Description	126
5.43.2 Field Documentation	126
5.43.2.1 [struct]	126
5.43.2.2 ferr	127
5.43.2.3 oerr	127
5.43.2.4 perr	127
5.43.2.5 reserved	127
5.43.2.6 status	127
5.44 uart2_status_t Union Reference	127
5.44.1 Detailed Description	127
5.44.2 Field Documentation	128
5.44.2.1 [struct]	128
5.44.2.2 ferr	128
5.44.2.3 oerr	128
5.44.2.4 perr	128
5.44.2.5 reserved	128
5.44.2.6 status	128
5.45 uart5_status_t Union Reference	128
5.45.1 Detailed Description	129
5.45.2 Field Documentation	129
5.45.2.1 [struct]	129
5.45.2.2 ferr	129
5.45.2.3 oerr	129
5.45.2.4 perr	129
5.45.2.5 reserved	129
5.45.2.6 status	129
5.46 V_type Struct Reference	129
5.46.1 Field Documentation	130

5.46.1.1 dmt_sosc_flag	130
5.46.1.2 fault_active	130
5.46.1.3 fault_count	130
5.46.1.4 fault_source	130
5.46.1.5 fault_ticks	130
5.46.1.6 modbus_rx	130
5.46.1.7 modbus_tx	130
5.46.1.8 motor_speed	131
5.46.1.9 pacing	131
5.46.1.10 pwm_stop	131
5.46.1.11 pwm_update	131
5.46.1.12 StartTime	131
5.46.1.13 TimeUsed	131
6 File Documentation	133
6.1 batmon.c File Reference	133
6.1.1 Function Documentation	134
6.1.1.1 compute_bm_data()	134
6.1.1.2 DATAEE_ReadByte()	134
6.1.1.3 DATAEE_WriteByte()	135
6.1.1.4 DeviceID_Read()	135
6.1.1.5 get_bm_data()	135
6.1.1.6 get_EEPROM()	136
6.1.1.7 initbm_data()	136
6.1.1.8 update_time()	137
6.1.1.9 Volts_to_SOC()	137
6.1.1.10 wr_bm_data()	138
6.1.2 Variable Documentation	138
6.1.2.1 ac_Wh_daily	138
6.1.2.2 ac_Wh_daily_prev	138
6.1.2.3 EBD	138
6.1.2.4 EBD_update	139
6.1.2.5 pv_Wh_daily	139
6.1.2.6 pv_Wh_daily_prev	139
6.2 batmon.h File Reference	139
6.2.1 Macro Definition Documentation	141
6.2.1.1 BAT_AH	141
6.2.1.2 BAT_CHARGED_V	141
6.2.1.3 BAT_CHARGED_W	141
6.2.1.4 BAT_CYCLES	141
6.2.1.5 BAT_DAY_COUNT	141
6.2.1.6 BAT_EFF_VAL	141

6.2.1.7 BAT_ENERGY	141
6.2.1.8 BAT_ENERGY_LOW	142
6.2.1.9 BAT_LOW_DISC	142
6.2.1.10 BAT_LOW_RECON	142
6.2.1.11 BAT_NIGHT_COUNT	142
6.2.1.12 BAT_OVER_RECON	142
6.2.1.13 BAT_OVER_VOLT	142
6.2.1.14 BAT_RUN_MAX	142
6.2.1.15 BAT_UNDER_VOLT	142
6.2.1.16 BAT_VOLT	142
6.2.1.17 BM_CM	142
6.2.1.18 BM_EEPROM_VER	143
6.2.1.19 BM_UPDATE	143
6.2.1.20 BM_UPDATE_RUN	143
6.2.1.21 BVSOC_SLOTS	143
6.2.1.22 CHK_DAY_TIME	143
6.2.1.23 IDLE_DRAIN	143
6.2.1.24 INV_EFF_VAL	143
6.2.1.25 LOG_VARS	143
6.2.1.26 MAX_ALT_DIS	143
6.2.1.27 TEN_SEC_HOUR	143
6.2.2 Typedef Documentation	144
6.2.2.1 device_id_address_t	144
6.2.2.2 device_id_data_t	144
6.2.2.3 EB_data	144
6.2.3 Function Documentation	144
6.2.3.1 compute_bm_data()	144
6.2.3.2 DATAEE_ReadByte()	145
6.2.3.3 DATAEE_WriteByte()	145
6.2.3.4 DeviceID_Read()	145
6.2.3.5 get_bm_data()	146
6.2.3.6 get_EEPROM()	146
6.2.3.7 initbm_data()	146
6.2.3.8 run_day_to_night()	147
6.2.3.9 run_night_to_day()	148
6.2.3.10 update_time()	148
6.2.3.11 Volts_to_SOC()	148
6.2.3.12 wr_bm_data()	149
6.2.4 Variable Documentation	149
6.2.4.1 ac_Wh_daily	149
6.2.4.2 ac_Wh_daily_prev	150
6.2.4.3 BVSOC_TABLE	150

6.2.4.4 can_newtime	150
6.2.4.5 EBD	150
6.2.4.6 EBD_update	150
6.2.4.7 log_format	150
6.2.4.8 pv_Wh_daily	150
6.2.4.9 pv_Wh_daily_prev	150
6.3 batmon.h	151
6.4 CAN-Examples/canfilter.c File Reference	152
6.4.1 Function Documentation	153
6.4.1.1 main()	153
6.5 CAN-Examples/canreceive.c File Reference	153
6.5.1 Function Documentation	153
6.5.1.1 main()	153
6.6 CAN-Examples/cantransmit.c File Reference	154
6.6.1 Function Documentation	154
6.6.1.1 main()	154
6.7 CAN-Examples/README.md File Reference	154
6.8 canfd.c File Reference	154
6.8.1 Function Documentation	155
6.8.1.1 Can1FIFO1NotEmptyHandler()	155
6.8.1.2 can_blob_get()	156
6.8.1.3 can_blob_set()	156
6.8.1.4 can_fd_lcd_mirror()	156
6.8.1.5 can_fd_tx()	157
6.8.1.6 can_mirror_print()	157
6.8.1.7 can_setup()	158
6.8.1.8 get_blob_rx()	158
6.8.1.9 get_blob_tx()	159
6.8.1.10 TXQNotFullHandler()	159
6.8.2 Variable Documentation	159
6.8.2.1 Byte1	159
6.8.2.2 Byte2	159
6.8.2.3 can_packets	159
6.8.2.4 can_rec_count	159
6.8.2.5 msg	160
6.8.2.6 [union]	160
6.8.2.7 rxMsgData	160
6.8.2.8 [struct]	160
6.8.2.9 Word	160
6.9 canfd.h File Reference	160
6.9.1 Macro Definition Documentation	162
6.9.1.1 CAN_ERROR_BUF	162

6.9.1.2 CAN_HIGH_BUF	162
6.9.1.3 CAN_INFO_BUF	162
6.9.1.4 CAN_LOW_BUF	162
6.9.1.5 CAN_MIRRORS	162
6.9.1.6 CAN_REC_BUFFERS	163
6.9.1.7 CAN_RX_TRIES	163
6.9.1.8 CANFD_BYTES	163
6.9.1.9 CANFD_NBUF	163
6.9.1.10 DATA_DEBUG	163
6.9.1.11 EMON_CO	163
6.9.1.12 EMON_DA	163
6.9.1.13 EMON_DM	163
6.9.1.14 EMON_DS	163
6.9.1.15 EMON_ER	163
6.9.1.16 EMON_M	164
6.9.1.17 EMON_MR	164
6.9.1.18 EMON_SL	164
6.9.1.19 EMON_SU	164
6.9.1.20 EMON_TM	164
6.9.1.21 LCD_BUF_SIZ	164
6.9.1.22 MIRROR_BUF	164
6.9.1.23 NO_NODE_ID	164
6.9.1.24 NODE_ID_SHIFT	164
6.9.1.25 USE_FD	164
6.9.2 Function Documentation	165
6.9.2.1 Can1FIFO1NotEmptyHandler()	165
6.9.2.2 can_blob_get()	165
6.9.2.3 can_blob_set()	165
6.9.2.4 can_fd_lcd_mirror()	165
6.9.2.5 can_fd_tx()	166
6.9.2.6 can_mirror_print()	166
6.9.2.7 can_setup()	167
6.9.2.8 get_blob_rx()	167
6.9.2.9 get_blob_tx()	168
6.9.2.10 TXQNotFullHandler()	168
6.9.3 Variable Documentation	168
6.9.3.1 can_newtime	168
6.9.3.2 can_packets	168
6.9.3.3 can_rec_count	168
6.9.3.4 can_timer	168
6.9.3.5 info_buffer	168
6.9.3.6 log_buffer	169

6.9.3.7 msg	169
6.9.3.8 rxMsgData	169
6.10 canfd.h	169
6.11 dio.c File Reference	170
6.11.1 Function Documentation	171
6.11.1.1 all_relays_off()	171
6.11.1.2 button_press_check()	171
6.11.1.3 init_all_switch()	171
6.12 dio.h File Reference	172
6.12.1 Macro Definition Documentation	173
6.12.1.1 CHARGER_RELAY_OFF	173
6.12.1.2 CHARGER_RELAY_ON	173
6.12.1.3 D_SW_A_PIN	173
6.12.1.4 D_SW_L_PIN	173
6.12.1.5 D_SW_M_PIN	174
6.12.1.6 DAY_RELAY_OFF	174
6.12.1.7 DAY_RELAY_ON	174
6.12.1.8 debounce_time	174
6.12.1.9 NIGHT_RELAY_OFF	174
6.12.1.10 NIGHT_RELAY_ON	174
6.12.1.11 SW_NC	174
6.12.1.12 SW_NO	174
6.12.2 Enumeration Type Documentation	174
6.12.2.1 D_SW	174
6.12.3 Function Documentation	175
6.12.3.1 all_relays_off()	175
6.12.3.2 button_press_check()	175
6.12.3.3 init_all_switch()	175
6.12.4 Variable Documentation	176
6.12.4.1 sw_contact_types	176
6.13 dio.h	176
6.14 eadog.c File Reference	177
6.14.1 Macro Definition Documentation	178
6.14.1.1 DONE_DELAY	178
6.14.1.2 E_TRACE	178
6.14.2 Function Documentation	179
6.14.2.1 check_lcd_dim()	179
6.14.2.2 clear_lcd_done()	179
6.14.2.3 eaDogM_ClearRow()	179
6.14.2.4 eaDogM_Scroll_String()	180
6.14.2.5 eaDogM_Scroll_Task()	180
6.14.2.6 eaDogM_SetPos()	181

6.14.2.7 eaDogM_WriteByteToCGRAM()	181
6.14.2.8 eaDogM_WriteChr()	181
6.14.2.9 eaDogM_WriteCommand()	181
6.14.2.10 eaDogM_WriteIntAtPos()	182
6.14.2.11 eaDogM_WriteString()	182
6.14.2.12 eaDogM_WriteStringAtPos()	182
6.14.2.13 init_display()	183
6.14.2.14 no_dma_set_lcd()	184
6.14.2.15 send_lcd_cmd_dma()	185
6.14.2.16 send_lcd_data_dma()	185
6.14.2.17 send_lcd_pos_dma()	186
6.14.2.18 set_lcd_dim()	187
6.14.2.19 spi_rec_done()	187
6.14.2.20 start_lcd()	187
6.14.2.21 wait_lcd_check()	188
6.14.2.22 wait_lcd_done()	188
6.14.2.23 wait_lcd_set()	188
6.14.3 Variable Documentation	189
6.14.3.1 spi_link	189
6.15 eadog.h File Reference	189
6.15.1 Macro Definition Documentation	191
6.15.1.1 DIM_DELAY	191
6.15.1.2 eaDogM_Cls	192
6.15.1.3 EADOGM_CMD_CGRAM_ADDR	192
6.15.1.4 EADOGM_CMD_CLR	192
6.15.1.5 EADOGM_CMD_CURSOR_OFF	192
6.15.1.6 EADOGM_CMD_CURSOR_ON	192
6.15.1.7 EADOGM_CMD_DDRAM_ADDR	192
6.15.1.8 EADOGM_CMD_DISPLAY_OFF	192
6.15.1.9 EADOGM_CMD_DISPLAY_ON	192
6.15.1.10 EADOGM_CMD_SELECT_R0	192
6.15.1.11 EADOGM_CMD_SELECT_R1	192
6.15.1.12 EADOGM_CMD_SET_TABLE2	193
6.15.1.13 EADOGM_COLSPAN	193
6.15.1.14 eaDogM_CursorOff	193
6.15.1.15 eaDogM_CursorOn	193
6.15.1.16 eaDogM_DisplayOff	193
6.15.1.17 eaDogM_DisplayOn	193
6.15.1.18 LCD0	193
6.15.1.19 LCD1	193
6.15.1.20 LCD2	194
6.15.1.21 LCD3	194

6.15.1.22 LCD_CLEAR_HOME	194
6.15.1.23 LCD_CMD_BRI	194
6.15.1.24 LCD_CMD_CLR	194
6.15.1.25 LCD_CMD_CONT	194
6.15.1.26 LCD_CMD_HOME	194
6.15.1.27 LCD_CMD_MASK	194
6.15.1.28 LCD_CMD_OFF	194
6.15.1.29 LCD_CMD_ON	194
6.15.1.30 LCD_CMD_SET	195
6.15.1.31 LCD_PWR_DELAY	195
6.15.1.32 LSB	195
6.15.1.33 max_strlen	195
6.15.1.34 NHD_BL_HIGH	195
6.15.1.35 NHD_BL_LOW	195
6.15.1.36 NHD_BL_MED	195
6.15.1.37 NHD_BL_OFF	195
6.15.1.38 NHD_CMD	195
6.15.1.39 NHD_CONT	195
6.15.1.40 NHD_L_DELAY	196
6.15.1.41 NHD_POS	196
6.15.1.42 NHD_S_DELAY	196
6.15.1.43 NHD_T_DELAY	196
6.15.1.44 NSB	196
6.15.1.45 USE_LCD_DMA	196
6.15.1.46 USEMCC_SPI	196
6.15.2 Function Documentation	196
6.15.2.1 can_fd_lcd_mirror()	196
6.15.2.2 check_lcd_dim()	197
6.15.2.3 clear_lcd_done()	197
6.15.2.4 eaDogM_ClearRow()	198
6.15.2.5 eaDogM_Scroll_String()	198
6.15.2.6 eaDogM_Scroll_Task()	198
6.15.2.7 eaDogM_SetPos()	199
6.15.2.8 eaDogM_WriteByteToCGRAM()	199
6.15.2.9 eaDogM_WriteChr()	199
6.15.2.10 eaDogM_WriteCommand()	200
6.15.2.11 eaDogM_WriteIntAtPos()	200
6.15.2.12 eaDogM_WriteString()	200
6.15.2.13 eaDogM_WriteStringAtPos()	201
6.15.2.14 init_display()	202
6.15.2.15 no_dma_set_lcd()	203
6.15.2.16 send_lcd_cmd_dma()	203

6.15.2.17	send_lcd_data_dma()	204
6.15.2.18	send_lcd_pos_dma()	205
6.15.2.19	set_lcd_dim()	205
6.15.2.20	spi_rec_done()	206
6.15.2.21	start_lcd()	206
6.15.2.22	wait_lcd_check()	206
6.15.2.23	wait_lcd_done()	207
6.15.2.24	wait_lcd_set()	207
6.15.2.25	wdtdelay()	207
6.16	eadog.h	208
6.17	linux/matesocketcan/.dep.inc File Reference	210
6.18	linux/matesocketcan/build/Debug/GNU-Linux/_ext/5c0/raw.githubusercontent.com_linux-can_can- utils_master_canfdtest.o.d File Reference	210
6.19	linux/matesocketcan/build/Release/GNU-Linux/_ext/5c0/raw.githubusercontent.com_linux-can_can- utils_master_canfdtest.o.d File Reference	210
6.20	linux/matesocketcan/mqtt.c File Reference	210
6.20.1	Detailed Description	210
6.21	linux/matesocketcan/mqtt.h File Reference	211
6.21.1	Detailed Description	215
6.21.2	Macro Definition Documentation	215
6.21.2.1	__ALL_MQTT_ERRORS	215
6.21.2.2	__mqtt_packed_cstrlen	215
6.21.2.3	GENERATE_ENUM	216
6.21.2.4	GENERATE_STRING	216
6.21.3	Function Documentation	216
6.21.3.1	__mqtt_pack_str()	216
6.21.3.2	__mqtt_pack_uint16()	217
6.21.3.3	__mqtt_ping()	218
6.21.3.4	__mqtt_unpack_uint16()	218
6.22	mqtt.h	219
6.23	linux/matesocketcan/mqtt_pal.c File Reference	225
6.23.1	Detailed Description	225
6.24	linux/matesocketcan/mqtt_pal.h File Reference	225
6.24.1	Detailed Description	226
6.25	mqtt_pal.h	226
6.26	linux/matesocketcan/mqtt_pub.c File Reference	228
6.26.1	Function Documentation	229
6.26.1.1	client_refresher()	229
6.26.1.2	exit_example()	230
6.26.1.3	mqtt_check()	230
6.26.1.4	mqtt_exit()	230
6.26.1.5	mqtt_socket()	231
6.26.1.6	publish_callback()	231

6.26.2 Variable Documentation	232
6.26.2.1 addr	232
6.26.2.2 client	232
6.26.2.3 client_daemon	232
6.26.2.4 client_id	232
6.26.2.5 connect_flags	232
6.26.2.6 port	232
6.26.2.7 recvbuf	232
6.26.2.8 sendbuf	232
6.26.2.9 sockfd	232
6.26.2.10 topic	232
6.27 linux/matesocketcan/mqtt_pub.h File Reference	233
6.27.1 Macro Definition Documentation	234
6.27.1.1 ADDR_MQTT	234
6.27.1.2 DATA_MQTT_SOLAR	234
6.27.1.3 MQTT_VERSION	234
6.27.2 Function Documentation	234
6.27.2.1 client_refresher()	234
6.27.2.2 exit_example()	235
6.27.2.3 mqtt_check()	236
6.27.2.4 mqtt_exit()	236
6.27.2.5 mqtt_socket()	236
6.27.2.6 publish_callback()	237
6.28 mqtt_pub.h	238
6.29 linux/matesocketcan/nbproject/private/c_standard_headers_indexer.c File Reference	238
6.30 linux/mqtt_cl/nbproject/private/c_standard_headers_indexer.c File Reference	239
6.31 linux/matesocketcan/nbproject/private/cpp_standard_headers_indexer.cpp File Reference	239
6.32 linux/mqtt_cl/nbproject/private/cpp_standard_headers_indexer.cpp File Reference	240
6.33 linux/matesocketcan/pge.h File Reference	241
6.33.1 Macro Definition Documentation	242
6.33.1.1 E_DAYS	242
6.33.1.2 E_MONTH	242
6.33.1.3 E_PER_DAY	242
6.33.1.4 E_PER_HOUR	242
6.33.1.5 G_MONTH	242
6.33.1.6 G_PER_DAY	242
6.33.1.7 G_PER_HOUR	242
6.33.1.8 PGE_ZERO	242
6.34 pge.h	243
6.35 linux/matesocketcan/templates/bearssl_sockets.h File Reference	243
6.35.1 Function Documentation	244
6.35.1.1 close_socket()	244

6.35.1.2 <code>open_nb_socket()</code>	244
6.36 <code>bearssl_sockets.h</code>	244
6.37 <code>linux/matesocketcan/templates/bio_sockets.h</code> File Reference	246
6.37.1 Function Documentation	246
6.37.1.1 <code>open_nb_socket()</code>	246
6.38 <code>bio_sockets.h</code>	247
6.39 <code>linux/matesocketcan/templates/mbedtls_sockets.h</code> File Reference	247
6.39.1 Macro Definition Documentation	248
6.39.1.1 <code>MBEDTLS_NET_POLL_READ</code>	248
6.39.1.2 <code>MBEDTLS_NET_POLL_WRITE</code>	248
6.39.2 Function Documentation	248
6.39.2.1 <code>cert_verify_failed()</code>	248
6.39.2.2 <code>failed()</code>	248
6.39.2.3 <code>mbedtls_net_poll()</code>	249
6.39.2.4 <code>open_nb_socket()</code>	249
6.40 <code>mbedtls_sockets.h</code>	249
6.41 <code>linux/matesocketcan/templates/openssl_sockets.h</code> File Reference	251
6.41.1 Function Documentation	252
6.41.1.1 <code>open_nb_socket()</code>	252
6.42 <code>openssl_sockets.h</code>	252
6.43 <code>linux/matesocketcan/templates/posix_sockets.h</code> File Reference	253
6.43.1 Function Documentation	254
6.43.1.1 <code>open_nb_socket_mqtt()</code>	254
6.44 <code>posix_sockets.h</code>	255
6.45 <code>linux/raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c</code> File Reference	256
6.45.1 Macro Definition Documentation	258
6.45.1.1 <code>_DEFAULT_SOURCE</code>	258
6.45.1.2 <code>ADDRESS</code>	258
6.45.1.3 <code>CAN_FULL_BUFFER</code>	258
6.45.1.4 <code>CAN_MSG_COUNT</code>	258
6.45.1.5 <code>CAN_MSG_ID_PING</code>	258
6.45.1.6 <code>CAN_MSG_ID_PING_X</code>	258
6.45.1.7 <code>CAN_MSG_ID_PONG</code>	258
6.45.1.8 <code>CAN_MSG_LEN</code>	259
6.45.1.9 <code>CAN_MSG_WAIT</code>	259
6.45.1.10 <code>CAN_TM_TIME</code>	259
6.45.1.11 <code>CLIENTID</code>	259
6.45.1.12 <code>DAY_SEC</code>	259
6.45.1.13 <code>E_DAYS</code>	259
6.45.1.14 <code>E_MONTH</code>	259
6.45.1.15 <code>E_PER_DAY</code>	259
6.45.1.16 <code>E_PER_HOUR</code>	259

6.45.1.17 EMON_CO	259
6.45.1.18 EMON_DA	260
6.45.1.19 EMON_ER	260
6.45.1.20 EMON_SH	260
6.45.1.21 EMON_SL	260
6.45.1.22 EMON_SU	260
6.45.1.23 EMON_TM	260
6.45.1.24 G_MONTH	260
6.45.1.25 G_PER_DAY	260
6.45.1.26 G_PER_HOUR	260
6.45.1.27 HR_SEC	260
6.45.1.28 LOG_TO_FILE	261
6.45.1.29 LOG_VERSION	261
6.45.1.30 MQTT_TIMEOUT	261
6.45.1.31 MQTT_VERSION	261
6.45.1.32 PGE_ZERO	261
6.45.1.33 QOS	261
6.45.1.34 SPACING_USEC	261
6.45.1.35 TIMEOUT	261
6.45.1.36 TOPIC_P	261
6.45.1.37 TOPIC_S	261
6.45.2 Function Documentation	262
6.45.2.1 connlost()	262
6.45.2.2 current_timestamp()	262
6.45.2.3 delivered()	262
6.45.2.4 log_time()	263
6.45.2.5 main()	263
6.45.2.6 msgarrvd()	263
6.45.2.7 timer_callback()	264
6.45.3 Variable Documentation	264
6.45.3.1 acamps	264
6.45.3.2 acenergy	265
6.45.3.3 acvolts	265
6.45.3.4 bamps	265
6.45.3.5 bat_energy_kw	265
6.45.3.6 bat_energy_scaled	265
6.45.3.7 benergy	265
6.45.3.8 bvolts	265
6.45.3.9 ccmode	265
6.45.3.10 client	265
6.45.3.11 conn_opts	265
6.45.3.12 data_buffer	266

6.45.3.13 day_time	266
6.45.3.14 deliveredtoken	266
6.45.3.15 fout	266
6.45.3.16 full_buffer	266
6.45.3.17 gasenergy	266
6.45.3.18 gridin	266
6.45.3.19 gridout	266
6.45.3.20 hour_time	266
6.45.3.21 json	266
6.45.3.22 load	267
6.45.3.23 mtoken	267
6.45.3.24 pamps	267
6.45.3.25 pubmsg	267
6.45.3.26 pvolts	267
6.45.3.27 pwatts	267
6.45.3.28 receivedtoken	267
6.45.3.29 runner	267
6.45.3.30 runtime	267
6.45.3.31 sec_30	267
6.45.3.32 sequence	268
6.45.3.33 solar	268
6.45.3.34 start_time	268
6.45.3.35 token	268
6.45.3.36 watergal	268
6.46 linux/mqtt_cl/main.c File Reference	268
6.46.1 Function Documentation	269
6.46.1.1 client_refresher()	269
6.46.1.2 exit_example()	269
6.46.1.3 main()	270
6.46.1.4 publish_callback()	270
6.47 main.c File Reference	271
6.47.1 Function Documentation	271
6.47.1.1 main()	271
6.48 mateQ84.X/main.c File Reference	271
6.48.1 Macro Definition Documentation	272
6.48.1.1 CMD_LEN	272
6.48.1.2 E_SAVE	272
6.48.1.3 E_UPDATE	273
6.48.1.4 PACE	273
6.48.1.5 REC_LEN	273
6.48.1.6 REC_LOG_LEN	273
6.48.1.7 REC_STATUS_LEN	273

6.48.2 Enumeration Type Documentation	273
6.48.2.1 state_type	273
6.48.3 Function Documentation	274
6.48.3.1 main()	274
6.48.3.2 run_day_to_night()	275
6.48.3.3 run_night_to_day()	275
6.48.3.4 spinners()	276
6.48.3.5 state_battery_cb()	276
6.48.3.6 state_batteryv_cb()	276
6.48.3.7 state_init_cb()	277
6.48.3.8 state_misc_cb()	277
6.48.3.9 state_mx_log_cb()	278
6.48.3.10 state_mx_status_cb()	278
6.48.3.11 state_panelv_cb()	279
6.48.3.12 state_status_cb()	279
6.48.3.13 state_watts_cb()	279
6.48.3.14 wdtdelay()	280
6.48.4 Variable Documentation	280
6.48.4.1 B	280
6.48.4.2 bat_amp_whole	280
6.48.4.3 buffer	281
6.48.4.4 build_date	281
6.48.4.5 build_time	281
6.48.4.6 can_newtime	281
6.48.4.7 can_timer	281
6.48.4.8 cc_mode	281
6.48.4.9 fw_state	281
6.48.4.10 info_buffer	281
6.48.4.11 log_buffer	281
6.48.4.12 mx_code	281
6.48.4.13 panel_watts	282
6.48.4.14 show_can	282
6.48.4.15 state	282
6.48.4.16 tickCount	282
6.48.4.17 time_show_can	282
6.48.4.18 vf	282
6.48.4.19 volt_fract	282
6.48.4.20 volt_whole	282
6.48.4.21 vw	282
6.49 mateQ84.X/build/default/production/_ext/1472/batmon.p1.d File Reference	284
6.50 mateQ84.X/build/default/production/_ext/1472/canfd.p1.d File Reference	284
6.51 mateQ84.X/build/default/production/_ext/1472/dio.p1.d File Reference	284

6.52	mateQ84.X/build/default/production/_ext/1472/eadog.p1.d File Reference	284
6.53	mateQ84.X/build/default/production/_ext/1472/modbus_master.p1.d File Reference	284
6.54	mateQ84.X/build/default/production/_ext/1472/timers.p1.d File Reference	284
6.55	mateQ84.X/build/default/production/main.p1.d File Reference	284
6.56	mateQ84.X/build/default/production/mcc_generated_files/adc.p1.d File Reference	284
6.57	mateQ84.X/build/default/production/mcc_generated_files/can1.p1.d File Reference	284
6.58	mateQ84.X/build/default/production/mcc_generated_files/clc7.p1.d File Reference	284
6.59	mateQ84.X/build/default/production/mcc_generated_files/clc8.p1.d File Reference	284
6.60	mateQ84.X/build/default/production/mcc_generated_files/device_config.p1.d File Reference	284
6.61	mateQ84.X/build/default/production/mcc_generated_files/dma1.p1.d File Reference	284
6.62	mateQ84.X/build/default/production/mcc_generated_files/interrupt_manager.p1.d File Reference	284
6.63	mateQ84.X/build/default/production/mcc_generated_files/mcc.p1.d File Reference	284
6.64	mateQ84.X/build/default/production/mcc_generated_files/pin_manager.p1.d File Reference	284
6.65	mateQ84.X/build/default/production/mcc_generated_files/spi1.p1.d File Reference	284
6.66	mateQ84.X/build/default/production/mcc_generated_files/tmr0.p1.d File Reference	284
6.67	mateQ84.X/build/default/production/mcc_generated_files/tmr2.p1.d File Reference	284
6.68	mateQ84.X/build/default/production/mcc_generated_files/tmr4.p1.d File Reference	284
6.69	mateQ84.X/build/default/production/mcc_generated_files/tmr5.p1.d File Reference	284
6.70	mateQ84.X/build/default/production/mcc_generated_files/tmr6.p1.d File Reference	284
6.71	mateQ84.X/build/default/production/mcc_generated_files/uart1.p1.d File Reference	284
6.72	mateQ84.X/build/default/production/mcc_generated_files/uart2.p1.d File Reference	284
6.73	mateQ84.X/build/default/production/mcc_generated_files/uart5.p1.d File Reference	284
6.74	mateQ84.X/build/default/production/mxcmd.p1.d File Reference	284
6.75	mateQ84.X/dist/default/production/doprnt.d File Reference	284
6.76	mateQ84.X/mcc_generated_files/adc.c File Reference	284
6.76.1	Function Documentation	287
6.76.1.1	__interrupt() [1/3]	287
6.76.1.2	__interrupt() [2/3]	287
6.76.1.3	__interrupt() [3/3]	287
6.77	mateQ84.X/mcc_generated_files/adc.h File Reference	288
6.78	adc.h	291
6.79	mateQ84.X/mcc_generated_files/can1.c File Reference	292
6.79.1	Macro Definition Documentation	294
6.79.1.1	BRS_POSN	294
6.79.1.2	DLCToPayloadBytes	294
6.79.1.3	EID_HIGH_MASK	294
6.79.1.4	EID_HIGH_WIDTH	295
6.79.1.5	EID_LOW_MASK	295
6.79.1.6	EID_LOW_POSN	295
6.79.1.7	EID_LOW_WIDTH	295
6.79.1.8	EID_MID_WIDTH	295
6.79.1.9	FDF_POSN	295

6.79.1.10 IDE_POSN	295
6.79.1.11 NUM_OF_RX_FIFO	295
6.79.1.12 PLSIZEToPayloadBytes	295
6.79.1.13 RTR_POSN	296
6.79.1.14 RX_FIFO_MSG_DATA	296
6.79.1.15 SID_HIGH_MASK	296
6.79.1.16 SID_LOW_WIDTH	296
6.79.2 Enumeration Type Documentation	296
6.79.2.1 CAN_RX_FIFO_STATUS	296
6.79.3 Function Documentation	296
6.79.3.1 __interrupt() [1/3]	296
6.79.3.2 __interrupt() [2/3]	297
6.79.3.3 __interrupt() [3/3]	297
6.79.3.4 CAN1_Initialize()	297
6.79.3.5 CAN1_IsBusOff()	298
6.79.3.6 CAN1_IsRxErrorActive()	298
6.79.3.7 CAN1_IsRxErrorPassive()	299
6.79.3.8 CAN1_IsRxErrorWarning()	300
6.79.3.9 CAN1_IsTxErrorActive()	300
6.79.3.10 CAN1_IsTxErrorPassive()	301
6.79.3.11 CAN1_IsTxErrorWarning()	302
6.79.3.12 CAN1_OperationModeGet()	303
6.79.3.13 CAN1_OperationModeSet()	303
6.79.3.14 CAN1_Receive()	304
6.79.3.15 CAN1_ReceivedMessageCountGet()	305
6.79.3.16 CAN1_ReceiveFrom()	305
6.79.3.17 CAN1_RX_FIFO_ResetInfo()	306
6.79.3.18 CAN1_SetBusErrorInterruptHandler()	306
6.79.3.19 CAN1_SetBusWakeUpActivityInterruptHandler()	307
6.79.3.20 CAN1_SetFIFO1NotEmptyHandler()	307
6.79.3.21 CAN1_SetInvalidMessageInterruptHandler()	308
6.79.3.22 CAN1_SetModeChangeInterruptHandler()	308
6.79.3.23 CAN1_SetRxBufferOverFlowInterruptHandler()	309
6.79.3.24 CAN1_SetSystemErrorInterruptHandler()	309
6.79.3.25 CAN1_SetTxAttemptInterruptHandler()	310
6.79.3.26 CAN1_SetTXQNotFullHandler()	311
6.79.3.27 CAN1_Sleep()	311
6.79.3.28 CAN1_Transmit()	312
6.79.3.29 CAN1_TransmitFIFOStatusGet()	313
6.80 mateQ84.X/mcc_generated_files/can1.h File Reference	314
6.80.1 Macro Definition Documentation	315
6.80.1.1 CAN1_TX_FIFO2	315

6.80.1.2 CAN1_TX_FIFO3	315
6.80.1.3 CAN1_TX_TXQ	316
6.80.2 Enumeration Type Documentation	316
6.80.2.1 CAN1_RX_FIFO_CHANNELS	316
6.80.2.2 CAN1_TX_FIFO_CHANNELS	316
6.80.3 Function Documentation	317
6.80.3.1 CAN1_Initialize()	317
6.80.3.2 CAN1_IsBusOff()	317
6.80.3.3 CAN1_IsRxErrorActive()	318
6.80.3.4 CAN1_IsRxErrorPassive()	319
6.80.3.5 CAN1_IsRxErrorWarning()	320
6.80.3.6 CAN1_IsTxErrorActive()	320
6.80.3.7 CAN1_IsTxErrorPassive()	321
6.80.3.8 CAN1_IsTxErrorWarning()	321
6.80.3.9 CAN1_OperationModeGet()	322
6.80.3.10 CAN1_OperationModeSet()	323
6.80.3.11 CAN1_Receive()	324
6.80.3.12 CAN1_ReceivedMessageCountGet()	324
6.80.3.13 CAN1_ReceiveFrom()	325
6.80.3.14 CAN1_SetBusErrorInterruptHandler()	325
6.80.3.15 CAN1_SetBusWakeUpActivityInterruptHandler()	326
6.80.3.16 CAN1_SetFIFO1NotEmptyHandler()	327
6.80.3.17 CAN1_SetFIFO2nullHandler()	327
6.80.3.18 CAN1_SetFIFO3nullHandler()	328
6.80.3.19 CAN1_SetInvalidMessageInterruptHandler()	328
6.80.3.20 CAN1_SetModeChangeInterruptHandler()	329
6.80.3.21 CAN1_SetRxBufferOverflowInterruptHandler()	329
6.80.3.22 CAN1_SetSystemErrorInterruptHandler()	330
6.80.3.23 CAN1_SetTxAttemptInterruptHandler()	331
6.80.3.24 CAN1_SetTXQNotFullHandler()	332
6.80.3.25 CAN1_Sleep()	332
6.80.3.26 CAN1_Transmit()	333
6.80.3.27 CAN1_TransmitFIFOStatusGet()	334
6.81 can1.h	335
6.82 mateQ84.X/mcc_generated_files/can_types.h File Reference	336
6.82.1 Enumeration Type Documentation	338
6.82.1.1 CAN_DLC	338
6.82.1.2 CAN_MSG_OBJ_BRS_MODE	338
6.82.1.3 CAN_MSG_OBJ_FRAME_TYPE	338
6.82.1.4 CAN_MSG_OBJ_ID_TYPE	339
6.82.1.5 CAN_MSG_OBJ_TYPE	339
6.82.1.6 CAN_OP_MODE_STATUS	339

6.82.1.7 CAN_OP_MODES	340
6.82.1.8 CAN_TX_FIFO_STATUS	341
6.82.1.9 CAN_TX_MSG_REQUEST_STATUS	341
6.83 can_types.h	342
6.84 mateQ84.X/mcc_generated_files/clc7.c File Reference	343
6.84.1 Function Documentation	344
6.84.1.1 CLC7_Initialize()	344
6.84.1.2 CLC7_OutputStatusGet()	345
6.85 mateQ84.X/mcc_generated_files/clc7.h File Reference	345
6.85.1 Function Documentation	346
6.85.1.1 CLC7_Initialize()	346
6.85.1.2 CLC7_OutputStatusGet()	347
6.86 clc7.h	347
6.87 mateQ84.X/mcc_generated_files/clc8.c File Reference	348
6.87.1 Function Documentation	348
6.87.1.1 CLC8_Initialize()	348
6.87.1.2 CLC8_OutputStatusGet()	349
6.88 mateQ84.X/mcc_generated_files/clc8.h File Reference	349
6.88.1 Function Documentation	350
6.88.1.1 CLC8_Initialize()	350
6.88.1.2 CLC8_OutputStatusGet()	351
6.89 clc8.h	351
6.90 mateQ84.X/mcc_generated_files/device_config.c File Reference	352
6.91 mateQ84.X/mcc_generated_files/device_config.h File Reference	352
6.91.1 Macro Definition Documentation	353
6.91.1.1 _XTAL_FREQ	353
6.92 device_config.h	353
6.93 mateQ84.X/mcc_generated_files/dma1.c File Reference	354
6.93.1 Function Documentation	355
6.93.1.1 __interrupt() [1/3]	355
6.93.1.2 __interrupt() [2/3]	355
6.93.1.3 __interrupt() [3/3]	355
6.93.1.4 DMA1_DefaultInterruptHandler()	355
6.93.1.5 DMA1_GetDestinationPointer()	356
6.93.1.6 DMA1_GetSourcePointer()	356
6.93.1.7 DMA1_Initialize()	357
6.93.1.8 DMA1_SelectSourceRegion()	357
6.93.1.9 DMA1_SetAbortTrigger()	358
6.93.1.10 DMA1_SetAllInterruptHandler()	358
6.93.1.11 DMA1_SetDestinationAddress()	358
6.93.1.12 DMA1_SetDestinationSize()	359
6.93.1.13 DMA1_SetDMAPriority()	359

6.93.1.14 DMA1_SetORInterruptHandler()	360
6.93.1.15 DMA1_SetSCNTInterruptHandler()	360
6.93.1.16 DMA1_SetSourceAddress()	361
6.93.1.17 DMA1_SetSourceSize()	361
6.93.1.18 DMA1_SetStartTrigger()	362
6.93.1.19 DMA1_StartTransfer()	362
6.93.1.20 DMA1_StartTransferWithTrigger()	362
6.93.1.21 DMA1_StopTransfer()	363
6.93.2 Variable Documentation	363
6.93.2.1 DMA1_AI_InterruptHandler	363
6.93.2.2 DMA1_ORI_InterruptHandler	363
6.93.2.3 DMA1_SCNTI_InterruptHandler	363
6.94 mateQ84.X/mcc_generated_files/dma1.h File Reference	364
6.94.1 Function Documentation	365
6.94.1.1 DMA1_DefaultInterruptHandler()	365
6.94.1.2 DMA1_GetDestinationPointer()	366
6.94.1.3 DMA1_GetSourcePointer()	366
6.94.1.4 DMA1_Initialize()	366
6.94.1.5 DMA1_SelectSourceRegion()	367
6.94.1.6 DMA1_SetAbortTrigger()	368
6.94.1.7 DMA1_SetAllInterruptHandler()	368
6.94.1.8 DMA1_SetDestinationAddress()	368
6.94.1.9 DMA1_SetDestinationSize()	369
6.94.1.10 DMA1_SetDMAPriority()	369
6.94.1.11 DMA1_SetORInterruptHandler()	370
6.94.1.12 DMA1_SetSCNTInterruptHandler()	370
6.94.1.13 DMA1_SetSourceAddress()	371
6.94.1.14 DMA1_SetSourceSize()	371
6.94.1.15 DMA1_SetStartTrigger()	372
6.94.1.16 DMA1_StartTransfer()	372
6.94.1.17 DMA1_StartTransferWithTrigger()	372
6.94.1.18 DMA1_StopTransfer()	373
6.94.2 Variable Documentation	373
6.94.2.1 lcd_dma_buf	373
6.95 dma1.h	374
6.96 mateQ84.X/mcc_generated_files/interrupt_manager.c File Reference	374
6.96.1 Function Documentation	375
6.96.1.1 __interrupt()	375
6.96.1.2 INTERRUPT_Initialize()	375
6.97 mateQ84.X/mcc_generated_files/interrupt_manager.h File Reference	376
6.97.1 Macro Definition Documentation	376
6.97.1.1 INTERRUPT_GlobalInterruptHighDisable	376

6.97.1.2 INTERRUPT_GlobalInterruptHighEnable	376
6.97.1.3 INTERRUPT_GlobalInterruptLowDisable	377
6.97.1.4 INTERRUPT_GlobalInterruptLowEnable	377
6.97.2 Function Documentation	377
6.97.2.1 INTERRUPT_Initialize()	377
6.98 interrupt_manager.h	378
6.99 mateQ84.X/mcc_generated_files/mcc.c File Reference	378
6.99.1 Function Documentation	379
6.99.1.1 OSCILLATOR_Initialize()	379
6.99.1.2 PMD_Initialize()	379
6.99.1.3 SYSTEM_Initialize()	379
6.99.1.4 SystemArbiter_Initialize()	381
6.100 mateQ84.X/mcc_generated_files/mcc.h File Reference	381
6.100.1 Function Documentation	382
6.100.1.1 OSCILLATOR_Initialize()	382
6.100.1.2 PMD_Initialize()	383
6.100.1.3 SYSTEM_Initialize()	383
6.100.1.4 SystemArbiter_Initialize()	385
6.101 mcc.h	385
6.102 mateQ84.X/mcc_generated_files/pin_manager.c File Reference	386
6.102.1 Function Documentation	387
6.102.1.1 __interrupt()	387
6.102.1.2 IOCAF1_DefaultInterruptHandler()	387
6.102.1.3 IOCAF1_ISR()	388
6.102.1.4 IOCAF1_SetInterruptHandler()	388
6.102.1.5 IOCAF2_DefaultInterruptHandler()	388
6.102.1.6 IOCAF2_ISR()	389
6.102.1.7 IOCAF2_SetInterruptHandler()	389
6.102.1.8 IOCAF5_DefaultInterruptHandler()	389
6.102.1.9 IOCAF5_ISR()	390
6.102.1.10 IOCAF5_SetInterruptHandler()	390
6.102.1.11 PIN_MANAGER_Initialize()	390
6.102.2 Variable Documentation	391
6.102.2.1 IOCAF1_InterruptHandler	391
6.102.2.2 IOCAF2_InterruptHandler	392
6.102.2.3 IOCAF5_InterruptHandler	392
6.103 mateQ84.X/mcc_generated_files/pin_manager.h File Reference	392
6.103.1 Macro Definition Documentation	402
6.103.1.1 A_SWITCH_ANS	402
6.103.1.2 A_SWITCH_GetValue	402
6.103.1.3 A_SWITCH_LAT	402
6.103.1.4 A_SWITCH_OD	403

6.103.1.5 A_SWITCH_PORT	403
6.103.1.6 A_SWITCH_ResetPullup	403
6.103.1.7 A_SWITCH_SetAnalogMode	403
6.103.1.8 A_SWITCH_SetDigitalInput	403
6.103.1.9 A_SWITCH_SetDigitalMode	403
6.103.1.10 A_SWITCH_SetDigitalOutput	403
6.103.1.11 A_SWITCH_SetHigh	404
6.103.1.12 A_SWITCH_SetLow	404
6.103.1.13 A_SWITCH_SetOpenDrain	404
6.103.1.14 A_SWITCH_SetPullup	404
6.103.1.15 A_SWITCH_SetPushPull	404
6.103.1.16 A_SWITCH_Toggle	404
6.103.1.17 A_SWITCH_TRIS	404
6.103.1.18 A_SWITCH_WPU	405
6.103.1.19 ANALOG	405
6.103.1.20 CHARGER_ANS	405
6.103.1.21 CHARGER_GetValue	405
6.103.1.22 CHARGER_LAT	405
6.103.1.23 CHARGER_OD	405
6.103.1.24 CHARGER_PORT	405
6.103.1.25 CHARGER_ResetPullup	405
6.103.1.26 CHARGER_SetAnalogMode	405
6.103.1.27 CHARGER_SetDigitalInput	406
6.103.1.28 CHARGER_SetDigitalMode	406
6.103.1.29 CHARGER_SetDigitalOutput	406
6.103.1.30 CHARGER_SetHigh	406
6.103.1.31 CHARGER_SetLow	406
6.103.1.32 CHARGER_SetOpenDrain	406
6.103.1.33 CHARGER_SetPullup	406
6.103.1.34 CHARGER_SetPushPull	407
6.103.1.35 CHARGER_Toggle	407
6.103.1.36 CHARGER_TRIS	407
6.103.1.37 CHARGER_WPU	407
6.103.1.38 CS_ANS	407
6.103.1.39 CS_GetValue	407
6.103.1.40 CS_LAT	407
6.103.1.41 CS_OD	407
6.103.1.42 CS_PORT	407
6.103.1.43 CS_ResetPullup	408
6.103.1.44 CS_SetAnalogMode	408
6.103.1.45 CS_SetDigitalInput	408
6.103.1.46 CS_SetDigitalMode	408

6.103.1.47 CS_SetDigitalOutput	408
6.103.1.48 CS_SetHigh	408
6.103.1.49 CS_SetLow	408
6.103.1.50 CS_SetOpenDrain	409
6.103.1.51 CS_SetPullup	409
6.103.1.52 CS_SetPushPull	409
6.103.1.53 CS_Toggle	409
6.103.1.54 CS_TRIS	409
6.103.1.55 CS_WPU	409
6.103.1.56 DB0_ANS	409
6.103.1.57 DB0_GetValue	409
6.103.1.58 DB0_LAT	410
6.103.1.59 DB0_OD	410
6.103.1.60 DB0_PORT	410
6.103.1.61 DB0_ResetPullup	410
6.103.1.62 DB0_SetAnalogMode	410
6.103.1.63 DB0_SetDigitalInput	410
6.103.1.64 DB0_SetDigitalMode	410
6.103.1.65 DB0_SetDigitalOutput	410
6.103.1.66 DB0_SetHigh	411
6.103.1.67 DB0_SetLow	411
6.103.1.68 DB0_SetOpenDrain	411
6.103.1.69 DB0_SetPullup	411
6.103.1.70 DB0_SetPushPull	411
6.103.1.71 DB0_Toggle	411
6.103.1.72 DB0_TRIS	411
6.103.1.73 DB0_WPU	412
6.103.1.74 DB1_ANS	412
6.103.1.75 DB1_GetValue	412
6.103.1.76 DB1_LAT	412
6.103.1.77 DB1_OD	412
6.103.1.78 DB1_PORT	412
6.103.1.79 DB1_ResetPullup	412
6.103.1.80 DB1_SetAnalogMode	412
6.103.1.81 DB1_SetDigitalInput	413
6.103.1.82 DB1_SetDigitalMode	413
6.103.1.83 DB1_SetDigitalOutput	413
6.103.1.84 DB1_SetHigh	413
6.103.1.85 DB1_SetLow	413
6.103.1.86 DB1_SetOpenDrain	413
6.103.1.87 DB1_SetPullup	413
6.103.1.88 DB1_SetPushPull	414

6.103.1.89 DB1_Toggle	414
6.103.1.90 DB1_TRIS	414
6.103.1.91 DB1_WPU	414
6.103.1.92 DERE_ANS	414
6.103.1.93 DERE_GetValue	414
6.103.1.94 DERE_LAT	414
6.103.1.95 DERE_OD	414
6.103.1.96 DERE_PORT	414
6.103.1.97 DERE_ResetPullup	415
6.103.1.98 DERE_SetAnalogMode	415
6.103.1.99 DERE_SetDigitalInput	415
6.103.1.100 DERE_SetDigitalMode	415
6.103.1.101 DERE_SetDigitalOutput	415
6.103.1.102 DERE_SetHigh	415
6.103.1.103 DERE_SetLow	415
6.103.1.104 DERE_SetOpenDrain	416
6.103.1.105 DERE_SetPullup	416
6.103.1.106 DERE_SetPushPull	416
6.103.1.107 DERE_Toggle	416
6.103.1.108 DERE_TRIS	416
6.103.1.109 DERE_WPU	416
6.103.1.110 DIGITAL	416
6.103.1.111 DLED1_ANS	416
6.103.1.112 DLED1_GetValue	417
6.103.1.113 DLED1_LAT	417
6.103.1.114 DLED1_OD	417
6.103.1.115 DLED1_PORT	417
6.103.1.116 DLED1_ResetPullup	417
6.103.1.117 DLED1_SetAnalogMode	417
6.103.1.118 DLED1_SetDigitalInput	417
6.103.1.119 DLED1_SetDigitalMode	417
6.103.1.120 DLED1_SetDigitalOutput	418
6.103.1.121 DLED1_SetHigh	418
6.103.1.122 DLED1_SetLow	418
6.103.1.123 DLED1_SetOpenDrain	418
6.103.1.124 DLED1_SetPullup	418
6.103.1.125 DLED1_SetPushPull	418
6.103.1.126 DLED1_Toggle	418
6.103.1.127 DLED1_TRIS	419
6.103.1.128 DLED1_WPU	419
6.103.1.129 DLED_ANS	419
6.103.1.130 DLED_GetValue	419

6.103.1.131 DLED_LAT	419
6.103.1.132 DLED_OD	419
6.103.1.133 DLED_PORT	419
6.103.1.134 DLED_ResetPullup	419
6.103.1.135 DLED_SetAnalogMode	419
6.103.1.136 DLED_SetDigitalInput	420
6.103.1.137 DLED_SetDigitalMode	420
6.103.1.138 DLED_SetDigitalOutput	420
6.103.1.139 DLED_SetHigh	420
6.103.1.140 DLED_SetLow	420
6.103.1.141 DLED_SetOpenDrain	420
6.103.1.142 DLED_SetPullup	420
6.103.1.143 DLED_SetPushPull	421
6.103.1.144 DLED_Toggle	421
6.103.1.145 DLED_TRIS	421
6.103.1.146 DLED_WPU	421
6.103.1.147 HIGH	421
6.103.1.148 INPUT	421
6.103.1.149 INT_TRACE_ANS	421
6.103.1.150 INT_TRACE_GetValue	422
6.103.1.151 INT_TRACE_LAT	422
6.103.1.152 INT_TRACE_OD	422
6.103.1.153 INT_TRACE_PORT	422
6.103.1.154 INT_TRACE_ResetPullup	422
6.103.1.155 INT_TRACE_SetAnalogMode	422
6.103.1.156 INT_TRACE_SetDigitalInput	422
6.103.1.157 INT_TRACE_SetDigitalMode	422
6.103.1.158 INT_TRACE_SetDigitalOutput	423
6.103.1.159 INT_TRACE_SetHigh	423
6.103.1.160 INT_TRACE_SetLow	423
6.103.1.161 INT_TRACE_SetOpenDrain	423
6.103.1.162 INT_TRACE_SetPullup	423
6.103.1.163 INT_TRACE_SetPushPull	423
6.103.1.164 INT_TRACE_Toggle	423
6.103.1.165 INT_TRACE_TRIS	424
6.103.1.166 INT_TRACE_WPU	424
6.103.1.167 IO_RB5_ANS	424
6.103.1.168 IO_RB5_GetValue	424
6.103.1.169 IO_RB5_LAT	424
6.103.1.170 IO_RB5_OD	424
6.103.1.171 IO_RB5_PORT	424
6.103.1.172 IO_RB5_ResetPullup	424

6.103.1.173 IO_RB5_SetAnalogMode	424
6.103.1.174 IO_RB5_SetDigitalInput	425
6.103.1.175 IO_RB5_SetDigitalMode	425
6.103.1.176 IO_RB5_SetDigitalOutput	425
6.103.1.177 IO_RB5_SetHigh	425
6.103.1.178 IO_RB5_SetLow	425
6.103.1.179 IO_RB5_SetOpenDrain	425
6.103.1.180 IO_RB5_SetPullup	425
6.103.1.181 IO_RB5_SetPushPull	426
6.103.1.182 IO_RB5_Toggle	426
6.103.1.183 IO_RB5_TRIS	426
6.103.1.184 IO_RB5_WPU	426
6.103.1.185 IO_RD7_ANS	426
6.103.1.186 IO_RD7_GetValue	426
6.103.1.187 IO_RD7_LAT	426
6.103.1.188 IO_RD7_OD	426
6.103.1.189 IO_RD7_PORT	426
6.103.1.190 IO_RD7_ResetPullup	427
6.103.1.191 IO_RD7_SetAnalogMode	427
6.103.1.192 IO_RD7_SetDigitalInput	427
6.103.1.193 IO_RD7_SetDigitalMode	427
6.103.1.194 IO_RD7_SetDigitalOutput	427
6.103.1.195 IO_RD7_SetHigh	427
6.103.1.196 IO_RD7_SetLow	427
6.103.1.197 IO_RD7_SetOpenDrain	428
6.103.1.198 IO_RD7_SetPullup	428
6.103.1.199 IO_RD7_SetPushPull	428
6.103.1.200 IO_RD7_Toggle	428
6.103.1.201 IO_RD7_TRIS	428
6.103.1.202 IO_RD7_WPU	428
6.103.1.203 L_SWITCH_ANS	428
6.103.1.204 L_SWITCH_GetValue	428
6.103.1.205 L_SWITCH_LAT	429
6.103.1.206 L_SWITCH_OD	429
6.103.1.207 L_SWITCH_PORT	429
6.103.1.208 L_SWITCH_ResetPullup	429
6.103.1.209 L_SWITCH_SetAnalogMode	429
6.103.1.210 L_SWITCH_SetDigitalInput	429
6.103.1.211 L_SWITCH_SetDigitalMode	429
6.103.1.212 L_SWITCH_SetDigitalOutput	429
6.103.1.213 L_SWITCH_SetHigh	430
6.103.1.214 L_SWITCH_SetLow	430

6.103.1.215 L_SWITCH_SetOpenDrain	430
6.103.1.216 L_SWITCH_SetPullup	430
6.103.1.217 L_SWITCH_SetPushPull	430
6.103.1.218 L_SWITCH_Toggle	430
6.103.1.219 L_SWITCH_TRIS	430
6.103.1.220 L_SWITCH_WPU	431
6.103.1.221 LOW	431
6.103.1.222 M_SWITCH_ANS	431
6.103.1.223 M_SWITCH_GetValue	431
6.103.1.224 M_SWITCH_LAT	431
6.103.1.225 M_SWITCH_OD	431
6.103.1.226 M_SWITCH_PORT	431
6.103.1.227 M_SWITCH_ResetPullup	431
6.103.1.228 M_SWITCH_SetAnalogMode	431
6.103.1.229 M_SWITCH_SetDigitalInput	432
6.103.1.230 M_SWITCH_SetDigitalMode	432
6.103.1.231 M_SWITCH_SetDigitalOutput	432
6.103.1.232 M_SWITCH_SetHigh	432
6.103.1.233 M_SWITCH_SetLow	432
6.103.1.234 M_SWITCH_SetOpenDrain	432
6.103.1.235 M_SWITCH_SetPullup	432
6.103.1.236 M_SWITCH_SetPushPull	433
6.103.1.237 M_SWITCH_Toggle	433
6.103.1.238 M_SWITCH_TRIS	433
6.103.1.239 M_SWITCH_WPU	433
6.103.1.240 MISC_ANS	433
6.103.1.241 MISC_GetValue	433
6.103.1.242 MISC_LAT	433
6.103.1.243 MISC_OD	433
6.103.1.244 MISC_PORT	433
6.103.1.245 MISC_ResetPullup	434
6.103.1.246 MISC_SetAnalogMode	434
6.103.1.247 MISC_SetDigitalInput	434
6.103.1.248 MISC_SetDigitalMode	434
6.103.1.249 MISC_SetDigitalOutput	434
6.103.1.250 MISC_SetHigh	434
6.103.1.251 MISC_SetLow	434
6.103.1.252 MISC_SetOpenDrain	435
6.103.1.253 MISC_SetPullup	435
6.103.1.254 MISC_SetPushPull	435
6.103.1.255 MISC_Toggle	435
6.103.1.256 MISC_TRIS	435

6.103.1.257 MISC_WPU	435
6.103.1.258 MLED_ANS	435
6.103.1.259 MLED_GetValue	435
6.103.1.260 MLED_LAT	436
6.103.1.261 MLED_OD	436
6.103.1.262 MLED_PORT	436
6.103.1.263 MLED_ResetPullup	436
6.103.1.264 MLED_SetAnalogMode	436
6.103.1.265 MLED_SetDigitalInput	436
6.103.1.266 MLED_SetDigitalMode	436
6.103.1.267 MLED_SetDigitalOutput	436
6.103.1.268 MLED_SetHigh	437
6.103.1.269 MLED_SetLow	437
6.103.1.270 MLED_SetOpenDrain	437
6.103.1.271 MLED_SetPullup	437
6.103.1.272 MLED_SetPushPull	437
6.103.1.273 MLED_Toggle	437
6.103.1.274 MLED_TRIS	437
6.103.1.275 MLED_WPU	438
6.103.1.276 OUTPUT	438
6.103.1.277 PROG_TRACE_ANS	438
6.103.1.278 PROG_TRACE_GetValue	438
6.103.1.279 PROG_TRACE_LAT	438
6.103.1.280 PROG_TRACE_OD	438
6.103.1.281 PROG_TRACE_PORT	438
6.103.1.282 PROG_TRACE_ResetPullup	438
6.103.1.283 PROG_TRACE_SetAnalogMode	438
6.103.1.284 PROG_TRACE_SetDigitalInput	439
6.103.1.285 PROG_TRACE_SetDigitalMode	439
6.103.1.286 PROG_TRACE_SetDigitalOutput	439
6.103.1.287 PROG_TRACE_SetHigh	439
6.103.1.288 PROG_TRACE_SetLow	439
6.103.1.289 PROG_TRACE_SetOpenDrain	439
6.103.1.290 PROG_TRACE_SetPullup	439
6.103.1.291 PROG_TRACE_SetPushPull	440
6.103.1.292 PROG_TRACE_Toggle	440
6.103.1.293 PROG_TRACE_TRIS	440
6.103.1.294 PROG_TRACE_WPU	440
6.103.1.295 PULL_UP_DISABLED	440
6.103.1.296 PULL_UP_ENABLED	440
6.103.1.297 RA0_GetValue	440
6.103.1.298 RA0_ResetPullup	440

6.103.1.299 RA0_SetAnalogMode	441
6.103.1.300 RA0_SetDigitalInput	441
6.103.1.301 RA0_SetDigitalMode	441
6.103.1.302 RA0_SetDigitalOutput	441
6.103.1.303 RA0_SetHigh	441
6.103.1.304 RA0_SetLow	441
6.103.1.305 RA0_SetPullup	441
6.103.1.306 RA0_Toggle	442
6.103.1.307 RA3_GetValue	442
6.103.1.308 RA3_ResetPullup	442
6.103.1.309 RA3_SetAnalogMode	442
6.103.1.310 RA3_SetDigitalInput	442
6.103.1.311 RA3_SetDigitalMode	442
6.103.1.312 RA3_SetDigitalOutput	442
6.103.1.313 RA3_SetHigh	443
6.103.1.314 RA3_SetLow	443
6.103.1.315 RA3_SetPullup	443
6.103.1.316 RA3_Toggle	443
6.103.1.317 RA4_GetValue	443
6.103.1.318 RA4_ResetPullup	443
6.103.1.319 RA4_SetAnalogMode	443
6.103.1.320 RA4_SetDigitalInput	444
6.103.1.321 RA4_SetDigitalMode	444
6.103.1.322 RA4_SetDigitalOutput	444
6.103.1.323 RA4_SetHigh	444
6.103.1.324 RA4_SetLow	444
6.103.1.325 RA4_SetPullup	444
6.103.1.326 RA4_Toggle	444
6.103.1.327 RB0_GetValue	445
6.103.1.328 RB0_ResetPullup	445
6.103.1.329 RB0_SetAnalogMode	445
6.103.1.330 RB0_SetDigitalInput	445
6.103.1.331 RB0_SetDigitalMode	445
6.103.1.332 RB0_SetDigitalOutput	445
6.103.1.333 RB0_SetHigh	445
6.103.1.334 RB0_SetLow	446
6.103.1.335 RB0_SetPullup	446
6.103.1.336 RB0_Toggle	446
6.103.1.337 RB4_GetValue	446
6.103.1.338 RB4_ResetPullup	446
6.103.1.339 RB4_SetAnalogMode	446
6.103.1.340 RB4_SetDigitalInput	446

6.103.1.341 RB4_SetDigitalMode	447
6.103.1.342 RB4_SetDigitalOutput	447
6.103.1.343 RB4_SetHigh	447
6.103.1.344 RB4_SetLow	447
6.103.1.345 RB4_SetPullup	447
6.103.1.346 RB4_Toggle	447
6.103.1.347 RB7_GetValue	447
6.103.1.348 RB7_ResetPullup	448
6.103.1.349 RB7_SetAnalogMode	448
6.103.1.350 RB7_SetDigitalInput	448
6.103.1.351 RB7_SetDigitalMode	448
6.103.1.352 RB7_SetDigitalOutput	448
6.103.1.353 RB7_SetHigh	448
6.103.1.354 RB7_SetLow	448
6.103.1.355 RB7_SetPullup	449
6.103.1.356 RB7_Toggle	449
6.103.1.357 RC0_GetValue	449
6.103.1.358 RC0_ResetPullup	449
6.103.1.359 RC0_SetAnalogMode	449
6.103.1.360 RC0_SetDigitalInput	449
6.103.1.361 RC0_SetDigitalMode	449
6.103.1.362 RC0_SetDigitalOutput	450
6.103.1.363 RC0_SetHigh	450
6.103.1.364 RC0_SetLow	450
6.103.1.365 RC0_SetPullup	450
6.103.1.366 RC0_Toggle	450
6.103.1.367 RC1_GetValue	450
6.103.1.368 RC1_ResetPullup	450
6.103.1.369 RC1_SetAnalogMode	451
6.103.1.370 RC1_SetDigitalInput	451
6.103.1.371 RC1_SetDigitalMode	451
6.103.1.372 RC1_SetDigitalOutput	451
6.103.1.373 RC1_SetHigh	451
6.103.1.374 RC1_SetLow	451
6.103.1.375 RC1_SetPullup	451
6.103.1.376 RC1_Toggle	452
6.103.1.377 RC3_GetValue	452
6.103.1.378 RC3_ResetPullup	452
6.103.1.379 RC3_SetAnalogMode	452
6.103.1.380 RC3_SetDigitalInput	452
6.103.1.381 RC3_SetDigitalMode	452
6.103.1.382 RC3_SetDigitalOutput	452

6.103.1.383 RC3_SetHigh	453
6.103.1.384 RC3_SetLow	453
6.103.1.385 RC3_SetPullup	453
6.103.1.386 RC3_Toggle	453
6.103.1.387 RC4_GetValue	453
6.103.1.388 RC4_ResetPullup	453
6.103.1.389 RC4_SetAnalogMode	453
6.103.1.390 RC4_SetDigitalInput	454
6.103.1.391 RC4_SetDigitalMode	454
6.103.1.392 RC4_SetDigitalOutput	454
6.103.1.393 RC4_SetHigh	454
6.103.1.394 RC4_SetLow	454
6.103.1.395 RC4_SetPullup	454
6.103.1.396 RC4_Toggle	454
6.103.1.397 RC5_GetValue	455
6.103.1.398 RC5_ResetPullup	455
6.103.1.399 RC5_SetAnalogMode	455
6.103.1.400 RC5_SetDigitalInput	455
6.103.1.401 RC5_SetDigitalMode	455
6.103.1.402 RC5_SetDigitalOutput	455
6.103.1.403 RC5_SetHigh	455
6.103.1.404 RC5_SetLow	456
6.103.1.405 RC5_SetPullup	456
6.103.1.406 RC5_Toggle	456
6.103.1.407 RC6_GetValue	456
6.103.1.408 RC6_ResetPullup	456
6.103.1.409 RC6_SetAnalogMode	456
6.103.1.410 RC6_SetDigitalInput	456
6.103.1.411 RC6_SetDigitalMode	457
6.103.1.412 RC6_SetDigitalOutput	457
6.103.1.413 RC6_SetHigh	457
6.103.1.414 RC6_SetLow	457
6.103.1.415 RC6_SetPullup	457
6.103.1.416 RC6_Toggle	457
6.103.1.417 RC7_GetValue	457
6.103.1.418 RC7_ResetPullup	458
6.103.1.419 RC7_SetAnalogMode	458
6.103.1.420 RC7_SetDigitalInput	458
6.103.1.421 RC7_SetDigitalMode	458
6.103.1.422 RC7_SetDigitalOutput	458
6.103.1.423 RC7_SetHigh	458
6.103.1.424 RC7_SetLow	458

6.103.1.425 RC7_SetPullup	459
6.103.1.426 RC7_Toggle	459
6.103.1.427 RD0_GetValue	459
6.103.1.428 RD0_ResetPullup	459
6.103.1.429 RD0_SetAnalogMode	459
6.103.1.430 RD0_SetDigitalInput	459
6.103.1.431 RD0_SetDigitalMode	459
6.103.1.432 RD0_SetDigitalOutput	460
6.103.1.433 RD0_SetHigh	460
6.103.1.434 RD0_SetLow	460
6.103.1.435 RD0_SetPullup	460
6.103.1.436 RD0_Toggle	460
6.103.1.437 RD6_GetValue	460
6.103.1.438 RD6_ResetPullup	460
6.103.1.439 RD6_SetAnalogMode	461
6.103.1.440 RD6_SetDigitalInput	461
6.103.1.441 RD6_SetDigitalMode	461
6.103.1.442 RD6_SetDigitalOutput	461
6.103.1.443 RD6_SetHigh	461
6.103.1.444 RD6_SetLow	461
6.103.1.445 RD6_SetPullup	461
6.103.1.446 RD6_Toggle	462
6.103.1.447 RELAY_ANS	462
6.103.1.448 RELAY_GetValue	462
6.103.1.449 RELAY_LAT	462
6.103.1.450 RELAY_OD	462
6.103.1.451 RELAY_PORT	462
6.103.1.452 RELAY_ResetPullup	462
6.103.1.453 RELAY_SetAnalogMode	462
6.103.1.454 RELAY_SetDigitalInput	463
6.103.1.455 RELAY_SetDigitalMode	463
6.103.1.456 RELAY_SetDigitalOutput	463
6.103.1.457 RELAY_SetHigh	463
6.103.1.458 RELAY_SetLow	463
6.103.1.459 RELAY_SetOpenDrain	463
6.103.1.460 RELAY_SetPullup	463
6.103.1.461 RELAY_SetPushPull	464
6.103.1.462 RELAY_Toggle	464
6.103.1.463 RELAY_TRIS	464
6.103.1.464 RELAY_WPU	464
6.103.1.465 RLED_ANS	464
6.103.1.466 RLED_GetValue	464

6.103.1.467 RLED_LAT	464
6.103.1.468 RLED_OD	464
6.103.1.469 RLED_PORT	464
6.103.1.470 RLED_ResetPullup	465
6.103.1.471 RLED_SetAnalogMode	465
6.103.1.472 RLED_SetDigitalInput	465
6.103.1.473 RLED_SetDigitalMode	465
6.103.1.474 RLED_SetDigitalOutput	465
6.103.1.475 RLED_SetHigh	465
6.103.1.476 RLED_SetLow	465
6.103.1.477 RLED_SetOpenDrain	466
6.103.1.478 RLED_SetPullup	466
6.103.1.479 RLED_SetPushPull	466
6.103.1.480 RLED_Toggle	466
6.103.1.481 RLED_TRIS	466
6.103.1.482 RLED_WPU	466
6.103.2 Function Documentation	467
6.103.2.1 IOCAF1_DefaultInterruptHandler()	467
6.103.2.2 IOCAF1_ISR()	467
6.103.2.3 IOCAF1_SetInterruptHandler()	468
6.103.2.4 IOCAF2_DefaultInterruptHandler()	468
6.103.2.5 IOCAF2_ISR()	469
6.103.2.6 IOCAF2_SetInterruptHandler()	469
6.103.2.7 IOCAF5_DefaultInterruptHandler()	470
6.103.2.8 IOCAF5_ISR()	470
6.103.2.9 IOCAF5_SetInterruptHandler()	471
6.103.2.10 PIN_MANAGER_Initialize()	471
6.103.3 Variable Documentation	472
6.103.3.1 IOCAF1_InterruptHandler	472
6.103.3.2 IOCAF2_InterruptHandler	473
6.103.3.3 IOCAF5_InterruptHandler	473
6.104 pin_manager.h	474
6.105 mateQ84.X/mcc_generated_files/spi1.c File Reference	481
6.105.1 Function Documentation	482
6.105.1.1 SPI1_Close()	482
6.105.1.2 SPI1_ExchangeBlock()	482
6.105.1.3 SPI1_ExchangeByte()	482
6.105.1.4 SPI1_Initialize()	483
6.105.1.5 SPI1_Open()	483
6.105.1.6 SPI1_ReadBlock()	483
6.105.1.7 SPI1_ReadByte()	483
6.105.1.8 SPI1_WriteBlock()	484

6.105.1.9 SPI1_WriteByte()	484
6.106 mateQ84.X/mcc_generated_files/spi1.h File Reference	484
6.106.1 Enumeration Type Documentation	485
6.106.1.1 spi1_modes_t	485
6.106.2 Function Documentation	486
6.106.2.1 SPI1_Close()	486
6.106.2.2 SPI1_ExchangeBlock()	486
6.106.2.3 SPI1_ExchangeByte()	486
6.106.2.4 SPI1_Initialize()	487
6.106.2.5 SPI1_Open()	487
6.106.2.6 SPI1_ReadBlock()	487
6.106.2.7 SPI1_ReadByte()	487
6.106.2.8 SPI1_WriteBlock()	488
6.106.2.9 SPI1_WriteByte()	488
6.107 spi1.h	488
6.108 mateQ84.X/mcc_generated_files/tmr0.c File Reference	489
6.108.1 Function Documentation	490
6.108.1.1 __interrupt()	490
6.108.1.2 TMR0_DefaultInterruptHandler()	490
6.108.1.3 TMR0_Initialize()	490
6.108.1.4 TMR0_ReadTimer()	491
6.108.1.5 TMR0_Reload()	492
6.108.1.6 TMR0_SetInterruptHandler()	492
6.108.1.7 TMR0_StartTimer()	493
6.108.1.8 TMR0_StopTimer()	493
6.108.1.9 TMR0_WriteTimer()	494
6.108.2 Variable Documentation	494
6.108.2.1 TMR0_InterruptHandler	494
6.109 mateQ84.X/mcc_generated_files/tmr0.h File Reference	495
6.109.1 Function Documentation	496
6.109.1.1 TMR0_DefaultInterruptHandler()	496
6.109.1.2 TMR0_Initialize()	496
6.109.1.3 TMR0_ReadTimer()	497
6.109.1.4 TMR0_Reload()	498
6.109.1.5 TMR0_SetInterruptHandler()	498
6.109.1.6 TMR0_StartTimer()	499
6.109.1.7 TMR0_StopTimer()	499
6.109.1.8 TMR0_WriteTimer()	500
6.109.2 Variable Documentation	500
6.109.2.1 TMR0_InterruptHandler	500
6.110 tmr0.h	501
6.111 mateQ84.X/mcc_generated_files/tmr2.c File Reference	501

6.111.1 Function Documentation	502
6.111.1.1 __interrupt()	502
6.111.1.2 TMR2_Counter8BitGet()	503
6.111.1.3 TMR2_Counter8BitSet()	503
6.111.1.4 TMR2_DefaultInterruptHandler()	504
6.111.1.5 TMR2_ExtResetSourceSet()	504
6.111.1.6 TMR2_Initialize()	505
6.111.1.7 TMR2_LoadPeriodRegister()	505
6.111.1.8 TMR2_ModeSet()	506
6.111.1.9 TMR2_Period8BitSet()	507
6.111.1.10 TMR2_ReadTimer()	507
6.111.1.11 TMR2_SetInterruptHandler()	508
6.111.1.12 TMR2_Start()	508
6.111.1.13 TMR2_StartTimer()	509
6.111.1.14 TMR2_Stop()	510
6.111.1.15 TMR2_StopTimer()	510
6.111.1.16 TMR2_WriteTimer()	511
6.111.2 Variable Documentation	511
6.111.2.1 TMR2_InterruptHandler	511
6.112 mateQ84.X/mcc_generated_files/tmr2.h File Reference	512
6.112.1 Enumeration Type Documentation	513
6.112.1.1 TMR2_HLT_EXT_RESET_SOURCE	513
6.112.1.2 TMR2_HLT_MODE	514
6.112.2 Function Documentation	515
6.112.2.1 TMR2_Counter8BitGet()	515
6.112.2.2 TMR2_Counter8BitSet()	516
6.112.2.3 TMR2_DefaultInterruptHandler()	517
6.112.2.4 TMR2_ExtResetSourceSet()	517
6.112.2.5 TMR2_Initialize()	518
6.112.2.6 TMR2_LoadPeriodRegister()	519
6.112.2.7 TMR2_ModeSet()	519
6.112.2.8 TMR2_Period8BitSet()	520
6.112.2.9 TMR2_ReadTimer()	520
6.112.2.10 TMR2_SetInterruptHandler()	521
6.112.2.11 TMR2_Start()	521
6.112.2.12 TMR2_StartTimer()	522
6.112.2.13 TMR2_Stop()	523
6.112.2.14 TMR2_StopTimer()	523
6.112.2.15 TMR2_WriteTimer()	524
6.112.3 Variable Documentation	524
6.112.3.1 TMR2_InterruptHandler	524
6.113 tmr2.h	525

6.114 mateQ84.X/mcc_generated_files/tmr4.c File Reference	529
6.114.1 Function Documentation	530
6.114.1.1 __interrupt()	530
6.114.1.2 TMR4_Counter8BitGet()	530
6.114.1.3 TMR4_Counter8BitSet()	531
6.114.1.4 TMR4_DefaultInterruptHandler()	531
6.114.1.5 TMR4_ExtResetSourceSet()	532
6.114.1.6 TMR4_Initialize()	532
6.114.1.7 TMR4_LoadPeriodRegister()	533
6.114.1.8 TMR4_ModeSet()	533
6.114.1.9 TMR4_Period8BitSet()	534
6.114.1.10 TMR4_ReadTimer()	534
6.114.1.11 TMR4_SetInterruptHandler()	535
6.114.1.12 TMR4_Start()	535
6.114.1.13 TMR4_StartTimer()	536
6.114.1.14 TMR4_Stop()	537
6.114.1.15 TMR4_StopTimer()	537
6.114.1.16 TMR4_WriteTimer()	538
6.114.2 Variable Documentation	538
6.114.2.1 TMR4_InterruptHandler	538
6.115 mateQ84.X/mcc_generated_files/tmr4.h File Reference	539
6.115.1 Enumeration Type Documentation	540
6.115.1.1 TMR4_HLT_EXT_RESET_SOURCE	540
6.115.1.2 TMR4_HLT_MODE	541
6.115.2 Function Documentation	542
6.115.2.1 TMR4_Counter8BitGet()	542
6.115.2.2 TMR4_Counter8BitSet()	543
6.115.2.3 TMR4_DefaultInterruptHandler()	544
6.115.2.4 TMR4_ExtResetSourceSet()	544
6.115.2.5 TMR4_Initialize()	545
6.115.2.6 TMR4_LoadPeriodRegister()	546
6.115.2.7 TMR4_ModeSet()	546
6.115.2.8 TMR4_Period8BitSet()	547
6.115.2.9 TMR4_ReadTimer()	547
6.115.2.10 TMR4_SetInterruptHandler()	548
6.115.2.11 TMR4_Start()	548
6.115.2.12 TMR4_StartTimer()	549
6.115.2.13 TMR4_Stop()	550
6.115.2.14 TMR4_StopTimer()	550
6.115.2.15 TMR4_WriteTimer()	551
6.115.3 Variable Documentation	551
6.115.3.1 TMR4_InterruptHandler	551

6.116 tmr4.h	552
6.117 mateQ84.X/mcc_generated_files/tmr5.c File Reference	556
6.117.1 Function Documentation	557
6.117.1.1 __interrupt()	557
6.117.1.2 TMR5_CheckGateValueStatus()	557
6.117.1.3 TMR5_DefaultInterruptHandler()	558
6.117.1.4 TMR5_Initialize()	558
6.117.1.5 TMR5_ReadTimer()	559
6.117.1.6 TMR5_Reload()	559
6.117.1.7 TMR5_SetInterruptHandler()	560
6.117.1.8 TMR5_StartSinglePulseAcquisition()	560
6.117.1.9 TMR5_StartTimer()	561
6.117.1.10 TMR5_StopTimer()	561
6.117.1.11 TMR5_WriteTimer()	562
6.117.2 Variable Documentation	562
6.117.2.1 timer5ReloadVal	562
6.117.2.2 TMR5_InterruptHandler	563
6.118 mateQ84.X/mcc_generated_files/tmr5.h File Reference	563
6.118.1 Function Documentation	564
6.118.1.1 TMR5_CheckGateValueStatus()	564
6.118.1.2 TMR5_DefaultInterruptHandler()	565
6.118.1.3 TMR5_Initialize()	565
6.118.1.4 TMR5_ReadTimer()	566
6.118.1.5 TMR5_Reload()	567
6.118.1.6 TMR5_SetInterruptHandler()	567
6.118.1.7 TMR5_StartSinglePulseAcquisition()	568
6.118.1.8 TMR5_StartTimer()	568
6.118.1.9 TMR5_StopTimer()	569
6.118.1.10 TMR5_WriteTimer()	569
6.118.2 Variable Documentation	570
6.118.2.1 TMR5_InterruptHandler	570
6.119 tmr5.h	570
6.120 mateQ84.X/mcc_generated_files/tmr6.c File Reference	571
6.120.1 Function Documentation	572
6.120.1.1 __interrupt()	572
6.120.1.2 TMR6_Counter8BitGet()	572
6.120.1.3 TMR6_Counter8BitSet()	573
6.120.1.4 TMR6_DefaultInterruptHandler()	573
6.120.1.5 TMR6_ExtResetSourceSet()	574
6.120.1.6 TMR6_Initialize()	574
6.120.1.7 TMR6_LoadPeriodRegister()	575
6.120.1.8 TMR6_ModeSet()	575

6.120.1.9 TMR6_Period8BitSet()	576
6.120.1.10 TMR6_ReadTimer()	576
6.120.1.11 TMR6_SetInterruptHandler()	577
6.120.1.12 TMR6_Start()	577
6.120.1.13 TMR6_StartTimer()	578
6.120.1.14 TMR6_Stop()	579
6.120.1.15 TMR6_StopTimer()	579
6.120.1.16 TMR6_WriteTimer()	580
6.120.2 Variable Documentation	580
6.120.2.1 TMR6_InterruptHandler	580
6.121 mateQ84.X/mcc_generated_files/tmr6.h File Reference	581
6.121.1 Enumeration Type Documentation	582
6.121.1.1 TMR6_HLT_EXT_RESET_SOURCE	582
6.121.1.2 TMR6_HLT_MODE	583
6.121.2 Function Documentation	584
6.121.2.1 TMR6_Counter8BitGet()	584
6.121.2.2 TMR6_Counter8BitSet()	585
6.121.2.3 TMR6_DefaultInterruptHandler()	586
6.121.2.4 TMR6_ExtResetSourceSet()	586
6.121.2.5 TMR6_Initialize()	587
6.121.2.6 TMR6_LoadPeriodRegister()	588
6.121.2.7 TMR6_ModeSet()	588
6.121.2.8 TMR6_Period8BitSet()	589
6.121.2.9 TMR6_ReadTimer()	589
6.121.2.10 TMR6_SetInterruptHandler()	590
6.121.2.11 TMR6_Start()	590
6.121.2.12 TMR6_StartTimer()	591
6.121.2.13 TMR6_Stop()	592
6.121.2.14 TMR6_StopTimer()	592
6.121.2.15 TMR6_WriteTimer()	593
6.121.3 Variable Documentation	593
6.121.3.1 TMR6_InterruptHandler	593
6.122 tmr6.h	594
6.123 mateQ84.X/mcc_generated_files/uart1.c File Reference	598
6.123.1 Function Documentation	599
6.123.1.1 UART1_DefaultErrorHandler()	599
6.123.1.2 UART1_DefaultFramingErrorHandler()	599
6.123.1.3 UART1_DefaultOverrunErrorHandler()	600
6.123.1.4 UART1_get_last_status()	600
6.123.1.5 UART1_Initialize()	601
6.123.1.6 UART1_is_rx_ready()	602
6.123.1.7 UART1_is_tx_done()	602

6.123.1.8 UART1_is_tx_ready()	603
6.123.1.9 UART1_Read()	603
6.123.1.10 UART1_SetAddressToTransmit()	604
6.123.1.11 UART1_SetErrorHandler()	604
6.123.1.12 UART1_SetFramingErrorHandler()	604
6.123.1.13 UART1_SetOverrunErrorHandler()	605
6.123.1.14 UART1_Write()	605
6.123.2 Variable Documentation	606
6.123.2.1 UART1_ErrorHandler	606
6.123.2.2 UART1_FramingErrorHandler	606
6.123.2.3 UART1_OverrunErrorHandler	606
6.124 mateQ84.X/mcc_generated_files/uart1.h File Reference	606
6.124.1 Macro Definition Documentation	607
6.124.1.1 UART1_DataReady	607
6.124.2 Function Documentation	608
6.124.2.1 UART1_get_last_status()	608
6.124.2.2 UART1_Initialize()	608
6.124.2.3 UART1_is_rx_ready()	609
6.124.2.4 UART1_is_tx_done()	610
6.124.2.5 UART1_is_tx_ready()	610
6.124.2.6 UART1_Read()	611
6.124.2.7 UART1_SetAddressToTransmit()	611
6.124.2.8 UART1_SetErrorHandler()	612
6.124.2.9 UART1_SetFramingErrorHandler()	612
6.124.2.10 UART1_SetOverrunErrorHandler()	613
6.124.2.11 UART1_Write()	613
6.125 uart1.h	614
6.126 mateQ84.X/mcc_generated_files/uart2.c File Reference	615
6.126.1 Macro Definition Documentation	616
6.126.1.1 INT_TRACE	616
6.126.1.2 IO_RB6_Toggle	616
6.126.1.3 TRACE	617
6.126.1.4 UART2_RX_BUFFER_SIZE	617
6.126.1.5 UART2_TX_BUFFER_SIZE	617
6.126.2 Function Documentation	617
6.126.2.1 __interrupt() [1/2]	617
6.126.2.2 __interrupt() [2/2]	617
6.126.2.3 getch()	617
6.126.2.4 putchar()	618
6.126.2.5 UART2_DefaultErrorHandler()	618
6.126.2.6 UART2_DefaultFramingErrorHandler()	618
6.126.2.7 UART2_DefaultOverrunErrorHandler()	619

6.126.2.8 UART2_get_last_status()	619
6.126.2.9 UART2_Initialize()	620
6.126.2.10 UART2_is_rx_ready()	621
6.126.2.11 UART2_is_tx_done()	621
6.126.2.12 UART2_is_tx_ready()	622
6.126.2.13 UART2_Read()	622
6.126.2.14 UART2_Receive_ISR()	623
6.126.2.15 UART2_RxDataHandler()	623
6.126.2.16 UART2_SetErrorHandler()	624
6.126.2.17 UART2_SetFramingErrorHandler()	624
6.126.2.18 UART2_SetOverrunErrorHandler()	625
6.126.2.19 UART2_SetRxInterruptHandler()	625
6.126.2.20 UART2_SetTxInterruptHandler()	625
6.126.2.21 UART2_Transmit_ISR()	626
6.126.2.22 UART2_Write()	626
6.126.3 Variable Documentation	627
6.126.3.1 UART2_ErrorHandler	627
6.126.3.2 UART2_FramingErrorHandler	627
6.126.3.3 UART2_OverrunErrorHandler	627
6.126.3.4 uart2RxCount	627
6.126.3.5 uart2TxBufferRemaining	627
6.127 mateQ84.X/mcc_generated_files/uart2.h File Reference	627
6.127.1 Macro Definition Documentation	629
6.127.1.1 UART2_DataReady	629
6.127.2 Function Documentation	629
6.127.2.1 UART2_get_last_status()	629
6.127.2.2 UART2_Initialize()	630
6.127.2.3 UART2_is_rx_ready()	631
6.127.2.4 UART2_is_tx_done()	631
6.127.2.5 UART2_is_tx_ready()	632
6.127.2.6 UART2_Read()	632
6.127.2.7 UART2_Receive_ISR()	633
6.127.2.8 UART2_RxDataHandler()	633
6.127.2.9 UART2_SetErrorHandler()	634
6.127.2.10 UART2_SetFramingErrorHandler()	634
6.127.2.11 UART2_SetOverrunErrorHandler()	635
6.127.2.12 UART2_SetRxInterruptHandler()	635
6.127.2.13 UART2_SetTxInterruptHandler()	635
6.127.2.14 UART2_Transmit_ISR()	636
6.127.2.15 UART2_Write()	636
6.127.3 Variable Documentation	637
6.127.3.1 UART2_RxInterruptHandler	637

6.127.3.2 UART2_TxInterruptHandler	637
6.127.3.3 uart2RxCount	637
6.127.3.4 uart2TxBufferRemaining	637
6.128 uart2.h	638
6.129 mateQ84.X/mcc_generated_files/uart5.c File Reference	639
6.129.1 Macro Definition Documentation	640
6.129.1.1 INT_TRACE	640
6.129.1.2 IO_RB6_Toggle	640
6.129.1.3 TRACE	641
6.129.1.4 UART5_RX_BUFFER_SIZE	641
6.129.1.5 UART5_TX_BUFFER_SIZE	641
6.129.2 Function Documentation	641
6.129.2.1 __interrupt() [1/2]	641
6.129.2.2 __interrupt() [2/2]	641
6.129.2.3 UART5_DefaultErrorHandler()	641
6.129.2.4 UART5_DefaultFramingErrorHandler()	642
6.129.2.5 UART5_DefaultOverrunErrorHandler()	642
6.129.2.6 UART5_get_last_status()	642
6.129.2.7 UART5_Initialize()	643
6.129.2.8 UART5_is_rx_ready()	644
6.129.2.9 UART5_is_tx_done()	644
6.129.2.10 UART5_is_tx_ready()	645
6.129.2.11 UART5_Read()	645
6.129.2.12 UART5_Receive_ISR()	646
6.129.2.13 UART5_RxDataHandler()	646
6.129.2.14 UART5_SetErrorHandler()	647
6.129.2.15 UART5_SetFramingErrorHandler()	647
6.129.2.16 UART5_SetOverrunErrorHandler()	648
6.129.2.17 UART5_SetRxInterruptHandler()	648
6.129.2.18 UART5_SetTxInterruptHandler()	649
6.129.2.19 UART5_Transmit_ISR()	649
6.129.2.20 UART5_Write()	649
6.129.3 Variable Documentation	650
6.129.3.1 UART5_ErrorHandler	650
6.129.3.2 UART5_FramingErrorHandler	650
6.129.3.3 UART5_OverrunErrorHandler	650
6.129.3.4 uart5RxCount	650
6.129.3.5 uart5TxBufferRemaining	650
6.130 mateQ84.X/mcc_generated_files/uart5.h File Reference	650
6.130.1 Macro Definition Documentation	652
6.130.1.1 UART5_DataReady	652
6.130.2 Function Documentation	652

6.130.2.1 UART5_get_last_status()	652
6.130.2.2 UART5_Initialize()	653
6.130.2.3 UART5_is_rx_ready()	654
6.130.2.4 UART5_is_tx_done()	654
6.130.2.5 UART5_is_tx_ready()	655
6.130.2.6 UART5_Read()	655
6.130.2.7 UART5_Receive_ISR()	656
6.130.2.8 UART5_RxDataHandler()	656
6.130.2.9 UART5_SetErrorHandler()	657
6.130.2.10 UART5_SetFramingErrorHandler()	657
6.130.2.11 UART5_SetOverrunErrorHandler()	658
6.130.2.12 UART5_SetRxInterruptHandler()	658
6.130.2.13 UART5_SetTxInterruptHandler()	659
6.130.2.14 UART5_Transmit_ISR()	659
6.130.2.15 UART5_Write()	659
6.130.3 Variable Documentation	660
6.130.3.1 UART5_RxInterruptHandler	660
6.130.3.2 UART5_TxInterruptHandler	660
6.130.3.3 uart5RxCount	660
6.130.3.4 uart5TxBufferRemaining	660
6.131 uart5.h	661
6.132 mateQ84.X/mxcmd.c File Reference	662
6.132.1 Function Documentation	662
6.132.1.1 calc_checksum()	662
6.132.1.2 FM_io()	663
6.132.1.3 FM_restart()	663
6.132.1.4 FM_rx()	663
6.132.1.5 FM_rx_count()	664
6.132.1.6 FM_rx_ready()	664
6.132.1.7 FM_tx()	664
6.132.1.8 FM_tx_empty()	664
6.132.1.9 lp_filter()	664
6.132.1.10 onesecc_io()	665
6.132.1.11 tennec_io()	665
6.133 mateQ84.X/mxcmd.h File Reference	665
6.133.1 Macro Definition Documentation	667
6.133.1.1 AMP_WHOLE_ZERO	667
6.133.1.2 BUFFER_SPACING	667
6.133.1.3 CMD_CRC_LEN	668
6.133.1.4 DTG_LEN	668
6.133.1.5 FM80_ID	668
6.133.1.6 FM_BUFFER	668

6.133.1.7 FMxx_STATE	668
6.133.1.8 IO_TEST	668
6.133.1.9 LP_BUFFER_SIZE	668
6.133.1.10 MAX_B_BUF	668
6.133.1.11 MAX_C_BUF	668
6.133.1.12 ONLINE_TIMEOUT	668
6.133.1.13 SPINNER_SPEED	669
6.133.1.14 VER	669
6.133.2 Typedef Documentation	669
6.133.2.1 B_type	669
6.133.3 Enumeration Type Documentation	669
6.133.3.1 status_type	669
6.133.4 Function Documentation	669
6.133.4.1 calc_checksum()	669
6.133.4.2 FM_io()	670
6.133.4.3 FM_restart()	670
6.133.4.4 FM_rx()	670
6.133.4.5 FM_rx_count()	670
6.133.4.6 FM_rx_ready()	670
6.133.4.7 FM_tx()	671
6.133.4.8 FM_tx_empty()	671
6.133.4.9 lp_filter()	671
6.133.4.10 onesec_io()	671
6.133.4.11 tensec_io()	672
6.133.4.12 wdtdelay()	672
6.133.5 Variable Documentation	672
6.133.5.1 B	672
6.133.5.2 build_version	672
6.133.5.3 canbus_name	672
6.133.5.4 cmd_battery	673
6.133.5.5 cmd_batteryv	673
6.133.5.6 cmd_date	673
6.133.5.7 cmd_fwreva	673
6.133.5.8 cmd_fwrevb	673
6.133.5.9 cmd_fwrevc	673
6.133.5.10 cmd_id	673
6.133.5.11 cmd_misc	673
6.133.5.12 cmd_mx_log	673
6.133.5.13 cmd_mx_status	673
6.133.5.14 cmd_panelv	674
6.133.5.15 cmd_status	674
6.133.5.16 cmd_time	674

6.133.5.17 cmd_watts	674
6.133.5.18 FM80_name	674
6.133.5.19 modbus_name	674
6.133.5.20 state_name	674
6.134 mxcmd.h	675
6.135 modbus_master.c File Reference	677
6.135.1 Macro Definition Documentation	678
6.135.1.1 OFF	678
6.135.1.2 ON	678
6.135.2 Function Documentation	679
6.135.2.1 clear_2hz()	679
6.135.2.2 clear_500ahz()	679
6.135.2.3 clear_500hz()	679
6.135.2.4 crc16()	680
6.135.2.5 get_2hz()	680
6.135.2.6 get_500ahz()	680
6.135.2.7 get_500hz()	681
6.135.2.8 init_mb_master_timers()	681
6.135.2.9 init_stream_params()	682
6.135.2.10 master_controller_work()	682
6.135.2.11 mb16_swap()	682
6.135.2.12 mb32_swap()	683
6.135.2.13 mb_setup()	683
6.135.2.14 mb_tx_test()	683
6.135.2.15 modbus_rtu_send_msg()	684
6.135.2.16 my_modbus_rx_32()	684
6.135.2.17 timer_2ms_tick()	685
6.135.2.18 timer_500ms_tick()	685
6.135.3 Variable Documentation	685
6.135.3.1 C	685
6.135.3.2 cc_buffer	685
6.135.3.3 cc_buffer_tx	686
6.135.3.4 cc_stream_file	686
6.135.3.5 em	686
6.135.3.6 em_config	686
6.135.3.7 em_data1	686
6.135.3.8 em_data2	686
6.135.3.9 em_id	686
6.135.3.10 em_light	686
6.135.3.11 em_passwd	686
6.135.3.12 em_serial	686
6.135.3.13 em_version	687

6.135.3.14 ems	687
6.135.3.15 emt	687
6.135.3.16 emv	687
6.135.3.17 M	687
6.135.3.18 modbus_em_config	687
6.135.3.19 modbus_em_data1	687
6.135.3.20 modbus_em_data2	687
6.135.3.21 modbus_em_id	687
6.135.3.22 modbus_em_light	688
6.135.3.23 modbus_em_passwd	688
6.135.3.24 modbus_em_serial	688
6.135.3.25 modbus_em_version	688
6.135.3.26 MT	688
6.135.3.27 V	688
6.136 modbus_master.h File Reference	689
6.136.1 Macro Definition Documentation	692
6.136.1.1 BOFF	692
6.136.1.2 BON	692
6.136.1.3 CDELAY	692
6.136.1.4 DEBUG_R	692
6.136.1.5 DUPL_DELAY	692
6.136.1.6 EM_DATA_LEN1	692
6.136.1.7 EM_DATA_LEN2	692
6.136.1.8 FASTQ	692
6.136.1.9 INT_TRACE	692
6.136.1.10 MADDR	692
6.136.1.11 MAX_DATA	693
6.136.1.12 MB_EM540	693
6.136.1.13 MB_EM540_ID_H	693
6.136.1.14 MB_EM540_ID_L	693
6.136.1.15 MB_MASTER	693
6.136.1.16 MM_ERROR_C	693
6.136.1.17 MM_ERROR_S	693
6.136.1.18 QDELAY	693
6.136.1.19 RDELAY	693
6.136.1.20 READ_COILS	693
6.136.1.21 READ_DISCRETE_INPUTS	694
6.136.1.22 READ_HOLDING_REGISTERS	694
6.136.1.23 READ_INPUT_REGISTERS	694
6.136.1.24 SERIAL_DATA_LEN	694
6.136.1.25 Serror	694
6.136.1.26 SPACING	694

6.136.1.27 Sbuffer	694
6.136.1.28 Sread	694
6.136.1.29 Strmt	694
6.136.1.30 SWMBMVER	694
6.136.1.31 Swrite	695
6.136.1.32 TDELAY	695
6.136.1.33 TEDELAY	695
6.136.1.34 TODELAY	695
6.136.1.35 WRITE_MULTIPLE_COILS	695
6.136.1.36 WRITE_MULTIPLE_REGISTERS	695
6.136.1.37 WRITE_SINGLE_COIL	695
6.136.1.38 WRITE_SINGLE_REGISTER	695
6.136.2 Typedef Documentation	695
6.136.2.1 C_data	695
6.136.2.2 cmd_type	695
6.136.2.3 comm_type	696
6.136.2.4 EM_data1	696
6.136.2.5 EM_data2	696
6.136.2.6 EM_serial	696
6.136.2.7 EM_version	696
6.136.2.8 filter_type	696
6.136.2.9 M_data	696
6.136.2.10 M_time_data	696
6.136.2.11 trace_type	696
6.136.3 Enumeration Type Documentation	696
6.136.3.1 cmd_type	696
6.136.3.2 comm_type	697
6.136.3.3 filter_type	697
6.136.3.4 trace_type	697
6.136.4 Function Documentation	698
6.136.4.1 clear_2hz()	698
6.136.4.2 clear_500ahz()	698
6.136.4.3 clear_500hz()	699
6.136.4.4 crc16()	699
6.136.4.5 get_2hz()	700
6.136.4.6 get_500ahz()	700
6.136.4.7 get_500hz()	700
6.136.4.8 init_mb_master_timers()	701
6.136.4.9 init_stream_params()	701
6.136.4.10 master_controller_work()	701
6.136.4.11 mb16_swap()	702
6.136.4.12 mb32_swap()	702

6.136.4.13 mb_setup()	702
6.136.4.14 mb_tx_test()	703
6.136.4.15 modbus_rtu_send_msg()	704
6.136.4.16 my_modbus_rx_32()	704
6.136.4.17 timer_2ms_tick()	704
6.136.4.18 timer_500ms_tick()	705
6.136.5 Variable Documentation	705
6.136.5.1 C	705
6.136.5.2 em	705
6.136.5.3 ems	705
6.136.5.4 emt	705
6.136.5.5 emv	705
6.136.5.6 M	705
6.136.5.7 MT	705
6.136.5.8 V	706
6.137 modbus_master.h	706
6.138 PIC18F47Q84-E_P/PADS/DIP1543W58P254L5175H635Q40N.d File Reference	710
6.139 PIC18F47Q84-E_P/PADS/PIC18F47Q84-E_P.c File Reference	710
6.140 qconfig.h File Reference	710
6.140.1 Macro Definition Documentation	711
6.140.1.1 INT_TRACE	711
6.140.1.2 MAX_SHAPES	712
6.140.1.3 NHD	712
6.140.1.4 SPIN_VAL_UPDATE	712
6.140.2 Function Documentation	712
6.140.2.1 spinners()	712
6.140.3 Variable Documentation	712
6.140.3.1 bat_amp_whole	712
6.140.3.2 cc_mode	712
6.140.3.3 panel_watts	712
6.140.3.4 spin	713
6.140.3.5 volt_fract	713
6.140.3.6 volt_whole	713
6.140.3.7 vw	713
6.141 qconfig.h	713
6.142 ringbufs.c File Reference	714
6.142.1 Function Documentation	715
6.142.1.1 modulo_dec()	715
6.142.1.2 modulo_inc()	715
6.142.1.3 ringBufS_empty()	716
6.142.1.4 ringBufS_flush()	716
6.142.1.5 ringBufS_full()	716

6.142.1.6 ringBufS_get()	716
6.142.1.7 ringBufS_init()	717
6.142.1.8 ringBufS_put()	717
6.142.1.9 ringBufS_put_dma()	717
6.142.1.10 ringBufS_put_dma_cpy()	717
6.143 ringbufs.h File Reference	717
6.143.1 Macro Definition Documentation	719
6.143.1.1 RBUF_SIZE	719
6.143.2 Typedef Documentation	719
6.143.2.1 ringBufS_t	719
6.143.3 Function Documentation	719
6.143.3.1 ringBufS_empty()	719
6.143.3.2 ringBufS_flush()	719
6.143.3.3 ringBufS_full()	719
6.143.3.4 ringBufS_get()	719
6.143.3.5 ringBufS_init()	720
6.143.3.6 ringBufS_put()	720
6.143.3.7 ringBufS_put_dma()	720
6.143.3.8 ringBufS_put_dma_cpy()	720
6.144 ringbufs.h	720
6.145 timers.c File Reference	721
6.145.1 Function Documentation	722
6.145.1.1 delay_ms()	722
6.145.1.2 delay_us()	722
6.145.1.3 StartTimer()	722
6.145.1.4 timer_ms_tick()	723
6.145.1.5 TimerDone()	723
6.145.1.6 WaitMs()	724
6.145.2 Variable Documentation	724
6.145.2.1 tickCount	724
6.146 timers.h File Reference	724
6.146.1 Enumeration Type Documentation	725
6.146.1.1 APP_TIMERS	725
6.146.2 Function Documentation	726
6.146.2.1 delay_ms()	726
6.146.2.2 StartTimer()	726
6.146.2.3 timer_ms_tick()	727
6.146.2.4 TimerDone()	727
6.146.2.5 WaitMs()	728
6.147 timers.h	728
6.148 trace.h File Reference	729
6.148.1 Macro Definition Documentation	729

6.148.1.1 TRACE	729
6.149 trace.h	729
7 Examples	731
7.1 simple_publisher.c	731
7.2 simple_subscriber.c	731
7.3 reconnect_subscriber.c	731
7.4 bio_publisher.c	731
7.5 openssl_publisher.c	731
Index	733

Chapter 1

Topic Index

1.1 Topics

Here is a list of all topics with brief descriptions:

API	9
Control Packet Serialization	20
Control Packet Deserialization	27
Utilities	33
Platform abstraction layer	39
ADC3: Analog-to-Digital Converter with Context	42

Chapter 2

Data Structure Index

2.1 Data Structures

Here are the data structures with brief descriptions:

B_type	67
blob_log	71
blob_type	72
C_data	73
CAN1_RX_FIFO	75
CAN_FIFOREG	76
can_frames_t	77
CAN_MSG_FIELD	77
CAN_MSG_OBJ	79
can_rec_count_t	80
EB_data	80
EM_data1	84
EM_data2	87
EM_serial	90
EM_version	90
M_data	91
M_time_data	93
mbedtls_context	93
mqtt_client	
An MQTT client	95
mqtt_fixed_header	
The fixed header of an MQTT control packet	100
mqtt_message_queue	
A message queue	102
mqtt_queued_message	
A message in a mqtt_message_queue	104
mqtt_response	
A struct used to deserialize/interpret an incoming packet from the broker	106
mqtt_response_connack	
A connection response datastructure	108
mqtt_response_pingresp	
The response to a ping request	109
mqtt_response_puback	
A publish acknowledgement for messages that were published with QoS level 1	110
mqtt_response_pubcomp	
The response to a PUBREL packet	111

mqtt_response_publish	
A publish packet received from the broker	112
mqtt_response_pubrec	
The response packet to a PUBLISH packet with QoS level 2	114
mqtt_response_pubrel	
The response to a PUBREC packet	114
mqtt_response_suback	
The response to a subscription request	115
mqtt_response_unsuback	
The brokers response to a UNSUBSCRIBE request	117
MREG	118
MREG32	119
MREG64	119
mx_log_packed_t	120
mx_logpage_t	120
mx_status_packed_t	122
PWMDC	122
ringBufS_t	123
spi1_configuration_t	124
spi_link_type	125
uart1_status_t	126
uart2_status_t	127
uart5_status_t	128
V_type	129

Chapter 3

File Index

3.1 File List

Here is a list of all files with brief descriptions:

batmon.c	133
batmon.h	139
canfd.c	154
canfd.h	160
dio.c	170
dio.h	172
eadog.c	177
eadog.h	189
main.c	271
modbus_master.c	677
modbus_master.h	689
qconfig.h	710
ringbufs.c	714
ringbufs.h	717
timers.c	721
timers.h	724
trace.h	729
CAN-Examples/canfilter.c	152
CAN-Examples/canreceive.c	153
CAN-Examples/cantransmit.c	154
linux/raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c	256
linux/matesocketcan/.dep.inc	210
linux/matesocketcan/mqtt.c	
Implements the functionality of MQTT-C	210
linux/matesocketcan/mqtt.h	
Declares all the MQTT-C functions and datastructures	211
linux/matesocketcan/mqtt_pal.c	
Implements mqtt_pal_sendall and mqtt_pal_recvall and any platform-specific helpers you'd like	225
linux/matesocketcan/mqtt_pal.h	
Includes/supports the types/calls required by the MQTT-C client	225
linux/matesocketcan/mqtt_pub.c	228
linux/matesocketcan/mqtt_pub.h	233
linux/matesocketcan/pge.h	241
linux/matesocketcan/build/Debug/GNU-Linux/_ext/5c0/raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.o.d	210

linux/matesocketcan/build/Release/GNU-Linux/_ext/5c0/raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.o.d	
210	
linux/matesocketcan/nbproject/private/c_standard_headers_indexer.c	238
linux/matesocketcan/nbproject/private/cpp_standard_headers_indexer.cpp	239
linux/matesocketcan/templates/bearssl_sockets.h	243
linux/matesocketcan/templates/bio_sockets.h	246
linux/matesocketcan/templates/mbedtls_sockets.h	247
linux/matesocketcan/templates/openssl_sockets.h	251
linux/matesocketcan/templates/posix_sockets.h	253
linux/mqtt_cl/main.c	268
linux/mqtt_cl/nbproject/private/c_standard_headers_indexer.c	239
linux/mqtt_cl/nbproject/private/cpp_standard_headers_indexer.cpp	240
mateQ84.X/main.c	271
mateQ84.X/mxcmd.c	662
mateQ84.X/mxcmd.h	665
mateQ84.X/build/default/production/main.p1.d	284
mateQ84.X/build/default/production/mxcmd.p1.d	284
mateQ84.X/build/default/production/_ext/1472/batmon.p1.d	284
mateQ84.X/build/default/production/_ext/1472/canfd.p1.d	284
mateQ84.X/build/default/production/_ext/1472/dio.p1.d	284
mateQ84.X/build/default/production/_ext/1472/eadog.p1.d	284
mateQ84.X/build/default/production/_ext/1472/modbus_master.p1.d	284
mateQ84.X/build/default/production/_ext/1472/timers.p1.d	284
mateQ84.X/build/default/production/mcc_generated_files/adc.p1.d	284
mateQ84.X/build/default/production/mcc_generated_files/can1.p1.d	284
mateQ84.X/build/default/production/mcc_generated_files/clc7.p1.d	284
mateQ84.X/build/default/production/mcc_generated_files/clc8.p1.d	284
mateQ84.X/build/default/production/mcc_generated_files/device_config.p1.d	284
mateQ84.X/build/default/production/mcc_generated_files/dma1.p1.d	284
mateQ84.X/build/default/production/mcc_generated_files/interrupt_manager.p1.d	284
mateQ84.X/build/default/production/mcc_generated_files/mcc.p1.d	284
mateQ84.X/build/default/production/mcc_generated_files/pin_manager.p1.d	284
mateQ84.X/build/default/production/mcc_generated_files/spi1.p1.d	284
mateQ84.X/build/default/production/mcc_generated_files/tmr0.p1.d	284
mateQ84.X/build/default/production/mcc_generated_files/tmr2.p1.d	284
mateQ84.X/build/default/production/mcc_generated_files/tmr4.p1.d	284
mateQ84.X/build/default/production/mcc_generated_files/tmr5.p1.d	284
mateQ84.X/build/default/production/mcc_generated_files/tmr6.p1.d	284
mateQ84.X/build/default/production/mcc_generated_files/uart1.p1.d	284
mateQ84.X/build/default/production/mcc_generated_files/uart2.p1.d	284
mateQ84.X/build/default/production/mcc_generated_files/uart5.p1.d	284
mateQ84.X/dist/default/production/doprnt.d	284
mateQ84.X/mcc_generated_files/adc.c	284
mateQ84.X/mcc_generated_files/adc.h	288
mateQ84.X/mcc_generated_files/can1.c	292
mateQ84.X/mcc_generated_files/can1.h	314
mateQ84.X/mcc_generated_files/can_types.h	336
mateQ84.X/mcc_generated_files/clc7.c	343
mateQ84.X/mcc_generated_files/clc7.h	345
mateQ84.X/mcc_generated_files/clc8.c	348
mateQ84.X/mcc_generated_files/clc8.h	349
mateQ84.X/mcc_generated_files/device_config.c	352
mateQ84.X/mcc_generated_files/device_config.h	352
mateQ84.X/mcc_generated_files/dma1.c	354
mateQ84.X/mcc_generated_files/dma1.h	364
mateQ84.X/mcc_generated_files/interrupt_manager.c	374
mateQ84.X/mcc_generated_files/interrupt_manager.h	376
mateQ84.X/mcc_generated_files/mcc.c	378

mateQ84.X/mcc_generated_files/mcc.h	381
mateQ84.X/mcc_generated_files/pin_manager.c	386
mateQ84.X/mcc_generated_files/pin_manager.h	392
mateQ84.X/mcc_generated_files/spi1.c	481
mateQ84.X/mcc_generated_files/spi1.h	484
mateQ84.X/mcc_generated_files/tmr0.c	489
mateQ84.X/mcc_generated_files/tmr0.h	495
mateQ84.X/mcc_generated_files/tmr2.c	501
mateQ84.X/mcc_generated_files/tmr2.h	512
mateQ84.X/mcc_generated_files/tmr4.c	529
mateQ84.X/mcc_generated_files/tmr4.h	539
mateQ84.X/mcc_generated_files/tmr5.c	556
mateQ84.X/mcc_generated_files/tmr5.h	563
mateQ84.X/mcc_generated_files/tmr6.c	571
mateQ84.X/mcc_generated_files/tmr6.h	581
mateQ84.X/mcc_generated_files/uart1.c	598
mateQ84.X/mcc_generated_files/uart1.h	606
mateQ84.X/mcc_generated_files/uart2.c	615
mateQ84.X/mcc_generated_files/uart2.h	627
mateQ84.X/mcc_generated_files/uart5.c	639
mateQ84.X/mcc_generated_files/uart5.h	650
PIC18F47Q84-E_P/PADS/DIP1543W58P254L5175H635Q40N.d	710
PIC18F47Q84-E_P/PADS/PIC18F47Q84-E_P.c	710

Chapter 4

Topic Documentation

4.1 API

Documentation of everything you need to know to use the MQTT-C client.

Enumerations

- enum `MQTTErrors` { `MQTT_ERROR_UNKNOWN` = INT_MIN , `MQTT_OK` = 1 }
- An enumeration of error codes. Error messages can be retrieved by calling `mqtt_error_str`.*

Functions

- const char * `mqtt_error_str` (enum `MQTTErrors` error)
Returns an error message for error code, `error`.
- enum `MQTTErrors` `mqtt_sync` (struct `mqtt_client` *client)
Function that does the actual sending and receiving of traffic from the network.
- enum `MQTTErrors` `mqtt_init` (struct `mqtt_client` *client, mqtt_pal_socket_handle sockfd, uint8_t *sendbuf, size_t sendbufsz, uint8_t *recvbuf, size_t recvbufsz, void(*publish_response_callback)(void **state, struct `mqtt_response_publish` *publish))
Initializes an MQTT client.
- void `mqtt_init_reconnect` (struct `mqtt_client` *client, void(*reconnect_callback)(struct `mqtt_client` *client, void **state), void *reconnect_state, void(*publish_response_callback)(void **state, struct `mqtt_response_publish` *publish))
Initializes an MQTT client and enables automatic reconnections.
- void `mqtt_reinit` (struct `mqtt_client` *client, mqtt_pal_socket_handle sockfd, uint8_t *sendbuf, size_t sendbufsz, uint8_t *recvbuf, size_t recvbufsz)
Safely assign/reassign a socket and buffers to an new/existing client.
- enum `MQTTErrors` `mqtt_connect` (struct `mqtt_client` *client, const char *client_id, const char *will_topic, const void *will_message, size_t will_message_size, const char *user_name, const char *password, uint8_t connect_flags, uint16_t keep_alive)
Establishes a session with the MQTT broker.
- enum `MQTTErrors` `mqtt_publish` (struct `mqtt_client` *client, const char *topic_name, const void *application_message, size_t application_message_size, uint8_t publish_flags)
Publish an application message.
- enum `MQTTErrors` `mqtt_subscribe` (struct `mqtt_client` *client, const char *topic_name, int max_qos_level)

Subscribe to a topic.

- enum [MQTTErrors mqtt_unsubscribe](#) (struct [mqtt_client](#) *client, const char *topic_name)

Unsubscribe from a topic.

- enum [MQTTErrors mqtt_ping](#) (struct [mqtt_client](#) *client)

Ping the broker.

- enum [MQTTErrors mqtt_disconnect](#) (struct [mqtt_client](#) *client)

Terminate the session with the MQTT broker.

- enum [MQTTErrors mqtt_reconnect](#) (struct [mqtt_client](#) *client)

Terminate the session with the MQTT broker and prepare to reconnect. Client code should call [mqtt_sync](#) immediately after this call to prevent message loss.

4.1.1 Detailed Description

Documentation of everything you need to know to use the MQTT-C client.

This module contains everything you need to know to use MQTT-C in your application. For usage examples see:

- [simple_publisher.c](#)
- [simple_subscriber.c](#)
- [reconnect_subscriber.c](#)
- [bio_publisher.c](#)
- [openssl_publisher.c](#)

Note

MQTT-C can be used in both single-threaded and multi-threaded applications. All the functions in [API](#) are thread-safe.

4.1.2 Enumeration Type Documentation

4.1.2.1 MQTTErrors

enum [MQTTErrors](#)

An enumeration of error codes. Error messages can be retrieved by calling [mqtt_error_str](#).

See also

[mqtt_error_str](#)

Enumerator

MQTT_ERROR_UNKNOWN	
MQTT_OK	

4.1.3 Function Documentation

4.1.3.1 mqtt_connect()

```
enum MQTTErrors mqtt_connect (
    struct mqtt_client * client,
    const char * client_id,
    const char * will_topic,
    const void * will_message,
    size_t will_message_size,
    const char * user_name,
    const char * password,
    uint8_t connect_flags,
    uint16_t keep_alive)
```

Establishes a session with the MQTT broker.

Precondition

mqtt_init must have been called.

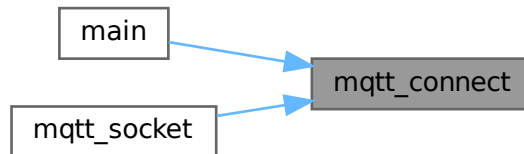
Parameters

in, out	<i>client</i>	The MQTT client.
in	<i>client_id</i>	The unique name identifying the client. (or NULL)
in	<i>will_topic</i>	The topic name of client's <i>will_message</i> . If no will message is desired set to NULL.
in	<i>will_message</i>	The application message (data) to be published in the event the client ungracefully disconnects. Set to NULL if <i>will_topic</i> is NULL.
in	<i>will_message_size</i>	The size of <i>will_message</i> in bytes.
in	<i>user_name</i>	The username to use when establishing the session with the MQTT broker. Set to NULL if a username is not required.
in	<i>password</i>	The password to use when establishing the session with the MQTT broker. Set to NULL if a password is not required.
in	<i>connect_flags</i>	Additional MQTTConnectFlags to use when establishing the connection. These flags are for forcing the session to start clean, MQTT_CONNECT_CLEAN_SESSION, the QOS level to publish the <i>will_message</i> with (provided <i>will_message</i> != NULL), MQTT_CONNECT_WILL_QOS_[0,1,2], and whether or not the broker should retain the <i>will_message</i> , MQTT_CONNECT_WILL_RETAIN.
in	<i>keep_alive</i>	The keep-alive time in seconds. A reasonable value for this is 400 [seconds].

Returns

MQTT_OK upon success, an [MQTTErrors](#) otherwise.

Here is the caller graph for this function:

**4.1.3.2 mqtt_disconnect()**

```
enum MQTTErrors mqtt_disconnect (
    struct mqtt\_client * client)
```

Terminate the session with the MQTT broker.

Precondition

mqtt_connect must have been called.

Parameters

in, out	<i>client</i>	The MQTT client.
---------	---------------	------------------

Note

To re-establish the session, mqtt_connect must be called.

Returns

MQTT_OK upon success, an [MQTTErrors](#) otherwise.

4.1.3.3 mqtt_error_str()

```
const char * mqtt_error_str (
    enum MQTTErrors error)
```

Returns an error message for error code, *error*.

Parameters

in	<i>error</i>	the error code.
----	--------------	-----------------

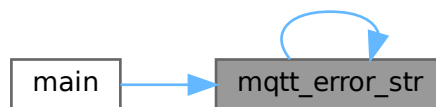
Returns

The associated error message.

Here is the call graph for this function:



Here is the caller graph for this function:

**4.1.3.4 mqtt_init()**

```

enum MQTTErrors mqtt_init (
    struct mqtt_client * client,
    mqtt_pal_socket_handle sockfd,
    uint8_t * sendbuf,
    size_t sendbufsz,
    uint8_t * recvbuf,
    size_t recvbufsz,
    void(* publish_response_callback )(void **state, struct mqtt_response_publish
*publish))
  
```

Initializes an MQTT client.

This function *must* be called before any other API function calls.

Precondition

None.

Parameters

out	<i>client</i>	The MQTT client.
in	<i>sockfd</i>	The socket file descriptor (or equivalent socket handle, e.g. BIO pointer for OpenSSL sockets) connected to the MQTT broker.
in	<i>sendbuf</i>	A buffer that will be used for sending messages to the broker.
in	<i>sendbufsz</i>	The size of <i>sendbuf</i> in bytes.
in	<i>recvbuf</i>	A buffer that will be used for receiving messages from the broker.
in	<i>recvbufsz</i>	The size of <i>recvbuf</i> in bytes.
in	<i>publish_response_callback</i>	The callback to call whenever application messages are received from the broker.

Postcondition

`mqtt_connect` must be called.

Note

`sockfd` is a non-blocking TCP connection.

If `sendbuf` fills up completely during runtime a `MQTT_ERROR_SEND_BUFFER_IS_FULL` error will be set. Similarly if `recvbuf` is ever too small to receive a message from the broker an `MQTT_ERROR_RECV_BUFFER_TOO_SMALL` error will be set.

A pointer to [publish_response_callback_state](#) is always passed as the `state` argument to `publish_response_callback`. Note that the second argument is the `mqtt_response_publish` that was received from the broker.

Attention

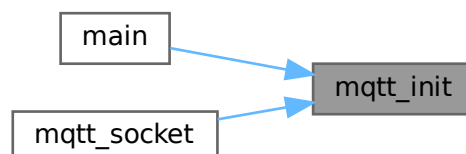
Only initialize an MQTT client once (i.e. don't call `mqtt_init` or `mqtt_init_reconnect` more than once per client). `sendbuf` internally mapped to client's message-to-send queue that actively uses pointer access. In the case of unaligned `sendbuf`, that may lead to Segmentation/Hard/Memory Faults on systems that do not support unaligned pointer access (e.g. ARMv6, ARMv7-M). To avoid that, you may use the following technique:

```
// example for ARMv7-M that requires pointers to be word aligned (4 byte boundary)
static unsigned char mqtt_tx_buffer[MAX_TX_BUFFER_SIZE] __attribute__((aligned(4)));
static unsigned char mqtt_rx_buffer[MAX_RX_BUFFER_SIZE];
// ...
int main(void) {
    // ...
    mqtt_init(p_client, p_client->socketfd, mqtt_tx_buffer, sizeof mqtt_tx_buffer, mqtt_rx_buffer,
              sizeof mqtt_rx_buffer, message_callback);
    // ...
}
```

Returns

`MQTT_OK` upon success, an [MQTTError](#)s otherwise.

Here is the caller graph for this function:



4.1.3.5 mqtt_init_reconnect()

```
void mqtt_init_reconnect (
    struct mqtt_client * client,
    void(* reconnect_callback )(struct mqtt_client *client, void **state),
    void * reconnect_state,
    void(* publish_response_callback )(void **state, struct mqtt_response_publish
*publish))
```

Initializes an MQTT client and enables automatic reconnections.

An alternative to [mqtt_init](#) that allows the client to automatically reconnect to the broker after an error occurs (e.g. socket error or internal buffer overflows).

This is accomplished by calling the `reconnect_callback` whenever the client enters an error state. The job of the `reconnect_callback` is to: (1) perform error handling/logging, (2) clean up the old connection (i.e. close client->socketfd), (3) [mqtt_reinit](#) the client, and (4) reconfigure the MQTT session by calling [mqtt_connect](#) followed by other API calls such as [mqtt_subscribe](#).

The first argument to the `reconnect_callback` is the client (which will be in an error state) and the second argument is a pointer to a void pointer where you can store some state information. Internally, MQTT-C calls the reconnect callback like so:

```
client->reconnect_callback(client, &client->reconnect_state)
```

Note that the `reconnect_callback` is also called to setup the initial session. After calling [mqtt_init_reconnect](#) the client will be in the error state `MQTT_ERROR_INITIAL_RECONNECT`.

Precondition

None.

Parameters

in, out	<i>client</i>	The MQTT client that will be initialized.
in	<i>reconnect_callback</i>	The callback that will be called to connect/reconnect the client to the broker and perform application level error handling.
in	<i>reconnect_state</i>	A pointer to some state data for your <code>reconnect_callback</code> . If your <code>reconnect_callback</code> does not require any state information set this to NULL. A pointer to the memory address where the client stores a copy of this pointer is passed as the second argumnet to <code>reconnect_callback</code> .
in	<i>publish_response_callback</i>	The callback to call whenever application messages are received from the broker.

Postcondition

Call `reconnect_callback` yourself, or call [mqtt_sync](#) (which will trigger the call to `reconnect_callback`).

Attention

Only initialize an MQTT client once (i.e. don't call [mqtt_init](#) or [mqtt_init_reconnect](#) more than once per client).

4.1.3.6 mqtt_ping()

```
enum MQTTErrors mqtt_ping (
    struct mqtt_client * client)
```

Ping the broker.

Precondition

mqtt_connect must have been called.

Parameters

in, out	client	The MQTT client.
---------	--------	------------------

Returns

MQTT_OK upon success, an [MQTTErrors](#) otherwise.

4.1.3.7 mqtt_publish()

```
enum MQTTErrors mqtt_publish (
    struct mqtt_client * client,
    const char * topic_name,
    const void * application_message,
    size_t application_message_size,
    uint8_t publish_flags)
```

Publish an application message.

Publishes an application message to the MQTT broker.

Precondition

mqtt_connect must have been called.

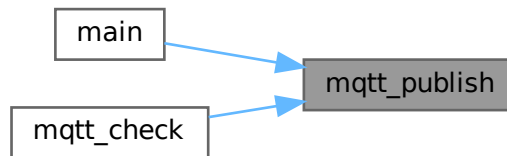
Parameters

in, out	client	The MQTT client.
in	topic_name	The name of the topic.
in	application_message	The data to be published.
in	application_message_size	The size of application_message in bytes.
in	publish_flags	MQTTPublishFlags to be used, namely the QOS level to publish at (MQTT_PUBLISH_QOS_[0,1,2]) or whether or not the broker should retain the publish (MQTT_PUBLISH_RETAIN).

Returns

MQTT_OK upon success, an [MQTTErrors](#) otherwise.

Here is the caller graph for this function:

**4.1.3.8 mqtt_reconnect()**

```
enum MQTTErrors mqtt_reconnect (  
    struct mqtt\_client * client)
```

Terminate the session with the MQTT broker and prepare to reconnect. Client code should call [mqtt_sync](#) immediately after this call to prevent message loss.

Note

The user must provide a reconnect callback function for this to work as expected. See `\r mqtt_client_↔reconnect`.

Precondition

`mqtt_connect` must have been called

Parameters

<i>in, out</i>	<i>client</i>	The MQTT client.
----------------	---------------	------------------

Returns

MQTT_OK upon success, an [MQTTErrors](#) otherwise.

4.1.3.9 mqtt_reinit()

```
void mqtt_reinit (
    struct mqtt_client * client,
    mqtt_pal_socket_handle sockfd,
    uint8_t * sendbuf,
    size_t sendbufsz,
    uint8_t * recvbuf,
    size_t recvbufsz)
```

Safely assign/reassign a socket and buffers to an new/existing client.

This function also clears the `client` error state. Upon exiting this function `client->error` will be `MQTT_ERROR_CONNECT_NOT_CALLED` (which will be cleared) as soon as `mqtt_connect` is called.

Precondition

This function must be called BEFORE `mqtt_connect`.

Parameters

in, out	<i>client</i>	The MQTT client.
in	<i>socketfd</i>	The new socket connected to the broker.
in	<i>sendbuf</i>	The buffer that will be used to buffer egress traffic to the broker.
in	<i>sendbufsz</i>	The size of <i>sendbuf</i> in bytes.
in	<i>recvbuf</i>	The buffer that will be used to buffer ingress traffic from the broker.
in	<i>recvbufsz</i>	The size of <i>recvbuf</i> in bytes.

Postcondition

Call `mqtt_connect`.

Attention

This function should be used in conjunction with clients that have been initialized with `mqtt_init_reconnect`.

4.1.3.10 mqtt_subscribe()

```
enum MQTTErrors mqtt_subscribe (
    struct mqtt_client * client,
    const char * topic_name,
    int max_qos_level)
```

Subscribe to a topic.

Precondition

`mqtt_connect` must have been called.

Parameters

in, out	<i>client</i>	The MQTT client.
in	<i>topic_name</i>	The name of the topic to subscribe to.
in	<i>max_qos_level</i>	The maximum QOS level with which the broker can send application messages for this topic.

Returns

MQTT_OK upon success, an [MQTTErrors](#) otherwise.

4.1.3.11 mqtt_sync()

```
enum MQTTErrors mqtt_sync (
    struct mqtt_client * client)
```

Function that does the actual sending and receiving of traffic from the network.

All the other functions in the [API](#) simply stage messages for being sent to the broker. This function does the actual sending of those messages. Additionally this function receives traffic (responses and acknowledgements) from the broker and responds to that traffic accordingly. Lastly this function also calls the `publish_response_callback` when any MQTT_CONTROL_PUBLISH messages are received.

Precondition

`mqtt_init` must have been called.

Parameters

in, out	<i>client</i>	The MQTT client.
---------	---------------	------------------

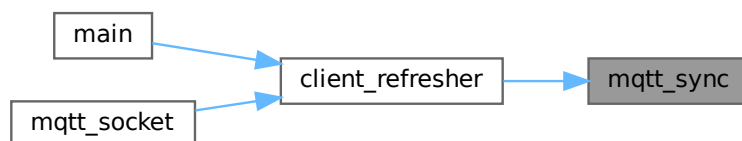
Attention

It is the responsibility of the application programmer to call this function periodically. All functions in the [API](#) are thread-safe so it is perfectly reasonable to have a thread dedicated to calling this function every 200 ms or so. MQTT-C can be used in single threaded application though by simply calling this function periodically inside your main thread. See [simple_publisher.c](#) and [simple_subscriber.c](#) for examples (specifically the `client_refresher` functions).

Returns

MQTT_OK upon success, an [MQTTErrors](#) otherwise.

Here is the caller graph for this function:



4.1.3.12 mqtt_unsubscribe()

```
enum MQTTErrors mqtt_unsubscribe (
    struct mqtt_client * client,
    const char * topic_name)
```

Unsubscribe from a topic.

Precondition

mqtt_connect must have been called.

Parameters

in, out	client	The MQTT client.
in	topic_name	The name of the topic to unsubscribe from.

Returns

MQTT_OK upon success, an MQTTErrors otherwise.

4.2 Control Packet Serialization

Developer documentation of the functions and datastructures used for serializing MQTT control packets.

Macros

- #define MQTT_PROTOCOL_LEVEL 0x04
The protocol identifier for MQTT v3.1.1.
- #define MQTT_SUBSCRIBE_REQUEST_MAX_NUM_TOPICS 8
The maximum number topics that can be subscribed to in a single call to mqtt_pack_subscribe_request.
- #define MQTT_UNSUBSCRIBE_REQUEST_MAX_NUM_TOPICS 8
The maximum number topics that can be subscribed to in a single call to mqtt_pack_unsubscribe_request.

Enumerations

- enum MQTTConnectFlags {
MQTT_CONNECT_RESERVED = 1u , MQTT_CONNECT_CLEAN_SESSION = 2u , MQTT_CONNECT_WILL_FLAG = 4u , MQTT_CONNECT_WILL_QOS_0 = (0u & 0x03) << 3 , MQTT_CONNECT_WILL_QOS_1 = (1u & 0x03) << 3 , MQTT_CONNECT_WILL_QOS_2 = (2u & 0x03) << 3 , MQTT_CONNECT_WILL_RETAIN = 32u , MQTT_CONNECT_PASSWORD = 64u , MQTT_CONNECT_USER_NAME = 128u }
An enumeration of CONNECT packet flags.
- enum MQTTPublishFlags {
MQTT_PUBLISH_DUP = 8u , MQTT_PUBLISH_QOS_0 = ((0u << 1) & 0x06) , MQTT_PUBLISH_QOS_1 = ((1u << 1) & 0x06) , MQTT_PUBLISH_QOS_2 = ((2u << 1) & 0x06) , MQTT_PUBLISH_QOS_MASK = ((3u << 1) & 0x06) , MQTT_PUBLISH_RETAIN = 0x01 }
An enumeration of the PUBLISH flags.

Functions

- `ssize_t mqtt_pack_fixed_header` (`uint8_t *buf`, `size_t bufsz`, `const struct mqtt_fixed_header *fixed_header`)
Serialize an `mqtt_fixed_header` and write it to `buf`.
- `ssize_t mqtt_pack_connection_request` (`uint8_t *buf`, `size_t bufsz`, `const char *client_id`, `const char *will_topic`, `const void *will_message`, `size_t will_message_size`, `const char *user_name`, `const char *password`, `uint8_t connect_flags`, `uint16_t keep_alive`)
Serialize a connection request into a buffer.
- `ssize_t mqtt_pack_publish_request` (`uint8_t *buf`, `size_t bufsz`, `const char *topic_name`, `uint16_t packet_id`, `const void *application_message`, `size_t application_message_size`, `uint8_t publish_flags`)
Serialize a PUBLISH request and put it in `buf`.
- `ssize_t mqtt_pack_pubxxx_request` (`uint8_t *buf`, `size_t bufsz`, `enum MQTTControlPacketType control_type`, `uint16_t packet_id`)
Serialize a PUBACK, PUBREC, PUBREL, or PUBCOMP packet and put it in `buf`.
- `ssize_t mqtt_pack_subscribe_request` (`uint8_t *buf`, `size_t bufsz`, `unsigned int packet_id`,...)
Serialize a SUBSCRIBE packet and put it in `buf`.
- `ssize_t mqtt_pack_unsubscribe_request` (`uint8_t *buf`, `size_t bufsz`, `unsigned int packet_id`,...)
Serialize a UNSUBSCRIBE packet and put it in `buf`.
- `ssize_t mqtt_pack_ping_request` (`uint8_t *buf`, `size_t bufsz`)
Serialize a PINGREQ and put it into `buf`.
- `ssize_t mqtt_pack_disconnect` (`uint8_t *buf`, `size_t bufsz`)
Serialize a DISCONNECT and put it into `buf`.

4.2.1 Detailed Description

Developer documentation of the functions and datastructures used for serializing MQTT control packets.

4.2.2 Macro Definition Documentation

4.2.2.1 MQTT_PROTOCOL_LEVEL

```
#define MQTT_PROTOCOL_LEVEL 0x04
```

The protocol identifier for MQTT v3.1.1.

See also

[MQTT v3.1.1: CONNECT Variable Header.](#)

4.2.2.2 MQTT_SUBSCRIBE_REQUEST_MAX_NUM_TOPICS

```
#define MQTT_SUBSCRIBE_REQUEST_MAX_NUM_TOPICS 8
```

The maximum number topics that can be subscribed to in a single call to `mqtt_pack_subscribe_request`.

See also

[mqtt_pack_subscribe_request](#)

4.2.2.3 MQTT_UNSUBSCRIBE_REQUEST_MAX_NUM_TOPICS

```
#define MQTT_UNSUBSCRIBE_REQUEST_MAX_NUM_TOPICS 8
```

The maximum number topics that can be subscribed to in a single call to `mqtt_pack_unsubscribe_request`.

See also

[mqtt_pack_unsubscribe_request](#)

4.2.3 Enumeration Type Documentation

4.2.3.1 MQTTConnectFlags

```
enum MQTTConnectFlags
```

An enumeration of CONNECT packet flags.

See also

[MQTT v3.1.1: CONNECT Variable Header.](#)

Enumerator

MQTT_CONNECT_RESERVED	
MQTT_CONNECT_CLEAN_SESSION	
MQTT_CONNECT_WILL_FLAG	
MQTT_CONNECT_WILL_QOS_0	
MQTT_CONNECT_WILL_QOS_1	
MQTT_CONNECT_WILL_QOS_2	
MQTT_CONNECT_WILL_RETAIN	
MQTT_CONNECT_PASSWORD	
MQTT_CONNECT_USER_NAME	

4.2.3.2 MQTTPublishFlags

```
enum MQTTPublishFlags
```

An enumeration of the PUBLISH flags.

See also

[MQTT v3.1.1: PUBLISH - Publish Message.](#)

Enumerator

MQTT_PUBLISH_DUP	
MQTT_PUBLISH_QOS_0	
MQTT_PUBLISH_QOS_1	
MQTT_PUBLISH_QOS_2	
MQTT_PUBLISH_QOS_MASK	
MQTT_PUBLISH_RETAIN	

4.2.4 Function Documentation

4.2.4.1 mqtt_pack_connection_request()

```
ssize_t mqtt_pack_connection_request (
    uint8_t * buf,
    size_t bufsz,
    const char * client_id,
    const char * will_topic,
    const void * will_message,
    size_t will_message_size,
    const char * user_name,
    const char * password,
    uint8_t connect_flags,
    uint16_t keep_alive)
```

Serialize a connection request into a buffer.

Parameters

out	<i>buf</i>	the buffer to pack the connection request packet into.
in	<i>bufsz</i>	the number of bytes left in <i>buf</i> .
in	<i>client_id</i>	the ID that identifies the local client. <i>client_id</i> can be NULL or an empty string for Anonymous clients.
in	<i>will_topic</i>	the topic under which the local client's will message will be published. Set to NULL for no will message. If <i>will_topic</i> is not NULL a <i>will_message</i> must also be provided.
in	<i>will_message</i>	the will message to be published upon a unsuccessful disconnection of the local client. Set to NULL if <i>will_topic</i> is NULL. <i>will_message</i> must <i>not</i> be NULL if <i>will_topic</i> is not NULL.
in	<i>will_message_size</i>	The size of <i>will_message</i> in bytes.
in	<i>user_name</i>	the username to be used to connect to the broker with. Set to NULL if no username is required.
in	<i>password</i>	the password to be used to connect to the broker with. Set to NULL if no password is required.
in	<i>connect_flags</i>	additional MQTTConnectFlags to be set. The only flags that need to be set manually are MQTT_CONNECT_CLEAN_SESSION, MQTT_CONNECT_WILL_QOS_X (for X ∈ {0, 1, 2}), and MQTT_CONNECT_WILL_RETAIN. Set to 0 if no additional flags are required.
in	<i>keep_alive</i>	the keep alive time in seconds. It is the responsibility of the client to ensure packets are sent to the server <i>{at least}</i> this frequently.

Note

If there is a *will_topic* and no additional *connect_flags* are given, then by default *will_message* will be published at QoS level 0.

See also

[MQTT v3.1.1: CONNECT – Client Requests a Connection to a Server.](#)

Returns

The number of bytes put into *buf*, 0 if *buf* is too small to fit the CONNECT packet, a negative value if there was a protocol violation.

4.2.4.2 mqtt_pack_disconnect()

```
ssize_t mqtt_pack_disconnect (
    uint8_t * buf,
    size_t bufsz)
```

Serialize a DISCONNECT and put it into `buf`.

Parameters

out	<i>buf</i>	the buffer to put the DISCONNECT packet in.
in	<i>bufsz</i>	the maximum number of bytes that can be put into <code>buf</code> .

See also

[MQTT v3.1.1: DISCONNECT - Disconnect Notification.](#)

Returns

The number of bytes put into `buf`, 0 if `buf` is too small to fit the DISCONNECT packet, a negative value if there was a protocol violation.

4.2.4.3 mqtt_pack_fixed_header()

```
ssize_t mqtt_pack_fixed_header (
    uint8_t * buf,
    size_t bufsz,
    const struct mqtt_fixed_header * fixed_header)
```

Serialize an [mqtt_fixed_header](#) and write it to `buf`.

Note

This function performs complete error checking and a positive return value guarantees the entire packet will fit into the given buffer.

Parameters

out	<i>buf</i>	the buffer to write to.
in	<i>bufsz</i>	the maximum number of bytes that can be put in to <code>buf</code> .
in	<i>fixed_header</i>	the fixed header that will be serialized.

Returns

The number of bytes written to `buf`, or 0 if `buf` is too small, or a negative value if there was a protocol violation.

4.2.4.4 mqtt_pack_ping_request()

```
ssize_t mqtt_pack_ping_request (
    uint8_t * buf,
    size_t bufsz)
```

Serialize a PINGREQ and put it into `buf`.

Parameters

out	<i>buf</i>	the buffer to put the PINGREQ packet in.
in	<i>bufsz</i>	the maximum number of bytes that can be put into <i>buf</i> .

See also

MQTT v3.1.1: PINGREQ – Ping Request.

Returns

The number of bytes put into *buf*, 0 if *buf* is too small to fit the PINGREQ packet, a negative value if there was a protocol violation.

4.2.4.5 mqtt_pack_publish_request()

```
ssize_t mqtt_pack_publish_request (
    uint8_t * buf,
    size_t bufsz,
    const char * topic_name,
    uint16_t packet_id,
    const void * application_message,
    size_t application_message_size,
    uint8_t publish_flags)
```

Serialize a PUBLISH request and put it in *buf*.

Parameters

out	<i>buf</i>	the buffer to put the PUBLISH packet in.
in	<i>bufsz</i>	the maximum number of bytes that can be put into <i>buf</i> .
in	<i>topic_name</i>	the topic to publish <i>application_message</i> under.
in	<i>packet_id</i>	this packets packet ID.
in	<i>application_message</i>	the application message to be published.
in	<i>application_message_size</i>	the size of <i>application_message</i> in bytes.
in	<i>publish_flags</i>	The flags to publish <i>application_message</i> with. These include the MQTT_PUBLISH_DUP flag, MQTT_PUBLISH_QOS_X (X ∈ {0, 1, 2}), and MQTT_PUBLISH_RETAIN flag.

Note

The default QoS is level 0.

See also

MQTT v3.1.1: PUBLISH – Publish Message.

Returns

The number of bytes put into *buf*, 0 if *buf* is too small to fit the PUBLISH packet, a negative value if there was a protocol violation.

4.2.4.6 mqtt_pack_pubxxx_request()

```
ssize_t mqtt_pack_pubxxx_request (
    uint8_t * buf,
    size_t bufsz,
    enum MQTTControlPacketType control_type,
    uint16_t packet_id)
```

Serialize a PUBACK, PUBREC, PUBREL, or PUBCOMP packet and put it in `buf`.

Parameters

out	<i>buf</i>	the buffer to put the PUBXXX packet in.
in	<i>bufsz</i>	the maximum number of bytes that can be put into <code>buf</code> .
in	<i>control_type</i>	the type of packet. Must be one of: MQTT_CONTROL_PUBACK, MQTT_CONTROL_PUBREC, MQTT_CONTROL_PUBREL, or MQTT_CONTROL_PUBCOMP.
in	<i>packet_id</i>	the packet ID of the packet being acknowledged.

See also

MQTT v3.1.1: PUBACK – Publish Acknowledgement.

MQTT v3.1.1: PUBREC – Publish Received.

MQTT v3.1.1: PUBREL – Publish Released.

MQTT v3.1.1: PUBCOMP – Publish Complete.

Returns

The number of bytes put into `buf`, 0 if `buf` is too small to fit the PUBXXX packet, a negative value if there was a protocol violation.

4.2.4.7 mqtt_pack_subscribe_request()

```
ssize_t mqtt_pack_subscribe_request (
    uint8_t * buf,
    size_t bufsz,
    unsigned int packet_id,
    ...)
```

Serialize a SUBSCRIBE packet and put it in `buf`.

Parameters

out	<i>buf</i>	the buffer to put the SUBSCRIBE packet in.
in	<i>bufsz</i>	the maximum number of bytes that can be put into <code>buf</code> .
in	<i>packet_id</i>	the packet ID to be used.
in	...	NULL terminated list of (<code>{const char *topic_name}, {int max_qos_level}</code>) pairs.

Note

The variadic arguments, ..., *must* be followed by a `NULL`. For example:

```
ssize_t n = mqtt_pack_subscribe_request(buf, bufsz, 1234, "topic_1", 0, "topic_2", 2, NULL);
```

See also

MQTT v3.1.1: [SUBSCRIBE – Subscribe to Topics.](#)

Returns

The number of bytes put into `buf`, 0 if `buf` is too small to fit the SUBSCRIBE packet, a negative value if there was a protocol violation.

4.2.4.8 mqtt_pack_unsubscribe_request()

```
ssize_t mqtt_pack_unsubscribe_request (
    uint8_t * buf,
    size_t bufsz,
    unsigned int packet_id,
    ...)
```

Serialize a UNSUBSCRIBE packet and put it in `buf`.

Parameters

out	<i>buf</i>	the buffer to put the UNSUBSCRIBE packet in.
in	<i>bufsz</i>	the maximum number of bytes that can be put into <code>buf</code> .
in	<i>packet_id</i>	the packet ID to be used.
in	...	<code>NULL</code> terminated list of <code>{const char *topic_name}</code> 's to unsubscribe from.

Note

The variadic arguments, ..., *must* be followed by a `NULL`. For example:

```
ssize_t n = mqtt_pack_unsubscribe_request(buf, bufsz, 4321, "topic_1", "topic_2", NULL);
```

See also

MQTT v3.1.1: [UNSUBSCRIBE – Unsubscribe from Topics.](#)

Returns

The number of bytes put into `buf`, 0 if `buf` is too small to fit the UNSUBSCRIBE packet, a negative value if there was a protocol violation.

4.3 Control Packet Deserialization

Developer documentation of the functions and datastructures used for deserializing MQTT control packets.

Data Structures

- struct `mqtt_fixed_header`
The fixed header of an MQTT control packet.
- struct `mqtt_response_connack`
A connection response datastructure.
- struct `mqtt_response_publish`
A publish packet received from the broker.
- struct `mqtt_response_puback`
A publish acknowledgement for messages that were published with QoS level 1.
- struct `mqtt_response_pubrec`
The response packet to a PUBLISH packet with QoS level 2.
- struct `mqtt_response_pubrel`
The response to a PUBREL packet.
- struct `mqtt_response_pubcomp`
The response to a PUBREL packet.
- struct `mqtt_response_suback`
The response to a subscription request.
- struct `mqtt_response_unsuback`
The brokers response to a UNSUBSCRIBE request.
- struct `mqtt_response_pingresp`
The response to a ping request.
- struct `mqtt_response`
A struct used to deserialize/interpret an incoming packet from the broker.

Enumerations

- enum `MQTTControlPacketType` {
`MQTT_CONTROL_CONNECT` = 1u , `MQTT_CONTROL_CONNACK` = 2u , `MQTT_CONTROL_PUBLISH` = 3u , `MQTT_CONTROL_PUBACK` = 4u ,
`MQTT_CONTROL_PUBREC` = 5u , `MQTT_CONTROL_PUBREL` = 6u , `MQTT_CONTROL_PUBCOMP` = 7u , `MQTT_CONTROL_SUBSCRIBE` = 8u ,
`MQTT_CONTROL_SUBACK` = 9u , `MQTT_CONTROL_UNSUBSCRIBE` = 10u , `MQTT_CONTROL_UNSUBACK` = 11u , `MQTT_CONTROL_PINGREQ` = 12u ,
`MQTT_CONTROL_PINGRESP` = 13u , `MQTT_CONTROL_DISCONNECT` = 14u }
An enumeration of the MQTT control packet types.
- enum `MQTTConnackReturnCode` {
`MQTT_CONNACK_ACCEPTED` = 0u , `MQTT_CONNACK_REFUSED_PROTOCOL_VERSION` = 1u ,
`MQTT_CONNACK_REFUSED_IDENTIFIER_REJECTED` = 2u , `MQTT_CONNACK_REFUSED_SERVER_UNAVAILABLE` = 3u ,
`MQTT_CONNACK_REFUSED_BAD_USER_NAME_OR_PASSWORD` = 4u , `MQTT_CONNACK_REFUSED_NOT_AUTHORIZED` = 5u }
An enumeration of the return codes returned in a CONNACK packet.
- enum `MQTTSubackReturnCodes` { `MQTT_SUBACK_SUCCESS_MAX_QOS_0` = 0u , `MQTT_SUBACK_SUCCESS_MAX_QOS_1` = 1u , `MQTT_SUBACK_SUCCESS_MAX_QOS_2` = 2u , `MQTT_SUBACK_FAILURE` = 128u }
An enumeration of subscription acknowledgement return codes.

Functions

- `ssize_t mqtt_unpack_fixed_header` (struct `mqtt_response` *response, const uint8_t *buf, size_t bufsz)
Deserialize the contents of buf into an mqtt_fixed_header object.
- `ssize_t mqtt_unpack_connack_response` (struct `mqtt_response` *mqtt_response, const uint8_t *buf)
Deserialize a CONNACK response from buf.
- `ssize_t mqtt_unpack_publish_response` (struct `mqtt_response` *mqtt_response, const uint8_t *buf)
Deserialize a publish response from buf.
- `ssize_t mqtt_unpack_pubxxx_response` (struct `mqtt_response` *mqtt_response, const uint8_t *buf)
Deserialize a PUBACK/PUBREC/PUBREL/PUBCOMP packet from buf.
- `ssize_t mqtt_unpack_response` (struct `mqtt_response` *response, const uint8_t *buf, size_t bufsz)
Deserialize a packet from the broker.

4.3.1 Detailed Description

Developer documentation of the functions and datastructures used for deserializing MQTT control packets.

4.3.2 Enumeration Type Documentation

4.3.2.1 MQTTConnackReturnCode

enum `MQTTConnackReturnCode`

An enumeration of the return codes returned in a CONNACK packet.

See also

[MQTT v3.1.1: CONNACK return codes.](#)

Enumerator

MQTT_CONNACK_ACCEPTED	
MQTT_CONNACK_REFUSED_PROTOCOL_VERSION	
MQTT_CONNACK_REFUSED_IDENTIFIER_REJECTED	
MQTT_CONNACK_REFUSED_SERVER_UNAVAILABLE	
MQTT_CONNACK_REFUSED_BAD_USER_NAME_OR_PASSWORD	
MQTT_CONNACK_REFUSED_NOT_AUTHORIZED	

4.3.2.2 MQTTControlPacketType

enum `MQTTControlPacketType`

An enumeration of the MQTT control packet types.

See also

[MQTT v3.1.1: MQTT Control Packet Types](#)

Enumerator

MQTT_CONTROL_CONNECT	
MQTT_CONTROL_CONNACK	
MQTT_CONTROL_PUBLISH	
MQTT_CONTROL_PUBACK	
MQTT_CONTROL_PUBREC	
MQTT_CONTROL_PUBREL	
MQTT_CONTROL_PUBCOMP	
MQTT_CONTROL_SUBSCRIBE	
MQTT_CONTROL_SUBACK	
MQTT_CONTROL_UNSUBSCRIBE	
MQTT_CONTROL_UNSUBACK	
MQTT_CONTROL_PINGREQ	
MQTT_CONTROL_PINGRESP	
MQTT_CONTROL_DISCONNECT	

4.3.2.3 MQTTSubackReturnCodes

enum [MQTTSubackReturnCodes](#)

An enumeration of subscription acknowledgement return codes.

See also

[MQTT v3.1.1: SUBACK Return Codes.](#)

Enumerator

MQTT_SUBACK_SUCCESS_MAX_QOS↔ _0	
MQTT_SUBACK_SUCCESS_MAX_QOS↔ _1	
MQTT_SUBACK_SUCCESS_MAX_QOS↔ _2	
MQTT_SUBACK_FAILURE	

4.3.3 Function Documentation

4.3.3.1 mqtt_unpack_connack_response()

```
ssize_t mqtt_unpack_connack_response (
    struct mqtt\_response * mqtt_response,
    const uint8_t * buf) [related]
```

Deserialize a CONNACK response from buf.

Precondition

[mqtt_unpack_fixed_header](#) must have returned a positive value and the control packet type must be MQTT↔
_CONTROL_CONNACK.

Parameters

out	mqtt_response	the mqtt_response that will be initialized.
in	<i>buf</i>	the buffer that contains the variable header and payload of the packet. The first byte of <i>buf</i> should be the first byte of the variable header.

Returns

The number of bytes that were consumed, or 0 if the buffer does not contain enough bytes to parse the packet, or a negative value if there was a protocol violation.

4.3.3.2 mqtt_unpack_fixed_header()

```
ssize_t mqtt_unpack_fixed_header (
    struct mqtt\_response * response,
    const uint8_t * buf,
    size_t bufsz)
```

Deserialize the contents of *buf* into an [mqtt_fixed_header](#) object.

Note

This function performs complete error checking and a positive return value means the entire [mqtt_response](#) can be deserialized from *buf*.

Parameters

out	<i>response</i>	the response who's fixed_header will be initialized.
in	<i>buf</i>	the buffer.
in	<i>bufsz</i>	the total number of bytes in the buffer.

Returns

The number of bytes that were consumed, or 0 if the buffer does not contain enough bytes to parse the packet, or a negative value if there was a protocol violation.

4.3.3.3 mqtt_unpack_publish_response()

```
ssize_t mqtt_unpack_publish_response (
    struct mqtt\_response * mqtt_response,
    const uint8_t * buf) [related]
```

Deserialize a publish response from *buf*.

Precondition

[mqtt_unpack_fixed_header](#) must have returned a positive value and the [mqtt_response](#) must have a control type of MQTT_CONTROL_PUBLISH.

Parameters

out	mqtt_response	the response that is initialized from the contents of <code>buf</code> .
in	<code>buf</code>	the buffer with the incoming data.

Returns

The number of bytes that were consumed, or 0 if the buffer does not contain enough bytes to parse the packet, or a negative value if there was a protocol violation.

4.3.3.4 mqtt_unpack_pubxxx_response()

```
ssize_t mqtt_unpack_pubxxx_response (
    struct mqtt\_response * mqtt_response,
    const uint8_t * buf) [related]
```

Deserialize a PUBACK/PUBREC/PUBREL/PUBCOMP packet from `buf`.

Precondition

[mqtt_unpack_fixed_header](#) must have returned a positive value and the [mqtt_response](#) must have a control type of MQTT_CONTROL_PUBACK, MQTT_CONTROL_PUBREC, MQTT_CONTROL_PUBREL or MQTT_CONTROL_PUBCOMP.

Parameters

out	mqtt_response	the response that is initialized from the contents of <code>buf</code> .
in	<code>buf</code>	the buffer with the incoming data.

[mqtt_response_pubrec](#) [mqtt_response_pubrel](#) [mqtt_response_pubcomp](#)

Returns

The number of bytes that were consumed, or 0 if the buffer does not contain enough bytes to parse the packet, or a negative value if there was a protocol violation.

4.3.3.5 mqtt_unpack_response()

```
ssize_t mqtt_unpack_response (
    struct mqtt\_response * response,
    const uint8_t * buf,
    size_t bufsz) [related]
```

Deserialize a packet from the broker.

Parameters

out	<code>response</code>	the mqtt_response that will be initialize from <code>buf</code> .
in	<code>buf</code>	the incoming data buffer.
in	<code>bufsz</code>	the number of bytes available in the buffer.

Returns

The number of bytes consumed on success, zero `buf` does not contain enough bytes to deserialize the packet, a negative value if a protocol violation was encountered.

4.4 Utilities

Developer documentation for the utilities used to implement the MQTT-C client.

Data Structures

- struct `mqtt_queued_message`
A message in a `mqtt_message_queue`.
- struct `mqtt_message_queue`
A message queue.
- struct `mqtt_client`
An MQTT client.

Macros

- #define `mqtt_mq_get`(mq_ptr, index)
Returns the `mqtt_queued_message` at `index`.
- #define `mqtt_mq_length`(mq_ptr)
Returns the number of messages in the message queue, `mq_ptr`.
- #define `mqtt_mq_currSZ`(mq_ptr)
Used internally to recalculate the `curr_sZ`.

Enumerations

- enum `MQTTQueuedMessageState` { `MQTT_QUEUED_UNSENT` , `MQTT_QUEUED_AWAITING_ACK` , `MQTT_QUEUED_COMPLETE` }
An enumeration of queued message states.

Functions

- uint16_t `__mqtt_next_pid` (struct `mqtt_client` *client)
Generate a new next packet ID.
- ssize_t `__mqtt_send` (struct `mqtt_client` *client)
Handles egress client traffic.
- ssize_t `__mqtt_recv` (struct `mqtt_client` *client)
Handles ingress client traffic.
- ssize_t `__mqtt_puback` (struct `mqtt_client` *client, uint16_t packet_id)
Acknowledge an ingree publish with QOS==1.
- ssize_t `__mqtt_pubrec` (struct `mqtt_client` *client, uint16_t packet_id)
Acknowledge an ingree publish with QOS==2.
- ssize_t `__mqtt_pubrel` (struct `mqtt_client` *client, uint16_t packet_id)
Acknowledge an ingree PUBREC packet.
- ssize_t `__mqtt_pubcomp` (struct `mqtt_client` *client, uint16_t packet_id)
Acknowledge an ingree PUBREL packet.
- void `mqtt_mq_init` (struct `mqtt_message_queue` *mq, void *buf, size_t bufsz)
Initialize a message queue.
- void `mqtt_mq_clean` (struct `mqtt_message_queue` *mq)
Clear as many messages from the front of the queue as possible.
- struct `mqtt_queued_message` * `mqtt_mq_register` (struct `mqtt_message_queue` *mq, size_t nbytes)
Register a message that was just added to the buffer.
- struct `mqtt_queued_message` * `mqtt_mq_find` (const struct `mqtt_message_queue` *mq, enum `MQTTControlPacketType` control_type, const uint16_t *packet_id)
Find a message in the message queue.

4.4.1 Detailed Description

Developer documentation for the utilities used to implement the MQTT-C client.

Note

To deserialize a packet from a buffer use `mqtt_unpack_response` (it's the only function you need).

4.4.2 Macro Definition Documentation

4.4.2.1 `mqtt_mq_currsz`

```
#define mqtt_mq_currsz(  
    mq_ptr)
```

Value:

```
((mq_ptr)->curr >= (uint8_t*) ((mq_ptr)->queue_tail - 1)) ? 0 : ((uint8_t*) ((mq_ptr)->queue_tail - 1)) -  
    (mq_ptr)->curr)
```

Used internally to recalculate the `curr_sz`.

4.4.2.2 `mqtt_mq_get`

```
#define mqtt_mq_get(  
    mq_ptr,  
    index)
```

Value:

```
((struct mqtt_queued_message*) ((mq_ptr)->mem_end)) - 1 - index)
```

Returns the [mqtt_queued_message](#) at `index`.

Parameters

<i>mq_ptr</i>	A pointer to the message queue.
<i>index</i>	The index of the message.

Returns

The [mqtt_queued_message](#) at `index`.

4.4.2.3 `mqtt_mq_length`

```
#define mqtt_mq_length(  
    mq_ptr)
```

Value:

```
((struct mqtt_queued_message*) ((mq_ptr)->mem_end)) - (mq_ptr)->queue_tail)
```

Returns the number of messages in the message queue, `mq_ptr`.

4.4.3 Enumeration Type Documentation

4.4.3.1 `MQTTQueuedMessageState`

```
enum MQTTQueuedMessageState
```

An enumeration of queued message states.

Enumerator

MQTT_QUEUED_UNSENT	
MQTT_QUEUED_AWAITING_ACK	
MQTT_QUEUED_COMPLETE	

4.4.4 Function Documentation

4.4.4.1 __mqtt_next_pid()

```
uint16_t __mqtt_next_pid (
    struct mqtt_client * client)
```

Generate a new next packet ID.

Packet ID's are generated using a max-length LFSR.

Parameters

<i>client</i>	The MQTT client.
---------------	------------------

Returns

The new packet ID that should be used.

4.4.4.2 __mqtt_puback()

```
ssize_t __mqtt_puback (
    struct mqtt_client * client,
    uint16_t packet_id)
```

Acknowledge an ingree publish with QOS==1.

Parameters

in, out	<i>client</i>	The MQTT client.
in	<i>packet_id</i>	The packet ID of the ingress publish being acknowledged.

Returns

MQTT_OK upon success, an [MQTTErrors](#) otherwise.

4.4.4.3 __mqtt_pubcomp()

```
ssize_t __mqtt_pubcomp (
    struct mqtt_client * client,
    uint16_t packet_id)
```

Acknowledge an ingree PUBREL packet.

Parameters

in, out	<i>client</i>	The MQTT client.
in	<i>packet↔ _id</i>	The packet ID of the ingress PUBREL being acknowledged.

Returns

MQTT_OK upon success, an [MQTTErrors](#) otherwise.

4.4.4.4 __mqtt_pubrec()

```
ssize_t __mqtt_pubrec (
    struct mqtt_client * client,
    uint16_t packet_id)
```

Acknowledge an ingree publish with QOS==2.

Parameters

in, out	<i>client</i>	The MQTT client.
in	<i>packet↔ _id</i>	The packet ID of the ingress publish being acknowledged.

Returns

MQTT_OK upon success, an [MQTTErrors](#) otherwise.

4.4.4.5 __mqtt_pubrel()

```
ssize_t __mqtt_pubrel (
    struct mqtt_client * client,
    uint16_t packet_id)
```

Acknowledge an ingree PUBREC packet.

Parameters

in, out	<i>client</i>	The MQTT client.
in	<i>packet↔ _id</i>	The packet ID of the ingress PUBREC being acknowledged.

Returns

MQTT_OK upon success, an [MQTTErrors](#) otherwise.

4.4.4.6 __mqtt_recv()

```
ssize_t __mqtt_recv (
    struct mqtt_client * client)
```

Handles ingress client traffic.

Parameters

<i>client</i>	The MQTT client.
---------------	------------------

Returns

MQTT_OK upon success, an [MQTTErrors](#) otherwise.

4.4.4.7 __mqtt_send()

```
ssize_t __mqtt_send (  
    struct mqtt_client * client)
```

Handles egress client traffic.

Parameters

<i>client</i>	The MQTT client.
---------------	------------------

Returns

MQTT_OK upon success, an [MQTTErrors](#) otherwise.

4.4.4.8 mqtt_mq_clean()

```
void mqtt_mq_clean (  
    struct mqtt_message_queue * mq) [related]
```

Clear as many messages from the front of the queue as possible.

Note

Calls to this function are the *only* way to remove messages from the queue.

Parameters

<i>mq</i>	The message queue.
-----------	--------------------

4.4.4.9 mqtt_mq_find()

```
struct mqtt_queued_message * mqtt_mq_find (  
    const struct mqtt_message_queue * mq,  
    enum MQTTControlPacketType control_type,  
    const uint16_t * packet_id) [related]
```

Find a message in the message queue.

Parameters

	<i>mq</i>	The message queue.
in	<i>control_type</i>	The control type of the message you want to find.
in	<i>packet_id</i>	The packet ID of the message you want to find. Set to <code>NULL</code> if you don't want to specify a packet ID.

Returns

The found message. `NULL` if the message was not found.

4.4.4.10 `mqtt_mq_init()`

```
void mqtt_mq_init (
    struct mqtt_message_queue * mq,
    void * buf,
    size_t bufsz) [related]
```

Initialize a message queue.

Parameters

out	<i>mq</i>	The message queue to initialize.
in	<i>buf</i>	The buffer for this message queue.
in	<i>bufsz</i>	The number of bytes in the buffer.

4.4.4.11 `mqtt_mq_register()`

```
struct mqtt_queued_message * mqtt_mq_register (
    struct mqtt_message_queue * mq,
    size_t nbytes) [related]
```

Register a message that was just added to the buffer.

Note

This function should be called immediately following a call to a packer function that returned a positive value. The positive value (number of bytes packed) should be passed to this function.

Parameters

	<i>mq</i>	The message queue.
in	<i>nbytes</i>	The number of bytes that were just packed.

Note

This function will step `mqtt_message_queue::curr` and update `mqtt_message_queue::curr_sz`.

Returns

The newly added struct `mqtt_queued_message`.

4.5 Platform abstraction layer

Documentation of the types and calls required to port MQTT-C to a new platform.

Functions

- `ssize_t mqtt_pal_sendall` (`mqtt_pal_socket_handle` fd, `const void *buf`, `size_t len`, `int flags`)
Sends all the bytes in a buffer.
- `ssize_t mqtt_pal_recvall` (`mqtt_pal_socket_handle` fd, `void *buf`, `size_t bufsz`, `int flags`)
Non-blocking receive all the byte available.

4.5.1 Detailed Description

Documentation of the types and calls required to port MQTT-C to a new platform.

`mqtt_pal.h` is the *only* header file included in `mqtt.c`. Therefore, to port MQTT-C to a new platform the following types, functions, constants, and macros must be defined in `mqtt_pal.h`:

- Types:
 - `size_t`, `ssize_t`
 - `uint8_t`, `uint16_t`, `uint32_t`
 - `va_list`
 - `mqtt_pal_time_t` : return type of `MQTT_PAL_TIME()`
 - `mqtt_pal_mutex_t` : type of the argument that is passed to `MQTT_PAL_MUTEX_LOCK` and `MQTT_PAL_MUTEX_RELEASE`
- Functions:
 - `memcpy`, `strlen`
 - `va_start`, `va_arg`, `va_end`
- Constants:
 - `INT_MIN`

Additionally, three macro's are required:

- `MQTT_PAL_HTONS(s)` : host-to-network endian conversion for `uint16_t`.
- `MQTT_PAL_NTOHS(s)` : network-to-host endian conversion for `uint16_t`.
- `MQTT_PAL_TIME()` : returns [type: `mqtt_pal_time_t`] current time in seconds.
- `MQTT_PAL_MUTEX_LOCK(mtx_pointer)` : macro that locks the mutex pointed to by `mtx_pointer`.
- `MQTT_PAL_MUTEX_RELEASE(mtx_pointer)` : macro that unlocks the mutex pointed to by `mtx_pointer`.

Lastly, `mqtt_pal_sendall` and `mqtt_pal_recvall`, must be implemented in `mqtt_pal.c` for sending and receiving data using the platforms socket calls.

4.5.2 Function Documentation

4.5.2.1 mqtt_pal_recvall()

```
ssize_t mqtt_pal_recvall (  
    mqtt_pal_socket_handle fd,  
    void * buf,  
    size_t bufsz,  
    int flags)
```

Non-blocking receive all the byte available.

Parameters

in	<i>fd</i>	The file-descriptor (or handle) of the socket.
in	<i>buf</i>	A pointer to the receive buffer.
in	<i>bufsz</i>	The max number of bytes that can be put into <i>buf</i> .
in	<i>flags</i>	Flags which are passed to the underlying socket.

Returns

The number of bytes received if successful, an [MQTTError](#)s otherwise.

Note about the error handling:

- On an error, if some bytes have been processed already, this function should return the number of bytes successfully processed. (partial success)
- Otherwise, if the error is an equivalent of EAGAIN, return 0.
- Otherwise, return MQTT_ERROR_SOCKET_ERROR.

4.5.2.2 mqtt_pal_sendall()

```
ssize_t mqtt_pal_sendall (
    mqtt_pal_socket_handle fd,
    const void * buf,
    size_t len,
    int flags)
```

Sends all the bytes in a buffer.

Parameters

in	<i>fd</i>	The file-descriptor (or handle) of the socket.
in	<i>buf</i>	A pointer to the first byte in the buffer to send.
in	<i>len</i>	The number of bytes to send (starting at <i>buf</i>).
in	<i>flags</i>	Flags which are passed to the underlying socket.

Returns

The number of bytes sent if successful, an [MQTTError](#)s otherwise.

Note about the error handling:

- On an error, if some bytes have been processed already, this function should return the number of bytes successfully processed. (partial success)
- Otherwise, if the error is an equivalent of EAGAIN, return 0.
- Otherwise, return MQTT_ERROR_SOCKET_ERROR.

4.6 ADC3: Analog-to-Digital Converter with Context

This file contains API prototypes and other datatypes for ADC module.

Typedefs

- typedef uint16_t [adc_result_t](#)
This typedef should be used for result of A/D conversion.

Enumerations

- enum [ADC_channel_t](#) {
 [channel_ANA0](#) = 0x0 , [channel_ANA4](#) = 0x4 , [channel_VSS](#) = 0x3B , [channel_Temp](#) = 0x3C ,
 [channel_DAC1](#) = 0x3D , [channel_FVR_Buffer1](#) = 0x3E , [channel_FVR_Buffer2](#) = 0x3F }
This enumeration contains available ADC channels.
- enum [ADC_context_t](#) { [mTouchContext](#) }
This enumeration contains available ADC contexts.

Functions

- void [ADC_Initialize](#) (void)
This API initializes the ADC module. This routine must be called before any other ADC routine.
- void [ADC_EnableChannelSequencer](#) (void)
This API enables ADC channel sequencer. ADC module should be initialized using [ADC_Initialize](#) before calling this API.
- void [ADC_DisableChannelSequencer](#) (void)
This API disables ADC channel sequencer. ADC module should be initialized using [ADC_Initialize](#) before calling this API.
- void [ADC_StartChannelSequencer](#) (void)
This API starts the channel sequencer by enabling GO bit. Channel sequencer should be enabled using [ADC_EnableChannelSequencer](#) before calling this API.
- void [ADC_SelectContext](#) ([ADC_context_t](#) context)
This API is used to select an ADC context to perform read/write operations on context specific registers.
- void [ADC_EnableChannelScan](#) ([ADC_context_t](#) context)
This API is used to include a channel context in the scan sequence. Correct context should be selected using [ADC_SelectContext](#) before calling this API.
- void [ADC_DisableChannelScan](#) ([ADC_context_t](#) context)
This API is used to exclude a channel context from the scan sequence. Correct context should be selected using [ADC_SelectContext](#) before calling this API.
- void [ADC_StartConversion](#) ([ADC_channel_t](#) channel)
This API starts A/D conversion on selected channel. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.
- bool [ADC_IsConversionDone](#) (void)
This API checks if ongoing A/D conversion is complete.
- [adc_result_t](#) [ADC_GetConversionResult](#) (void)
This API is used to retrieve the result of latest A/D conversion. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.
- [adc_result_t](#) [ADC_GetSingleConversion](#) ([ADC_channel_t](#) channel)
This API is used to retrieve the result of single A/D conversion on given channel. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

- void [ADC_StopConversion](#) (void)
This API is used to stop ongoing A/D conversion. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.
- void [ADC_SetStopOnInterrupt](#) (void)
This API is used enable Stop On Interrupt bit for selected A/D context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.
- void [ADC_DischargeSampleCapacitor](#) (void)
This API is used to discharge input sample capacitor for selected context by setting the channel to AVss. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.
- void [ADC_LoadAcquisitionRegister](#) (uint16_t acquisitionValue)
This API is used to load ADC Acquisition Time Control register with specified value. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.
- void [ADC_SetPrechargeTime](#) (uint16_t prechargeTime)
This API is used to load ADC Precharge Time Control register with specified value. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.
- void [ADC_SetRepeatCount](#) (uint8_t repeatCount)
This API is used to load repeat counter for given context with specified value. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.
- uint8_t [ADC_GetCurrentCountofConversions](#) (void)
This API retrieves the current value of ADC Repeat Count register for selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.
- void [ADC_ClearAccumulator](#) (void)
This API clears the accumulator of selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.
- uint24_t [ADC_GetAccumulatorValue](#) (void)
This API retrieves 18-bit value of ADC accumulator of selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.
- bool [ADC_HasAccumulatorOverflowed](#) (void)
This API is used to determine whether ADC accumulator has overflowed for selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.
- uint16_t [ADC_GetFilterValue](#) (void)
This API retrieves the value of ADC Filter register of selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.
- uint16_t [ADC_GetPreviousResult](#) (void)
This API retrieves the value of ADC Previous register of selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.
- void [ADC_DefineSetPoint](#) (uint16_t setPoint)
This API used to set value of ADC Threshold Set-point of selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.
- void [ADC_SetUpperThreshold](#) (uint16_t upperThreshold)
This API used to set value of ADC Upper Threshold register of selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.
- void [ADC_SetLowerThreshold](#) (uint16_t lowerThreshold)
This API used to set value of ADC Lower Threshold register of selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.
- uint16_t [ADC_GetErrorCalculation](#) (void)
This API retrieves the value of ADC Set-point Error register of selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.
- void [ADC_EnableDoubleSampling](#) (void)
This API enables double-sampling bit for selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.
- void [ADC_EnableContinuousConversion](#) (void)
This API enables continuous conversion for selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

- void [ADC_DisableContinuousConversion](#) (void)
This API disables continuous conversion for selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.
- bool [ADC_HasErrorCrossedUpperThreshold](#) (void)
This API is used to determine if ADC error has crossed the upper threshold for selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.
- bool [ADC_HasErrorCrossedLowerThreshold](#) (void)
This API is used to determine if ADC error is less than the lower threshold for selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.
- uint8_t [ADC_GetConversionStageStatus](#) (void)
This API is used retrieve the multi-stage status for selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.
- void [ADC_EnableChargePump](#) (void)
This API is used to enable ADC charge pump.
- void [ADC_DisableChargePump](#) (void)
This API is used to disable ADC charge pump.
- void [ADC_SetADInterruptHandler](#) (void(*InterruptHandler)(void))
Setter for A/D conversion ISR.
- void [ADC_SetActiveClockTuningInterruptHandler](#) (void(*InterruptHandler)(void))
Setter for A/D Active Clock Tuning ISR.
- void [ADC_SetContext1ThresholdInterruptHandler](#) (void(*InterruptHandler)(void))
Setter for Context-1 threshold interrupt.
- void [ADC_SetContext2ThresholdInterruptHandler](#) (void(*InterruptHandler)(void))
Setter for Context-2 threshold interrupt.
- void [ADC_SetContext3ThresholdInterruptHandler](#) (void(*InterruptHandler)(void))
Setter for Context-3 threshold interrupt.
- void [ADC_SetContext4ThresholdInterruptHandler](#) (void(*InterruptHandler)(void))
Setter for Context-4 threshold interrupt.

4.6.1 Detailed Description

This file contains API prototypes and other datatypes for ADC module.

ADC Generated Driver API Header File

@Company Microchip Technology Inc.

@File Name [adc.h](#)

@Summary This is the generated header file for the ADC driver using PIC10 / PIC12 / PIC16 / PIC18 MCUs

@Description This header file provides APIs for driver for ADC. Generation Information : Product Revision : PIC10 / PIC12 / PIC16 / PIC18 MCUs - 1.81.8 Device : PIC18F47Q84 Driver Version : 1.0.0 The generated drivers are tested against the following: Compiler : XC8 2.36 and above MPLAB : MPLAB X 6.00

4.6.2 Typedef Documentation

4.6.2.1 `adc_result_t`

```
typedef uint16_t adc\_result\_t
```

This typedef should be used for result of A/D conversion.

4.6.3 Enumeration Type Documentation

4.6.3.1 ADC_channel_t

enum [ADC_channel_t](#)

This enumeration contains available ADC channels.

Enumerator

channel_ANA0	
channel_ANA4	
channel_VSS	
channel_Temp	
channel_DAC1	
channel_FVR_Buffer1	
channel_FVR_Buffer2	

4.6.3.2 ADC_context_t

enum [ADC_context_t](#)

This enumeration contains available ADC contexts.

Enumerator

mTouchContext	
---------------	--

4.6.4 Function Documentation

4.6.4.1 ADC_ClearAccumulator()

```
void ADC_ClearAccumulator (
    void ) [inline]
```

This API clears the accumulator of selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

Parameters

<i>void</i>	
-------------	--

Returns

void

```
void main(void)
{
    ADC_Initialize();
    ADC_SelectContext(CONTEXT_1);
    ADC_ClearAccumulator();

    //Application code

    while(1);
}
```

4.6.4.2 ADC_DefineSetPoint()

```
void ADC_DefineSetPoint (
    uint16_t setPoint)
```

This API used to set value of ADC Threshold Set-point of selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

Parameters

in	<i>setPoint</i>	- 16-bit value for set point.
----	-----------------	-------------------------------

Returns

void

```
void main(void)
{
    uint16_t setPoint = 90;
    ADC_Initialize();
    ADC_SelectContext (CONTEXT_1);
    ADC_DefineSetPoint (setPoint);

    //Application code

    while (1);
}
```

4.6.4.3 ADC_DisableChannelScan()

```
void ADC_DisableChannelScan (
    ADC_context_t context)
```

This API is used to exclude a channel context from the scan sequence. Correct context should be selected using [ADC_SelectContext](#) before calling this API.

Parameters

in	<i>context</i>	- Context which needs to be excluded from scan. Refer ADC_context_t for available contexts
----	----------------	--

Returns

void

```
void main(void)
{
    ADC_Initialize();
    ADC_EnableChannelScan (CONTEXT_1);
    ADC_DisableChannelScan (CONTEXT_2);
    ADC_DisableChannelScan (CONTEXT_3);
    ADC_EnableChannelScan (CONTEXT_4);

    //Application code

    while (1);
}
```

4.6.4.4 ADC_DisableChannelSequencer()

```
void ADC_DisableChannelSequencer (
    void ) [inline]
```

This API disables ADC channel sequencer. ADC module should be initialized using [ADC_Initialize](#) before calling this API.

Parameters

<i>void</i>	
-------------	--

Returns

void

```
void main(void)
{
    adc_result_t convertedValue;
    ADC_Initialize();
    ADC_DisableChannelSequencer();    //Disable scanner
    ADC_SelectContext(CONTEXT_1);
    convertedValue = ADC_GetSingleConversion(channel_ANA0);

    while(1);
}
```

4.6.4.5 ADC_DisableChargePump()

```
void ADC_DisableChargePump (
    void ) [inline]
```

This API is used to disable ADC charge pump.

Parameters

<i>void</i>	
-------------	--

Returns

void

```
void main(void)
{
    ADC_Initialize();
    ADC_DisableChargePump();
    ADC_StartConversion(channel_ANA0);

    //Application code

    while(1);
}
```

4.6.4.6 ADC_DisableContinuousConversion()

```
void ADC_DisableContinuousConversion (
    void ) [inline]
```

This API disables continuous conversion for selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

Parameters

<i>void</i>	
-------------	--

Returns

void

```
void main(void)
{
    ADC_Initialize();
    ADC_SelectContext (CONTEXT_1);
    ADC_DisableContinuousConversion();
    ADC_StartConversion (channel_ANA0);

    //Application code

    while(1);
}
```

4.6.4.7 ADC_DischargeSampleCapacitor()

```
void ADC_DischargeSampleCapacitor (
    void ) [inline]
```

This API is used to discharge input sample capacitor for selected context by setting the channel to AVss. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

Parameters

<i>void</i>	
-------------	--

Returns

void

```
void main(void)
{
    adc_result_t convertedValue;
    ADC_Initialize();
    ADC_SelectContext (CONTEXT_1);
    ADC_DischargeSampleCapacitor();
    ADC_StartConversion (channel_ANA0);
    while (!ADC_IsConversionDone());
    convertedValue = ADC_GetConversionResult();

    //Application code

    while(1);
}
```

4.6.4.8 ADC_EnableChannelScan()

```
void ADC_EnableChannelScan (
    ADC_context_t context)
```

This API is used to include a channel context in the scan sequence. Correct context should be selected using [ADC_SelectContext](#) before calling this API.

Parameters

in	<i>context</i>	- Context which needs to be included in scan. Refer ADC_context_t for available contexts
----	----------------	--

Returns

void

```
void main(void)
{
    ADC_Initialize();
    ADC_EnableChannelScan (CONTEXT_1);
    ADC_EnableChannelScan (CONTEXT_2);

    //Application code

    while(1);
}
```

4.6.4.9 ADC_EnableChannelSequencer()

```
void ADC_EnableChannelSequencer (
    void ) [inline]
```

This API enables ADC channel sequencer. ADC module should be initialized using [ADC_Initialize](#) before calling this API.

Parameters

<i>void</i>	
-------------	--

Returns

void

```
void main(void)
{
    adc_result_t convertedValue1, convertedValue2, convertedValue3, convertedValue4;
    ADC_Initialize();
    ADC_EnableChannelSequencer();    //Enable auto-scanning if not already enabled
    ADC_StartChannelSequencer();    // Software starts the sequencer

    while(!ADC_IsConversionDone());
    ADC_SelectContext (CONTEXT_1);
    convertedValue1 = ADC_GetConversionResult();
    ADC_SelectContext (CONTEXT_2);
    convertedValue2 = ADC_GetConversionResult();
    ADC_SelectContext (CONTEXT_3);
    convertedValue3 = ADC_GetConversionResult();
    ADC_SelectContext (CONTEXT_4);
    convertedValue4 = ADC_GetConversionResult();

    while(1);
}
```

4.6.4.10 ADC_EnableChargePump()

```
void ADC_EnableChargePump (
    void ) [inline]
```

This API is used to enable ADC charge pump.

Parameters

<i>void</i>	
-------------	--

Returns

void

```
void main(void)
{
    ADC_Initialize();
    ADC_EnableChargePump();
    ADC_StartConversion(channel_ANA0);

    //Application code

    while(1);
}
```

4.6.4.11 ADC_EnableContinuousConversion()

```
void ADC_EnableContinuousConversion (
    void ) [inline]
```

This API enables continuous conversion for selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

Parameters

<code>void</code>	
-------------------	--

Returns

`void`

```
void main(void)
{
    ADC_Initialize();
    ADC_SelectContext (CONTEXT_1);
    ADC_EnableContinuousConversion();
    ADC_StartConversion(channel_ANA0);

    //Application code
    while(1);
}
```

4.6.4.12 ADC_EnableDoubleSampling()

```
void ADC_EnableDoubleSampling (
    void ) [inline]
```

This API enables double-sampling bit for selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

Parameters

<code>void</code>	
-------------------	--

Returns

`void`

```
void main(void)
{
    ADC_Initialize();
    ADC_SelectContext (CONTEXT_1);
    ADC_EnableDoubleSampling();
    ADC_StartConversion(channel_ANA0);

    //Application code
    while(1);
}
```

4.6.4.13 ADC_GetAccumulatorValue()

```
uint24_t ADC_GetAccumulatorValue (
    void )
```

This API retrieves 18-bit value of ADC accumulator of selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

Parameters

<code>void</code>	
-------------------	--

Returns

Value of ADC accumulator.

```
void main(void)
{
    uint24_t accumulatorValue;
    ADC_Initialize();
    ADC_SelectContext(CONTEXT_1);
    accumulatorValue = ADC_GetAccumulatorValue();

    //Application code

    while(1);
}
```

4.6.4.14 ADC_GetConversionResult()

```
adc_result_t ADC_GetConversionResult (
    void )
```

This API is used to retrieve the result of latest A/D conversion. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

Parameters

void	
------	--

Returns

The result of A/D conversion. Refer [adc_result_t](#)

```
void main(void)
{
    adc_result_t convertedValue;
    ADC_Initialize();
    ADC_SelectContext(CONTEXT_1);
    ADC_StartConversion(channel_ANA0);
    while(!ADC_IsConversionDone());
    convertedValue = ADC_GetConversionResult();

    //Application code

    while(1);
}
```

4.6.4.15 ADC_GetConversionStageStatus()

```
uint8_t ADC_GetConversionStageStatus (
    void )
```

This API is used to retrieve the multi-stage status for selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

Parameters

void	
------	--

Returns

Contents of ADC STATUS register.

```
void main(void)
{
    uint8_t status;
    ADC_Initialize();
    ADC_SelectContext (CONTEXT_1);
    ADC_StartConversion (channel_ANA0);
    status = ADC_GetConversionStageStatus ();

    //Application code

    while (1);
}
```

4.6.4.16 ADC_GetCurrentCountofConversions()

```
uint8_t ADC_GetCurrentCountofConversions (
    void )
```

This API retrieves the current value of ADC Repeat Count register for selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

Parameters

<i>void</i>	
-------------	--

Returns

Current value of ADC Repeat Count register

```
void main(void)
{
    uint8_t conversionCount;
    ADC_Initialize();
    ADC_SelectContext (CONTEXT_1);
    conversionCount = ADC_GetCurrentCountofConversions ();

    //Application code

    while (1);
}
```

4.6.4.17 ADC_GetErrorCalculation()

```
uint16_t ADC_GetErrorCalculation (
    void )
```

This API retrieves the value of ADC Set-point Error register of selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

Parameters

<i>void</i>	
-------------	--

Returns

16-bit value obtained from ADERRH and ADERRL registers.

```
void main(void)
{
    uint16_t errorCalc;
    ADC_Initialize();
    ADC_SelectContext (CONTEXT_1);
    ADC_StartConversion (channel_ANA0);
    errorCalc = ADC_GetErrorCalculation ();

    //Application code

    while (1);
}
```

4.6.4.18 ADC_GetFilterValue()

```
uint16_t ADC_GetFilterValue (
    void )
```

This API retrieves the value of ADC Filter register of selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

Parameters

<i>void</i>	
-------------	--

Returns

16-bit value obtained from ADFLTRH and ADFLTRL registers.

```
void main(void)
{
    uint16_t filterValue;
    ADC_Initialize();
    ADC_SelectContext (CONTEXT_1);
    filterValue = ADC_GetFilterValue();

    //Application code

    while (1);
}
```

4.6.4.19 ADC_GetPreviousResult()

```
uint16_t ADC_GetPreviousResult (
    void )
```

This API retrieves the value of ADC Previous register of selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

Parameters

<i>void</i>	
-------------	--

Returns

16-bit value obtained from ADPREVH and ADPREVL registers.

```
void main(void)
{
    uint16_t previousResult;
    ADC_Initialize();
    ADC_SelectContext (CONTEXT_1);
    previousResult = ADC_GetPreviousResult();

    //Application code

    while (1);
}
```

4.6.4.20 ADC_GetSingleConversion()

```
adc_result_t ADC_GetSingleConversion (
    ADC_channel_t channel)
```

This API is used to retrieve the result of single A/D conversion on given channel. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

Parameters

in	<i>channel</i>	- Analog channel number on which A/D conversion has to be applied. Refer ADC_channel_t for available channels
----	----------------	---

Returns

The result of A/D conversion. Refer [adc_result_t](#)

```
void main(void)
{
    adc_result_t convertedValue;
    ADC_Initialize();
    ADC_DisableChannelSequencer();    //Disable scanner
    ADC_SelectContext (CONTEXT_1);
    convertedValue = ADC_GetSingleConversion(channel_ANA0);

    //Application code
    while(1);
}
```

4.6.4.21 ADC_HasAccumulatorOverflowed()

```
bool ADC_HasAccumulatorOverflowed (
    void )
```

This API is used to determine whether ADC accumulator has overflowed for selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

Parameters

<i>void</i>	
-------------	--

Returns

Status of accumulator. true - ADC accumulator has overflowed. false - ADC accumulator has not overflowed.

```
void main(void)
{
    bool hasAccOvf;
    ADC_Initialize();
    ADC_SelectContext (CONTEXT_1);
    ADC_StartConversion(channel_ANA0);
    hasAccOvf = ADC_HasAccumulatorOverflowed();

    //Application code
    while(1);
}
```

4.6.4.22 ADC_HasErrorCrossedLowerThreshold()

```
bool ADC_HasErrorCrossedLowerThreshold (
    void )
```

This API is used to determine if ADC error is less than the lower threshold for selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

Parameters

<i>void</i>	
-------------	--

Returns

Status of operation. true - if $ERR < LTH$. false - if $ERR \geq LTH$.

```
void main(void)
{
    bool ulThr;
    ADC_Initialize();
    ADC_SelectContext (CONTEXT_1);
    ADC_StartConversion (channel_ANA0);
    ulThr = ADC_HasErrorCrossedLowerThreshold();

    //Application code

    while (1);
}
```

4.6.4.23 ADC_HasErrorCrossedUpperThreshold()

```
bool ADC_HasErrorCrossedUpperThreshold (
    void )
```

This API is used to determine if ADC error has crossed the upper threshold for selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

Parameters

<i>void</i>	
-------------	--

Returns

Status of operation. true - if $ERR > UTH$ false - if $ERR \leq UTH$

```
void main(void)
{
    bool uThr;
    ADC_Initialize();
    ADC_SelectContext (CONTEXT_1);
    ADC_StartConversion (channel_ANA0);
    uThr = ADC_HasErrorCrossedUpperThreshold();

    //Application code

    while (1);
}
```

4.6.4.24 ADC_Initialize()

```
void ADC_Initialize (
    void )
```

This API initializes the ADC module. This routine must be called before any other ADC routine.

Parameters

<i>void.</i>	
--------------	--

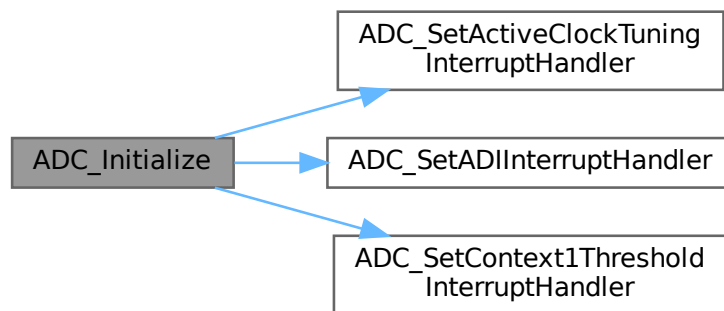
Returns

void.

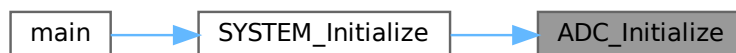
```
void main(void)
{
    adc_result_t convertedValue;
    ADC_Initialize();
    ADC_SelectContext (CONTEXT_1);
    ADC_StartConversion (channel_ANA0);
    while (!ADC_IsConversionDone());
    convertedValue = ADC_GetConversionResult();

    while (1);
}
```

Here is the call graph for this function:



Here is the caller graph for this function:

**4.6.4.25 ADC_IsConversionDone()**

```
bool ADC_IsConversionDone (
    void )
```

This API checks if ongoing A/D conversion is complete.

Parameters

void	
------	--

Returns

bool - Status of A/D conversion. true - if conversion is complete. false - if conversion is ongoing.

```
void main(void)
{
    adc_result_t convertedValue;
    ADC_Initialize();
    ADC_SelectContext (CONTEXT_1);
    ADC_StartConversion (channel_ANA0);
    while (!ADC_IsConversionDone());
    convertedValue = ADC_GetConversionResult();

    //Application code

    while(1);
}
```

4.6.4.26 ADC_LoadAcquisitionRegister()

```
void ADC_LoadAcquisitionRegister (
    uint16_t acquisitionValue)
```

This API is used to load ADC Acquisition Time Control register with specified value. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

Parameters

in	<i>acquisitionValue</i>	- Value to be loaded in the acquisition time control register.
----	-------------------------	--

Returns

void

```
void main(void)
{
    uint16_t acquisitionValue = 98;
    ADC_Initialize();
    ADC_SelectContext (CONTEXT_1);
    ADC_LoadAcquisitionRegister (acquisitionValue);

    //Application code

    while(1);
}
```

4.6.4.27 ADC_SelectContext()

```
void ADC_SelectContext (
    ADC_context_t context) [inline]
```

This API is used to select an ADC context to perform read/write operations on context specific registers.

Parameters

in	<i>context</i>	- Context to be selected. Refer ADC_context_t for available contexts
----	----------------	--

Returns

void

```
void main(void)
{
    adc_result_t convertedValue;
    ADC_Initialize();
    ADC_SelectContext (CONTEXT_1);
    ADC_StartConversion (channel_ANA0);
    while (!ADC_IsConversionDone());
    convertedValue = ADC_GetConversionResult();

    while(1);
}
```

4.6.4.28 ADC_SetActiveClockTuningInterruptHandler()

```
void ADC_SetActiveClockTuningInterruptHandler (
    void(* InterruptHandler ) (void))
```

Setter for A/D Active Clock Tuning ISR.

Parameters

in	<i>InterruptHandler</i>	- Pointer to custom ISR.
----	-------------------------	--------------------------

Returns

void

```
void customACTIHandler(void)
{
    //Custom ISR code
}

void main(void)
{
    ADC_Initialize();
    ADC_SetActiveClockTuningInterruptHandler(customACTIHandler);
    ADC_SelectContext (CONTEXT_1);
    ADC_StartConversion(channel_ANA0);

    //Replace with your application code
    while(1)
    {
    }
}
```

Here is the caller graph for this function:



4.6.4.29 ADC_SetADIInterruptHandler()

```
void ADC_SetADIInterruptHandler (
    void(* InterruptHandler ) (void))
```

Setter for A/D conversion ISR.

Parameters

in	<i>InterruptHandler</i>	- Pointer to custom ISR.
----	-------------------------	--------------------------

Returns

void

```

void customADHandler(void)
{
    //Custom ISR code
}

void main(void)
{
    ADC_Initialize();
    ADC_SetADInterruptHandler(customADHandler);
    ADC_SelectContext(CONTEXT_1);
    ADC_StartConversion(channel_ANA0);

    //Replace with your application code
    while(1)
    {
    }
}

```

Here is the caller graph for this function:

**4.6.4.30 ADC_SetContext1ThresholdInterruptHandler()**

```

void ADC_SetContext1ThresholdInterruptHandler (
    void(* InterruptHandler ) (void))

```

Setter for Context-1 threshold interrupt.

Parameters

in	<i>InterruptHandler</i>	- Pointer to custom ISR.
----	-------------------------	--------------------------

Returns

void

```

void customADCH1InterruptHandler(void)
{
    //Custom ISR code
}

void main(void)
{
    ADC_Initialize();
    ADC_SetContext1ThresholdInterruptHandler(customADCH1InterruptHandler);
    ADC_SelectContext(CONTEXT_1);
    ADC_StartConversion(channel_ANA0);

    //Replace with your application code
    while(1)
    {
    }
}

```

Here is the caller graph for this function:



4.6.4.31 ADC_SetContext2ThresholdInterruptHandler()

```
void ADC_SetContext2ThresholdInterruptHandler (
    void(* InterruptHandler ) (void))
```

Setter for Context-2 threshold interrupt.

Parameters

in	<i>InterruptHandler</i>	- Pointer to custom ISR.
----	-------------------------	--------------------------

Returns

void

```
void customADCH2InterruptHandler(void)
{
    //Custom ISR code
}

void main(void)
{
    ADC_Initialize();
    ADC_SetContext2ThresholdInterruptHandler(customADCH1InterruptHandler);
    ADC_SelectContext (CONTEXT_2);
    ADC_StartConversion (channel_ANA0);

    //Replace with your application code
    while (1)
    {
    }
}
```

4.6.4.32 ADC_SetContext3ThresholdInterruptHandler()

```
void ADC_SetContext3ThresholdInterruptHandler (
    void(* InterruptHandler ) (void))
```

Setter for Context-3 threshold interrupt.

Parameters

in	<i>InterruptHandler</i>	- Pointer to custom ISR.
----	-------------------------	--------------------------

Returns

void

```

void customADCH3InterruptHandler(void)
{
    //Custom ISR code
}

void main(void)
{
    ADC_Initialize();
    ADC_SetContext3ThresholdInterruptHandler(customADCH1InterruptHandler);
    ADC_SelectContext (CONTEXT_3);
    ADC_StartConversion(channel_ANA0);

    //Replace with your application code
    while(1)
    {
    }
}

```

4.6.4.33 ADC_SetContext4ThresholdInterruptHandler()

```

void ADC_SetContext4ThresholdInterruptHandler (
    void(* InterruptHandler ) (void))

```

Setter for Context-4 threshold interrupt.

Parameters

in	<i>InterruptHandler</i>	- Pointer to custom ISR.
----	-------------------------	--------------------------

Returns

void

```

void customADCH4InterruptHandler(void)
{
    //Custom ISR code
}

void main(void)
{
    ADC_Initialize();
    ADC_SetContext4ThresholdInterruptHandler(customADCH1InterruptHandler);
    ADC_SelectContext (CONTEXT_4);
    ADC_StartConversion(channel_ANA0);

    //Replace with your application code
    while(1)
    {
    }
}

```

4.6.4.34 ADC_SetLowerThreshold()

```

void ADC_SetLowerThreshold (
    uint16_t lowerThreshold)

```

This API used to set value of ADC Lower Threshold register of selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

Parameters

in	<i>lowerThreshold</i>	- 16-bit value for lower threshold.
----	-----------------------	-------------------------------------

Returns

void

```
void main(void)
{
    uint16_t lowerThreshold = 500;
    ADC_Initialize();
    ADC_SelectContext(CONTEXT_1);
    ADC_SetLowerThreshold(lowerThreshold);

    //Application code
    while(1);
}
```

4.6.4.35 ADC_SetPrechargeTime()

```
void ADC_SetPrechargeTime (
    uint16_t prechargeTime)
```

This API is used to load ADC Precharge Time Control register with specified value. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

Parameters

in	<i>prechargeTime</i>	- Value to be loaded in the precharge time control register.
----	----------------------	--

Returns

void

```
void main(void)
{
    uint16_t prechargeTime = 98;
    ADC_Initialize();
    ADC_SelectContext(CONTEXT_1);
    ADC_SetPrechargeTime(prechargeTime);

    //Application code
    while(1);
}
```

4.6.4.36 ADC_SetRepeatCount()

```
void ADC_SetRepeatCount (
    uint8_t repeatCount) [inline]
```

This API is used to load repeat counter for given context with specified value. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

Parameters

in	<i>repeatCount</i>	- Value to be loaded to repeat counter.
----	--------------------	---

Returns

void

```
void main(void)
{
    uint8_t repeatCount = 32;
    ADC_Initialize();
    ADC_SelectContext(CONTEXT_1);
    ADC_SetRepeatCount(repeatCount);

    //Application code
    while(1);
}
```

4.6.4.37 ADC_SetStopOnInterrupt()

```
void ADC_SetStopOnInterrupt (
    void ) [inline]
```

This API is used enable Stop On Interrupt bit for selected A/D context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

Parameters

<i>void</i>	
-------------	--

Returns

void

```
void main(void)
{
    ADC_Initialize();
    ADC_SelectContext (CONTEXT_1);
    ADC_EnableContinuousConversion();
    ADC_SetStopOnInterrupt();

    //Application code

    while(1);
}
```

4.6.4.38 ADC_SetUpperThreshold()

```
void ADC_SetUpperThreshold (
    uint16_t upperThreshold)
```

This API used to set value of ADC Upper Threshold register of selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

Parameters

in	<i>upperThreshold</i>	- 16-bit value for upper threshold.
----	-----------------------	-------------------------------------

Returns

void

```
void main(void)
{
    uint16_t upperThreshold = 5000;
    ADC_Initialize();
    ADC_SelectContext (CONTEXT_1);
    ADC_SetUpperThreshold(upperThreshold);

    //Application code

    while(1);
}
```

4.6.4.39 ADC_StartChannelSequencer()

```
void ADC_StartChannelSequencer (
    void ) [inline]
```

This API starts the channel sequencer by enabling GO bit. Channel sequencer should be enabled using [ADC_EnableChannelSequencer](#) before calling this API.

Parameters

<i>void</i>	
-------------	--

Returns

void

```
void main(void)
{
    adc_result_t convertedValue1, convertedValue2, convertedValue3, convertedValue4;
    ADC_Initialize();
    ADC_EnableChannelSequencer();    //Enable auto-scanning if not already enabled
    ADC_StartChannelSequencer();    // Software starts the sequencer

    while(!ADC_IsConversionDone());
    ADC_SelectContext(CONTEXT_1);
    convertedValue1 = ADC_GetConversionResult();
    ADC_SelectContext(CONTEXT_2);
    convertedValue2 = ADC_GetConversionResult();
    ADC_SelectContext(CONTEXT_3);
    convertedValue3 = ADC_GetConversionResult();
    ADC_SelectContext(CONTEXT_4);
    convertedValue4 = ADC_GetConversionResult();

    while(1);
}
```

4.6.4.40 ADC_StartConversion()

```
void ADC_StartConversion (
    ADC_channel_t channel)
```

This API starts A/D conversion on selected channel. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

Parameters

in	<i>channel</i>	- Analog channel number on which A/D conversion has to be applied. Refer ADC_channel_t for available list of channels
----	----------------	---

Returns

void

```
void main(void)
{
    adc_result_t convertedValue;
    ADC_Initialize();
    ADC_SelectContext(CONTEXT_1);
    ADC_StartConversion(channel_ANA0);
    while(!ADC_IsConversionDone());
    convertedValue = ADC_GetConversionResult();

    //Application code

    while(1);
}
```

4.6.4.41 ADC_StopConversion()

```
void ADC_StopConversion (
    void ) [inline]
```

This API is used to stop ongoing A/D conversion. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

Parameters

<i>void</i>	
-------------	--

Returns

void

```
void main(void)
{
    adc_result_t convertedValue;
    ADC_Initialize();
    ADC_SelectContext(CONTEXT_1);
    ADC_StartConversion(channel_ANA0);
    ADC_StopConversion();

    //Application code

    while(1);
}
```

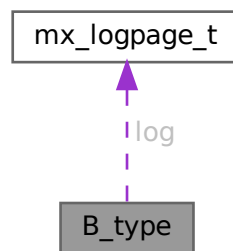

Chapter 5

Data Structure Documentation

5.1 B_type Struct Reference

```
#include <mxcmd.h>
```

Collaboration diagram for B_type:



Data Fields

- volatile bool [ten_sec_flag](#)
- volatile bool [one_sec_flag](#)
- volatile bool [FM80_charged](#)
- volatile bool [pv_high](#)
- volatile bool [pv_update](#)
- volatile bool [once](#)
- volatile bool [a_switch](#) [D_SW_COUNT]
- volatile bool [a_trigger](#) [D_SW_COUNT]
- volatile bool [a_type](#) [D_SW_COUNT]
- volatile uint16_t [pacing](#)
- volatile uint16_t [rx_count](#)
- volatile uint16_t [flush](#)
- volatile uint16_t [pv_prev](#)

- volatile uint16_t [day_check](#)
- volatile uint16_t [node_id](#)
- volatile uint16_t [dim_delay](#)
- volatile bool [FM80_online](#)
- volatile bool [FM80_io](#)
- volatile bool [LOG](#)
- volatile bool [display_dim](#)
- volatile bool [display_update](#)
- volatile bool [display_on](#)
- volatile uint8_t [canbus_online](#)
- volatile uint8_t [modbus_online](#)
- volatile uint8_t [alt_display](#)
- volatile uint8_t [a_pin](#) [D_SW_COUNT]
- float [run_time](#)
- float [net_balance](#)
- uint16_t [mui](#) [10]
- uint16_t [fwrev](#) [3]
- [mx_logpage_t](#) [log](#)

5.1.1 Field Documentation

5.1.1.1 a_pin

```
volatile uint8_t a_pin[D_SW_COUNT]
```

5.1.1.2 a_switch

```
volatile bool a_switch[D_SW_COUNT]
```

5.1.1.3 a_trigger

```
volatile bool a_trigger[D_SW_COUNT]
```

5.1.1.4 a_type

```
volatile bool a_type[D_SW_COUNT]
```

5.1.1.5 alt_display

```
volatile uint8_t alt_display
```

5.1.1.6 canbus_online

```
volatile uint8_t canbus_online
```

5.1.1.7 day_check

```
volatile uint16_t day_check
```

5.1.1.8 dim_delay

```
volatile uint16_t dim_delay
```

5.1.1.9 display_dim

```
volatile bool display_dim
```

5.1.1.10 display_on

```
volatile bool display_on
```

5.1.1.11 display_update

```
volatile bool display_update
```

5.1.1.12 flush

```
volatile uint16_t flush
```

5.1.1.13 FM80_charged

```
volatile bool FM80_charged
```

5.1.1.14 FM80_io

```
volatile bool FM80_io
```

5.1.1.15 FM80_online

```
volatile bool FM80_online
```

5.1.1.16 fwrev

```
uint16_t fwrev[3]
```

5.1.1.17 LOG

```
volatile bool LOG
```

5.1.1.18 log

```
mx_logpage_t log
```

5.1.1.19 modbus_online

```
volatile uint8_t modbus_online
```

5.1.1.20 mui

```
uint16_t mui[10]
```

5.1.1.21 net_balance

```
float net_balance
```

5.1.1.22 node_id

```
volatile uint16_t node_id
```

5.1.1.23 once

```
volatile bool once
```

5.1.1.24 one_sec_flag

```
volatile bool one_sec_flag
```

5.1.1.25 pacing

```
volatile uint16_t pacing
```

5.1.1.26 pv_high

```
volatile bool pv_high
```

5.1.1.27 pv_prev

```
volatile uint16_t pv_prev
```

5.1.1.28 pv_update

```
volatile bool pv_update
```

5.1.1.29 run_time

```
float run_time
```

5.1.1.30 rx_count

```
volatile uint16_t rx_count
```

5.1.1.31 ten_sec_flag

```
volatile bool ten_sec_flag
```

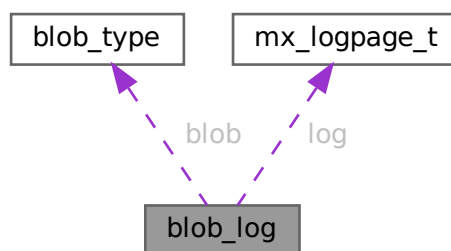
The documentation for this struct was generated from the following file:

- [mateQ84.X/mxcmd.h](#)

5.2 blob_log Union Reference

```
#include <canfd.h>
```

Collaboration diagram for blob_log:



Data Fields

- [blob_type](#) blob
- [mx_logpage_t](#) log

5.2.1 Field Documentation

5.2.1.1 blob

[blob_type](#) blob

5.2.1.2 log

[mx_logpage_t](#) log

The documentation for this union was generated from the following file:

- [canfd.h](#)

5.3 blob_type Struct Reference

```
#include <canfd.h>
```

Data Fields

- volatile uint8_t [blob](#) [[CANFD_BYTES](#)]
- volatile bool [tx_flag](#)
- volatile bool [rx_flag](#)

5.3.1 Field Documentation

5.3.1.1 blob

volatile uint8_t [blob](#)[[CANFD_BYTES](#)]

5.3.1.2 rx_flag

volatile bool [rx_flag](#)

5.3.1.3 tx_flag

volatile bool [tx_flag](#)

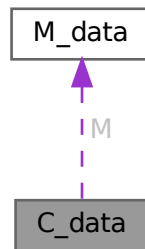
The documentation for this struct was generated from the following file:

- [canfd.h](#)

5.4 C_data Struct Reference

```
#include <modbus_master.h>
```

Collaboration diagram for C_data:



Data Fields

- uint8_t [mcmd](#)
- [comm_type](#) [cstate](#)
- [cmd_type](#) [modbus_command](#)
- uint16_t [req_length](#)
- int8_t [trace](#)
- bool [id_ok](#)
- bool [passwd_ok](#)
- bool [config_ok](#)
- bool [data_ok](#)
- bool [light_ok](#)
- bool [serial_ok](#)
- bool [version_ok](#)
- bool [tm_ok](#)
- uint32_t [data_count](#)
- uint32_t [data_prev](#)
- volatile [M_data](#) [M](#)

5.4.1 Field Documentation

5.4.1.1 config_ok

```
bool config_ok
```

5.4.1.2 cstate

```
comm\_type cstate
```

5.4.1.3 data_count

`uint32_t data_count`

5.4.1.4 data_ok

`bool data_ok`

5.4.1.5 data_prev

`uint32_t data_prev`

5.4.1.6 id_ok

`bool id_ok`

5.4.1.7 light_ok

`bool light_ok`

5.4.1.8 M

`volatile M_data M`

5.4.1.9 mcmd

`uint8_t mcmd`

5.4.1.10 modbus_command

`cmd_type modbus_command`

5.4.1.11 passwd_ok

`bool passwd_ok`

5.4.1.12 req_length

`uint16_t req_length`

5.4.1.13 serial_ok

`bool serial_ok`

5.4.1.14 tm_ok

`bool tm_ok`

5.4.1.15 trace

`int8_t trace`

5.4.1.16 version_ok

`bool version_ok`

The documentation for this struct was generated from the following file:

- [modbus_master.h](#)

5.5 CAN1_RX_FIFO Struct Reference

Data Fields

- [CAN1_RX_FIFO_CHANNELS](#) channel
- volatile uint8_t [fifoHead](#)

5.5.1 Field Documentation

5.5.1.1 channel

[CAN1_RX_FIFO_CHANNELS](#) channel

5.5.1.2 fifoHead

volatile uint8_t [fifoHead](#)

The documentation for this struct was generated from the following file:

- mateQ84.X/mcc_generated_files/[can1.c](#)

5.6 CAN_FIFOREG Struct Reference

Data Fields

- uint8_t [CONL](#)
- uint8_t [CONH](#)
- uint8_t [CONU](#)
- uint8_t [CONT](#)
- uint8_t [STAL](#)
- uint8_t [STAH](#)
- uint8_t [STAU](#)
- uint8_t [STAT](#)
- uint32_t [UA](#)

5.6.1 Field Documentation

5.6.1.1 CONH

uint8_t CONH

5.6.1.2 CONL

uint8_t CONL

5.6.1.3 CONT

uint8_t CONT

5.6.1.4 CONU

uint8_t CONU

5.6.1.5 STAH

uint8_t STAH

5.6.1.6 STAL

uint8_t STAL

5.6.1.7 STAT

uint8_t STAT

5.6.1.8 STAU

```
uint8_t STAU
```

5.6.1.9 UA

```
uint32_t UA
```

The documentation for this struct was generated from the following file:

- [mateQ84.X/mcc_generated_files/can1.c](#)

5.7 can_frames_t Struct Reference

```
#include <canfd.h>
```

Data Fields

- `uint32_t` [rec_count](#)
- `uint32_t` [tx_count](#)

5.7.1 Field Documentation

5.7.1.1 rec_count

```
uint32_t rec_count
```

5.7.1.2 tx_count

```
uint32_t tx_count
```

The documentation for this struct was generated from the following file:

- [canfd.h](#)

5.8 CAN_MSG_FIELD Union Reference

```
#include <can_types.h>
```

Data Fields

- uint8_t [msgfields](#)
- struct {
 - uint8_t [idType](#):1
 - uint8_t [frameType](#):1
 - uint8_t [dlc](#):4
 - uint8_t [formatType](#):1
 - uint8_t [brs](#):1
- };

5.8.1 Detailed Description

CAN1 Generated Driver File

@Company Microchip Technology Inc.

@File Name [can1.c](#)

@Summary This is the generated driver implementation file for the CAN1 driver using PIC10 / PIC12 / PIC16 / PIC18 MCUs

@Description This header file provides implementations for driver APIs for CAN1. Generation Information : Product Revision : PIC10 / PIC12 / PIC16 / PIC18 MCUs - 1.81.8 Device : PIC18F47Q84 Driver Version : 1.0.0 The generated drivers are tested against the following: Compiler : XC8 2.36 and above or later MPLAB : MPLAB X 6.00 Section: Included Files CAN Message Object data structure

@Summary Defines the CAN Message Object data structure.

@Description This Data structure is to implement a CAN FIFO message object.

5.8.2 Field Documentation

5.8.2.1 [struct]

```
struct { ... }
```

5.8.2.2 brs

```
uint8_t brs
```

5.8.2.3 dlc

```
uint8_t dlc
```

5.8.2.4 formatType

```
uint8_t formatType
```

5.8.2.5 frameType

```
uint8_t frameType
```

5.8.2.6 idType

```
uint8_t idType
```

5.8.2.7 msgfields

```
uint8_t msgfields
```

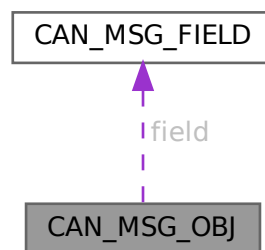
The documentation for this union was generated from the following file:

- [mateQ84.X/mcc_generated_files/can_types.h](#)

5.9 CAN_MSG_OBJ Struct Reference

```
#include <can_types.h>
```

Collaboration diagram for CAN_MSG_OBJ:



Data Fields

- `uint32_t msgId`
- `CAN_MSG_FIELD` field
- `uint8_t * data`

5.9.1 Field Documentation

5.9.1.1 data

```
uint8_t* data
```

5.9.1.2 field

`CAN_MSG_FIELD` field

5.9.1.3 msgId

`uint32_t` msgId

The documentation for this struct was generated from the following file:

- [mateQ84.X/mcc_generated_files/can_types.h](#)

5.10 can_rec_count_t Struct Reference

```
#include <canfd.h>
```

Data Fields

- `uint32_t` [rec_count](#)
- `bool` [rec_flag](#)

5.10.1 Field Documentation

5.10.1.1 rec_count

`uint32_t` rec_count

5.10.1.2 rec_flag

`bool` rec_flag

The documentation for this struct was generated from the following file:

- [canfd.h](#)

5.11 EB_data Struct Reference

```
#include <batmon.h>
```

Data Fields

- uint8_t [checkmark](#)
- uint8_t [version](#)
- uint8_t [alt_display](#)
- bool [loaded](#)
- float [FMw](#)
- float [FMpv](#)
- float [FMa](#)
- float [FMbv](#)
- float [ENw](#)
- float [ENva](#)
- float [ENvar](#)
- float [ENac](#)
- float [volt_whole](#)
- float [bat_amp_whole](#)
- float [bat_energy](#)
- uint16_t [cc_mode](#)
- uint16_t [bat_cycles](#)
- uint16_t [bat_mode](#)
- uint16_t [time](#)
- uint16_t [date](#)
- uint32_t [bat_time](#)
- uint32_t [fm80_time](#)
- uint32_t [q84_sequence](#)
- uint16_t [crc](#)

5.11.1 Field Documentation

5.11.1.1 alt_display

```
uint8_t alt_display
```

5.11.1.2 bat_amp_whole

```
float bat_amp_whole
```

5.11.1.3 bat_cycles

```
uint16_t bat_cycles
```

5.11.1.4 bat_energy

```
float bat_energy
```

5.11.1.5 bat_mode

```
uint16_t bat_mode
```

5.11.1.6 bat_time

```
uint32_t bat_time
```

5.11.1.7 cc_mode

```
uint16_t cc_mode
```

5.11.1.8 checkmark

```
uint8_t checkmark
```

5.11.1.9 crc

```
uint16_t crc
```

5.11.1.10 date

```
uint16_t date
```

5.11.1.11 ENac

```
float ENac
```

5.11.1.12 ENva

```
float ENva
```

5.11.1.13 ENvar

```
float ENvar
```

5.11.1.14 ENw

```
float ENw
```

5.11.1.15 fm80_time

```
uint32_t fm80_time
```


5.11.1.16 FMa

`float FMa`

5.11.1.17 FMbv

`float FMbv`

5.11.1.18 FMpv

`float FMpv`

5.11.1.19 FMw

`float FMw`

5.11.1.20 loaded

`bool loaded`

5.11.1.21 q84_sequence

`uint32_t q84_sequence`

5.11.1.22 time

`uint16_t time`

5.11.1.23 version

`uint8_t version`

5.11.1.24 volt_whole

`float volt_whole`

The documentation for this struct was generated from the following file:

- [batmon.h](#)

5.12 EM_data1 Struct Reference

```
#include <modbus_master.h>
```

Data Fields

- volatile int32_t [vl1n](#)
- volatile int32_t [vl2n](#)
- volatile int32_t [vl3n](#)
- volatile int32_t [vl1l2](#)
- volatile int32_t [vl2l3](#)
- volatile int32_t [vl3l1](#)
- volatile int32_t [al1](#)
- volatile int32_t [al2](#)
- volatile int32_t [al3](#)
- volatile int32_t [wl1](#)
- volatile int32_t [wl2](#)
- volatile int32_t [wl3](#)
- volatile int32_t [val1](#)
- volatile int32_t [val2](#)
- volatile int32_t [val3](#)
- volatile int32_t [varl1](#)
- volatile int32_t [varl2](#)
- volatile int32_t [varl3](#)
- volatile int32_t [vlnsys](#)
- volatile int32_t [vllsys](#)
- volatile int32_t [wsys](#)
- volatile int32_t [vasys](#)
- volatile int32_t [varsys](#)
- volatile int16_t [pfl1](#)
- volatile int16_t [pfl2](#)
- volatile int16_t [pfl3](#)
- volatile int16_t [pfsys](#)
- volatile int16_t [phaseseq](#)
- volatile int16_t [hz](#)

5.12.1 Field Documentation

5.12.1.1 al1

```
volatile int32_t al1
```

5.12.1.2 al2

```
volatile int32_t al2
```

5.12.1.3 al3

```
volatile int32_t al3
```

5.12.1.4 hz

```
volatile int16_t hz
```

5.12.1.5 pfl1

```
volatile int16_t pfl1
```

5.12.1.6 pfl2

```
volatile int16_t pfl2
```

5.12.1.7 pfl3

```
volatile int16_t pfl3
```

5.12.1.8 pfsys

```
volatile int16_t pfsys
```

5.12.1.9 phaseseq

```
volatile int16_t phaseseq
```

5.12.1.10 val1

```
volatile int32_t val1
```

5.12.1.11 val2

```
volatile int32_t val2
```

5.12.1.12 val3

```
volatile int32_t val3
```

5.12.1.13 var1

```
volatile int32_t var1
```

5.12.1.14 varl2

```
volatile int32_t varl2
```

5.12.1.15 varl3

```
volatile int32_t varl3
```

5.12.1.16 varsys

```
volatile int32_t varsys
```

5.12.1.17 vasys

```
volatile int32_t vasys
```

5.12.1.18 vl1l2

```
volatile int32_t vl1l2
```

5.12.1.19 vl1n

```
volatile int32_t vl1n
```

5.12.1.20 vl2l3

```
volatile int32_t vl2l3
```

5.12.1.21 vl2n

```
volatile int32_t vl2n
```

5.12.1.22 vl3l1

```
volatile int32_t vl3l1
```

5.12.1.23 vl3n

```
volatile int32_t vl3n
```

5.12.1.24 vllsys

```
volatile int32_t vllsys
```

5.12.1.25 vlmsys

```
volatile int32_t vlmsys
```

5.12.1.26 wl1

```
volatile int32_t wl1
```

5.12.1.27 wl2

```
volatile int32_t wl2
```

5.12.1.28 wl3

```
volatile int32_t wl3
```

5.12.1.29 wsys

```
volatile int32_t wsys
```

The documentation for this struct was generated from the following file:

- [modbus_master.h](#)

5.13 EM_data2 Struct Reference

```
#include <modbus_master.h>
```

Data Fields

- volatile int64_t [kwhpt](#)
- volatile int64_t [kvarhpt](#)
- volatile int64_t [kwhpp](#)
- volatile int64_t [kvarhpp](#)
- volatile int64_t [kwhpl1](#)
- volatile int64_t [kwhpl2](#)
- volatile int64_t [kwhpl3](#)
- volatile int64_t [kwhnt](#)
- volatile int64_t [kvarhnt](#)
- volatile int64_t [kwhnp](#)
- volatile int64_t [kvarhnp](#)
- volatile int64_t [kvaht](#)
- volatile int64_t [kvahtp](#)
- volatile int32_t [rhm](#)
- volatile int32_t [rhmk](#)
- volatile int32_t [rhmp](#)
- volatile int32_t [rhmkp](#)
- volatile int32_t [hz](#)
- volatile int32_t [rhlc](#)

5.13.1 Field Documentation

5.13.1.1 hz

```
volatile int32_t hz
```

5.13.1.2 kvahp

```
volatile int64_t kvahp
```

5.13.1.3 kvaht

```
volatile int64_t kvaht
```

5.13.1.4 kvarhnp

```
volatile int64_t kvarhnp
```

5.13.1.5 kvarhnt

```
volatile int64_t kvarhnt
```

5.13.1.6 kvarhpp

```
volatile int64_t kvarhpp
```

5.13.1.7 kvarhpt

```
volatile int64_t kvarhpt
```

5.13.1.8 kwhnp

```
volatile int64_t kwhnp
```

5.13.1.9 kwhnt

```
volatile int64_t kwhnt
```

5.13.1.10 kwhp11

```
volatile int64_t kwhp11
```

5.13.1.11 kwhpl2

```
volatile int64_t kwhpl2
```

5.13.1.12 kwhpl3

```
volatile int64_t kwhpl3
```

5.13.1.13 kwhpp

```
volatile int64_t kwhpp
```

5.13.1.14 kwhpt

```
volatile int64_t kwhpt
```

5.13.1.15 rhlc

```
volatile int32_t rhlc
```

5.13.1.16 rhm

```
volatile int32_t rhm
```

5.13.1.17 rhmk

```
volatile int32_t rhmk
```

5.13.1.18 rhmkp

```
volatile int32_t rhmkp
```

5.13.1.19 rhmp

```
volatile int32_t rhmp
```

The documentation for this struct was generated from the following file:

- [modbus_master.h](#)

5.14 EM_serial Struct Reference

```
#include <modbus_master.h>
```

Data Fields

- volatile char [serial](#) [14]
- volatile int16_t [year](#)

5.14.1 Field Documentation

5.14.1.1 serial

```
volatile char serial[14]
```

5.14.1.2 year

```
volatile int16_t year
```

The documentation for this struct was generated from the following file:

- [modbus_master.h](#)

5.15 EM_version Struct Reference

```
#include <modbus_master.h>
```

Data Fields

- volatile int16_t [firmware](#)

5.15.1 Field Documentation

5.15.1.1 firmware

```
volatile int16_t firmware
```

The documentation for this struct was generated from the following file:

- [modbus_master.h](#)

5.16 M_data Struct Reference

```
#include <modbus_master.h>
```

Data Fields

- uint8_t [blink_lock](#): 1
- uint8_t [config](#): 1
- uint8_t [stable](#): 1
- uint8_t [boot_code](#): 1
- uint8_t [power_on](#): 1
- uint8_t [send_count](#)
- uint8_t [recv_count](#)
- uint8_t [pwm_volts](#)
- uint16_t [error](#)
- uint16_t [crc_data](#)
- uint16_t [crc_calc](#)
- uint32_t [crc_error](#)
- uint32_t [to_error](#)
- uint32_t [sends](#)
- volatile bool [rx](#)

5.16.1 Field Documentation

5.16.1.1 [blink_lock](#)

```
uint8_t blink_lock
```

5.16.1.2 [boot_code](#)

```
uint8_t boot_code
```

5.16.1.3 [config](#)

```
uint8_t config
```

5.16.1.4 [crc_calc](#)

```
uint16_t crc_calc
```

5.16.1.5 [crc_data](#)

```
uint16_t crc_data
```

5.16.1.6 crc_error

```
uint32_t crc_error
```

5.16.1.7 error

```
uint16_t error
```

5.16.1.8 power_on

```
uint8_t power_on
```

5.16.1.9 pwm_volts

```
uint8_t pwm_volts
```

5.16.1.10 recv_count

```
uint8_t recv_count
```

5.16.1.11 rx

```
volatile bool rx
```

5.16.1.12 send_count

```
uint8_t send_count
```

5.16.1.13 sends

```
uint32_t sends
```

5.16.1.14 stable

```
uint8_t stable
```

5.16.1.15 to_error

```
uint32_t to_error
```

The documentation for this struct was generated from the following file:

- [modbus_master.h](#)

5.17 M_time_data Struct Reference

```
#include <modbus_master.h>
```

Data Fields

- uint32_t [clock_500hz](#)
- uint32_t [clock_500ahz](#)
- uint32_t [clock_2hz](#)

5.17.1 Field Documentation

5.17.1.1 clock_2hz

```
uint32_t clock_2hz
```

5.17.1.2 clock_500ahz

```
uint32_t clock_500ahz
```

5.17.1.3 clock_500hz

```
uint32_t clock_500hz
```

The documentation for this struct was generated from the following file:

- [modbus_master.h](#)

5.18 mbedtls_context Struct Reference

```
#include <mbedtls_sockets.h>
```

Data Fields

- mbedtls_net_context [net_ctx](#)
- mbedtls_ssl_context [ssl_ctx](#)
- mbedtls_ssl_config [ssl_conf](#)
- mbedtls_x509_crt [ca_crt](#)
- mbedtls_entropy_context [entropy](#)
- mbedtls_ctr_drbg_context [ctr_drbg](#)

5.18.1 Field Documentation

5.18.1.1 ca_cert

`MBEDTLS_X509_CERT CA_CERT`

5.18.1.2 ctr_drbg

`MBEDTLS_CTR_DRBG_CONTEXT CTR_DRBG`

5.18.1.3 entropy

`MBEDTLS_ENTROPY_CONTEXT ENTROPY`

5.18.1.4 net_ctx

`MBEDTLS_NET_CONTEXT NET_CTX`

5.18.1.5 ssl_conf

`MBEDTLS_SSL_CONFIG SSL_CONF`

5.18.1.6 ssl_ctx

`MBEDTLS_SSL_CONTEXT SSL_CTX`

The documentation for this struct was generated from the following file:

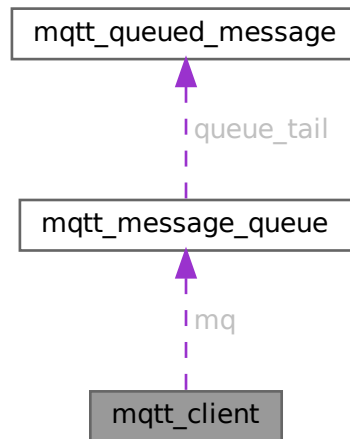
- [linux/mbedtlscan/templates/mbedtls_sockets.h](#)

5.19 mqtt_client Struct Reference

An MQTT client.

```
#include <mqtt.h>
```

Collaboration diagram for mqtt_client:



Data Fields

- `mqtt_pal_socket_handle` [socketfd](#)
The socket connecting to the MQTT broker.
- `uint16_t` [pid_lfsr](#)
The LFSR state used to generate packet ID's.
- `uint16_t` [keep_alive](#)
The keep-alive time in seconds.
- `int` [number_of_keep_alives](#)
A counter counting pings that have been sent to keep the connection alive.
- `size_t` [send_offset](#)
The current sent offset.
- `mqtt_pal_time_t` [time_of_last_send](#)
The timestamp of the last message sent to the buffer.
- `enum` [MQTTErrors](#) [error](#)
The error state of the client.
- `int` [response_timeout](#)
The timeout period in seconds.
- `int` [number_of_timeouts](#)
A counter counting the number of timeouts that have occurred.
- `float` [typical_response_time](#)
Approximately much time it has typically taken to receive responses from the broker.
- `void(*` [publish_response_callback](#) `)(void **state, struct mqtt\_response\_publish *publish)`

The callback that is called whenever a publish is received from the broker.

- void * [publish_response_callback_state](#)

A pointer to any publish_response_callback state information you need.

- enum [MQTTErrors](#)(* [inspector_callback](#))(struct [mqtt_client](#) *)

A user-specified callback, triggered on each [mqtt_sync](#), allowing the user to perform state inspections (and custom socket error detection) on the client.

- void(* [reconnect_callback](#))(struct [mqtt_client](#) *, void **)

A callback that is called whenever the client is in an error state.

- void * [reconnect_state](#)

A pointer to some state. A pointer to this member is passed to [reconnect_callback](#).

- struct {

uint8_t * [mem_start](#)

The start of the receive buffer's memory.

size_t [mem_size](#)

The size of the receive buffer's memory.

uint8_t * [curr](#)

A pointer to the next writable location in the receive buffer.

size_t [curr_sz](#)

The number of bytes that are still writable at curr.

} [recv_buffer](#)

The buffer where ingress data is temporarily stored.

- mqtt_pal_mutex_t [mutex](#)

A variable passed to support thread-safety.

- struct [mqtt_message_queue](#) [mq](#)

The sending message queue.

5.19.1 Detailed Description

An MQTT client.

Note

All members can be manipulated via the related functions.

5.19.2 Field Documentation

5.19.2.1 curr

```
uint8_t* curr
```

A pointer to the next writable location in the receive buffer.

5.19.2.2 curr_sz

```
size_t curr_sz
```

The number of bytes that are still writable at curr.

5.19.2.3 error

```
enum MQTTErrors error
```

The error state of the client.

error should be MQTT_OK for the entirety of the connection.

Note

The error state will be MQTT_ERROR_CONNECT_NOT_CALLED until you call mqtt_connect.

5.19.2.4 inspector_callback

```
enum MQTTErrors(* inspector_callback) (struct mqtt_client *)
```

A user-specified callback, triggered on each [mqtt_sync](#), allowing the user to perform state inspections (and custom socket error detection) on the client.

This callback is triggered on each call to [mqtt_sync](#). If it returns MQTT_OK then [mqtt_sync](#) will continue normally (performing reads and writes). If it returns an error then [mqtt_sync](#) will not call reads and writes.

This callback can be used to perform custom error detection, namely platform specific socket error detection, and force the client into an error state.

This member is always initialized to NULL but it can be manually set at any time.

5.19.2.5 keep_alive

```
uint16_t keep_alive
```

The keep-alive time in seconds.

5.19.2.6 mem_size

```
size_t mem_size
```

The size of the receive buffer's memory.

5.19.2.7 mem_start

```
uint8_t* mem_start
```

The start of the receive buffer's memory.

5.19.2.8 mq

```
struct mqtt_message_queue mq
```

The sending message queue.

5.19.2.9 mutex

```
mqtt_pal_mutex_t mutex
```

A variable passed to support thread-safety.

A pointer to this variable is passed to `MQTT_PAL_MUTEX_LOCK`, and `MQTT_PAL_MUTEX_UNLOCK`.

5.19.2.10 number_of_keep_alives

```
int number_of_keep_alives
```

A counter counting pings that have been sent to keep the connection alive.

See also

[keep_alive](#)

5.19.2.11 number_of_timeouts

```
int number_of_timeouts
```

A counter counting the number of timeouts that have occurred.

5.19.2.12 pid_lfsr

```
uint16_t pid_lfsr
```

The LFSR state used to generate packet ID's.

5.19.2.13 publish_response_callback

```
void(* publish_response_callback) (void **state, struct mqtt_response_publish *publish)
```

The callback that is called whenever a publish is received from the broker.

Any topics that you have subscribed to will be returned from the broker as `mqtt_response_publish` messages. All the publishes received from the broker will be passed to this function.

Note

A pointer to `publish_response_callback_state` is always passed to the callback. Use `publish_response_callback_state` to keep track of any state information you need.

5.19.2.14 publish_response_callback_state

```
void* publish_response_callback_state
```

A pointer to any publish_response_callback state information you need.

Note

A pointer to this pointer will always be publish_response_callback upon receiving a publish message from the broker.

5.19.2.15 reconnect_callback

```
void(* reconnect_callback) (struct mqtt_client *, void **)
```

A callback that is called whenever the client is in an error state.

This callback is responsible for: application level error handling, closing previous sockets, and reestablishing the connection to the broker and session configurations (i.e. subscriptions).

5.19.2.16 reconnect_state

```
void* reconnect_state
```

A pointer to some state. A pointer to this member is passed to [reconnect_callback](#).

5.19.2.17 [struct]

```
struct { ... } recv_buffer
```

The buffer where ingress data is temporarily stored.

5.19.2.18 response_timeout

```
int response_timeout
```

The timeout period in seconds.

If the broker doesn't return an ACK within response_timeout seconds a timeout will occur and the message will be retransmitted.

Note

The default value is 30 [seconds] but you can change it at any time.

5.19.2.19 `send_offset`

```
size_t send_offset
```

The current sent offset.

This is used to allow partial send commands.

5.19.2.20 `socketfd`

```
mqtt_pal_socket_handle socketfd
```

The socket connecting to the MQTT broker.

5.19.2.21 `time_of_last_send`

```
mqtt_pal_time_t time_of_last_send
```

The timestamp of the last message sent to the buffer.

This is used to detect the need for keep-alive pings.

See also

[keep_alive](#)

5.19.2.22 `typical_response_time`

```
float typical_response_time
```

Approximately much time it has typically taken to receive responses from the broker.

Note

This is tracked using a exponential-averaging.

The documentation for this struct was generated from the following file:

- [linux/matesocketcan/mqtt.h](#)

5.20 `mqtt_fixed_header` Struct Reference

The fixed header of an MQTT control packet.

```
#include <mqtt.h>
```

Data Fields

- enum [MQTTControlPacketType](#) control_type
- uint32_t control_flags: 4
- uint32_t remaining_length

5.20.1 Detailed Description

The fixed header of an MQTT control packet.

See also

[MQTT v3.1.1: Fixed Header](#)

5.20.2 Field Documentation

5.20.2.1 control_flags

uint32_t control_flags

The packets control flags.

5.20.2.2 control_type

enum [MQTTControlPacketType](#) control_type

The type of packet.

5.20.2.3 remaining_length

uint32_t remaining_length

The remaining size of the packet in bytes (i.e. the size of variable header and payload).

The documentation for this struct was generated from the following file:

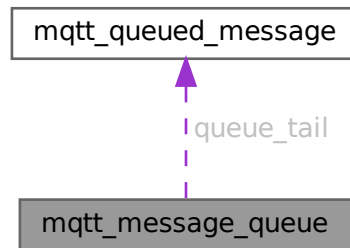
- [linux/matesocketcan/mqtt.h](#)

5.21 mqtt_message_queue Struct Reference

A message queue.

```
#include <mqtt.h>
```

Collaboration diagram for mqtt_message_queue:



Data Fields

- void * [mem_start](#)
The start of the message queue's memory block.
- void * [mem_end](#)
The end of the message queue's memory block.
- uint8_t * [curr](#)
A pointer to the position in the buffer you can pack bytes at.
- size_t [curr_sz](#)
The number of bytes that can be written to [curr](#).
- struct [mqtt_queued_message](#) * [queue_tail](#)
The tail of the array of [mqtt_queued_message](#)'s.

Related Symbols

(Note that these are not member symbols.)

- void [mqtt_mq_init](#) (struct [mqtt_message_queue](#) *mq, void *buf, size_t bufsz)
Initialize a message queue.
- void [mqtt_mq_clean](#) (struct [mqtt_message_queue](#) *mq)
Clear as many messages from the front of the queue as possible.
- struct [mqtt_queued_message](#) * [mqtt_mq_register](#) (struct [mqtt_message_queue](#) *mq, size_t nbytes)
Register a message that was just added to the buffer.
- struct [mqtt_queued_message](#) * [mqtt_mq_find](#) (const struct [mqtt_message_queue](#) *mq, enum [MQTTControlPacketType](#) control_type, const uint16_t *packet_id)
Find a message in the message queue.

5.21.1 Detailed Description

A message queue.

Note

This struct is used internally to manage sending messages.

The only members the user should use are `curr` and `curr_sz`.

5.21.2 Field Documentation

5.21.2.1 `curr`

```
uint8_t* curr
```

A pointer to the position in the buffer you can pack bytes at.

Note

Immediately after packing bytes at `curr` you *must* call `mqtt_mq_register`.

5.21.2.2 `curr_sz`

```
size_t curr_sz
```

The number of bytes that can be written to `curr`.

Note

`curr_sz` will decrease by more than the number of bytes you write to `curr`. This is because the [mqtt_queued_message](#) structs share the same memory (and thus, a [mqtt_queued_message](#) must be allocated in the message queue's memory whenever a new message is registered).

5.21.2.3 `mem_end`

```
void* mem_end
```

The end of the message queue's memory block.

5.21.2.4 `mem_start`

```
void* mem_start
```

The start of the message queue's memory block.

Warning

This member should *not* be manually changed.

5.21.2.5 queue_tail

```
struct mqtt_queued_message* queue_tail
```

The tail of the array of mqtt_queued_messages's.

Note

This member should not be used manually.

The documentation for this struct was generated from the following file:

- [linux/matesocketcan/mqtt.h](#)

5.22 mqtt_queued_message Struct Reference

A message in a [mqtt_message_queue](#).

```
#include <mqtt.h>
```

Data Fields

- `uint8_t * start`
A pointer to the start of the message.
- `size_t size`
The number of bytes in the message.
- `enum MQTTQueuedMessageState state`
The state of the message.
- `mqtt_pal_time_t time_sent`
The time at which the message was sent..
- `enum MQTTControlPacketType control_type`
The control type of the message.
- `uint16_t packet_id`
The packet id of the message.

5.22.1 Detailed Description

A message in a [mqtt_message_queue](#).

5.22.2 Field Documentation

5.22.2.1 control_type

```
enum MQTTControlPacketType control_type
```

The control type of the message.

5.22.2.2 packet_id

```
uint16_t packet_id
```

The packet id of the message.

Note

This field is only used if the associate `control_type` has a `packet_id` field.

5.22.2.3 size

```
size_t size
```

The number of bytes in the message.

5.22.2.4 start

```
uint8_t* start
```

A pointer to the start of the message.

5.22.2.5 state

```
enum MQTTQueuedMessageState state
```

The state of the message.

5.22.2.6 time_sent

```
mqtt_pal_time_t time_sent
```

The time at which the message was sent..

Note

A timeout will only occur if the message is in the `MQTT_QUEUED_AWAITING_ACK` state.

The documentation for this struct was generated from the following file:

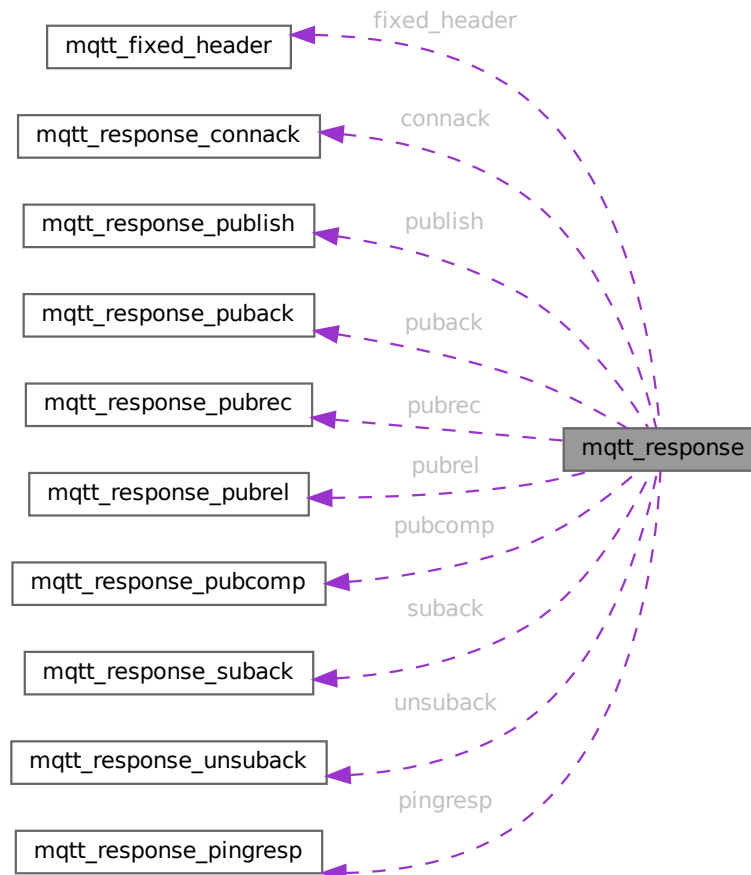
- [linux/matesocketcan/mqtt.h](#)

5.23 mqtt_response Struct Reference

A struct used to deserialize/interpret an incoming packet from the broker.

```
#include <mqtt.h>
```

Collaboration diagram for mqtt_response:



Data Fields

- struct `mqtt_fixed_header` `fixed_header`
The `mqtt_fixed_header` of the deserialized packet.
 - union {
 - struct `mqtt_response_connack` `connack`
 - struct `mqtt_response_publish` `publish`
 - struct `mqtt_response_puback` `puback`
 - struct `mqtt_response_pubrec` `pubrec`
 - struct `mqtt_response_pubrel` `pubrel`
 - struct `mqtt_response_pubcomp` `pubcomp`
 - struct `mqtt_response_suback` `suback`
 - struct `mqtt_response_unsuback` `unsuback`
 - struct `mqtt_response_pingresp` `pingresp`
- } `decoded`

A union of the possible responses from the broker.

Related Symbols

(Note that these are not member symbols.)

- `ssize_t mqtt_unpack_response` (struct `mqtt_response` *response, const uint8_t *buf, size_t bufsz)
Deserialize a packet from the broker.

5.23.1 Detailed Description

A struct used to deserialize/interpret an incoming packet from the broker.

5.23.2 Field Documentation

5.23.2.1 connack

```
struct mqtt_response_connack connack
```

5.23.2.2 [union]

```
union { ... } decoded
```

A union of the possible responses from the broker.

Note

The `fixed_header` contains the control type. This control type corresponds to the member of this union that should be accessed. For example if `fixed_header#control_type == MQTT_CONTROL_PUBLISH` then `decoded#publish` should be accessed.

5.23.2.3 fixed_header

```
struct mqtt_fixed_header fixed_header
```

The `mqtt_fixed_header` of the deserialized packet.

5.23.2.4 pingresp

```
struct mqtt_response_pingresp pingresp
```

5.23.2.5 puback

```
struct mqtt_response_puback puback
```

5.23.2.6 pubcomp

```
struct mqtt_response_pubcomp pubcomp
```

5.23.2.7 publish

```
struct mqtt_response_publish publish
```

5.23.2.8 pubrec

```
struct mqtt_response_pubrec pubrec
```

5.23.2.9 pubrel

```
struct mqtt_response_pubrel pubrel
```

5.23.2.10 suback

```
struct mqtt_response_suback suback
```

5.23.2.11 unsuback

```
struct mqtt_response_unsuback unsuback
```

The documentation for this struct was generated from the following file:

- [linux/matesocketcan/mqtt.h](#)

5.24 mqtt_response_connack Struct Reference

A connection response datastructure.

```
#include <mqtt.h>
```

Data Fields

- `uint8_t session_present_flag`
Allows client and broker to check if they have a consistent view about whether there is already a stored session state.
- `enum MQTTConnackReturnCode return_code`
The return code of the connection request.

Related Symbols

(Note that these are not member symbols.)

- `ssize_t mqtt_unpack_connack_response` (struct `mqtt_response` *`mqtt_response`, const `uint8_t` *`buf`)
Deserialize a CONNACK response from buf.

5.24.1 Detailed Description

A connection response datastructure.

See also

MQTT v3.1.1: CONNACK – Acknowledgement connection response.

5.24.2 Field Documentation

5.24.2.1 return_code

```
enum MQTTConnackReturnCode return_code
```

The return code of the connection request.

See also

[MQTTConnackReturnCode](#)

5.24.2.2 session_present_flag

```
uint8_t session_present_flag
```

Allows client and broker to check if they have a consistent view about whether there is already a stored session state.

The documentation for this struct was generated from the following file:

- [linux/matesocketcan/mqtt.h](#)

5.25 mqtt_response_pingresp Struct Reference

The response to a ping request.

```
#include <mqtt.h>
```

Data Fields

- int [dummy](#)

5.25.1 Detailed Description

The response to a ping request.

Note

This response contains no members.

See also

[MQTT v3.1.1: PINGRESP – Ping Response.](#)

5.25.2 Field Documentation

5.25.2.1 dummy

int dummy

The documentation for this struct was generated from the following file:

- [linux/matesocketcan/mqtt.h](#)

5.26 mqtt_response_puback Struct Reference

A publish acknowledgement for messages that were published with QoS level 1.

```
#include <mqtt.h>
```

Data Fields

- uint16_t [packet_id](#)
The published messages packet ID.

Related Symbols

(Note that these are not member symbols.)

- ssize_t [mqtt_unpack_pubxxx_response](#) (struct [mqtt_response](#) *[mqtt_response](#), const uint8_t *buf)
Deserialize a PUBACK/PUBREC/PUBREL/PUBCOMP packet from buf.

5.26.1 Detailed Description

A publish acknowledgement for messages that were published with QoS level 1.

See also

MQTT v3.1.1: PUBACK – Publish Acknowledgement.

5.26.2 Field Documentation

5.26.2.1 packet_id

```
uint16_t packet_id
```

The published messages packet ID.

The documentation for this struct was generated from the following file:

- [linux/matesocketcan/mqtt.h](#)

5.27 mqtt_response_pubcomp Struct Reference

The response to a PUBREL packet.

```
#include <mqtt.h>
```

Data Fields

- uint16_t [packet_id](#)
he published messages packet ID.

5.27.1 Detailed Description

The response to a PUBREL packet.

See also

MQTT v3.1.1: PUBCOMP – Publish Complete.

5.27.2 Field Documentation

5.27.2.1 packet_id

```
uint16_t packet_id
```

he published messages packet ID.

T

The documentation for this struct was generated from the following file:

- [linux/matesocketcan/mqtt.h](#)

5.28 mqtt_response_publish Struct Reference

A publish packet received from the broker.

```
#include <mqtt.h>
```

Data Fields

- uint8_t [dup_flag](#)
The DUP flag. DUP flag is 0 if its the first attempt to send this publish packet. A DUP flag of 1 means that this might be a re-delivery of the packet.
- uint8_t [qos_level](#)
The quality of service level.
- uint8_t [retain_flag](#)
The retain flag of this publish message.
- uint16_t [topic_name_size](#)
Size of the topic name (number of characters).
- const void * [topic_name](#)
The topic name.
- uint16_t [packet_id](#)
The publish message's packet ID.
- const void * [application_message](#)
The publish message's application message.
- size_t [application_message_size](#)
The size of the application message in bytes.

Related Symbols

(Note that these are not member symbols.)

- ssize_t [mqtt_unpack_publish_response](#) (struct [mqtt_response](#) *[mqtt_response](#), const uint8_t *buf)
Deserialize a publish response from buf.

5.28.1 Detailed Description

A publish packet received from the broker.

A publish packet is received from the broker when a client publishes to a topic that the {local client} is subscribed to.

See also

MQTT v3.1.1: PUBLISH – Publish Message.

5.28.2 Field Documentation

5.28.2.1 application_message

```
const void* application_message
```

The publish message's application message.

5.28.2.2 application_message_size

```
size_t application_message_size
```

The size of the application message in bytes.

5.28.2.3 dup_flag

```
uint8_t dup_flag
```

The DUP flag. DUP flag is 0 if its the first attempt to send this publish packet. A DUP flag of 1 means that this might be a re-delivery of the packet.

5.28.2.4 packet_id

```
uint16_t packet_id
```

The publish message's packet ID.

5.28.2.5 qos_level

```
uint8_t qos_level
```

The quality of service level.

See also

[MQTT v3.1.1: QoS Definitions](#)

5.28.2.6 retain_flag

```
uint8_t retain_flag
```

The retain flag of this publish message.

5.28.2.7 topic_name

```
const void* topic_name
```

The topic name.

Note

topic_name is not null terminated. Therefore topic_name_size must be used to get the string length.

5.28.2.8 topic_name_size

```
uint16_t topic_name_size
```

Size of the topic name (number of characters).

The documentation for this struct was generated from the following file:

- [linux/matesocketcan/mqtt.h](#)

5.29 mqtt_response_pubrec Struct Reference

The response packet to a PUBLISH packet with QoS level 2.

```
#include <mqtt.h>
```

Data Fields

- `uint16_t packet_id`
The published messages packet ID.

5.29.1 Detailed Description

The response packet to a PUBLISH packet with QoS level 2.

See also

[MQTT v3.1.1: PUBREC - Publish Received.](#)

5.29.2 Field Documentation

5.29.2.1 packet_id

```
uint16_t packet_id
```

The published messages packet ID.

The documentation for this struct was generated from the following file:

- [linux/matesocketcan/mqtt.h](#)

5.30 mqtt_response_pubrel Struct Reference

The response to a PUBREC packet.

```
#include <mqtt.h>
```


Data Fields

- uint16_t [packet_id](#)
The published messages packet ID.

5.30.1 Detailed Description

The response to a PUBREC packet.

See also

MQTT v3.1.1: PUBREL – Publish Release.

5.30.2 Field Documentation

5.30.2.1 packet_id

uint16_t packet_id

The published messages packet ID.

The documentation for this struct was generated from the following file:

- [linux/matesocketcan/mqtt.h](#)

5.31 mqtt_response_suback Struct Reference

The response to a subscription request.

```
#include <mqtt.h>
```

Data Fields

- uint16_t [packet_id](#)
The published messages packet ID.
- const uint8_t * [return_codes](#)
- size_t [num_return_codes](#)

Related Symbols

(Note that these are not member symbols.)

- ssize_t [mqtt_unpack_suback_response](#) (struct [mqtt_response](#) *[mqtt_response](#), const uint8_t *[buf](#))
Deserialize a SUBACK packet from buf.

5.31.1 Detailed Description

The response to a subscription request.

See also

[MQTT v3.1.1: SUBACK – Subscription Acknowledgement.](#)

5.31.2 Friends And Related Symbol Documentation

5.31.2.1 mqtt_unpack_suback_response()

```
ssize_t mqtt_unpack_suback_response (
    struct mqtt_response * mqtt_response,
    const uint8_t * buf) [related]
```

Deserialize a SUBACK packet from `buf`.

Precondition

[mqtt_unpack_fixed_header](#) must have returned a positive value and the [mqtt_response](#) must have a control type of `MQTT_CONTROL_SUBACK`.

Parameters

out	mqtt_response	the response that is initialized from the contents of <code>buf</code> .
in	<code>buf</code>	the buffer with the incoming data.

Returns

The number of bytes that were consumed, or 0 if the buffer does not contain enough bytes to parse the packet, or a negative value if there was a protocol violation.

5.31.3 Field Documentation

5.31.3.1 num_return_codes

```
size_t num_return_codes
```

The number of return codes.

5.31.3.2 packet_id

```
uint16_t packet_id
```

The published messages packet ID.

5.31.3.3 return_codes

```
const uint8_t* return_codes
```

Array of return codes corresponding to the requested subscribe topics.

See also

[MQTTSubackReturnCodes](#)

The documentation for this struct was generated from the following file:

- [linux/matesocketcan/mqtt.h](#)

5.32 mqtt_response_unsuback Struct Reference

The brokers response to a UNSUBSCRIBE request.

```
#include <mqtt.h>
```

Data Fields

- `uint16_t` [packet_id](#)
The published messages packet ID.

Related Symbols

(Note that these are not member symbols.)

- `ssize_t` [mqtt_unpack_unsuback_response](#) (struct [mqtt_response](#) *[mqtt_response](#), const `uint8_t` *[buf](#))
Deserialize an UNSUBACK packet from [buf](#).

5.32.1 Detailed Description

The brokers response to a UNSUBSCRIBE request.

See also

[MQTT v3.1.1: UNSUBACK – Unsubscribe Acknowledgement.](#)

5.32.2 Friends And Related Symbol Documentation

5.32.2.1 mqtt_unpack_unsuback_response()

```
ssize_t mqtt_unpack_unsuback_response (
    struct mqtt\_response * mqtt\_response,
    const uint8_t * buf) [related]
```

Deserialize an UNSUBACK packet from [buf](#).

Precondition

[mqtt_unpack_fixed_header](#) must have returned a positive value and the [mqtt_response](#) must have a control type of MQTT_CONTROL_UNSUBACK.

Parameters

out	mqtt_response	the response that is initialized from the contents of buf.
in	buf	the buffer with the incoming data.

Returns

The number of bytes that were consumed, or 0 if the buffer does not contain enough bytes to parse the packet, or a negative value if there was a protocol violation.

5.32.3 Field Documentation

5.32.3.1 packet_id

```
uint16_t packet_id
```

The published messages packet ID.

The documentation for this struct was generated from the following file:

- [linux/matesocketcan/mqtt.h](#)

5.33 MREG Union Reference

```
#include <modbus_master.h>
```

Data Fields

- [int16_t value](#)
- [char bytes](#) [2]

5.33.1 Field Documentation

5.33.1.1 bytes

```
char bytes[2]
```

5.33.1.2 value

```
int16_t value
```

The documentation for this union was generated from the following file:

- [modbus_master.h](#)

5.34 MREG32 Union Reference

```
#include <modbus_master.h>
```

Data Fields

- [int32_t value](#)
- [char bytes](#) [4]

5.34.1 Field Documentation

5.34.1.1 bytes

```
char bytes[4]
```

5.34.1.2 value

```
int32_t value
```

The documentation for this union was generated from the following file:

- [modbus_master.h](#)

5.35 MREG64 Union Reference

```
#include <modbus_master.h>
```

Data Fields

- [int64_t value](#)
- [char bytes](#) [8]

5.35.1 Field Documentation

5.35.1.1 bytes

```
char bytes[8]
```

5.35.1.2 value

```
int64_t value
```

The documentation for this union was generated from the following file:

- [modbus_master.h](#)

5.36 mx_log_packed_t Struct Reference

```
#include <mxcmd.h>
```

Data Fields

- uint8_t [a](#) [16]

5.36.1 Field Documentation

5.36.1.1 a

```
uint8_t a[16]
```

The documentation for this struct was generated from the following file:

- mateQ84.X/[mxcmd.h](#)

5.37 mx_logpage_t Struct Reference

```
#include <mxcmd.h>
```

Data Fields

- uint8_t [type](#)
- int16_t [day](#)
- int16_t [amp_hours](#)
- int16_t [kilowatt_hours](#)
- int16_t [volts_peak](#)
- int16_t [amps_peak](#)
- int16_t [kilowatts_peak](#)
- int16_t [bat_min](#)
- int16_t [bat_max](#)
- int16_t [absorb_time](#)
- int16_t [float_time](#)
- uint8_t [select](#)

5.37.1 Field Documentation

5.37.1.1 absorb_time

```
int16_t absorb_time
```

5.37.1.2 amp_hours

int16_t amp_hours

5.37.1.3 amps_peak

int16_t amps_peak

5.37.1.4 bat_max

int16_t bat_max

5.37.1.5 bat_min

int16_t bat_min

5.37.1.6 day

int16_t day

5.37.1.7 float_time

int16_t float_time

5.37.1.8 kilowatt_hours

int16_t kilowatt_hours

5.37.1.9 kilowatts_peak

int16_t kilowatts_peak

5.37.1.10 select

uint8_t select

5.37.1.11 type

uint8_t type

5.37.1.12 volts_peak

```
int16_t volts_peak
```

The documentation for this struct was generated from the following file:

- [mateQ84.X/mxcmd.h](#)

5.38 mx_status_packed_t Struct Reference

```
#include <mxcmd.h>
```

Data Fields

- [uint8_t a](#) [16]

5.38.1 Field Documentation

5.38.1.1 a

```
uint8_t a[16]
```

The documentation for this struct was generated from the following file:

- [mateQ84.X/mxcmd.h](#)

5.39 PWMDC Union Reference

```
#include <modbus_master.h>
```

Data Fields

- [uint16_t lpwm](#)
- [uint8_t bpwm](#) [2]

5.39.1 Field Documentation

5.39.1.1 bpwm

```
uint8_t bpwm[2]
```


5.39.1.2 lpwm

```
uint16_t lpwm
```

The documentation for this union was generated from the following file:

- [modbus_master.h](#)

5.40 ringBufS_t Struct Reference

```
#include <ringbufs.h>
```

Data Fields

- uint8_t [buf](#) [[RBUF_SIZE](#)]
- uint8_t [head](#)
- uint8_t [tail](#)
- uint8_t [count](#)

5.40.1 Field Documentation

5.40.1.1 buf

```
uint8_t buf[RBUF_SIZE]
```

5.40.1.2 count

```
uint8_t count
```

5.40.1.3 head

```
uint8_t head
```

5.40.1.4 tail

```
uint8_t tail
```

The documentation for this struct was generated from the following file:

- [ringbufs.h](#)

5.41 spi1_configuration_t Struct Reference

```
#include <eadog.h>
```

Data Fields

- uint8_t [con0](#)
- uint8_t [con1](#)
- uint8_t [con2](#)
- uint8_t [baud](#)
- uint8_t [operation](#)

5.41.1 Detailed Description

SPI1 Generated Driver File

@Company Microchip Technology Inc.

@File Name [spi1.c](#)

@Summary This is the generated driver implementation file for the SPI1 driver using PIC10 / PIC12 / PIC16 / PIC18 MCUs

@Description This header file provides implementations for driver APIs for SPI1. Generation Information : Product Revision : PIC10 / PIC12 / PIC16 / PIC18 MCUs - 1.81.8 Device : PIC18F47Q84 Driver Version : 1.0.0 The generated drivers are tested against the following: Compiler : XC8 2.36 and above or later MPLAB : MPLAB X 6.00

5.41.2 Field Documentation

5.41.2.1 baud

uint8_t baud

5.41.2.2 con0

uint8_t con0

5.41.2.3 con1

uint8_t con1

5.41.2.4 con2

uint8_t con2

5.41.2.5 operation

uint8_t operation

The documentation for this struct was generated from the following files:

- [eadog.h](#)
- [mateQ84.X/mcc_generated_files/spi1.c](#)

5.42 spi_link_type Struct Reference

```
#include <qconfig.h>
```

Data Fields

- uint8_t [SPI_LCD](#): 1
- uint8_t [SPI_AUX](#): 1
- uint8_t [LCD_TIMER](#): 1
- volatile uint8_t [LCD_DATA](#): 1
- uint16_t [delay](#)
- uint8_t [config](#)
- uint8_t * [txbuf](#)
- volatile int32_t [int_count](#)

5.42.1 Field Documentation

5.42.1.1 config

uint8_t config

5.42.1.2 delay

uint16_t delay

5.42.1.3 int_count

volatile int32_t int_count

5.42.1.4 LCD_DATA

volatile uint8_t LCD_DATA

5.42.1.5 LCD_TIMER

```
uint8_t LCD_TIMER
```

5.42.1.6 SPI_AUX

```
uint8_t SPI_AUX
```

5.42.1.7 SPI_LCD

```
uint8_t SPI_LCD
```

5.42.1.8 txbuf

```
uint8_t* txbuf
```

The documentation for this struct was generated from the following file:

- [qconfig.h](#)

5.43 uart1_status_t Union Reference

```
#include <uart1.h>
```

Data Fields

- struct {
 - unsigned [perr](#): 1
 - unsigned [ferr](#): 1
 - unsigned [oerr](#): 1
 - unsigned [reserved](#): 5
- };
- uint8_t [status](#)

5.43.1 Detailed Description

Section: Data Type Definitions

5.43.2 Field Documentation

5.43.2.1 [struct]

```
struct { ... }
```

5.43.2.2 ferr

unsigned ferr

5.43.2.3 oerr

unsigned oerr

5.43.2.4 perr

unsigned perr

5.43.2.5 reserved

unsigned reserved

5.43.2.6 status

uint8_t status

The documentation for this union was generated from the following file:

- mateQ84.X/mcc_generated_files/[uart1.h](#)

5.44 uart2_status_t Union Reference

```
#include <uart2.h>
```

Data Fields

- struct {
 - unsigned [perr](#): 1
 - unsigned [ferr](#): 1
 - unsigned [oerr](#): 1
 - unsigned [reserved](#): 5
- };
- uint8_t [status](#)

5.44.1 Detailed Description

Section: Data Type Definitions

5.44.2 Field Documentation

5.44.2.1 [struct]

```
struct { ... }
```

5.44.2.2 ferr

```
unsigned ferr
```

5.44.2.3 oerr

```
unsigned oerr
```

5.44.2.4 perr

```
unsigned perr
```

5.44.2.5 reserved

```
unsigned reserved
```

5.44.2.6 status

```
uint8_t status
```

The documentation for this union was generated from the following file:

- [mateQ84.X/mcc_generated_files/uart2.h](#)

5.45 uart5_status_t Union Reference

```
#include <uart5.h>
```

Data Fields

- struct {
 - unsigned [perr](#): 1
 - unsigned [ferr](#): 1
 - unsigned [oerr](#): 1
 - unsigned [reserved](#): 5
- };
- uint8_t [status](#)

5.45.1 Detailed Description

Section: Data Type Definitions

5.45.2 Field Documentation

5.45.2.1 [struct]

```
struct { ... }
```

5.45.2.2 ferr

```
unsigned ferr
```

5.45.2.3 oerr

```
unsigned oerr
```

5.45.2.4 perr

```
unsigned perr
```

5.45.2.5 reserved

```
unsigned reserved
```

5.45.2.6 status

```
uint8_t status
```

The documentation for this union was generated from the following file:

- [mateQ84.X/mcc_generated_files/uart5.h](#)

5.46 V_type Struct Reference

```
#include <modbus_master.h>
```

Data Fields

- uint32_t [StartTime](#)
- uint32_t [TimeUsed](#)
- volatile uint32_t [pacing](#)
- volatile uint32_t [pwm_update](#)
- volatile uint32_t [pwm_stop](#)
- volatile uint32_t [fault_count](#)
- volatile uint32_t [fault_ticks](#)
- volatile uint32_t [fault_source](#)
- volatile uint32_t [modbus_rx](#)
- volatile uint32_t [modbus_tx](#)
- int32_t [motor_speed](#)
- volatile bool [fault_active](#)
- volatile bool [dmt_sosc_flag](#)

5.46.1 Field Documentation

5.46.1.1 dmt_sosc_flag

```
volatile bool dmt_sosc_flag
```

5.46.1.2 fault_active

```
volatile bool fault_active
```

5.46.1.3 fault_count

```
volatile uint32_t fault_count
```

5.46.1.4 fault_source

```
volatile uint32_t fault_source
```

5.46.1.5 fault_ticks

```
volatile uint32_t fault_ticks
```

5.46.1.6 modbus_rx

```
volatile uint32_t modbus_rx
```

5.46.1.7 modbus_tx

```
volatile uint32_t modbus_tx
```


5.46.1.8 motor_speed

```
int32_t motor_speed
```

5.46.1.9 pacing

```
volatile uint32_t pacing
```

5.46.1.10 pwm_stop

```
volatile uint32_t pwm_stop
```

5.46.1.11 pwm_update

```
volatile uint32_t pwm_update
```

5.46.1.12 StartTime

```
uint32_t StartTime
```

5.46.1.13 TimeUsed

```
uint32_t TimeUsed
```

The documentation for this struct was generated from the following file:

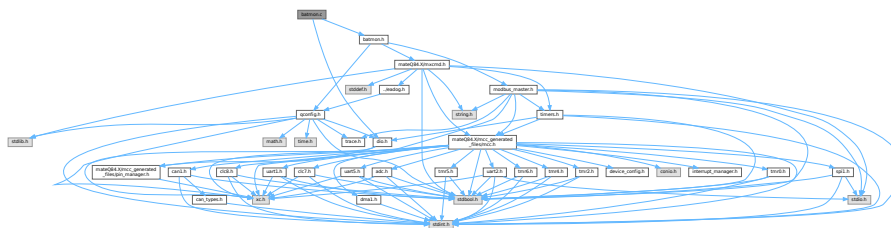
- [modbus_master.h](#)

Chapter 6

File Documentation

6.1 batmon.c File Reference

```
#include "batmon.h"
#include "dio.h"
Include dependency graph for batmon.c:
```



Functions

- [EB_data *](#) [get_EEPROM](#) (void)
- bool [initbm_data](#) (uint8_t *EB)
- void [wr_bm_data](#) (uint8_t *EB)
- void [get_bm_data](#) (EB_data *EB)
- void [compute_bm_data](#) (EB_data *EB)
- void [DATAEE_WriteByte](#) (const uint16_t bAdd, const uint8_t bData)
- uint8_t [DATAEE_ReadByte](#) (const uint16_t bAdd)
- uint16_t [Volts_to_SOC](#) (const uint16_t bvww, const uint16_t bvf)
- [device_id_data_t](#) [DeviceID_Read](#) ([device_id_address_t](#) address)
- void [update_time](#) (struct tm *ts, EB_data *EB)

Variables

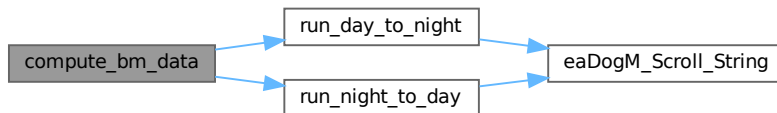
- [EB_data](#) [EBD](#)
- uint16_t [EBD_update](#) = 0
- float [pv_Wh_daily](#) = 0.0f
- float [pv_Wh_daily_prev](#) = 0.0f
- float [ac_Wh_daily](#) = 0.0f
- float [ac_Wh_daily_prev](#) = 0.0f

6.1.1 Function Documentation

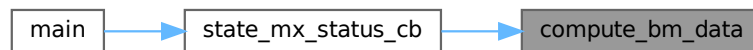
6.1.1.1 compute_bm_data()

```
void compute_bm_data (  
    EB_data * EB)
```

Here is the call graph for this function:



Here is the caller graph for this function:



6.1.1.2 DATAEE_ReadByte()

```
uint8_t DATAEE_ReadByte (  
    const uint16_t bAdd)
```

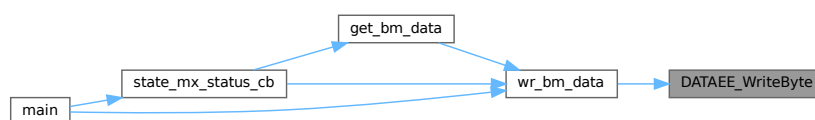
Here is the caller graph for this function:



6.1.1.3 DATAEE_WriteByte()

```
void DATAEE_WriteByte (  
    const uint16_t bAdd,  
    const uint8_t bData)
```

Here is the caller graph for this function:



6.1.1.4 DeviceID_Read()

```
device_id_data_t DeviceID_Read (  
    device_id_address_t address)
```

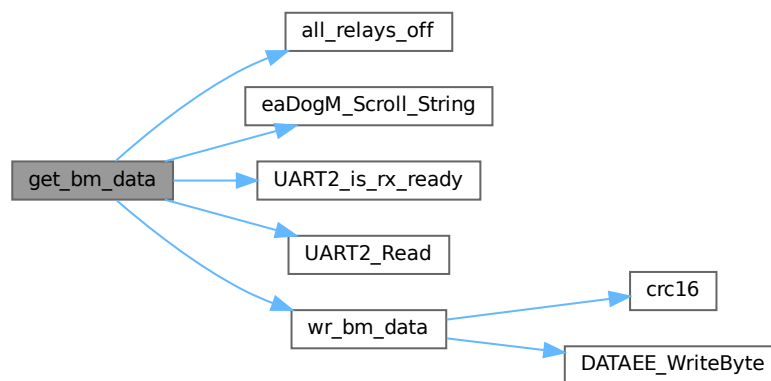
Here is the caller graph for this function:



6.1.1.5 get_bm_data()

```
void get_bm_data (  
    EB_data * EB)
```

Here is the call graph for this function:



Here is the caller graph for this function:



6.1.1.6 get_EEPROM()

```
EB_data * get_EEPROM (
    void )
```

6.1.1.7 initbm_data()

```
bool initbm_data (
    uint8_t * EB)
```

Here is the call graph for this function:



Here is the caller graph for this function:



6.1.1.8 update_time()

```
void update_time (  
    struct tm * ts,  
    EB_data * EB)
```

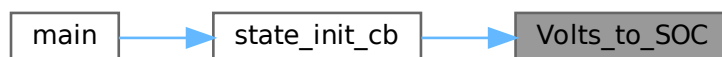
Here is the caller graph for this function:



6.1.1.9 Volts_to_SOC()

```
uint16_t Volts_to_SOC (  
    const uint16_t bvw,  
    const uint16_t bvf)
```

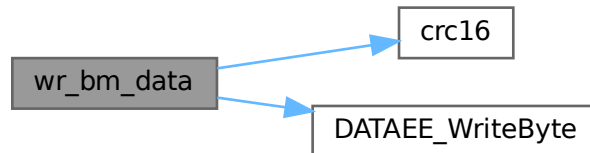
Here is the caller graph for this function:



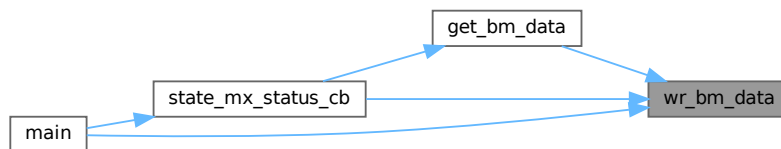
6.1.1.10 wr_bm_data()

```
void wr_bm_data (
    uint8_t * EB)
```

Here is the call graph for this function:



Here is the caller graph for this function:



6.1.2 Variable Documentation

6.1.2.1 ac_Wh_daily

```
float ac_Wh_daily = 0.0f
```

6.1.2.2 ac_Wh_daily_prev

```
float ac_Wh_daily_prev = 0.0f
```

6.1.2.3 EBD

`EB_data` EBD

Initial value:

```
= {
    .checkmark = BM_CM,
    .version = BM_EEPROM_VER,
    .loaded = false,
    .bat_cycles = 0,
    .bat_energy = BAT_ENERGY,
    .bat_mode = 0,
    .bat_time = 0,
    .crc = 0,
    .alt_display = 1,
    .q84_sequence = 0,
}
```


6.1.2.4 EBD_update

```
uint16_t EBD_update = 0
```

6.1.2.5 pv_Wh_daily

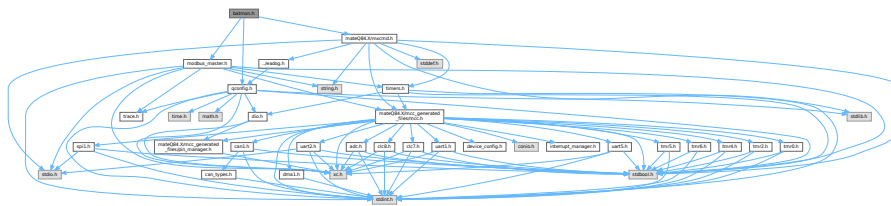
```
float pv_Wh_daily = 0.0f
```

6.1.2.6 pv_Wh_daily_prev

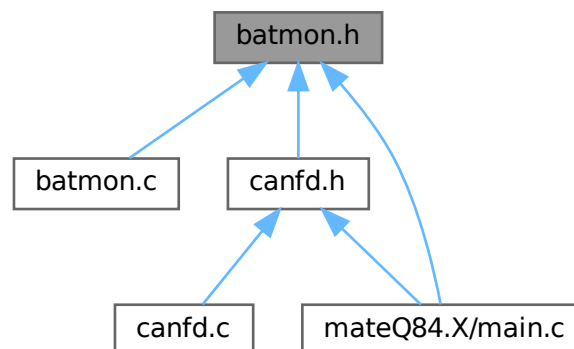
```
float pv_Wh_daily_prev = 0.0f
```

6.2 batmon.h File Reference

```
#include "qconfig.h"
#include "modbus_master.h"
#include "mateQ84.X/mxcmd.h"
Include dependency graph for batmon.h:
```



This graph shows which files directly or indirectly include this file:



Data Structures

- struct [EB_data](#)

Macros

- #define [CHK_DAY_TIME](#) 1200
- #define [TEN_SEC_HOUR](#) 360.0f
- #define [BM_EEPROM_VER](#) 1
- #define [BM_CM](#) 0x57+[BM_EEPROM_VER](#)
- #define [BM_UPDATE](#) 3600
- #define [BM_UPDATE_RUN](#) 1800
- #define [BAT_VOLT](#) 25.6f
- #define [BAT_AH](#) 200.0f
- #define [BAT_ENERGY](#) [BAT_VOLT](#)*[BAT_AH](#)*[TEN_SEC_HOUR](#)
- #define [BAT_ENERGY_LOW](#) [BAT_ENERGY](#)*0.5f
- #define [BAT_OVER_VOLT](#) 30.0f
- #define [BAT_OVER_RECON](#) 28.4f
- #define [BAT_UNDER_VOLT](#) 23.2f
- #define [BAT_LOW_DISC](#) 21.6f
- #define [BAT_LOW_RECON](#) 24.8f
- #define [BAT_EFF_VAL](#) 0.985f
- #define [BAT_CYCLES](#) 8
- #define [BAT_CHARGED_W](#) [BAT_AH](#)
- #define [BAT_CHARGED_V](#) 120.0f
- #define [BAT_DAY_COUNT](#) 45
- #define [BAT_NIGHT_COUNT](#) 90
- #define [BAT_RUN_MAX](#) 280.0f
- #define [IDLE_DRAIN](#) 10.0f
- #define [INV_EFF_VAL](#) 1.06f
- #define [MAX_ALT_DIS](#) 3
- #define [LOG_VARS](#) abuf[3] - 128, abuf[1]&0x0f, [vw](#), [vf](#), abuf[2] - 128, [volt_whole](#), [volt_fract](#), [panel_watts](#), [pv_Wh_daily](#), [ac_Wh_daily](#), B.run_time, B.net_balance, [cc_mode](#), ((float) em.wl1) / 10.0f, ((float) em.val1) / 10.0f, ((float) em.var11) / 10.0f, ((float) em.vl1l2) / 10.0f, EBD.bat_energy / 3600.0f, EBD.bat_cycles, ((float) em.pfl1) / 1000.0f, ((float) emt.hz) / 1000.0f, B.rx_count++,[buffer](#)
- #define [BVSOC_SLOTS](#) 12

Typedefs

- typedef struct EB_data [EB_data](#)
- typedef uint16_t [device_id_data_t](#)
- typedef uint24_t [device_id_address_t](#)

Functions

- [device_id_data_t DeviceID_Read](#) ([device_id_address_t](#))
- void [run_night_to_day](#) (void)
- void [run_day_to_night](#) (void)
- bool [initbm_data](#) (uint8_t *)
- void [wr_bm_data](#) (uint8_t *)
- void [get_bm_data](#) ([EB_data](#) *)
- void [compute_bm_data](#) ([EB_data](#) *)
- void [update_time](#) (struct tm *, [EB_data](#) *)
- [EB_data](#) * [get_EEPROM](#) (void)
- void [DATAEE_WriteByte](#) (const uint16_t, const uint8_t)
- uint8_t [DATAEE_ReadByte](#) (const uint16_t)
- uint16_t [Volts_to_SOC](#) (const uint16_t, const uint16_t)

Variables

- const char [log_format](#) [] = "^,%d.%01d,%d.%01d,%d,%d.%01d,%d,%d.%1f,%d.%1f,%d.%2f,%d.%2f,%d,%d.%1f,%d.%1f,%d.%1f,%d.%2f,%u,%5.3f,%5.3f,%u,%s,~\r\n"
- const uint32_t [BVSOC_TABLE](#) [[BVSOC_SLOTS](#)][2]
- [EB_data](#) EBD
- uint16_t [EBD_update](#)
- struct tm * [can_newtime](#)
- float [pv_Wh_daily](#)
- float [pv_Wh_daily_prev](#)
- float [ac_Wh_daily](#)
- float [ac_Wh_daily_prev](#)

6.2.1 Macro Definition Documentation

6.2.1.1 BAT_AH

```
#define BAT_AH 200.0f
```

6.2.1.2 BAT_CHARGED_V

```
#define BAT_CHARGED_V 120.0f
```

6.2.1.3 BAT_CHARGED_W

```
#define BAT_CHARGED_W BAT\_AH
```

6.2.1.4 BAT_CYCLES

```
#define BAT_CYCLES 8
```

6.2.1.5 BAT_DAY_COUNT

```
#define BAT_DAY_COUNT 45
```

6.2.1.6 BAT_EFF_VAL

```
#define BAT_EFF_VAL 0.985f
```

6.2.1.7 BAT_ENERGY

```
#define BAT_ENERGY BAT\_VOLT*BAT\_AH*TEN\_SEC\_HOUR
```

6.2.1.8 BAT_ENERGY_LOW

```
#define BAT_ENERGY_LOW BAT_ENERGY*0.5f
```

6.2.1.9 BAT_LOW_DISC

```
#define BAT_LOW_DISC 21.6f
```

6.2.1.10 BAT_LOW_RECON

```
#define BAT_LOW_RECON 24.8f
```

6.2.1.11 BAT_NIGHT_COUNT

```
#define BAT_NIGHT_COUNT 90
```

6.2.1.12 BAT_OVER_RECON

```
#define BAT_OVER_RECON 28.4f
```

6.2.1.13 BAT_OVER_VOLT

```
#define BAT_OVER_VOLT 30.0f
```

6.2.1.14 BAT_RUN_MAX

```
#define BAT_RUN_MAX 280.0f
```

6.2.1.15 BAT_UNDER_VOLT

```
#define BAT_UNDER_VOLT 23.2f
```

6.2.1.16 BAT_VOLT

```
#define BAT_VOLT 25.6f
```

6.2.1.17 BM_CM

```
#define BM_CM 0x57+BM_EEPROM_VER
```

6.2.1.18 BM_EEPROM_VER

```
#define BM_EEPROM_VER 1
```

6.2.1.19 BM_UPDATE

```
#define BM_UPDATE 3600
```

6.2.1.20 BM_UPDATE_RUN

```
#define BM_UPDATE_RUN 1800
```

6.2.1.21 BVSOC_SLOTS

```
#define BVSOC_SLOTS 12
```

6.2.1.22 CHK_DAY_TIME

```
#define CHK_DAY_TIME 1200
```

6.2.1.23 IDLE_DRAIN

```
#define IDLE_DRAIN 10.0f
```

6.2.1.24 INV_EFF_VAL

```
#define INV_EFF_VAL 1.06f
```

6.2.1.25 LOG_VARS

```
#define LOG_VARS abuf[3] - 128, abuf[1]&0x0f, vw, vf, abuf[2] - 128, volt_whole, volt_fract,  
panel_watts, pv_Wh_daily, ac_Wh_daily, B.run_time, B.net_balance, cc_mode, ((float) em.wl1)  
/ 10.0f, ((float) em.val1) / 10.0f, ((float) em.varl1) / 10.0f, ((float) em.vl1l2) / 10.↔  
0f, EBD.bat_energy / 3600.0f, EBD.bat_cycles, ((float) em.pfl1) / 1000.0f, ((float) emt.hz) /  
1000.0f, B.rx_count++,buffer
```

6.2.1.26 MAX_ALT_DIS

```
#define MAX_ALT_DIS 3
```

6.2.1.27 TEN_SEC_HOUR

```
#define TEN_SEC_HOUR 360.0f
```

6.2.2 Typedef Documentation

6.2.2.1 device_id_address_t

```
typedef uint24_t device_id_address_t
```

6.2.2.2 device_id_data_t

```
typedef uint16_t device_id_data_t
```

6.2.2.3 EB_data

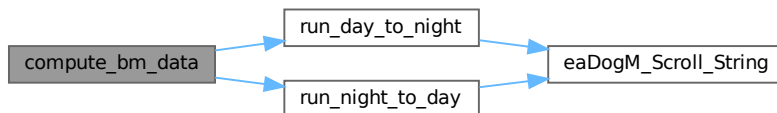
```
typedef struct EB_data EB_data
```

6.2.3 Function Documentation

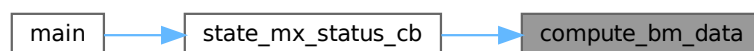
6.2.3.1 compute_bm_data()

```
void compute_bm_data (  
    EB_data * EB)
```

Here is the call graph for this function:



Here is the caller graph for this function:



6.2.3.2 DATAEE_ReadByte()

```
uint8_t DATAEE_ReadByte (  
    const uint16_t bAdd)
```

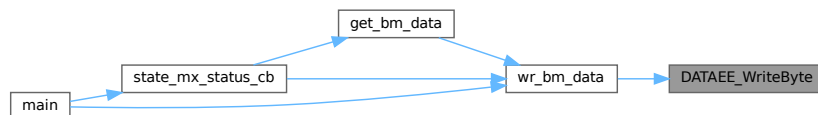
Here is the caller graph for this function:



6.2.3.3 DATAEE_WriteByte()

```
void DATAEE_WriteByte (  
    const uint16_t bAdd,  
    const uint8_t bData)
```

Here is the caller graph for this function:



6.2.3.4 DeviceID_Read()

```
device_id_data_t DeviceID_Read (  
    device_id_address_t address)
```

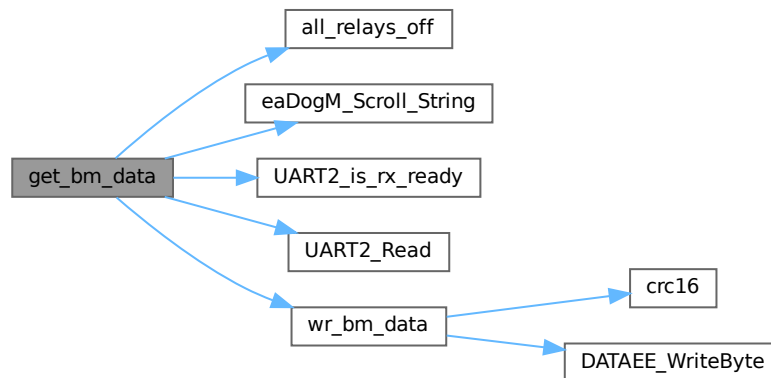
Here is the caller graph for this function:



6.2.3.5 get_bm_data()

```
void get_bm_data (
    EB_data * EB)
```

Here is the call graph for this function:



Here is the caller graph for this function:



6.2.3.6 get_EEPROM()

```
EB_data * get_EEPROM (
    void )
```

6.2.3.7 initbm_data()

```
bool initbm_data (
    uint8_t * EB)
```


Here is the call graph for this function:



Here is the caller graph for this function:



6.2.3.8 run_day_to_night()

```
void run_day_to_night (  
    void ) [extern]
```

Here is the call graph for this function:



Here is the caller graph for this function:



6.2.3.9 run_night_to_day()

```
void run_night_to_day (  
    void ) [extern]
```

Here is the call graph for this function:



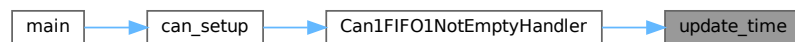
Here is the caller graph for this function:



6.2.3.10 update_time()

```
void update_time (  
    struct tm * ts,  
    EB_data * EB)
```

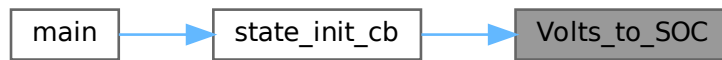
Here is the caller graph for this function:



6.2.3.11 Volts_to_SOC()

```
uint16_t Volts_to_SOC (  
    const uint16_t bvw,  
    const uint16_t bvf)
```

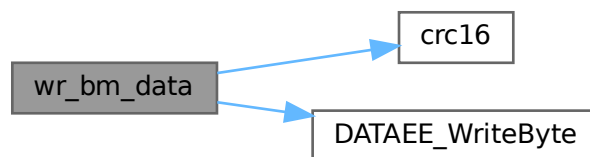
Here is the caller graph for this function:



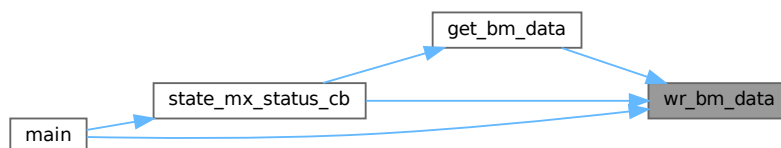
6.2.3.12 wr_bm_data()

```
void wr_bm_data (  
    uint8_t * EB)
```

Here is the call graph for this function:



Here is the caller graph for this function:



6.2.4 Variable Documentation

6.2.4.1 ac_Wh_daily

```
float ac_Wh_daily
```

6.2.4.2 ac_Wh_daily_prev

```
float ac_Wh_daily_prev
```

6.2.4.3 BVSOC_TABLE

```
const uint32_t BVSOC_TABLE[BVSOC_SLOTS][2]
```

Initial value:

$$= \{ \begin{array}{ll} 20000, & 0, \\ 24000, & 5, \\ 25000, & 14, \\ 25600, & 17, \\ 25800, & 20, \\ 26000, & 30, \\ 26200, & 40, \\ 26400, & 70, \\ 26600, & 90, \\ 26800, & 99, \\ 27200, & 100, \\ 29200, & 100 \end{array} \}$$

6.2.4.4 can_newtime

```
struct tm* can_newtime [extern]
```

6.2.4.5 EBD

```
EB_data EBD [extern]
```

6.2.4.6 EBD_update

```
uint16_t EBD_update [extern]
```

6.2.4.7 log_format

```
const char log_format[] = "%^,%d,%01d,%d,%d,%01d,%d,%1f,%1f,%2f,%2f,%d,%1f,%1f,%1f,%4.1f,%2f,%u,%5.3f,%5.3f,%u,%s,~\r\n"
```

6.2.4.8 pv_Wh_daily

```
float pv_Wh_daily [extern]
```

6.2.4.9 pv_Wh_daily_prev

```
float pv_Wh_daily_prev
```

6.3 batmon.h

[Go to the documentation of this file.](#)

```
00001 /*  
00002 * File: batmon.h  
00003 * Author: root  
00004 *  
00005 * Created on June 25, 2023, 10:39 AM  
00006 */  
00007  
00008 #ifndef BATMON_H  
00009 #define BATMON_H  
00010  
00011 #ifdef __cplusplus  
00012 extern "C" {  
00013 #endif  
00014  
00015 #include "qconfig.h"  
00016 #include "modbus_master.h"  
00017 #include "mateQ84.X/mxcmd.h"  
00018  
00019 #define CHK_DAY_TIME 1200  
00020 #define TEN_SEC_HOUR 360.0f  
00021 #define BM_EEPROM_VER 1  
00022 #define BM_CM 0x57+BM_EEPROM_VER  
00023 #define BM_UPDATE 3600 // seconds between EEPROM updates, sleeping  
00024 #define BM_UPDATE_RUN 1800 // while running  
00025  
00026 /*  
00027 * Energy Battery Bank  
00028 * 24V LiFePO4 Battery Parameters  
00029 */  
00030 #define BAT_VOLT 25.6f  
00031 #define BAT_AH 200.0f  
00032 #define BAT_ENERGY BAT_VOLT*BAT_AH*TEN_SEC_HOUR // nominal battery voltage and AH rating for 10  
second updates  
00033 #define BAT_ENERGY_LOW BAT_ENERGY*0.5f  
00034 #define BAT_OVER_VOLT 30.0f  
00035 #define BAT_OVER_RECON 28.4f  
00036 #define BAT_UNDER_VOLT 23.2f  
00037 #define BAT_LOW_DISC 21.6f  
00038 #define BAT_LOW_RECON 24.8f  
00039 #define BAT_EFF_VAL 0.985f // battery storage energy efficiency  
00040 #define BAT_CYCLES 8  
00041 #define BAT_CHARGED_W BAT_AH  
00042 #define BAT_CHARGED_V 120.0f  
00043 #define BAT_DAY_COUNT 45 // number of reports before updates  
00044 #define BAT_NIGHT_COUNT 90  
00045 #define BAT_RUN_MAX 280.0f // max displayed run time at current load  
00046 /*  
00047 * System correction factors  
00048 */  
00049 #define IDLE_DRAIN 10.0f // system operational drain losses in Wh  
00050 #define INV_EFF_VAL 1.06f // DC watts to AC watt inverter correction  
00051  
00052 #define MAX_ALT_DIS 3  
00053 // EEPROM data storage structure  
00054  
00055 typedef struct EB_data {  
00056     uint8_t checkmark;  
00057     uint8_t version, alt_display;  
00058     bool loaded;  
00059     float FMw, Fmpv, FMa, Fmbv, ENw, ENva, ENvar, ENac;  
00060     float volt_whole, bat_amp_whole;  
00061     float bat_energy;  
00062     uint16_t cc_mode, bat_cycles, bat_mode, time, date;  
00063     uint32_t bat_time, fm80_time, q84_sequence;  
00064     uint16_t crc;  
00065 } EB_data;  
00066  
00067 /*  
00068 * logging data format for printf and variables  
00069 */  
00070 const char log_format[] =  
00071     "^,%d.%01d,%d.%01d,%d,%d.%01d,%d,%1f,%1f,%1f,%2f,%d,%1f,%1f,%1f,%4.1f,%2f,%u,%5.3f,%5.3f,%u,%s,%s,%r\n";  
00072  
00073 #define LOG_VARS abuf[3] - 128, abuf[1]&0x0f, vw, vf, abuf[2] - 128, volt_whole, volt_fract,  
panel_watts, pv_Wh_daily, ac_Wh_daily, B.run_time, B.net_balance, cc_mode, ((float) em.wll) / 10.0f,  
((float) em.vall) / 10.0f, ((float) em.varll) / 10.0f, ((float) em.vlll2) / 10.0f, EBD.bat_energy /  
3600.0f, EBD.bat_cycles, ((float) em.pfl1) / 1000.0f, ((float) emt.hz) / 1000.0f, B.rx_count++,buffer  
00074  
00075 #define BVSOC_SLOTS 12 // 24V LiFePO4 Battery to SOC data table slots  
00076 const uint32_t BVSOC_TABLE[BVSOC_SLOTS][2] = {  
00077     20000, 0,  
00078     24000, 5,  
00079     25000, 14.  

```

```

00078         25600, 17,
00079         25800, 20,
00080         26000, 30,
00081         26200, 40,
00082         26400, 70,
00083         26600, 90,
00084         26800, 99,
00085         27200, 100,
00086         29200, 100 // charging voltage guess
00087     };
00088
00089     typedef uint16_t device_id_data_t;
00090     typedef uint24_t device_id_address_t;
00091     device_id_data_t DeviceID_Read(device_id_address_t);
00092
00093     extern EB_data EBD;
00094     extern uint16_t EBD_update;
00095     extern struct tm *can_newtime;
00096     extern float pv_Wh_daily, pv_Wh_daily_prev, ac_Wh_daily, ac_Wh_daily_prev;
00097     extern void run_night_to_day(void);
00098     extern void run_day_to_night(void);
00099
00100     bool initbm_data(uint8_t *);
00101     void wr_bm_data(uint8_t *);
00102     void get_bm_data(EB_data *);
00103     void compute_bm_data(EB_data *);
00104     void update_time(struct tm *, EB_data *);
00105
00106     EB_data * get_EEPROM(void);
00107     void DATAEE_WriteByte(const uint16_t, const uint8_t);
00108     uint8_t DATAEE_ReadByte(const uint16_t);
00109
00110     uint16_t Volts_to_SOC(const uint16_t, const uint16_t);
00111
00112 #ifdef __cplusplus
00113 }
00114 #endif
00115
00116 #endif /* BATMON_H */
00117

```

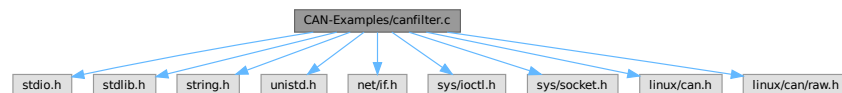
6.4 CAN-Examples/canfilter.c File Reference

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <net/if.h>
#include <sys/ioctl.h>
#include <sys/socket.h>
#include <linux/can.h>
#include <linux/can/raw.h>

```

Include dependency graph for canfilter.c:



Functions

- int [main](#) (int argc, char **argv)

6.4.1 Function Documentation

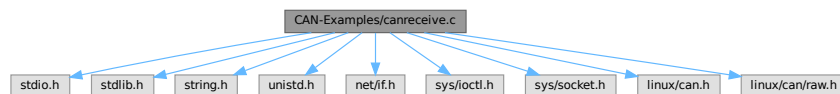
6.4.1.1 main()

```
int main (  
    int argc,  
    char ** argv)
```

6.5 CAN-Examples/canreceive.c File Reference

```
#include <stdio.h>  
#include <stdlib.h>  
#include <string.h>  
#include <unistd.h>  
#include <net/if.h>  
#include <sys/ioctl.h>  
#include <sys/socket.h>  
#include <linux/can.h>  
#include <linux/can/raw.h>
```

Include dependency graph for canreceive.c:



Functions

- int [main](#) (int argc, char **argv)

6.5.1 Function Documentation

6.5.1.1 main()

```
int main (  
    int argc,  
    char ** argv)
```


Functions

- void [Can1FIFO1NotEmptyHandler](#) (void)
- void [can_mirror_print](#) (void)
- void [can_fd_tx](#) (void)
- void [can_setup](#) (void)
- void [can_fd_lcd_mirror](#) (const uint8_t r, char *strPtr)
- void [TXQNotFullHandler](#) (void)
- void [can_blob_set](#) (blob_type *data)
- bool [get_blob_tx](#) (void)
- blob_type * [can_blob_get](#) (blob_type *data)
- bool [get_blob_rx](#) (void)

Variables

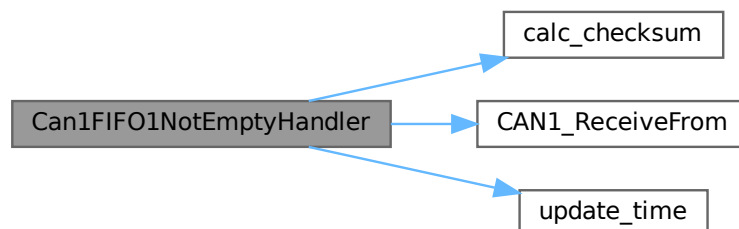
- [CAN_MSG_OBJ](#) msg [CANFD_NBUF]
- volatile uint8_t rxMsgData [CAN_REC_BUFFERS][CANFD_BYTES]
- volatile [can_frames_t](#) can_packets
- volatile [can_rec_count_t](#) can_rec_count
- union {
 - uint16_t [Word](#)
 - struct {
 - uint8_t [Byte1](#): 8
 - uint8_t [Byte2](#): 8
 - } [structBytes](#)
- } [myVar](#)

6.8.1 Function Documentation

6.8.1.1 Can1FIFO1NotEmptyHandler()

```
void Can1FIFO1NotEmptyHandler (
    void )
```

Here is the call graph for this function:



Here is the caller graph for this function:



6.8.1.2 can_blob_get()

```

blob_type * can_blob_get (
    blob_type * data)
  
```

6.8.1.3 can_blob_set()

```

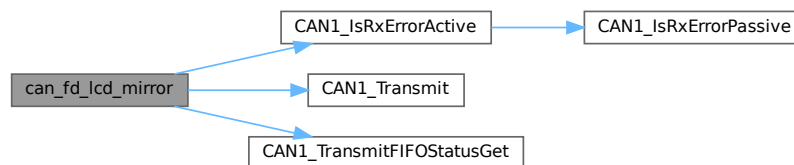
void can_blob_set (
    blob_type * data)
  
```

6.8.1.4 can_fd_lcd_mirror()

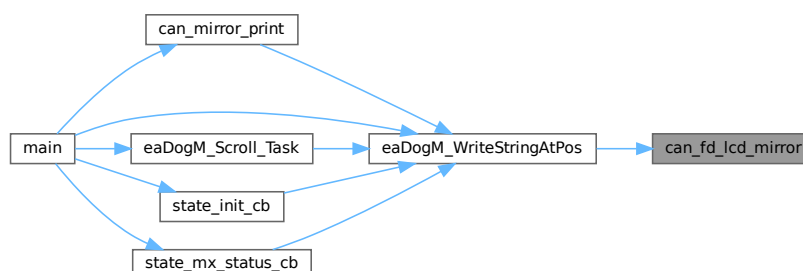
```

void can_fd_lcd_mirror (
    const uint8_t r,
    char * strPtr)
  
```

Here is the call graph for this function:



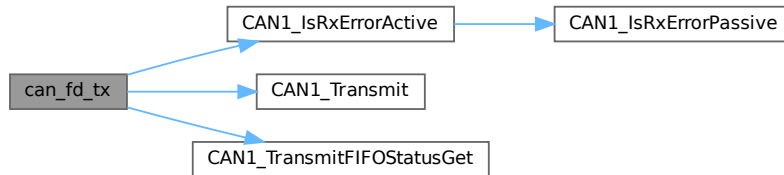
Here is the caller graph for this function:



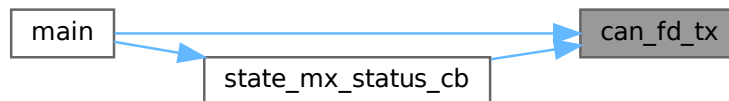
6.8.1.5 can_fd_tx()

```
void can_fd_tx (
    void )
```

Here is the call graph for this function:



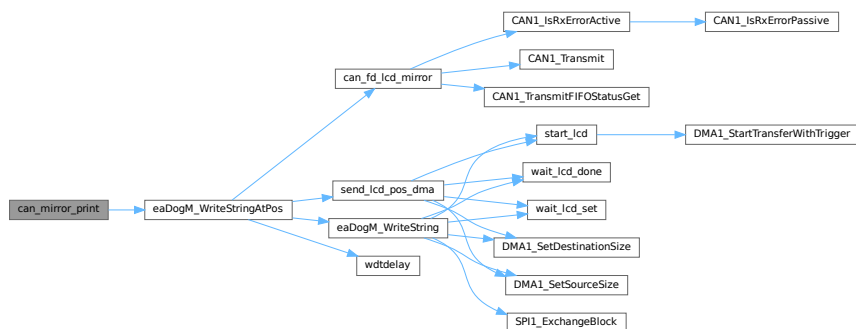
Here is the caller graph for this function:



6.8.1.6 can_mirror_print()

```
void can_mirror_print (
    void )
```

Here is the call graph for this function:



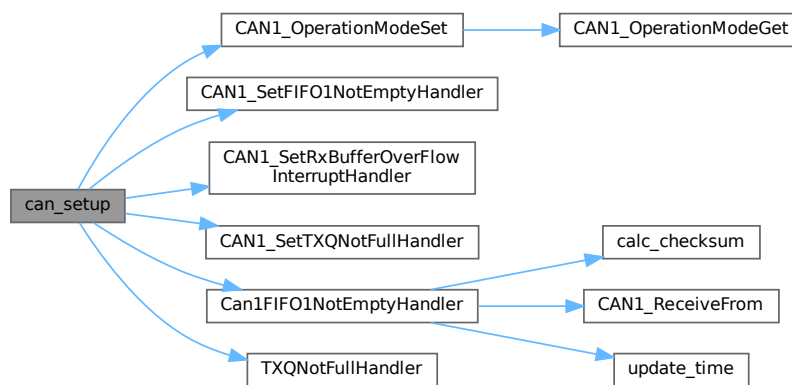
Here is the caller graph for this function:



6.8.1.7 can_setup()

```
void can_setup (  
    void )
```

Here is the call graph for this function:



Here is the caller graph for this function:



6.8.1.8 get_blob_rx()

```
bool get_blob_rx (  
    void )
```

6.8.1.9 get_blob_tx()

```
bool get_blob_tx (  
    void )
```

6.8.1.10 TXQNotFullHandler()

```
void TXQNotFullHandler (  
    void )
```

Here is the caller graph for this function:



6.8.2 Variable Documentation

6.8.2.1 Byte1

```
uint8_t Byte1
```

6.8.2.2 Byte2

```
uint8_t Byte2
```

6.8.2.3 can_packets

```
volatile can_frames_t can_packets
```

Initial value:

```
= {  
    .rec_count = 0,  
    .tx_count = 0,  
}
```

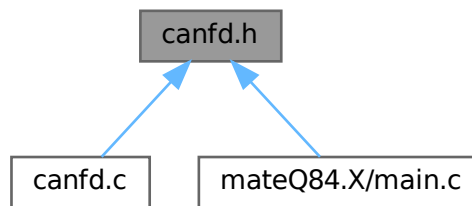
6.8.2.4 can_rec_count

```
volatile can_rec_count_t can_rec_count
```

Initial value:

```
= {  
    .rec_count = 0,  
    .rec_flag = false,  
}
```


This graph shows which files directly or indirectly include this file:



Data Structures

- struct [can_rec_count_t](#)
- struct [can_frames_t](#)
- struct [blob_type](#)
- union [blob_log](#)

Macros

- #define [DATA_DEBUG](#)
- #define [NO_NODE_ID](#)
- #define [NODE_ID_SHIFT](#) 4
- #define [USE_FD](#)
- #define [CANFD_BYTES](#) 64
- #define [CANFD_NBUF](#) 3
- #define [CAN_RX_TRIES](#) 8
- #define [CAN_REC_BUFFERS](#) 4
- #define [CAN_LOW_BUF](#) 0
- #define [CAN_HIGH_BUF](#) 1
- #define [CAN_INFO_BUF](#) 2
- #define [CAN_ERROR_BUF](#) 3
- #define [CAN_MIRRORS](#) 4
- #define [EMON_M](#) 0x1
- #define [EMON_SL](#) 0x2
- #define [EMON_SU](#) 0x3
- #define [EMON_MR](#) 0x4
- #define [EMON_TM](#) 0xA
- #define [EMON_DS](#) 0xB
- #define [EMON_DM](#) 0xE
- #define [EMON_ER](#) 0xF
- #define [EMON_CO](#) 0xC
- #define [EMON_DA](#) 0xD
- #define [MIRROR_BUF](#) [CANFD_NBUF](#)-1
- #define [LCD_BUF_SIZ](#) 22

Functions

- void [TXQNotFullHandler](#) (void)
- void [Can1FIFO1NotEmptyHandler](#) (void)
- void [can_fd_tx](#) (void)
- void [can_setup](#) (void)
- void [can_fd_lcd_mirror](#) (const uint8_t, char *)
- void [can_mirror_print](#) (void)
- void [can_blob_set](#) (blob_type *)
- blob_type * [can_blob_get](#) (blob_type *)
- bool [get_blob_tx](#) (void)
- bool [get_blob_rx](#) (void)

Variables

- volatile [can_rec_count_t](#) [can_rec_count](#)
- [CAN_MSG_OBJ](#) msg [3]
- volatile uint8_t rxMsgData [CAN_REC_BUFFERS][CANFD_BYTES]
- time_t [can_timer](#)
- struct tm * [can_newtime](#)
- volatile [can_frames_t](#) [can_packets](#)
- char [info_buffer](#) [MAX_B_BUF]
- char [log_buffer](#) [MAX_B_BUF]

6.9.1 Macro Definition Documentation

6.9.1.1 CAN_ERROR_BUF

```
#define CAN_ERROR_BUF 3
```

6.9.1.2 CAN_HIGH_BUF

```
#define CAN_HIGH_BUF 1
```

6.9.1.3 CAN_INFO_BUF

```
#define CAN_INFO_BUF 2
```

6.9.1.4 CAN_LOW_BUF

```
#define CAN_LOW_BUF 0
```

6.9.1.5 CAN_MIRRORS

```
#define CAN_MIRRORS 4
```


6.9.1.6 CAN_REC_BUFFERS

```
#define CAN_REC_BUFFERS 4
```

6.9.1.7 CAN_RX_TRIES

```
#define CAN_RX_TRIES 8
```

6.9.1.8 CANFD_BYTES

```
#define CANFD_BYTES 64
```

6.9.1.9 CANFD_NBUF

```
#define CANFD_NBUF 3
```

6.9.1.10 DATA_DEBUG

```
#define DATA_DEBUG
```

6.9.1.11 EMON_CO

```
#define EMON_CO 0xC
```

6.9.1.12 EMON_DA

```
#define EMON_DA 0xD
```

6.9.1.13 EMON_DM

```
#define EMON_DM 0xE
```

6.9.1.14 EMON_DS

```
#define EMON_DS 0xB
```

6.9.1.15 EMON_ER

```
#define EMON_ER 0xF
```

6.9.1.16 EMON_M

```
#define EMON_M 0x1
```

6.9.1.17 EMON_MR

```
#define EMON_MR 0x4
```

6.9.1.18 EMON_SL

```
#define EMON_SL 0x2
```

6.9.1.19 EMON_SU

```
#define EMON_SU 0x3
```

6.9.1.20 EMON_TM

```
#define EMON_TM 0xA
```

6.9.1.21 LCD_BUF_SIZ

```
#define LCD_BUF_SIZ 22
```

6.9.1.22 MIRROR_BUF

```
#define MIRROR_BUF CANFD\_NBUF-1
```

6.9.1.23 NO_NODE_ID

```
#define NO_NODE_ID
```

6.9.1.24 NODE_ID_SHIFT

```
#define NODE_ID_SHIFT 4
```

6.9.1.25 USE_FD

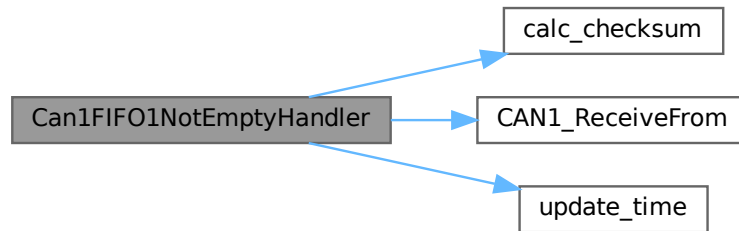
```
#define USE_FD
```

6.9.2 Function Documentation

6.9.2.1 Can1FIFO1NotEmptyHandler()

```
void Can1FIFO1NotEmptyHandler (  
    void )
```

Here is the call graph for this function:



Here is the caller graph for this function:



6.9.2.2 can_blob_get()

```
blob_type * can_blob_get (  
    blob_type * data)
```

6.9.2.3 can_blob_set()

```
void can_blob_set (  
    blob_type * data)
```

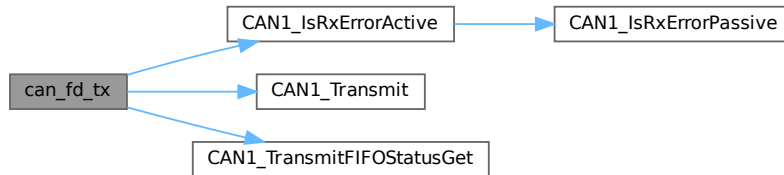
6.9.2.4 can_fd_lcd_mirror()

```
void can_fd_lcd_mirror (  
    const uint8_t r,  
    char * strPtr)
```

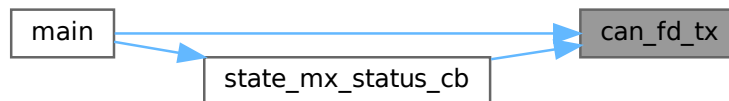
6.9.2.5 can_fd_tx()

```
void can_fd_tx (
    void )
```

Here is the call graph for this function:



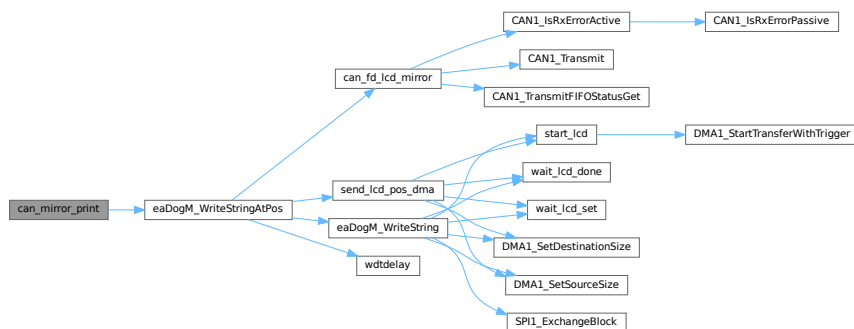
Here is the caller graph for this function:



6.9.2.6 can_mirror_print()

```
void can_mirror_print (
    void )
```

Here is the call graph for this function:



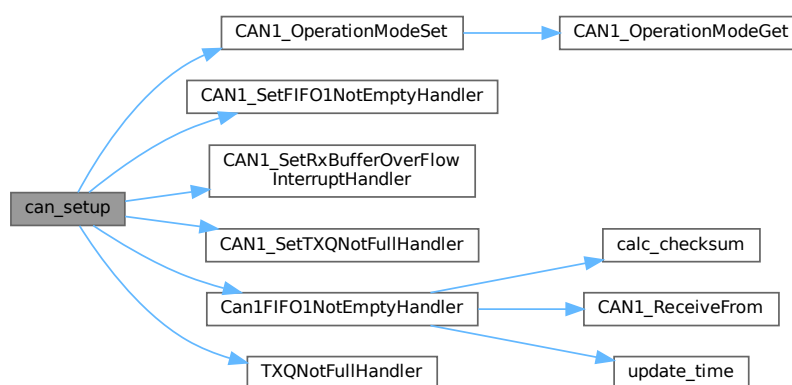
Here is the caller graph for this function:



6.9.2.7 can_setup()

```
void can_setup (  
    void )
```

Here is the call graph for this function:



Here is the caller graph for this function:



6.9.2.8 get_blob_rx()

```
bool get_blob_rx (  
    void )
```

6.9.2.9 get_blob_tx()

```
bool get_blob_tx (  
    void )
```

6.9.2.10 TXQNotFullHandler()

```
void TXQNotFullHandler (  
    void )
```

Here is the caller graph for this function:



6.9.3 Variable Documentation

6.9.3.1 can_newtime

```
struct tm* can_newtime [extern]
```

6.9.3.2 can_packets

```
volatile can_frames_t can_packets [extern]
```

6.9.3.3 can_rec_count

```
volatile can_rec_count_t can_rec_count [extern]
```

6.9.3.4 can_timer

```
time_t can_timer [extern]
```

6.9.3.5 info_buffer

```
char info_buffer[MAX_B_BUF] [extern]
```

6.9.3.6 log_buffer

```
char log_buffer[MAX_B_BUF]
```

6.9.3.7 msg

```
CAN_MSG_OBJ msg[3] [extern]
```

6.9.3.8 rxMsgData

```
volatile uint8_t rxMsgData[CAN_REC_BUFFERS][CANFD_BYTES] [extern]
```

6.10 canfd.h

[Go to the documentation of this file.](#)

```
00001 /*
00002  * File:   canfd.h
00003  * Author: root
00004  *
00005  * Created on June 16, 2023, 12:55 PM
00006  */
00007
00008 /*
00009  * simple CAN FD transfers using blocking TX and interrupts for RX
00010  */
00011 #ifndef CANFD_H
00012 #define CANFD_H
00013
00014 #ifdef __cplusplus
00015 extern "C" {
00016 #endif
00017
00018 #include "qconfig.h"
00019 #include "mateQ84.X/mxcmd.h"
00020 #include "modbus_master.h"
00021 #include "batmon.h"
00022
00023 #define DATA_DEBUG
00024 #define NO_NODE_ID
00025 #define NODE_ID_SHIFT 4
00026
00027 #ifdef CAN_REMOTE
00028 #define LCD_MIRROR
00029 #endif
00030
00031 #define USE_FD // select classic or FD
00032 #define CANFD_BYTES 64
00033 #define CANFD_NBUF 3
00034 #define CAN_RX_TRIES 8
00035 #define CAN_REC_BUFFERS 4
00036 #define CAN_LOW_BUF 0
00037 #define CAN_HIGH_BUF 1
00038 #define CAN_INFO_BUF 2
00039 #define CAN_ERROR_BUF 3
00040 #define CAN_MIRRORS 4
00041
00042 #define EMON_M 0x1 // FM80 host
00043 #define EMON_SL 0x2 // remote display lower data
00044 #define EMON_SU 0x3 // remote display upper data
00045 #define EMON_MR 0x4 // LCD mirror data [0..3]
00046 #define EMON_TM 0xA // remote time from time server
00047 #define EMON_DS 0xB // remote display server
00048 #define EMON_DM 0xE // set display mode to 1
00049 #define EMON_ER 0xF // error reporting
00050 #define EMON_CO 0xC // configuration reporting
00051 #define EMON_DA 0xD // 64 byte binary blob
00052
00053 #define MIRROR_BUF CANFD_NBUF-1 // LCD mirror CANBUS buffer
00054 #define LCD_BUF_SIZ 22
00055
```


6.11.1 Function Documentation

6.11.1.1 all_relays_off()

```
void all_relays_off (  
    void )
```

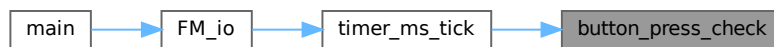
Here is the caller graph for this function:



6.11.1.2 button_press_check()

```
void button_press_check (  
    void )
```

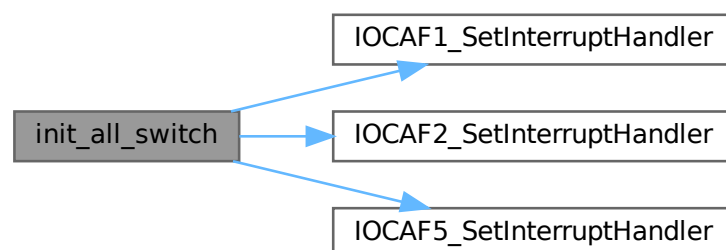
Here is the caller graph for this function:



6.11.1.3 init_all_switch()

```
void init_all_switch (  
    void )
```

Here is the call graph for this function:



Macros

- `#define DAY_RELAY_ON RELAY_SetHigh()`
- `#define DAY_RELAY_OFF RELAY_SetLow()`
- `#define NIGHT_RELAY_ON MISC_SetHigh()`
- `#define NIGHT_RELAY_OFF MISC_SetLow()`
- `#define CHARGER_RELAY_ON CHARGER_SetHigh()`
- `#define CHARGER_RELAY_OFF CHARGER_SetLow()`
- `#define D_SW_A_PIN A_SWITCH_GetValue()`
- `#define D_SW_L_PIN L_SWITCH_GetValue()`
- `#define D_SW_M_PIN M_SWITCH_GetValue()`
- `#define debounce_time 300`
- `#define SW_NO 0`
- `#define SW_NC 1`

Enumerations

- `enum D_SW { D_SW_A = 0 , D_SW_L , D_SW_M , D_SW_COUNT }`

Functions

- `void init_all_switch (void)`
- `void button_press_check (void)`
- `void all_relays_off (void)`

Variables

- `const uint8_t sw_contact_types [D_SW_COUNT]`

6.12.1 Macro Definition Documentation

6.12.1.1 CHARGER_RELAY_OFF

```
#define CHARGER_RELAY_OFF CHARGER_SetLow()
```

6.12.1.2 CHARGER_RELAY_ON

```
#define CHARGER_RELAY_ON CHARGER_SetHigh()
```

6.12.1.3 D_SW_A_PIN

```
#define D_SW_A_PIN A_SWITCH_GetValue()
```

6.12.1.4 D_SW_L_PIN

```
#define D_SW_L_PIN L_SWITCH_GetValue()
```

6.12.1.5 D_SW_M_PIN

```
#define D_SW_M_PIN M_SWITCH_GetValue()
```

6.12.1.6 DAY_RELAY_OFF

```
#define DAY_RELAY_OFF RELAY_SetLow()
```

6.12.1.7 DAY_RELAY_ON

```
#define DAY_RELAY_ON RELAY_SetHigh()
```

6.12.1.8 debounce_time

```
#define debounce_time 300
```

6.12.1.9 NIGHT_RELAY_OFF

```
#define NIGHT_RELAY_OFF MISC_SetLow()
```

6.12.1.10 NIGHT_RELAY_ON

```
#define NIGHT_RELAY_ON MISC_SetHigh()
```

6.12.1.11 SW_NC

```
#define SW_NC 1
```

6.12.1.12 SW_NO

```
#define SW_NO 0
```

6.12.2 Enumeration Type Documentation

6.12.2.1 D_SW

```
enum D_SW
```

Enumerator

D_SW_A	
D_SW_L	
D_SW_M	
D_SW_COUNT	

6.12.3 Function Documentation

6.12.3.1 all_relays_off()

```
void all_relays_off (  
    void )
```

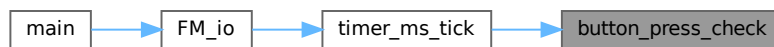
Here is the caller graph for this function:



6.12.3.2 button_press_check()

```
void button_press_check (  
    void )
```

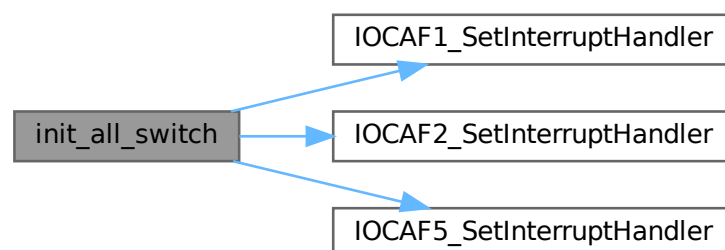
Here is the caller graph for this function:



6.12.3.3 init_all_switch()

```
void init_all_switch (  
    void )
```

Here is the call graph for this function:



Here is the caller graph for this function:



6.12.4 Variable Documentation

6.12.4.1 sw_contact_types

```
const uint8_t sw_contact_types[D_SW_COUNT]
```

Initial value:

```
= {
    SW_NO,
    SW_NO,
    SW_NC,
}
```

6.13 dio.h

[Go to the documentation of this file.](#)

```

00001 /*
00002  * File:   dio.h
00003  * Author: root
00004  *
00005  * Created on October 13, 2023, 9:29 AM
00006  */
00007
00008 #ifndef DIO_H
00009 #define DIO_H
00010
00011 #ifdef __cplusplus
00012 extern "C" {
00013 #endif
00014
00015 #include <xc.h>
00016 #include "mateQ84.X/mcc_generated_files/pin_manager.h"
00017 /*
00018  * MISC_IO header
00019  * pin 3, CHARGER
00020  * pin 2, MISC
00021  * pin 1, RELAY
00022  *
00023  * DIGITAL header
00024  *
00025  * pin 2, DB1
00026  * pin 1, DB0
00027  */
00028
00029 #define DAY_RELAY_ON      RELAY_SetHigh()
00030 #define DAY_RELAY_OFF     RELAY_SetLow()
00031 #define NIGHT_RELAY_ON   MISC_SetHigh()
00032 #define NIGHT_RELAY_OFF  MISC_SetLow()
00033 #define CHARGER_RELAY_ON CHARGER_SetHigh()
00034 #define CHARGER_RELAY_OFF CHARGER_SetLow()
00035
00036 /*
00037  * assign a physical pin for each switch
00038  * ANALOG header
00039  * A: pin 5
  
```

```

00040      * L: pin 2
00041      * M: pin 1
00042      */
00043 #define D_SW_A_PIN  A_SWITCH_GetValue()
00044 #define D_SW_L_PIN  L_SWITCH_GetValue()
00045 #define D_SW_M_PIN  M_SWITCH_GetValue()
00046
00047 #define debounce_time  300 // debounce counts, at least a few tens of milliseconds for a typical
switch
00048
00049 #define SW_NO  0    // switch, normally open
00050 #define SW_NC  1    // switch, normally closed
00051
00052 /*
00053  * switch inputs and flags, uses the IOC interrupt and the software
00054  * timing ISR for processing
00055  */
00056 enum D_SW {
00057     D_SW_A = 0,    // alternate LCD display
00058     D_SW_L,        // history logging from FM80
00059     D_SW_M,
00060     D_SW_COUNT // one extra for number of switches to check
00061 };
00062
00063 /*
00064  * contact type per configured switch
00065  * used to verify the de-bounced switch condition for setting the button flag
00066  */
00067 const uint8_t sw_contact_types[D_SW_COUNT] = {
00068     SW_NO,
00069     SW_NO,
00070     SW_NC,
00071 };
00072
00073 void init_all_switch(void); // setup data and interrupt functions for each switch
00074 void button_press_check(void); // for de-bounce timer ISR
00075
00076 void all_relays_off(void);
00077
00078 #ifdef __cplusplus
00079 }
00080 #endif
00081
00082 #endif /* DIO_H */
00083

```

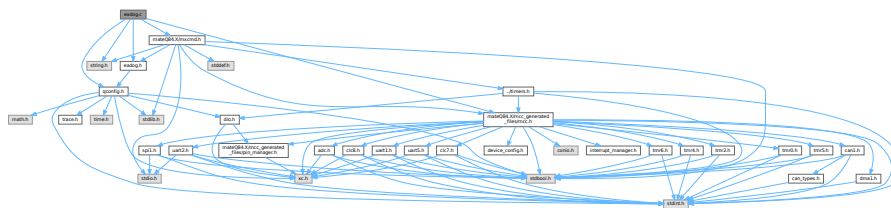
6.14 eadog.c File Reference

```

#include <string.h>
#include "qconfig.h"
#include "eadog.h"
#include "mateQ84.X/mcc_generated_files/mcc.h"
#include "mateQ84.X/mxcmd.h"

```

Include dependency graph for eadog.c:



Macros

- #define `E_TRACE IO_RB5_Toggle()`
- #define `DONE_DELAY 99999`

Functions

- bool [init_display](#) (void)
- void [eaDogM_WriteString](#) (char *strPtr)
- void [send_lcd_cmd_dma](#) (const uint8_t strPtr)
- void [send_lcd_data_dma](#) (const uint8_t strPtr)
- void [send_lcd_pos_dma](#) (const uint8_t strPtr)
- void [eaDogM_WriteStringAtPos](#) (const uint8_t r, const uint8_t c, char *strPtr)
- void [eaDogM_WriteIntAtPos](#) (const uint8_t r, const uint8_t c, const uint8_t i)
- void [eaDogM_SetPos](#) (const uint8_t r, const uint8_t c)
- void [eaDogM_ClearRow](#) (const uint8_t r)
- void [eaDogM_WriteByteToCGRAM](#) (const uint8_t ndx, const uint8_t data)
- void [eaDogM_WriteCommand](#) (const uint8_t cmd)
- void [eaDogM_WriteChr](#) (const int8_t value)
- void [start_lcd](#) (void)
- void [wait_lcd_set](#) (void)
- bool [wait_lcd_check](#) (void)
- void [wait_lcd_done](#) (void)
- void [clear_lcd_done](#) (void)
- void [spi_rec_done](#) (void)
- char * [eaDogM_Scroll_String](#) (char *strPtr)
- void [eaDogM_Scroll_Task](#) (void)
- void [no_dma_set_lcd](#) (void)
- void [set_lcd_dim](#) (const bool dim)
- void [check_lcd_dim](#) (const bool dim)

Variables

- volatile struct [spi_link_type](#) [spi_link](#)

6.14.1 Macro Definition Documentation

6.14.1.1 DONE_DELAY

```
#define DONE_DELAY 99999
```

6.14.1.2 E_TRACE

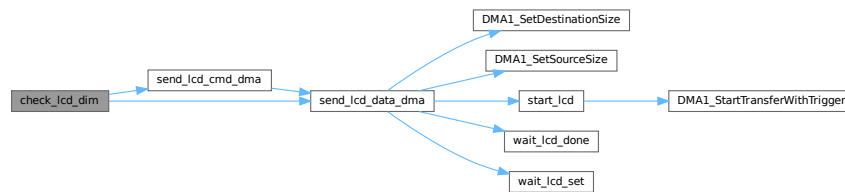
```
#define E_TRACE IO_RB5_Toggle()
```


6.14.2 Function Documentation

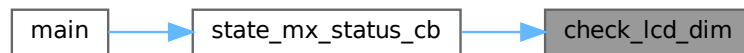
6.14.2.1 check_lcd_dim()

```
void check_lcd_dim (
    const bool dim)
```

Here is the call graph for this function:



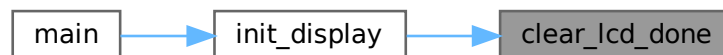
Here is the caller graph for this function:



6.14.2.2 clear_lcd_done()

```
void clear_lcd_done (
    void )
```

Here is the caller graph for this function:



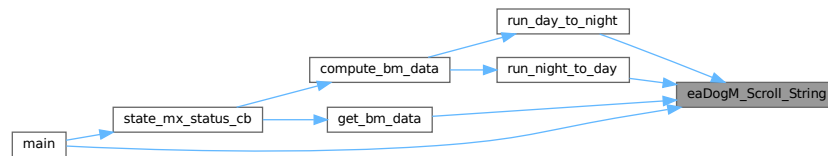
6.14.2.3 eaDogM_ClearRow()

```
void eaDogM_ClearRow (
    const uint8_t r)
```

6.14.2.4 eaDogM_Scroll_String()

```
char * eaDogM_Scroll_String (
    char * strPtr)
```

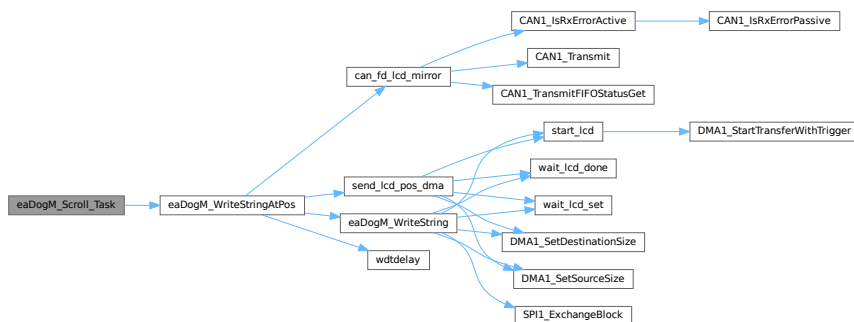
Here is the caller graph for this function:



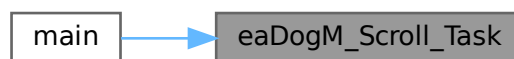
6.14.2.5 eaDogM_Scroll_Task()

```
void eaDogM_Scroll_Task (
    void )
```

Here is the call graph for this function:



Here is the caller graph for this function:



6.14.2.6 eaDogM_SetPos()

```
void eaDogM_SetPos (
    const uint8_t r,
    const uint8_t c)
```

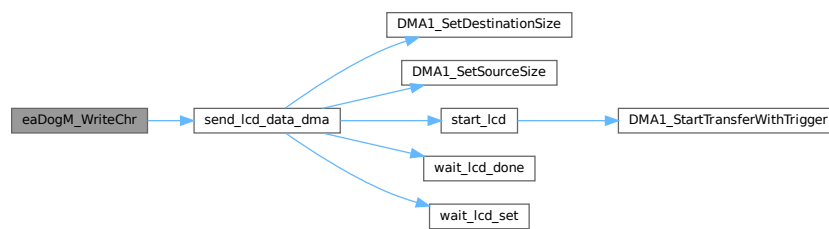
6.14.2.7 eaDogM_WriteByteToCGRAM()

```
void eaDogM_WriteByteToCGRAM (
    const uint8_t ndx,
    const uint8_t data)
```

6.14.2.8 eaDogM_WriteChr()

```
void eaDogM_WriteChr (
    const int8_t value)
```

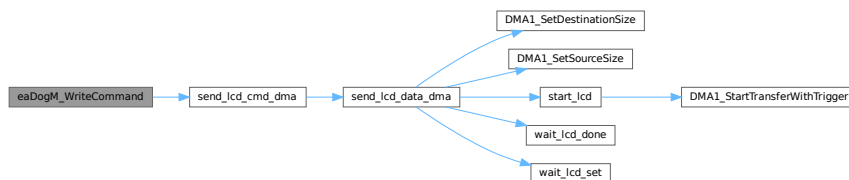
Here is the call graph for this function:



6.14.2.9 eaDogM_WriteCommand()

```
void eaDogM_WriteCommand (
    const uint8_t cmd)
```

Here is the call graph for this function:



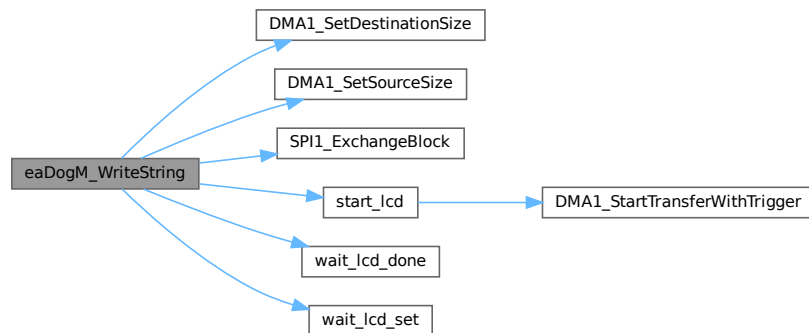
6.14.2.10 eaDogM_WriteIntAtPos()

```
void eaDogM_WriteIntAtPos (
    const uint8_t r,
    const uint8_t c,
    const uint8_t i)
```

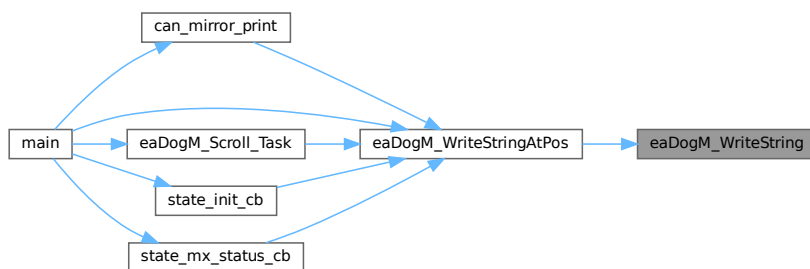
6.14.2.11 eaDogM_WriteString()

```
void eaDogM_WriteString (
    char * strPtr)
```

Here is the call graph for this function:



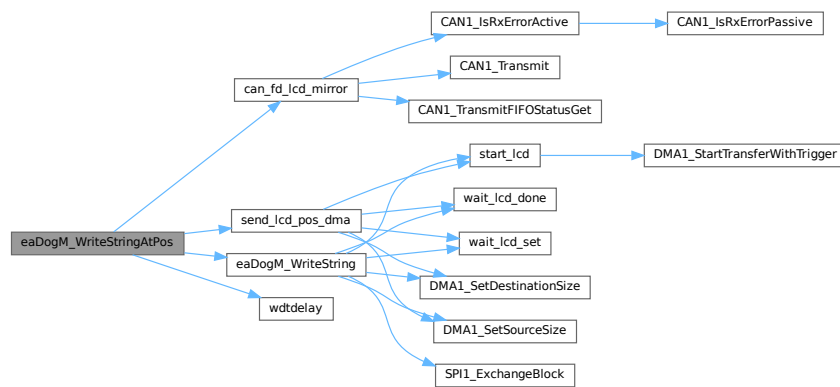
Here is the caller graph for this function:



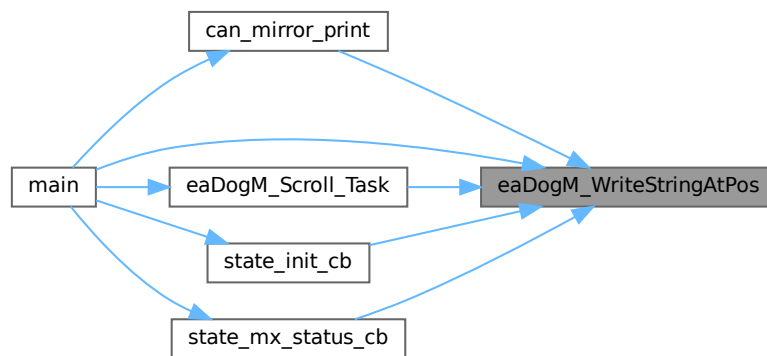
6.14.2.12 eaDogM_WriteStringAtPos()

```
void eaDogM_WriteStringAtPos (
    const uint8_t r,
    const uint8_t c,
    char * strPtr)
```

Here is the call graph for this function:



Here is the caller graph for this function:

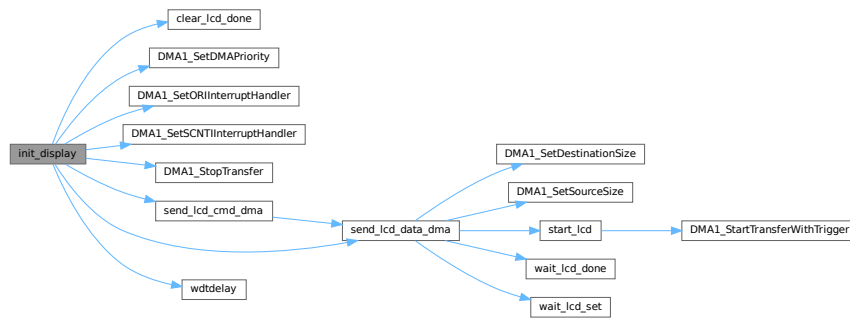


6.14.2.13 init_display()

```

bool init_display (
    void )
  
```

Here is the call graph for this function:



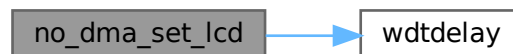
Here is the caller graph for this function:



6.14.2.14 no_dma_set_lcd()

```
void no_dma_set_lcd (
    void )
```

Here is the call graph for this function:



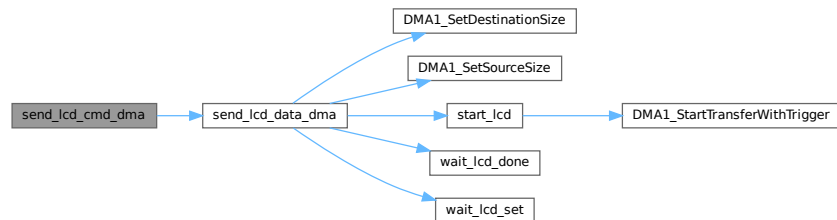
Here is the caller graph for this function:



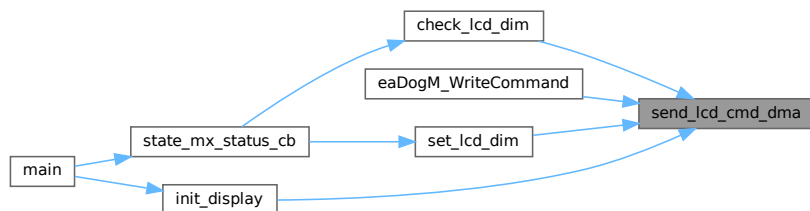
6.14.2.15 send_lcd_cmd_dma()

```
void send_lcd_cmd_dma (
    const uint8_t strPtr)
```

Here is the call graph for this function:



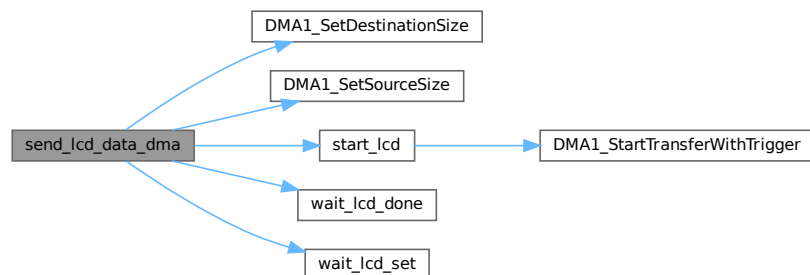
Here is the caller graph for this function:



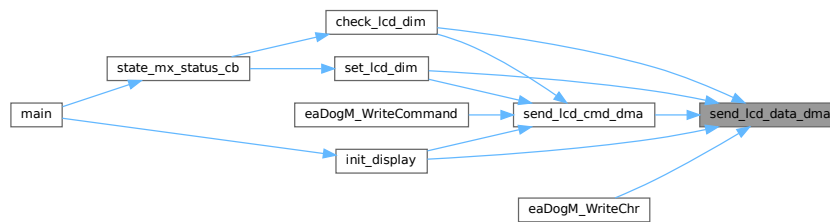
6.14.2.16 send_lcd_data_dma()

```
void send_lcd_data_dma (
    const uint8_t strPtr)
```

Here is the call graph for this function:



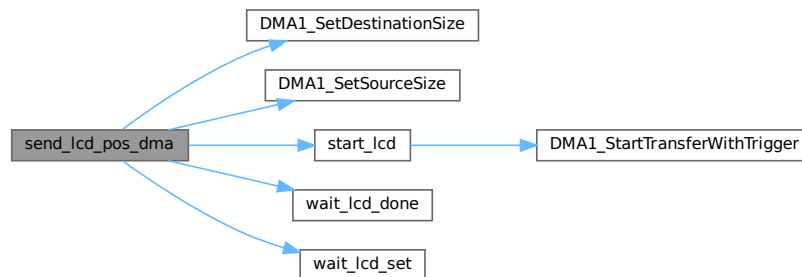
Here is the caller graph for this function:



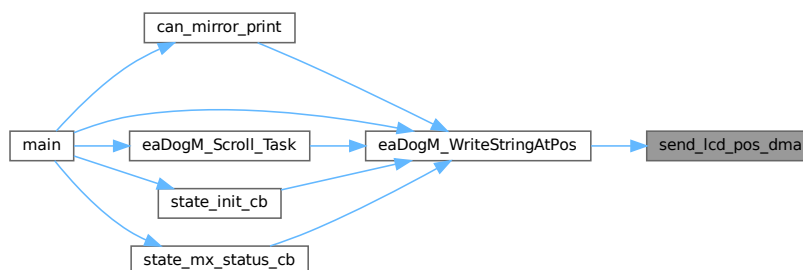
6.14.2.17 send_lcd_pos_dma()

```
void send_lcd_pos_dma (
    const uint8_t strPtr)
```

Here is the call graph for this function:



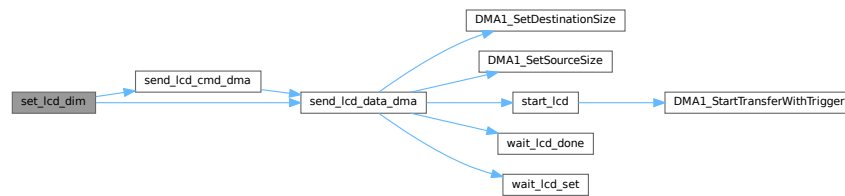
Here is the caller graph for this function:



6.14.2.18 set_lcd_dim()

```
void set_lcd_dim (
    const bool dim)
```

Here is the call graph for this function:



Here is the caller graph for this function:



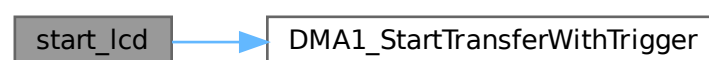
6.14.2.19 spi_rec_done()

```
void spi_rec_done (
    void )
```

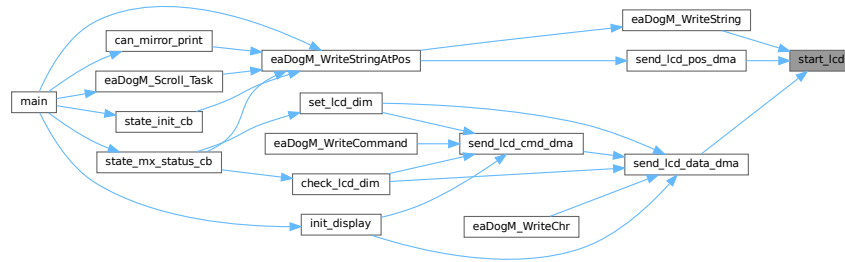
6.14.2.20 start_lcd()

```
void start_lcd (
    void )
```

Here is the call graph for this function:



Here is the caller graph for this function:



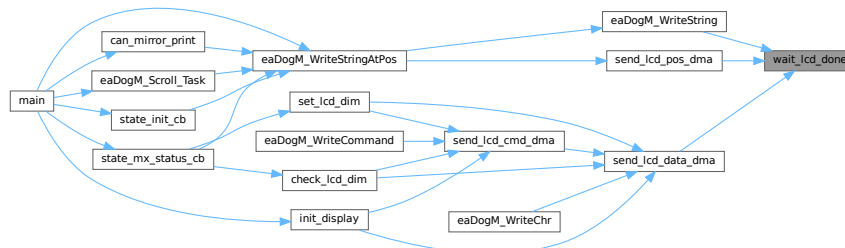
6.14.2.21 wait_lcd_check()

```
bool wait_lcd_check (
    void )
```

6.14.2.22 wait_lcd_done()

```
void wait_lcd_done (
    void )
```

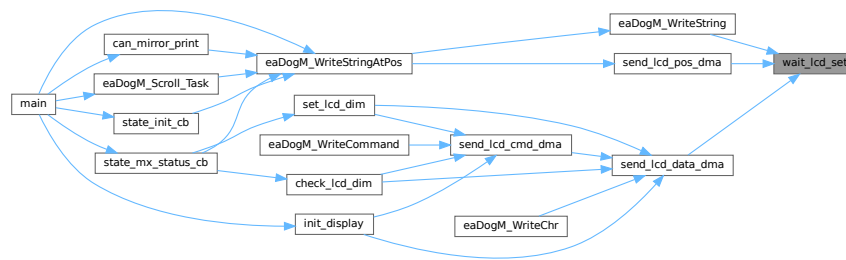
Here is the caller graph for this function:



6.14.2.23 wait_lcd_set()

```
void wait_lcd_set (
    void )
```

Here is the caller graph for this function:



6.14.3 Variable Documentation

6.14.3.1 spi_link

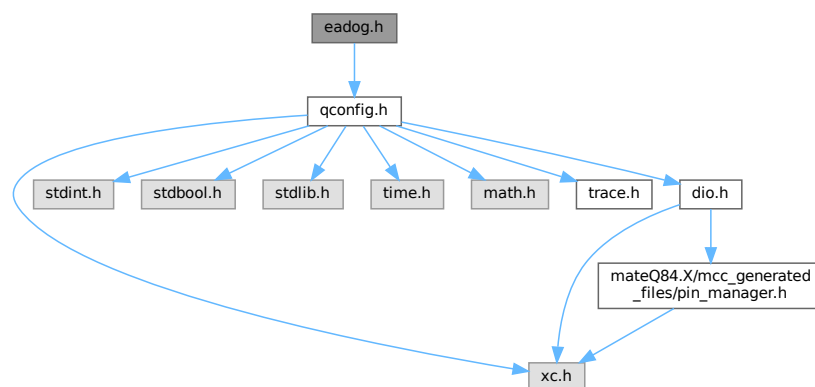
```
volatile struct spi_link_type spi_link
```

Initial value:

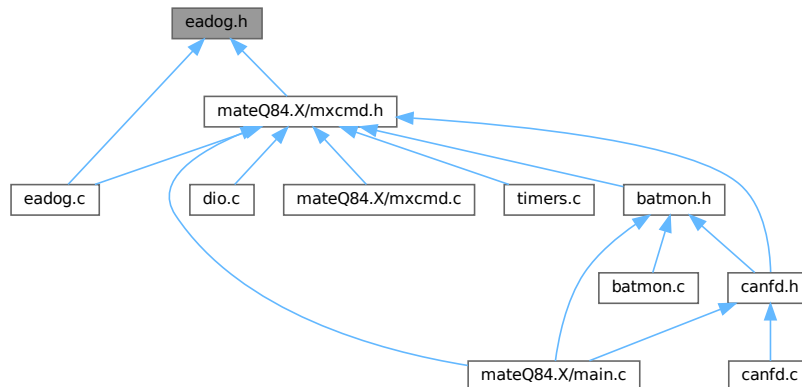
```
= {
    .LCD_DATA = false,
}
```

6.15 eadog.h File Reference

```
#include "qconfig.h"
Include dependency graph for eadog.h:
```



This graph shows which files directly or indirectly include this file:



Data Structures

- struct [spi1_configuration_t](#)

Macros

- #define [USEMCC_SPI](#)
- #define [USE_LCD_DMA](#)
- #define [EADOGM_CMD_CLR](#) 1
- #define [EADOGM_CMD_CURSOR_ON](#) 0b00001111
- #define [EADOGM_CMD_CURSOR_OFF](#) 0b00001100
- #define [EADOGM_CMD_DISPLAY_ON](#) 0b00001100
- #define [EADOGM_CMD_DISPLAY_OFF](#) 0b00001000
- #define [EADOGM_CMD_DDRAM_ADDR](#) 0b10000000
- #define [EADOGM_CMD_CGRAM_ADDR](#) 0b01000000
- #define [EADOGM_CMD_SELECT_R0](#) 0b00011000
- #define [EADOGM_CMD_SELECT_R1](#) 0b00010000
- #define [EADOGM_CMD_SET_TABLE2](#) 0b00101010
- #define [EADOGM_COLSPAN](#) 16
- #define [LCD_CMD_MASK](#) 0x01
- #define [LCD_CMD_SET](#) 0x100
- #define [LCD_CLEAR_HOME](#) 0x04
- #define [LCD_CMD_ON](#) 0x41
- #define [LCD_CMD_OFF](#) 0x42
- #define [LCD_CMD_HOME](#) 0x46
- #define [LCD_CMD_CLR](#) 0x51
- #define [LCD_CMD_CONT](#) 0x52
- #define [LCD_CMD_BRI](#) 0x53
- #define [NHD_CMD](#) 0xFE
- #define [NHD_POS](#) 0x45
- #define [NHD_BL_OFF](#) 1
- #define [NHD_BL_LOW](#) 2
- #define [NHD_BL_MED](#) 5
- #define [NHD_BL_HIGH](#) 8

- `#define NHD_CONT 50`
- `#define NHD_T_DELAY 8`
- `#define NHD_S_DELAY 200`
- `#define NHD_L_DELAY 800`
- `#define LCD0 0`
- `#define LCD1 1`
- `#define LCD2 2`
- `#define LCD3 3`
- `#define DIM_DELAY 6`
- `#define NSB 5`
- `#define LSB 21`
- `#define LCD_PWR_DELAY 350000`
- `#define eaDogM_Cls()`
- `#define eaDogM_CursorOn()`
- `#define eaDogM_CursorOff()`
- `#define eaDogM_DisplayOn()`
- `#define eaDogM_DisplayOff()`
- `#define max_strlen 20`

Functions

- void `wtdelay` (const uint32_t)
- bool `init_display` (void)
- void `no_dma_set_lcd` (void)
- void `send_lcd_data_dma` (const uint8_t)
- void `send_lcd_cmd_dma` (const uint8_t)
- void `send_lcd_pos_dma` (const uint8_t)
- void `start_lcd` (void)
- void `wait_lcd_set` (void)
- bool `wait_lcd_check` (void)
- void `wait_lcd_done` (void)
- void `eaDogM_WriteChr` (const int8_t)
- void `eaDogM_WriteCommand` (const uint8_t)
- void `eaDogM_SetPos` (const uint8_t, const uint8_t)
- void `eaDogM_ClearRow` (const uint8_t)
- void `eaDogM_WriteString` (char *)
- void `eaDogM_WriteStringAtPos` (const uint8_t, const uint8_t, char *)
- void `eaDogM_WriteIntAtPos` (const uint8_t, const uint8_t, const uint8_t)
- void `eaDogM_WriteByteToCGRAM` (const uint8_t, const uint8_t)
- void `set_lcd_dim` (const bool)
- void `check_lcd_dim` (const bool)
- char * `eaDogM_Scroll_String` (char *)
- void `eaDogM_Scroll_Task` (void)
- void `clear_lcd_done` (void)
- void `spi_rec_done` (void)
- void `can_fd_lcd_mirror` (const uint8_t, char *)

6.15.1 Macro Definition Documentation

6.15.1.1 DIM_DELAY

```
#define DIM_DELAY 6
```

6.15.1.2 eaDogM_Cls

```
#define eaDogM_Cls()
```

Value:

```
eaDogM_WriteCommand(EADOGM_CMD_CLR)
```

6.15.1.3 EADOGM_CMD_CGRAM_ADDR

```
#define EADOGM_CMD_CGRAM_ADDR 0b01000000
```

6.15.1.4 EADOGM_CMD_CLR

```
#define EADOGM_CMD_CLR 1
```

6.15.1.5 EADOGM_CMD_CURSOR_OFF

```
#define EADOGM_CMD_CURSOR_OFF 0b00001100
```

6.15.1.6 EADOGM_CMD_CURSOR_ON

```
#define EADOGM_CMD_CURSOR_ON 0b00001111
```

6.15.1.7 EADOGM_CMD_DDRAM_ADDR

```
#define EADOGM_CMD_DDRAM_ADDR 0b10000000
```

6.15.1.8 EADOGM_CMD_DISPLAY_OFF

```
#define EADOGM_CMD_DISPLAY_OFF 0b00001000
```

6.15.1.9 EADOGM_CMD_DISPLAY_ON

```
#define EADOGM_CMD_DISPLAY_ON 0b00001100
```

6.15.1.10 EADOGM_CMD_SELECT_R0

```
#define EADOGM_CMD_SELECT_R0 0b00011000
```

6.15.1.11 EADOGM_CMD_SELECT_R1

```
#define EADOGM_CMD_SELECT_R1 0b00010000
```

6.15.1.12 EADOGM_CMD_SET_TABLE2

```
#define EADOGM_CMD_SET_TABLE2 0b00101010
```

6.15.1.13 EADOGM_COLSPAN

```
#define EADOGM_COLSPAN 16
```

6.15.1.14 eaDogM_CursorOff

```
#define eaDogM_CursorOff()
```

Value:

```
eaDogM_WriteCommand(EADOGM_CMD_CURSOR_OFF)
```

6.15.1.15 eaDogM_CursorOn

```
#define eaDogM_CursorOn()
```

Value:

```
eaDogM_WriteCommand(EADOGM_CMD_CURSOR_ON)
```

6.15.1.16 eaDogM_DisplayOff

```
#define eaDogM_DisplayOff()
```

Value:

```
eaDogM_WriteCommand(EADOGM_CMD_DISPLAY_OFF)
```

6.15.1.17 eaDogM_DisplayOn

```
#define eaDogM_DisplayOn()
```

Value:

```
eaDogM_WriteCommand(EADOGM_CMD_DISPLAY_ON)
```

6.15.1.18 LCD0

```
#define LCD0 0
```

6.15.1.19 LCD1

```
#define LCD1 1
```

6.15.1.20 LCD2

```
#define LCD2 2
```

6.15.1.21 LCD3

```
#define LCD3 3
```

6.15.1.22 LCD_CLEAR_HOME

```
#define LCD_CLEAR_HOME 0x04
```

6.15.1.23 LCD_CMD_BRI

```
#define LCD_CMD_BRI 0x53
```

6.15.1.24 LCD_CMD_CLR

```
#define LCD_CMD_CLR 0x51
```

6.15.1.25 LCD_CMD_CONT

```
#define LCD_CMD_CONT 0x52
```

6.15.1.26 LCD_CMD_HOME

```
#define LCD_CMD_HOME 0x46
```

6.15.1.27 LCD_CMD_MASK

```
#define LCD_CMD_MASK 0x01
```

6.15.1.28 LCD_CMD_OFF

```
#define LCD_CMD_OFF 0x42
```

6.15.1.29 LCD_CMD_ON

```
#define LCD_CMD_ON 0x41
```


6.15.1.30 LCD_CMD_SET

```
#define LCD_CMD_SET 0x100
```

6.15.1.31 LCD_PWR_DELAY

```
#define LCD_PWR_DELAY 350000
```

6.15.1.32 LSB

```
#define LSB 21
```

6.15.1.33 max_strlen

```
#define max_strlen 20
```

6.15.1.34 NHD_BL_HIGH

```
#define NHD_BL_HIGH 8
```

6.15.1.35 NHD_BL_LOW

```
#define NHD_BL_LOW 2
```

6.15.1.36 NHD_BL_MED

```
#define NHD_BL_MED 5
```

6.15.1.37 NHD_BL_OFF

```
#define NHD_BL_OFF 1
```

6.15.1.38 NHD_CMD

```
#define NHD_CMD 0xFE
```

6.15.1.39 NHD_CONT

```
#define NHD_CONT 50
```

6.15.1.40 NHD_L_DELAY

```
#define NHD_L_DELAY 800
```

6.15.1.41 NHD_POS

```
#define NHD_POS 0x45
```

6.15.1.42 NHD_S_DELAY

```
#define NHD_S_DELAY 200
```

6.15.1.43 NHD_T_DELAY

```
#define NHD_T_DELAY 8
```

6.15.1.44 NSB

```
#define NSB 5
```

6.15.1.45 USE_LCD_DMA

```
#define USE_LCD_DMA
```

6.15.1.46 USEMCC_SPI

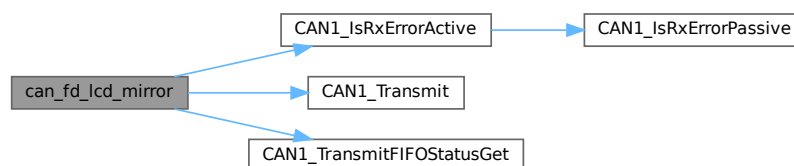
```
#define USEMCC_SPI
```

6.15.2 Function Documentation

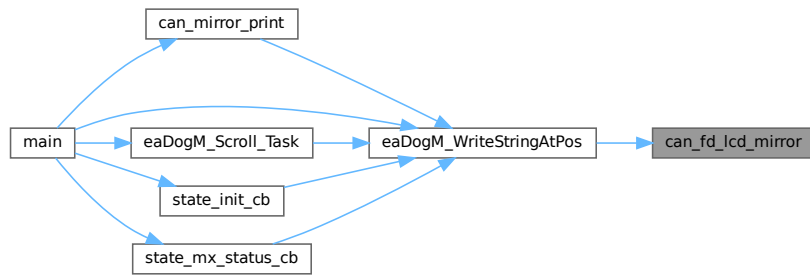
6.15.2.1 can_fd_lcd_mirror()

```
void can_fd_lcd_mirror (  
    const uint8_t r,  
    char * strPtr) [extern]
```

Here is the call graph for this function:



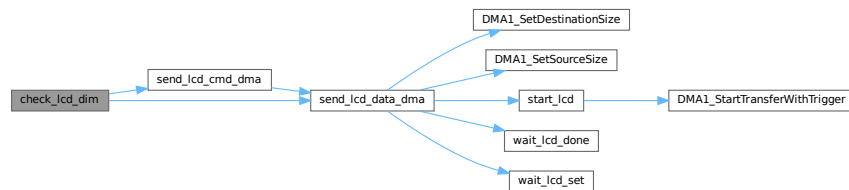
Here is the caller graph for this function:



6.15.2.2 check_lcd_dim()

```
void check_lcd_dim (
    const bool dim)
```

Here is the call graph for this function:



Here is the caller graph for this function:



6.15.2.3 clear_lcd_done()

```
void clear_lcd_done (
    void )
```

Here is the caller graph for this function:



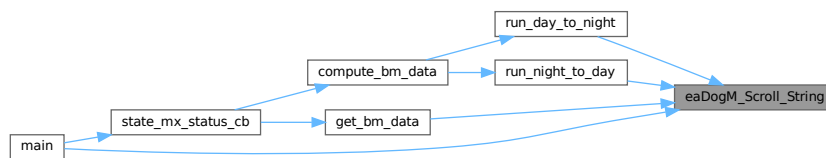
6.15.2.4 eaDogM_ClearRow()

```
void eaDogM_ClearRow (
    const uint8_t r)
```

6.15.2.5 eaDogM_Scroll_String()

```
char * eaDogM_Scroll_String (
    char * strPtr)
```

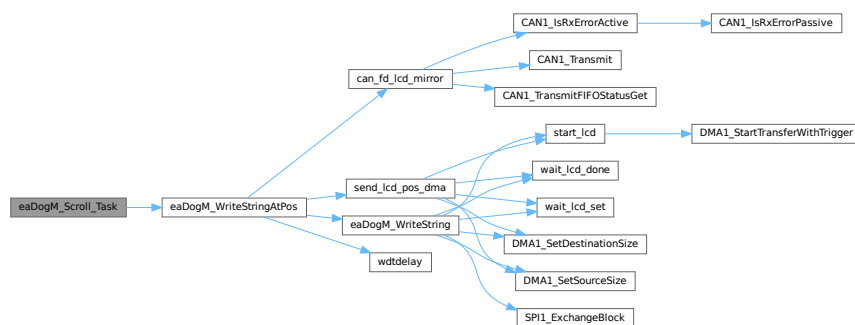
Here is the caller graph for this function:



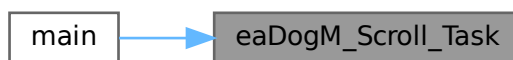
6.15.2.6 eaDogM_Scroll_Task()

```
void eaDogM_Scroll_Task (
    void )
```

Here is the call graph for this function:



Here is the caller graph for this function:



6.15.2.7 eaDogM_SetPos()

```
void eaDogM_SetPos (  
    const uint8_t r,  
    const uint8_t c)
```

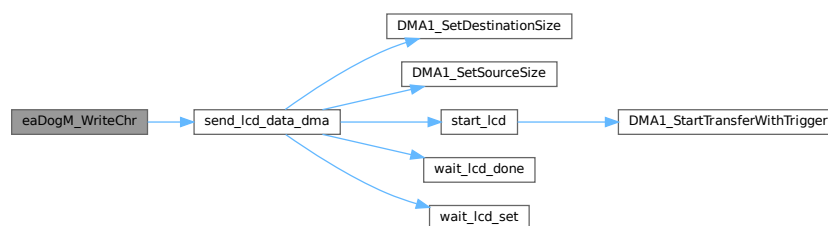
6.15.2.8 eaDogM_WriteByteToCGRAM()

```
void eaDogM_WriteByteToCGRAM (  
    const uint8_t ndx,  
    const uint8_t data)
```

6.15.2.9 eaDogM_WriteChr()

```
void eaDogM_WriteChr (  
    const int8_t value)
```

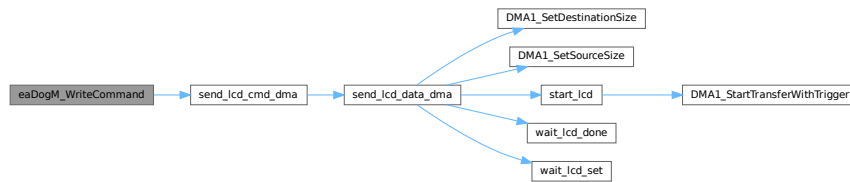
Here is the call graph for this function:



6.15.2.10 eaDogM_WriteCommand()

```
void eaDogM_WriteCommand (
    const uint8_t cmd)
```

Here is the call graph for this function:



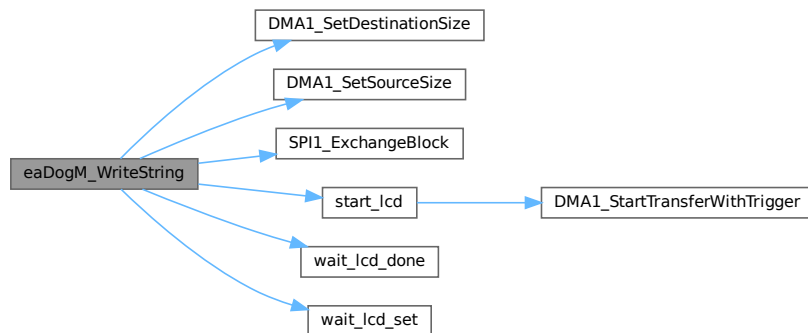
6.15.2.11 eaDogM_WriteIntAtPos()

```
void eaDogM_WriteIntAtPos (
    const uint8_t r,
    const uint8_t c,
    const uint8_t i)
```

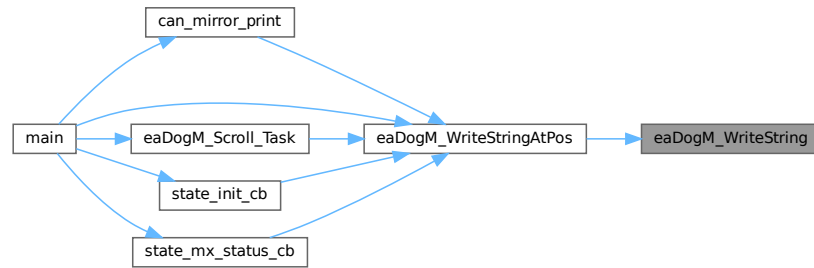
6.15.2.12 eaDogM_WriteString()

```
void eaDogM_WriteString (
    char * strPtr)
```

Here is the call graph for this function:



Here is the caller graph for this function:

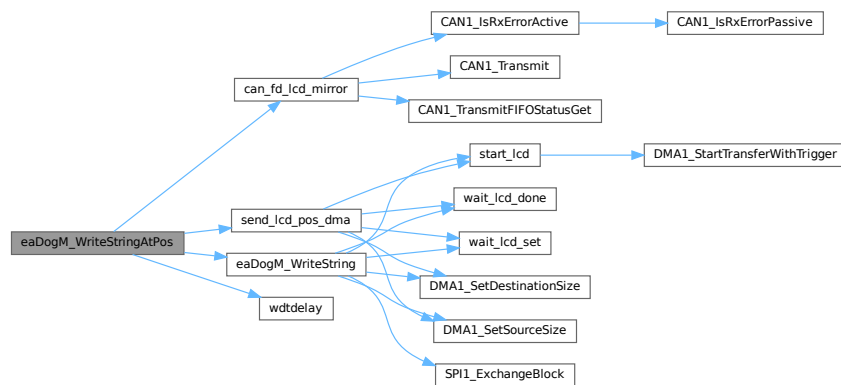


6.15.2.13 eadogM_WriteStringAtPos()

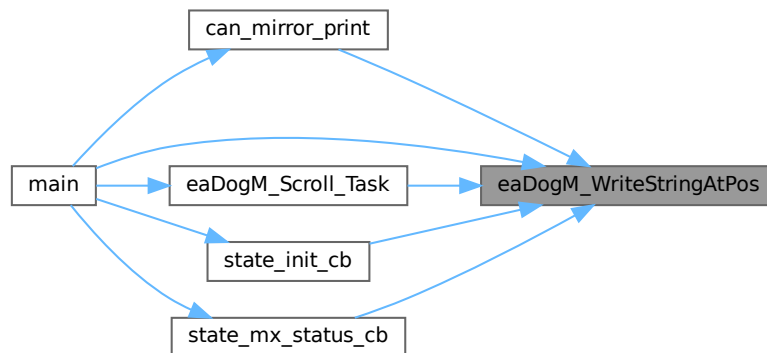
```

void eadogM_WriteStringAtPos (
    const uint8_t r,
    const uint8_t c,
    char * strPtr)
  
```

Here is the call graph for this function:



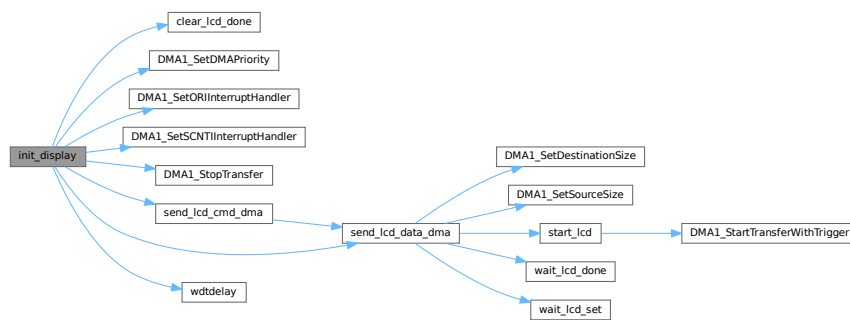
Here is the caller graph for this function:



6.15.2.14 init_display()

```
bool init_display (
    void )
```

Here is the call graph for this function:



Here is the caller graph for this function:



6.15.2.15 no_dma_set_lcd()

```
void no_dma_set_lcd (  
    void )
```

Here is the call graph for this function:



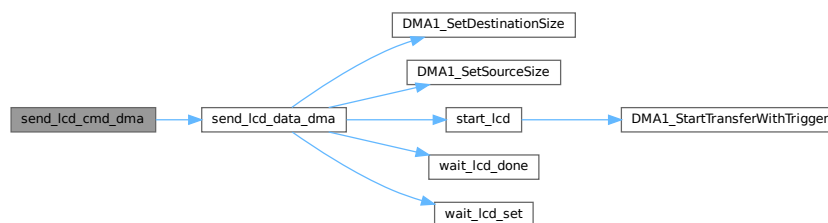
Here is the caller graph for this function:



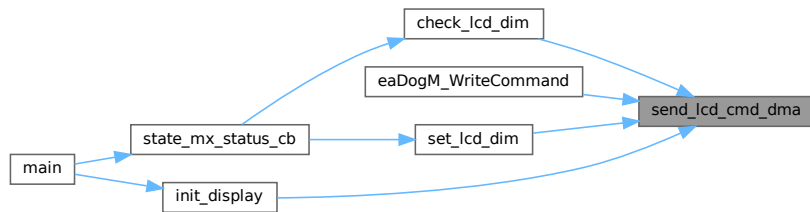
6.15.2.16 send_lcd_cmd_dma()

```
void send_lcd_cmd_dma (  
    const uint8_t strPtr)
```

Here is the call graph for this function:



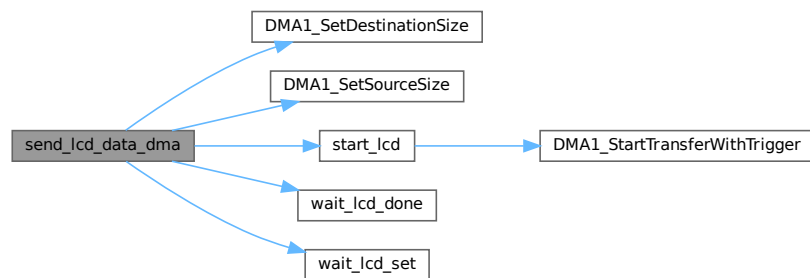
Here is the caller graph for this function:



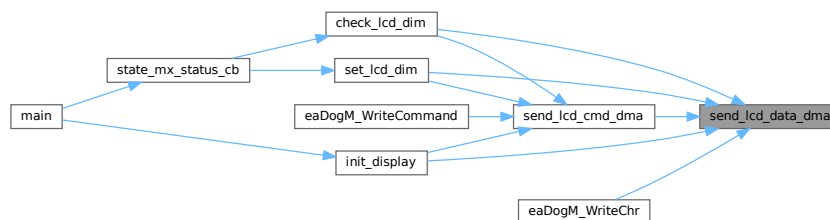
6.15.2.17 send_lcd_data_dma()

```
void send_lcd_data_dma (
    const uint8_t strPtr)
```

Here is the call graph for this function:



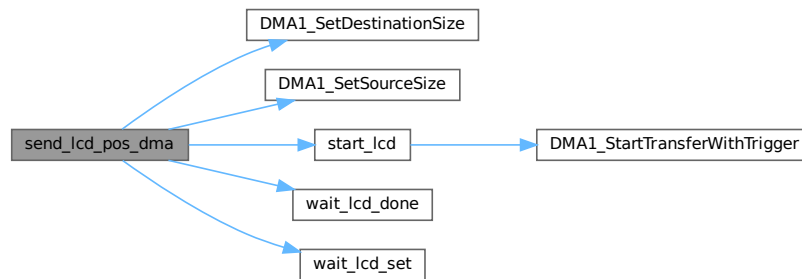
Here is the caller graph for this function:



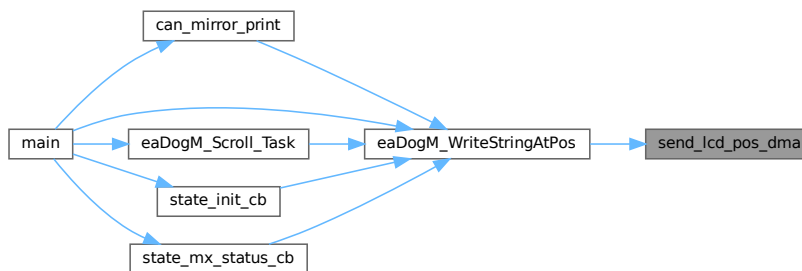
6.15.2.18 send_lcd_pos_dma()

```
void send_lcd_pos_dma (
    const uint8_t strPtr)
```

Here is the call graph for this function:



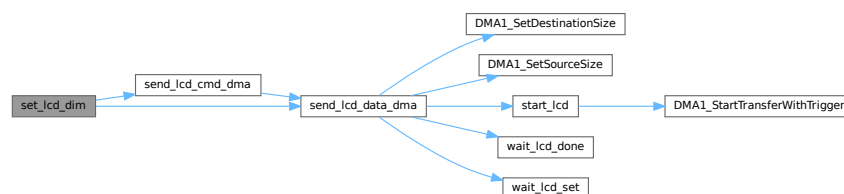
Here is the caller graph for this function:



6.15.2.19 set_lcd_dim()

```
void set_lcd_dim (
    const bool dim)
```

Here is the call graph for this function:



Here is the caller graph for this function:



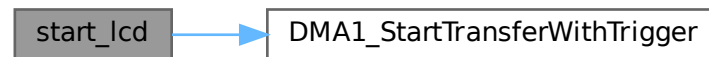
6.15.2.20 spi_rec_done()

```
void spi_rec_done (
    void )
```

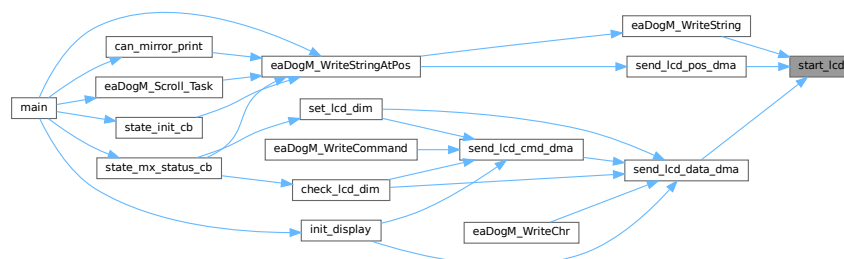
6.15.2.21 start_lcd()

```
void start_lcd (
    void )
```

Here is the call graph for this function:



Here is the caller graph for this function:



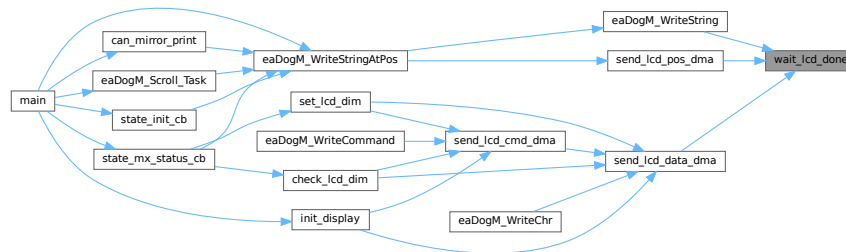
6.15.2.22 wait_lcd_check()

```
bool wait_lcd_check (
    void )
```

6.15.2.23 wait_lcd_done()

```
void wait_lcd_done (
    void )
```

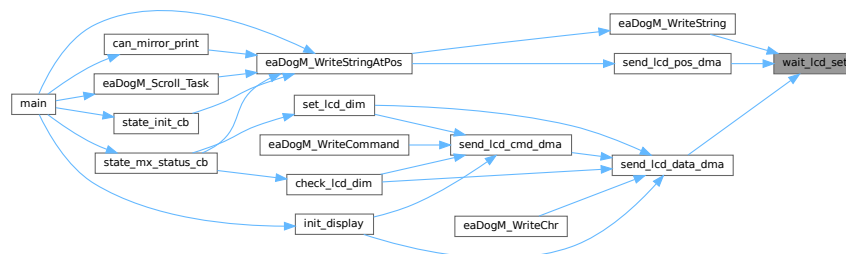
Here is the caller graph for this function:



6.15.2.24 wait_lcd_set()

```
void wait_lcd_set (
    void )
```

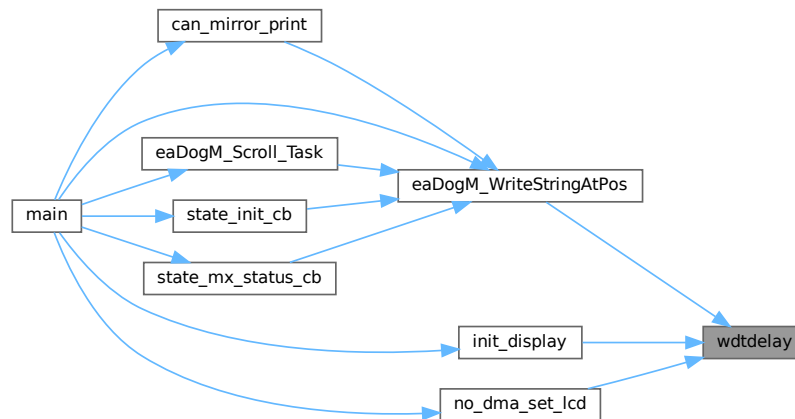
Here is the caller graph for this function:



6.15.2.25 wtdtdelay()

```
void wtdtdelay (
    const uint32_t delay) [extern]
```

Here is the caller graph for this function:



6.16 eadog.h

[Go to the documentation of this file.](#)

```

00001 /*
00002  * File:   eadog.h
00003  * Author: root
00004  *
00005  * Created on July 27, 2015, 2:05 PM
00006  */
00007
00008 /* Parts taken from:
00009  * Modified for XC8 and with SPI using ring buffers with interrupts
00010  *
00011  *      file: EA-DOGM_MIO.c
00012  *      version: 2.03
00013  *      description: Multi I/O driver for EA DOGM displays
00014  *      : Uses 8Bit, SPI HW or SPI SW (bitbang)
00015  *      written by : Michael Bradley (mbradley@mcu-labs.com)
00016  *      contributions: Imaginos (CCS forum), Emil Nad (8Bit testing)
00017  *      jgschmidt (CCS forum)
00018  */
00019
00020 #ifndef EADOG_H
00021 #define EADOG_H
00022
00023 #ifdef __cplusplus
00024 extern "C" {
00025 #endif
00026 #include "qconfig.h"
00027
00028 #define USEMCC_SPI
00029 #ifndef CAN_REMOTE_NODMA
00030 #define USE_LCD_DMA
00031 #endif
00032
00033 typedef struct {
00034     uint8_t con0;
00035     uint8_t con1;
00036     uint8_t con2;
00037     uint8_t baud;
00038     uint8_t operation;
00039 } spil_configuration_t;
00040
00041 #define EADOGM_CMD_CLR          1
00042 #define EADOGM_CMD_CURSOR_ON   0b00001111
00043 #define EADOGM_CMD_CURSOR_OFF  0b00001100
00044 #define EADOGM_CMD_DISPLAY_ON  0b00001100
00045 #define EADOGM_CMD_DISPLAY_OFF 0b00001000
00046 #define EADOGM_CMD_DDRAM_ADDR  0b10000000
00047 #define EADOGM_CMD_CGRAM_ADDR  0b01000000

```

```

00048 #define EADOGM_CMD_SELECT_R0    0b00011000
00049 #define EADOGM_CMD_SELECT_R1    0b00010000
00050 #define EADOGM_CMD_SET_TABLE2    0b00101010
00051 #define EADOGM_COLSPAN           16
00052
00053 #define LCD_CMD_MASK              0x01
00054 #define LCD_CMD_SET               0x100
00055 #define LCD_CLEAR_HOME           0x04
00056 #define LCD_CMD_ON               0x41
00057 #define LCD_CMD_OFF              0x42
00058 #define LCD_CMD_HOME             0x46
00059 #define LCD_CMD_CLR              0x51
00060 #define LCD_CMD_CONT             0x52
00061 #define LCD_CMD_BRI              0x53
00062 #define NHD_CMD                  0xFE
00063 #define NHD_POS                   0x45
00064 #define NHD_BL_OFF               1
00065 #define NHD_BL_LOW               2
00066 #define NHD_BL_MED               5
00067 #define NHD_BL_HIGH              8
00068 #define NHD_CONT                  50
00069
00070 #define NHD_T_DELAY              8
00071 #define NHD_S_DELAY              200
00072 #define NHD_L_DELAY              800
00073
00074 #define LCD0                      0
00075 #define LCD1                      1
00076 #define LCD2                      2
00077 #define LCD3                      3
00078
00079 #define DIM_DELAY                6
00080
00081 #define NSB                       5
00082 #define LSB                       21
00083
00084 #define LCD_PWR_DELAY             350000
00085
00086     extern void wdtdelay(const uint32_t);
00087     bool init_display(void);
00088     void no_dma_set_lcd(void);
00089     void send_lcd_data_dma(const uint8_t);
00090     void send_lcd_cmd_dma(const uint8_t);
00091     void send_lcd_pos_dma(const uint8_t);
00092     void start_lcd(void);
00093     void wait_lcd_set(void);
00094     bool wait_lcd_check(void);
00095     void wait_lcd_done(void);
00096     void eaDogM_WriteChr(const int8_t);
00097     void eaDogM_WriteCommand(const uint8_t);
00098     void eaDogM_SetPos(const uint8_t, const uint8_t);
00099     void eaDogM_ClearRow(const uint8_t);
00100     void eaDogM_WriteString(char *);
00101     void eaDogM_WriteStringAtPos(const uint8_t, const uint8_t, char *);
00102     void eaDogM_WriteIntAtPos(const uint8_t, const uint8_t, const uint8_t);
00103     void eaDogM_WriteByteToCGRAM(const uint8_t, const uint8_t);
00104     void set_lcd_dim(const bool);
00105     void check_lcd_dim(const bool);
00106
00107     char * eaDogM_Scroll_String(char *);
00108     void eaDogM_Scroll_Task(void);
00109
00110     // DMA complete flag
00111     void clear_lcd_done(void);
00112     void spi_rec_done(void);
00113     extern void can_fd_lcd_mirror(const uint8_t, char *);
00114
00115 #define eaDogM_Cls()               eaDogM_WriteCommand(EADOGM_CMD_CLR)
00116 #define eaDogM_CursorOn()          eaDogM_WriteCommand(EADOGM_CMD_CURSOR_ON)
00117 #define eaDogM_CursorOff()         eaDogM_WriteCommand(EADOGM_CMD_CURSOR_OFF)
00118 #define eaDogM_DisplayOn()         eaDogM_WriteCommand(EADOGM_CMD_DISPLAY_ON)
00119 #define eaDogM_DisplayOff()        eaDogM_WriteCommand(EADOGM_CMD_DISPLAY_OFF)
00120
00121     /*
00122     * characters per line on the display
00123     */
00124 #define max_strlen 20
00125
00126 #ifdef __cplusplus
00127 }
00128 #endif
00129
00130 #endif /* EADOG_H */
00131

```

6.17 linux/matesocketcan/.dep.inc File Reference

6.18 linux/matesocketcan/build/Debug/GNU-Linux/_↵ ext/5c0/raw.githubusercontent.com_linux-can_can-utils_master_↵ canfdtest.o.d File Reference

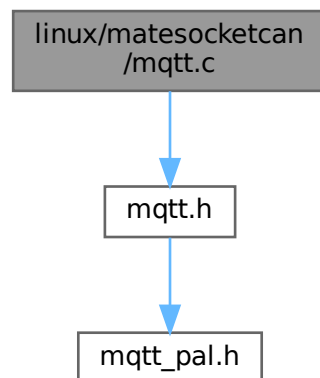
6.19 linux/matesocketcan/build/Release/GNU-Linux/_↵ ext/5c0/raw.githubusercontent.com_linux-can_can-utils_master_↵ canfdtest.o.d File Reference

6.20 linux/matesocketcan/mqtt.c File Reference

Implements the functionality of MQTT-C.

```
#include "mqtt.h"
```

Include dependency graph for mqtt.c:



6.20.1 Detailed Description

Implements the functionality of MQTT-C.

Note

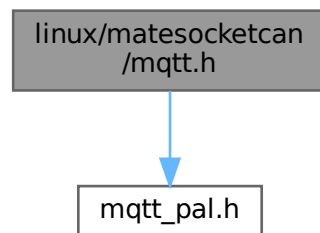
The only files that are included are [mqtt.h](#) and [mqtt_pal.h](#).

6.21 linux/matesocketcan/mqtt.h File Reference

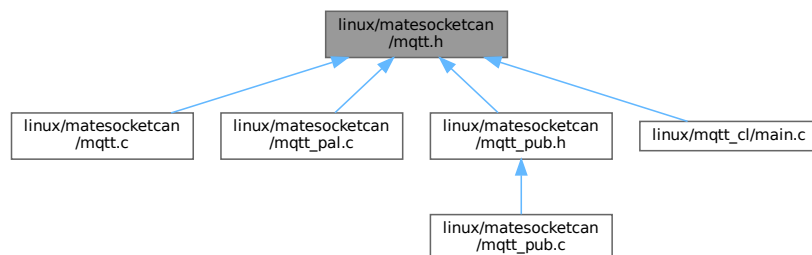
Declares all the MQTT-C functions and datastructures.

```
#include "mqtt_pal.h"
```

Include dependency graph for mqtt.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct [mqtt_fixed_header](#)
The fixed header of an MQTT control packet.
- struct [mqtt_response_connack](#)
A connection response datastructure.
- struct [mqtt_response_publish](#)
A publish packet received from the broker.
- struct [mqtt_response_puback](#)
A publish acknowledgement for messages that were published with QoS level 1.
- struct [mqtt_response_pubrec](#)
The response packet to a PUBLISH packet with QoS level 2.
- struct [mqtt_response_pubrel](#)
The response to a PUBREC packet.
- struct [mqtt_response_pubcomp](#)

- The response to a PUBREL packet.*

 - struct [mqtt_response_suback](#)
- The response to a subscription request.*

 - struct [mqtt_response_unsuback](#)
- The brokers response to a UNSUBSCRIBE request.*

 - struct [mqtt_response_pingresp](#)
- The response to a ping request.*

 - struct [mqtt_response](#)
- A struct used to deserialize/interpret an incoming packet from the broker.*

 - struct [mqtt_queued_message](#)
- A message in a [mqtt_message_queue](#).*

 - struct [mqtt_message_queue](#)
- A message queue.*

 - struct [mqtt_client](#)
- An MQTT client.*

Macros

- #define [MQTT_PROTOCOL_LEVEL](#) 0x04
- The protocol identifier for MQTT v3.1.1.*
- #define [__ALL_MQTT_ERRORS](#)(MQTT_ERROR)
- A macro used to declare the enum [MQTTErrors](#) and associated error messages (the members of the num) at the same time.*
- #define [GENERATE_ENUM](#)(ENUM)
- A macro used to generate the enum [MQTTErrors](#) from [__ALL_MQTT_ERRORS](#).*
- #define [GENERATE_STRING](#)(STRING)
- A macro used to generate the error messages associated with [MQTTErrors](#) from [__ALL_MQTT_ERRORS](#).*
- #define [__mqtt_packed_cstrlen](#)(x)
- A macro to get the MQTT string length from a c-string.*
- #define [MQTT_SUBSCRIBE_REQUEST_MAX_NUM_TOPICS](#) 8
- The maximum number topics that can be subscribed to in a single call to [mqtt_pack_subscribe_request](#).*
- #define [MQTT_UNSUBSCRIBE_REQUEST_MAX_NUM_TOPICS](#) 8
- The maximum number topics that can be subscribed to in a single call to [mqtt_pack_unsubscribe_request](#).*
- #define [mqtt_mq_get](#)(mq_ptr, index)
- Returns the [mqtt_queued_message](#) at *index*.*
- #define [mqtt_mq_length](#)(mq_ptr)
- Returns the number of messages in the message queue, *mq_ptr*.*
- #define [mqtt_mq_currSZ](#)(mq_ptr)
- Used internally to recalculate the *currSZ*.*

Enumerations

- enum [MQTTControlPacketType](#) {
[MQTT_CONTROL_CONNECT](#) = 1u , [MQTT_CONTROL_CONNACK](#) = 2u , [MQTT_CONTROL_PUBLISH](#) = 3u , [MQTT_CONTROL_PUBACK](#) = 4u ,
[MQTT_CONTROL_PUBREC](#) = 5u , [MQTT_CONTROL_PUBREL](#) = 6u , [MQTT_CONTROL_PUBCOMP](#) = 7u , [MQTT_CONTROL_SUBSCRIBE](#) = 8u ,
[MQTT_CONTROL_SUBACK](#) = 9u , [MQTT_CONTROL_UNSUBSCRIBE](#) = 10u , [MQTT_CONTROL_UNSUBACK](#) = 11u , [MQTT_CONTROL_PINGREQ](#) = 12u ,
[MQTT_CONTROL_PINGRESP](#) = 13u , [MQTT_CONTROL_DISCONNECT](#) = 14u }
- An enumeration of the MQTT control packet types.*

- enum [MQTTErrors](#) { [MQTT_ERROR_UNKNOWN](#) = INT_MIN , [MQTT_OK](#) = 1 }
An enumeration of error codes. Error messages can be retrieved by calling [mqtt_error_str](#).
- enum [MQTTConnackReturnCode](#) {
[MQTT_CONNACK_ACCEPTED](#) = 0u , [MQTT_CONNACK_REFUSED_PROTOCOL_VERSION](#) = 1u ,
[MQTT_CONNACK_REFUSED_IDENTIFIER_REJECTED](#) = 2u , [MQTT_CONNACK_REFUSED_SERVER_UNAVAILABLE](#)
= 3u ,
[MQTT_CONNACK_REFUSED_BAD_USER_NAME_OR_PASSWORD](#) = 4u , [MQTT_CONNACK_REFUSED_NOT_AUTHORIZED](#)
= 5u }
An enumeration of the return codes returned in a CONNACK packet.
- enum [MQTTSubackReturnCodes](#) { [MQTT_SUBACK_SUCCESS_MAX_QOS_0](#) = 0u , [MQTT_SUBACK_SUCCESS_MAX_QOS_1](#) = 1u , [MQTT_SUBACK_SUCCESS_MAX_QOS_2](#) = 2u , [MQTT_SUBACK_FAILURE](#) = 128u }
An enumeration of subscription acknowledgement return codes.
- enum [MQTTConnectFlags](#) {
[MQTT_CONNECT_RESERVED](#) = 1u , [MQTT_CONNECT_CLEAN_SESSION](#) = 2u , [MQTT_CONNECT_WILL_FLAG](#)
= 4u , [MQTT_CONNECT_WILL_QOS_0](#) = (0u & 0x03) << 3 ,
[MQTT_CONNECT_WILL_QOS_1](#) = (1u & 0x03) << 3 , [MQTT_CONNECT_WILL_QOS_2](#) = (2u & 0x03)
<< 3 , [MQTT_CONNECT_WILL_RETAIN](#) = 32u , [MQTT_CONNECT_PASSWORD](#) = 64u ,
[MQTT_CONNECT_USER_NAME](#) = 128u }
An enumeration of CONNECT packet flags.
- enum [MQTTPublishFlags](#) {
[MQTT_PUBLISH_DUP](#) = 8u , [MQTT_PUBLISH_QOS_0](#) = ((0u << 1) & 0x06) , [MQTT_PUBLISH_QOS_1](#) =
((1u << 1) & 0x06) , [MQTT_PUBLISH_QOS_2](#) = ((2u << 1) & 0x06) ,
[MQTT_PUBLISH_QOS_MASK](#) = ((3u << 1) & 0x06) , [MQTT_PUBLISH_RETAIN](#) = 0x01 }
An enumeration of the PUBLISH flags.
- enum [MQTTQueuedMessageState](#) { [MQTT_QUEUED_UNSENT](#) , [MQTT_QUEUED_AWAITING_ACK](#) ,
[MQTT_QUEUED_COMPLETE](#) }
An enumeration of queued message states.

Functions

- const char * [mqtt_error_str](#) (enum [MQTTErrors](#) error)
Returns an error message for error code, *error*.
- ssize_t [__mqtt_pack_uint16](#) (uint8_t *buf, uint16_t integer)
Pack a MQTT 16 bit integer, given a native 16 bit integer .
- uint16_t [__mqtt_unpack_uint16](#) (const uint8_t *buf)
Unpack a MQTT 16 bit integer to a native 16 bit integer.
- ssize_t [__mqtt_pack_str](#) (uint8_t *buf, const char *str)
Pack a MQTT string, given a c-string *str*.
- ssize_t [mqtt_unpack_fixed_header](#) (struct [mqtt_response](#) *response, const uint8_t *buf, size_t bufsz)
Deserialize the contents of *buf* into an [mqtt_fixed_header](#) object.
- ssize_t [mqtt_pack_fixed_header](#) (uint8_t *buf, size_t bufsz, const struct [mqtt_fixed_header](#) *fixed_header)
Serialize an [mqtt_fixed_header](#) and write it to *buf*.
- ssize_t [mqtt_pack_connection_request](#) (uint8_t *buf, size_t bufsz, const char *client_id, const char *will_topic, const void *will_message, size_t will_message_size, const char *user_name, const char *password, uint8_t connect_flags, uint16_t keep_alive)
Serialize a connection request into a buffer.
- ssize_t [mqtt_pack_publish_request](#) (uint8_t *buf, size_t bufsz, const char *topic_name, uint16_t packet_id, const void *application_message, size_t application_message_size, uint8_t publish_flags)
Serialize a PUBLISH request and put it in *buf*.
- ssize_t [mqtt_pack_pubxxx_request](#) (uint8_t *buf, size_t bufsz, enum [MQTTControlPacketType](#) control_type, uint16_t packet_id)
Serialize a PUBACK, PUBREC, PUBREL, or PUBCOMP packet and put it in *buf*.
- ssize_t [mqtt_pack_subscribe_request](#) (uint8_t *buf, size_t bufsz, unsigned int packet_id,...)

- Serialize a SUBSCRIBE packet and put it in buf.*

 - ssize_t [mqtt_pack_unsubscribe_request](#) (uint8_t *buf, size_t bufsz, unsigned int packet_id,...)
- Serialize a UNSUBSCRIBE packet and put it in buf.*

 - ssize_t [mqtt_pack_ping_request](#) (uint8_t *buf, size_t bufsz)
- Serialize a PINGREQ and put it into buf.*

 - ssize_t [mqtt_pack_disconnect](#) (uint8_t *buf, size_t bufsz)
- Serialize a DISCONNECT and put it into buf.*

 - uint16_t [__mqtt_next_pid](#) (struct [mqtt_client](#) *client)
- Generate a new next packet ID.*

 - ssize_t [__mqtt_send](#) (struct [mqtt_client](#) *client)
- Handles egress client traffic.*

 - ssize_t [__mqtt_rcv](#) (struct [mqtt_client](#) *client)
- Handles ingress client traffic.*

 - enum [MQTTErrors](#) [mqtt_sync](#) (struct [mqtt_client](#) *client)
- Function that does the actual sending and receiving of traffic from the network.*

 - enum [MQTTErrors](#) [mqtt_init](#) (struct [mqtt_client](#) *client, mqtt_pal_socket_handle [sockfd](#), uint8_t *sendbuf, size_t sendbufsz, uint8_t *recvbuf, size_t recvbufsz, void(*publish_response_callback)(void **state, struct [mqtt_response_publish](#) *publish))
- Initializes an MQTT client.*

 - void [mqtt_init_reconnect](#) (struct [mqtt_client](#) *client, void(*reconnect_callback)(struct [mqtt_client](#) *client, void **state), void *reconnect_state, void(*publish_response_callback)(void **state, struct [mqtt_response_publish](#) *publish))
- Initializes an MQTT client and enables automatic reconnections.*

 - void [mqtt_reinit](#) (struct [mqtt_client](#) *client, mqtt_pal_socket_handle sockfd, uint8_t *sendbuf, size_t sendbufsz, uint8_t *recvbuf, size_t recvbufsz)
- Safely assign/reassign a socket and buffers to an new/existing client.*

 - enum [MQTTErrors](#) [mqtt_connect](#) (struct [mqtt_client](#) *client, const char *client_id, const char *will_topic, const void *will_message, size_t will_message_size, const char *user_name, const char *password, uint8_t connect_flags, uint16_t keep_alive)
- Establishes a session with the MQTT broker.*

 - enum [MQTTErrors](#) [mqtt_publish](#) (struct [mqtt_client](#) *client, const char *topic_name, const void *application_message, size_t application_message_size, uint8_t publish_flags)
- Publish an application message.*

 - ssize_t [__mqtt_puback](#) (struct [mqtt_client](#) *client, uint16_t packet_id)
- Acknowledge an ingree publish with QOS==1.*

 - ssize_t [__mqtt_pubrec](#) (struct [mqtt_client](#) *client, uint16_t packet_id)
- Acknowledge an ingree publish with QOS==2.*

 - ssize_t [__mqtt_pubrel](#) (struct [mqtt_client](#) *client, uint16_t packet_id)
- Acknowledge an ingree PUBREC packet.*

 - ssize_t [__mqtt_pubcomp](#) (struct [mqtt_client](#) *client, uint16_t packet_id)
- Acknowledge an ingree PUBREL packet.*

 - enum [MQTTErrors](#) [mqtt_subscribe](#) (struct [mqtt_client](#) *client, const char *topic_name, int max_qos_level)
- Subscribe to a topic.*

 - enum [MQTTErrors](#) [mqtt_unsubscribe](#) (struct [mqtt_client](#) *client, const char *topic_name)
- Unsubscribe from a topic.*

 - enum [MQTTErrors](#) [mqtt_ping](#) (struct [mqtt_client](#) *client)
- Ping the broker.*

 - enum [MQTTErrors](#) [__mqtt_ping](#) (struct [mqtt_client](#) *client)
- Ping the broker without locking/unlocking the mutex.*

 - enum [MQTTErrors](#) [mqtt_disconnect](#) (struct [mqtt_client](#) *client)
- Terminate the session with the MQTT broker.*

 - enum [MQTTErrors](#) [mqtt_reconnect](#) (struct [mqtt_client](#) *client)
- Terminate the session with the MQTT broker and prepare to reconnect. Client code should call [mqtt_sync](#) immediately after this call to prevent message loss.*

6.21.1 Detailed Description

Declares all the MQTT-C functions and datastructures.

Note

You should `#include <mqtt.h>`.

6.21.2 Macro Definition Documentation

6.21.2.1 `__ALL_MQTT_ERRORS`

```
#define __ALL_MQTT_ERRORS (
    MQTT_ERROR)
```

Value:

```
MQTT_ERROR(MQTT_ERROR_NULLPTR)           \|
MQTT_ERROR(MQTT_ERROR_CONTROL_FORBIDDEN_TYPE) \|
MQTT_ERROR(MQTT_ERROR_CONTROL_INVALID_FLAGS) \|
MQTT_ERROR(MQTT_ERROR_CONTROL_WRONG_TYPE)   \|
MQTT_ERROR(MQTT_ERROR_CONNECT_CLIENT_ID_REFUSED) \|
MQTT_ERROR(MQTT_ERROR_CONNECT_NULL_WILL_MESSAGE) \|
MQTT_ERROR(MQTT_ERROR_CONNECT_FORBIDDEN_WILL_QOS) \|
MQTT_ERROR(MQTT_ERROR_CONNACK_FORBIDDEN_FLAGS) \|
MQTT_ERROR(MQTT_ERROR_CONNACK_FORBIDDEN_CODE) \|
MQTT_ERROR(MQTT_ERROR_PUBLISH_FORBIDDEN_QOS) \|
MQTT_ERROR(MQTT_ERROR_SUBSCRIBE_TOO_MANY_TOPICS) \|
MQTT_ERROR(MQTT_ERROR_MALFORMED_RESPONSE) \|
MQTT_ERROR(MQTT_ERROR_UNSUBSCRIBE_TOO_MANY_TOPICS) \|
MQTT_ERROR(MQTT_ERROR_RESPONSE_INVALID_CONTROL_TYPE) \|
MQTT_ERROR(MQTT_ERROR_CONNECT_NOT_CALLED) \|
MQTT_ERROR(MQTT_ERROR_SEND_BUFFER_IS_FULL) \|
MQTT_ERROR(MQTT_ERROR_SOCKET_ERROR) \|
MQTT_ERROR(MQTT_ERROR_MALFORMED_REQUEST) \|
MQTT_ERROR(MQTT_ERROR_RECV_BUFFER_TOO_SMALL) \|
MQTT_ERROR(MQTT_ERROR_ACK_OF_UNKNOWN) \|
MQTT_ERROR(MQTT_ERROR_NOT_IMPLEMENTED) \|
MQTT_ERROR(MQTT_ERROR_CONNECTION_REFUSED) \|
MQTT_ERROR(MQTT_ERROR_SUBSCRIBE_FAILED) \|
MQTT_ERROR(MQTT_ERROR_CONNECTION_CLOSED) \|
MQTT_ERROR(MQTT_ERROR_INITIAL_RECONNECT) \|
MQTT_ERROR(MQTT_ERROR_INVALID_REMAINING_LENGTH) \|
MQTT_ERROR(MQTT_ERROR_CLEAN_SESSION_IS_REQUIRED) \|
MQTT_ERROR(MQTT_ERROR_RECONNECT_FAILED) \|
MQTT_ERROR(MQTT_ERROR_RECONNECTING)
```

A macro used to declare the enum [MQTTErrors](#) and associated error messages (the members of the num) at the same time.

6.21.2.2 `__mqtt_packed_cstrlen`

```
#define __mqtt_packed_cstrlen(
    x)
```

Value:

```
(2 + (unsigned int)strlen(x))
```

A macro to get the MQTT string length from a c-string.

6.21.2.3 GENERATE_ENUM

```
#define GENERATE_ENUM(  
    ENUM)
```

Value:

ENUM,

A macro used to generate the enum [MQTTErrors](#) from [__ALL_MQTT_ERRORS](#).

See also

[__ALL_MQTT_ERRORS](#)

6.21.2.4 GENERATE_STRING

```
#define GENERATE_STRING(  
    STRING)
```

Value:

#STRING,

A macro used to generate the error messages associated with [MQTTErrors](#) from [__ALL_MQTT_ERRORS](#).

See also

[__ALL_MQTT_ERRORS](#)

6.21.3 Function Documentation

6.21.3.1 __mqtt_pack_str()

```
ssize_t __mqtt_pack_str (  
    uint8_t * buf,  
    const char * str)
```

Pack a MQTT string, given a c-string *str*.

Parameters

out	<i>buf</i>	the buffer that the MQTT string will be written to.
in	<i>str</i>	the c-string to be written to <i>buf</i> .

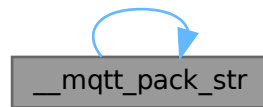
Warning

This function provides no error checking.

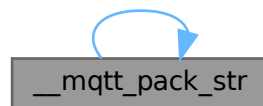
Returns

`strlen(str) + 2`

Here is the call graph for this function:



Here is the caller graph for this function:

**6.21.3.2 __mqtt_pack_uint16()**

```
ssize_t __mqtt_pack_uint16 (  
    uint8_t * buf,  
    uint16_t integer)
```

Pack a MQTT 16 bit integer, given a native 16 bit integer .

Parameters

out	<i>buf</i>	the buffer that the MQTT integer will be written to.
in	<i>integer</i>	the native integer to be written to <i>buf</i> .

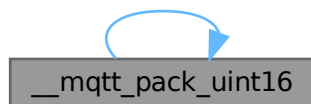
Warning

This function provides no error checking.

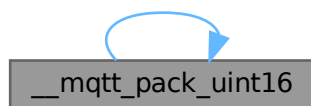
Returns

2

Here is the call graph for this function:



Here is the caller graph for this function:

**6.21.3.3 __mqtt_ping()**

```
enum MQTTErrors __mqtt_ping (  
    struct mqtt_client * client)
```

Ping the broker without locking/unlocking the mutex.

See also[mqtt_ping](#)**6.21.3.4 __mqtt_unpack_uint16()**

```
uint16_t __mqtt_unpack_uint16 (  
    const uint8_t * buf)
```

Unpack a MQTT 16 bit integer to a native 16 bit integer.

Parameters

in	buf	the buffer that the MQTT integer will be read from.
----	-----	---

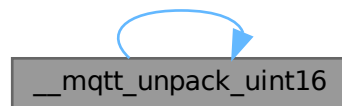
Warning

This function provides no error checking and does not modify `buf`.

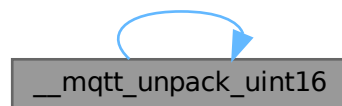
Returns

The native integer

Here is the call graph for this function:



Here is the caller graph for this function:



6.22 mqtt.h

[Go to the documentation of this file.](#)

```

00001 #if !defined(__MQTT_H__)
00002 #define __MQTT_H__
00003
00004 /*
00005 MIT License
00006
00007 Copyright(c) 2018 Liam Bindle
00008
00009 Permission is hereby granted, free of charge, to any person obtaining a copy
00010 of this software and associated documentation files(the "Software"), to deal
00011 in the Software without restriction, including without limitation the rights
00012 to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
00013 copies of the Software, and to permit persons to whom the Software is
00014 furnished to do so, subject to the following conditions :

```

```

00015
00016 The above copyright notice and this permission notice shall be included in all
00017 copies or substantial portions of the Software.
00018
00019 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
00020 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
00021 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
00022 AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
00023 LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
00024 OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
00025 SOFTWARE.
00026 */
00027
00028 #if defined(__cplusplus)
00029 extern "C" {
00030 #endif
00031
00032 // Users can override mqtt_pal.h with their own configuration by defining
00033 // MQTTTC_PAL_FILE as a header file to include (-DMQTTTC_PAL_FILE=my_mqtt_pal.h).
00034 //
00035 // If MQTTTC_PAL_FILE is used, none of the default utils will be emitted and must be
00036 // provided by the config file. To start, I would suggest copying mqtt_pal.h
00037 // and modifying as needed.
00038 #if defined(MQTTTC_PAL_FILE)
00039 #define MQTTTC_STR2(x) #x
00040 #define MQTTTC_STR(x) MQTTTC_STR2(x)
00041 #include MQTTTC_STR(MQTTTC_PAL_FILE)
00042 #else
00043 #include "mqtt_pal.h"
00044 #endif /* MQTTTC_PAL_FILE */
00045
00046
00047 enum MQTTControlPacketType {
00048     MQTT_CONTROL_CONNECT = 1u,
00049     MQTT_CONTROL_CONNACK = 2u,
00050     MQTT_CONTROL_PUBLISH = 3u,
00051     MQTT_CONTROL_PUBACK = 4u,
00052     MQTT_CONTROL_PUBREC = 5u,
00053     MQTT_CONTROL_PUBREL = 6u,
00054     MQTT_CONTROL_PUBCOMP = 7u,
00055     MQTT_CONTROL_SUBSCRIBE = 8u,
00056     MQTT_CONTROL_SUBACK = 9u,
00057     MQTT_CONTROL_UNSUBSCRIBE = 10u,
00058     MQTT_CONTROL_UNSUBACK = 11u,
00059     MQTT_CONTROL_PINGREQ = 12u,
00060     MQTT_CONTROL_PINGRESP = 13u,
00061     MQTT_CONTROL_DISCONNECT = 14u
00062 };
00063
00064 struct mqtt_fixed_header {
00065     enum MQTTControlPacketType control_type;
00066
00067     uint32_t control_flags : 4;
00068
00069     uint32_t remaining_length;
00070 };
00071
00072 #define MQTT_PROTOCOL_LEVEL 0x04
00073
00074 #define __ALL_MQTT_ERRORS(MQTT_ERROR)
00075 MQTT_ERROR(MQTT_ERROR_NULLPTR)
00076 MQTT_ERROR(MQTT_ERROR_CONTROL_FORBIDDEN_TYPE)
00077 MQTT_ERROR(MQTT_ERROR_CONTROL_INVALID_FLAGS)
00078 MQTT_ERROR(MQTT_ERROR_CONTROL_WRONG_TYPE)
00079 MQTT_ERROR(MQTT_ERROR_CONNECT_CLIENT_ID_REFUSED)
00080 MQTT_ERROR(MQTT_ERROR_CONNECT_NULL_WILL_MESSAGE)
00081 MQTT_ERROR(MQTT_ERROR_CONNECT_FORBIDDEN_WILL_QOS)
00082 MQTT_ERROR(MQTT_ERROR_CONNACK_FORBIDDEN_FLAGS)
00083 MQTT_ERROR(MQTT_ERROR_CONNACK_FORBIDDEN_CODE)
00084 MQTT_ERROR(MQTT_ERROR_PUBLISH_FORBIDDEN_QOS)
00085 MQTT_ERROR(MQTT_ERROR_SUBSCRIBE_TOO_MANY_TOPICS)
00086 MQTT_ERROR(MQTT_ERROR_MALFORMED_RESPONSE)
00087 MQTT_ERROR(MQTT_ERROR_UNSUBSCRIBE_TOO_MANY_TOPICS)
00088 MQTT_ERROR(MQTT_ERROR_RESPONSE_INVALID_CONTROL_TYPE)
00089 MQTT_ERROR(MQTT_ERROR_CONNECT_NOT_CALLED)
00090 MQTT_ERROR(MQTT_ERROR_SEND_BUFFER_IS_FULL)
00091 MQTT_ERROR(MQTT_ERROR_SOCKET_ERROR)
00092 MQTT_ERROR(MQTT_ERROR_MALFORMED_REQUEST)
00093 MQTT_ERROR(MQTT_ERROR_RECV_BUFFER_TOO_SMALL)
00094 MQTT_ERROR(MQTT_ERROR_ACK_OF_UNKNOWN)
00095 MQTT_ERROR(MQTT_ERROR_NOT_IMPLEMENTED)
00096 MQTT_ERROR(MQTT_ERROR_CONNECTION_REFUSED)
00097 MQTT_ERROR(MQTT_ERROR_SUBSCRIBE_FAILED)
00098 MQTT_ERROR(MQTT_ERROR_CONNECTION_CLOSED)
00099 MQTT_ERROR(MQTT_ERROR_INITIAL_RECONNECT)
00100 MQTT_ERROR(MQTT_ERROR_INVALID_REMAINING_LENGTH)
00101 MQTT_ERROR(MQTT_ERROR_CLEAN_SESSION_IS_REQUIRED)

```

```

00208     MQTT_ERROR(MQTT_ERROR_RECONNECT_FAILED)          \
00209     MQTT_ERROR(MQTT_ERROR_RECONNECTING)
00210
00211     /* todo: add more connection refused errors */
00212
00218 #define GENERATE_ENUM(ENUM) ENUM,
00219
00225 #define GENERATE_STRING(STRING) #STRING,
00226
00233     enum MQTTErrors {
00234         MQTT_ERROR_UNKNOWN = INT_MIN,
00235         __ALL_MQTT_ERRORS(GENERATE_ENUM)
00236         MQTT_OK = 1
00237     };
00238
00247     const char* mqtt_error_str(enum MQTTErrors error);
00248
00259     ssize_t __mqtt_pack_uint16(uint8_t *buf, uint16_t integer);
00260
00270     uint16_t __mqtt_unpack_uint16(const uint8_t *buf);
00271
00282     ssize_t __mqtt_pack_str(uint8_t *buf, const char* str);
00283
00285 #define __mqtt_packed_cstrlen(x) (2 + (unsigned int)strlen(x))
00286
00287     /* RESPONSES */
00288
00297     enum MQTTConnackReturnCode {
00298         MQTT_CONNACK_ACCEPTED = 0u,
00299         MQTT_CONNACK_REFUSED_PROTOCOL_VERSION = 1u,
00300         MQTT_CONNACK_REFUSED_IDENTIFIER_REJECTED = 2u,
00301         MQTT_CONNACK_REFUSED_SERVER_UNAVAILABLE = 3u,
00302         MQTT_CONNACK_REFUSED_BAD_USER_NAME_OR_PASSWORD = 4u,
00303         MQTT_CONNACK_REFUSED_NOT_AUTHORIZED = 5u
00304     };
00305
00314     struct mqtt_response_connack {
00319         uint8_t session_present_flag;
00320
00326         enum MQTTConnackReturnCode return_code;
00327     };
00328
00340     struct mqtt_response_publish {
00345         uint8_t dup_flag;
00346
00354         uint8_t qos_level;
00355
00357         uint8_t retain_flag;
00358
00360         uint16_t topic_name_size;
00361
00367         const void* topic_name;
00368
00370         uint16_t packet_id;
00371
00373         const void* application_message;
00374
00376         size_t application_message_size;
00377     };
00378
00388     struct mqtt_response_puback {
00390         uint16_t packet_id;
00391     };
00392
00402     struct mqtt_response_pubrec {
00404         uint16_t packet_id;
00405     };
00406
00416     struct mqtt_response_pubrel {
00418         uint16_t packet_id;
00419     };
00420
00430     struct mqtt_response_pubcomp {
00432         uint16_t packet_id;
00433     };
00434
00443     enum MQTTSubackReturnCodes {
00444         MQTT_SUBACK_SUCCESS_MAX_QOS_0 = 0u,
00445         MQTT_SUBACK_SUCCESS_MAX_QOS_1 = 1u,
00446         MQTT_SUBACK_SUCCESS_MAX_QOS_2 = 2u,
00447         MQTT_SUBACK_FAILURE = 128u
00448     };
00449
00458     struct mqtt_response_suback {
00460         uint16_t packet_id;
00461
00467         const uint8_t *return_codes;

```

```

00468
00470         size_t num_return_codes;
00471     };
00472
00481     struct mqtt_response_unsuback {
00483         uint16_t packet_id;
00484     };
00485
00496     struct mqtt_response_pingresp {
00497         int dummy;
00498     };
00499
00504     struct mqtt_response {
00506         struct mqtt_fixed_header fixed_header;
00507
00516         union {
00517             struct mqtt_response_connack connack;
00518             struct mqtt_response_publish publish;
00519             struct mqtt_response_puback puback;
00520             struct mqtt_response_pubrec pubrec;
00521             struct mqtt_response_pubrel pubrel;
00522             struct mqtt_response_pubcomp pubcomp;
00523             struct mqtt_response_suback suback;
00524             struct mqtt_response_unsuback unsuback;
00525             struct mqtt_response_pingresp pingresp;
00526         } decoded;
00527     };
00528
00543     ssize_t mqtt_unpack_fixed_header(struct mqtt_response *response, const uint8_t *buf, size_t
00544     bufsz);
00545
00561     ssize_t mqtt_unpack_connack_response(struct mqtt_response *mqtt_response, const uint8_t *buf);
00562
00578     ssize_t mqtt_unpack_publish_response(struct mqtt_response *mqtt_response, const uint8_t *buf);
00579
00596     ssize_t mqtt_unpack_pubxxx_response(struct mqtt_response *mqtt_response, const uint8_t *buf);
00597
00613     ssize_t mqtt_unpack_suback_response(struct mqtt_response *mqtt_response, const uint8_t *buf);
00614
00630     ssize_t mqtt_unpack_unsuback_response(struct mqtt_response *mqtt_response, const uint8_t
00631     *buf);
00632
00645     ssize_t mqtt_unpack_response(struct mqtt_response* response, const uint8_t *buf, size_t
00646     bufsz);
00647
00647     /* REQUESTS */
00648
00663     ssize_t mqtt_pack_fixed_header(uint8_t *buf, size_t bufsz, const struct mqtt_fixed_header
00664     *fixed_header);
00665
00673     enum MQTTConnectFlags {
00674         MQTT_CONNECT_RESERVED = 1u,
00675         MQTT_CONNECT_CLEAN_SESSION = 2u,
00676         MQTT_CONNECT_WILL_FLAG = 4u,
00677         MQTT_CONNECT_WILL_QOS_0 = (0u & 0x03) << 3,
00678         MQTT_CONNECT_WILL_QOS_1 = (1u & 0x03) << 3,
00679         MQTT_CONNECT_WILL_QOS_2 = (2u & 0x03) << 3,
00680         MQTT_CONNECT_WILL_RETAIN = 32u,
00681         MQTT_CONNECT_PASSWORD = 64u,
00682         MQTT_CONNECT_USER_NAME = 128u
00683     };
00684
00723     ssize_t mqtt_pack_connection_request(uint8_t* buf, size_t bufsz,
00724     const char* client_id,
00725     const char* will_topic,
00726     const void* will_message,
00727     size_t will_message_size,
00728     const char* user_name,
00729     const char* password,
00730     uint8_t connect_flags,
00731     uint16_t keep_alive);
00732
00741     enum MQTTPublishFlags {
00742         MQTT_PUBLISH_DUP = 8u,
00743         MQTT_PUBLISH_QOS_0 = ((0u << 1) & 0x06),
00744         MQTT_PUBLISH_QOS_1 = ((1u << 1) & 0x06),
00745         MQTT_PUBLISH_QOS_2 = ((2u << 1) & 0x06),
00746         MQTT_PUBLISH_QOS_MASK = ((3u << 1) & 0x06),
00747         MQTT_PUBLISH_RETAIN = 0x01
00748     };
00749
00773     ssize_t mqtt_pack_publish_request(uint8_t *buf, size_t bufsz,
00774     const char* topic_name,
00775     uint16_t packet_id,
00776     const void* application_message,
00777     size_t application_message_size,
00778     uint8_t publish_flags);

```

```

00779
00808     ssize_t mqtt_pack_pubxxx_request(uint8_t *buf, size_t bufsz,
00809                                     enum MQTTControlPacketType control_type,
00810                                     uint16_t packet_id);
00811
00819 #define MQTT_SUBSCRIBE_REQUEST_MAX_NUM_TOPICS 8
00820
00843     ssize_t mqtt_pack_subscribe_request(uint8_t *buf, size_t bufsz,
00844                                         unsigned int packet_id,
00845                                         ...); /* null terminated */
00846
00854 #define MQTT_UNSUBSCRIBE_REQUEST_MAX_NUM_TOPICS 8
00855
00877     ssize_t mqtt_pack_unsubscribe_request(uint8_t *buf, size_t bufsz,
00878                                           unsigned int packet_id,
00879                                           ...); /* null terminated */
00880
00895     ssize_t mqtt_pack_ping_request(uint8_t *buf, size_t bufsz);
00896
00911     ssize_t mqtt_pack_disconnect(uint8_t *buf, size_t bufsz);
00912
00917     enum MQTTQueuedMessageState {
00918         MQTT_QUEUED_UNSENT,
00919         MQTT_QUEUED_AWAITING_ACK,
00920         MQTT_QUEUED_COMPLETE
00921     };
00922
00927     struct mqtt_queued_message {
00929         uint8_t *start;
00930
00932         size_t size;
00933
00934
00936         enum MQTTQueuedMessageState state;
00937
00944         mqtt_pal_time_t time_sent;
00945
00949         enum MQTTControlPacketType control_type;
00950
00957         uint16_t packet_id;
00958     };
00959
00967     struct mqtt_message_queue {
00973         void *mem_start;
00974
00976         void *mem_end;
00977
00984         uint8_t *curr;
00985
00994         size_t curr_sz;
00995
01001         struct mqtt_queued_message *queue_tail;
01002     };
01003
01014     void mqtt_mq_init(struct mqtt_message_queue *mq, void *buf, size_t bufsz);
01015
01026     void mqtt_mq_clean(struct mqtt_message_queue *mq);
01027
01044     struct mqtt_queued_message* mqtt_mq_register(struct mqtt_message_queue *mq, size_t nbytes);
01045
01058     struct mqtt_queued_message* mqtt_mq_find(const struct mqtt_message_queue *mq, enum
MQTTControlPacketType control_type, const uint16_t *packet_id);
01059
01069 #define mqtt_mq_get(mq_ptr, index) (((struct mqtt_queued_message*) ((mq_ptr)->mem_end)) - 1 - index)
01070
01075 #define mqtt_mq_length(mq_ptr) (((struct mqtt_queued_message*) ((mq_ptr)->mem_end)) -
(mq_ptr)->queue_tail)
01076
01081 #define mqtt_mq_currsize(mq_ptr) (((mq_ptr)->curr >= (uint8_t*) ((mq_ptr)->queue_tail - 1)) ? 0 :
((uint8_t*) ((mq_ptr)->queue_tail - 1)) - (mq_ptr)->curr)
01082
01083     /* CLIENT */
01084
01091     struct mqtt_client {
01093         mqtt_pal_socket_handle sockfd;
01094
01096         uint16_t pid_lfsr;
01097
01099         uint16_t keep_alive;
01100
01105         int number_of_keep_alives;
01106
01112         size_t send_offset;
01113
01121         mqtt_pal_time_t time_of_last_send;
01122
01131         enum MQTTErrors error;

```

```

01132
01141         int response_timeout;
01142
01144         int number_of_timeouts;
01145
01152         float typical_response_time;
01153
01165         void (*publish_response_callback)(void** state, struct mqtt_response_publish
01166 *publish);
01166
01173         void* publish_response_callback_state;
01174
01190         enum MQTTErrors(*inspector_callback)(struct mqtt_client*);
01191
01199         void (*reconnect_callback)(struct mqtt_client*, void**);
01200
01205         void* reconnect_state;
01206
01210         struct {
01212             uint8_t *mem_start;
01213
01215             size_t mem_size;
01216
01218             uint8_t *curr;
01219
01221             size_t curr_sz;
01222         } recv_buffer;
01223
01230         mqtt_pal_mutex_t mutex;
01231
01233         struct mqtt_message_queue mq;
01234     };
01235
01246     uint16_t __mqtt_next_pid(struct mqtt_client *client);
01247
01256     ssize_t __mqtt_send(struct mqtt_client *client);
01257
01266     ssize_t __mqtt_recv(struct mqtt_client *client);
01267
01294     enum MQTTErrors mqtt_sync(struct mqtt_client *client);
01295
01345     enum MQTTErrors mqtt_init(struct mqtt_client *client,
01346                               mqtt_pal_socket_handle sockfd,
01347                               uint8_t *sendbuf, size_t sendbufsz,
01348                               uint8_t *recvbuf, size_t recvbufsz,
01349                               void (*publish_response_callback)(void** state, struct mqtt_response_publish
01350 *publish));
01350
01395     void mqtt_init_reconnect(struct mqtt_client *client,
01396                              void (*reconnect_callback)(struct mqtt_client *client, void** state),
01397                              void *reconnect_state,
01398                              void (*publish_response_callback)(void** state, struct mqtt_response_publish
01399 *publish));
01399
01422     void mqtt_reinit(struct mqtt_client* client,
01423                     mqtt_pal_socket_handle sockfd,
01424                     uint8_t *sendbuf, size_t sendbufsz,
01425                     uint8_t *recvbuf, size_t recvbufsz);
01426
01453     enum MQTTErrors mqtt_connect(struct mqtt_client *client,
01454                                  const char* client_id,
01455                                  const char* will_topic,
01456                                  const void* will_message,
01457                                  size_t will_message_size,
01458                                  const char* user_name,
01459                                  const char* password,
01460                                  uint8_t connect_flags,
01461                                  uint16_t keep_alive);
01462
01463     /*
01464      * todo: will_message should be a void*
01465      */
01466
01485     enum MQTTErrors mqtt_publish(struct mqtt_client *client,
01486                                  const char* topic_name,
01487                                  const void* application_message,
01488                                  size_t application_message_size,
01489                                  uint8_t publish_flags);
01490
01500     ssize_t __mqtt_puback(struct mqtt_client *client, uint16_t packet_id);
01501
01511     ssize_t __mqtt_pubrec(struct mqtt_client *client, uint16_t packet_id);
01512
01522     ssize_t __mqtt_pubrel(struct mqtt_client *client, uint16_t packet_id);
01523
01533     ssize_t __mqtt_pubcomp(struct mqtt_client *client, uint16_t packet_id);
01534

```

```

01535
01549     enum MQTTErrors mqtt_subscribe(struct mqtt_client *client,
01550         const char* topic_name,
01551         int max_qos_level);
01552
01564     enum MQTTErrors mqtt_unsubscribe(struct mqtt_client *client,
01565         const char* topic_name);
01566
01577     enum MQTTErrors mqtt_ping(struct mqtt_client *client);
01578
01583     enum MQTTErrors __mqtt_ping(struct mqtt_client *client);
01584
01597     enum MQTTErrors mqtt_disconnect(struct mqtt_client *client);
01598
01614     enum MQTTErrors mqtt_reconnect(struct mqtt_client *client);
01615
01616 #if defined(__cplusplus)
01617 }
01618 #endif
01619
01620 #endif

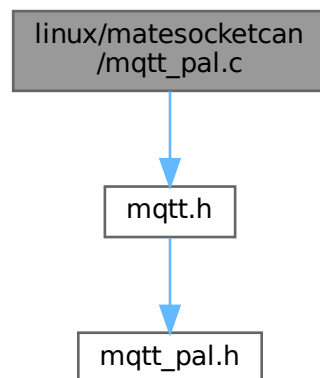
```

6.23 linux/matesocketcan/mqtt_pal.c File Reference

Implements [mqtt_pal_sendall](#) and [mqtt_pal_recvall](#) and any platform-specific helpers you'd like.

```
#include "mqtt.h"
```

Include dependency graph for mqtt_pal.c:



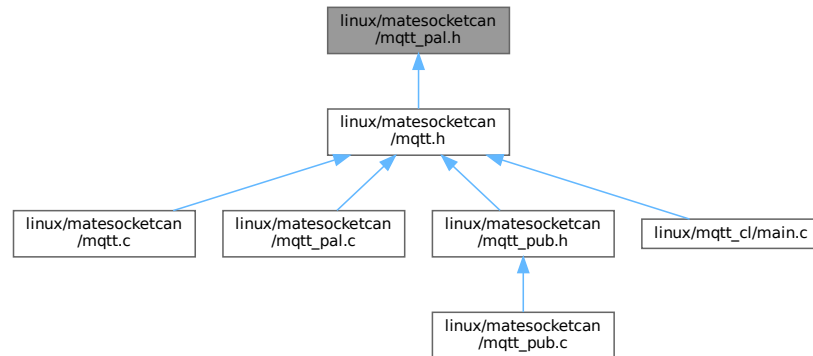
6.23.1 Detailed Description

Implements [mqtt_pal_sendall](#) and [mqtt_pal_recvall](#) and any platform-specific helpers you'd like.

6.24 linux/matesocketcan/mqtt_pal.h File Reference

Includes/supports the types/calls required by the MQTT-C client.

This graph shows which files directly or indirectly include this file:



Functions

- `ssize_t mqtt_pal_sendall` (mqtt_pal_socket_handle fd, const void *buf, size_t len, int flags)
Sends all the bytes in a buffer.
- `ssize_t mqtt_pal_recvall` (mqtt_pal_socket_handle fd, void *buf, size_t bufsz, int flags)
Non-blocking receive all the byte available.

6.24.1 Detailed Description

Includes/supports the types/calls required by the MQTT-C client.

Note

This is the *only* file included in `mqtt.h`, and `mqtt.c`. It is therefore responsible for including/supporting all the required types and calls.

6.25 mqtt_pal.h

[Go to the documentation of this file.](#)

```

00001 #if !defined(__MQTT_PAL_H_)
00002 #define __MQTT_PAL_H_
00003
00004 /*
00005 MIT License
00006
00007 Copyright(c) 2018 Liam Bindle
00008
00009 Permission is hereby granted, free of charge, to any person obtaining a copy
00010 of this software and associated documentation files(the "Software"), to deal
00011 in the Software without restriction, including without limitation the rights
00012 to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
00013 copies of the Software, and to permit persons to whom the Software is
00014 furnished to do so, subject to the following conditions :
00015
00016 The above copyright notice and this permission notice shall be included in all
00017 copies or substantial portions of the Software.
00018
00019 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
00020 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
00021 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT.IN NO EVENT SHALL THE
  
```



```

00022 AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
00023 LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
00024 OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
00025 SOFTWARE.
00026 */
00027
00028 #if defined(__cplusplus)
00029 extern "C" {
00030 #endif
00031
00069
00070
00071 /* UNIX-like platform support */
00072 #if defined(__unix__) || defined(__APPLE__) || defined(__NuttX__)
00073     #include <limits.h>
00074     #include <string.h>
00075     #include <stdarg.h>
00076     #include <time.h>
00077     #include <arpa/inet.h>
00078     #include <pthread.h>
00079
00080     #define MQTT_PAL_HTONS(s) htons(s)
00081     #define MQTT_PAL_NTOHS(s) ntohs(s)
00082
00083     #define MQTT_PAL_TIME() time(NULL)
00084
00085     typedef time_t mqtt_pal_time_t;
00086     typedef pthread_mutex_t mqtt_pal_mutex_t;
00087
00088     #define MQTT_PAL_MUTEX_INIT(mtx_ptr) pthread_mutex_init(mtx_ptr, NULL)
00089     #define MQTT_PAL_MUTEX_LOCK(mtx_ptr) pthread_mutex_lock(mtx_ptr)
00090     #define MQTT_PAL_MUTEX_UNLOCK(mtx_ptr) pthread_mutex_unlock(mtx_ptr)
00091
00092     #if !defined(MQTT_USE_CUSTOM_SOCKET_HANDLE)
00093         #if defined(MQTT_USE_MBEDTLS)
00094             struct mbedtls_ssl_context;
00095             typedef struct mbedtls_ssl_context *mqtt_pal_socket_handle;
00096         #elif defined(MQTT_USE_WOLFSSL)
00097             #include <wolfssl/ssl.h>
00098             typedef WOLFSSL* mqtt_pal_socket_handle;
00099         #elif defined(MQTT_USE_BIO)
00100             #include <openssl/bio.h>
00101             typedef BIO* mqtt_pal_socket_handle;
00102         #elif defined(MQTT_USE_BEARSSL)
00103             #include <bearssl.h>
00104
00105             typedef struct _bearssl_context {
00106                 br_ssl_client_context sc;
00107                 br_x509_minimal_context xc;
00108                 br_sslio_context ioc;
00109                 size_t ta_count;
00110                 br_x509_trust_anchor *anchOut;
00111                 int fd;
00112                 int (*low_read)(void *read_context, unsigned char *buf, size_t len);
00113                 int (*low_write)(void *write_context, const unsigned char *buf, size_t len);
00114             } bearssl_context;
00115
00116             typedef bearssl_context* mqtt_pal_socket_handle;
00117         #else
00118             typedef int mqtt_pal_socket_handle;
00119         #endif
00120     #endif
00121 #elif defined(_MSC_VER) || defined(WIN32)
00122     #include <limits.h>
00123     #include <winsock2.h>
00124     #include <windows.h>
00125     #include <time.h>
00126     #include <stdint.h>
00127
00128     typedef SSIZE_T ssize_t;
00129     #define MQTT_PAL_HTONS(s) htons(s)
00130     #define MQTT_PAL_NTOHS(s) ntohs(s)
00131
00132     #define MQTT_PAL_TIME() time(NULL)
00133
00134     typedef time_t mqtt_pal_time_t;
00135     typedef CRITICAL_SECTION mqtt_pal_mutex_t;
00136
00137     #define MQTT_PAL_MUTEX_INIT(mtx_ptr) InitializeCriticalSection(mtx_ptr)
00138     #define MQTT_PAL_MUTEX_LOCK(mtx_ptr) EnterCriticalSection(mtx_ptr)
00139     #define MQTT_PAL_MUTEX_UNLOCK(mtx_ptr) LeaveCriticalSection(mtx_ptr)
00140
00141
00142     #if !defined(MQTT_USE_CUSTOM_SOCKET_HANDLE)
00143         #if defined(MQTT_USE_BIO)
00144             #include <openssl/bio.h>
00145             typedef BIO* mqtt_pal_socket_handle;

```

```

00146         #else
00147             typedef SOCKET mqtt_pal_socket_handle;
00148         #endif
00149     #endif
00150
00151 #endif
00152
00171 ssize_t mqtt_pal_sendall(mqtt_pal_socket_handle fd, const void* buf, size_t len, int flags);
00172
00191 ssize_t mqtt_pal_recvall(mqtt_pal_socket_handle fd, void* buf, size_t bufsz, int flags);
00192
00193 #if defined(__cplusplus)
00194 }
00195 #endif
00196
00197
00198 #endif

```

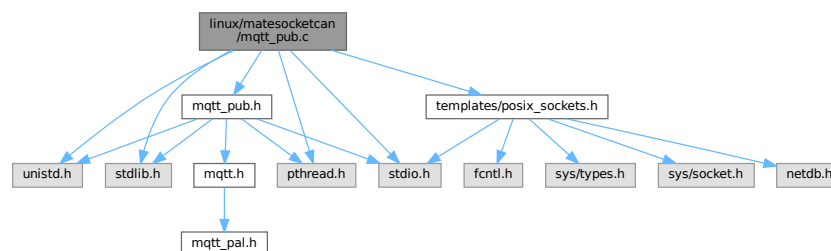
6.26 linux/matesocketcan/mqtt_pub.c File Reference

```

#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>
#include <pthread.h>
#include "templates/posix_sockets.h"
#include "mqtt_pub.h"

```

Include dependency graph for mqtt_pub.c:



Functions

- int [mqtt_socket](#) (void)
- int [mqtt_check](#) (uint8_t *application_message)
- void [mqtt_exit](#) (void)
- void [exit_example](#) (int status, int [sockfd](#), pthread_t *[client_daemon](#))

Safely closes the sockfd and cancels the client_daemon before exit.
- void [publish_callback](#) (void **unused, struct [mqtt_response_publish](#) *published)

The function that would be called whenever a PUBLISH is received.
- void * [client_refresher](#) (void *client)

The client's refresher. This function triggers back-end routines to handle ingress/egress traffic to the broker.

Variables

- const char * [addr](#)
- const char * [port](#)
- const char * [topic](#)
- int [sockfd](#)
- struct [mqtt_client](#) [client](#)
- uint8_t [sendbuf](#) [2048]
- uint8_t [recvbuf](#) [1024]
- const char * [client_id](#)
- uint8_t [connect_flags](#)
- pthread_t [client_daemon](#)

6.26.1 Function Documentation

6.26.1.1 [client_refresher\(\)](#)

```
void * client_refresher (  
    void * client)
```

The client's refresher. This function triggers back-end routines to handle ingress/egress traffic to the broker.

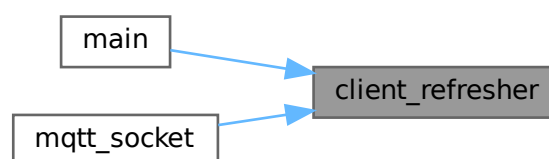
Note

All this function needs to do is call [__mqtt_rcv](#) and [__mqtt_send](#) every so often. I've picked 100 ms meaning that client ingress/egress traffic will be handled every 100 ms.

Here is the call graph for this function:



Here is the caller graph for this function:

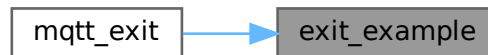


6.26.1.2 `exit_example()`

```
void exit_example (  
    int status,  
    int sockfd,  
    pthread_t * client_daemon)
```

Safety closes the `sockfd` and cancels the `client_daemon` before `exit`.

Here is the caller graph for this function:



6.26.1.3 `mqtt_check()`

```
int mqtt_check (  
    uint8_t * application_message)
```

Here is the call graph for this function:



6.26.1.4 `mqtt_exit()`

```
void mqtt_exit (  
    void )
```

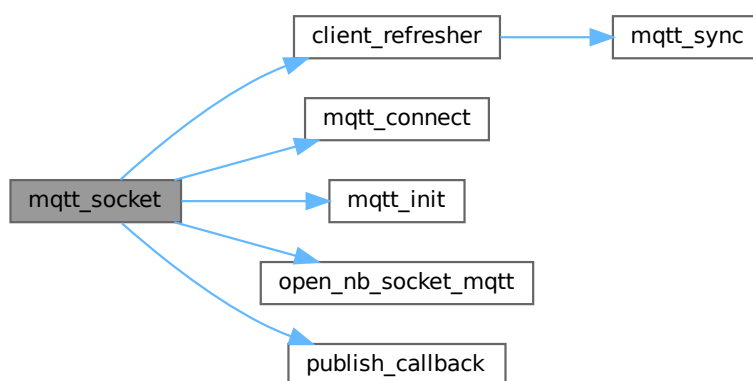
Here is the call graph for this function:



6.26.1.5 mqtt_socket()

```
int mqtt_socket (  
    void )
```

Here is the call graph for this function:



6.26.1.6 publish_callback()

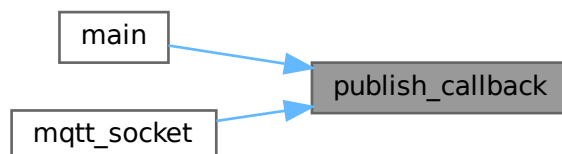
```
void publish_callback (  
    void ** unused,  
    struct mqtt_response_publish * published)
```

The function that would be called whenever a PUBLISH is received.

Note

This function is not used in this example.

Here is the caller graph for this function:



6.26.2 Variable Documentation

6.26.2.1 addr

```
const char* addr
```

6.26.2.2 client

```
struct mqtt_client client
```

6.26.2.3 client_daemon

```
pthread_t client_daemon
```

6.26.2.4 client_id

```
const char* client_id
```

6.26.2.5 connect_flags

```
uint8_t connect_flags
```

6.26.2.6 port

```
const char* port
```

6.26.2.7 recvbuf

```
uint8_t recvbuf[1024]
```

6.26.2.8 sendbuf

```
uint8_t sendbuf[2048]
```

6.26.2.9 sockfd

```
int sockfd
```

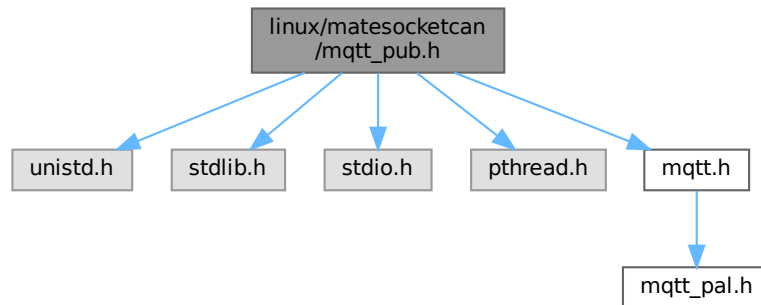
6.26.2.10 topic

```
const char* topic
```

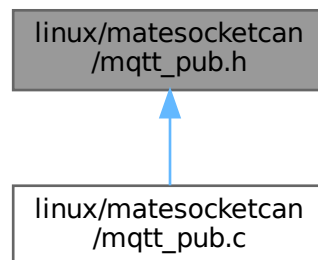
6.27 linux/matesocketcan/mqtt_pub.h File Reference

```
#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>
#include <pthread.h>
#include "mqtt.h"
```

Include dependency graph for mqtt_pub.h:



This graph shows which files directly or indirectly include this file:



Macros

- `#define MQTT_VERSION "v01.2"`
- `#define DATA_MQTT_SOLAR "mateq84/data/solar"`
- `#define ADDR_MQTT "10.1.1.172"`

Functions

- int [mqtt_socket](#) (void)
- int [mqtt_check](#) (uint8_t *)
- void [mqtt_exit](#) (void)
- void [publish_callback](#) (void **unused, struct [mqtt_response_publish](#) *published)
The function that would be called whenever a PUBLISH is received.
- void * [client_refresher](#) (void *client)
The client's refresher. This function triggers back-end routines to handle ingress/egress traffic to the broker.
- void [exit_example](#) (int status, int [sockfd](#), pthread_t *[client_daemon](#))
Safely closes the sockfd and cancels the client_daemon before exit.

6.27.1 Macro Definition Documentation

6.27.1.1 ADDR_MQTT

```
#define ADDR_MQTT "10.1.1.172"
```

6.27.1.2 DATA_MQTT_SOLAR

```
#define DATA_MQTT_SOLAR "mateq84/data/solar"
```

6.27.1.3 MQTT_VERSION

```
#define MQTT_VERSION "v01.2"
```

6.27.2 Function Documentation

6.27.2.1 client_refresher()

```
void * client_refresher (  
    void * client)
```

The client's refresher. This function triggers back-end routines to handle ingress/egress traffic to the broker.

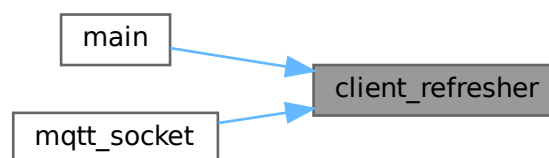
Note

All this function needs to do is call `__mqtt_rcv` and `__mqtt_send` every so often. I've picked 100 ms meaning that client ingress/egress traffic will be handled every 100 ms.

Here is the call graph for this function:



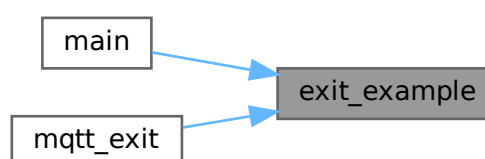
Here is the caller graph for this function:

**6.27.2.2 exit_example()**

```
void exit_example (  
    int status,  
    int sockfd,  
    pthread_t * client_daemon)
```

Safely closes the `sockfd` and cancels the `client_daemon` before `exit`.

Here is the caller graph for this function:



6.27.2.3 mqtt_check()

```
int mqtt_check (  
    uint8_t * application_message)
```

Here is the call graph for this function:



6.27.2.4 mqtt_exit()

```
void mqtt_exit (  
    void )
```

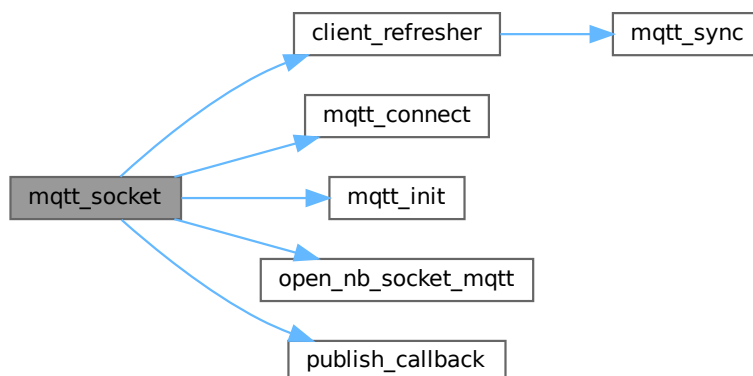
Here is the call graph for this function:



6.27.2.5 mqtt_socket()

```
int mqtt_socket (  
    void )
```

Here is the call graph for this function:



6.27.2.6 publish_callback()

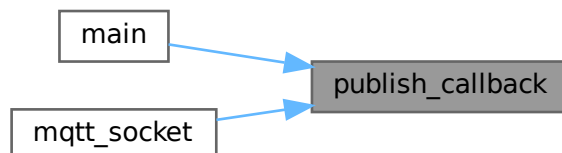
```
void publish_callback (  
    void ** unused,  
    struct mqtt_response_publish * published)
```

The function that would be called whenever a PUBLISH is received.

Note

This function is not used in this example.

Here is the caller graph for this function:



6.28 mqtt_pub.h

[Go to the documentation of this file.](#)

```

00001 /*
00002  * File:   mqtt_pub.h
00003  * Author: root
00004  *
00005  * Created on November 3, 2023, 8:52 AM
00006  */
00007
00008 #ifndef MQTT_PUB_H
00009 #define MQTT_PUB_H
00010
00011 #ifdef __cplusplus
00012 extern "C" {
00013 #endif
00014
00015 #include <unistd.h>
00016 #include <stdlib.h>
00017 #include <stdio.h>
00018 #include <pthread.h>
00019
00020 #include "mqtt.h"
00021
00022 #define MQTT_VERSION          "v01.2"
00023 #define DATA_MQTT_SOLAR     "mateq84/data/solar"
00024 // #define DATA_MQTT_SOLAR    "comedi/data/p8055/set"
00025 // #define ADDR_MQTT           "test.mosquitto.org" // testing broker
00026 #define ADDR_MQTT            "10.1.1.172" // local broker
00027
00028     int mqtt_socket(void);
00029     int mqtt_check(uint8_t *);
00030     void mqtt_exit(void);
00031
00032     void publish_callback(void** unused, struct mqtt_response_publish *published);
00033
00034     void* client_refresher(void* client);
00035
00036     void exit_example(int status, int sockfd, pthread_t *client_daemon);
00037
00038 #ifdef __cplusplus
00039 }
00040 #endif
00041 #endif /* MQTT_PUB_H */

```

6.29 linux/matesocketcan/nbproject/private/c_standard_headers_indexer.c File Reference

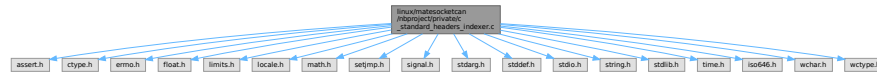
```

#include <assert.h>
#include <ctype.h>
#include <errno.h>
#include <float.h>
#include <limits.h>
#include <locale.h>
#include <math.h>
#include <setjmp.h>
#include <signal.h>
#include <stdarg.h>
#include <stddef.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <time.h>
#include <iso646.h>
#include <wchar.h>

```

```
#include <wctype.h>
```

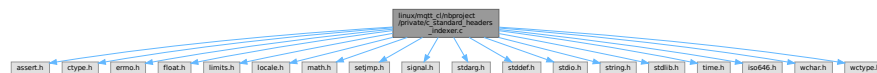
Include dependency graph for c_standard_headers_indexer.c:



6.30 linux/mqtt_cl/nbproject/private/c_standard_headers_indexer.c File Reference

```
#include <assert.h>
#include <ctype.h>
#include <errno.h>
#include <float.h>
#include <limits.h>
#include <locale.h>
#include <math.h>
#include <setjmp.h>
#include <signal.h>
#include <stdarg.h>
#include <stddef.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <time.h>
#include <iso646.h>
#include <wchar.h>
#include <wctype.h>
```

Include dependency graph for c_standard_headers_indexer.c:



6.31 linux/matesocketcan/nbproject/private/cpp_standard_headers_indexer.cpp File Reference

```
#include <cstdlib>
#include <csignal>
#include <csetjmp>
#include <cstdarg>
#include <typeinfo>
#include <bitset>
#include <functional>
#include <utility>
#include <ctime>
#include <cstddef>
#include <new>
```

```
#include <memory>
#include <climits>
#include <cfloat>
#include <limits>
#include <exception>
#include <stdexcept>
#include <cassert>
#include <cerrno>
#include <cctype>
#include <cwctype>
#include <cstring>
#include <wchar>
#include <string>
#include <vector>
#include <deque>
#include <list>
#include <set>
#include <map>
#include <stack>
#include <queue>
#include <algorithm>
#include <iterator>
#include <cmath>
#include <complex>
#include <valarray>
#include <numeric>
#include <iosfwd>
#include <ios>
#include <istream>
#include <ostream>
#include <iostream>
#include <fstream>
#include <sstream>
#include <sstream>
#include <iomanip>
#include <streambuf>
#include <cstdio>
#include <locale>
#include <clocale>
#include <ciso646>
```

6.32 linux/mqtt_cl/nbproject/private/cpp_standard_headers_indexer.cpp

File Reference

```
#include <cstdlib>
#include <csignal>
#include <setjmp>
#include <cstdint>
#include <typeinfo>
#include <bitset>
#include <functional>
#include <utility>
#include <ctime>
#include <cstdint>
```

```
#include <new>
#include <memory>
#include <climits>
#include <cfloat>
#include <limits>
#include <exception>
#include <stdexcept>
#include <cassert>
#include <cerrno>
#include <cctype>
#include <cwctype>
#include <cstring>
#include <wchar>
#include <string>
#include <vector>
#include <deque>
#include <list>
#include <set>
#include <map>
#include <stack>
#include <queue>
#include <algorithm>
#include <iterator>
#include <cmath>
#include <complex>
#include <valarray>
#include <numeric>
#include <iosfwd>
#include <ios>
#include <istream>
#include <ostream>
#include <iostream>
#include <fstream>
#include <sstream>
#include <strstream>
#include <iomanip>
#include <streambuf>
#include <cstdio>
#include <locale>
#include <clocale>
#include <ciso646>
```

6.33 linux/matesocketcan/pge.h File Reference

Macros

- #define [E_MONTH](#) 2266.0f
- #define [G_MONTH](#) 1000.0f
- #define [E_DAYS](#) 31.0f
- #define [E_PER_DAY](#) [E_MONTH](#)/[E_DAYS](#)
- #define [E_PER_HOUR](#) [E_PER_DAY](#)/24.0f
- #define [G_PER_DAY](#) [G_MONTH](#)/[E_DAYS](#)
- #define [G_PER_HOUR](#) [G_PER_DAY](#)/24.0f
- #define [PGE_ZERO](#)

6.33.1 Macro Definition Documentation

6.33.1.1 E_DAYS

```
#define E_DAYS 31.0f
```

6.33.1.2 E_MONTH

```
#define E_MONTH 2266.0f
```

6.33.1.3 E_PER_DAY

```
#define E_PER_DAY E_MONTH/E_DAYS
```

6.33.1.4 E_PER_HOUR

```
#define E_PER_HOUR E_PER_DAY/24.0f
```

6.33.1.5 G_MONTH

```
#define G_MONTH 1000.0f
```

6.33.1.6 G_PER_DAY

```
#define G_PER_DAY G_MONTH/E_DAYS
```

6.33.1.7 G_PER_HOUR

```
#define G_PER_HOUR G_PER_DAY/24.0f
```

6.33.1.8 PGE_ZERO

```
#define PGE_ZERO
```


6.34 pge.h

[Go to the documentation of this file.](#)

```

00001 /*
00002  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
00003  * Click nbfs://nbhost/SystemFileSystem/Templates/cFiles/file.h to edit this template
00004  */
00005
00006 /*
00007  * File:   pge.h
00008  * Author: root
00009  *
00010  * Created on December 31, 2023, 5:55 PM
00011  */
00012
00013 #ifndef PGE_H
00014 #define PGE_H
00015
00016 #ifdef __cplusplus
00017 extern "C" {
00018 #endif
00019
00020
00021 #define E_MONTH      2266.0f // Kwh
00022 #define G_MONTH      1000.0f // kWh
00023 #define E_DAYS       31.0f
00024 #define E_PER_DAY    E_MONTH/E_DAYS
00025 #define E_PER_HOUR   E_PER_DAY/24.0f
00026 #define G_PER_DAY    G_MONTH/E_DAYS
00027 #define G_PER_HOUR   G_PER_DAY/24.0f
00028
00029 #define PGE_ZERO
00030
00031 #ifdef __cplusplus
00032 }
00033 #endif
00034
00035 #endif /* PGE_H */
00036

```

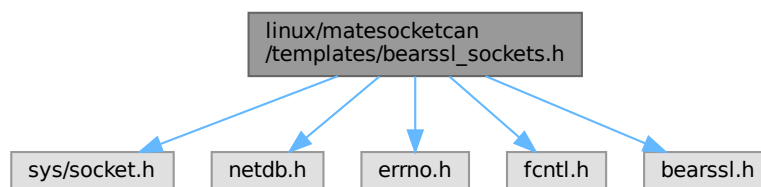
6.35 linux/matesocketcan/templates/bearssl_sockets.h File Reference

```

#include <sys/socket.h>
#include <netdb.h>
#include <errno.h>
#include <fcntl.h>
#include <bearssl.h>

```

Include dependency graph for bearssl_sockets.h:



Functions

- `int open_nb_socket` (bearssl_context *ctx, const char *hostname, const char *port, unsigned char *bearssl_iobuf, size_t bearssl_iobuf_len)
- `int close_socket` (bearssl_context *ctx)

6.35.1 Function Documentation

6.35.1.1 close_socket()

```
int close_socket (
    bearssl_context * ctx)
```

6.35.1.2 open_nb_socket()

```
int open_nb_socket (
    bearssl_context * ctx,
    const char * hostname,
    const char * port,
    unsigned char * bearssl_iobuf,
    size_t bearssl_iobuf_len)
```

Here is the caller graph for this function:



6.36 bearssl_sockets.h

[Go to the documentation of this file.](#)

```
00001 #if !defined(__BEARSSL_SOCKET_TEMPLATE_H__)
00002 #define __BEARSSL_SOCKET_TEMPLATE_H__
00003
00004 #include <sys/socket.h>
00005 #include <netdb.h>
00006 #include <errno.h>
00007 #include <fcntl.h>
00008
00009 #include <bearssl.h>
00010
00011 // #include "mqtt_pal.h"
00012
00013 /*
00014  * Low-level data read callback for the simplified SSL I/O API.
00015  */
00016 static int sock_read(void *ctx, unsigned char *buf, size_t len) {
00017     ssize_t rlen;
00018
00019     for (;;) {
00020         rlen = read(*(int *)ctx, buf, len);
00021
00022         if (rlen < 0) {
00023             if (errno == EINTR) {
00024                 continue;
00025             }
00026             else if (errno == EWOULDBLOCK || errno == EAGAIN) {
00027                 rlen = 0;
00028                 break;
00029             }
00030             else {
00031                 break;
00032             }
00033         }
00034     }
00035 }
```

```

00033     }
00034     else {
00035         break;
00036     }
00037 }
00038
00039 return (int)rlen;
00040 }
00041
00042 /*
00043  * Low-level data write callback for the simplified SSL I/O API.
00044  */
00045 static int sock_write(void *ctx, const unsigned char *buf, size_t len) {
00046     ssize_t wlen;
00047
00048     for (;;) {
00049
00050         wlen = write(*(int *)ctx, buf, len);
00051         if (wlen <= 0 && errno == EINTR) {
00052             continue;
00053         }
00054         return (int)wlen;
00055     }
00056 }
00057
00058 static int host_connect(const char *host, const char *port) {
00059     struct hostent *he;
00060     struct in_addr **addr_list;
00061
00062     if (NULL == (he = gethostbyname(host)))
00063     {
00064         fprintf(stderr, "Failed to resolve host name\n");
00065         return -1;
00066     }
00067
00068     addr_list = (struct in_addr **) he->h_addr_list;
00069
00070     int sockfd = -1;
00071     int rv;
00072     int i;
00073
00074     if (-1 == (sockfd = socket(AF_INET, SOCK_STREAM, IPPROTO_IP)))
00075         return -1;
00076
00077     struct sockaddr_in server;
00078
00079     server.sin_family = AF_INET;
00080     server.sin_port = htons(atoi(port));
00081
00082     /* open the first possible socket */
00083     for (i = 0; addr_list[i] != NULL; i++)
00084     {
00085         memcpy(&server.sin_addr.s_addr, he->h_addr_list[0], he->h_length);
00086
00087         if (0 > (rv = connect(sockfd, (struct sockaddr *)&server, sizeof(server)))) {
00088             continue;
00089         }
00090         else {
00091             int flags = fcntl(sockfd, F_GETFL, 0);
00092             flags |= O_NONBLOCK;
00093             fcntl(sockfd, F_SETFL, flags);
00094             break;
00095         }
00096     }
00097
00098     if (addr_list[i] == NULL)
00099         sockfd = -1;
00100
00101     /* return the new socket fd */
00102     return sockfd;
00103 }
00104
00105 int open_nb_socket(bearssl_context *ctx,
00106                  const char *hostname,
00107                  const char *port,
00108                  unsigned char *bearssl_iobuf,
00109                  size_t bearssl_iobuf_len) {
00110
00111     /* initialize the BearSSL engine */
00112     br_ssl_client_init_full(&ctx->sc, &ctx->xc, ctx->anchOut, ctx->ta_count);
00113     br_ssl_engine_set_buffer(&ctx->sc.eng, bearssl_iobuf, bearssl_iobuf_len, 1);
00114     br_ssl_client_reset(&ctx->sc, hostname, 0);
00115
00116     if (-1 == (ctx->fd = host_connect(hostname, port)))
00117         return -1;
00118
00119     ctx->low_read = sock_read;

```

```

00120     ctx->low_write = sock_write;
00121
00122     return 0;
00123 }
00124
00125 int close_socket(bearssl_context *ctx) {
00126     int rc;
00127
00128     br_ssl_engine_close(&ctx->sc.eng);
00129
00130     if (ctx->fd != 0) {
00131         shutdown(ctx->fd, SHUT_RDWR);
00132         rc = close(ctx->fd);
00133         ctx->fd = 0;
00134     }
00135
00136     return rc;
00137 }
00138
00139 #endif

```

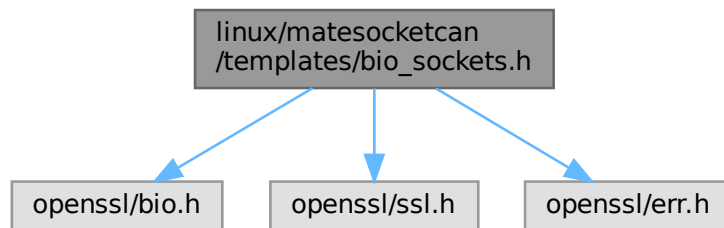
6.37 linux/matesocketcan/templates/bio_sockets.h File Reference

```

#include <openssl/bio.h>
#include <openssl/ssl.h>
#include <openssl/err.h>

```

Include dependency graph for bio_sockets.h:



Functions

- BIO * [open_nb_socket](#) (const char *[addr](#), const char *[port](#))

6.37.1 Function Documentation

6.37.1.1 open_nb_socket()

```

BIO * open_nb_socket (
    const char * addr,
    const char * port)

```

6.38 bio_sockets.h

[Go to the documentation of this file.](#)

```

00001 #if !defined(__BIO_SOCKET_TEMPLATE_H__)
00002 #define __BIO_SOCKET_TEMPLATE_H__
00003
00004 #include <openssl/bio.h>
00005 #include <openssl/ssl.h>
00006 #include <openssl/err.h>
00007
00008 /*
00009  * A template for opening a non-blocking BIO socket.
00010  */
00011 BIO* open_nb_socket(const char* addr, const char* port) {
00012     BIO* bio = BIO_new_connect(addr);
00013     BIO_set_nbio(bio, 1);
00014     BIO_set_conn_port(bio, port);
00015
00016     /* timeout after 10 seconds */
00017     int start_time = time(NULL);
00018     while(BIO_do_connect(bio) == 0 && (int)time(NULL) - start_time < 10);
00019
00020     if (BIO_do_connect(bio) <= 0) {
00021         fprintf(stderr, "Failed to open socket: BIO_do_connect returned <= 0\n");
00022         return NULL;
00023     }
00024
00025     return bio;
00026 }
00027
00028 #endif

```

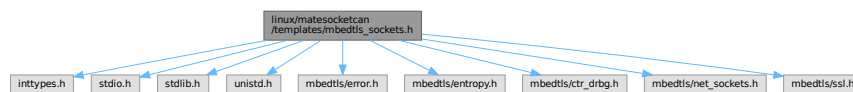
6.39 linux/matesocketcan/templates/mbedtls_sockets.h File Reference

```

#include <inttypes.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <mbedtls/error.h>
#include <mbedtls/entropy.h>
#include <mbedtls/ctr_drbg.h>
#include <mbedtls/net_sockets.h>
#include <mbedtls/ssl.h>

```

Include dependency graph for mbedtls_sockets.h:



Data Structures

- struct [mbedtls_context](#)

Macros

- #define [MBEDTLS_NET_POLL_READ](#) 1
- #define [MBEDTLS_NET_POLL_WRITE](#) 1

Functions

- int [mbedtls_net_poll](#) (mbedtls_net_context *ctx, uint32_t rw, uint32_t timeout)
- void [failed](#) (const char *fn, int rv)
- void [cert_verify_failed](#) (uint32_t rv)
- void [open_nb_socket](#) (struct [mbedtls_context](#) *ctx, const char *hostname, const char *port, const char *ca↔_file)

6.39.1 Macro Definition Documentation

6.39.1.1 MBEDTLS_NET_POLL_READ

```
#define MBEDTLS_NET_POLL_READ 1
```

6.39.1.2 MBEDTLS_NET_POLL_WRITE

```
#define MBEDTLS_NET_POLL_WRITE 1
```

6.39.2 Function Documentation

6.39.2.1 cert_verify_failed()

```
void cert_verify_failed (
    uint32_t rv)
```

Here is the caller graph for this function:



6.39.2.2 failed()

```
void failed (
    const char * fn,
    int rv)
```

Here is the caller graph for this function:



6.39.2.3 mbedtls_net_poll()

```
int mbedtls_net_poll (
    mbedtls_net_context * ctx,
    uint32_t rw,
    uint32_t timeout)
```

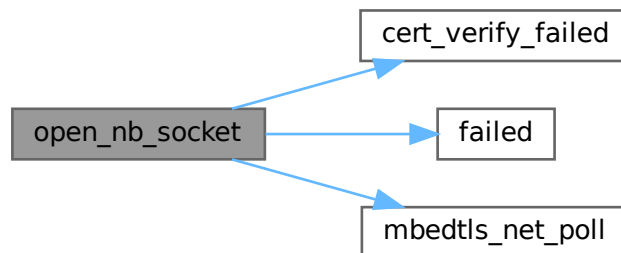
Here is the caller graph for this function:



6.39.2.4 open_nb_socket()

```
void open_nb_socket (
    struct mbedtls_context * ctx,
    const char * hostname,
    const char * port,
    const char * ca_file)
```

Here is the call graph for this function:



6.40 mbedtls_sockets.h

[Go to the documentation of this file.](#)

```
00001 #if !defined(__MBEDTLS_SOCKET_TEMPLATE_H__)
00002 #define __MBEDTLS_SOCKET_TEMPLATE_H__
00003
00004 #include <inttypes.h>
00005 #include <stdio.h>
```

```

00006 #include <stdlib.h>
00007 #include <unistd.h>
00008
00009 #include <mbedtls/error.h>
00010 #include <mbedtls/entropy.h>
00011 #include <mbedtls/ctr_drbg.h>
00012 #include <mbedtls/net_sockets.h>
00013 #include <mbedtls/ssl.h>
00014
00015 #if !defined(MBEDTLS_NET_POLL_READ)
00016 /* compat for older mbedtls */
00017 #define MBEDTLS_NET_POLL_READ 1
00018 #define MBEDTLS_NET_POLL_WRITE 1
00019
00020 int
00021 mbedtls_net_poll(mbedtls_net_context * ctx, uint32_t rw, uint32_t timeout)
00022 {
00023     /* XXX this is not ideal but good enough for an example */
00024     usleep(300);
00025     return 1;
00026 }
00027 #endif
00028
00029 struct mbedtls_context {
00030     mbedtls_net_context net_ctx;
00031     mbedtls_ssl_context ssl_ctx;
00032     mbedtls_ssl_config ssl_conf;
00033     mbedtls_x509_crt ca_crt;
00034     mbedtls_entropy_context entropy;
00035     mbedtls_ctr_drbg_context ctr_drbg;
00036 };
00037
00038 void failed(const char *fn, int rv);
00039 void cert_verify_failed(uint32_t rv);
00040 void open_nb_socket(struct mbedtls_context *ctx,
00041                    const char *hostname,
00042                    const char *port,
00043                    const char *ca_file);
00044
00045
00046 void failed(const char *fn, int rv) {
00047     char buf[100];
00048     mbedtls_strerror(rv, buf, sizeof(buf));
00049     printf("%s failed with %x (%s)\n", fn, -rv, buf);
00050     exit(1);
00051 }
00052
00053 void cert_verify_failed(uint32_t rv) {
00054     char buf[512];
00055     mbedtls_x509_crt_verify_info(buf, sizeof(buf), "\t", rv);
00056     printf("Certificate verification failed (%0" PRIx32 ") \n%s\n", rv, buf);
00057     exit(1);
00058 }
00059
00060 /*
00061  A template for opening a non-blocking mbed TLS connection.
00062  */
00063 void open_nb_socket(struct mbedtls_context *ctx,
00064                    const char *hostname,
00065                    const char *port,
00066                    const char *ca_file) {
00067     const unsigned char additional = (const unsigned char *) "MQTT-C";
00068     size_t additional_len = 6;
00069     int rv;
00070
00071     mbedtls_net_context *net_ctx = &ctx->net_ctx;
00072     mbedtls_ssl_context *ssl_ctx = &ctx->ssl_ctx;
00073     mbedtls_ssl_config *ssl_conf = &ctx->ssl_conf;
00074     mbedtls_x509_crt *ca_crt = &ctx->ca_crt;
00075     mbedtls_entropy_context *entropy = &ctx->entropy;
00076     mbedtls_ctr_drbg_context *ctr_drbg = &ctx->ctr_drbg;
00077
00078     mbedtls_entropy_init(entropy);
00079     mbedtls_ctr_drbg_init(ctr_drbg);
00080     rv = mbedtls_ctr_drbg_seed(ctr_drbg, mbedtls_entropy_func, entropy,
00081                               additional, additional_len);
00082     if (rv != 0) {
00083         failed("mbedtls_ctr_drbg_seed", rv);
00084     }
00085
00086     mbedtls_x509_crt_init(ca_crt);
00087     rv = mbedtls_x509_crt_parse_file(ca_crt, ca_file);
00088     if (rv != 0) {
00089         failed("mbedtls_x509_crt_parse_file", rv);
00090     }
00091
00092     mbedtls_ssl_config_init(ssl_conf);

```



```

00093     rv = mbedtls_ssl_config_defaults(ssl_conf,  MBEDTLS_SSL_IS_CLIENT,
00094                                     MBEDTLS_SSL_TRANSPORT_STREAM,
00095                                     MBEDTLS_SSL_PRESET_DEFAULT);
00096     if (rv != 0) {
00097         failed("mbedtls_ssl_config_defaults", rv);
00098     }
00099     mbedtls_ssl_conf_ca_chain(ssl_conf, ca_cert, NULL);
00100     mbedtls_ssl_conf_authmode(ssl_conf, MBEDTLS_SSL_VERIFY_OPTIONAL);
00101     mbedtls_ssl_conf_rng(ssl_conf, mbedtls_ctr_drbg_random, ctr_drbg);
00102
00103     mbedtls_net_init(&net_ctx);
00104     rv = mbedtls_net_connect(&net_ctx, hostname, port, MBEDTLS_NET_PROTO_TCP);
00105     if (rv != 0) {
00106         failed("mbedtls_net_connect", rv);
00107     }
00108     rv = mbedtls_net_set_nonblock(&net_ctx);
00109     if (rv != 0) {
00110         failed("mbedtls_net_set_nonblock", rv);
00111     }
00112
00113     mbedtls_ssl_init(&ssl_ctx);
00114     rv = mbedtls_ssl_setup(&ssl_ctx, ssl_conf);
00115     if (rv != 0) {
00116         failed("mbedtls_ssl_setup", rv);
00117     }
00118     rv = mbedtls_ssl_set_hostname(&ssl_ctx, hostname);
00119     if (rv != 0) {
00120         failed("mbedtls_ssl_set_hostname", rv);
00121     }
00122     mbedtls_ssl_set_bio(&ssl_ctx, &net_ctx,
00123                        mbedtls_net_send, mbedtls_net_recv, NULL);
00124
00125     for (;;) {
00126         rv = mbedtls_ssl_handshake(&ssl_ctx);
00127         uint32_t want = 0;
00128         if (rv == MBEDTLS_ERR_SSL_WANT_READ) {
00129             want |= MBEDTLS_NET_POLL_READ;
00130         } else if (rv == MBEDTLS_ERR_SSL_WANT_WRITE) {
00131             want |= MBEDTLS_NET_POLL_WRITE;
00132         } else {
00133             break;
00134         }
00135         rv = mbedtls_net_poll(&net_ctx, want, (uint32_t)-1);
00136         if (rv < 0) {
00137             failed("mbedtls_net_poll", rv);
00138         }
00139     }
00140     if (rv != 0) {
00141         failed("mbedtls_ssl_handshake", rv);
00142     }
00143     uint32_t result = mbedtls_ssl_get_verify_result(&ssl_ctx);
00144     if (result != 0) {
00145         if (result == (uint32_t)-1) {
00146             failed("mbedtls_ssl_get_verify_result", (int)result);
00147         } else {
00148             cert_verify_failed(result);
00149         }
00150     }
00151 }
00152
00153 #endif

```

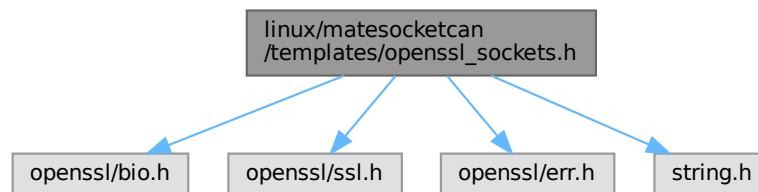
6.41 linux/matesocketcan/templates/openssl_sockets.h File Reference

```

#include <openssl/bio.h>
#include <openssl/ssl.h>
#include <openssl/err.h>
#include <string.h>

```

Include dependency graph for openssl_sockets.h:



Functions

- void `open_nb_socket` (BIO ***bio*, SSL_CTX ***ssl_ctx*, const char **addr*, const char **port*, const char **ca_file*, const char **ca_path*, const char **cert_file*, const char **key_file*)

6.41.1 Function Documentation

6.41.1.1 open_nb_socket()

```

void open_nb_socket (
    BIO ** bio,
    SSL_CTX ** ssl_ctx,
    const char * addr,
    const char * port,
    const char * ca_file,
    const char * ca_path,
    const char * cert_file,
    const char * key_file)

```

6.42 openssl_sockets.h

[Go to the documentation of this file.](#)

```

00001 #if !defined(__OPENSSL_SOCKET_TEMPLATE_H__)
00002 #define __OPENSSL_SOCKET_TEMPLATE_H__
00003
00004 #include <openssl/bio.h>
00005 #include <openssl/ssl.h>
00006 #include <openssl/err.h>
00007
00008 #include <string.h>
00009
00010 /*
00011  * A template for opening a non-blocking OpenSSL connection.
00012  */
00013 void open_nb_socket (BIO**      bio,
00014                     SSL_CTX**  ssl_ctx,
00015                     const char* addr,
00016                     const char* port,
00017                     const char* ca_file,
00018                     const char* ca_path,
00019                     const char* cert_file,
00020                     const char* key_file);
00021
00022 void open_nb_socket (BIO**      bio,

```

```

00023         SSL_CTX**  ssl_ctx,
00024         const char* addr,
00025         const char* port,
00026         const char* ca_file,
00027         const char* ca_path,
00028         const char* cert_file,
00029         const char* key_file)
00030 {
00031     *ssl_ctx = SSL_CTX_new(SSLv23_client_method());
00032     SSL* ssl;
00033
00034     /* load certificate */
00035     if (!SSL_CTX_load_verify_locations(*ssl_ctx, ca_file, ca_path)) {
00036         printf("error: failed to load ca certificate\n");
00037         exit(1);
00038     }
00039
00040     if (cert_file && key_file)
00041     {
00042         if (!SSL_CTX_use_certificate_file(*ssl_ctx, cert_file, SSL_FILETYPE_PEM))
00043         {
00044             printf("error: failed to load client certificate\n");
00045             exit(1);
00046         }
00047
00048         if (!SSL_CTX_use_PrivateKey_file(*ssl_ctx, key_file, SSL_FILETYPE_PEM))
00049         {
00050             printf("error: failed to load client key\n");
00051             exit(1);
00052         }
00053     }
00054
00055     /* open BIO socket */
00056     char * addr_copy = (char*)malloc(strlen(addr) + 1);
00057     strcpy(addr_copy, addr);
00058     char * port_copy = (char*)malloc(strlen(port) + 1);
00059     strcpy(port_copy, port);
00060
00061     *bio = BIO_new_ssl_connect(*ssl_ctx);
00062     BIO_get_ssl(*bio, &ssl);
00063     SSL_set_mode(ssl, SSL_MODE_AUTO_RETRY);
00064     BIO_set_conn_hostname(*bio, addr_copy);
00065     BIO_set_nbio(*bio, 1);
00066     BIO_set_conn_port(*bio, port_copy);
00067
00068     free(addr_copy);
00069     free(port_copy);
00070
00071     /* wait for connect with 10 second timeout */
00072     int start_time = (int)time(NULL);
00073     int do_connect_rv = (int)BIO_do_connect(*bio);
00074     while(do_connect_rv <= 0 && BIO_should_retry(*bio) && (int)time(NULL) - start_time < 10) {
00075         do_connect_rv = (int)BIO_do_connect(*bio);
00076     }
00077     if (do_connect_rv <= 0) {
00078         printf("error: %s\n", ERR_reason_error_string(ERR_get_error()));
00079         BIO_free_all(*bio);
00080         SSL_CTX_free(*ssl_ctx);
00081         *bio = NULL;
00082         *ssl_ctx=NULL;
00083         return;
00084     }
00085
00086     /* verify certificate */
00087     if (SSL_get_verify_result(ssl) != X509_V_OK) {
00088         /* Handle the failed verification */
00089         printf("error: x509 certificate verification failed\n");
00090         exit(1);
00091     }
00092 }
00093
00094 #endif

```

6.43 linux/matesocketcan/templates/posix_sockets.h File Reference

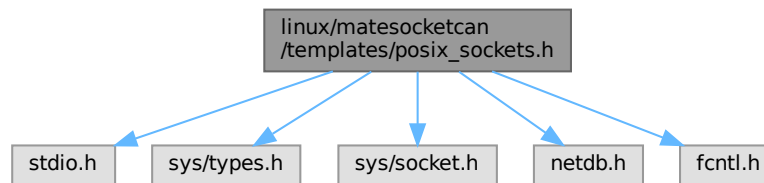
```

#include <stdio.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netdb.h>

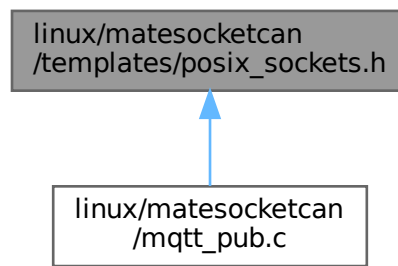
```

```
#include <fcntl.h>
```

Include dependency graph for `posix_sockets.h`:



This graph shows which files directly or indirectly include this file:



Functions

- int `open_nb_socket_mqtt` (const char *`addr`, const char *`port`)

6.43.1 Function Documentation

6.43.1.1 `open_nb_socket_mqtt()`

```
int open_nb_socket_mqtt (
    const char * addr,
    const char * port)
```

Here is the caller graph for this function:



6.44 posix_sockets.h

[Go to the documentation of this file.](#)

```

00001 #if !defined(__POSIX_SOCKET_TEMPLATE_H__)
00002 #define __POSIX_SOCKET_TEMPLATE_H__
00003
00004 #include <stdio.h>
00005 #include <sys/types.h>
00006 #if !defined(WIN32)
00007 #include <sys/socket.h>
00008 #include <netdb.h>
00009 #else
00010 #include <ws2tcpip.h>
00011 #endif
00012 #if defined(__VMS)
00013 #include <iocctl.h>
00014 #endif
00015 #include <fcntl.h>
00016
00017 /*
00018  A template for opening a non-blocking POSIX socket.
00019 */
00020 int open_nb_socket_mqtt(const char* addr, const char* port);
00021
00022 int open_nb_socket_mqtt(const char* addr, const char* port) {
00023     struct addrinfo hints = {0};
00024
00025     hints.ai_family = AF_UNSPEC; /* IPv4 or IPv6 */
00026     hints.ai_socktype = SOCK_STREAM; /* Must be TCP */
00027     int sockfd = -1;
00028     int rv;
00029     struct addrinfo *p, *servinfo;
00030
00031     /* get address information */
00032     rv = getaddrinfo(addr, port, &hints, &servinfo);
00033     if(rv != 0) {
00034         fprintf(stderr, "Failed to open socket (getaddrinfo): %s\n", gai_strerror(rv));
00035         return -1;
00036     }
00037
00038     /* open the first possible socket */
00039     for(p = servinfo; p != NULL; p = p->ai_next) {
00040         sockfd = socket(p->ai_family, p->ai_socktype, p->ai_protocol);
00041         if (sockfd == -1) continue;
00042
00043         /* connect to server */
00044         rv = connect(sockfd, p->ai_addr, p->ai_addrlen);
00045         if(rv == -1) {
00046             close(sockfd);
00047             sockfd = -1;
00048             continue;
00049         }
00050         break;
00051     }
00052
00053     /* free servinfo */
00054     freeaddrinfo(servinfo);
00055
00056     /* make non-blocking */
00057 #if !defined(WIN32)
00058     if (sockfd != -1) fcntl(sockfd, F_SETFL, fcntl(sockfd, F_GETFL) | O_NONBLOCK);
00059 #else
00060     if (sockfd != INVALID_SOCKET) {
00061         int iMode = 1;
00062         ioctlsocket(sockfd, FIONBIO, &iMode);
00063     }
00064 #endif
00065 #if defined(__VMS)
00066     /*
00067      OpenVMS only partially implements fcntl. It works on file descriptors
00068      but silently fails on socket descriptors. So we need to fall back on
00069      to the older ioctl system to set non-blocking IO
00070     */
00071     int on = 1;
00072     if (sockfd != -1) ioctl(sockfd, FIONBIO, &on);
00073 #endif
00074
00075     /* return the new socket fd */
00076     return sockfd;
00077 }
00078
00079 #endif

```

6.45 linux/raw.githubusercontent.com_linux-can_can-utils_master_ canfdtest.c File Reference

```
#include <errno.h>
#include <getopt.h>
#include <libgen.h>
#include <limits.h>
#include <sched.h>
#include <signal.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>
#include <time.h>
#include <unistd.h>
#include <sys/stat.h>
#include <syslog.h>
#include <net/if.h>
#include <sys/ioctl.h>
#include <sys/socket.h>
#include <sys/time.h>
#include <sys/types.h>
#include <cjson/cJSON.h>
#include "MQTTClient.h"
#include <linux/can.h>
#include <linux/can/raw.h>
```

Include dependency graph for raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c:



Macros

- #define [_DEFAULT_SOURCE](#)
- #define [LOG_TO_FILE](#) "/store/logs/canfd.log"
- #define [CAN_MSG_ID_PING](#) 0x80000002
- #define [CAN_MSG_ID_PING_X](#) 0x80000003
- #define [EMON_SL](#) 0x80000002
- #define [EMON_SU](#) 0x80000003
- #define [EMON_SH](#) 0x80000004
- #define [EMON_ER](#) 0x8000000F
- #define [EMON_CO](#) 0x8000000C
- #define [EMON_DA](#) 0x8000000D
- #define [EMON_TM](#) 0x8000000A
- #define [CAN_MSG_ID_PONG](#) 0x3
- #define [CAN_MSG_LEN](#) 64
- #define [CAN_FULL_BUFFER](#) [CAN_MSG_LEN](#)+[CAN_MSG_LEN](#)+[CAN_MSG_LEN](#)+1
- #define [CAN_MSG_COUNT](#) 1
- #define [CAN_MSG_WAIT](#) 27
- #define [CAN_TM_TIME](#) 30
- #define [HR_SEC](#) 3600
- #define [DAY_SEC](#) [HR_SEC](#)*24

- #define LOG_VERSION "v1.12"
- #define MQTT_VERSION "V3.11"
- #define ADDRESS "tcp://10.1.1.172:1883"
- #define CLIENTID "MateQ84_Mqtt"
- #define TOPIC_P "mateq84/data/solar"
- #define TOPIC_S "mateq84/data/solar/sub"
- #define QOS 1
- #define TIMEOUT 10000L
- #define SPACING_USEC 500 * 1000
- #define MQTT_TIMEOUT 150
- #define E_MONTH 2266.0f
- #define G_MONTH 1000.0f
- #define E_DAYS 31.0f
- #define E_PER_DAY E_MONTH/E_DAYS
- #define E_PER_HOUR E_PER_DAY/24.0f
- #define G_PER_DAY G_MONTH/E_DAYS
- #define G_PER_HOUR G_PER_DAY/24.0f
- #define PGE_ZERO

Functions

- long long current_timestamp (void)
- void timer_callback (int32_t)
- void delivered (void *, MQTTClient_deliveryToken)
- int32_t msgarrvd (void *, char *, int, MQTTClient_message *)
- void connlost (void *, char *)
- char * log_time (bool)
- int main (int argc, char *argv[])

Variables

- uint8_t full_buffer [CAN_FULL_BUFFER]
- uint8_t data_buffer [CAN_FULL_BUFFER]
- int32_t sec_30
- char * token
- cJSON * json
- FILE * fout
- volatile MQTTClient_deliveryToken deliveredtoken
- volatile MQTTClient_deliveryToken receivedtoken = false
- volatile bool runner = false
- MQTTClient client
- MQTTClient_connectOptions conn_opts = MQTTClient_connectOptions_initializer
- MQTTClient_message pubmsg = MQTTClient_message_initializer
- MQTTClient_deliveryToken mtoken
- time_t start_time = 0
- time_t hour_time = 0
- time_t day_time = 0
- double benergy
- double acenergy
- double load
- double solar
- double bvolts
- double bamps

- double [pvolts](#)
- double [pamps](#)
- double [pwatts](#)
- double [runtime](#)
- double [bat_energy_scaled](#)
- double [bat_energy_kw](#)
- double [acvolts](#)
- double [acamps](#) = 0.0f
- double [gridin](#) = 0.001
- double [gridout](#) = 0.001
- double [gasenergy](#) = 0.001
- double [watergal](#) = 0.1
- int32_t [ccmode](#) = 0
- int32_t [sequence](#)

6.45.1 Macro Definition Documentation

6.45.1.1 `_DEFAULT_SOURCE`

```
#define _DEFAULT_SOURCE
```

6.45.1.2 `ADDRESS`

```
#define ADDRESS "tcp://10.1.1.172:1883"
```

6.45.1.3 `CAN_FULL_BUFFER`

```
#define CAN_FULL_BUFFER CAN\_MSG\_LEN+CAN\_MSG\_LEN+CAN\_MSG\_LEN+1
```

6.45.1.4 `CAN_MSG_COUNT`

```
#define CAN_MSG_COUNT 1
```

6.45.1.5 `CAN_MSG_ID_PING`

```
#define CAN_MSG_ID_PING 0x80000002
```

6.45.1.6 `CAN_MSG_ID_PING_X`

```
#define CAN_MSG_ID_PING_X 0x80000003
```

6.45.1.7 `CAN_MSG_ID_PONG`

```
#define CAN_MSG_ID_PONG 0x3
```


6.45.1.8 CAN_MSG_LEN

```
#define CAN_MSG_LEN 64
```

6.45.1.9 CAN_MSG_WAIT

```
#define CAN_MSG_WAIT 27
```

6.45.1.10 CAN_TM_TIME

```
#define CAN_TM_TIME 30
```

6.45.1.11 CLIENTID

```
#define CLIENTID "MateQ84_Mqtt"
```

6.45.1.12 DAY_SEC

```
#define DAY_SEC HR\_SEC*24
```

6.45.1.13 E_DAYS

```
#define E_DAYS 31.0f
```

6.45.1.14 E_MONTH

```
#define E_MONTH 2266.0f
```

6.45.1.15 E_PER_DAY

```
#define E_PER_DAY E\_MONTH/E\_DAYS
```

6.45.1.16 E_PER_HOUR

```
#define E_PER_HOUR E\_PER\_DAY/24.0f
```

6.45.1.17 EMON_CO

```
#define EMON_CO 0x8000000C
```

6.45.1.18 EMON_DA

```
#define EMON_DA 0x8000000D
```

6.45.1.19 EMON_ER

```
#define EMON_ER 0x8000000F
```

6.45.1.20 EMON_SH

```
#define EMON_SH 0x80000004
```

6.45.1.21 EMON_SL

```
#define EMON_SL 0x80000002
```

6.45.1.22 EMON_SU

```
#define EMON_SU 0x80000003
```

6.45.1.23 EMON_TM

```
#define EMON_TM 0x8000000A
```

6.45.1.24 G_MONTH

```
#define G_MONTH 1000.0f
```

6.45.1.25 G_PER_DAY

```
#define G_PER_DAY G_MONTH/E_DAYS
```

6.45.1.26 G_PER_HOUR

```
#define G_PER_HOUR G_PER_DAY/24.0f
```

6.45.1.27 HR_SEC

```
#define HR_SEC 3600
```

6.45.1.28 LOG_TO_FILE

```
#define LOG_TO_FILE "/store/logs/canfd.log"
```

6.45.1.29 LOG_VERSION

```
#define LOG_VERSION "v1.12"
```

6.45.1.30 MQTT_TIMEOUT

```
#define MQTT_TIMEOUT 150
```

6.45.1.31 MQTT_VERSION

```
#define MQTT_VERSION "V3.11"
```

6.45.1.32 PGE_ZERO

```
#define PGE_ZERO
```

6.45.1.33 QOS

```
#define QOS 1
```

6.45.1.34 SPACING_USEC

```
#define SPACING_USEC 500 * 1000
```

6.45.1.35 TIMEOUT

```
#define TIMEOUT 10000L
```

6.45.1.36 TOPIC_P

```
#define TOPIC_P "mateq84/data/solar"
```

6.45.1.37 TOPIC_S

```
#define TOPIC_S "mateq84/data/solar/sub"
```

6.45.2 Function Documentation

6.45.2.1 connlost()

```
void connlost (  
    void * context,  
    char * cause)
```

Here is the caller graph for this function:



6.45.2.2 current_timestamp()

```
long long current_timestamp (  
    void )
```

6.45.2.3 delivered()

```
void delivered (  
    void * context,  
    MQTTClient_deliveryToken dt)
```

Here is the caller graph for this function:



6.45.2.4 log_time()

```
char * log_time (  
    bool log)
```

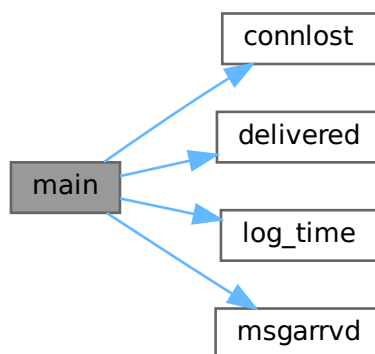
Here is the caller graph for this function:



6.45.2.5 main()

```
int main (  
    int argc,  
    char * argv[])
```

Here is the call graph for this function:



6.45.2.6 msgarrvd()

```
int32_t msgarrvd (  
    void * context,  
    char * topicName,  
    int topicLen,  
    MQTTClient_message * message)
```

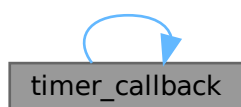
Here is the caller graph for this function:



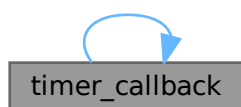
6.45.2.7 timer_callback()

```
void timer_callback (  
    int32_t signum)
```

Here is the call graph for this function:



Here is the caller graph for this function:



6.45.3 Variable Documentation

6.45.3.1 acamps

```
double acamps = 0.0f
```

6.45.3.2 acenergy

```
double acenergy
```

6.45.3.3 acvolts

```
double acvolts
```

6.45.3.4 bamps

```
double bamps
```

6.45.3.5 bat_energy_kw

```
double bat_energy_kw
```

6.45.3.6 bat_energy_scaled

```
double bat_energy_scaled
```

6.45.3.7 benergy

```
double benergy
```

6.45.3.8 bvolts

```
double bvolts
```

6.45.3.9 ccmode

```
int32_t ccmode = 0
```

6.45.3.10 client

```
MQTTClient client
```

6.45.3.11 conn_opts

```
MQTTClient_connectOptions conn_opts = MQTTClient_connectOptions_initializer
```

6.45.3.12 data_buffer

```
uint8_t data_buffer[CAN_FULL_BUFFER]
```

6.45.3.13 day_time

```
time_t day_time = 0
```

6.45.3.14 deliveredtoken

```
volatile MQTTClient_deliveryToken deliveredtoken
```

6.45.3.15 fout

```
FILE* fout
```

6.45.3.16 full_buffer

```
uint8_t full_buffer[CAN_FULL_BUFFER]
```

6.45.3.17 gasenergy

```
double gasenergy = 0.001
```

6.45.3.18 gridin

```
double gridin = 0.001
```

6.45.3.19 gridout

```
double gridout = 0.001
```

6.45.3.20 hour_time

```
time_t hour_time = 0
```

6.45.3.21 json

```
cJSON* json
```


6.45.3.22 load

```
double load
```

6.45.3.23 mtoken

```
MQTTClient_deliveryToken mtoken
```

6.45.3.24 pamps

```
double pamps
```

6.45.3.25 pubmsg

```
MQTTClient_message pubmsg = MQTTClient_message_initializer
```

6.45.3.26 pvolts

```
double pvolts
```

6.45.3.27 pwatts

```
double pwatts
```

6.45.3.28 receivedtoken

```
volatile MQTTClient_deliveryToken receivedtoken = false
```

6.45.3.29 runner

```
volatile bool runner = false
```

6.45.3.30 runtime

```
double runtime
```

6.45.3.31 sec_30

```
int32_t sec_30
```

6.45.3.32 sequence

```
int32_t sequence
```

6.45.3.33 solar

```
double solar
```

6.45.3.34 start_time

```
time_t start_time = 0
```

6.45.3.35 token

```
char* token
```

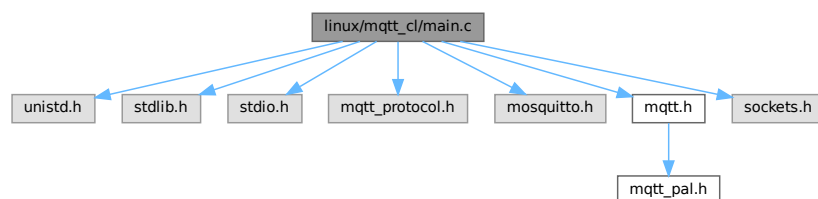
6.45.3.36 watergal

```
double watergal = 0.1
```

6.46 linux/mqtt_cl/main.c File Reference

```
#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>
#include <mqtt_protocol.h>
#include <mosquitto.h>
#include <mqtt.h>
#include <sockets.h>
```

Include dependency graph for main.c:



Functions

- void [publish_callback](#) (void **unused, struct [mqtt_response_publish](#) *published)
The function that would be called whenever a PUBLISH is received.
- void * [client_refresher](#) (void *client)
The client's refresher. This function triggers back-end routines to handle ingress/egress traffic to the broker.
- void [exit_example](#) (int status, int [sockfd](#), pthread_t *client_daemon)
Safely closes the sockfd and cancels the client_daemon before exit.
- int [main](#) (int argc, const char *argv[])

6.46.1 Function Documentation

6.46.1.1 client_refresher()

```
void * client_refresher (  
    void * client)
```

The client's refresher. This function triggers back-end routines to handle ingress/egress traffic to the broker.

Note

All this function needs to do is call [__mqtt_rcv](#) and [__mqtt_send](#) every so often. I've picked 100 ms meaning that client ingress/egress traffic will be handled every 100 ms.

Here is the call graph for this function:

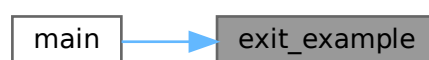


6.46.1.2 exit_example()

```
void exit_example (  
    int status,  
    int sockfd,  
    pthread_t * client_daemon)
```

Safely closes the sockfd and cancels the client_daemon before exit.

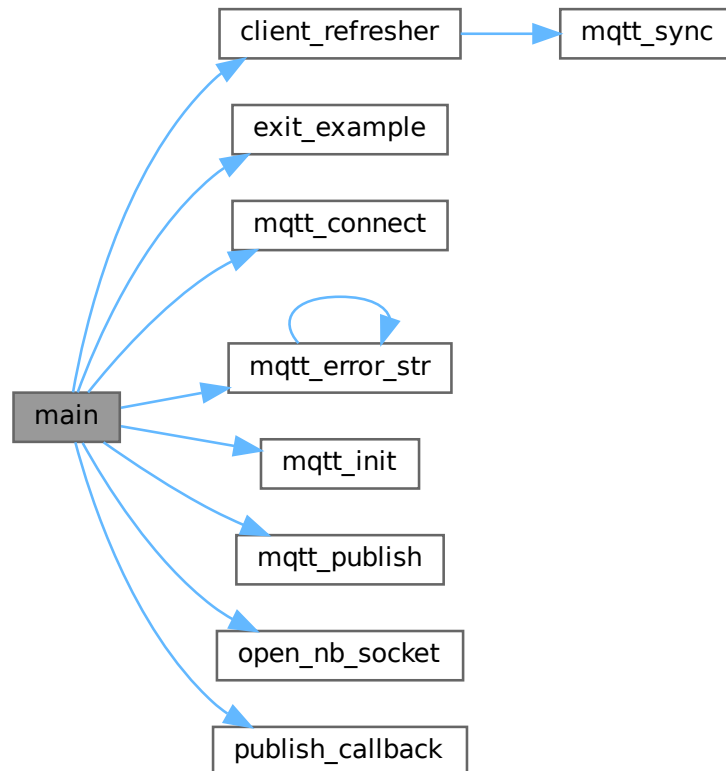
Here is the caller graph for this function:



6.46.1.3 main()

```
int main (  
    int argc,  
    const char * argv[])
```

Here is the call graph for this function:



6.46.1.4 publish_callback()

```
void publish_callback (  
    void ** unused,  
    struct mqtt_response_publish * published)
```

The function that would be called whenever a PUBLISH is received.

Note

This function is not used in this example.

6.47 main.c File Reference

Functions

- void [main](#) (void)

6.47.1 Function Documentation

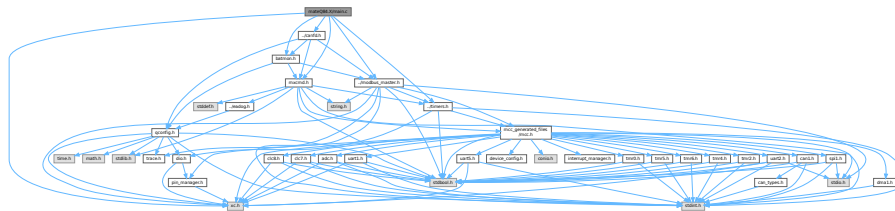
6.47.1.1 main()

```
void main (
    void )
```

6.48 mateQ84.X/main.c File Reference

```
#include <xc.h>
#include "mxcmd.h"
#include "../timers.h"
#include "../modbus_master.h"
#include "../canfd.h"
#include "../batmon.h"
```

Include dependency graph for main.c:



Macros

- #define [PACE](#) 31000
- #define [CMD_LEN](#) 8
- #define [REC_LEN](#) 5
- #define [REC_STATUS_LEN](#) 16
- #define [REC_LOG_LEN](#) 17
- #define [E_UPDATE](#) 10
- #define [E_SAVE](#) 3

Enumerations

- enum [state_type](#) {
[state_init](#) , [state_status](#) , [state_panel](#) , [state_batteryv](#) ,
[state_batterya](#) , [state_watts](#) , [state_fwrev](#) , [state_time](#) ,
[state_date](#) , [state_mx_log](#) , [state_misc](#) , [state_mx_status](#) ,
[state_last](#) }

Functions

- void [state_init_cb](#) (void)
- void [state_status_cb](#) (void)
- void [state_panelv_cb](#) (void)
- void [state_batteryv_cb](#) (void)
- void [state_battery_a_cb](#) (void)
- void [state_watts_cb](#) (void)
- void [state_misc_cb](#) (void)
- void [state_mx_status_cb](#) (void)
- void [state_mx_log_cb](#) (void)
- void [wdtdelay](#) (const uint32_t delay)
- void [main](#) (void)
- char [spinners](#) (uint8_t shape, const uint8_t reset)
- void [run_day_to_night](#) (void)
- void [run_night_to_day](#) (void)

Variables

- volatile uint16_t [cc_mode](#) = [STATUS_LAST](#)
- volatile uint16_t [mx_code](#) = 0x00
- uint16_t [volt_whole](#)
- uint16_t [bat_amp_whole](#) = [AMP_WHOLE_ZERO](#)
- uint16_t [panel_watts](#)
- uint16_t [volt_fract](#)
- uint16_t [vf](#)
- uint16_t [vw](#)
- enum [state_type](#) [state](#) = [state_init](#)
- char [buffer](#) [[MAX_B_BUF](#)] = "Boot Init Display "
- char [info_buffer](#) [[MAX_B_BUF](#)]
- char [log_buffer](#) [[MAX_B_BUF](#)]
- const char * [build_date](#) = [__DATE__](#)
- const char * [build_time](#) = [__TIME__](#)
- volatile uint16_t [tickCount](#) [[TMR_COUNT](#)]
- uint8_t [fw_state](#) = 0
- time_t [can_timer](#) = 1694196350
- struct tm * [can_newtime](#)
- bool [show_can](#)
- uint8_t [time_show_can](#)
- [B_type](#) [B](#)

6.48.1 Macro Definition Documentation

6.48.1.1 CMD_LEN

```
#define CMD_LEN 8
```

6.48.1.2 E_SAVE

```
#define E_SAVE 3
```

6.48.1.3 E_UPDATE

```
#define E_UPDATE 10
```

6.48.1.4 PACE

```
#define PACE 31000
```

Generated Main Source File

Company: Microchip Technology Inc.

File Name: main.c

Summary: This is the main file generated using PIC10 / PIC12 / PIC16 / PIC18 MCUs

Description: This header file provides implementations for driver APIs for all modules selected in the GUI. Generation Information : Product Revision : PIC10 / PIC12 / PIC16 / PIC18 MCUs - 1.81.8 Device : PIC18F14Q41 Driver Version : 2.00

6.48.1.5 REC_LEN

```
#define REC_LEN 5
```

6.48.1.6 REC_LOG_LEN

```
#define REC_LOG_LEN 17
```

6.48.1.7 REC_STATUS_LEN

```
#define REC_STATUS_LEN 16
```

6.48.2 Enumeration Type Documentation

6.48.2.1 state_type

```
enum state_type
```

Enumerator

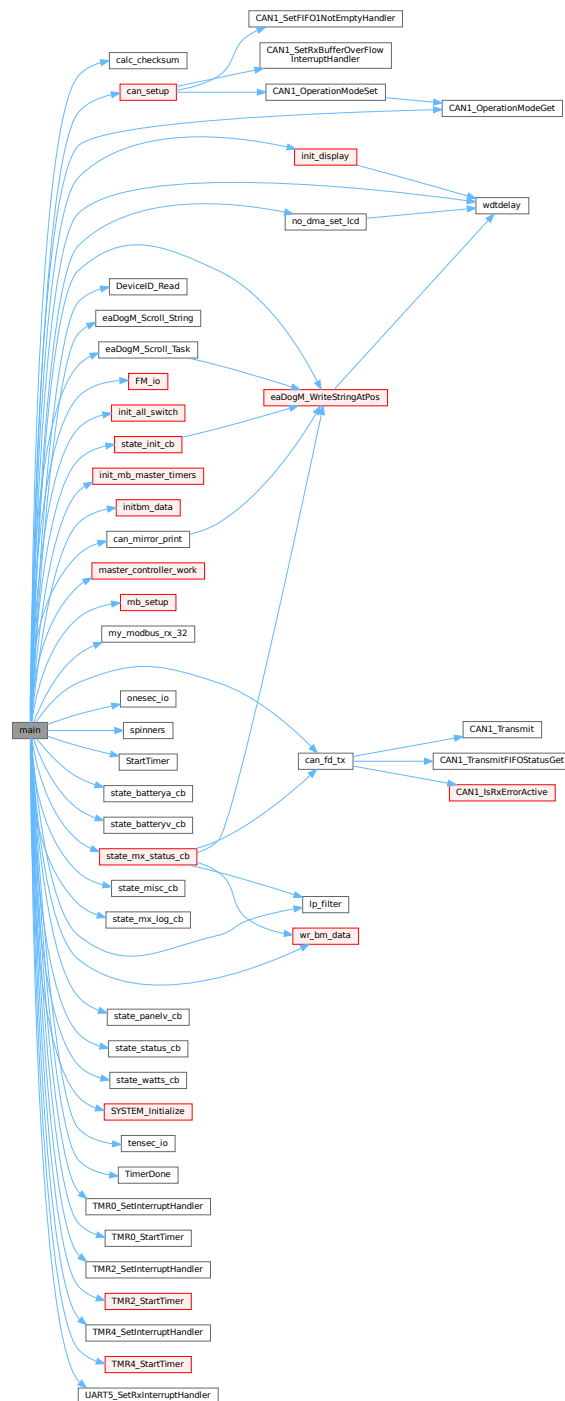
state_init	
state_status	
state_panel	
state_batteryv	
state_batterya	
state_watts	
state_fwrev	
state_time	
state_date	
state_mx_log	
state_misc	
state_mx_status	
state_last	

6.48.3 Function Documentation

6.48.3.1 main()

```
void main (
    void )
```

Here is the call graph for this function:



6.48.3.2 run_day_to_night()

```
void run_day_to_night (  
    void )
```

Here is the call graph for this function:



Here is the caller graph for this function:



6.48.3.3 run_night_to_day()

```
void run_night_to_day (  
    void )
```

Here is the call graph for this function:



Here is the caller graph for this function:



6.48.3.4 spinners()

```
char spinners (  
    uint8_t shape,  
    const uint8_t reset)
```

Here is the caller graph for this function:



6.48.3.5 state_battery_cb()

```
void state_battery_cb (  
    void )
```

Here is the caller graph for this function:



6.48.3.6 state_batteryv_cb()

```
void state_batteryv_cb (  
    void )
```

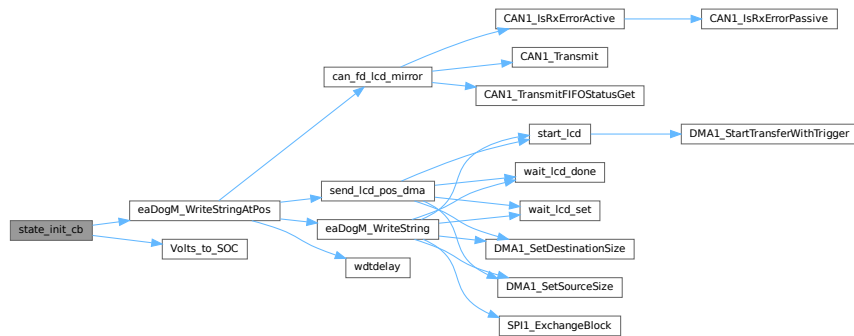
Here is the caller graph for this function:



6.48.3.7 state_init_cb()

```
void state_init_cb (
    void )
```

Here is the call graph for this function:



Here is the caller graph for this function:



6.48.3.8 state_misc_cb()

```
void state_misc_cb (
    void )
```

Here is the caller graph for this function:



6.48.3.9 state_mx_log_cb()

```
void state_mx_log_cb (
    void )
```

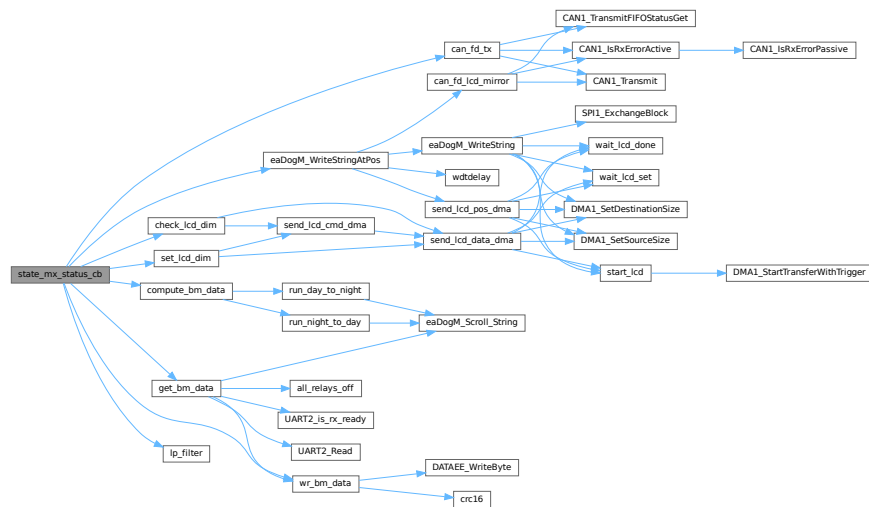
Here is the caller graph for this function:



6.48.3.10 state_mx_status_cb()

```
void state_mx_status_cb (
    void )
```

Here is the call graph for this function:



Here is the caller graph for this function:



6.48.3.11 state_panelv_cb()

```
void state_panelv_cb (  
    void )
```

Here is the caller graph for this function:



6.48.3.12 state_status_cb()

```
void state_status_cb (  
    void )
```

Here is the caller graph for this function:



6.48.3.13 state_watts_cb()

```
void state_watts_cb (  
    void )
```

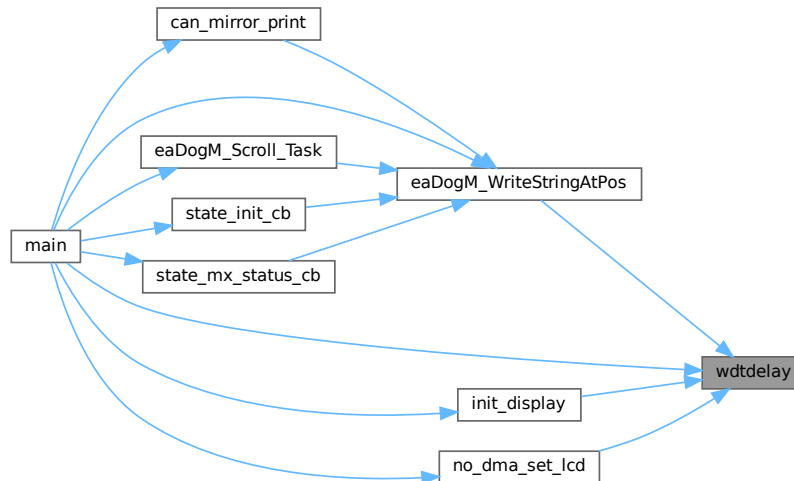
Here is the caller graph for this function:



6.48.3.14 wdtdelay()

```
void wdtdelay (
    const uint32_t delay)
```

Here is the caller graph for this function:



6.48.4 Variable Documentation

6.48.4.1 B

`B_type` B

Initial value:

```
{
    .one_sec_flag = false,
    .ten_sec_flag = false,
    .pacing = 0,
    .rx_count = 0,
    .flush = 0,
    .canbus_online = 0,
    .modbus_online = 0,
    .log.select = 1,
    .pv_high = false,
    .pv_prev = STATUS_SLEEPING,
    .pv_update = false,
    .once = false,
    .log.type = 1,
    .display_dim = false,
    .display_update = false,
    .dim_delay = DIM_DELAY,
    .display_on = true,
}
```

6.48.4.2 bat_amp_whole

```
uint16_t bat_amp_whole = AMP_WHOLE_ZERO
```

6.48.4.3 buffer

```
char buffer[MAX_B_BUF] = "Boot Init Display "
```

6.48.4.4 build_date

```
const char* build_date = __DATE__
```

6.48.4.5 build_time

```
const char * build_time = __TIME__
```

6.48.4.6 can_newtime

```
struct tm* can_newtime
```

6.48.4.7 can_timer

```
time_t can_timer = 1694196350
```

6.48.4.8 cc_mode

```
volatile uint16_t cc_mode = STATUS_LAST
```

6.48.4.9 fw_state

```
uint8_t fw_state = 0
```

6.48.4.10 info_buffer

```
char info_buffer[MAX_B_BUF]
```

6.48.4.11 log_buffer

```
char log_buffer[MAX_B_BUF]
```

6.48.4.12 mx_code

```
volatile uint16_t mx_code = 0x00
```

6.48.4.13 panel_watts

```
uint16_t panel_watts
```

6.48.4.14 show_can

```
bool show_can
```

6.48.4.15 state

```
enum state_type state = state_init
```

6.48.4.16 tickCount

```
volatile uint16_t tickCount[TMR_COUNT]
```

6.48.4.17 time_show_can

```
uint8_t time_show_can
```

6.48.4.18 vf

```
uint16_t vf
```

6.48.4.19 volt_fract

```
uint16_t volt_fract
```

6.48.4.20 volt_whole

```
uint16_t volt_whole
```

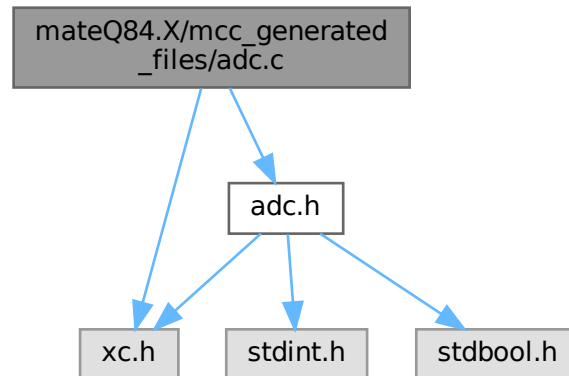
6.48.4.21 vw

```
uint16_t vw
```


- 6.49** `mateQ84.X/build/default/production/_ext/1472/batmon.p1.d` File Reference
- 6.50** `mateQ84.X/build/default/production/_ext/1472/canfd.p1.d` File Reference
- 6.51** `mateQ84.X/build/default/production/_ext/1472/dio.p1.d` File Reference
- 6.52** `mateQ84.X/build/default/production/_ext/1472/eadog.p1.d` File Reference
- 6.53** `mateQ84.X/build/default/production/_ext/1472/modbus_master.p1.d` File Reference
- 6.54** `mateQ84.X/build/default/production/_ext/1472/timers.p1.d` File Reference
- 6.55** `mateQ84.X/build/default/production/main.p1.d` File Reference
- 6.56** `mateQ84.X/build/default/production/mcc_generated_files/adc.p1.d` File Reference
- 6.57** `mateQ84.X/build/default/production/mcc_generated_files/can1.p1.d` File Reference
- 6.58** `mateQ84.X/build/default/production/mcc_generated_files/clc7.p1.d` File Reference
- 6.59** `mateQ84.X/build/default/production/mcc_generated_files/clc8.p1.d` File Reference
- 6.60** `mateQ84.X/build/default/production/mcc_generated_files/device_↔
config.p1.d` File Reference
- 6.61** `mateQ84.X/build/default/production/mcc_generated_↔
files/dma1.p1.d` File Reference
- 6.62** `mateQ84.X/build/default/production/mcc_generated_files/interrupt_↔
_manager.p1.d` File Reference
- 6.63** `mateQ84.X/build/default/production/mcc_generated_files/mcc.p1.d` File Reference
- 6.64** `mateQ84.X/build/default/production/mcc_generated_files/pin_↔
manager.p1.d` File Reference

```
#include "adc.h"
```

Include dependency graph for adc.c:



Functions

- void [ADC_Initialize](#) (void)
This API initializes the ADC module. This routine must be called before any other ADC routine.
- void [ADC_EnableChannelSequencer](#) (void)
This API enables ADC channel sequencer. ADC module should be initialized using [ADC_Initialize](#) before calling this API.
- void [ADC_DisableChannelSequencer](#) (void)
This API disables ADC channel sequencer. ADC module should be initialized using [ADC_Initialize](#) before calling this API.
- void [ADC_StartChannelSequencer](#) (void)
This API starts the channel sequencer by enabling GO bit. Channel sequencer should be enabled using [ADC_EnableChannelSequencer](#) before calling this API.
- void [ADC_SelectContext](#) ([ADC_context_t](#) context)
This API is used to select an ADC context to perform read/write operations on context specific registers.
- void [ADC_EnableChannelScan](#) ([ADC_context_t](#) context)
This API is used to include a channel context in the scan sequence. Correct context should be selected using [ADC_SelectContext](#) before calling this API.
- void [ADC_DisableChannelScan](#) ([ADC_context_t](#) context)
This API is used to exclude a channel context from the scan sequence. Correct context should be selected using [ADC_SelectContext](#) before calling this API.
- void [ADC_StartConversion](#) ([ADC_channel_t](#) channel)
This API starts A/D conversion on selected channel. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.
- bool [ADC_IsConversionDone](#) (void)
This API checks if ongoing A/D conversion is complete.
- [adc_result_t](#) [ADC_GetConversionResult](#) (void)
This API is used to retrieve the result of latest A/D conversion. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.
- [adc_result_t](#) [ADC_GetSingleConversion](#) ([ADC_channel_t](#) channel)

This API is used to retrieve the result of single A/D conversion on given channel. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

- void [ADC_StopConversion](#) (void)

This API is used to stop ongoing A/D conversion. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

- void [ADC_SetStopOnInterrupt](#) (void)

This API is used enable Stop On Interrupt bit for selected A/D context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

- void [ADC_DischargeSampleCapacitor](#) (void)

This API is used to discharge input sample capacitor for selected context by setting the channel to AVss. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

- void [ADC_LoadAcquisitionRegister](#) (uint16_t acquisitionValue)

This API is used to load ADC Acquisition Time Control register with specified value. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

- void [ADC_SetPrechargeTime](#) (uint16_t prechargeTime)

This API is used to load ADC Precharge Time Control register with specified value. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

- void [ADC_SetRepeatCount](#) (uint8_t repeatCount)

This API is used to load repeat counter for given context with specified value. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

- uint8_t [ADC_GetCurrentCountofConversions](#) (void)

This API retrieves the current value of ADC Repeat Count register for selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

- void [ADC_ClearAccumulator](#) (void)

This API clears the accumulator of selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

- uint24_t [ADC_GetAccumulatorValue](#) (void)

This API retrieves 18-bit value of ADC accumulator of selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

- bool [ADC_HasAccumulatorOverflowed](#) (void)

This API is used to determine whether ADC accumulator has overflowed for selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

- uint16_t [ADC_GetFilterValue](#) (void)

This API retrieves the value of ADC Filter register of selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

- uint16_t [ADC_GetPreviousResult](#) (void)

This API retrieves the value of ADC Previous register of selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

- void [ADC_DefineSetPoint](#) (uint16_t setPoint)

This API used to set value of ADC Threshold Set-point of selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

- void [ADC_SetUpperThreshold](#) (uint16_t upperThreshold)

This API used to set value of ADC Upper Threshold register of selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

- void [ADC_SetLowerThreshold](#) (uint16_t lowerThreshold)

This API used to set value of ADC Lower Threshold register of selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

- uint16_t [ADC_GetErrorCalculation](#) (void)

This API retrieves the value of ADC Set-point Error register of selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

- void [ADC_EnableDoubleSampling](#) (void)

This API enables double-sampling bit for selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

- void [ADC_EnableContinuousConversion](#) (void)

This API enables continuous conversion for selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

- void [ADC_DisableContinuousConversion](#) (void)

This API disables continuous conversion for selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

- bool [ADC_HasErrorCrossedUpperThreshold](#) (void)

This API is used to determine if ADC error has crossed the upper threshold for selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

- bool [ADC_HasErrorCrossedLowerThreshold](#) (void)

This API is used to determine if ADC error is less than the lower threshold for selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

- uint8_t [ADC_GetConversionStageStatus](#) (void)

This API is used retrieve the multi-stage status for selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

- void [ADC_EnableChargePump](#) (void)

This API is used to enable ADC charge pump.

- void [ADC_DisableChargePump](#) (void)

This API is used to disable ADC charge pump.

- void [__interrupt](#) (irq(AD), base(8))

- void [__interrupt](#) (irq(ACT), base(8))

- void [__interrupt](#) (irq(ADCH1), base(8))

- void [ADC_SetADInterruptHandler](#) (void(*InterruptHandler)(void))

Setter for A/D conversion ISR.

- void [ADC_SetContext1ThresholdInterruptHandler](#) (void(*InterruptHandler)(void))

Setter for Context-1 threshold interrupt.

- void [ADC_SetActiveClockTuningInterruptHandler](#) (void(*InterruptHandler)(void))

Setter for A/D Active Clock Tuning ISR.

6.76.1 Function Documentation

6.76.1.1 [__interrupt\(\)](#) [1/3]

```
void __interrupt (
    irq(ACT) ,
    base(8) )
```

6.76.1.2 [__interrupt\(\)](#) [2/3]

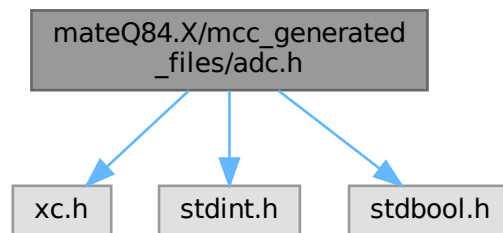
```
void __interrupt (
    irq(AD) ,
    base(8) )
```

6.76.1.3 [__interrupt\(\)](#) [3/3]

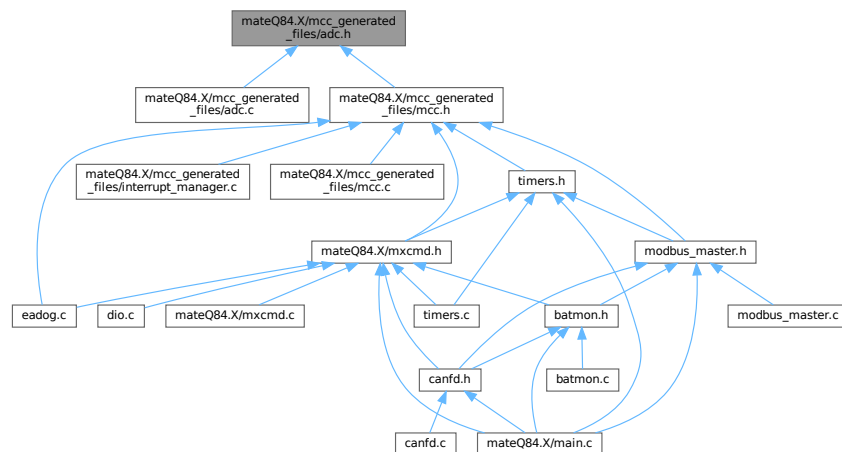
```
void __interrupt (
    irq(ADCH1) ,
    base(8) )
```

6.77 mateQ84.X/mcc_generated_files/adc.h File Reference

```
#include <xc.h>
#include <stdint.h>
#include <stdbool.h>
Include dependency graph for adc.h:
```



This graph shows which files directly or indirectly include this file:



Typedefs

- typedef uint16_t [adc_result_t](#)
This typedef should be used for result of A/D conversion.

Enumerations

- enum [ADC_channel_t](#) {
channel_ANA0 = 0x0 , channel_ANA4 = 0x4 , channel_VSS = 0x3B , channel_Temp = 0x3C ,
channel_DAC1 = 0x3D , channel_FVR_Buffer1 = 0x3E , channel_FVR_Buffer2 = 0x3F }
This enumeration contains available ADC channels.
- enum [ADC_context_t](#) { [mTouchContext](#) }
This enumeration contains available ADC contexts.

Functions

- void [ADC_Initialize](#) (void)
This API initializes the ADC module. This routine must be called before any other ADC routine.
- void [ADC_EnableChannelSequencer](#) (void)
This API enables ADC channel sequencer. ADC module should be initialized using [ADC_Initialize](#) before calling this API.
- void [ADC_DisableChannelSequencer](#) (void)
This API disables ADC channel sequencer. ADC module should be initialized using [ADC_Initialize](#) before calling this API.
- void [ADC_StartChannelSequencer](#) (void)
This API starts the channel sequencer by enabling GO bit. Channel sequencer should be enabled using [ADC_EnableChannelSequencer](#) before calling this API.
- void [ADC_SelectContext](#) ([ADC_context_t](#) context)
This API is used to select an ADC context to perform read/write operations on context specific registers.
- void [ADC_EnableChannelScan](#) ([ADC_context_t](#) context)
This API is used to include a channel context in the scan sequence. Correct context should be selected using [ADC_SelectContext](#) before calling this API.
- void [ADC_DisableChannelScan](#) ([ADC_context_t](#) context)
This API is used to exclude a channel context from the scan sequence. Correct context should be selected using [ADC_SelectContext](#) before calling this API.
- void [ADC_StartConversion](#) ([ADC_channel_t](#) channel)
This API starts A/D conversion on selected channel. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.
- bool [ADC_IsConversionDone](#) (void)
This API checks if ongoing A/D conversion is complete.
- [adc_result_t](#) [ADC_GetConversionResult](#) (void)
This API is used to retrieve the result of latest A/D conversion. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.
- [adc_result_t](#) [ADC_GetSingleConversion](#) ([ADC_channel_t](#) channel)
This API is used to retrieve the result of single A/D conversion on given channel. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.
- void [ADC_StopConversion](#) (void)
This API is used to stop ongoing A/D conversion. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.
- void [ADC_SetStopOnInterrupt](#) (void)
This API is used enable Stop On Interrupt bit for selected A/D context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.
- void [ADC_DischargeSampleCapacitor](#) (void)
This API is used to discharge input sample capacitor for selected context by setting the channel to AVss. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.
- void [ADC_LoadAcquisitionRegister](#) ([uint16_t](#) acquisitionValue)
This API is used to load ADC Acquisition Time Control register with specified value. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.
- void [ADC_SetPrechargeTime](#) ([uint16_t](#) prechargeTime)
This API is used to load ADC Precharge Time Control register with specified value. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.
- void [ADC_SetRepeatCount](#) ([uint8_t](#) repeatCount)
This API is used to load repeat counter for given context with specified value. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.
- [uint8_t](#) [ADC_GetCurrentCountofConversions](#) (void)
This API retrieves the current value of ADC Repeat Count register for selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.
- void [ADC_ClearAccumulator](#) (void)

- This API clears the accumulator of selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.*
- `uint24_t ADC_GetAccumulatorValue` (void)
This API retrieves 18-bit value of ADC accumulator of selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.
 - `bool ADC_HasAccumulatorOverflowed` (void)
This API is used to determine whether ADC accumulator has overflowed for selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.
 - `uint16_t ADC_GetFilterValue` (void)
This API retrieves the value of ADC Filter register of selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.
 - `uint16_t ADC_GetPreviousResult` (void)
This API retrieves the value of ADC Previous register of selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.
 - `void ADC_DefineSetPoint` (uint16_t setPoint)
This API used to set value of ADC Threshold Set-point of selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.
 - `void ADC_SetUpperThreshold` (uint16_t upperThreshold)
This API used to set value of ADC Upper Threshold register of selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.
 - `void ADC_SetLowerThreshold` (uint16_t lowerThreshold)
This API used to set value of ADC Lower Threshold register of selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.
 - `uint16_t ADC_GetErrorCalculation` (void)
This API retrieves the value of ADC Set-point Error register of selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.
 - `void ADC_EnableDoubleSampling` (void)
This API enables double-sampling bit for selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.
 - `void ADC_EnableContinuousConversion` (void)
This API enables continuous conversion for selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.
 - `void ADC_DisableContinuousConversion` (void)
This API disables continuous conversion for selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.
 - `bool ADC_HasErrorCrossedUpperThreshold` (void)
This API is used to determine if ADC error has crossed the upper threshold for selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.
 - `bool ADC_HasErrorCrossedLowerThreshold` (void)
This API is used to determine if ADC error is less than the lower threshold for selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.
 - `uint8_t ADC_GetConversionStageStatus` (void)
This API is used retrieve the multi-stage status for selected context. Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.
 - `void ADC_EnableChargePump` (void)
This API is used to enable ADC charge pump.
 - `void ADC_DisableChargePump` (void)
This API is used to disable ADC charge pump.
 - `void ADC_SetADInterruptHandler` (void(*InterruptHandler)(void))
Setter for A/D conversion ISR.
 - `void ADC_SetActiveClockTuningInterruptHandler` (void(*InterruptHandler)(void))
Setter for A/D Active Clock Tuning ISR.
 - `void ADC_SetContext1ThresholdInterruptHandler` (void(*InterruptHandler)(void))
Setter for Context-1 threshold interrupt.

- void [ADC_SetContext2ThresholdInterruptHandler](#) (void(*InterruptHandler)(void))
Setter for Context-2 threshold interrupt.
- void [ADC_SetContext3ThresholdInterruptHandler](#) (void(*InterruptHandler)(void))
Setter for Context-3 threshold interrupt.
- void [ADC_SetContext4ThresholdInterruptHandler](#) (void(*InterruptHandler)(void))
Setter for Context-4 threshold interrupt.

6.78 adc.h

[Go to the documentation of this file.](#)

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029     party
00030     license terms applicable to your use of third party software (including open source software) that
00031
00032     may accompany Microchip software.
00033
00034     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00035     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00036     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00037     FOR A PARTICULAR PURPOSE.
00038
00039     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00040     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00041     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00042     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00043     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00044     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00045     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00046     SOFTWARE.
00047 */
00048 #ifndef ADC_H
00049 #define ADC_H
00050
00051 #include <xc.h>
00052 #include <stdint.h>
00053 #include <stdbool.h>
00054
00055 typedef uint16_t adc_result_t;
00056
00057 typedef enum
00058 {
00059     channel_ANA0 = 0x0,
00060     channel_ANA4 = 0x4,
00061     channel_VSS = 0x3B,
00062     channel_Temp = 0x3C,
00063     channel_DAC1 = 0x3D,
00064     channel_FVR_Buffer1 = 0x3E,
00065     channel_FVR_Buffer2 = 0x3F
00066 } ADC_channel_t;
00067
00068 typedef enum
00069 {
00070     mTouchContext,
00071 } ADC_context_t;
00072
00073 void ADC_Initialize(void);
00074
00075 inline void ADC_EnableChannelSequencer(void);
00076
00077 inline void ADC_DisableChannelSequencer(void);
00078
00079 inline void ADC_StartChannelSequencer(void);
00080
00081 inline void ADC_SelectContext(ADC_context_t context);
00082
00083 void ADC_EnableChannelScan(ADC_context_t context);
00084
00085 void ADC_DisableChannelScan(ADC_context_t context);
00086

```

```

00295 void ADC_StartConversion(ADC_channel_t channel);
00296
00321 bool ADC_IsConversionDone(void);
00322
00347 adc_result_t ADC_GetConversionResult(void);
00348
00373 adc_result_t ADC_GetSingleConversion(ADC_channel_t channel);
00374
00398 inline void ADC_StopConversion(void);
00399
00422 inline void ADC_SetStopOnInterrupt(void);
00423
00450 inline void ADC_DischargeSampleCapacitor(void);
00451
00474 void ADC_LoadAcquisitionRegister(uint16_t acquisitionValue);
00475
00498 void ADC_SetPrechargeTime(uint16_t prechargeTime);
00499
00522 inline void ADC_SetRepeatCount(uint8_t repeatCount);
00523
00546 uint8_t ADC_GetCurrentCountofConversions(void);
00547
00569 inline void ADC_ClearAccumulator(void);
00570
00593 uint24_t ADC_GetAccumulatorValue(void);
00594
00620 bool ADC_HasAccumulatorOverflowed(void);
00621
00644 uint16_t ADC_GetFilterValue(void);
00645
00668 uint16_t ADC_GetPreviousResult(void);
00669
00692 void ADC_DefineSetPoint(uint16_t setPoint);
00693
00716 void ADC_SetUpperThreshold(uint16_t upperThreshold);
00717
00740 void ADC_SetLowerThreshold(uint16_t lowerThreshold);
00741
00765 uint16_t ADC_GetErrorCalculation(void);
00766
00789 inline void ADC_EnableDoubleSampling(void);
00790
00813 inline void ADC_EnableContinuousConversion(void);
00814
00837 inline void ADC_DisableContinuousConversion(void);
00838
00864 bool ADC_HasErrorCrossedUpperThreshold(void);
00865
00891 bool ADC_HasErrorCrossedLowerThreshold(void);
00892
00916 uint8_t ADC_GetConversionStageStatus(void);
00917
00937 inline void ADC_EnableChargePump(void);
00938
00958 inline void ADC_DisableChargePump(void);
00959
00960
00987 void ADC_SetADIIInterruptHandler(void (* InterruptHandler)(void));
00988
00989
01016 void ADC_SetActiveClockTuningInterruptHandler(void (* InterruptHandler)(void));
01017
01018
01045 void ADC_SetContext1ThresholdInterruptHandler(void (* InterruptHandler)(void));
01046
01047
01074 void ADC_SetContext2ThresholdInterruptHandler(void (* InterruptHandler)(void));
01075
01076
01103 void ADC_SetContext3ThresholdInterruptHandler(void (* InterruptHandler)(void));
01104
01105
01132 void ADC_SetContext4ThresholdInterruptHandler(void (* InterruptHandler)(void));
01133
01137 #endif //ADC_H

```

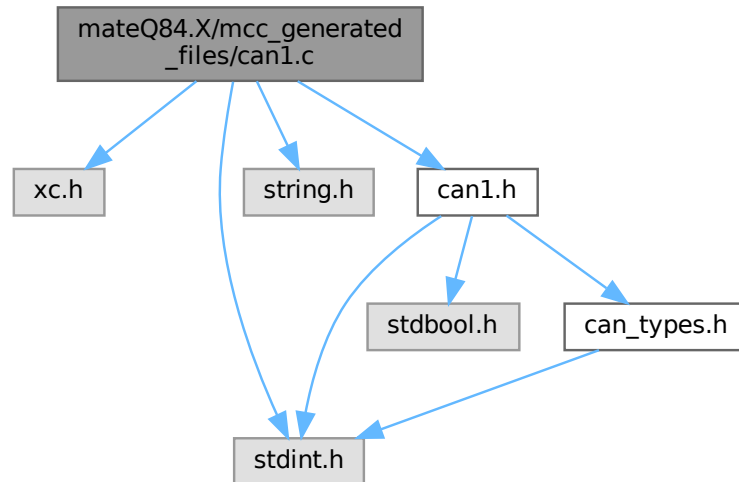
6.79 mateQ84.X/mcc_generated_files/can1.c File Reference

```

#include <xc.h>
#include <stdint.h>

```

```
#include <string.h>
#include "can1.h"
Include dependency graph for can1.c:
```



Data Structures

- struct [CAN_FIFOREG](#)
- struct [CAN1_RX_FIFO](#)

Macros

- `#define` [RX_FIFO_MSG_DATA](#) (64U)
- `#define` [NUM_OF_RX_FIFO](#) (1U)
- `#define` [SID_LOW_WIDTH](#) (8U)
- `#define` [SID_HIGH_MASK](#) (0x07U)
- `#define` [EID_LOW_WIDTH](#) (5U)
- `#define` [EID_LOW_POSN](#) (3U)
- `#define` [EID_LOW_MASK](#) (0xF8U)
- `#define` [EID_MID_WIDTH](#) (8U)
- `#define` [EID_HIGH_WIDTH](#) (5U)
- `#define` [EID_HIGH_MASK](#) (0x1FU)
- `#define` [IDE_POSN](#) (4U)
- `#define` [RTR_POSN](#) (5U)
- `#define` [BRS_POSN](#) (6U)
- `#define` [FDF_POSN](#) (7U)
- `#define` [DLCToPayloadBytes](#)(x)
- `#define` [PLSIZEToPayloadBytes](#)(x)

Enumerations

- enum [CAN_RX_FIFO_STATUS](#) { [CAN_RX_MSG_NOT_AVAILABLE](#) = 0U , [CAN_RX_MSG_AVAILABLE](#) = 1U , [CAN_RX_MSG_OVERFLOW](#) = 8U }

Functions

- void [CAN1_RX_FIFO_ResetInfo](#) (void)
- void [CAN1_Initialize](#) (void)
- [CAN_OP_MODE_STATUS](#) [CAN1_OperationModeSet](#) (const [CAN_OP_MODES](#) requestMode)
- [CAN_OP_MODES](#) [CAN1_OperationModeGet](#) (void)
- bool [CAN1_Receive](#) ([CAN_MSG_OBJ](#) *rxCanMsg)
- bool [CAN1_ReceiveFrom](#) (const [CAN1_RX_FIFO_CHANNELS](#) channel, [CAN_MSG_OBJ](#) *rxCanMsg)
- uint8_t [CAN1_ReceivedMessageCountGet](#) (void)
- [CAN_TX_MSG_REQUEST_STATUS](#) [CAN1_Transmit](#) (const [CAN1_TX_FIFO_CHANNELS](#) fifoChannel, [CAN_MSG_OBJ](#) *txCanMsg)
- [CAN_TX_FIFO_STATUS](#) [CAN1_TransmitFIFOStatusGet](#) (const [CAN1_TX_FIFO_CHANNELS](#) fifoChannel)
- bool [CAN1_IsBusOff](#) (void)
- bool [CAN1_IsRxErrorPassive](#) (void)
- bool [CAN1_IsRxErrorWarning](#) (void)
- bool [CAN1_IsRxErrorActive](#) (void)
- bool [CAN1_IsTxErrorPassive](#) (void)
- bool [CAN1_IsTxErrorWarning](#) (void)
- bool [CAN1_IsTxErrorActive](#) (void)
- void [CAN1_Sleep](#) (void)
- void [CAN1_SetInvalidMessageInterruptHandler](#) (void(*handler)(void))
- void [CAN1_SetBusWakeUpActivityInterruptHandler](#) (void(*handler)(void))
- void [CAN1_SetBusErrorInterruptHandler](#) (void(*handler)(void))
- void [CAN1_SetModeChangeInterruptHandler](#) (void(*handler)(void))
- void [CAN1_SetSystemErrorInterruptHandler](#) (void(*handler)(void))
- void [CAN1_SetTxAttemptInterruptHandler](#) (void(*handler)(void))
- void [CAN1_SetRxBufferOverflowInterruptHandler](#) (void(*handler)(void))
- void [__interrupt](#) (irq(CAN), base(8))
- void [CAN1_SetFIFO1NotEmptyHandler](#) (void(*handler)(void))
- void [CAN1_SetTXQNotFullHandler](#) (void(*handler)(void))
- void [__interrupt](#) (irq(CANRX), base(8))
- void [__interrupt](#) (irq(CANTX), base(8))

6.79.1 Macro Definition Documentation

6.79.1.1 BRS_POSN

```
#define BRS_POSN (6U)
```

6.79.1.2 DLCToPayloadBytes

```
#define DLCToPayloadBytes(  
    x)
```

Value:

```
(DLC_BYTES[(x)])
```

6.79.1.3 EID_HIGH_MASK

```
#define EID_HIGH_MASK (0x1FU)
```

6.79.1.4 EID_HIGH_WIDTH

```
#define EID_HIGH_WIDTH (5U)
```

6.79.1.5 EID_LOW_MASK

```
#define EID_LOW_MASK (0xF8U)
```

6.79.1.6 EID_LOW_POSN

```
#define EID_LOW_POSN (3U)
```

6.79.1.7 EID_LOW_WIDTH

```
#define EID_LOW_WIDTH (5U)
```

6.79.1.8 EID_MID_WIDTH

```
#define EID_MID_WIDTH (8U)
```

6.79.1.9 FDF_POSN

```
#define FDF_POSN (7U)
```

6.79.1.10 IDE_POSN

```
#define IDE_POSN (4U)
```

6.79.1.11 NUM_OF_RX_FIFO

```
#define NUM_OF_RX_FIFO (1U)
```

6.79.1.12 PLSIZEToPayloadBytes

```
#define PLSIZEToPayloadBytes(  
    x)
```

Value:

```
(DLCToPayloadBytes(8u + (x)))
```

6.79.1.13 RTR_POSN

```
#define RTR_POSN (5U)
```

6.79.1.14 RX_FIFO_MSG_DATA

```
#define RX_FIFO_MSG_DATA (64U)
```

CAN1 Generated Driver File

@Company Microchip Technology Inc.

@File Name [can1.c](#)

@Summary This is the generated driver implementation file for the CAN1 driver using PIC10 / PIC12 / PIC16 / PIC18 MCUs

@Description This header file provides implementations for driver APIs for CAN1. Generation Information : Product Revision : PIC10 / PIC12 / PIC16 / PIC18 MCUs - 1.81.8 Device : PIC18F47Q84 Driver Version : 1.1.1 The generated drivers are tested against the following: Compiler : XC8 2.36 and above or later MPLAB : MPLAB X 6.00

6.79.1.15 SID_HIGH_MASK

```
#define SID_HIGH_MASK (0x07U)
```

6.79.1.16 SID_LOW_WIDTH

```
#define SID_LOW_WIDTH (8U)
```

6.79.2 Enumeration Type Documentation

6.79.2.1 CAN_RX_FIFO_STATUS

```
enum CAN\_RX\_FIFO\_STATUS
```

Enumerator

CAN_RX_MSG_NOT_AVAILABLE	
CAN_RX_MSG_AVAILABLE	
CAN_RX_MSG_OVERFLOW	

6.79.3 Function Documentation

6.79.3.1 __interrupt() [1/3]

```
void __interrupt (
    irq(CAN) ,
    base(8) )
```

6.79.3.2 __interrupt() [2/3]

```
void __interrupt (
    irq(CANRX) ,
    base(8) )
```

6.79.3.3 __interrupt() [3/3]

```
void __interrupt (
    irq(CANTX) ,
    base(8) )
```

6.79.3.4 CAN1_Initialize()

```
void CAN1_Initialize (
    void )
```

Section: CAN Module APIs @Summary Initialization routine that takes inputs from the CAN GUI.

@Description This routine initializes the CAN module. This routine must be called before any other CAN routine is called. This routine should only be called once during system initialization.

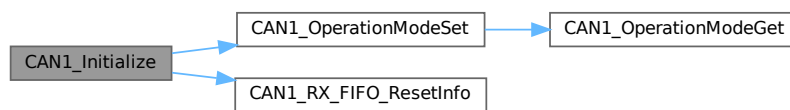
@Preconditions None

@Param None

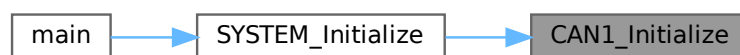
@Returns None

@Example void main(void) { SYSTEM_Initialize();

```
while (1) { User Application code }
} Here is the call graph for this function:
```



Here is the caller graph for this function:



6.79.3.5 CAN1_IsBusOff()

```
bool CAN1_IsBusOff (
    void )
```

@Summary Checks whether the transmitter is in bus off state.

@Description This routine checks whether the transmitter is in bus off state.

@Preconditions CAN1_Initialize function should be called before calling this function.

@Param None

@Returns true - Transmitter in Bus Off state false - Transmitter not in Bus Off state

@Example #define CAN_TX_BUFF "TEMPERATURE SENSOR ON"

```
void main(void) { CAN_MSG_OBJ msg; uint8_t data[32] = CAN_TX_BUFF;

SYSTEM_Initialize();

if(CAN_CONFIGURATION_MODE == CAN1_OperationModeGet()) { if(CAN_OP_MODE_↵
REQUEST_SUCCESS == CAN1_OperationModeSet(CAN_NORMAL_FD_MODE)) { msg.msgId =
0x1FFFF; msg.field.formatType = CAN_FD_FORMAT; msg.field.brs = CAN_NON_BRS_↵
_MODE; msg.field.frameType = CAN_FRAME_DATA; msg.field.idType = CAN_FRAME_↵
EXT; msg.field.dlc = DLC_32; msg.data = data;

if(CAN1_IsBusOff() == false) { if(CAN_TX_FIFO_AVAILABLE == (CAN1_Transmit_↵
FIFOStatusGet(CAN1_TX_FIFO1) & CAN_TX_FIFO_AVAILABLE)) { CAN1_Transmit(↵
CAN1_TX_FIFO1, &msg); } } } }

while(1); }
```

6.79.3.6 CAN1_IsRxErrorActive()

```
bool CAN1_IsRxErrorActive (
    void )
```

@Summary Checks whether the Receiver is in the error active state.

@Description This routine checks whether the receive is in the error active state. If Receiver error counter is above 0 to below 128, then receiver error active state is set.

@Preconditions CAN1_Initialize function should be called before calling this function.

@Param None

@Returns true - Receiver in Error active state false - Receiver not in Error active state

@Example void main(void) { CAN_MSG_OBJ msg;

```
SYSTEM_Initialize();

if(CAN_CONFIGURATION_MODE == CAN1_OperationModeGet()) { if(CAN_OP_MODE_↵
REQUEST_SUCCESS == CAN1_OperationModeSet(CAN_NORMAL_FD_MODE)) { while(1) {
```

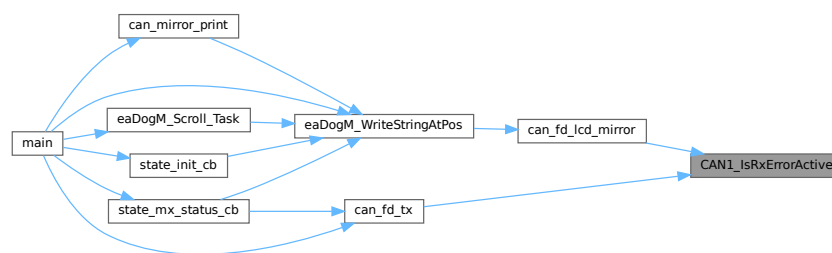


```
if(CAN1_IsRxErrorActive() == false) { if(CAN1_ReceivedMessageCountGet() > 0) { CAN1_Receive(&msg); } } } }
```

Here is the call graph for this function:



Here is the caller graph for this function:



6.79.3.7 CAN1_IsRxErrorPassive()

```
bool CAN1_IsRxErrorPassive (
    void )
```

@Summary Checks whether the Receiver is in the error passive state.

@Description This routine checks whether the receiver is in the error passive state. If Receiver error counter is above 127, then receiver error passive state is set.

@Preconditions CAN1_Initialize function should be called before calling this function.

@Param None

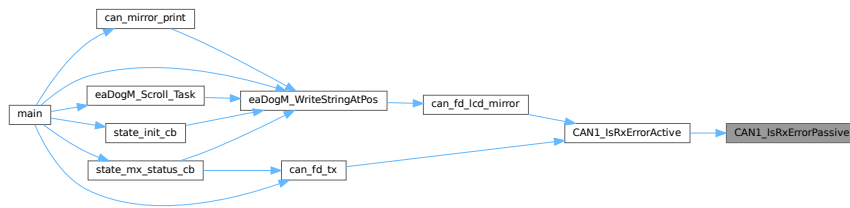
@Returns true - Receiver in Error passive state false - Receiver not in Error passive state

@Example void main(void) { CAN_MSG_OBJ msg;

```
SYSTEM_Initialize();
```

```
if(CAN_CONFIGURATION_MODE == CAN1_OperationModeGet()) { if(CAN_OP_MODE_↔
REQUEST_SUCCESS == CAN1_OperationModeSet(CAN_NORMAL_FD_MODE)) { while(1) {
```

```
if(CAN1_IsRxErrorPassive() == false) { if(CAN1_ReceivedMessageCountGet() > 0) { CAN1_Receive(&msg); } } } } } Here is the caller graph for this function:
```



6.79.3.8 CAN1_IsRxErrorWarning()

```
bool CAN1_IsRxErrorWarning (
    void )
```

@Summary Checks whether the Receiver is in the error warning state.

@Description This routine checks whether the receive is in the error warning state. If Receiver error counter is above 95 to below 128, then receiver error warning state is set.

@Preconditions CAN1_Initialize function should be called before calling this function.

@Param None

@Returns true - Receiver in Error warning state false - Receiver not in Error warning state

@Example void main(void) { CAN_MSG_OBJ msg;

```
SYSTEM_Initialize();
```

```
if(CAN_CONFIGURATION_MODE == CAN1_OperationModeGet()) { if(CAN_OP_MODE_←
REQUEST_SUCCESS == CAN1_OperationModeSet(CAN_NORMAL_FD_MODE)) { while(1) {
if(CAN1_IsRxErrorWarning() == false) { if(CAN1_ReceivedMessageCountGet() >
0) { CAN1_Receive(&msg); } } } } } }
```

6.79.3.9 CAN1_IsTxErrorActive()

```
bool CAN1_IsTxErrorActive (
    void )
```

@Summary Checks whether the transmitter is in the error active state.

@Description This routine checks whether the transmitter is in the error active state. If Transmitter error counter is above 0 to below 128, then transmitter error active state is set.

@Preconditions CAN1_Initialize function should be called before calling this function.

@Param None

@Returns true - Transmitter in Error active state false - Transmitter not in Error active state

@Example #define CAN_TX_BUFF "TEMPERATURE SENSOR ON"

```
void main(void) { CAN_MSG_OBJ msg; uint8_t data[32] = CAN_TX_BUFF;
```

```
SYSTEM_Initialize();
```

```
if(CAN_CONFIGURATION_MODE == CAN1_OperationModeGet()) { if(CAN_OP_MODE_↵
REQUEST_SUCCESS == CAN1_OperationModeSet(CAN_NORMAL_FD_MODE)) { msg.msgId =
0x1FFFF; msg.field.formatType = CAN_FD_FORMAT; msg.field.brs = CAN_NON_BRS_↵
_MODE; msg.field.frameType = CAN_FRAME_DATA; msg.field.idType = CAN_FRAME_↵
EXT; msg.field.dlc = DLC_32; msg.data = data;
```

```
if(CAN1_IsTxErrorActive() == false) { if(CAN_TX_FIFO_AVAILABLE == (CAN1_↵
_TransmitFIFOStatusGet(CAN1_TX_FIFO1) & CAN_TX_FIFO_AVAILABLE)) { CAN1_↵
Transmit(CAN1_TX_FIFO1, &msg); } } }
```

while(1); } Here is the call graph for this function:



6.79.3.10 CAN1_IsTxErrorPassive()

```
bool CAN1_IsTxErrorPassive (
    void )
```

@Summary Checks whether the transmitter is in the error passive state.

@Description This routine checks whether the transmitter is in the error passive state. If Transmitter error counter is above 127, then the transmitter error passive state is set.

@Preconditions CAN1_Initialize function should be called before calling this function.

@Param None

@Returns true - Transmitter in Error Passive state false - Transmitter not in Error Passive state

@Example #define CAN_TX_BUFF "TEMPERATURE SENSOR ON"

```
void main(void) { CAN_MSG_OBJ msg; uint8_t data[32] = CAN_TX_BUFF;
```

```
SYSTEM_Initialize();
```

```
if(CAN_CONFIGURATION_MODE == CAN1_OperationModeGet()) { if(CAN_OP_MODE_↵
REQUEST_SUCCESS == CAN1_OperationModeSet(CAN_NORMAL_FD_MODE)) { msg.msgId =
0x1FFFF; msg.field.formatType = CAN_FD_FORMAT; msg.field.brs = CAN_NON_BRS_↵
_MODE; msg.field.frameType = CAN_FRAME_DATA; msg.field.idType = CAN_FRAME_↵
EXT; msg.field.dlc = DLC_32; msg.data = data;
```

```
if(CAN1_IsTxErrorPassive() == false) { if(CAN_TX_FIFO_AVAILABLE == (CAN1_
_TransmitFIFOStatusGet(CAN1_TX_FIFO1) & CAN_TX_FIFO_AVAILABLE)) { CAN1_
Transmit(CAN1_TX_FIFO1, &msg); } } }
```

while(1); } Here is the caller graph for this function:



6.79.3.11 CAN1_IsTxErrorWarning()

```
bool CAN1_IsTxErrorWarning (
    void )
```

@Summary Checks whether the transmitter is in the error warning state.

@Description This routine checks whether the transmitter is in the error warning state. If Transmitter error counter is above 95 to below 128, then transmitter error warning state is set.

@Preconditions CAN1_Initialize function should be called before calling this function.

@Param None

@Returns true - Transmitter in Error warning state false - Transmitter not in Error warning state

@Example #define CAN_TX_BUFF "TEMPERATURE SENSOR ON"

```
void main(void) { CAN_MSG_OBJ msg; uint8_t data[32] = CAN_TX_BUFF;
```

```
SYSTEM_Initialize();
```

```
if(CAN_CONFIGURATION_MODE == CAN1_OperationModeGet()) { if(CAN_OP_MODE_
REQUEST_SUCCESS == CAN1_OperationModeSet(CAN_NORMAL_FD_MODE)) { msg.msgId =
0x1FFFF; msg.field.formatType = CAN_FD_FORMAT; msg.field.brs = CAN_NON_BRS_
_MODE; msg.field.frameType = CAN_FRAME_DATA; msg.field.idType = CAN_FRAME_
EXT; msg.field.dlc = DLC_32; msg.data = data;
```

```
if(CAN1_IsTxErrorWarning() == false) { if(CAN_TX_FIFO_AVAILABLE == (CAN1_
_TransmitFIFOStatusGet(CAN1_TX_FIFO1) & CAN_TX_FIFO_AVAILABLE)) { CAN1_
Transmit(CAN1_TX_FIFO1, &msg); } } }
```

```
while(1); }
```

6.79.3.12 CAN1_OperationModeGet()

```
CAN_OP_MODES CAN1_OperationModeGet (
    void )
```

@Summary Get the CAN operation mode.

@Description This function gets the CAN operation mode.

@Preconditions [CAN1_Initialize\(\)](#) function should be called before calling this function.

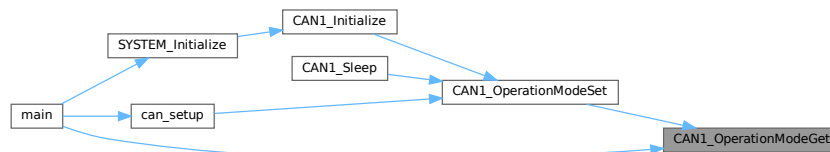
@Param None

@Returns Return the present CAN operation mode.

```
@Example void main(void) {
SYSTEM_Initialize();

if (CAN_CONFIGURATION_MODE == CAN1_OperationModeGet()) { if (CAN_OP_MODE_↵
REQUEST_SUCCESS == CAN1_OperationModeSet(CAN_NORMAL_FD_MODE)) { User Application
code } }

while (1) {
} } Here is the caller graph for this function:
```



6.79.3.13 CAN1_OperationModeSet()

```
CAN_OP_MODE_STATUS CAN1_OperationModeSet (
    const CAN_OP_MODES requestMode)
```

@Summary Set the CAN operation mode.

@Description This function sets the CAN operation mode.

@Preconditions [CAN1_Initialize\(\)](#) function should be called before calling this function.

@Param requestMode - CAN operation mode

@Returns CAN_OP_MODE_REQUEST_SUCCESS - Requested Operation mode set successfully CAN_OP_↵
 MODE_REQUEST_FAIL - Requested Operation mode set failure. Set configuration mode before setting CAN nor-
 mal or debug operation mode. CAN_OP_MODE_SYS_ERROR_OCCURED - System error occurred while setting
 Operation mode.

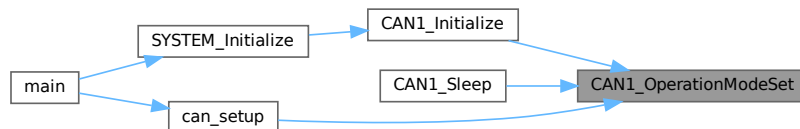
```
@Example void main(void) { SYSTEM_Initialize();
```

```
if (CAN_CONFIGURATION_MODE == CAN1_OperationModeGet ()) { if (CAN_OP_MODE_↵
REQUEST_SUCCESS == CAN1_OperationModeSet (CAN_NORMAL_FD_MODE)) { User Application
code } }
```

while (1) { } } Here is the call graph for this function:



Here is the caller graph for this function:



6.79.3.14 CAN1_Receive()

```
bool CAN1_Receive (
    CAN_MSG_OBJ * rxCanMsg)
```

@Summary Reads the message object from CAN receive FIFO.

@Description This routine reads a message object from the CAN receive FIFO.

@Preconditions [CAN1_Initialize\(\)](#) function should be called before calling this function. The [CAN1_ReceivedMessageCountGet\(\)](#) function should be checked to see if the receiver is not empty before calling this function.

@Param rxCanMsg - pointer to the message object

@Returns true - Receive successful false - Receive failure

@Example void [main\(void\)](#) { [CAN_MSG_OBJ](#) msg;

```
SYSTEM\_Initialize\(\);
```

```
if (CAN_CONFIGURATION_MODE == CAN1\_OperationModeGet\(\)) { if (CAN_OP_MODE_↵
REQUEST_SUCCESS == CAN1_OperationModeSet (CAN_NORMAL_FD_MODE)) { while(1) {
if (CAN1_ReceivedMessageCountGet () > 0) { if (true == CAN1_Receive(&msg)) {
break; } } } } }
```

```
while (1); }
```

6.79.3.15 CAN1_ReceivedMessageCountGet()

```
uint8_t CAN1_ReceivedMessageCountGet (
    void )
```

@Summary CAN RX FIFO number of messages that are received.

@Description This returns the number of messages that are received.

@Preconditions CAN1_Initialize function should be called before calling this function.

@Param None

@Returns Number of message received.

```
@Example void main(void) { CAN_MSG_OBJ msg;

SYSTEM_Initialize();

if (CAN_CONFIGURATION_MODE == CAN1_OperationModeGet()) { if (CAN_OP_MODE_↵
REQUEST_SUCCESS == CAN1_OperationModeSet(CAN_NORMAL_FD_MODE)) { while(1) {
if (CAN1_ReceivedMessageCountGet() > 0) { CAN1_Receive(&msg); } } } } }
```

6.79.3.16 CAN1_ReceiveFrom()

```
bool CAN1_ReceiveFrom (
    const CAN1_RX_FIFO_CHANNELS fifoChannel,
    CAN_MSG_OBJ * rxCanMsg)
```

@Summary Reads the message object from the specified CAN receive FIFO.

@Description This routine reads a message object from the specified CAN receive FIFO.

@Preconditions CAN1_Initialize() function should be called before calling this function.

@Param fifoChannel - CAN RX FIFO channel rxCanMsg - pointer to the message object

@Returns true - Receive successful false - Receive failure

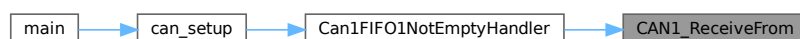
```
@Example volatile CAN_MSG_OBJ gMsg;
```

```
void CustomFIFO1Handler(void) { CAN1_ReceiveFrom(FIFO1, &gMsg); }
```

```
void main(void) { SYSTEM_Initialize(); CAN1_SetFIFO1FullHandler(&Custom↵
FIFO1Handler);
```

```
INTERRUPT_GlobalInterruptEnable();
```

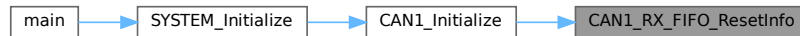
```
while(1); } Here is the caller graph for this function:
```



6.79.3.17 CAN1_RX_FIFO_ResetInfo()

```
void CAN1_RX_FIFO_ResetInfo (
    void )
```

Here is the caller graph for this function:



6.79.3.18 CAN1_SetBusErrorInterruptHandler()

```
void CAN1_SetBusErrorInterruptHandler (
    void(* handler ) (void))
```

@Summary Sets the CAN bus error interrupt handler.

@Description This routine sets the CAN bus error interrupt handler.

@Param Address of the callback routine.

@Returns None

@Example Note: Example code here is not based on MCC UI configuration, this is a sample code to demonstrate CAN transmit APIs usage.

```
volatile bool gBusErrorOccurred = false;
```

```
void CAN1_BusError(void)
{
    gBusErrorOccurred = true;
```

```
CAN Bus Error application code }
```

```
void main(void) { CAN_MSG_OBJ msg; uint8_t data[8] = {0x41,0x42,0x43,0x44,0x45,0x46,0x47,0x48};
```

```
SYSTEM_Initialize(); CAN1_SetBusErrorInterruptHandler(&CAN1_BusError); CAN1_
_OperationModeSet(CAN_CONFIGURATION_MODE);
```

```
if(CAN_CONFIGURATION_MODE == CAN1_OperationModeGet()) {
if(CAN_OP_MODE_REQUEST_SUCCESS == CAN1_OperationModeSet(CAN_NORMAL_FD_
MODE)) { msg.msgId = 0x1FFFF; msg.field.formatType = CAN_FD_FORMAT; msg.↵
field.brs = CAN_NON_BRS_MODE; msg.field.frameType = CAN_FRAME_DATA; msg.↵
field.idType = CAN_FRAME_EXT; msg.field.dlc = DLC_8; msg.data = data;
```

```
while(1) {
if(CAN_TX_FIFO_AVAILABLE == (CAN1_TransmitFIFOStatusGet(CAN1_TX_FIFO1) &
CAN_TX_FIFO_AVAILABLE)) { CAN1_Transmit(CAN1_TX_FIFO1, &msg); }
```

```
if(gBusErrorOccurred == true) { break; } } }
```

```
while(1); }
```


6.79.3.19 CAN1_SetBusWakeUpActivityInterruptHandler()

```
void CAN1_SetBusWakeUpActivityInterruptHandler (
    void(* handler ) (void))
```

@Summary Sets the CAN bus wake-Up activity interrupt handler.

@Description This routine sets the CAN bus wake-Up activity interrupt handler.

@Param Address of the callback routine.

@Returns None

@Example volatile bool gBusWakeUpOccurred = false;

```
void CAN1_BusWakeUpActivity(void) { gBusWakeUpOccurred = true; CAN Bus Wake↔
Up activity application code }
```

```
void main(void) { SYSTEM_Initialize(); CAN1_SetBusWakeUpActivityInterrupt↔
Handler(&CAN1_BusWakeUpActivity); CAN1_OperationModeSet(CAN_CONFIGURATION_↔
MODE);
```

```
if(CAN_CONFIGURATION_MODE == CAN1_OperationModeGet()) { if(CAN_OP_MODE_↔
REQUEST_SUCCESS == CAN1_OperationModeSet(CAN_NORMAL_FD_MODE)) { CAN1_Sleep();
```

```
Check CAN1 module is in CAN_DISABLE_MODE if(CAN_DISABLE_MODE == CAN1_OperationModeGet())
{ Sleep(); //Call sleep instruction
```

```
while(1) { if(gBusWakeUpOccurred == true) { break; }
} } }
```

```
while(1); }
```

6.79.3.20 CAN1_SetFIFO1NotEmptyHandler()

```
void CAN1_SetFIFO1NotEmptyHandler (
    void(* handler ) (void))
```

@Summary Sets the RX FIFO Not Empty interrupt handler.

@Description This routine sets the RX FIFO Not Empty interrupt handler for FIFO1.

@Param Address of the callback routine.

@Returns None

@Example volatile CAN_MSG_OBJ gMsg;

```
void CustomFIFO1Handler(void) { CAN1_ReceiveFrom(FIFO1, &gMsg); }
```

```
void main(void) { SYSTEM_Initialize(); CAN1_SetFIFO1NotEmptyHandler(&Custom↔
FIFO1Handler);
```

```
INTERRUPT_GlobalInterruptEnable();
```

```
while(1); } Here is the caller graph for this function:
```



6.79.3.21 CAN1_SetInvalidMessageInterruptHandler()

```
void CAN1_SetInvalidMessageInterruptHandler (
    void(* handler ) (void))
```

@Summary Sets the invalid message interrupt handler.

@Description This routine sets the invalid message interrupt handler.

@Param Address of the callback routine.

@Returns None

@Example Note: Example code here is not based on MCC UI configuration, this is a sample code to demonstrate CAN transmit APIs usage.

```
volatile bool gInvalidMsgOccurred = false;
```

```
void CAN1_InvalidMessage(void)
{
    gInvalidMsgOccurred = true;
```

```
    CAN Invalid Message application code }
```

```
void main(void) { CAN_MSG_OBJ msg; uint8_t data[8] = {0x41,0x42,0x43,0x44,0x45,0x46,0x47,0x48};
```

```
    SYSTEM_Initialize(); CAN1_SetInvalidMessageInterruptHandler(&CAN1_InvalidMessage); CAN1_OperationModeSet(CAN_CONFIGURATION_MODE);
```

```
    if(CAN_CONFIGURATION_MODE == CAN1_OperationModeGet()) {
        if(CAN_OP_MODE_REQUEST_SUCCESS == CAN1_OperationModeSet(CAN_NORMAL_FD_MODE)) { msg.msgId = 0x1FFFF; msg.field.formatType = CAN_FD_FORMAT; msg.field.brs = CAN_NON_BRS_MODE; msg.field.frameType = CAN_FRAME_DATA; msg.field.idType = CAN_FRAME_EXT; msg.field.dlc = DLC_8; msg.data = data;
```

```
        while(1) {
            if(CAN_TX_FIFO_AVAILABLE == (CAN1_TransmitFIFOStatusGet(CAN1_TX_FIFO1) & CAN_TX_FIFO_AVAILABLE)) { CAN1_Transmit(CAN1_TX_FIFO1, &msg); }
```

```
        if(gInvalidMsgOccurred == true) { break; } } }
```

```
    while(1); }
```

6.79.3.22 CAN1_SetModeChangeInterruptHandler()

```
void CAN1_SetModeChangeInterruptHandler (
    void(* handler ) (void))
```

@Summary Sets the CAN mode change interrupt handler.

@Description This routine sets the CAN mode change interrupt handler.

@Param Address of the callback routine.

@Returns None

@Example volatile bool gModeChangeOccurred = false;

```
void CAN1_ModeChange(void) { gModeChangeOccurred = true; CAN Mode Change application code }
```

```
void main(void) { CAN_MSG_OBJ msg;
```

```
    SYSTEM_Initialize(); CAN1_SetModeChangeInterruptHandler(&CAN1_ModeChange); CAN1_OperationModeSet(CAN_CONFIGURATION_MODE);
```

```
    if(CAN_CONFIGURATION_MODE == CAN1_OperationModeGet()) { if(CAN_OP_MODE_REQUEST_SUCCESS == CAN1_OperationModeSet(CAN_NORMAL_FD_MODE)) { while(1) { if(gModeChangeOccurred == true) { break; } } }
```

```
    while (1); }
```

6.79.3.23 CAN1_SetRxBufferOverFlowInterruptHandler()

```
void CAN1_SetRxBufferOverFlowInterruptHandler (
    void(* handler ) (void))
```

@Summary Sets the CAN receive overflow interrupt handler.

@Description This routine sets the CAN receive overflow interrupt handler.

@Param Address of the callback routine.

@Returns None

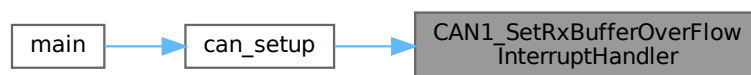
@Example `volatile bool gRxOverflowOccurred = false;`

```
void CAN1_RxBufferOverFlow(void) { gRxOverflowOccurred = true; CAN Receive
Buffer OverFlow application code }
```

```
void main(void) {
    SYSTEM_Initialize(); CAN1_SetRxBufferOverFlowInterruptHandler(&CAN1_Rx↵
BufferOverFlow); CAN1_OperationModeSet(CAN_CONFIGURATION_MODE);

    if(CAN_CONFIGURATION_MODE == CAN1_OperationModeGet()) {
        if(CAN_OP_MODE_REQUEST_SUCCESS == CAN1_OperationModeSet(CAN_NORMAL_FD_↵
MODE)) { while(1) { if(gRxOverflowOccurred == true) { gRxOverflowOccurred =
false; User Application code break;
        } } } }
```

`while(1); }` Here is the caller graph for this function:

**6.79.3.24 CAN1_SetSystemErrorInterruptHandler()**

```
void CAN1_SetSystemErrorInterruptHandler (
    void(* handler ) (void))
```

@Summary Sets the CAN system error interrupt handler.

@Description This routine sets the CAN system error interrupt handler.

@Param Address of the callback routine.

@Returns None

@Example Note: Example code here is not based on MCC UI configuration, this is a sample code to demonstrate CAN transmit APIs usage.

```

volatile bool gSystemOccurred = false;

void CAN1_SystemError(void)
{
    gSystemOccurred = true;

    CAN System Error application code }

void main(void) { CAN_MSG_OBJ msg; uint8_t data[8] = {0x41,0x42,0x43,0x44,0x45,0x46,0x47,0x48};
SYSTEM_Initialize(); CAN1_SetSystemErrorInterruptHandler(&CAN1_SystemError);
CAN1_OperationModeSet(CAN_CONFIGURATION_MODE);

if(CAN_CONFIGURATION_MODE == CAN1_OperationModeGet()) {
if(CAN_OP_MODE_REQUEST_SUCCESS == CAN1_OperationModeSet(CAN_NORMAL_FD_↵
MODE)) { msg.msgId = 0x1FFFF; msg.field.formatType = CAN_FD_FORMAT; msg.↵
field.brs = CAN_NON_BRS_MODE; msg.field.frameType = CAN_FRAME_DATA; msg.↵
field.idType = CAN_FRAME_EXT; msg.field.dlc = DLC_8; msg.data = data;

while(1) {
if(CAN_TX_FIFO_AVAILABLE == (CAN1_TransmitFIFOStatusGet(CAN1_TX_FIFO1) &
CAN_TX_FIFO_AVAILABLE)) { CAN1_Transmit(CAN1_TX_FIFO1, &msg); }

if(gSystemOccurred == true) { break; } }

} }

while(1); }

```

6.79.3.25 CAN1_SetTxAttemptInterruptHandler()

```

void CAN1_SetTxAttemptInterruptHandler (
    void(* handler )(void))

```

@Summary Sets the CAN transmit attempt interrupt handler.

@Description This routine sets the CAN transmit attempt interrupt handler.

@Param Address of the callback routine.

@Returns None

@Example Note: Example code here is not based on MCC UI configuration, this is a sample code to demonstrate CAN transmit APIs usage.

```

volatile bool gTxAttemptOccurred = false;

void CAN1_TxAttempt(void)
{
    gTxAttemptOccurred = true;

    CAN Transmit Attempt application code }

void main(void) { CAN_MSG_OBJ msg; uint8_t data[8] = {0x41,0x42,0x43,0x44,0x45,0x46,0x47,0x48};
SYSTEM_Initialize(); CAN1_SetTxAttemptInterruptHandler(&CAN1_TxAttempt);
CAN1_OperationModeSet(CAN_CONFIGURATION_MODE);

if(CAN_CONFIGURATION_MODE == CAN1_OperationModeGet()) {
if(CAN_OP_MODE_REQUEST_SUCCESS == CAN1_OperationModeSet(CAN_NORMAL_FD_↵
MODE)) { msg.msgId = 0x1FFFF; msg.field.formatType = CAN_FD_FORMAT; msg.↵
field.brs = CAN_NON_BRS_MODE; msg.field.frameType = CAN_FRAME_DATA; msg.↵
field.idType = CAN_FRAME_EXT; msg.field.dlc = DLC_8; msg.data = data;

while(1) {
if(CAN_TX_FIFO_AVAILABLE == (CAN1_TransmitFIFOStatusGet(CAN1_TX_FIFO1) &
CAN_TX_FIFO_AVAILABLE)) { CAN1_Transmit(CAN1_TX_FIFO1, &msg); }

if(gTxAttemptOccurred == true) { break; } } } }

while(1); }

```

6.79.3.26 CAN1_SetTXQNotFullHandler()

```
void CAN1_SetTXQNotFullHandler (
    void(* handler ) (void))
```

@Summary Sets the TXQ Not Full interrupt handler.

@Description This routine sets the TXQ Not Full interrupt handler for TXQ.

@Param Address of the callback routine.

@Returns None

@Example volatile CAN_MSG_OBJ gMsg;

```
void CustomTXQHandler(void) { CAN1_Transmit(CAN1_TX_FIFO1, &gMsg); }
```

```
void main(void) { uint8_t data[8] = {0x41,0x42,0x43,0x44,0x45,0x46,0x47,0x48};
gMsg.msgId = 0x1FFFF; gMsg.field.formatType = CAN_FD_FORMAT; gMsg.field.brs
= CAN_NON_BRS_MODE; gMsg.field.frameType = CAN_FRAME_DATA; gMsg.field.id←
Type = CAN_FRAME_EXT; gMsg.field.dlc = DLC_8; gMsg.data = data;
```

```
SYSTEM_Initialize(); CAN1_SetTXQNotFullHandler(&CustomTXQHandler);
```

```
INTERRUPT_GlobalInterruptEnable();
```

```
while(1); } Here is the caller graph for this function:
```

**6.79.3.27 CAN1_Sleep()**

```
void CAN1_Sleep (
    void )
```

@Summary Puts CAN module in sleep mode.

@Description This routine puts CAN module in sleep mode.

@Preconditions CAN1_Initialize function should be called before calling this function.

@Param None

@Returns None

```
@Example void main(void) {
SYSTEM_Initialize();
```

```

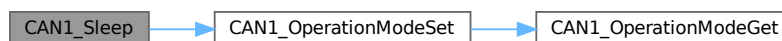
if(CAN_CONFIGURATION_MODE == CAN1_OperationModeGet()) { if(CAN_OP_MODE_↵
REQUEST_SUCCESS == CAN1_OperationModeSet(CAN_NORMAL_FD_MODE)) { CAN1_Sleep();

if(CAN_DISABLE_MODE == CAN1_OperationModeGet()) { Sleep(); //Call sleep
instruction C1INTLbits.WAKIF = 0; // Clear Wake-Up interrupt flag

Recover to Normal mode if(CAN_OP_MODE_REQUEST_SUCCESS == CAN1_Operation_↵
ModeSet(CAN_CONFIGURATION_MODE)) { if(CAN_OP_MODE_REQUEST_SUCCESS == CAN1_↵
_OperationModeSet(CAN_NORMAL_FD_MODE)) { User Application code
} } } } }

while(1); } Here is the call graph for this function:

```



6.79.3.28 CAN1_Transmit()

```

CAN_TX_MSG_REQUEST_STATUS CAN1_Transmit (
    const CAN1_TX_FIFO_CHANNELS fifoChannel,
    CAN_MSG_OBJ * txCanMsg)

```

@Summary Writes a message object to the CAN TX FIFO.

@Description This routine writes a message object to the CAN TX FIFO.

@Preconditions CAN1_Initialize function should be called before calling this function. The transfer status should be checked to see if transmitter is not full before calling this function.

@Param fifoChannel - CAN TX priority FIFO selection txCanMsg - pointer to the message object

@Returns CAN Transmit message status. CAN_TX_MSG_REQUEST_SUCCESS - Transmit message object successfully placed into transmit FIFO CAN_TX_MSG_REQUEST_DLC_EXCEED_ERROR - Transmit message object DLC size is more than transmit FIFO configured DLC size CAN_TX_MSG_REQUEST_BRS_ERROR - Transmit FIFO configured has Non BRS mode and CAN TX Message object has BRS enabled CAN_TX_MSG_REQUEST_↵_FIFO_FULL - Transmit FIFO Full

@Example #define CAN_TX_BUFF "TEMPERATURE SENSOR ON"

```

void main(void) { CAN_MSG_OBJ msg; uint8_t data[32] = CAN_TX_BUFF;

```

```

SYSTEM_Initialize();

```

```

if(CAN_CONFIGURATION_MODE == CAN1_OperationModeGet()) { if(CAN_OP_MODE_↵
REQUEST_SUCCESS == CAN1_OperationModeSet(CAN_NORMAL_FD_MODE)) { msg.msgId =
0xFFFF; msg.field.formatType = CAN_FD_FORMAT; msg.field.brs = CAN_NON_BRS_↵
_MODE; msg.field.frameType = CAN_FRAME_DATA; msg.field.idType = CAN_FRAME_↵
EXT; msg.field.dlc = DLC_32; msg.data = data;

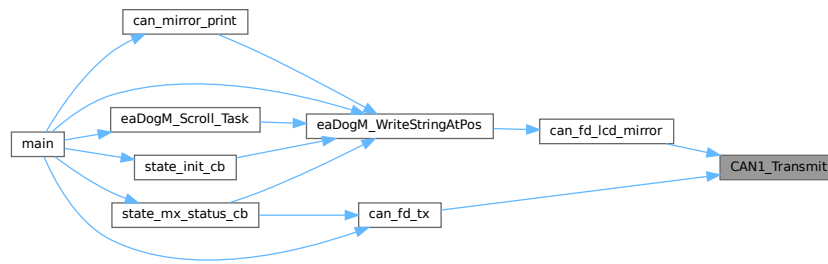
```

```

if(CAN_TX_FIFO_AVAILABLE == (CAN1_TransmitFIFOStatusGet(CAN1_TX_FIFO1) &
CAN_TX_FIFO_AVAILABLE)) { CAN1_Transmit(CAN1_TX_FIFO1, &msg); } } }

```

while(1); } Here is the caller graph for this function:



6.79.3.29 CAN1_TransmitFIFOStatusGet()

```

CAN_TX_FIFO_STATUS CAN1_TransmitFIFOStatusGet (
    const CAN1_TX_FIFO_CHANNELS fifoChannel)

```

@Summary CAN transmitter FIFO status.

@Description This returns the CAN transmitter FIFO status.

@Preconditions CAN1_Initialize function should be called before calling this function.

@Param fifoChannel - CAN TX priority FIFO selection

@Returns CAN Transmit FIFO status. CAN_TX_FIFO_FULL - CAN Transmit FIFO is full CAN_TX_FIFO_↔
AVAILABLE - CAN Transmit FIFO message space is available

@Example #define CAN_TX_BUFF "TEMPERATURE SENSOR ON"

```

void main(void) { CAN_MSG_OBJ msg; uint8_t data[32] = CAN_TX_BUFF;

```

```

SYSTEM_Initialize();

```

```

if(CAN_CONFIGURATION_MODE == CAN1_OperationModeGet()) {
if(CAN_OP_MODE_REQUEST_SUCCESS == CAN1_OperationModeSet(CAN_NORMAL_FD_↔
MODE)) { msg.msgId = 0x1FFFF; msg.field.formatType = CAN_FD_FORMAT; msg.↔
field.brs = CAN_NON_BRS_MODE; msg.field.frameType = CAN_FRAME_DATA; msg.↔
field.idType = CAN_FRAME_EXT; msg.field.dlc = DLC_32; msg.data = data;

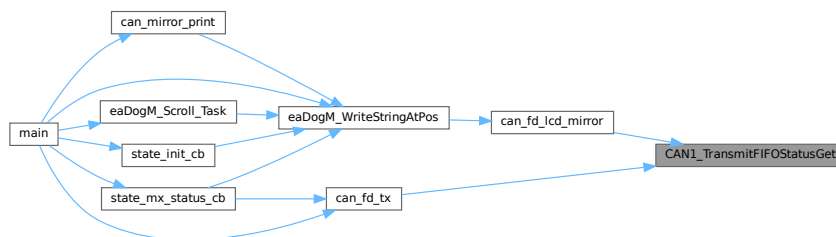
```

```

if(CAN_TX_FIFO_AVAILABLE == (CAN1_TransmitFIFOStatusGet(CAN1_TX_FIFO1) &
CAN_TX_FIFO_AVAILABLE)) { CAN1_Transmit(CAN1_TX_FIFO1, &msg); } } }

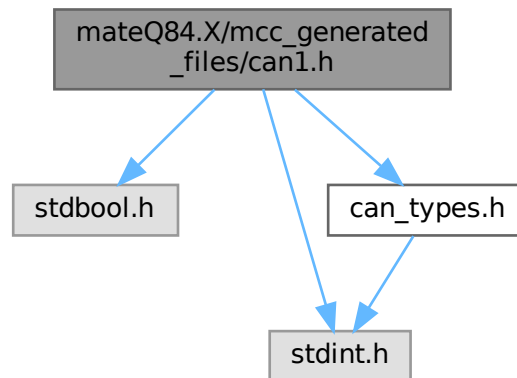
```

while(1); } Here is the caller graph for this function:

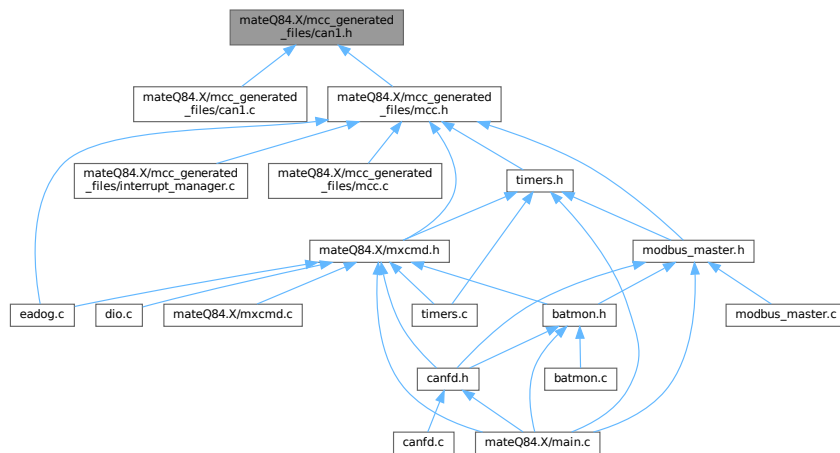


6.80 mateQ84.X/mcc_generated_files/can1.h File Reference

```
#include <stdbool.h>
#include <stdint.h>
#include "can_types.h"
Include dependency graph for can1.h:
```



This graph shows which files directly or indirectly include this file:



Macros

- `#define CAN1_TX_TXQ TXQ`
- `#define CAN1_TX_FIFO2 FIFO2`
- `#define CAN1_TX_FIFO3 FIFO3`

Enumerations

- enum `CAN1_TX_FIFO_CHANNELS` { `TXQ` = 0 , `FIFO2` = 2 , `FIFO3` = 3 }
- enum `CAN1_RX_FIFO_CHANNELS` { `FIFO1` = 1 }

Functions

- void `CAN1_Initialize` (void)
- `CAN_OP_MODE_STATUS` `CAN1_OperationModeSet` (const `CAN_OP_MODES` requestMode)
- `CAN_OP_MODES` `CAN1_OperationModeGet` (void)
- bool `CAN1_Receive` (`CAN_MSG_OBJ` *rxCanMsg)
- bool `CAN1_ReceiveFrom` (const `CAN1_RX_FIFO_CHANNELS` fifoChannel, `CAN_MSG_OBJ` *rxCanMsg)
- `CAN_TX_MSG_REQUEST_STATUS` `CAN1_Transmit` (const `CAN1_TX_FIFO_CHANNELS` fifoChannel, `CAN_MSG_OBJ` *txCanMsg)
- bool `CAN1_IsBusOff` (void)
- bool `CAN1_IsTxErrorPassive` (void)
- bool `CAN1_IsTxErrorWarning` (void)
- bool `CAN1_IsTxErrorActive` (void)
- bool `CAN1_IsRxErrorPassive` (void)
- bool `CAN1_IsRxErrorWarning` (void)
- bool `CAN1_IsRxErrorActive` (void)
- void `CAN1_Sleep` (void)
- `CAN_TX_FIFO_STATUS` `CAN1_TransmitFIFOStatusGet` (const `CAN1_TX_FIFO_CHANNELS` fifoChannel)
- uint8_t `CAN1_ReceivedMessageCountGet` (void)
- void `CAN1_SetInvalidMessageInterruptHandler` (void(*handler)(void))
- void `CAN1_SetBusWakeUpActivityInterruptHandler` (void(*handler)(void))
- void `CAN1_SetBusErrorInterruptHandler` (void(*handler)(void))
- void `CAN1_SetModeChangeInterruptHandler` (void(*handler)(void))
- void `CAN1_SetSystemErrorInterruptHandler` (void(*handler)(void))
- void `CAN1_SetTxAttemptInterruptHandler` (void(*handler)(void))
- void `CAN1_SetRxBufferOverflowInterruptHandler` (void(*handler)(void))
- void `CAN1_SetFIFO1NotEmptyHandler` (void(*handler)(void))
- void `CAN1_SetTXQNotFullHandler` (void(*handler)(void))
- void `CAN1_SetFIFO2nullHandler` (void(*handler)(void))
- void `CAN1_SetFIFO3nullHandler` (void(*handler)(void))

6.80.1 Macro Definition Documentation

6.80.1.1 CAN1_TX_FIFO2

```
#define CAN1_TX_FIFO2 FIFO2
```

6.80.1.2 CAN1_TX_FIFO3

```
#define CAN1_TX_FIFO3 FIFO3
```

6.80.1.3 CAN1_TX_TXQ

```
#define CAN1_TX_TXQ TXQ
```

CAN1 Generated Driver File

@Company Microchip Technology Inc.

@File Name `can1.c`

@Summary This is the generated driver implementation file for the CAN1 driver using PIC10 / PIC12 / PIC16 / PIC18 MCUs

@Description This header file provides implementations for driver APIs for CAN1. Generation Information : Product Revision : PIC10 / PIC12 / PIC16 / PIC18 MCUs - 1.81.8 Device : PIC18F47Q84 Driver Version : 1.1.1 The generated drivers are tested against the following: Compiler : XC8 2.36 and above or later MPLAB : MPLAB X 6.00
Section: Included Files

6.80.2 Enumeration Type Documentation

6.80.2.1 CAN1_RX_FIFO_CHANNELS

```
enum CAN1_RX_FIFO_CHANNELS
```

Enumerator

FIFO1	
-------	--

6.80.2.2 CAN1_TX_FIFO_CHANNELS

```
enum CAN1_TX_FIFO_CHANNELS
```

Enumerator

TXQ	
FIFO2	
FIFO3	

6.80.3 Function Documentation

6.80.3.1 CAN1_Initialize()

```
void CAN1_Initialize (
    void )
```

Section: CAN Module APIs @Summary Initialization routine that takes inputs from the CAN GUI.

@Description This routine initializes the CAN module. This routine must be called before any other CAN routine is called. This routine should only be called once during system initialization.

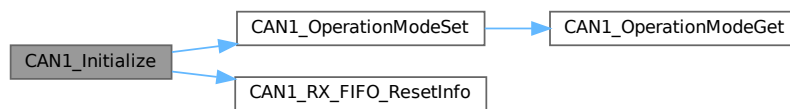
@Preconditions None

@Param None

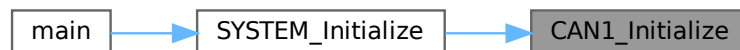
@Returns None

@Example void `main(void)` { `SYSTEM_Initialize()` ;

```
while (1) { User Application code }
} Here is the call graph for this function:
```



Here is the caller graph for this function:



6.80.3.2 CAN1_IsBusOff()

```
bool CAN1_IsBusOff (
    void )
```

@Summary Checks whether the transmitter is in bus off state.

@Description This routine checks whether the transmitter is in bus off state.

@Preconditions `CAN1_Initialize` function should be called before calling this function.

@Param None

@Returns true - Transmitter in Bus Off state false - Transmitter not in Bus Off state

@Example #define CAN_TX_BUFF "TEMPERATURE SENSOR ON"

```
void main(void) { CAN_MSG_OBJ msg; uint8_t data[32] = CAN_TX_BUFF;

SYSTEM_Initialize();

if(CAN_CONFIGURATION_MODE == CAN1_OperationModeGet()) { if(CAN_OP_MODE_↵
REQUEST_SUCCESS == CAN1_OperationModeSet(CAN_NORMAL_FD_MODE)) { msg.msgId =
0x1FFFF; msg.field.formatType = CAN_FD_FORMAT; msg.field.brs = CAN_NON_BRS_↵
_MODE; msg.field.frameType = CAN_FRAME_DATA; msg.field.idType = CAN_FRAME_↵
EXT; msg.field.dlc = DLC_32; msg.data = data;

if(CAN1_IsBusOff() == false) { if(CAN_TX_FIFO_AVAILABLE == (CAN1_Transmit_↵
FIFOStatusGet(CAN1_TX_FIFO1) & CAN_TX_FIFO_AVAILABLE)) { CAN1_Transmit(↵
CAN1_TX_FIFO1, &msg); } } } }

while(1); }
```

6.80.3.3 CAN1_IsRxErrorActive()

```
bool CAN1_IsRxErrorActive (
    void )
```

@Summary Checks whether the Receiver is in the error active state.

@Description This routine checks whether the receive is in the error active state. If Receiver error counter is above 0 to below 128, then receiver error active state is set.

@Preconditions CAN1_Initialize function should be called before calling this function.

@Param None

@Returns true - Receiver in Error active state false - Receiver not in Error active state

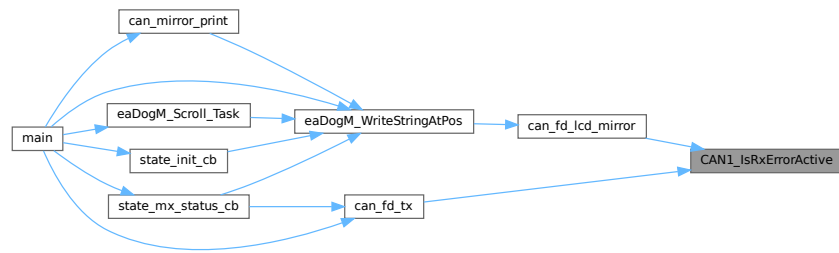
@Example void main(void) { CAN_MSG_OBJ msg;

```
SYSTEM_Initialize();
```

```
if(CAN_CONFIGURATION_MODE == CAN1_OperationModeGet()) { if(CAN_OP_MODE_↵
REQUEST_SUCCESS == CAN1_OperationModeSet(CAN_NORMAL_FD_MODE)) { while(1) {
if(CAN1_IsRxErrorActive() == false) { if(CAN1_ReceivedMessageCountGet() >
0) { CAN1_Receive(&msg); } } } } } } Here is the call graph for this function:
```



Here is the caller graph for this function:



6.80.3.4 CAN1_IsRxErrorPassive()

```
bool CAN1_IsRxErrorPassive (
    void )
```

@Summary Checks whether the Receiver is in the error passive state.

@Description This routine checks whether the receive is in the error passive state. If Receiver error counter is above 127, then receiver error passive state is set.

@Preconditions CAN1_Initialize function should be called before calling this function.

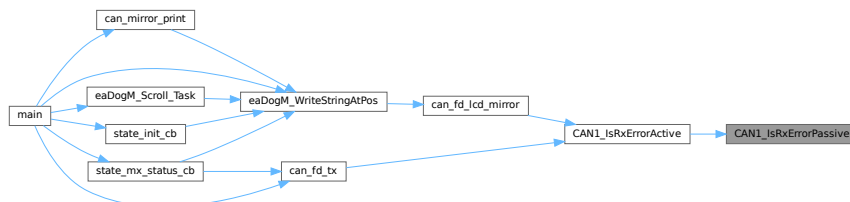
@Param None

@Returns true - Receiver in Error passive state false - Receiver not in Error passive state

@Example void main(void) { CAN_MSG_OBJ msg;

```
SYSTEM_Initialize();
```

```
if(CAN_CONFIGURATION_MODE == CAN1_OperationModeGet()) { if(CAN_OP_MODE_←
REQUEST_SUCCESS == CAN1_OperationModeSet(CAN_NORMAL_FD_MODE)) { while(1) {
if(CAN1_IsRxErrorPassive() == false) { if(CAN1_ReceivedMessageCountGet() >
0) { CAN1_Receive(&msg); } } } } } Here is the caller graph for this function:
```



6.80.3.5 CAN1_IsRxErrorWarning()

```
bool CAN1_IsRxErrorWarning (
    void )
```

@Summary Checks whether the Receiver is in the error warning state.

@Description This routine checks whether the receive is in the error warning state. If Receiver error counter is above 95 to below 128, then receiver error warning state is set.

@Preconditions CAN1_Initialize function should be called before calling this function.

@Param None

@Returns true - Receiver in Error warning state false - Receiver not in Error warning state

@Example void main(void) { CAN_MSG_OBJ msg;

SYSTEM_Initialize();

```
if(CAN_CONFIGURATION_MODE == CAN1_OperationModeGet()) { if(CAN_OP_MODE_↵
REQUEST_SUCCESS == CAN1_OperationModeSet(CAN_NORMAL_FD_MODE)) { while(1) {
if(CAN1_IsRxErrorWarning() == false) { if(CAN1_ReceivedMessageCountGet() >
0) { CAN1_Receive(&msg); } } } } }
```

6.80.3.6 CAN1_IsTxErrorActive()

```
bool CAN1_IsTxErrorActive (
    void )
```

@Summary Checks whether the transmitter is in the error active state.

@Description This routine checks whether the transmitter is in the error active state. If Transmitter error counter is above 0 to below 128, then transmitter error active state is set.

@Preconditions CAN1_Initialize function should be called before calling this function.

@Param None

@Returns true - Transmitter in Error active state false - Transmitter not in Error active state

@Example #define CAN_TX_BUFF "TEMPERATURE SENSOR ON"

```
void main(void) { CAN_MSG_OBJ msg; uint8_t data[32] = CAN_TX_BUFF;
```

SYSTEM_Initialize();

```
if(CAN_CONFIGURATION_MODE == CAN1_OperationModeGet()) { if(CAN_OP_MODE_↵
REQUEST_SUCCESS == CAN1_OperationModeSet(CAN_NORMAL_FD_MODE)) { msg.msgId =
0x1FFFF; msg.field.formatType = CAN_FD_FORMAT; msg.field.brs = CAN_NON_BRS_↵
_MODE; msg.field.frameType = CAN_FRAME_DATA; msg.field.idType = CAN_FRAME_↵
EXT; msg.field.dlc = DLC_32; msg.data = data;
```

```
if(CAN1_IsTxErrorActive() == false) { if(CAN_TX_FIFO_AVAILABLE == (CAN1_↵
_TransmitFIFOStatusGet(CAN1_TX_FIFO1) & CAN_TX_FIFO_AVAILABLE)) { CAN1_↵
Transmit(CAN1_TX_FIFO1, &msg); } } } }
```

while(1); } Here is the call graph for this function:



6.80.3.7 CAN1_IsTxErrorPassive()

```
bool CAN1_IsTxErrorPassive (
    void )
```

@Summary Checks whether the transmitter is in the error passive state.

@Description This routine checks whether the transmitter is in the error passive state. If Transmitter error counter is above 127, then the transmitter error passive state is set.

@Preconditions CAN1_Initialize function should be called before calling this function.

@Param None

@Returns true - Transmitter in Error Passive state false - Transmitter not in Error Passive state

@Example #define CAN_TX_BUFF "TEMPERATURE SENSOR ON"

```
void main(void) { CAN_MSG_OBJ msg; uint8_t data[32] = CAN_TX_BUFF;
```

```
SYSTEM_Initialize();
```

```
if(CAN_CONFIGURATION_MODE == CAN1_OperationModeGet()) { if(CAN_OP_MODE_↔
REQUEST_SUCCESS == CAN1_OperationModeSet(CAN_NORMAL_FD_MODE)) { msg.msgId =
0x1FFFF; msg.field.formatType = CAN_FD_FORMAT; msg.field.brs = CAN_NON_BRS_↔
_MODE; msg.field.frameType = CAN_FRAME_DATA; msg.field.idType = CAN_FRAME_↔
EXT; msg.field.dlc = DLC_32; msg.data = data;
```

```
if(CAN1_IsTxErrorPassive() == false) { if(CAN_TX_FIFO_AVAILABLE == (CAN1_↔
_TransmitFIFOStatusGet(CAN1_TX_FIFO1) & CAN_TX_FIFO_AVAILABLE)) { CAN1_↔
_Transmit(CAN1_TX_FIFO1, &msg); } } }
```

```
while(1); } Here is the caller graph for this function:
```



6.80.3.8 CAN1_IsTxErrorWarning()

```
bool CAN1_IsTxErrorWarning (
    void )
```

@Summary Checks whether the transmitter is in the error warning state.

@Description This routine checks whether the transmitter is in the error warning state. If Transmitter error counter is above 95 to below 128, then transmitter error warning state is set.

@Preconditions CAN1_Initialize function should be called before calling this function.

@Param None

@Returns true - Transmitter in Error warning state false - Transmitter not in Error warning state

@Example #define CAN_TX_BUFF "TEMPERATURE SENSOR ON"

```
void main(void) { CAN_MSG_OBJ msg; uint8_t data[32] = CAN_TX_BUFF;

SYSTEM_Initialize();

if(CAN_CONFIGURATION_MODE == CAN1_OperationModeGet()) { if(CAN_OP_MODE_↵
REQUEST_SUCCESS == CAN1_OperationModeSet(CAN_NORMAL_FD_MODE)) { msg.msgId =
0x1FFFF; msg.field.formatType = CAN_FD_FORMAT; msg.field.brs = CAN_NON_BRS_↵
_MODE; msg.field.frameType = CAN_FRAME_DATA; msg.field.idType = CAN_FRAME_↵
EXT; msg.field.dlc = DLC_32; msg.data = data;

if(CAN1_IsTxErrorWarning() == false) { if(CAN_TX_FIFO_AVAILABLE == (CAN1_↵
_TransmitFIFOStatusGet(CAN1_TX_FIFO1) & CAN_TX_FIFO_AVAILABLE)) { CAN1_↵
Transmit(CAN1_TX_FIFO1, &msg); } } } }

while(1); }
```

6.80.3.9 CAN1_OperationModeGet()

```
CAN_OP_MODES CAN1_OperationModeGet (
    void )
```

@Summary Get the CAN operation mode.

@Description This function gets the CAN operation mode.

@Preconditions [CAN1_Initialize\(\)](#) function should be called before calling this function.

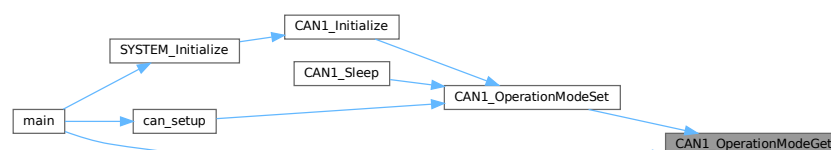
@Param None

@Returns Return the present CAN operation mode.

```
@Example void main(void) {
SYSTEM_Initialize();

if(CAN_CONFIGURATION_MODE == CAN1_OperationModeGet()) { if(CAN_OP_MODE_↵
REQUEST_SUCCESS == CAN1_OperationModeSet(CAN_NORMAL_FD_MODE)) { User Application
code } }

while (1) {
} } Here is the caller graph for this function:
```



6.80.3.10 CAN1_OperationModeSet()

```
CAN_OP_MODE_STATUS CAN1_OperationModeSet (
    const CAN_OP_MODES requestMode)
```

@Summary Set the CAN operation mode.

@Description This function sets the CAN operation mode.

@Preconditions [CAN1_Initialize\(\)](#) function should be called before calling this function.

@Param requestMode - CAN operation mode

@Returns CAN_OP_MODE_REQUEST_SUCCESS - Requested Operation mode set successfully CAN_OP_MODE_REQUEST_FAIL - Requested Operation mode set failure. Set configuration mode before setting CAN normal or debug operation mode. CAN_OP_MODE_SYS_ERROR_OCCURED - System error occurred while setting Operation mode.

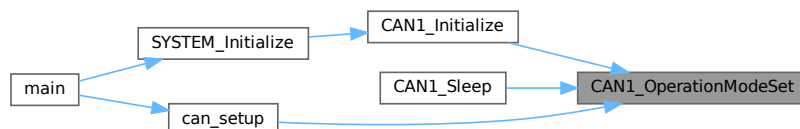
@Example void main(void) { [SYSTEM_Initialize\(\)](#);

```
if(CAN_CONFIGURATION_MODE == CAN1\_OperationModeGet\(\)) { if(CAN_OP_MODE_REQUEST_SUCCESS == CAN1_OperationModeSet(CAN_NORMAL_FD_MODE)) { User Application code } }
```

while (1) { } } Here is the call graph for this function:



Here is the caller graph for this function:



6.80.3.11 CAN1_Receive()

```
bool CAN1_Receive (
    CAN_MSG_OBJ * rxCanMsg)
```

@Summary Reads the message object from CAN receive FIFO.

@Description This routine reads a message object from the CAN receive FIFO.

@Preconditions [CAN1_Initialize\(\)](#) function should be called before calling this function. The [CAN1_ReceivedMessageCountGet\(\)](#) function should be checked to see if the receiver is not empty before calling this function.

@Param rxCanMsg - pointer to the message object

@Returns true - Receive successful false - Receive failure

@Example void [main\(void\)](#) { [CAN_MSG_OBJ](#) msg;

```
SYSTEM\_Initialize\(\);
```

```
if(CAN_CONFIGURATION_MODE == CAN1\_OperationModeGet\(\)) { if(CAN_OP_MODE_↵
REQUEST_SUCCESS == CAN1_OperationModeSet(CAN_NORMAL_FD_MODE)) { while(1) {
if(CAN1_ReceivedMessageCountGet() > 0) { if(true == CAN1_Receive(&msg)) {
break; } } } } }
```

```
while (1); }
```

6.80.3.12 CAN1_ReceivedMessageCountGet()

```
uint8_t CAN1_ReceivedMessageCountGet (
    void )
```

@Summary CAN RX FIFO number of messages that are received.

@Description This returns the number of messages that are received.

@Preconditions [CAN1_Initialize](#) function should be called before calling this function.

@Param None

@Returns Number of message received.

@Example void [main\(void\)](#) { [CAN_MSG_OBJ](#) msg;

```
SYSTEM\_Initialize\(\);
```

```
if(CAN_CONFIGURATION_MODE == CAN1\_OperationModeGet\(\)) { if(CAN_OP_MODE_↵
REQUEST_SUCCESS == CAN1_OperationModeSet(CAN_NORMAL_FD_MODE)) { while(1) {
if(CAN1_ReceivedMessageCountGet() > 0) { CAN1\_Receive(&msg); } } } } }
```

6.80.3.13 CAN1_ReceiveFrom()

```
bool CAN1_ReceiveFrom (
    const CAN1_RX_FIFO_CHANNELS fifoChannel,
    CAN_MSG_OBJ * rxCanMsg)
```

@Summary Reads the message object from the specified CAN receive FIFO.

@Description This routine reads a message object from the specified CAN receive FIFO.

@Preconditions [CAN1_Initialize\(\)](#) function should be called before calling this function.

@Param fifoChannel - CAN RX FIFO channel rxCanMsg - pointer to the message object

@Returns true - Receive successful false - Receive failure

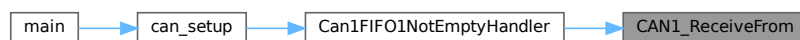
@Example volatile [CAN_MSG_OBJ](#) gMsg;

```
void CustomFIFO1Handler(void) { CAN1_ReceiveFrom(FIFO1, &gMsg)); }
```

```
void main(void) { SYSTEM\_Initialize\(\); CAN1_SetFIFO1FullHandler(&CustomFIFO1Handler);
```

```
INTERRUPT_GlobalInterruptEnable();
```

```
while(1); } Here is the caller graph for this function:
```

**6.80.3.14 CAN1_SetBusErrorInterruptHandler()**

```
void CAN1_SetBusErrorInterruptHandler (
    void(* handler ) (void))
```

@Summary Sets the CAN bus error interrupt handler.

@Description This routine sets the CAN bus error interrupt handler.

@Param Address of the callback routine.

@Returns None

@Example Note: Example code here is not based on MCC UI configuration, this is a sample code to demonstrate CAN transmit APIs usage.

```
volatile bool gBusErrorOccurred = false;
```

```
void CAN1_BusError(void)
{
    gBusErrorOccurred = true;
```

```
CAN Bus Error application code }
```

```
void main(void) { CAN_MSG_OBJ msg; uint8_t data[8] = {0x41,0x42,0x43,0x44,0x45,0x46,0x47,0x48};
```

```
SYSTEM_Initialize(); CAN1_SetBusErrorInterruptHandler(&CAN1_BusError); CAN1_↵
_OperationModeSet(CAN_CONFIGURATION_MODE);
```

```
if(CAN_CONFIGURATION_MODE == CAN1_OperationModeGet()) {
if(CAN_OP_MODE_REQUEST_SUCCESS == CAN1_OperationModeSet(CAN_NORMAL_FD_↵
MODE)) { msg.msgId = 0x1FFFF; msg.field.formatType = CAN_FD_FORMAT; msg.↵
field.brs = CAN_NON_BRS_MODE; msg.field.frameType = CAN_FRAME_DATA; msg.↵
field.idType = CAN_FRAME_EXT; msg.field.dlc = DLC_8; msg.data = data;
```

```
while(1) {
if(CAN_TX_FIFO_AVAILABLE == (CAN1_TransmitFIFOStatusGet(CAN1_TX_FIFO1) &
CAN_TX_FIFO_AVAILABLE)) { CAN1_Transmit(CAN1_TX_FIFO1, &msg); }
```

```
if(gBusErrorOccurred == true) { break; } } }
```

```
while(1); }
```

6.80.3.15 CAN1_SetBusWakeUpActivityInterruptHandler()

```
void CAN1_SetBusWakeUpActivityInterruptHandler (
    void(* handler ) (void))
```

@Summary Sets the CAN bus wake-Up activity interrupt handler.

@Description This routine sets the CAN bus wake-Up activity interrupt handler.

@Param Address of the callback routine.

@Returns None

@Example volatile bool gBusWakeUpOccurred = false;

```
void CAN1_BusWakeUpActivity(void) { gBusWakeUpOccurred = true; CAN Bus Wake↵
Up activity application code }
```

```
void main(void) { SYSTEM_Initialize(); CAN1_SetBusWakeUpActivityInterrupt↵
Handler(&CAN1_BusWakeUpActivity); CAN1_OperationModeSet(CAN_CONFIGURATION_↵
MODE);
```

```
if(CAN_CONFIGURATION_MODE == CAN1_OperationModeGet()) { if(CAN_OP_MODE_↵
REQUEST_SUCCESS == CAN1_OperationModeSet(CAN_NORMAL_FD_MODE)) { CAN1_Sleep();
```

```
Check CAN1 module is in CAN_DISABLE_MODE if(CAN_DISABLE_MODE == CAN1_OperationModeGet())
{ Sleep(); //Call sleep instruction
```

```
while(1) { if(gBusWakeUpOccurred == true) { break; }
} } }
```

```
while(1); }
```

6.80.3.16 CAN1_SetFIFO1NotEmptyHandler()

```
void CAN1_SetFIFO1NotEmptyHandler (
    void(* handler ) (void))
```

@Summary Sets the RX FIFO Not Empty interrupt handler.

@Description This routine sets the RX FIFO Not Empty interrupt handler for FIFO1.

@Param Address of the callback routine.

@Returns None

@Example volatile `CAN_MSG_OBJ` gMsg;

```
void CustomFIFO1Handler(void) { CAN1_ReceiveFrom(FIFO1, &gMsg); }
```

```
void main(void) { SYSTEM_Initialize(); CAN1_SetFIFO1NotEmptyHandler(&CustomFIFO1Handler);
```

```
INTERRUPT_GlobalInterruptEnable();
```

```
while(1); } Here is the caller graph for this function:
```

**6.80.3.17 CAN1_SetFIFO2nullHandler()**

```
void CAN1_SetFIFO2nullHandler (
    void(* handler ) (void))
```

@Summary Sets the Disable TX FIFO Interrupt interrupt handler.

@Description This routine sets the Disable TX FIFO Interrupt interrupt handler for FIFO2.

@Param Address of the callback routine.

@Returns None

@Example volatile `CAN_MSG_OBJ` gMsg;

```
void CustomFIFO2Handler(void) { CAN1_Transmit(CAN1_TX_FIFO1, &gMsg); }
```

```
void main(void) { uint8_t data[8] = {0x41,0x42,0x43,0x44,0x45,0x46,0x47,0x48};
gMsg.msgId = 0x1FFFF; gMsg.field.formatType = CAN_FD_FORMAT; gMsg.field.brs
= CAN_NON_BRS_MODE; gMsg.field.frameType = CAN_FRAME_DATA; gMsg.field.id
Type = CAN_FRAME_EXT; gMsg.field.dlc = DLC_8; gMsg.data = data;
```

```
SYSTEM_Initialize(); CAN1_SetFIFO2nullHandler(&CustomFIFO2Handler);
```

```
INTERRUPT_GlobalInterruptEnable();
```

```
while(1); }
```

6.80.3.18 CAN1_SetFIFO3nullHandler()

```
void CAN1_SetFIFO3nullHandler (
    void(* handler ) (void))
```

@Summary Sets the Disable TX FIFO Interrupt interrupt handler.

@Description This routine sets the Disable TX FIFO Interrupt interrupt handler for FIFO3.

@Param Address of the callback routine.

@Returns None

@Example volatile [CAN_MSG_OBJ](#) gMsg;

```
void CustomFIFO3Handler(void) { CAN1_Transmit(CAN1_TX_FIFO1, &gMsg); }

void main(void) { uint8_t data[8] = {0x41,0x42,0x43,0x44,0x45,0x46,0x47,0x48};
gMsg.msgId = 0x1FFFF; gMsg.field.formatType = CAN_FD_FORMAT; gMsg.field.brs
= CAN_NON_BRS_MODE; gMsg.field.frameType = CAN_FRAME_DATA; gMsg.field.id↵
Type = CAN_FRAME_EXT; gMsg.field.dlc = DLC_8; gMsg.data = data;

SYSTEM_Initialize(); CAN1_SetFIFO3nullHandler(&CustomFIFO3Handler);

INTERRUPT_GlobalInterruptEnable();

while(1); }
```

6.80.3.19 CAN1_SetInvalidMessageInterruptHandler()

```
void CAN1_SetInvalidMessageInterruptHandler (
    void(* handler ) (void))
```

@Summary Sets the invalid message interrupt handler.

@Description This routine sets the invalid message interrupt handler.

@Param Address of the callback routine.

@Returns None

@Example Note: Example code here is not based on MCC UI configuration, this is a sample code to demonstrate CAN transmit APIs usage.

```
volatile bool gInvalidMsgOccurred = false;

void CAN1_InvalidMessage(void)
{
    gInvalidMsgOccurred = true;

    CAN Invalid Message application code }

void main(void) { CAN\_MSG\_OBJ msg; uint8_t data[8] = {0x41,0x42,0x43,0x44,0x45,0x46,0x47,0x48};

SYSTEM_Initialize(); CAN1_SetInvalidMessageInterruptHandler(&CAN1_Invalid↵
Message); CAN1_OperationModeSet(CAN_CONFIGURATION_MODE);

if(CAN_CONFIGURATION_MODE == CAN1\_OperationModeGet\(\)) {
if(CAN_OP_MODE_REQUEST_SUCCESS == CAN1_OperationModeSet(CAN_NORMAL_FD_↵
MODE)) { msg.msgId = 0x1FFFF; msg.field.formatType = CAN_FD_FORMAT; msg.↵
field.brs = CAN_NON_BRS_MODE; msg.field.frameType = CAN_FRAME_DATA; msg.↵
field.idType = CAN_FRAME_EXT; msg.field.dlc = DLC_8; msg.data = data;

while(1) {
if(CAN_TX_FIFO_AVAILABLE == (CAN1_TransmitFIFOStatusGet(CAN1_TX_FIFO1) &
CAN_TX_FIFO_AVAILABLE)) { CAN1_Transmit(CAN1_TX_FIFO1, &msg); }

if(gInvalidMsgOccurred == true) { break; } } }

while(1); }
```

6.80.3.20 CAN1_SetModeChangeInterruptHandler()

```
void CAN1_SetModeChangeInterruptHandler (
    void(* handler ) (void))
```

@Summary Sets the CAN mode change interrupt handler.

@Description This routine sets the CAN mode change interrupt handler.

@Param Address of the callback routine.

@Returns None

@Example volatile bool gModeChangeOccurred = false;

```
void CAN1_ModeChange(void) { gModeChangeOccurred = true; CAN Mode Change
application code }
```

```
void main(void) { CAN_MSG_OBJ msg;
```

```
SYSTEM_Initialize(); CAN1_SetModeChangeInterruptHandler(&CAN1_ModeChange);
CAN1_OperationModeSet(CAN_CONFIGURATION_MODE);
```

```
if(CAN_CONFIGURATION_MODE == CAN1_OperationModeGet()) { if(CAN_OP_MODE_↵
REQUEST_SUCCESS == CAN1_OperationModeSet(CAN_NORMAL_FD_MODE)) { while(1) {
if(gModeChangeOccurred == true) { break; }
} } }
```

```
while (1); }
```

6.80.3.21 CAN1_SetRxBufferOverflowInterruptHandler()

```
void CAN1_SetRxBufferOverflowInterruptHandler (
    void(* handler ) (void))
```

@Summary Sets the CAN receive overflow interrupt handler.

@Description This routine sets the CAN receive overflow interrupt handler.

@Param Address of the callback routine.

@Returns None

@Example volatile bool gRxOverflowOccurred = false;

```
void CAN1_RxBufferOverflow(void) { gRxOverflowOccurred = true; CAN Receive
Buffer Overflow application code }
```

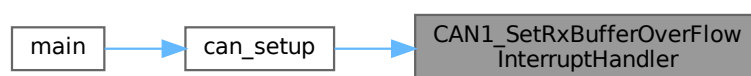
```
void main(void) {
```

```
SYSTEM_Initialize(); CAN1_SetRxBufferOverflowInterruptHandler(&CAN1_Rx↵
BufferOverflow); CAN1_OperationModeSet(CAN_CONFIGURATION_MODE);
```

```
if(CAN_CONFIGURATION_MODE == CAN1_OperationModeGet()) {
if(CAN_OP_MODE_REQUEST_SUCCESS == CAN1_OperationModeSet(CAN_NORMAL_FD_↵
MODE)) { while(1) { if(gRxOverflowOccurred == true) { gRxOverflowOccurred =
false; User Application code break;
} } } }
```

```
while(1); }
```

Here is the caller graph for this function:



6.80.3.22 CAN1_SetSystemErrorInterruptHandler()

```
void CAN1_SetSystemErrorInterruptHandler (
    void(* handler ) (void))
```

@Summary Sets the CAN system error interrupt handler.

@Description This routine sets the CAN system error interrupt handler.

@Param Address of the callback routine.

@Returns None

@Example Note: Example code here is not based on MCC UI configuration, this is a sample code to demonstrate CAN transmit APIs usage.

```
volatile bool gSystemOccurred = false;
```

```
void CAN1_SystemError(void)
{
    gSystemOccurred = true;
```

```
    CAN System Error application code }
```

```
void main(void) { CAN_MSG_OBJ msg; uint8_t data[8] = {0x41,0x42,0x43,0x44,0x45,0x46,0x47
```

```
SYSTEM_Initialize(); CAN1_SetSystemErrorInterruptHandler(&CAN1_SystemError);
CAN1_OperationModeSet (CAN_CONFIGURATION_MODE);
```

```
if (CAN_CONFIGURATION_MODE == CAN1_OperationModeGet ()) {
if (CAN_OP_MODE_REQUEST_SUCCESS == CAN1_OperationModeSet (CAN_NORMAL_FD_↵
MODE)) { msg.msgId = 0x1FFFF; msg.field.formatType = CAN_FD_FORMAT; msg.↵
field.brs = CAN_NON_BRS_MODE; msg.field.frameType = CAN_FRAME_DATA; msg.↵
field.idType = CAN_FRAME_EXT; msg.field.dlc = DLC_8; msg.data = data;
```

```
while(1) {
if (CAN_TX_FIFO_AVAILABLE == (CAN1_TransmitFIFOStatusGet (CAN1_TX_FIFO1) &
CAN_TX_FIFO_AVAILABLE)) { CAN1_Transmit (CAN1_TX_FIFO1, &msg); }
```

```
if(gSystemOccurred == true) { break; } }
```

```
} }
```

```
while(1); }
```


6.80.3.23 CAN1_SetTxAttemptInterruptHandler()

```
void CAN1_SetTxAttemptInterruptHandler (
    void(* handler ) (void))
```

@Summary Sets the CAN transmit attempt interrupt handler.

@Description This routine sets the CAN transmit attempt interrupt handler.

@Param Address of the callback routine.

@Returns None

@Example **Note:** Example code here is not based on MCC UI configuration, this is a sample code to demonstrate CAN transmit APIs usage.

```
volatile bool gTxAttemptOccurred = false;
```

```
void CAN1_TxAttempt(void)
{
    gTxAttemptOccurred = true;
```

```
CAN Transmit Attempt application code }
```

```
void main(void) { CAN_MSG_OBJ msg; uint8_t data[8] = {0x41,0x42,0x43,0x44,0x45,0x46,0x47,0x48};
```

```
SYSTEM_Initialize(); CAN1_SetTxAttemptInterruptHandler(&CAN1_TxAttempt);
CAN1_OperationModeSet(CAN_CONFIGURATION_MODE);
```

```
if(CAN_CONFIGURATION_MODE == CAN1_OperationModeGet()) {
if(CAN_OP_MODE_REQUEST_SUCCESS == CAN1_OperationModeSet(CAN_NORMAL_FD_↵
MODE)) { msg.msgId = 0x1FFFF; msg.field.formatType = CAN_FD_FORMAT; msg.↵
field.brs = CAN_NON_BRS_MODE; msg.field.frameType = CAN_FRAME_DATA; msg.↵
field.idType = CAN_FRAME_EXT; msg.field.dlc = DLC_8; msg.data = data;
```

```
while(1) {
if(CAN_TX_FIFO_AVAILABLE == (CAN1_TransmitFIFOStatusGet(CAN1_TX_FIFO1) &
CAN_TX_FIFO_AVAILABLE)) { CAN1_Transmit(CAN1_TX_FIFO1, &msg); }
```

```
if(gTxAttemptOccurred == true) { break; } } }
```

```
while(1); }
```

6.80.3.24 CAN1_SetTXQNotFullHandler()

```
void CAN1_SetTXQNotFullHandler (
    void(* handler ) (void))
```

@Summary Sets the TXQ Not Full interrupt handler.

@Description This routine sets the TXQ Not Full interrupt handler for TXQ.

@Param Address of the callback routine.

@Returns None

@Example volatile CAN_MSG_OBJ gMsg;

```
void CustomTXQHandler(void) { CAN1_Transmit(CAN1_TX_FIFO1, &gMsg); }
```

```
void main(void) { uint8_t data[8] = {0x41,0x42,0x43,0x44,0x45,0x46,0x47,0x48};
gMsg.msgId = 0x1FFFF; gMsg.field.formatType = CAN_FD_FORMAT; gMsg.field.brs
= CAN_NON_BRS_MODE; gMsg.field.frameType = CAN_FRAME_DATA; gMsg.field.id←
Type = CAN_FRAME_EXT; gMsg.field.dlc = DLC_8; gMsg.data = data;
```

```
SYSTEM_Initialize(); CAN1_SetTXQNotFullHandler(&CustomTXQHandler);
```

```
INTERRUPT_GlobalInterruptEnable();
```

```
while(1); } Here is the caller graph for this function:
```



6.80.3.25 CAN1_Sleep()

```
void CAN1_Sleep (
    void )
```

@Summary Puts CAN module in sleep mode.

@Description This routine puts CAN module in sleep mode.

@Preconditions CAN1_Initialize function should be called before calling this function.

@Param None

@Returns None

```
@Example void main(void) {
SYSTEM_Initialize();
```

```

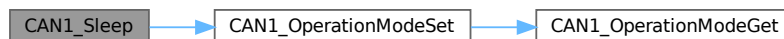
if(CAN_CONFIGURATION_MODE == CAN1_OperationModeGet()) { if(CAN_OP_MODE_↵
REQUEST_SUCCESS == CAN1_OperationModeSet(CAN_NORMAL_FD_MODE)) { CAN1_Sleep();

if(CAN_DISABLE_MODE == CAN1_OperationModeGet()) { Sleep(); //Call sleep
instruction C1INTLbits.WAKIF = 0; // Clear Wake-Up interrupt flag

Recover to Normal mode if(CAN_OP_MODE_REQUEST_SUCCESS == CAN1_Operation_↵
ModeSet(CAN_CONFIGURATION_MODE)) { if(CAN_OP_MODE_REQUEST_SUCCESS == CAN1_↵
_OperationModeSet(CAN_NORMAL_FD_MODE)) { User Application code
} } } } }

while(1); } Here is the call graph for this function:

```



6.80.3.26 CAN1_Transmit()

```

CAN_TX_MSG_REQUEST_STATUS CAN1_Transmit (
    const CAN1_TX_FIFO_CHANNELS fifoChannel,
    CAN_MSG_OBJ * txCanMsg)

```

@Summary Writes a message object to the CAN TX FIFO.

@Description This routine writes a message object to the CAN TX FIFO.

@Preconditions CAN1_Initialize function should be called before calling this function. The transfer status should be checked to see if transmitter is not full before calling this function.

@Param fifoChannel - CAN TX priority FIFO selection txCanMsg - pointer to the message object

@Returns CAN Transmit message status. CAN_TX_MSG_REQUEST_SUCCESS - Transmit message object successfully placed into transmit FIFO CAN_TX_MSG_REQUEST_DLC_EXCEED_ERROR - Transmit message object DLC size is more than transmit FIFO configured DLC size CAN_TX_MSG_REQUEST_BRS_ERROR - Transmit FIFO configured has Non BRS mode and CAN TX Message object has BRS enabled CAN_TX_MSG_REQUEST_↵_FIFO_FULL - Transmit FIFO Full

@Example #define CAN_TX_BUFF "TEMPERATURE SENSOR ON"

```

void main(void) { CAN_MSG_OBJ msg; uint8_t data[32] = CAN_TX_BUFF;

```

```

SYSTEM_Initialize();

```

```

if(CAN_CONFIGURATION_MODE == CAN1_OperationModeGet()) { if(CAN_OP_MODE_↵
REQUEST_SUCCESS == CAN1_OperationModeSet(CAN_NORMAL_FD_MODE)) { msg.msgId =
0xFFFF; msg.field.formatType = CAN_FD_FORMAT; msg.field.brs = CAN_NON_BRS_↵
_MODE; msg.field.frameType = CAN_FRAME_DATA; msg.field.idType = CAN_FRAME_↵
EXT; msg.field.dlc = DLC_32; msg.data = data;

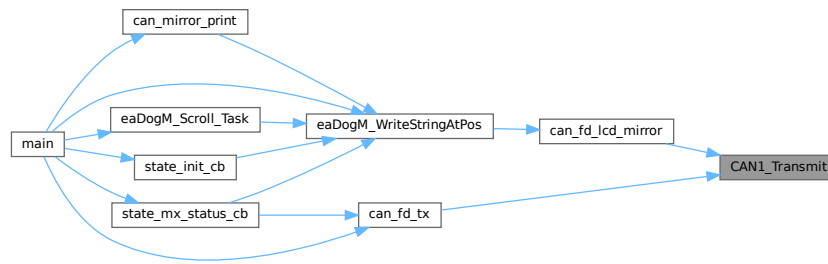
```

```

if(CAN_TX_FIFO_AVAILABLE == (CAN1_TransmitFIFOStatusGet(CAN1_TX_FIFO1) &
CAN_TX_FIFO_AVAILABLE)) { CAN1_Transmit(CAN1_TX_FIFO1, &msg); } } }

```

`while(1); }` Here is the caller graph for this function:



6.80.3.27 CAN1_TransmitFIFOStatusGet()

```

CAN_TX_FIFO_STATUS CAN1_TransmitFIFOStatusGet (
    const CAN1_TX_FIFO_CHANNELS fifoChannel)

```

@Summary CAN transmitter FIFO status.

@Description This returns the CAN transmitter FIFO status.

@Preconditions CAN1_Initialize function should be called before calling this function.

@Param fifoChannel - CAN TX priority FIFO selection

@Returns CAN Transmit FIFO status. CAN_TX_FIFO_FULL - CAN Transmit FIFO is full CAN_TX_FIFO_↔
AVAILABLE - CAN Transmit FIFO message space is available

@Example `#define CAN_TX_BUFF "TEMPERATURE SENSOR ON"`

```

void main(void) { CAN_MSG_OBJ msg; uint8_t data[32] = CAN_TX_BUFF;

```

```

SYSTEM_Initialize();

```

```

if(CAN_CONFIGURATION_MODE == CAN1_OperationModeGet()) {
    if(CAN_OP_MODE_REQUEST_SUCCESS == CAN1_OperationModeSet(CAN_NORMAL_FD_↔
    MODE)) { msg.msgId = 0x1FFFF; msg.field.formatType = CAN_FD_FORMAT; msg.↔
    field.brs = CAN_NON_BRS_MODE; msg.field.frameType = CAN_FRAME_DATA; msg.↔
    field.idType = CAN_FRAME_EXT; msg.field.dlc = DLC_32; msg.data = data;

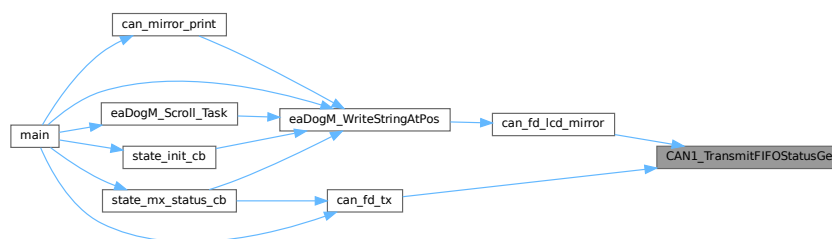
```

```

    if(CAN_TX_FIFO_AVAILABLE == (CAN1_TransmitFIFOStatusGet(CAN1_TX_FIFO1) &
    CAN_TX_FIFO_AVAILABLE)) { CAN1_Transmit(CAN1_TX_FIFO1, &msg); } }

```

`while(1); }` Here is the caller graph for this function:



6.81 can1.h

[Go to the documentation of this file.](#)

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029     party
00030     license terms applicable to your use of third party software (including open source software) that
00031     may accompany Microchip software.
00032
00033     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00034     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00035     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00036     FOR A PARTICULAR PURPOSE.
00037
00038     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00039     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00040     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00041     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00042     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00043     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00044     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00045     SOFTWARE.
00046 */
00047 #ifndef CAN1_H
00048 #define CAN1_H
00049
00053
00054 #include <stdbool.h>
00055 #include <stdint.h>
00056 #include "can_types.h"
00057
00058 // Transmit FIFO's Custom Name
00059 #define CAN1_TX_TXQ TXQ
00060 #define CAN1_TX_FIFO2 FIFO2
00061 #define CAN1_TX_FIFO3 FIFO3
00062
00063 typedef enum
00064 {
00065     TXQ = 0,
00066     FIFO2 = 2,
00067     FIFO3 = 3
00068 } CAN1_TX_FIFO_CHANNELS;
00069
00070 typedef enum
00071 {
00072     FIFO1 = 1
00073 } CAN1_RX_FIFO_CHANNELS;
00074
00078
00110 void CAN1_Initialize(void);
00111
00151 CAN_OP_MODE_STATUS CAN1_OperationModeSet(const CAN_OP_MODES requestMode);
00152
00189 CAN_OP_MODES CAN1_OperationModeGet(void);
00190
00239 bool CAN1_Receive(CAN_MSG_OBJ *rxCanMsg);
00240
00279 bool CAN1_ReceiveFrom(const CAN1_RX_FIFO_CHANNELS fifoChannel, CAN_MSG_OBJ *rxCanMsg);
00280
00338 CAN_TX_MSG_REQUEST_STATUS CAN1_Transmit(const CAN1_TX_FIFO_CHANNELS fifoChannel, CAN_MSG_OBJ
    *txCanMsg);
00339
00394 bool CAN1_IsBusOff(void);
00395
00452 bool CAN1_IsTxErrorPassive(void);
00453
00511 bool CAN1_IsTxErrorWarning(void);
00512
00570 bool CAN1_IsTxErrorActive(void);
00571
00618 bool CAN1_IsRxErrorPassive(void);
00619
00666 bool CAN1_IsRxErrorWarning(void);
00667
00714 bool CAN1_IsRxErrorActive(void);
00715
00765 void CAN1_Sleep(void);
00766

```

```

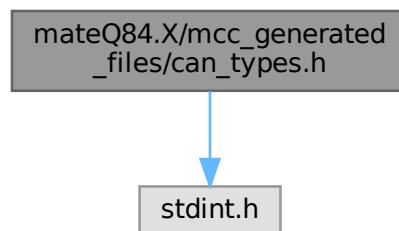
00819 CAN_TX_FIFO_STATUS CAN1_TransmitFIFOStatusGet(const CAN1_TX_FIFO_CHANNELS fifoChannel);
00820
00861 uint8_t CAN1_ReceivedMessageCountGet(void);
00862
00928 void CAN1_SetInvalidMessageInterruptHandler(void (*handler)(void));
00929
00985 void CAN1_SetBusWakeUpActivityInterruptHandler(void (*handler)(void));
00986
01053 void CAN1_SetBusErrorInterruptHandler(void (*handler)(void));
01054
01104 void CAN1_SetModeChangeInterruptHandler(void (*handler)(void));
01105
01173 void CAN1_SetSystemErrorInterruptHandler(void (*handler)(void));
01174
01241 void CAN1_SetTxAttemptInterruptHandler(void (*handler)(void));
01242
01293 void CAN1_SetRxBufferOverflowInterruptHandler(void (*handler)(void));
01294
01328 void CAN1_SetFIFO1NotEmptyHandler(void (*handler)(void));
01329
01372 void CAN1_SetTXQNotFullHandler(void (*handler)(void));
01373
01416 void CAN1_SetFIFO2nullHandler(void (*handler)(void));
01417
01460 void CAN1_SetFIFO3nullHandler(void (*handler)(void));
01461
01462
01463
01464 #endif //CAN1_H

```

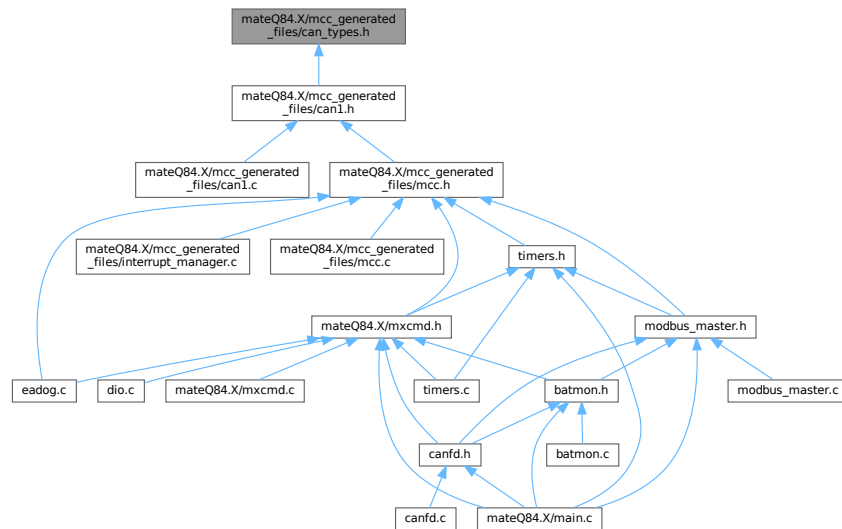
6.82 mateQ84.X/mcc_generated_files/can_types.h File Reference

#include <stdint.h>

Include dependency graph for can_types.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- union [CAN_MSG_FIELD](#)
- struct [CAN_MSG_OBJ](#)

Enumerations

- enum [CAN_MSG_OBJ_BRS_MODE](#) { [CAN_NON_BRS_MODE](#) = 0 , [CAN_BRS_MODE](#) = 1 }
- enum [CAN_MSG_OBJ_ID_TYPE](#) { [CAN_FRAME_STD](#) = 0 , [CAN_FRAME_EXT](#) = 1 }
- enum [CAN_MSG_OBJ_FRAME_TYPE](#) { [CAN_FRAME_DATA](#) = 0 , [CAN_FRAME_RTR](#) = 1 }
- enum [CAN_MSG_OBJ_TYPE](#) { [CAN_2_0_FORMAT](#) = 0 , [CAN_FD_FORMAT](#) = 1 }
- enum [CAN_TX_MSG_REQUEST_STATUS](#) { [CAN_TX_MSG_REQUEST_SUCCESS](#) = 0 , [CAN_TX_MSG_REQUEST_DLC_ERROR](#) = 1 , [CAN_TX_MSG_REQUEST_BRS_ERROR](#) = 2 , [CAN_TX_MSG_REQUEST_FIFO_FULL](#) = 3 }
- enum [CAN_OP_MODES](#) { [CAN_NORMAL_FD_MODE](#) = 0 , [CAN_DISABLE_MODE](#) = 1 , [CAN_INTERNAL_LOOPBACK_MODE](#) = 2 , [CAN_LISTEN_ONLY_MODE](#) = 3 , [CAN_CONFIGURATION_MODE](#) = 4 , [CAN_EXTERNAL_LOOPBACK_MODE](#) = 5 , [CAN_NORMAL_2_0_MODE](#) = 6 , [CAN_RESTRICTED_OPERATION_MODE](#) = 7 }
- enum [CAN_OP_MODE_STATUS](#) { [CAN_OP_MODE_REQUEST_SUCCESS](#) , [CAN_OP_MODE_REQUEST_FAIL](#) , [CAN_OP_MODE_SYS_ERROR_OCCURED](#) }
- enum [CAN_TX_FIFO_STATUS](#) { [CAN_TX_FIFO_FULL](#) , [CAN_TX_FIFO_AVAILABLE](#) }
- enum [CAN_DLC](#) { [DLC_0](#) , [DLC_1](#) , [DLC_2](#) , [DLC_3](#) , [DLC_4](#) , [DLC_5](#) , [DLC_6](#) , [DLC_7](#) , [DLC_8](#) , [DLC_12](#) , [DLC_16](#) , [DLC_20](#) , [DLC_24](#) , [DLC_32](#) , [DLC_48](#) , [DLC_64](#) }

6.82.1 Enumeration Type Documentation

6.82.1.1 CAN_DLC

enum [CAN_DLC](#)

CAN Message Object Data Length code Enumeration

@Summary Defines the CAN message object data length code enumeration.

@Description The Data Length Code (DLC) specifies the number of data bytes a message frame contains.

Enumerator

DLC_0	
DLC_1	
DLC_2	
DLC_3	
DLC_4	
DLC_5	
DLC_6	
DLC_7	
DLC_8	
DLC_12	
DLC_16	
DLC_20	
DLC_24	
DLC_32	
DLC_48	
DLC_64	

6.82.1.2 CAN_MSG_OBJ_BRS_MODE

enum [CAN_MSG_OBJ_BRS_MODE](#)

CAN Message Object Bit Rate Switch Selection Enumeration

@Summary Defines the CAN message object bit rate switch selection enumeration.

@Description This enumeration defines the CAN message object bit rate switch selection option.

Enumerator

CAN_NON_BRS_MODE	
CAN_BRS_MODE	

6.82.1.3 CAN_MSG_OBJ_FRAME_TYPE

enum [CAN_MSG_OBJ_FRAME_TYPE](#)

CAN Message Object Frame Type Selection Enumeration

@Summary Defines the CAN message object frame type selection enumeration.

@Description This enumeration defines the CAN message object frame type selection option.

Enumerator

CAN_FRAME_DATA	
CAN_FRAME_RTR	

6.82.1.4 CAN_MSG_OBJ_ID_TYPE

enum [CAN_MSG_OBJ_ID_TYPE](#)

CAN Message Object Identifier Selection Enumeration

@Summary Defines the CAN message object identifier selection enumeration.

@Description This enumeration defines the CAN message object identifier selection option.

Enumerator

CAN_FRAME_STD	
CAN_FRAME_EXT	

6.82.1.5 CAN_MSG_OBJ_TYPE

enum [CAN_MSG_OBJ_TYPE](#)

CAN Message Object Format Type Selection Enumeration

@Summary Defines the CAN message object format type selection enumeration.

@Description This enumeration defines the CAN message object format type selection option.

Enumerator

CAN_2_0_FORMAT	
CAN_FD_FORMAT	

6.82.1.6 CAN_OP_MODE_STATUS

enum [CAN_OP_MODE_STATUS](#)

CAN Operation Modes Request Status Enumeration

@Summary Defines the CAN operation modes request status enumeration.

@Description This enumeration defines the CAN operation mode request status.

Enumerator

CAN_OP_MODE_REQUEST_SUCCESS	
CAN_OP_MODE_REQUEST_FAIL	
CAN_OP_MODE_SYS_ERROR_OCCURED	

6.82.1.7 CAN_OP_MODES

enum [CAN_OP_MODES](#)

CAN Operation Modes Enumeration

@Summary Defines the CAN operation modes enumeration.

@Description This enumeration defines the CAN different operating modes.

Enumerator

CAN_NORMAL_FD_MODE	
CAN_DISABLE_MODE	
CAN_INTERNAL_LOOPBACK_MODE	
CAN_LISTEN_ONLY_MODE	
CAN_CONFIGURATION_MODE	
CAN_EXTERNAL_LOOPBACK_MODE	
CAN_NORMAL_2_0_MODE	
CAN_RESTRICTED_OPERATION_MODE	

6.82.1.8 CAN_TX_FIFO_STATUS

enum [CAN_TX_FIFO_STATUS](#)

CAN Transmit FIFO Status Enumeration

@Summary Defines the CAN transmit FIFO status Enumeration.

@Description This enumeration defines the CAN transmit FIFO status.

Enumerator

CAN_TX_FIFO_FULL	
CAN_TX_FIFO_AVAILABLE	

6.82.1.9 CAN_TX_MSG_REQUEST_STATUS

enum [CAN_TX_MSG_REQUEST_STATUS](#)

CAN Transmit Message Object Request Status Enumeration

@Summary Defines the CAN transmit message object request status enumeration.

@Description This enumeration defines the CAN transmit message object request status.

Enumerator

CAN_TX_MSG_REQUEST_SUCCESS	
CAN_TX_MSG_REQUEST_DLC_EXCEED_ERROR	
CAN_TX_MSG_REQUEST_BRS_ERROR	
CAN_TX_MSG_REQUEST_FIFO_FULL	

6.83 can_types.h

[Go to the documentation of this file.](#)

```

00001
00002
00003 /*
00004  (c) 2018 Microchip Technology Inc. and its subsidiaries.
00005
00006  Subject to your compliance with these terms, you may use Microchip software and any
00007  derivatives exclusively with Microchip products. It is your responsibility to comply with third
00008  party
00009  license terms applicable to your use of third party software (including open source software) that
00010
00011  may accompany Microchip software.
00012
00013  THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00014  EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00015  IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00016  FOR A PARTICULAR PURPOSE.
00017
00018  IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00019  INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00020  WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00021  HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00022  THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00023  CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00024  OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00025  SOFTWARE.
00026 */
00027
00028 #ifndef CAN_TYPES_H
00029 #define CAN_TYPES_H
00030
00031 #include <stdint.h>
00032
00033 typedef union
00034 {
00035     uint8_t msgfields;
00036     struct
00037     {
00038         uint8_t idType:1;           // 1 bit (Standard Frame or Extended Frame)
00039         uint8_t frameType:1;        // 1 bit (Data Frame or RTR Frame)
00040         uint8_t dlc:4;              // 4 bit (No of data bytes a message frame contains)
00041         uint8_t formatType:1;       // 1 bit (CAN 2.0 Format or CAN_FD Format)
00042         uint8_t brs:1;              // 1 bit (Bit Rate Switch)
00043     };
00044 } CAN_MSG_FIELD;
00045
00046 typedef struct
00047 {
00048     uint32_t msgId;                // 29 bit (SID: 11bit, EID:18bit)
00049     CAN_MSG_FIELD field;           // CAN TX/RX Message Object Control
00050     uint8_t *data;                 // Pointer to message data
00051 } CAN_MSG_OBJ;
00052
00053 typedef enum
00054 {
00055     CAN_NON_BRS_MODE = 0,
00056     CAN_BRS_MODE = 1              //Supported only in CAN FD mode
00057 } CAN_MSG_OBJ_BRS_MODE;
00058
00059 typedef enum
00060 {
00061     CAN_FRAME_STD = 0,
00062     CAN_FRAME_EXT = 1,
00063 } CAN_MSG_OBJ_ID_TYPE;
00064
00065 typedef enum
00066 {
00067     CAN_FRAME_DATA = 0,
00068     CAN_FRAME_RTR = 1,
00069 } CAN_MSG_OBJ_FRAME_TYPE;
00070
00071 typedef enum
00072 {
00073     CAN_2_0_FORMAT = 0,
00074     CAN_FD_FORMAT = 1             //Supported only in CAN FD mode
00075 } CAN_MSG_OBJ_TYPE;
00076
00077 typedef enum
00078 {
00079     CAN_TX_MSG_REQUEST_SUCCESS = 0, // Transmit message object successfully placed into
00080                                     Transmit FIFO

```

```

00157     CAN_TX_MSG_REQUEST_DLC_EXCEED_ERROR = 1,    // Transmit message object DLC size is more than
Transmit FIFO configured DLC size
00158     CAN_TX_MSG_REQUEST_BRS_ERROR = 2,           // Transmit FIFO is configured has Non BRS mode and
CAN TX Message object has BRS enabled
00159     CAN_TX_MSG_REQUEST_FIFO_FULL = 3,           // Transmit FIFO is Full
00160 } CAN_TX_MSG_REQUEST_STATUS;
00161
00171 typedef enum
00172 {
00173     CAN_NORMAL_FD_MODE = 0,    //Supported only in CAN FD mode
00174     CAN_DISABLE_MODE = 1,
00175     CAN_INTERNAL_LOOPBACK_MODE = 2,
00176     CAN_LISTEN_ONLY_MODE = 3,
00177     CAN_CONFIGURATION_MODE = 4,
00178     CAN_EXTERNAL_LOOPBACK_MODE = 5,
00179     CAN_NORMAL_2_0_MODE = 6,
00180     CAN_RESTRICTED_OPERATION_MODE = 7,
00181 } CAN_OP_MODES;
00182
00192 typedef enum
00193 {
00194     CAN_OP_MODE_REQUEST_SUCCESS,    // Requested Operation mode set successfully
00195     CAN_OP_MODE_REQUEST_FAIL,       // Requested Operation mode set failure. Set configuration mode
before setting CAN normal or debug operation mode.
00196     CAN_OP_MODE_SYS_ERROR_OCCURED   // System error occurred while setting Operation mode.
00197 } CAN_OP_MODE_STATUS;
00198
00208 typedef enum
00209 {
00210     CAN_TX_FIFO_FULL,
00211     CAN_TX_FIFO_AVAILABLE,
00212 } CAN_TX_FIFO_STATUS;
00213
00223 typedef enum
00224 {
00225     /*DLC_0 to DLC_8 for CAN 2.0 and CAN FD*/
00226     DLC_0,
00227     DLC_1,
00228     DLC_2,
00229     DLC_3,
00230     DLC_4,
00231     DLC_5,
00232     DLC_6,
00233     DLC_7,
00234     DLC_8,
00235
00236     //Supported only in CAN FD mode
00237     /*DLC_12 to DLC_64 for CAN FD*/
00238     DLC_12,
00239     DLC_16,
00240     DLC_20,
00241     DLC_24,
00242     DLC_32,
00243     DLC_48,
00244     DLC_64,
00245 } CAN_DLC;
00246
00247 #endif //CAN_TYPES_H

```

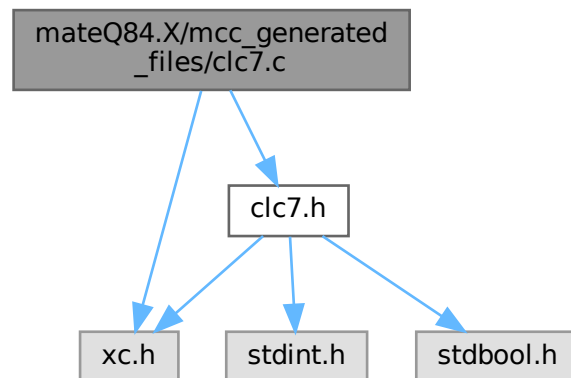
6.84 mateQ84.X/mcc_generated_files/clc7.c File Reference

```

#include <xc.h>
#include "clc7.h"

```

Include dependency graph for `clc7.c`:



Functions

- void [CLC7_Initialize](#) (void)
- bool [CLC7_OutputStatusGet](#) (void)

6.84.1 Function Documentation

6.84.1.1 CLC7_Initialize()

```
void CLC7_Initialize (
    void )
```

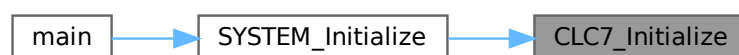
CLC7 Generated Driver File

@Company Microchip Technology Inc.

@File Name [clc7.c](#)

@Summary This is the generated driver implementation file for the CLC7 driver using PIC10 / PIC12 / PIC16 / PIC18 MCUs

@Description This source file provides implementations for driver APIs for CLC7. Generation Information : Product Revision : PIC10 / PIC12 / PIC16 / PIC18 MCUs - 1.81.8 Device : PIC18F47Q84 Driver Version : 1.0.0 The generated drivers are tested against the following: Compiler : XC8 2.36 and above or later MPLAB : MPLAB X 6.00
Section: Included Files Section: CLC7 APIs Here is the caller graph for this function:



6.84.1.2 CLC7_OutputStatusGet()

```
bool CLC7_OutputStatusGet (
    void )
```

@Summary Returns output pin status of the CLC module.

@Description This routine returns output pin status of the CLC module.

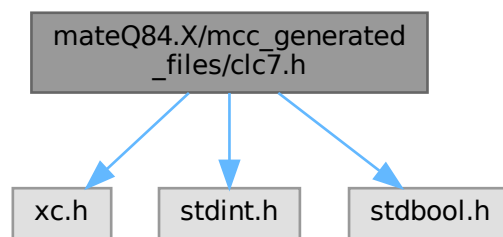
@Param None.

@Returns Output pin status

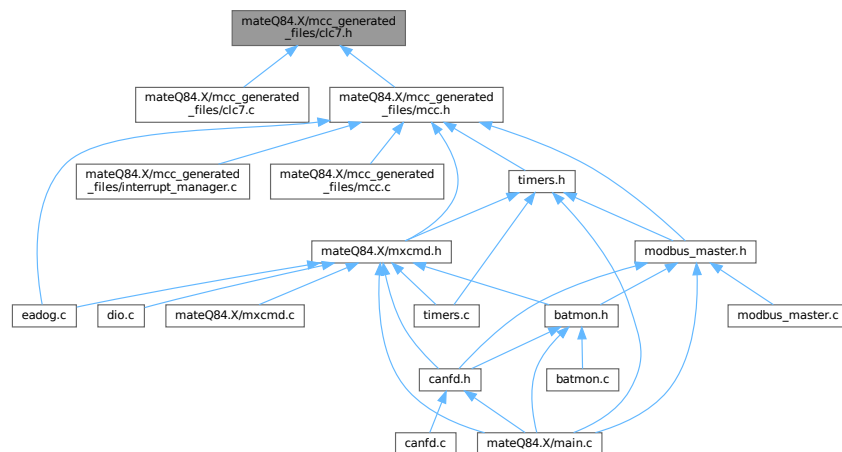
@Example `bool outputStatus; outputStatus = CLC7_OutputStatusGet ();`

6.85 mateQ84.X/mcc_generated_files/clc7.h File Reference

```
#include <xc.h>
#include <stdint.h>
#include <stdbool.h>
Include dependency graph for clc7.h:
```



This graph shows which files directly or indirectly include this file:



Functions

- void [CLC7_Initialize](#) (void)
- bool [CLC7_OutputStatusGet](#) (void)

6.85.1 Function Documentation

6.85.1.1 CLC7_Initialize()

```
void CLC7_Initialize (
    void )
```

CLC7 Generated Driver API Header File

@Company Microchip Technology Inc.

@File Name [clc7.h](#)

@Summary This is the generated header file for the CLC7 driver using PIC10 / PIC12 / PIC16 / PIC18 MCUs

@Description This header file provides APIs for driver for CLC7. Generation Information : Product Revision : PIC10 / PIC12 / PIC16 / PIC18 MCUs - 1.81.8 Device : PIC18F47Q84 Driver Version : 1.0.0 The generated drivers are tested against the following: Compiler : XC8 2.36 and above or later MPLAB : MPLAB X 6.00 Section: Included Files Section: CLC7 APIs @Summary Initializes the CLC7

@Description This routine configures the CLC7 specific control registers

@Preconditions None

@Returns None

@Param None

@Comment

@Example [CLC7_Initialize\(\)](#) ;

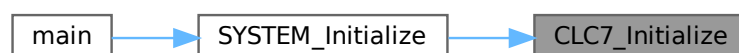
CLC7 Generated Driver File

@Company Microchip Technology Inc.

@File Name [clc7.c](#)

@Summary This is the generated driver implementation file for the CLC7 driver using PIC10 / PIC12 / PIC16 / PIC18 MCUs

@Description This source file provides implementations for driver APIs for CLC7. Generation Information : Product Revision : PIC10 / PIC12 / PIC16 / PIC18 MCUs - 1.81.8 Device : PIC18F47Q84 Driver Version : 1.0.0 The generated drivers are tested against the following: Compiler : XC8 2.36 and above or later MPLAB : MPLAB X 6.00 Section: Included Files Section: CLC7 APIs Here is the caller graph for this function:



6.85.1.2 CLC7_OutputStatusGet()

```
bool CLC7_OutputStatusGet (
    void )
```

@Summary Returns output pin status of the CLC module.

@Description This routine returns output pin status of the CLC module.

@Param None.

@Returns Output pin status

@Example `bool outputStatus; outputStatus = CLC7_OutputStatusGet ();`

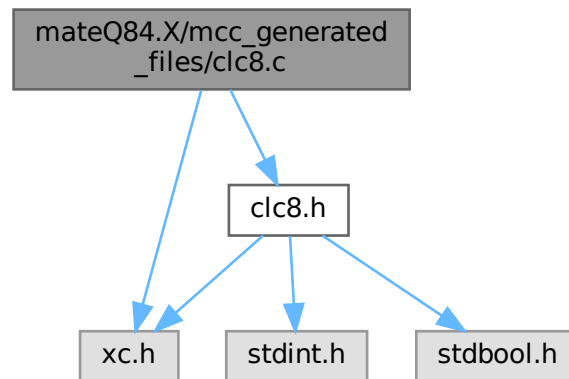
6.86 clc7.h

[Go to the documentation of this file.](#)

```
00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029     party
00030     license terms applicable to your use of third party software (including open source software) that
00031
00032     may accompany Microchip software.
00033
00034     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00035     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00036     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00037     FOR A PARTICULAR PURPOSE.
00038
00039     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00040     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00041     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00042     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00043     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00044     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00045     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00046     SOFTWARE.
00047 */
00048 #ifndef CLC7_H
00049 #define CLC7_H
00050
00051 #include <xc.h>
00052 #include <stdint.h>
00053 #include <stdbool.h>
00054
00055 #ifdef __cplusplus // Provide C++ Compatibility
00056     extern "C" {
00057 #endif
00058
00059 void CLC7_Initialize(void);
00060
00061 bool CLC7_OutputStatusGet(void);
00062
00063 #ifdef __cplusplus // Provide C++ Compatibility
00064     }
00065 #endif
00066 #endif // CLC7_H
00067
```

6.87 mateQ84.X/mcc_generated_files/clc8.c File Reference

```
#include <xc.h>
#include "clc8.h"
Include dependency graph for clc8.c:
```



Functions

- void [CLC8_Initialize](#) (void)
- bool [CLC8_OutputStatusGet](#) (void)

6.87.1 Function Documentation

6.87.1.1 CLC8_Initialize()

```
void CLC8_Initialize (
    void )
```

CLC8 Generated Driver File

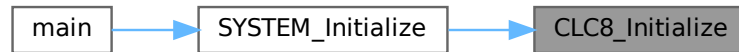
@Company Microchip Technology Inc.

@File Name [clc8.c](#)

@Summary This is the generated driver implementation file for the CLC8 driver using PIC10 / PIC12 / PIC16 / PIC18 MCUs

@Description This source file provides implementations for driver APIs for CLC8. Generation Information : Product Revision : PIC10 / PIC12 / PIC16 / PIC18 MCUs - 1.81.8 Device : PIC18F47Q84 Driver Version : 1.0.0 The

generated drivers are tested against the following: Compiler : XC8 2.36 and above or later MPLAB : MPLAB X 6.00
 Section: Included Files Section: CLC8 APIs Here is the caller graph for this function:



6.87.1.2 CLC8_OutputStatusGet()

```
bool CLC8_OutputStatusGet (
    void )
```

@Summary Returns output pin status of the CLC module.

@Description This routine returns output pin status of the CLC module.

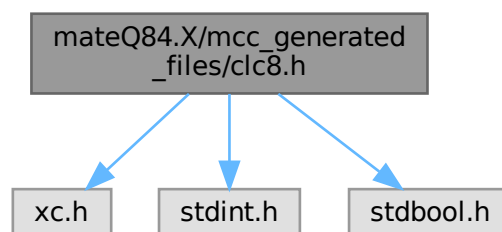
@Param None.

@Returns Output pin status

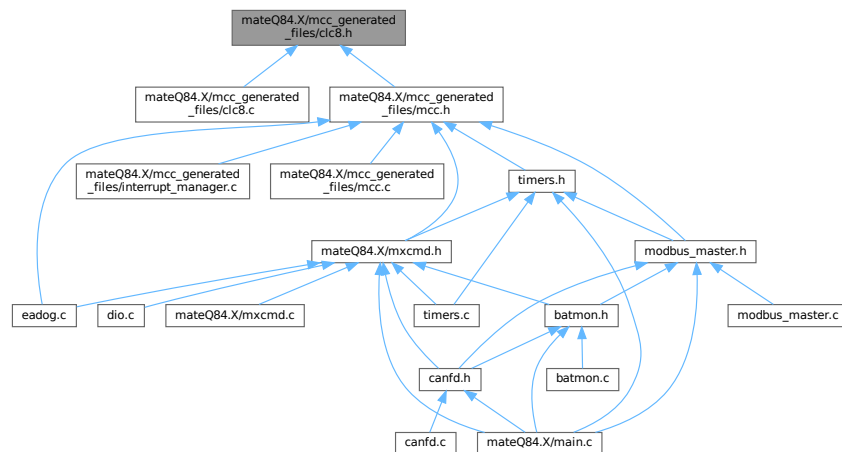
@Example `bool outputStatus; outputStatus = CLC8_OutputStatusGet ();`

6.88 mateQ84.X/mcc_generated_files/clc8.h File Reference

```
#include <xc.h>
#include <stdint.h>
#include <stdbool.h>
Include dependency graph for clc8.h:
```



This graph shows which files directly or indirectly include this file:



Functions

- void [CLC8_Initialize](#) (void)
- bool [CLC8_OutputStatusGet](#) (void)

6.88.1 Function Documentation

6.88.1.1 CLC8_Initialize()

```
void CLC8_Initialize (
    void )
```

CLC8 Generated Driver API Header File

@Company Microchip Technology Inc.

@File Name [clc8.h](#)

@Summary This is the generated header file for the CLC8 driver using PIC10 / PIC12 / PIC16 / PIC18 MCUs

@Description This header file provides APIs for driver for CLC8. Generation Information : Product Revision : PIC10 / PIC12 / PIC16 / PIC18 MCUs - 1.81.8 Device : PIC18F47Q84 Driver Version : 1.0.0 The generated drivers are tested against the following: Compiler : XC8 2.36 and above or later MPLAB : MPLAB X 6.00 Section: Included Files Section: CLC8 APIs @Summary Initializes the CLC8

@Description This routine configures the CLC8 specific control registers

@Preconditions None

@Returns None

@Param None

@Comment

@Example `CLC8_Initialize()`;

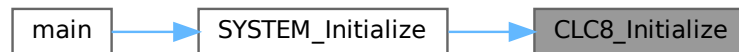
CLC8 Generated Driver File

@Company Microchip Technology Inc.

@File Name `clc8.c`

@Summary This is the generated driver implementation file for the CLC8 driver using PIC10 / PIC12 / PIC16 / PIC18 MCUs

@Description This source file provides implementations for driver APIs for CLC8. Generation Information : Product Revision : PIC10 / PIC12 / PIC16 / PIC18 MCUs - 1.81.8 Device : PIC18F47Q84 Driver Version : 1.0.0 The generated drivers are tested against the following: Compiler : XC8 2.36 and above or later MPLAB : MPLAB X 6.00 Section: Included Files Section: CLC8 APIs Here is the caller graph for this function:



6.88.1.2 CLC8_OutputStatusGet()

```
bool CLC8_OutputStatusGet (
    void )
```

@Summary Returns output pin status of the CLC module.

@Description This routine returns output pin status of the CLC module.

@Param None.

@Returns Output pin status

@Example `bool outputStatus; outputStatus = CLC8_OutputStatusGet ();`

6.89 clc8.h

[Go to the documentation of this file.](#)

```

00001
00023
00024 /*
00025      (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027      Subject to your compliance with these terms, you may use Microchip software and any
00028      derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029      party
00029      license terms applicable to your use of third party software (including open source software) that
00030
00030      may accompany Microchip software.
00031
00032      THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033      EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
```

```

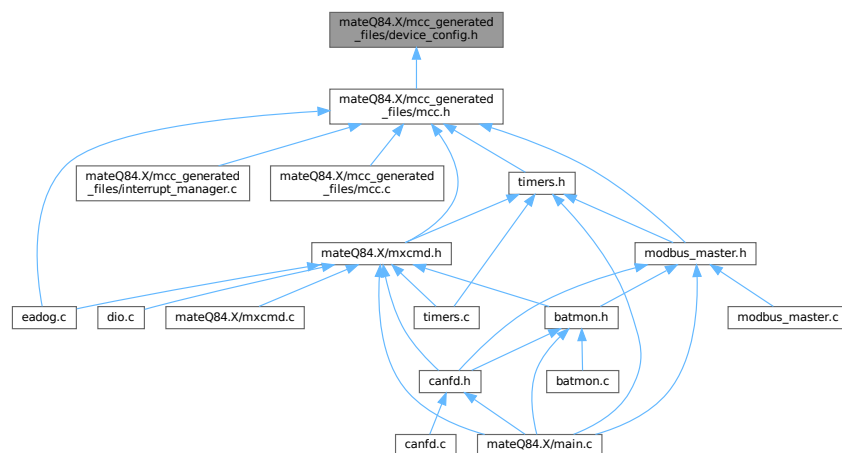
00034     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035     FOR A PARTICULAR PURPOSE.
00036
00037     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044     SOFTWARE.
00045 */
00046
00047 #ifndef CLC8_H
00048 #define CLC8_H
00049
00053
00054 #include <xc.h>
00055 #include <stdint.h>
00056 #include <stdbool.h>
00057
00058 #ifdef __cplusplus // Provide C++ Compatibility
00059     extern "C" {
00060
00061 #endif
00062
00063
00067
00091 void CLC8_Initialize(void);
00092
00112
00113 bool CLC8_OutputStatusGet(void);
00114
00115 #ifdef __cplusplus // Provide C++ Compatibility
00116     }
00117 #endif
00118
00119 #endif
00120
00121 #endif // CLC8_H
00125

```

6.90 mateQ84.X/mcc_generated_files/device_config.c File Reference

6.91 mateQ84.X/mcc_generated_files/device_config.h File Reference

This graph shows which files directly or indirectly include this file:



Macros

- #define [_XTAL_FREQ](#) 40000000

6.91.1 Macro Definition Documentation**6.91.1.1 _XTAL_FREQ**

```
#define _XTAL_FREQ 40000000
```

@Generated PIC10 / PIC12 / PIC16 / PIC18 MCUs Header File

@Company: Microchip Technology Inc.

@File Name: [mcc.c](#)

@Summary: This is the [device_config.h](#) file generated using PIC10 / PIC12 / PIC16 / PIC18 MCUs

@Description: This header file provides implementations for driver APIs for all modules selected in the GUI. Generation Information : Product Revision : PIC10 / PIC12 / PIC16 / PIC18 MCUs - 1.81.8 Device : PIC18F47Q84 Driver Version : 2.00 The generated drivers are tested against the following: Compiler : XC8 2.36 and above or later MPLAB : MPLAB X 6.00

6.92 device_config.h

[Go to the documentation of this file.](#)

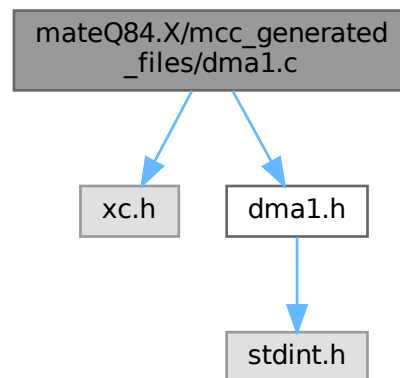
```
00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029     party
00030     license terms applicable to your use of third party software (including open source software) that
00031     may accompany Microchip software.
00032
00033     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00034     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00035     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00036     FOR A PARTICULAR PURPOSE.
00037
00038     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00039     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00040     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00041     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00042     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00043     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00044     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00045     SOFTWARE.
00046 */
00047 #ifndef DEVICE_CONFIG_H
00048 #define DEVICE_CONFIG_H
00049
00050 #define _XTAL_FREQ 40000000
00051
00052 #endif /* DEVICE_CONFIG_H */
```

6.93 mateQ84.X/mcc_generated_files/dma1.c File Reference

```
#include <xc.h>
```

```
#include "dma1.h"
```

Include dependency graph for dma1.c:



Functions

- void [DMA1_Initialize](#) (void)
Initializes the DMA1 module This routine must be called before any other DMA1 routine.
- void [DMA1_SelectSourceRegion](#) (uint8_t region)
Sets the source region as per user selection.
- void [DMA1_SetSourceAddress](#) (uint24_t address)
Sets the Source Address for the DMA packet.
- void [DMA1_SetDestinationAddress](#) (uint16_t address)
Sets the destination address for the DMA packet.
- void [DMA1_SetSourceSize](#) (uint16_t size)
Sets the size of the Source array.
- void [DMA1_SetDestinationSize](#) (uint16_t size)
Sets the size of the destination array.
- uint24_t [DMA1_GetSourcePointer](#) (void)
Returns the current location from which the DMA has read.
- uint16_t [DMA1_GetDestinationPointer](#) (void)
Returns the current location where the DMA last wrote.
- void [DMA1_SetStartTrigger](#) (uint8_t sirq)
Sets the Start Trigger for DMA, doesn't start the transfer.
- void [DMA1_SetAbortTrigger](#) (uint8_t airq)
Sets the Abort Trigger for DMA, doesn't abort the transfer.
- void [DMA1_StartTransfer](#) (void)
Starts the DMA Transfer by setting GO bit.
- void [DMA1_StartTransferWithTrigger](#) (void)
Starts the DMA transfer by enabling the trigger.

- void [DMA1_StopTransfer](#) (void)
Stops all the possible triggers and also clears the GO bit.
- void [DMA1_SetDMAPriority](#) (uint8_t priority)
Unlocks Arbiter - changes priority - locks Arbiter.
- void [__interrupt](#) (irq(IRQ_DMA1SCNT), base(8))
- void [DMA1_SetSCNTInterruptHandler](#) (void(*InterruptHandler)(void))
This routine is used to set the callback for the SCNTI Interrupt.
- void [__interrupt](#) (irq(IRQ_DMA1A), base(8))
- void [DMA1_SetAllInterruptHandler](#) (void(*InterruptHandler)(void))
This routine is used to set the callback for the AI Interrupt.
- void [__interrupt](#) (irq(IRQ_DMA1OR), base(8))
- void [DMA1_SetORInterruptHandler](#) (void(*InterruptHandler)(void))
This routine is used to set the callback for the ORI Interrupt.
- void [DMA1_DefaultInterruptHandler](#) (void)
This is the default Interrupt Handler function.

Variables

- void(* [DMA1_SCNTI_InterruptHandler](#))(void)
- void(* [DMA1_AI_InterruptHandler](#))(void)
- void(* [DMA1_ORI_InterruptHandler](#))(void)

6.93.1 Function Documentation

6.93.1.1 [__interrupt\(\)](#) [1/3]

```
void __interrupt (
    irq(IRQ_DMA1A) ,
    base(8) )
```

6.93.1.2 [__interrupt\(\)](#) [2/3]

```
void __interrupt (
    irq(IRQ_DMA1OR) ,
    base(8) )
```

6.93.1.3 [__interrupt\(\)](#) [3/3]

```
void __interrupt (
    irq(IRQ_DMA1SCNT) ,
    base(8) )
```

6.93.1.4 [DMA1_DefaultInterruptHandler\(\)](#)

```
void DMA1_DefaultInterruptHandler (
    void )
```

This is the default Interrupt Handler function.

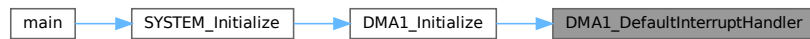
Returns

None

Parameters

<i>None</i>	
-------------	--

Here is the caller graph for this function:

**6.93.1.5 DMA1_GetDestinationPointer()**

```
uint16_t DMA1_GetDestinationPointer (  
    void )
```

Returns the current location where the DMA last wrote.

Returns

Current Destination pointer

Parameters

<i>None</i>	
-------------	--

6.93.1.6 DMA1_GetSourcePointer()

```
uint24_t DMA1_GetSourcePointer (  
    void )
```

Returns the current location from which the DMA has read.

Returns

Current Source pointer

Parameters

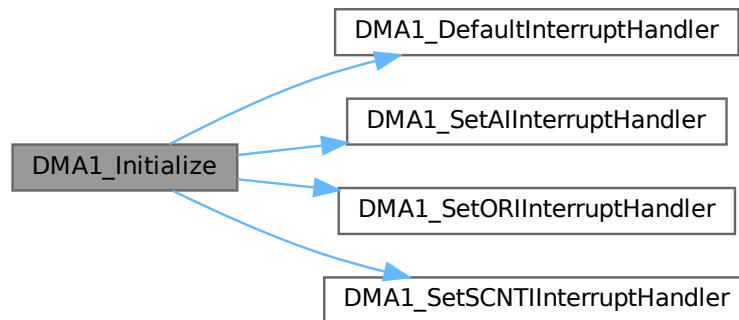
<i>None</i>	
-------------	--

6.93.1.7 DMA1_Initialize()

```
void DMA1_Initialize (
    void )
```

Initializes the DMA1 module This routine must be called before any other DMA1 routine.

Section: DMA1 APIs Here is the call graph for this function:



Here is the caller graph for this function:

**6.93.1.8 DMA1_SelectSourceRegion()**

```
void DMA1_SelectSourceRegion (
    uint8_t region)
```

Sets the source region as per user selection.

Returns

None.

Parameters

in	<i>Desired</i>	source region
----	----------------	---------------

6.93.1.9 DMA1_SetAbortTrigger()

```
void DMA1_SetAbortTrigger (
    uint8_t airq)
```

Sets the Abort Trigger for DMA, doesn't abort the transfer.

Returns

None.

Parameters

in	<i>Abort</i>	Trigger Source
----	--------------	----------------

6.93.1.10 DMA1_SetAllInterruptHandler()

```
void DMA1_SetAllInterruptHandler (
    void(* InterruptHandler ) (void))
```

This routine is used to set the callback for the AI Interrupt.

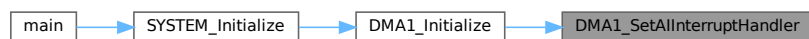
Returns

None

Parameters

<i>Callback</i>	Function to be called
-----------------	-----------------------

Here is the caller graph for this function:



6.93.1.11 DMA1_SetDestinationAddress()

```
void DMA1_SetDestinationAddress (
    uint16_t address)
```

Sets the destination address for the DMA packet.

Returns

None.

Parameters

in	<i>Desired</i>	Destination Address
----	----------------	---------------------

6.93.1.12 DMA1_SetDestinationSize()

```
void DMA1_SetDestinationSize (
    uint16_t size)
```

Sets the size of the destination array.

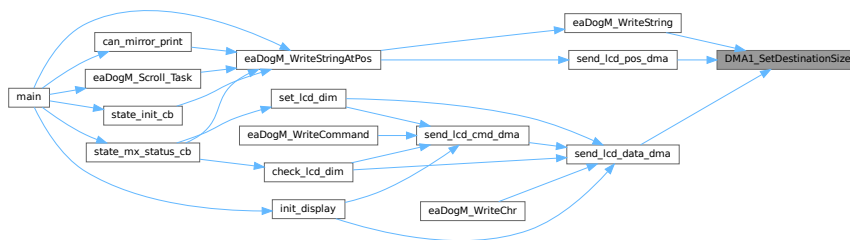
Returns

None.

Parameters

in	<i>Size</i>	of Destination array
----	-------------	----------------------

Here is the caller graph for this function:



6.93.1.13 DMA1_SetDMAPriority()

```
void DMA1_SetDMAPriority (
    uint8_t priority)
```

Unlocks Arbiter - changes priority - locks Arbiter.

Returns

None.

Parameters

in	<i>Priority</i>	of DMA
----	-----------------	--------

Here is the caller graph for this function:



6.93.1.14 DMA1_SetORInterruptHandler()

```
void DMA1_SetORInterruptHandler (
    void(* InterruptHandler ) (void))
```

This routine is used to set the callback for the ORI Interrupt.

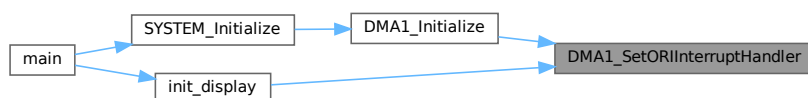
Returns

None

Parameters

<i>Callback</i>	Function to be called
-----------------	-----------------------

Here is the caller graph for this function:



6.93.1.15 DMA1_SetSCNTInterruptHandler()

```
void DMA1_SetSCNTInterruptHandler (
    void(* InterruptHandler ) (void))
```

This routine is used to set the callback for the SCNTI Interrupt.

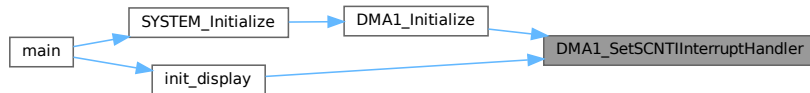
Returns

None

Parameters

<i>Callback</i>	Function to be called
-----------------	-----------------------

Here is the caller graph for this function:



6.93.1.16 DMA1_SetSourceAddress()

```
void DMA1_SetSourceAddress (
    uint24_t address)
```

Sets the Source Address for the DMA packet.

Returns

None.

Parameters

in	<i>Desired</i>	Source Address
----	----------------	----------------

6.93.1.17 DMA1_SetSourceSize()

```
void DMA1_SetSourceSize (
    uint16_t size)
```

Sets the size of the Source array.

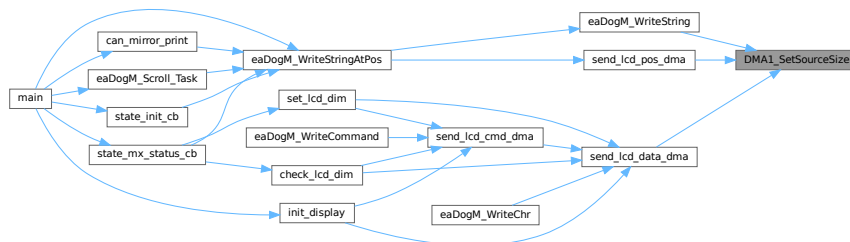
Returns

None.

Parameters

in	<i>Size</i>	of Source Array
----	-------------	-----------------

Here is the caller graph for this function:



6.93.1.18 DMA1_SetStartTrigger()

```
void DMA1_SetStartTrigger (
    uint8_t irq)
```

Sets the Start Trigger for DMA, doesn't start the transfer.

Returns

None.

Parameters

in	Start	Trigger Source
----	-------	----------------

6.93.1.19 DMA1_StartTransfer()

```
void DMA1_StartTransfer (
    void )
```

Starts the DMA Transfer by setting GO bit.

Returns

None.

Parameters

None.	
-------	--

6.93.1.20 DMA1_StartTransferWithTrigger()

```
void DMA1_StartTransferWithTrigger (
    void )
```

Starts the DMA transfer by enabling the trigger.

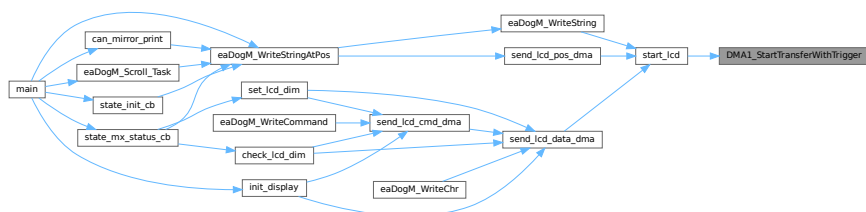
Returns

None.

Parameters

None.	
-------	--

Here is the caller graph for this function:



6.93.1.21 DMA1_StopTransfer()

```
void DMA1_StopTransfer (
    void )
```

Stops all the possible triggers and also clears the GO bit.

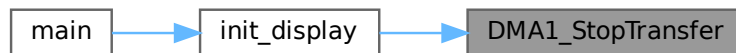
Returns

None.

Parameters

None.	
-------	--

Here is the caller graph for this function:



6.93.2 Variable Documentation

6.93.2.1 DMA1_AI_InterruptHandler

```
void(* DMA1_AI_InterruptHandler) (void) (
    void )
```

6.93.2.2 DMA1_ORI_InterruptHandler

```
void(* DMA1_ORI_InterruptHandler) (void) (
    void )
```

6.93.2.3 DMA1_SCNTI_InterruptHandler

```
void(* DMA1_SCNTI_InterruptHandler) (void) (
    void )
```

DMA1 Generated Driver File

@Company Microchip Technology Inc.

@File Name [dma1.c](#)

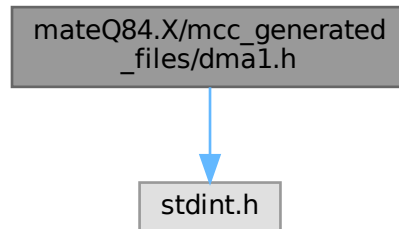
@Summary This is the generated driver implementation file for the DMA1 driver using PIC10 / PIC12 / PIC16 / PIC18 MCUs

@Description This source file provides APIs for DMA1. Generation Information : Product Revision : PIC10 / PIC12 / PIC16 / PIC18 MCUs - 1.81.8 Device : PIC18F47Q84 Driver Version : 1.0.0 The generated drivers are tested against the following: Compiler : XC8 2.36 and above MPLAB : MPLAB X 6.00 Section: Included Files

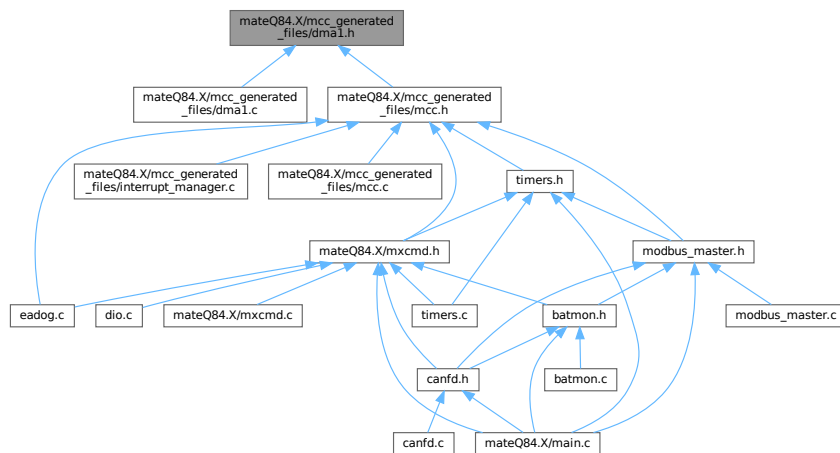
6.94 mateQ84.X/mcc_generated_files/dma1.h File Reference

```
#include <stdint.h>
```

Include dependency graph for dma1.h:



This graph shows which files directly or indirectly include this file:



Functions

- void [DMA1_Initialize](#) (void)
Initializes the DMA1 module This routine must be called before any other DMA1 routine.
- void [DMA1_SelectSourceRegion](#) (uint8_t region)
Sets the source region as per user selection.
- void [DMA1_SetSourceAddress](#) (uint24_t address)
Sets the Source Address for the DMA packet.
- void [DMA1_SetDestinationAddress](#) (uint16_t address)
Sets the destination address for the DMA packet.
- void [DMA1_SetSourceSize](#) (uint16_t size)
Sets the size of the Source array.

- void [DMA1_SetDestinationSize](#) (uint16_t size)
Sets the size of the destination array.
- uint24_t [DMA1_GetSourcePointer](#) (void)
Returns the current location from which the DMA has read.
- uint16_t [DMA1_GetDestinationPointer](#) (void)
Returns the current location where the DMA last wrote.
- void [DMA1_SetStartTrigger](#) (uint8_t sirq)
Sets the Start Trigger for DMA, doesn't start the transfer.
- void [DMA1_SetAbortTrigger](#) (uint8_t airq)
Sets the Abort Trigger for DMA, doesn't abort the transfer.
- void [DMA1_StartTransfer](#) (void)
Starts the DMA Transfer by setting GO bit.
- void [DMA1_StartTransferWithTrigger](#) (void)
Starts the DMA transfer by enabling the trigger.
- void [DMA1_StopTransfer](#) (void)
Stops all the possible triggers and also clears the GO bit.
- void [DMA1_SetDMAPriority](#) (uint8_t priority)
Unlocks Arbiter - changes priority - locks Arbiter.
- void [DMA1_SetSCNTIInterruptHandler](#) (void(*InterruptHandler)(void))
This routine is used to set the callback for the SCNTI Interrupt.
- void [DMA1_SetAllInterruptHandler](#) (void(*InterruptHandler)(void))
This routine is used to set the callback for the AI Interrupt.
- void [DMA1_SetORIIInterruptHandler](#) (void(*InterruptHandler)(void))
This routine is used to set the callback for the ORI Interrupt.
- void [DMA1_DefaultInterruptHandler](#) (void)
This is the default Interrupt Handler function.

Variables

- uint8_t [lcd_dma_buf](#) [32]

6.94.1 Function Documentation

6.94.1.1 DMA1_DefaultInterruptHandler()

```
void DMA1_DefaultInterruptHandler (
    void )
```

This is the default Interrupt Handler function.

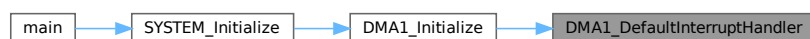
Returns

None

Parameters

None	
------	--

Here is the caller graph for this function:



6.94.1.2 DMA1_GetDestinationPointer()

```
uint16_t DMA1_GetDestinationPointer (
    void )
```

Returns the current location where the DMA last wrote.

Returns

Current Destination pointer

Parameters

None	
------	--

6.94.1.3 DMA1_GetSourcePointer()

```
uint24_t DMA1_GetSourcePointer (
    void )
```

Returns the current location from which the DMA has read.

Returns

Current Source pointer

Parameters

None	
------	--

6.94.1.4 DMA1_Initialize()

```
void DMA1_Initialize (
    void )
```

Initializes the DMA1 module This routine must be called before any other DMA1 routine.

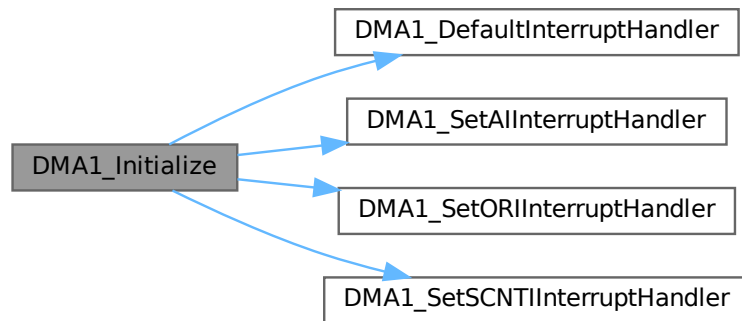
Returns

None.

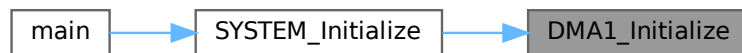
Parameters

None.	
-------	--

Section: DMA1 APIs Here is the call graph for this function:



Here is the caller graph for this function:



6.94.1.5 DMA1_SelectSourceRegion()

```
void DMA1_SelectSourceRegion (
    uint8_t region)
```

Sets the source region as per user selection.

Returns

None.

Parameters

in	<i>Desired</i>	source region
----	----------------	---------------

6.94.1.6 DMA1_SetAbortTrigger()

```
void DMA1_SetAbortTrigger (
    uint8_t airq)
```

Sets the Abort Trigger for DMA, doesn't abort the transfer.

Returns

None.

Parameters

in	<i>Abort</i>	Trigger Source
----	--------------	----------------

6.94.1.7 DMA1_SetAllInterruptHandler()

```
void DMA1_SetAllInterruptHandler (
    void(* InterruptHandler ) (void))
```

This routine is used to set the callback for the AI Interrupt.

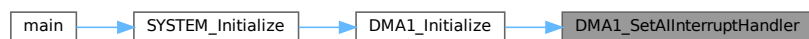
Returns

None

Parameters

<i>Callback</i>	Function to be called
-----------------	-----------------------

Here is the caller graph for this function:



6.94.1.8 DMA1_SetDestinationAddress()

```
void DMA1_SetDestinationAddress (
    uint16_t address)
```

Sets the destination address for the DMA packet.

Returns

None.

Parameters

in	<i>Desired</i>	Destination Address
----	----------------	---------------------

6.94.1.9 DMA1_SetDestinationSize()

```
void DMA1_SetDestinationSize (
    uint16_t size)
```

Sets the size of the destination array.

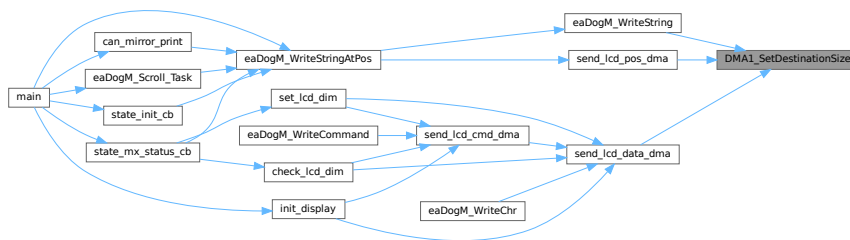
Returns

None.

Parameters

in	<i>Size</i>	of Destination array
----	-------------	----------------------

Here is the caller graph for this function:



6.94.1.10 DMA1_SetDMAPriority()

```
void DMA1_SetDMAPriority (
    uint8_t priority)
```

Unlocks Arbiter - changes priority - locks Arbiter.

Returns

None.

Parameters

in	<i>Priority</i>	of DMA
----	-----------------	--------

Here is the caller graph for this function:



6.94.1.11 DMA1_SetORInterruptHandler()

```
void DMA1_SetORInterruptHandler (
    void(* InterruptHandler ) (void))
```

This routine is used to set the callback for the ORI Interrupt.

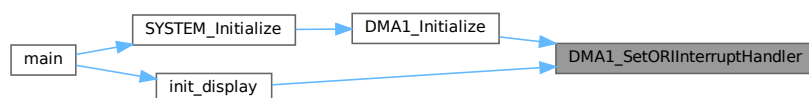
Returns

None

Parameters

<i>Callback</i>	Function to be called
-----------------	-----------------------

Here is the caller graph for this function:



6.94.1.12 DMA1_SetSCNTInterruptHandler()

```
void DMA1_SetSCNTInterruptHandler (
    void(* InterruptHandler ) (void))
```

This routine is used to set the callback for the SCNTI Interrupt.

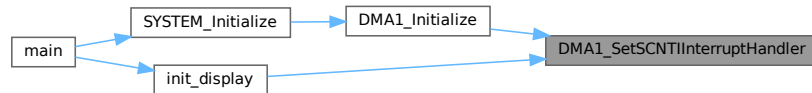
Returns

None

Parameters

<i>Callback</i>	Function to be called
-----------------	-----------------------

Here is the caller graph for this function:



6.94.1.13 DMA1_SetSourceAddress()

```
void DMA1_SetSourceAddress (
    uint24_t address)
```

Sets the Source Address for the DMA packet.

Returns

None.

Parameters

in	<i>Desired</i>	Source Address
----	----------------	----------------

6.94.1.14 DMA1_SetSourceSize()

```
void DMA1_SetSourceSize (
    uint16_t size)
```

Sets the size of the Source array.

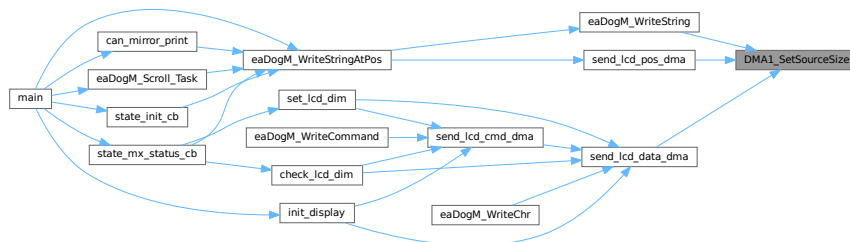
Returns

None.

Parameters

in	<i>Size</i>	of Source Array
----	-------------	-----------------

Here is the caller graph for this function:



6.94.1.15 DMA1_SetStartTrigger()

```
void DMA1_SetStartTrigger (
    uint8_t irq)
```

Sets the Start Trigger for DMA, doesn't start the transfer.

Returns

None.

Parameters

in	Start	Trigger Source
----	-------	----------------

6.94.1.16 DMA1_StartTransfer()

```
void DMA1_StartTransfer (
    void )
```

Starts the DMA Transfer by setting GO bit.

Returns

None.

Parameters

None.	
-------	--

6.94.1.17 DMA1_StartTransferWithTrigger()

```
void DMA1_StartTransferWithTrigger (
    void )
```

Starts the DMA transfer by enabling the trigger.

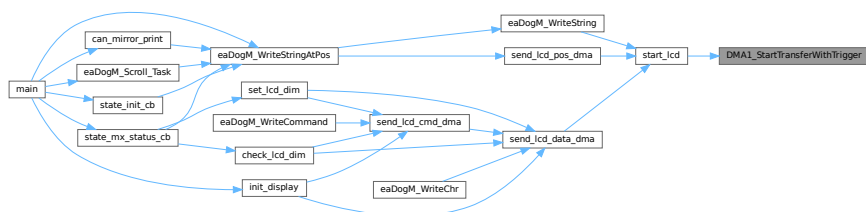
Returns

None.

Parameters

None.	
-------	--

Here is the caller graph for this function:



6.94.1.18 DMA1_StopTransfer()

```
void DMA1_StopTransfer (
    void )
```

Stops all the possible triggers and also clears the GO bit.

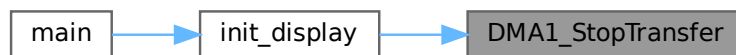
Returns

None.

Parameters

None.	
-------	--

Here is the caller graph for this function:



6.94.2 Variable Documentation

6.94.2.1 lcd_dma_buf

```
uint8_t lcd_dma_buf[32]
```

DMA1 Generated Driver API Header File

@Company Microchip Technology Inc.

@File Name [dma1.h](#)

@Summary This is the generated header file for the DMA1 driver using PIC10 / PIC12 / PIC16 / PIC18 MCUs

@Description This header file provides APIs for driver for DMA1. Generation Information : Product Revision : PIC10 / PIC12 / PIC16 / PIC18 MCUs - 1.81.8 Device : PIC18F47Q84 Driver Version : 1.0.0 The generated drivers are tested against the following: Compiler : XC8 2.36 and above or later MPLAB : MPLAB X 6.00 Section: Included Files

6.95 dma1.h

[Go to the documentation of this file.](#)

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029     party
    license terms applicable to your use of third party software (including open source software) that
    may accompany Microchip software.
00030
00031
00032     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035     FOR A PARTICULAR PURPOSE.
00036
00037     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044     SOFTWARE.
00045 */
00046
00047 #ifndef DMA1_H
00048 #define DMA1_H
00049
00053 #include <stdint.h>
00054
00055 uint8_t lcd_dma_buf[32];
00056
00063 void DMA1_Initialize(void);
00064
00070 void DMA1_SelectSourceRegion(uint8_t region);
00071
00077 void DMA1_SetSourceAddress(uint24_t address);
00078
00084 void DMA1_SetDestinationAddress(uint16_t address);
00085
00091 void DMA1_SetSourceSize(uint16_t size);
00092
00098 void DMA1_SetDestinationSize(uint16_t size);
00099
00105 uint24_t DMA1_GetSourcePointer(void);
00106
00112 uint16_t DMA1_GetDestinationPointer(void);
00113
00119 void DMA1_SetStartTrigger(uint8_t sirq);
00120
00126 void DMA1_SetAbortTrigger(uint8_t airq);
00127
00133 void DMA1_StartTransfer(void);
00134
00140 void DMA1_StartTransferWithTrigger(void);
00141
00147 void DMA1_StopTransfer(void);
00148
00154 void DMA1_SetDMAPriority(uint8_t priority);
00155
00161 void DMA1_SetSCNTIInterruptHandler(void (* InterruptHandler)(void));
00162
00163
00169 void DMA1_SetAIInterruptHandler(void (* InterruptHandler)(void));
00170
00176 void DMA1_SetORIInterruptHandler(void (* InterruptHandler)(void));
00182 void DMA1_DefaultInterruptHandler(void);
00183 #endif //DMA1_H

```

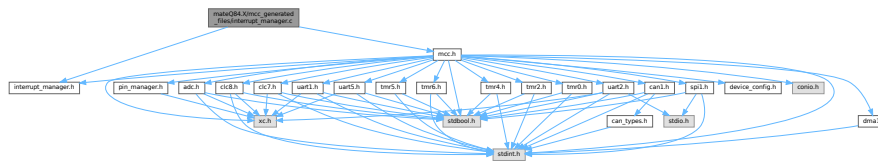
6.96 mateQ84.X/mcc_generated_files/interrupt_manager.c File Reference

```

#include "interrupt_manager.h"
#include "mcc.h"

```

Include dependency graph for interrupt_manager.c:



Functions

- void [INTERRUPT_Initialize](#) (void)
- void [__interrupt](#) (irq(default), base(8))

6.96.1 Function Documentation

6.96.1.1 __interrupt()

```
void __interrupt (
    irq(default) ,
    base(8) )
```

6.96.1.2 INTERRUPT_Initialize()

```
void INTERRUPT_Initialize (
    void )
```

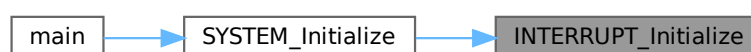
Generated Interrupt Manager Header File

@Company: Microchip Technology Inc.

@File Name: [interrupt_manager.h](#)

@Summary: This is the Interrupt Manager file generated using PIC10 / PIC12 / PIC16 / PIC18 MCUs

@Description: This header file provides implementations for global interrupt handling. For individual peripheral handlers please see the peripheral driver for all modules selected in the GUI. Generation Information : Product Revision : PIC10 / PIC12 / PIC16 / PIC18 MCUs - 1.81.8 Device : PIC18F47Q84 Driver Version : 2.12 The generated drivers are tested against the following: Compiler : XC8 2.36 and above or later MPLAB : MPLAB X 6.00 Here is the caller graph for this function:



6.97.1.3 INTERRUPT_GlobalInterruptLowDisable

```
#define INTERRUPT_GlobalInterruptLowDisable()
```

Value:

```
(INTCON0bits.GIEL = 0)
```

@Param none @Returns none @Description This macro will disable low priority global interrupts. @Example [INTERRUPT_GlobalInterruptLowDisable\(\);](#)

6.97.1.4 INTERRUPT_GlobalInterruptLowEnable

```
#define INTERRUPT_GlobalInterruptLowEnable()
```

Value:

```
(INTCON0bits.GIEL = 1)
```

@Param none @Returns none @Description This macro will enable low priority global interrupts. @Example [INTERRUPT_GlobalInterruptLowEnable\(\);](#)

6.97.2 Function Documentation

6.97.2.1 INTERRUPT_Initialize()

```
void INTERRUPT_Initialize (
    void )
```

@Param none @Returns none @Description Initializes PIC18 peripheral interrupt priorities; enables/disables priority vectors @Example [INTERRUPT_Initialize\(\);](#)

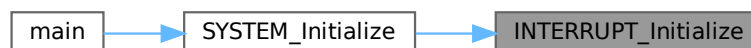
Generated Interrupt Manager Header File

@Company: Microchip Technology Inc.

@File Name: [interrupt_manager.h](#)

@Summary: This is the Interrupt Manager file generated using PIC10 / PIC12 / PIC16 / PIC18 MCUs

@Description: This header file provides implementations for global interrupt handling. For individual peripheral handlers please see the peripheral driver for all modules selected in the GUI. Generation Information : Product Revision : PIC10 / PIC12 / PIC16 / PIC18 MCUs - 1.81.8 Device : PIC18F47Q84 Driver Version : 2.12 The generated drivers are tested against the following: Compiler : XC8 2.36 and above or later MPLAB : MPLAB X 6.00 Here is the caller graph for this function:



6.98 interrupt_manager.h

[Go to the documentation of this file.](#)

```

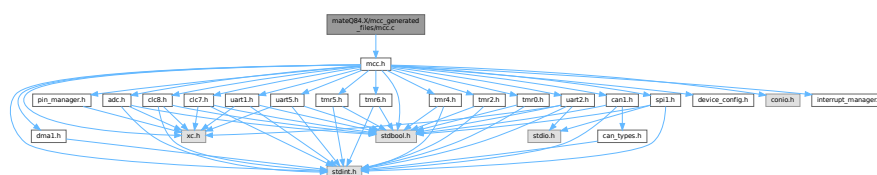
00001
00025
00026 /*
00027     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00028
00029     Subject to your compliance with these terms, you may use Microchip software and any
00030     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00031     party
00032     license terms applicable to your use of third party software (including open source software) that
00033     may accompany Microchip software.
00034
00035     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00036     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00037     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00038     FOR A PARTICULAR PURPOSE.
00039
00040     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00041     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00042     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00043     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00044     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00045     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00046     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00047     SOFTWARE.
00048 */
00049 #ifndef INTERRUPT_MANAGER_H
00050 #define INTERRUPT_MANAGER_H
00051
00052 #define INTERRUPT_GlobalInterruptHighEnable() (INTCON0bits.GIEH = 1)
00053
00054 #define INTERRUPT_GlobalInterruptHighDisable() (INTCON0bits.GIEH = 0)
00055
00056 #define INTERRUPT_GlobalInterruptLowEnable() (INTCON0bits.GIEL = 1)
00057
00058 #define INTERRUPT_GlobalInterruptLowDisable() (INTCON0bits.GIEL = 0)
00059
00060 void INTERRUPT_Initialize (void);
00061
00062 #endif // INTERRUPT_MANAGER_H

```

6.99 mateQ84.X/mcc_generated_files/mcc.c File Reference

```
#include "mcc.h"
```

Include dependency graph for mcc.c:



Functions

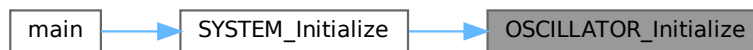
- void [SYSTEM_Initialize](#) (void)
- void [OSCILLATOR_Initialize](#) (void)
- void [PMD_Initialize](#) (void)
- void [SystemArbiter_Initialize](#) (void)

6.99.1 Function Documentation

6.99.1.1 OSCILLATOR_Initialize()

```
void OSCILLATOR_Initialize (
    void )
```

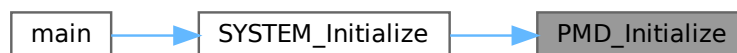
@Param none @Returns none @Description Initializes the oscillator to the default states configured in the MCC GUI @Example [OSCILLATOR_Initialize\(void\)](#); Here is the caller graph for this function:



6.99.1.2 PMD_Initialize()

```
void PMD_Initialize (
    void )
```

@Param none @Returns none @Description Initializes the PMD module to the default states configured in the MCC GUI @Example [PMD_Initialize\(void\)](#); Here is the caller graph for this function:



6.99.1.3 SYSTEM_Initialize()

```
void SYSTEM_Initialize (
    void )
```

@Generated PIC10 / PIC12 / PIC16 / PIC18 MCUs Source File

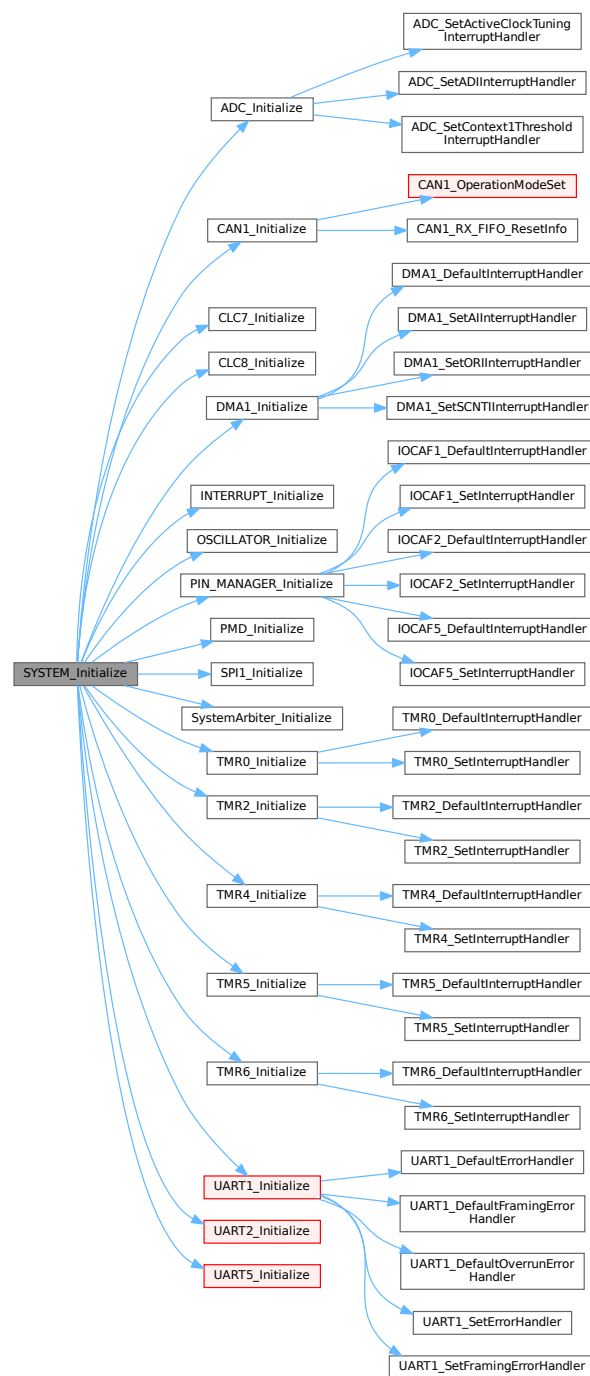
@Company: Microchip Technology Inc.

@File Name: [mcc.c](#)

@Summary: This is the [mcc.c](#) file generated using PIC10 / PIC12 / PIC16 / PIC18 MCUs

@Description: This header file provides implementations for driver APIs for all modules selected in the GUI. Generation Information : Product Revision : PIC10 / PIC12 / PIC16 / PIC18 MCUs - 1.81.8 Device : PIC18F47Q84

Driver Version : 2.00 The generated drivers are tested against the following: Compiler : XC8 2.36 and above or later
MPLAB : MPLAB X 6.00 Here is the call graph for this function:



Here is the caller graph for this function:



6.99.1.4 SystemArbiter_Initialize()

```
void SystemArbiter_Initialize (
    void )
```

@Param none @Returns none @Description Initializes the System Arbiter for DMA to the default priority. @↔ Example SystemArbiter_DMA_Initialize(); Here is the caller graph for this function:

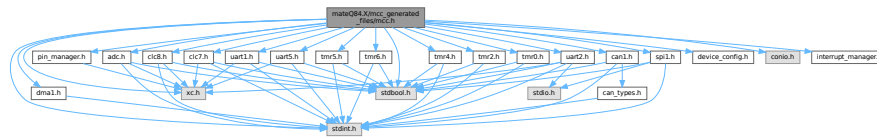


6.100 mateQ84.X/mcc_generated_files/mcc.h File Reference

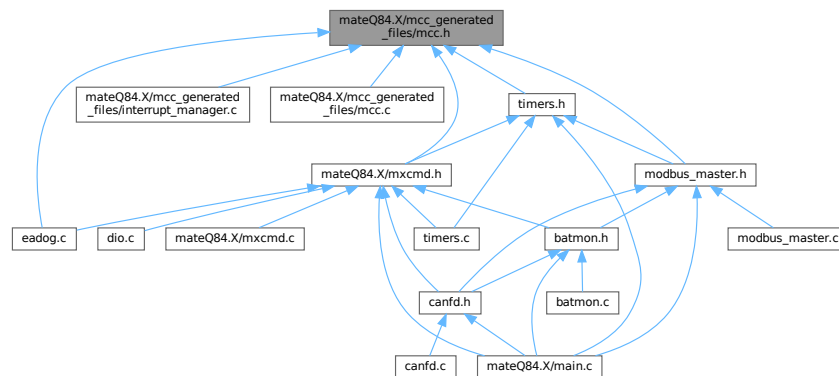
```
#include <xc.h>
#include "device_config.h"
#include "pin_manager.h"
#include <stdint.h>
#include <stdbool.h>
#include <conio.h>
#include "interrupt_manager.h"
#include "adc.h"
#include "dma1.h"
#include "tmr5.h"
#include "tmr6.h"
#include "tmr4.h"
#include "tmr2.h"
#include "tmr0.h"
#include "clc8.h"
#include "clc7.h"
#include "uart2.h"
#include "uart1.h"
#include "uart5.h"
#include "can1.h"
```

```
#include "spi1.h"
```

Include dependency graph for `mcc.h`:



This graph shows which files directly or indirectly include this file:



Functions

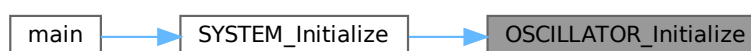
- void [SYSTEM_Initialize](#) (void)
- void [OSCILLATOR_Initialize](#) (void)
- void [PMD_Initialize](#) (void)
- void [SystemArbiter_Initialize](#) (void)

6.100.1 Function Documentation

6.100.1.1 OSCILLATOR_Initialize()

```
void OSCILLATOR_Initialize (
    void )
```

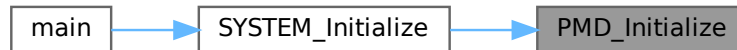
@Param none @Returns none @Description Initializes the oscillator to the default states configured in the MCC GUI @Example [OSCILLATOR_Initialize\(void\)](#); Here is the caller graph for this function:



6.100.1.2 PMD_Initialize()

```
void PMD_Initialize (
    void )
```

@Param none @Returns none @Description Initializes the PMD module to the default states configured in the MCC GUI @Example [PMD_Initialize\(void\)](#); Here is the caller graph for this function:



6.100.1.3 SYSTEM_Initialize()

```
void SYSTEM_Initialize (
    void )
```

@Generated PIC10 / PIC12 / PIC16 / PIC18 MCUs Header File

@Company: Microchip Technology Inc.

@File Name: [mcc.h](#)

@Summary: This is the [mcc.h](#) file generated using PIC10 / PIC12 / PIC16 / PIC18 MCUs

@Description: This header file provides implementations for driver APIs for all modules selected in the GUI. Generation Information : Product Revision : PIC10 / PIC12 / PIC16 / PIC18 MCUs - 1.81.8 Device : PIC18F47Q84 Driver Version : 2.00 The generated drivers are tested against the following: Compiler : XC8 2.36 and above or later MPLAB : MPLAB X 6.00 @Param none @Returns none @Description Initializes the device to the default states configured in the MCC GUI @Example [SYSTEM_Initialize\(void\)](#);

@Generated PIC10 / PIC12 / PIC16 / PIC18 MCUs Source File

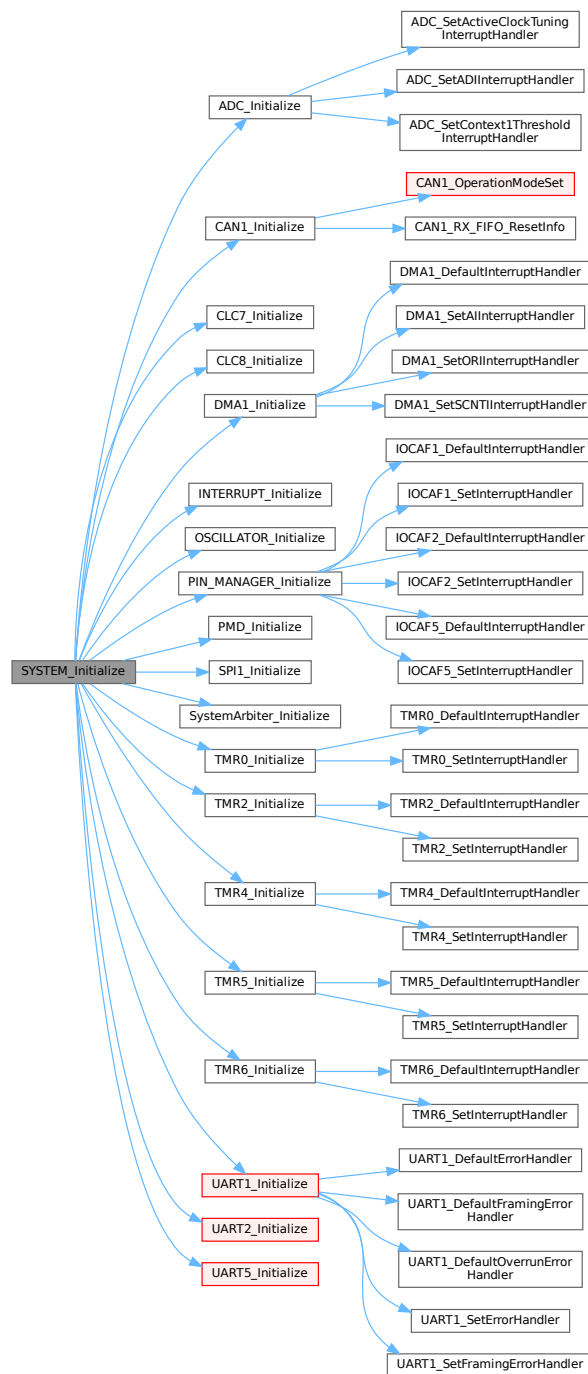
@Company: Microchip Technology Inc.

@File Name: [mcc.c](#)

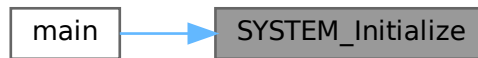
@Summary: This is the [mcc.c](#) file generated using PIC10 / PIC12 / PIC16 / PIC18 MCUs

@Description: This header file provides implementations for driver APIs for all modules selected in the GUI. Generation Information : Product Revision : PIC10 / PIC12 / PIC16 / PIC18 MCUs - 1.81.8 Device : PIC18F47Q84

Driver Version : 2.00 The generated drivers are tested against the following: Compiler : XC8 2.36 and above or later
MPLAB : MPLAB X 6.00 Here is the call graph for this function:



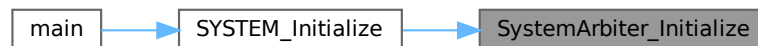
Here is the caller graph for this function:



6.100.1.4 SystemArbiter_Initialize()

```
void SystemArbiter_Initialize (
    void )
```

@Param none @Returns none @Description Initializes the System Arbiter for DMA to the default priority. @↔
Example SystemArbiter_DMA_Initialize(); Here is the caller graph for this function:



6.101 mcc.h

[Go to the documentation of this file.](#)

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029     party
00029     license terms applicable to your use of third party software (including open source software) that
00030     may accompany Microchip software.
00031
00032     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035     FOR A PARTICULAR PURPOSE.
00036
00037     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044     SOFTWARE.
00045 */
00046
00047 #ifndef MCC_H
00048 #define MCC_H
00049 #include <xc.h>
00050 #include "device_config.h"
```

```

00051 #include "pin_manager.h"
00052 #include <stdint.h>
00053 #include <stdbool.h>
00054 #include <conio.h>
00055 #include "interrupt_manager.h"
00056 #include "adc.h"
00057 #include "dma1.h"
00058 #include "tmr5.h"
00059 #include "tmr6.h"
00060 #include "tmr4.h"
00061 #include "tmr2.h"
00062 #include "tmr0.h"
00063 #include "clc8.h"
00064 #include "clc7.h"
00065 #include "uart2.h"
00066 #include "uart1.h"
00067 #include "uart5.h"
00068 #include "can1.h"
00069 #include "spi1.h"
00070
00071
00072
00084 void SYSTEM_Initialize(void);
00085
00097 void OSCILLATOR_Initialize(void);
00098
00110 void PMD_Initialize(void);
00111
00122 void SystemArbiter_Initialize(void);
00123
00124 #endif /* MCC_H */

```

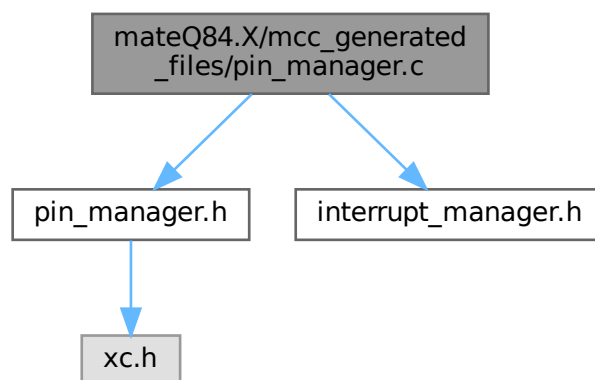
6.102 mateQ84.X/mcc_generated_files/pin_manager.c File Reference

```

#include "pin_manager.h"
#include "interrupt_manager.h"

```

Include dependency graph for pin_manager.c:



Functions

- void `PIN_MANAGER_Initialize` (void)
- void `__interrupt` (irq(IOC), base(8), low_priority)
- void `IOCAF1_ISR` (void)
- void `IOCAF1_SetInterruptHandler` (void(*InterruptHandler)(void))

- void [IOCAF1_DefaultInterruptHandler](#) (void)
- void [IOCAF2_ISR](#) (void)
- void [IOCAF2_SetInterruptHandler](#) (void(*InterruptHandler)(void))
- void [IOCAF2_DefaultInterruptHandler](#) (void)
- void [IOCAF5_ISR](#) (void)
- void [IOCAF5_SetInterruptHandler](#) (void(*InterruptHandler)(void))
- void [IOCAF5_DefaultInterruptHandler](#) (void)

Variables

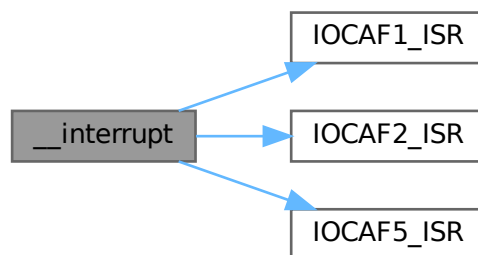
- void(* [IOCAF1_InterruptHandler](#))(void)
- void(* [IOCAF2_InterruptHandler](#))(void)
- void(* [IOCAF5_InterruptHandler](#))(void)

6.102.1 Function Documentation

6.102.1.1 `__interrupt()`

```
void __interrupt (
    irq(IOC) ,
    base(8) ,
    low_priority )
```

Here is the call graph for this function:



6.102.1.2 `IOCAF1_DefaultInterruptHandler()`

```
void IOCAF1_DefaultInterruptHandler (
    void )
```

Default interrupt handler for IOCAF1 Here is the caller graph for this function:



6.102.1.3 IOCAF1_ISR()

```
void IOCAF1_ISR (
    void )
```

IOCAF1 Interrupt Service Routine Here is the caller graph for this function:



6.102.1.4 IOCAF1_SetInterruptHandler()

```
void IOCAF1_SetInterruptHandler (
    void(* InterruptHandler ) (void))
```

Allows selecting an interrupt handler for IOCAF1 at application runtime Here is the caller graph for this function:



6.102.1.5 IOCAF2_DefaultInterruptHandler()

```
void IOCAF2_DefaultInterruptHandler (
    void )
```

Default interrupt handler for IOCAF2 Here is the caller graph for this function:



6.102.1.6 IOCAF2_ISR()

```
void IOCAF2_ISR (  
    void )
```

IOCAF2 Interrupt Service Routine Here is the caller graph for this function:



6.102.1.7 IOCAF2_SetInterruptHandler()

```
void IOCAF2_SetInterruptHandler (  
    void(* InterruptHandler ) (void))
```

Allows selecting an interrupt handler for IOCAF2 at application runtime Here is the caller graph for this function:



6.102.1.8 IOCAF5_DefaultInterruptHandler()

```
void IOCAF5_DefaultInterruptHandler (  
    void )
```

Default interrupt handler for IOCAF5 Here is the caller graph for this function:



6.102.1.9 IOCAF5_ISR()

```
void IOCAF5_ISR (
    void )
```

IOCAF5 Interrupt Service Routine Here is the caller graph for this function:



6.102.1.10 IOCAF5_SetInterruptHandler()

```
void IOCAF5_SetInterruptHandler (
    void(* InterruptHandler ) (void))
```

Allows selecting an interrupt handler for IOCAF5 at application runtime Here is the caller graph for this function:



6.102.1.11 PIN_MANAGER_Initialize()

```
void PIN_MANAGER_Initialize (
    void )
```

@Param none @Returns none @Description GPIO and peripheral I/O initialization @Example [PIN_MANAGER_Initialize\(\)](#);
LATx registers

TRISx registers

ANSELx registers

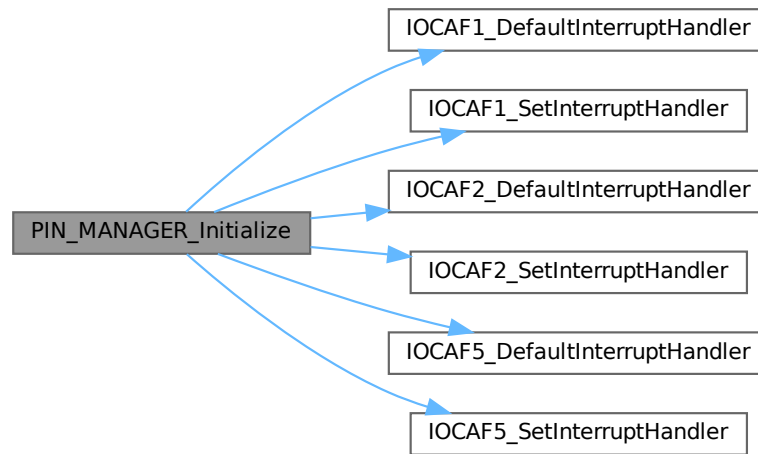
WPUx registers

ODx registers

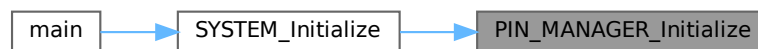
SLRCONx registers

INLVLx registers

IOCx registersHere is the call graph for this function:



Here is the caller graph for this function:



6.102.2 Variable Documentation

6.102.2.1 IOCAF1_InterruptHandler

```
void(* IOCAF1_InterruptHandler) (void) (
    void )
```

Generated Pin Manager File

Company: Microchip Technology Inc.

File Name: [pin_manager.c](#)

Summary: This is the Pin Manager file generated using PIC10 / PIC12 / PIC16 / PIC18 MCUs

Description: This header file provides implementations for pin APIs for all pins selected in the GUI. Generation Information : Product Revision : PIC10 / PIC12 / PIC16 / PIC18 MCUs - 1.81.8 Device : PIC18F47Q84 Driver Version : 2.11 The generated drivers are tested against the following: Compiler : XC8 2.36 and above MPLAB : MPLAB X 6.00

Copyright (c) 2013 - 2015 released Microchip Technology Inc. All rights reserved.

6.102.2.2 IOCAF2_InterruptHandler

```
void(* IOCAF2_InterruptHandler) (void) (
    void )
```

@Summary Dynamic Interrupt Handler for IOCAF2 pin

@Description This is a dynamic interrupt handler to be used together with the [IOCAF2_SetInterruptHandler\(\)](#) method. This handler is called every time the IOCAF2 ISR is executed and allows any function to be registered at runtime.

@Preconditions Pin Manager initializer called

@Returns None.

@Param None.

@Example [PIN_MANAGER_Initialize\(\)](#); IOCAF2_SetInterruptHandler(IOCAF2_InterruptHandler);

6.102.2.3 IOCAF5_InterruptHandler

```
void(* IOCAF5_InterruptHandler) (void) (
    void )
```

@Summary Dynamic Interrupt Handler for IOCAF5 pin

@Description This is a dynamic interrupt handler to be used together with the [IOCAF5_SetInterruptHandler\(\)](#) method. This handler is called every time the IOCAF5 ISR is executed and allows any function to be registered at runtime.

@Preconditions Pin Manager initializer called

@Returns None.

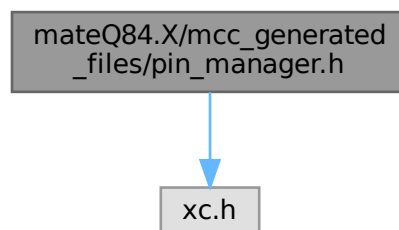
@Param None.

@Example [PIN_MANAGER_Initialize\(\)](#); IOCAF5_SetInterruptHandler(IOCAF5_InterruptHandler);

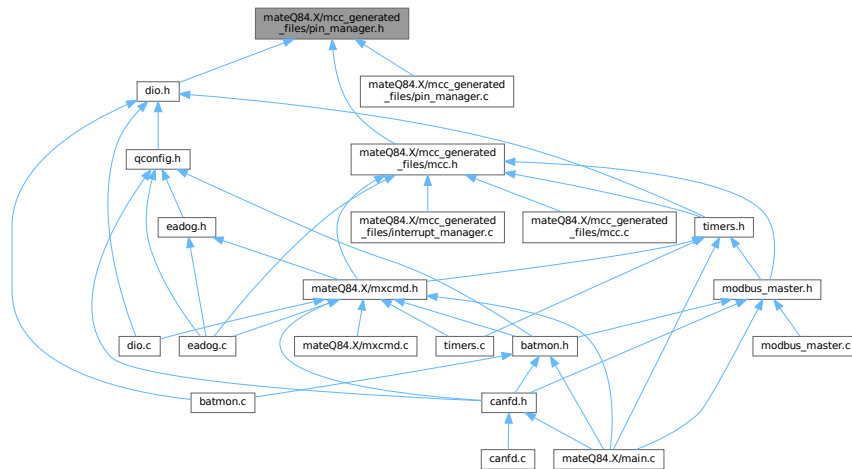
6.103 mateQ84.X/mcc_generated_files/pin_manager.h File Reference

```
#include <xc.h>
```

Include dependency graph for pin_manager.h:



This graph shows which files directly or indirectly include this file:



Macros

- #define INPUT 1
- #define OUTPUT 0
- #define HIGH 1
- #define LOW 0
- #define ANALOG 1
- #define DIGITAL 0
- #define PULL_UP_ENABLED 1
- #define PULL_UP_DISABLED 0
- #define RA0_SetHigh()
- #define RA0_SetLow()
- #define RA0_Toggle()
- #define RA0_GetValue()
- #define RA0_SetDigitalInput()
- #define RA0_SetDigitalOutput()
- #define RA0_SetPullup()
- #define RA0_ResetPullup()
- #define RA0_SetAnalogMode()
- #define RA0_SetDigitalMode()
- #define M_SWITCH_TRIS TRISAbits.TRISA1
- #define M_SWITCH_LAT LATAbits.LATA1
- #define M_SWITCH_PORT PORTAbits.RA1
- #define M_SWITCH_WPU WPUAbits.WPUA1
- #define M_SWITCH_OD ODCONAbits.ODCA1
- #define M_SWITCH_ANS ANSELAbits.ANSELA1
- #define M_SWITCH_SetHigh()
- #define M_SWITCH_SetLow()
- #define M_SWITCH_Toggle()
- #define M_SWITCH_GetValue()
- #define M_SWITCH_SetDigitalInput()
- #define M_SWITCH_SetDigitalOutput()
- #define M_SWITCH_SetPullup()
- #define M_SWITCH_ResetPullup()

- #define [M_SWITCH_SetPushPull\(\)](#)
- #define [M_SWITCH_SetOpenDrain\(\)](#)
- #define [M_SWITCH_SetAnalogMode\(\)](#)
- #define [M_SWITCH_SetDigitalMode\(\)](#)
- #define [L_SWITCH_TRIS](#) TRISAbits.TRISA2
- #define [L_SWITCH_LAT](#) LATAbits.LATA2
- #define [L_SWITCH_PORT](#) PORTAbits.RA2
- #define [L_SWITCH_WPU](#) WPUAbits.WPUA2
- #define [L_SWITCH_OD](#) ODCONAbits.ODCA2
- #define [L_SWITCH_ANS](#) ANSELAbits.ANSELA2
- #define [L_SWITCH_SetHigh\(\)](#)
- #define [L_SWITCH_SetLow\(\)](#)
- #define [L_SWITCH_Toggle\(\)](#)
- #define [L_SWITCH_GetValue\(\)](#)
- #define [L_SWITCH_SetDigitalInput\(\)](#)
- #define [L_SWITCH_SetDigitalOutput\(\)](#)
- #define [L_SWITCH_SetPullup\(\)](#)
- #define [L_SWITCH_ResetPullup\(\)](#)
- #define [L_SWITCH_SetPushPull\(\)](#)
- #define [L_SWITCH_SetOpenDrain\(\)](#)
- #define [L_SWITCH_SetAnalogMode\(\)](#)
- #define [L_SWITCH_SetDigitalMode\(\)](#)
- #define [RA3_SetHigh\(\)](#)
- #define [RA3_SetLow\(\)](#)
- #define [RA3_Toggle\(\)](#)
- #define [RA3_GetValue\(\)](#)
- #define [RA3_SetDigitalInput\(\)](#)
- #define [RA3_SetDigitalOutput\(\)](#)
- #define [RA3_SetPullup\(\)](#)
- #define [RA3_ResetPullup\(\)](#)
- #define [RA3_SetAnalogMode\(\)](#)
- #define [RA3_SetDigitalMode\(\)](#)
- #define [RA4_SetHigh\(\)](#)
- #define [RA4_SetLow\(\)](#)
- #define [RA4_Toggle\(\)](#)
- #define [RA4_GetValue\(\)](#)
- #define [RA4_SetDigitalInput\(\)](#)
- #define [RA4_SetDigitalOutput\(\)](#)
- #define [RA4_SetPullup\(\)](#)
- #define [RA4_ResetPullup\(\)](#)
- #define [RA4_SetAnalogMode\(\)](#)
- #define [RA4_SetDigitalMode\(\)](#)
- #define [A_SWITCH_TRIS](#) TRISAbits.TRISA5
- #define [A_SWITCH_LAT](#) LATAbits.LATA5
- #define [A_SWITCH_PORT](#) PORTAbits.RA5
- #define [A_SWITCH_WPU](#) WPUAbits.WPUA5
- #define [A_SWITCH_OD](#) ODCONAbits.ODCA5
- #define [A_SWITCH_ANS](#) ANSELAbits.ANSELA5
- #define [A_SWITCH_SetHigh\(\)](#)
- #define [A_SWITCH_SetLow\(\)](#)
- #define [A_SWITCH_Toggle\(\)](#)
- #define [A_SWITCH_GetValue\(\)](#)
- #define [A_SWITCH_SetDigitalInput\(\)](#)
- #define [A_SWITCH_SetDigitalOutput\(\)](#)
- #define [A_SWITCH_SetPullup\(\)](#)

- #define [A_SWITCH_ResetPullup\(\)](#)
- #define [A_SWITCH_SetPushPull\(\)](#)
- #define [A_SWITCH_SetOpenDrain\(\)](#)
- #define [A_SWITCH_SetAnalogMode\(\)](#)
- #define [A_SWITCH_SetDigitalMode\(\)](#)
- #define [RB0_SetHigh\(\)](#)
- #define [RB0_SetLow\(\)](#)
- #define [RB0_Toggle\(\)](#)
- #define [RB0_GetValue\(\)](#)
- #define [RB0_SetDigitalInput\(\)](#)
- #define [RB0_SetDigitalOutput\(\)](#)
- #define [RB0_SetPullup\(\)](#)
- #define [RB0_ResetPullup\(\)](#)
- #define [RB0_SetAnalogMode\(\)](#)
- #define [RB0_SetDigitalMode\(\)](#)
- #define [MLED_TRIS](#) TRISBbits.TRISB1
- #define [MLED_LAT](#) LATBbits.LATB1
- #define [MLED_PORT](#) PORTBbits.RB1
- #define [MLED_WPU](#) WPUBbits.WPUB1
- #define [MLED_OD](#) ODCONBbits.ODCB1
- #define [MLED_ANS](#) ANSELBbits.ANSELB1
- #define [MLED_SetHigh\(\)](#)
- #define [MLED_SetLow\(\)](#)
- #define [MLED_Toggle\(\)](#)
- #define [MLED_GetValue\(\)](#)
- #define [MLED_SetDigitalInput\(\)](#)
- #define [MLED_SetDigitalOutput\(\)](#)
- #define [MLED_SetPullup\(\)](#)
- #define [MLED_ResetPullup\(\)](#)
- #define [MLED_SetPushPull\(\)](#)
- #define [MLED_SetOpenDrain\(\)](#)
- #define [MLED_SetAnalogMode\(\)](#)
- #define [MLED_SetDigitalMode\(\)](#)
- #define [RLED_TRIS](#) TRISBbits.TRISB2
- #define [RLED_LAT](#) LATBbits.LATB2
- #define [RLED_PORT](#) PORTBbits.RB2
- #define [RLED_WPU](#) WPUBbits.WPUB2
- #define [RLED_OD](#) ODCONBbits.ODCB2
- #define [RLED_ANS](#) ANSELBbits.ANSELB2
- #define [RLED_SetHigh\(\)](#)
- #define [RLED_SetLow\(\)](#)
- #define [RLED_Toggle\(\)](#)
- #define [RLED_GetValue\(\)](#)
- #define [RLED_SetDigitalInput\(\)](#)
- #define [RLED_SetDigitalOutput\(\)](#)
- #define [RLED_SetPullup\(\)](#)
- #define [RLED_ResetPullup\(\)](#)
- #define [RLED_SetPushPull\(\)](#)
- #define [RLED_SetOpenDrain\(\)](#)
- #define [RLED_SetAnalogMode\(\)](#)
- #define [RLED_SetDigitalMode\(\)](#)
- #define [DLED1_TRIS](#) TRISBbits.TRISB3
- #define [DLED1_LAT](#) LATBbits.LATB3
- #define [DLED1_PORT](#) PORTBbits.RB3
- #define [DLED1_WPU](#) WPUBbits.WPUB3

- `#define DLED1_OD ODCONBbits.ODCB3`
- `#define DLED1_ANS ANSELBbits.ANSELB3`
- `#define DLED1_SetHigh()`
- `#define DLED1_SetLow()`
- `#define DLED1_Toggle()`
- `#define DLED1_GetValue()`
- `#define DLED1_SetDigitalInput()`
- `#define DLED1_SetDigitalOutput()`
- `#define DLED1_SetPullup()`
- `#define DLED1_ResetPullup()`
- `#define DLED1_SetPushPull()`
- `#define DLED1_SetOpenDrain()`
- `#define DLED1_SetAnalogMode()`
- `#define DLED1_SetDigitalMode()`
- `#define RB4_SetHigh()`
- `#define RB4_SetLow()`
- `#define RB4_Toggle()`
- `#define RB4_GetValue()`
- `#define RB4_SetDigitalInput()`
- `#define RB4_SetDigitalOutput()`
- `#define RB4_SetPullup()`
- `#define RB4_ResetPullup()`
- `#define RB4_SetAnalogMode()`
- `#define RB4_SetDigitalMode()`
- `#define IO_RB5_TRIS TRISBbits.TRISB5`
- `#define IO_RB5_LAT LATBbits.LATB5`
- `#define IO_RB5_PORT PORTBbits.RB5`
- `#define IO_RB5_WPU WPUBbits.WPUB5`
- `#define IO_RB5_OD ODCONBbits.ODCB5`
- `#define IO_RB5_ANS ANSELBbits.ANSELB5`
- `#define IO_RB5_SetHigh()`
- `#define IO_RB5_SetLow()`
- `#define IO_RB5_Toggle()`
- `#define IO_RB5_GetValue()`
- `#define IO_RB5_SetDigitalInput()`
- `#define IO_RB5_SetDigitalOutput()`
- `#define IO_RB5_SetPullup()`
- `#define IO_RB5_ResetPullup()`
- `#define IO_RB5_SetPushPull()`
- `#define IO_RB5_SetOpenDrain()`
- `#define IO_RB5_SetAnalogMode()`
- `#define IO_RB5_SetDigitalMode()`
- `#define INT_TRACE_TRIS TRISBbits.TRISB6`
- `#define INT_TRACE_LAT LATBbits.LATB6`
- `#define INT_TRACE_PORT PORTBbits.RB6`
- `#define INT_TRACE_WPU WPUBbits.WPUB6`
- `#define INT_TRACE_OD ODCONBbits.ODCB6`
- `#define INT_TRACE_ANS ANSELBbits.ANSELB6`
- `#define INT_TRACE_SetHigh()`
- `#define INT_TRACE_SetLow()`
- `#define INT_TRACE_Toggle()`
- `#define INT_TRACE_GetValue()`
- `#define INT_TRACE_SetDigitalInput()`
- `#define INT_TRACE_SetDigitalOutput()`
- `#define INT_TRACE_SetPullup()`

- #define INT_TRACE_ResetPullup()
- #define INT_TRACE_SetPushPull()
- #define INT_TRACE_SetOpenDrain()
- #define INT_TRACE_SetAnalogMode()
- #define INT_TRACE_SetDigitalMode()
- #define RB7_SetHigh()
- #define RB7_SetLow()
- #define RB7_Toggle()
- #define RB7_GetValue()
- #define RB7_SetDigitalInput()
- #define RB7_SetDigitalOutput()
- #define RB7_SetPullup()
- #define RB7_ResetPullup()
- #define RB7_SetAnalogMode()
- #define RB7_SetDigitalMode()
- #define RC0_SetHigh()
- #define RC0_SetLow()
- #define RC0_Toggle()
- #define RC0_GetValue()
- #define RC0_SetDigitalInput()
- #define RC0_SetDigitalOutput()
- #define RC0_SetPullup()
- #define RC0_ResetPullup()
- #define RC0_SetAnalogMode()
- #define RC0_SetDigitalMode()
- #define RC1_SetHigh()
- #define RC1_SetLow()
- #define RC1_Toggle()
- #define RC1_GetValue()
- #define RC1_SetDigitalInput()
- #define RC1_SetDigitalOutput()
- #define RC1_SetPullup()
- #define RC1_ResetPullup()
- #define RC1_SetAnalogMode()
- #define RC1_SetDigitalMode()
- #define DERE_TRIS TRISCbits.TRISC2
- #define DERE_LAT LATCbits.LATC2
- #define DERE_PORT PORTCbits.RC2
- #define DERE_WPU WPUCbits.WPUC2
- #define DERE_OD ODCONCbits.ODCC2
- #define DERE_ANS ANSELCbits.ANSELC2
- #define DERE_SetHigh()
- #define DERE_SetLow()
- #define DERE_Toggle()
- #define DERE_GetValue()
- #define DERE_SetDigitalInput()
- #define DERE_SetDigitalOutput()
- #define DERE_SetPullup()
- #define DERE_ResetPullup()
- #define DERE_SetPushPull()
- #define DERE_SetOpenDrain()
- #define DERE_SetAnalogMode()
- #define DERE_SetDigitalMode()
- #define RC3_SetHigh()
- #define RC3_SetLow()

- `#define RC3_Toggle()`
- `#define RC3_GetValue()`
- `#define RC3_SetDigitalInput()`
- `#define RC3_SetDigitalOutput()`
- `#define RC3_SetPullup()`
- `#define RC3_ResetPullup()`
- `#define RC3_SetAnalogMode()`
- `#define RC3_SetDigitalMode()`
- `#define RC4_SetHigh()`
- `#define RC4_SetLow()`
- `#define RC4_Toggle()`
- `#define RC4_GetValue()`
- `#define RC4_SetDigitalInput()`
- `#define RC4_SetDigitalOutput()`
- `#define RC4_SetPullup()`
- `#define RC4_ResetPullup()`
- `#define RC4_SetAnalogMode()`
- `#define RC4_SetDigitalMode()`
- `#define RC5_SetHigh()`
- `#define RC5_SetLow()`
- `#define RC5_Toggle()`
- `#define RC5_GetValue()`
- `#define RC5_SetDigitalInput()`
- `#define RC5_SetDigitalOutput()`
- `#define RC5_SetPullup()`
- `#define RC5_ResetPullup()`
- `#define RC5_SetAnalogMode()`
- `#define RC5_SetDigitalMode()`
- `#define RC6_SetHigh()`
- `#define RC6_SetLow()`
- `#define RC6_Toggle()`
- `#define RC6_GetValue()`
- `#define RC6_SetDigitalInput()`
- `#define RC6_SetDigitalOutput()`
- `#define RC6_SetPullup()`
- `#define RC6_ResetPullup()`
- `#define RC6_SetAnalogMode()`
- `#define RC6_SetDigitalMode()`
- `#define RC7_SetHigh()`
- `#define RC7_SetLow()`
- `#define RC7_Toggle()`
- `#define RC7_GetValue()`
- `#define RC7_SetDigitalInput()`
- `#define RC7_SetDigitalOutput()`
- `#define RC7_SetPullup()`
- `#define RC7_ResetPullup()`
- `#define RC7_SetAnalogMode()`
- `#define RC7_SetDigitalMode()`
- `#define RD0_SetHigh()`
- `#define RD0_SetLow()`
- `#define RD0_Toggle()`
- `#define RD0_GetValue()`
- `#define RD0_SetDigitalInput()`
- `#define RD0_SetDigitalOutput()`
- `#define RD0_SetPullup()`

- #define [RD0_ResetPullup\(\)](#)
- #define [RD0_SetAnalogMode\(\)](#)
- #define [RD0_SetDigitalMode\(\)](#)
- #define [DB0_TRIS](#) TRISDbits.TRISD1
- #define [DB0_LAT](#) LATDbits.LATD1
- #define [DB0_PORT](#) PORTDbits.RD1
- #define [DB0_WPU](#) WPUDbits.WPUD1
- #define [DB0_OD](#) ODCONDbits.ODCD1
- #define [DB0_ANS](#) ANSELDbits.ANSELD1
- #define [DB0_SetHigh\(\)](#)
- #define [DB0_SetLow\(\)](#)
- #define [DB0_Toggle\(\)](#)
- #define [DB0_GetValue\(\)](#)
- #define [DB0_SetDigitalInput\(\)](#)
- #define [DB0_SetDigitalOutput\(\)](#)
- #define [DB0_SetPullup\(\)](#)
- #define [DB0_ResetPullup\(\)](#)
- #define [DB0_SetPushPull\(\)](#)
- #define [DB0_SetOpenDrain\(\)](#)
- #define [DB0_SetAnalogMode\(\)](#)
- #define [DB0_SetDigitalMode\(\)](#)
- #define [DB1_TRIS](#) TRISDbits.TRISD2
- #define [DB1_LAT](#) LATDbits.LATD2
- #define [DB1_PORT](#) PORTDbits.RD2
- #define [DB1_WPU](#) WPUDbits.WPUD2
- #define [DB1_OD](#) ODCONDbits.ODCD2
- #define [DB1_ANS](#) ANSELDbits.ANSELD2
- #define [DB1_SetHigh\(\)](#)
- #define [DB1_SetLow\(\)](#)
- #define [DB1_Toggle\(\)](#)
- #define [DB1_GetValue\(\)](#)
- #define [DB1_SetDigitalInput\(\)](#)
- #define [DB1_SetDigitalOutput\(\)](#)
- #define [DB1_SetPullup\(\)](#)
- #define [DB1_ResetPullup\(\)](#)
- #define [DB1_SetPushPull\(\)](#)
- #define [DB1_SetOpenDrain\(\)](#)
- #define [DB1_SetAnalogMode\(\)](#)
- #define [DB1_SetDigitalMode\(\)](#)
- #define [CS_TRIS](#) TRISDbits.TRISD3
- #define [CS_LAT](#) LATDbits.LATD3
- #define [CS_PORT](#) PORTDbits.RD3
- #define [CS_WPU](#) WPUDbits.WPUD3
- #define [CS_OD](#) ODCONDbits.ODCD3
- #define [CS_ANS](#) ANSELDbits.ANSELD3
- #define [CS_SetHigh\(\)](#)
- #define [CS_SetLow\(\)](#)
- #define [CS_Toggle\(\)](#)
- #define [CS_GetValue\(\)](#)
- #define [CS_SetDigitalInput\(\)](#)
- #define [CS_SetDigitalOutput\(\)](#)
- #define [CS_SetPullup\(\)](#)
- #define [CS_ResetPullup\(\)](#)
- #define [CS_SetPushPull\(\)](#)
- #define [CS_SetOpenDrain\(\)](#)

- `#define CS_SetAnalogMode()`
- `#define CS_SetDigitalMode()`
- `#define DLED_TRIS TRISDbits.TRISD4`
- `#define DLED_LAT LATDbits.LATD4`
- `#define DLED_PORT PORTDbits.RD4`
- `#define DLED_WPU WPUDbits.WPUD4`
- `#define DLED_OD ODCONDbits.ODCD4`
- `#define DLED_ANS ANSELDbits.ANSELD4`
- `#define DLED_SetHigh()`
- `#define DLED_SetLow()`
- `#define DLED_Toggle()`
- `#define DLED_GetValue()`
- `#define DLED_SetDigitalInput()`
- `#define DLED_SetDigitalOutput()`
- `#define DLED_SetPullup()`
- `#define DLED_ResetPullup()`
- `#define DLED_SetPushPull()`
- `#define DLED_SetOpenDrain()`
- `#define DLED_SetAnalogMode()`
- `#define DLED_SetDigitalMode()`
- `#define PROG_TRACE_TRIS TRISDbits.TRISD5`
- `#define PROG_TRACE_LAT LATDbits.LATD5`
- `#define PROG_TRACE_PORT PORTDbits.RD5`
- `#define PROG_TRACE_WPU WPUDbits.WPUD5`
- `#define PROG_TRACE_OD ODCONDbits.ODCD5`
- `#define PROG_TRACE_ANS ANSELDbits.ANSELD5`
- `#define PROG_TRACE_SetHigh()`
- `#define PROG_TRACE_SetLow()`
- `#define PROG_TRACE_Toggle()`
- `#define PROG_TRACE_GetValue()`
- `#define PROG_TRACE_SetDigitalInput()`
- `#define PROG_TRACE_SetDigitalOutput()`
- `#define PROG_TRACE_SetPullup()`
- `#define PROG_TRACE_ResetPullup()`
- `#define PROG_TRACE_SetPushPull()`
- `#define PROG_TRACE_SetOpenDrain()`
- `#define PROG_TRACE_SetAnalogMode()`
- `#define PROG_TRACE_SetDigitalMode()`
- `#define RD6_SetHigh()`
- `#define RD6_SetLow()`
- `#define RD6_Toggle()`
- `#define RD6_GetValue()`
- `#define RD6_SetDigitalInput()`
- `#define RD6_SetDigitalOutput()`
- `#define RD6_SetPullup()`
- `#define RD6_ResetPullup()`
- `#define RD6_SetAnalogMode()`
- `#define RD6_SetDigitalMode()`
- `#define IO_RD7_TRIS TRISDbits.TRISD7`
- `#define IO_RD7_LAT LATDbits.LATD7`
- `#define IO_RD7_PORT PORTDbits.RD7`
- `#define IO_RD7_WPU WPUDbits.WPUD7`
- `#define IO_RD7_OD ODCONDbits.ODCD7`
- `#define IO_RD7_ANS ANSELDbits.ANSELD7`
- `#define IO_RD7_SetHigh()`

- #define [IO_RD7_SetLow\(\)](#)
- #define [IO_RD7_Toggle\(\)](#)
- #define [IO_RD7_GetValue\(\)](#)
- #define [IO_RD7_SetDigitalInput\(\)](#)
- #define [IO_RD7_SetDigitalOutput\(\)](#)
- #define [IO_RD7_SetPullup\(\)](#)
- #define [IO_RD7_ResetPullup\(\)](#)
- #define [IO_RD7_SetPushPull\(\)](#)
- #define [IO_RD7_SetOpenDrain\(\)](#)
- #define [IO_RD7_SetAnalogMode\(\)](#)
- #define [IO_RD7_SetDigitalMode\(\)](#)
- #define [RELAY_TRIS](#) TRISEbits.TRISE0
- #define [RELAY_LAT](#) LATEbits.LATE0
- #define [RELAY_PORT](#) PORTEbits.RE0
- #define [RELAY_WPU](#) WPUEbits.WPUE0
- #define [RELAY_OD](#) ODCONEbits.ODCE0
- #define [RELAY_ANS](#) ANSELEbits.ANSELE0
- #define [RELAY_SetHigh\(\)](#)
- #define [RELAY_SetLow\(\)](#)
- #define [RELAY_Toggle\(\)](#)
- #define [RELAY_GetValue\(\)](#)
- #define [RELAY_SetDigitalInput\(\)](#)
- #define [RELAY_SetDigitalOutput\(\)](#)
- #define [RELAY_SetPullup\(\)](#)
- #define [RELAY_ResetPullup\(\)](#)
- #define [RELAY_SetPushPull\(\)](#)
- #define [RELAY_SetOpenDrain\(\)](#)
- #define [RELAY_SetAnalogMode\(\)](#)
- #define [RELAY_SetDigitalMode\(\)](#)
- #define [MISC_TRIS](#) TRISEbits.TRISE1
- #define [MISC_LAT](#) LATEbits.LATE1
- #define [MISC_PORT](#) PORTEbits.RE1
- #define [MISC_WPU](#) WPUEbits.WPUE1
- #define [MISC_OD](#) ODCONEbits.ODCE1
- #define [MISC_ANS](#) ANSELEbits.ANSELE1
- #define [MISC_SetHigh\(\)](#)
- #define [MISC_SetLow\(\)](#)
- #define [MISC_Toggle\(\)](#)
- #define [MISC_GetValue\(\)](#)
- #define [MISC_SetDigitalInput\(\)](#)
- #define [MISC_SetDigitalOutput\(\)](#)
- #define [MISC_SetPullup\(\)](#)
- #define [MISC_ResetPullup\(\)](#)
- #define [MISC_SetPushPull\(\)](#)
- #define [MISC_SetOpenDrain\(\)](#)
- #define [MISC_SetAnalogMode\(\)](#)
- #define [MISC_SetDigitalMode\(\)](#)
- #define [CHARGER_TRIS](#) TRISEbits.TRISE2
- #define [CHARGER_LAT](#) LATEbits.LATE2
- #define [CHARGER_PORT](#) PORTEbits.RE2
- #define [CHARGER_WPU](#) WPUEbits.WPUE2
- #define [CHARGER_OD](#) ODCONEbits.ODCE2
- #define [CHARGER_ANS](#) ANSELEbits.ANSELE2
- #define [CHARGER_SetHigh\(\)](#)
- #define [CHARGER_SetLow\(\)](#)

- `#define CHARGER_Toggle()`
- `#define CHARGER_GetValue()`
- `#define CHARGER_SetDigitalInput()`
- `#define CHARGER_SetDigitalOutput()`
- `#define CHARGER_SetPullup()`
- `#define CHARGER_ResetPullup()`
- `#define CHARGER_SetPushPull()`
- `#define CHARGER_SetOpenDrain()`
- `#define CHARGER_SetAnalogMode()`
- `#define CHARGER_SetDigitalMode()`

Functions

- `void PIN_MANAGER_Initialize (void)`
- `void IOCAF1_ISR (void)`
- `void IOCAF1_SetInterruptHandler (void(*InterruptHandler)(void))`
- `void IOCAF1_DefaultInterruptHandler (void)`
- `void IOCAF2_ISR (void)`
- `void IOCAF2_SetInterruptHandler (void(*InterruptHandler)(void))`
- `void IOCAF2_DefaultInterruptHandler (void)`
- `void IOCAF5_ISR (void)`
- `void IOCAF5_SetInterruptHandler (void(*InterruptHandler)(void))`
- `void IOCAF5_DefaultInterruptHandler (void)`

Variables

- `void(* IOCAF1_InterruptHandler )(void)`
- `void(* IOCAF2_InterruptHandler )(void)`
- `void(* IOCAF5_InterruptHandler )(void)`

6.103.1 Macro Definition Documentation

6.103.1.1 A_SWITCH_ANS

```
#define A_SWITCH_ANS ANSELAbits.ANSELA5
```

6.103.1.2 A_SWITCH_GetValue

```
#define A_SWITCH_GetValue()
```

Value:

```
PORTAbits.RA5
```

6.103.1.3 A_SWITCH_LAT

```
#define A_SWITCH_LAT LATAbits.LATA5
```


6.103.1.4 A_SWITCH_OD

```
#define A_SWITCH_OD ODCONAbits.ODCA5
```

6.103.1.5 A_SWITCH_PORT

```
#define A_SWITCH_PORT PORTAbits.RA5
```

6.103.1.6 A_SWITCH_ResetPullup

```
#define A_SWITCH_ResetPullup()
```

Value:

```
do { WPUAbits.WPUA5 = 0; } while(0)
```

6.103.1.7 A_SWITCH_SetAnalogMode

```
#define A_SWITCH_SetAnalogMode()
```

Value:

```
do { ANSELAbits.ANSELA5 = 1; } while(0)
```

6.103.1.8 A_SWITCH_SetDigitalInput

```
#define A_SWITCH_SetDigitalInput()
```

Value:

```
do { TRISAbits.TRISA5 = 1; } while(0)
```

6.103.1.9 A_SWITCH_SetDigitalMode

```
#define A_SWITCH_SetDigitalMode()
```

Value:

```
do { ANSELAbits.ANSELA5 = 0; } while(0)
```

6.103.1.10 A_SWITCH_SetDigitalOutput

```
#define A_SWITCH_SetDigitalOutput()
```

Value:

```
do { TRISAbits.TRISA5 = 0; } while(0)
```

6.103.1.11 A_SWITCH_SetHigh

```
#define A_SWITCH_SetHigh()
```

Value:

```
do { LATAbits.LATA5 = 1; } while(0)
```

6.103.1.12 A_SWITCH_SetLow

```
#define A_SWITCH_SetLow()
```

Value:

```
do { LATAbits.LATA5 = 0; } while(0)
```

6.103.1.13 A_SWITCH_SetOpenDrain

```
#define A_SWITCH_SetOpenDrain()
```

Value:

```
do { ODCONAbits.ODCA5 = 1; } while(0)
```

6.103.1.14 A_SWITCH_SetPullup

```
#define A_SWITCH_SetPullup()
```

Value:

```
do { WPUAbits.WPUA5 = 1; } while(0)
```

6.103.1.15 A_SWITCH_SetPushPull

```
#define A_SWITCH_SetPushPull()
```

Value:

```
do { ODCONAbits.ODCA5 = 0; } while(0)
```

6.103.1.16 A_SWITCH_Toggle

```
#define A_SWITCH_Toggle()
```

Value:

```
do { LATAbits.LATA5 = ~LATAbits.LATA5; } while(0)
```

6.103.1.17 A_SWITCH_TRIS

```
#define A_SWITCH_TRIS TRISAbits.TRISA5
```

6.103.1.18 A_SWITCH_WPU

```
#define A_SWITCH_WPU WPUAbits.WPUA5
```

6.103.1.19 ANALOG

```
#define ANALOG 1
```

6.103.1.20 CHARGER_ANS

```
#define CHARGER_ANS ANSELEbits.ANSELE2
```

6.103.1.21 CHARGER_GetValue

```
#define CHARGER_GetValue()
```

Value:

```
PORTEbits.RE2
```

6.103.1.22 CHARGER_LAT

```
#define CHARGER_LAT LATEbits.LATE2
```

6.103.1.23 CHARGER_OD

```
#define CHARGER_OD ODCONEbits.ODCE2
```

6.103.1.24 CHARGER_PORT

```
#define CHARGER_PORT PORTEbits.RE2
```

6.103.1.25 CHARGER_ResetPullup

```
#define CHARGER_ResetPullup()
```

Value:

```
do { WPUEbits.WPUE2 = 0; } while(0)
```

6.103.1.26 CHARGER_SetAnalogMode

```
#define CHARGER_SetAnalogMode()
```

Value:

```
do { ANSELEbits.ANSELE2 = 1; } while(0)
```

6.103.1.27 CHARGER_SetDigitalInput

```
#define CHARGER_SetDigitalInput()
```

Value:

```
do { TRISEbits.TRISE2 = 1; } while(0)
```

6.103.1.28 CHARGER_SetDigitalMode

```
#define CHARGER_SetDigitalMode()
```

Value:

```
do { ANSELEbits.ANSELE2 = 0; } while(0)
```

6.103.1.29 CHARGER_SetDigitalOutput

```
#define CHARGER_SetDigitalOutput()
```

Value:

```
do { TRISEbits.TRISE2 = 0; } while(0)
```

6.103.1.30 CHARGER_SetHigh

```
#define CHARGER_SetHigh()
```

Value:

```
do { LATEbits.LATE2 = 1; } while(0)
```

6.103.1.31 CHARGER_SetLow

```
#define CHARGER_SetLow()
```

Value:

```
do { LATEbits.LATE2 = 0; } while(0)
```

6.103.1.32 CHARGER_SetOpenDrain

```
#define CHARGER_SetOpenDrain()
```

Value:

```
do { ODCONEbits.ODCE2 = 1; } while(0)
```

6.103.1.33 CHARGER_SetPullup

```
#define CHARGER_SetPullup()
```

Value:

```
do { WPUEbits.WPUE2 = 1; } while(0)
```

6.103.1.34 CHARGER_SetPushPull

```
#define CHARGER_SetPushPull()
```

Value:

```
do { ODCONEbits.ODCE2 = 0; } while(0)
```

6.103.1.35 CHARGER_Toggle

```
#define CHARGER_Toggle()
```

Value:

```
do { LATEbits.LATE2 = ~LATEbits.LATE2; } while(0)
```

6.103.1.36 CHARGER_TRIS

```
#define CHARGER_TRIS TRISEbits.TRISE2
```

6.103.1.37 CHARGER_WPU

```
#define CHARGER_WPU WPUEbits.WPUE2
```

6.103.1.38 CS_ANS

```
#define CS_ANS ANSELDbits.ANSELD3
```

6.103.1.39 CS_GetValue

```
#define CS_GetValue()
```

Value:

```
PORTDbits.RD3
```

6.103.1.40 CS_LAT

```
#define CS_LAT LATDbits.LATD3
```

6.103.1.41 CS_OD

```
#define CS_OD ODCONDbits.ODCD3
```

6.103.1.42 CS_PORT

```
#define CS_PORT PORTDbits.RD3
```

6.103.1.43 CS_ResetPullup

```
#define CS_ResetPullup()
```

Value:

```
do { WPUDbits.WPUD3 = 0; } while(0)
```

6.103.1.44 CS_SetAnalogMode

```
#define CS_SetAnalogMode()
```

Value:

```
do { ANSELDbits.ANSELD3 = 1; } while(0)
```

6.103.1.45 CS_SetDigitalInput

```
#define CS_SetDigitalInput()
```

Value:

```
do { TRISDbits.TRISD3 = 1; } while(0)
```

6.103.1.46 CS_SetDigitalMode

```
#define CS_SetDigitalMode()
```

Value:

```
do { ANSELDbits.ANSELD3 = 0; } while(0)
```

6.103.1.47 CS_SetDigitalOutput

```
#define CS_SetDigitalOutput()
```

Value:

```
do { TRISDbits.TRISD3 = 0; } while(0)
```

6.103.1.48 CS_SetHigh

```
#define CS_SetHigh()
```

Value:

```
do { LATDbits.LATD3 = 1; } while(0)
```

6.103.1.49 CS_SetLow

```
#define CS_SetLow()
```

Value:

```
do { LATDbits.LATD3 = 0; } while(0)
```

6.103.1.50 CS_SetOpenDrain

```
#define CS_SetOpenDrain()
```

Value:

```
do { ODCONDbits.ODCD3 = 1; } while(0)
```

6.103.1.51 CS_SetPullup

```
#define CS_SetPullup()
```

Value:

```
do { WPUDbits.WPUD3 = 1; } while(0)
```

6.103.1.52 CS_SetPushPull

```
#define CS_SetPushPull()
```

Value:

```
do { ODCONDbits.ODCD3 = 0; } while(0)
```

6.103.1.53 CS_Toggle

```
#define CS_Toggle()
```

Value:

```
do { LATDbits.LATD3 = ~LATDbits.LATD3; } while(0)
```

6.103.1.54 CS_TRIS

```
#define CS_TRIS TRISDbits.TRISD3
```

6.103.1.55 CS_WPU

```
#define CS_WPU WPUDbits.WPUD3
```

6.103.1.56 DB0_ANS

```
#define DB0_ANS ANSELDbits.ANSELD1
```

6.103.1.57 DB0_GetValue

```
#define DB0_GetValue()
```

Value:

```
PORTDbits.RD1
```

6.103.1.58 DB0_LAT

```
#define DB0_LAT LATDbits.LATD1
```

6.103.1.59 DB0_OD

```
#define DB0_OD ODCONDbits.ODCD1
```

6.103.1.60 DB0_PORT

```
#define DB0_PORT PORTDbits.RD1
```

6.103.1.61 DB0_ResetPullup

```
#define DB0_ResetPullup()
```

Value:

```
do { WPUDbits.WPUD1 = 0; } while(0)
```

6.103.1.62 DB0_SetAnalogMode

```
#define DB0_SetAnalogMode()
```

Value:

```
do { ANSELDbits.ANSELD1 = 1; } while(0)
```

6.103.1.63 DB0_SetDigitalInput

```
#define DB0_SetDigitalInput()
```

Value:

```
do { TRISDbits.TRISD1 = 1; } while(0)
```

6.103.1.64 DB0_SetDigitalMode

```
#define DB0_SetDigitalMode()
```

Value:

```
do { ANSELDbits.ANSELD1 = 0; } while(0)
```

6.103.1.65 DB0_SetDigitalOutput

```
#define DB0_SetDigitalOutput()
```

Value:

```
do { TRISDbits.TRISD1 = 0; } while(0)
```


6.103.1.66 DB0_SetHigh

```
#define DB0_SetHigh()
```

Value:

```
do { LATDbits.LATD1 = 1; } while(0)
```

6.103.1.67 DB0_SetLow

```
#define DB0_SetLow()
```

Value:

```
do { LATDbits.LATD1 = 0; } while(0)
```

6.103.1.68 DB0_SetOpenDrain

```
#define DB0_SetOpenDrain()
```

Value:

```
do { ODCONDbits.ODCD1 = 1; } while(0)
```

6.103.1.69 DB0_SetPullup

```
#define DB0_SetPullup()
```

Value:

```
do { WPUDbits.WPUD1 = 1; } while(0)
```

6.103.1.70 DB0_SetPushPull

```
#define DB0_SetPushPull()
```

Value:

```
do { ODCONDbits.ODCD1 = 0; } while(0)
```

6.103.1.71 DB0_Toggle

```
#define DB0_Toggle()
```

Value:

```
do { LATDbits.LATD1 = ~LATDbits.LATD1; } while(0)
```

6.103.1.72 DB0_TRIS

```
#define DB0_TRIS TRISDbits.TRISD1
```

6.103.1.73 DB0_WPU

```
#define DB0_WPU WPUDbits.WPUD1
```

6.103.1.74 DB1_ANS

```
#define DB1_ANS ANSELDbits.ANSELD2
```

6.103.1.75 DB1_GetValue

```
#define DB1_GetValue()
```

Value:

```
PORTDbits.RD2
```

6.103.1.76 DB1_LAT

```
#define DB1_LAT LATDbits.LATD2
```

6.103.1.77 DB1_OD

```
#define DB1_OD ODCONDbits.ODCD2
```

6.103.1.78 DB1_PORT

```
#define DB1_PORT PORTDbits.RD2
```

6.103.1.79 DB1_ResetPullup

```
#define DB1_ResetPullup()
```

Value:

```
do { WPUDbits.WPUD2 = 0; } while(0)
```

6.103.1.80 DB1_SetAnalogMode

```
#define DB1_SetAnalogMode()
```

Value:

```
do { ANSELDbits.ANSELD2 = 1; } while(0)
```

6.103.1.81 DB1_SetDigitalInput

```
#define DB1_SetDigitalInput()
```

Value:

```
do { TRISDbits.TRISD2 = 1; } while(0)
```

6.103.1.82 DB1_SetDigitalMode

```
#define DB1_SetDigitalMode()
```

Value:

```
do { ANSELDbits.ANSELD2 = 0; } while(0)
```

6.103.1.83 DB1_SetDigitalOutput

```
#define DB1_SetDigitalOutput()
```

Value:

```
do { TRISDbits.TRISD2 = 0; } while(0)
```

6.103.1.84 DB1_SetHigh

```
#define DB1_SetHigh()
```

Value:

```
do { LATDbits.LATD2 = 1; } while(0)
```

6.103.1.85 DB1_SetLow

```
#define DB1_SetLow()
```

Value:

```
do { LATDbits.LATD2 = 0; } while(0)
```

6.103.1.86 DB1_SetOpenDrain

```
#define DB1_SetOpenDrain()
```

Value:

```
do { ODCONDbits.ODCD2 = 1; } while(0)
```

6.103.1.87 DB1_SetPullup

```
#define DB1_SetPullup()
```

Value:

```
do { WPUDbits.WPUD2 = 1; } while(0)
```

6.103.1.88 DB1_SetPushPull

```
#define DB1_SetPushPull()
```

Value:

```
do { ODCONDbits.ODCD2 = 0; } while(0)
```

6.103.1.89 DB1_Toggle

```
#define DB1_Toggle()
```

Value:

```
do { LATDbits.LATD2 = ~LATDbits.LATD2; } while(0)
```

6.103.1.90 DB1_TRIS

```
#define DB1_TRIS TRISDbits.TRISD2
```

6.103.1.91 DB1_WPU

```
#define DB1_WPU WPUdbits.WPUD2
```

6.103.1.92 DERE_ANS

```
#define DERE_ANS ANSELbits.ANSEL2
```

6.103.1.93 DERE_GetValue

```
#define DERE_GetValue()
```

Value:

```
PORTbits.RC2
```

6.103.1.94 DERE_LAT

```
#define DERE_LAT LATbits.LATC2
```

6.103.1.95 DERE_OD

```
#define DERE_OD ODCONbits.ODCC2
```

6.103.1.96 DERE_PORT

```
#define DERE_PORT PORTbits.RC2
```

6.103.1.97 DERE_ResetPullup

```
#define DERE_ResetPullup()
```

Value:

```
do { WPUCbits.WPUC2 = 0; } while(0)
```

6.103.1.98 DERE_SetAnalogMode

```
#define DERE_SetAnalogMode()
```

Value:

```
do { ANSELbits.ANSEL2 = 1; } while(0)
```

6.103.1.99 DERE_SetDigitalInput

```
#define DERE_SetDigitalInput()
```

Value:

```
do { TRISCbits.TRISC2 = 1; } while(0)
```

6.103.1.100 DERE_SetDigitalMode

```
#define DERE_SetDigitalMode()
```

Value:

```
do { ANSELbits.ANSEL2 = 0; } while(0)
```

6.103.1.101 DERE_SetDigitalOutput

```
#define DERE_SetDigitalOutput()
```

Value:

```
do { TRISCbits.TRISC2 = 0; } while(0)
```

6.103.1.102 DERE_SetHigh

```
#define DERE_SetHigh()
```

Value:

```
do { LATCbits.LATC2 = 1; } while(0)
```

6.103.1.103 DERE_SetLow

```
#define DERE_SetLow()
```

Value:

```
do { LATCbits.LATC2 = 0; } while(0)
```

6.103.1.104 DERE_SetOpenDrain

```
#define DERE_SetOpenDrain()
```

Value:

```
do { ODCONCbits.ODCC2 = 1; } while(0)
```

6.103.1.105 DERE_SetPullup

```
#define DERE_SetPullup()
```

Value:

```
do { WPUCbits.WPUC2 = 1; } while(0)
```

6.103.1.106 DERE_SetPushPull

```
#define DERE_SetPushPull()
```

Value:

```
do { ODCONCbits.ODCC2 = 0; } while(0)
```

6.103.1.107 DERE_Toggle

```
#define DERE_Toggle()
```

Value:

```
do { LATCbits.LATC2 = ~LATCbits.LATC2; } while(0)
```

6.103.1.108 DERE_TRIS

```
#define DERE_TRIS TRISCbits.TRISC2
```

6.103.1.109 DERE_WPU

```
#define DERE_WPU WPUCbits.WPUC2
```

6.103.1.110 DIGITAL

```
#define DIGITAL 0
```

6.103.1.111 DLED1_ANS

```
#define DLED1_ANS ANSELBbits.ANSELB3
```

6.103.1.112 DLED1_GetValue

```
#define DLED1_GetValue()
```

Value:

```
PORTBbits.RB3
```

6.103.1.113 DLED1_LAT

```
#define DLED1_LAT LATBbits.LATB3
```

6.103.1.114 DLED1_OD

```
#define DLED1_OD ODCONBbits.ODCB3
```

6.103.1.115 DLED1_PORT

```
#define DLED1_PORT PORTBbits.RB3
```

6.103.1.116 DLED1_ResetPullup

```
#define DLED1_ResetPullup()
```

Value:

```
do { WPUBbits.WPUB3 = 0; } while(0)
```

6.103.1.117 DLED1_SetAnalogMode

```
#define DLED1_SetAnalogMode()
```

Value:

```
do { ANSELBbits.ANSELB3 = 1; } while(0)
```

6.103.1.118 DLED1_SetDigitalInput

```
#define DLED1_SetDigitalInput()
```

Value:

```
do { TRISBbits.TRISB3 = 1; } while(0)
```

6.103.1.119 DLED1_SetDigitalMode

```
#define DLED1_SetDigitalMode()
```

Value:

```
do { ANSELBbits.ANSELB3 = 0; } while(0)
```

6.103.1.120 DLED1_SetDigitalOutput

```
#define DLED1_SetDigitalOutput()
```

Value:

```
do { TRISBbits.TRISB3 = 0; } while(0)
```

6.103.1.121 DLED1_SetHigh

```
#define DLED1_SetHigh()
```

Value:

```
do { LATBbits.LATB3 = 1; } while(0)
```

6.103.1.122 DLED1_SetLow

```
#define DLED1_SetLow()
```

Value:

```
do { LATBbits.LATB3 = 0; } while(0)
```

6.103.1.123 DLED1_SetOpenDrain

```
#define DLED1_SetOpenDrain()
```

Value:

```
do { ODCONBbits.ODCB3 = 1; } while(0)
```

6.103.1.124 DLED1_SetPullup

```
#define DLED1_SetPullup()
```

Value:

```
do { WPUBbits.WPUB3 = 1; } while(0)
```

6.103.1.125 DLED1_SetPushPull

```
#define DLED1_SetPushPull()
```

Value:

```
do { ODCONBbits.ODCB3 = 0; } while(0)
```

6.103.1.126 DLED1_Toggle

```
#define DLED1_Toggle()
```

Value:

```
do { LATBbits.LATB3 = ~LATBbits.LATB3; } while(0)
```


6.103.1.127 DLED1_TRIS

```
#define DLED1_TRIS TRISBbits.TRISB3
```

6.103.1.128 DLED1_WPU

```
#define DLED1_WPU WPUBbits.WPUB3
```

6.103.1.129 DLED_ANS

```
#define DLED_ANS ANSELDbits.ANSELD4
```

6.103.1.130 DLED_GetValue

```
#define DLED_GetValue()
```

Value:

```
PORTDbits.RD4
```

6.103.1.131 DLED_LAT

```
#define DLED_LAT LATDbits.LATD4
```

6.103.1.132 DLED_OD

```
#define DLED_OD ODCONDbits.ODCD4
```

6.103.1.133 DLED_PORT

```
#define DLED_PORT PORTDbits.RD4
```

6.103.1.134 DLED_ResetPullup

```
#define DLED_ResetPullup()
```

Value:

```
do { WPUDbits.WPUD4 = 0; } while(0)
```

6.103.1.135 DLED_SetAnalogMode

```
#define DLED_SetAnalogMode()
```

Value:

```
do { ANSELDbits.ANSELD4 = 1; } while(0)
```

6.103.1.136 DLED_SetDigitalInput

```
#define DLED_SetDigitalInput()
```

Value:

```
do { TRISDbits.TRISD4 = 1; } while(0)
```

6.103.1.137 DLED_SetDigitalMode

```
#define DLED_SetDigitalMode()
```

Value:

```
do { ANSELDbits.ANSEL4 = 0; } while(0)
```

6.103.1.138 DLED_SetDigitalOutput

```
#define DLED_SetDigitalOutput()
```

Value:

```
do { TRISDbits.TRISD4 = 0; } while(0)
```

6.103.1.139 DLED_SetHigh

```
#define DLED_SetHigh()
```

Value:

```
do { LATDbits.LATD4 = 1; } while(0)
```

6.103.1.140 DLED_SetLow

```
#define DLED_SetLow()
```

Value:

```
do { LATDbits.LATD4 = 0; } while(0)
```

6.103.1.141 DLED_SetOpenDrain

```
#define DLED_SetOpenDrain()
```

Value:

```
do { ODCONDbits.ODCD4 = 1; } while(0)
```

6.103.1.142 DLED_SetPullup

```
#define DLED_SetPullup()
```

Value:

```
do { WPUDbits.WPUD4 = 1; } while(0)
```

6.103.1.143 DLED_SetPushPull

```
#define DLED_SetPushPull()
```

Value:

```
do { ODCONDbits.ODCD4 = 0; } while(0)
```

6.103.1.144 DLED_Toggle

```
#define DLED_Toggle()
```

Value:

```
do { LATDbits.LATD4 = ~LATDbits.LATD4; } while(0)
```

6.103.1.145 DLED_TRIS

```
#define DLED_TRIS TRISDbits.TRISD4
```

6.103.1.146 DLED_WPU

```
#define DLED_WPU WPUDbits.WPUD4
```

6.103.1.147 HIGH

```
#define HIGH 1
```

6.103.1.148 INPUT

```
#define INPUT 1
```

@Generated Pin Manager Header File

@Company: Microchip Technology Inc.

@File Name: [pin_manager.h](#)

@Summary: This is the Pin Manager file generated using PIC10 / PIC12 / PIC16 / PIC18 MCUs

@Description This header file provides APIs for driver for . Generation Information : Product Revision : PIC10 / PIC12 / PIC16 / PIC18 MCUs - 1.81.8 Device : PIC18F47Q84 Driver Version : 2.11 The generated drivers are tested against the following: Compiler : XC8 2.36 and above MPLAB : MPLAB X 6.00 Section: Included Files

6.103.1.149 INT_TRACE_ANS

```
#define INT_TRACE_ANS ANSELBbits.ANSELB6
```

6.103.1.150 INT_TRACE_GetValue

```
#define INT_TRACE_GetValue()
```

Value:

```
PORTBbits.RB6
```

6.103.1.151 INT_TRACE_LAT

```
#define INT_TRACE_LAT LATBbits.LATB6
```

6.103.1.152 INT_TRACE_OD

```
#define INT_TRACE_OD ODCONBbits.ODCB6
```

6.103.1.153 INT_TRACE_PORT

```
#define INT_TRACE_PORT PORTBbits.RB6
```

6.103.1.154 INT_TRACE_ResetPullup

```
#define INT_TRACE_ResetPullup()
```

Value:

```
do { WPUBbits.WPUB6 = 0; } while(0)
```

6.103.1.155 INT_TRACE_SetAnalogMode

```
#define INT_TRACE_SetAnalogMode()
```

Value:

```
do { ANSELBbits.ANSELB6 = 1; } while(0)
```

6.103.1.156 INT_TRACE_SetDigitalInput

```
#define INT_TRACE_SetDigitalInput()
```

Value:

```
do { TRISBbits.TRISB6 = 1; } while(0)
```

6.103.1.157 INT_TRACE_SetDigitalMode

```
#define INT_TRACE_SetDigitalMode()
```

Value:

```
do { ANSELBbits.ANSELB6 = 0; } while(0)
```

6.103.1.158 INT_TRACE_SetDigitalOutput

```
#define INT_TRACE_SetDigitalOutput()
```

Value:

```
do { TRISBbits.TRISB6 = 0; } while(0)
```

6.103.1.159 INT_TRACE_SetHigh

```
#define INT_TRACE_SetHigh()
```

Value:

```
do { LATBbits.LATB6 = 1; } while(0)
```

6.103.1.160 INT_TRACE_SetLow

```
#define INT_TRACE_SetLow()
```

Value:

```
do { LATBbits.LATB6 = 0; } while(0)
```

6.103.1.161 INT_TRACE_SetOpenDrain

```
#define INT_TRACE_SetOpenDrain()
```

Value:

```
do { ODCONBbits.ODCB6 = 1; } while(0)
```

6.103.1.162 INT_TRACE_SetPullup

```
#define INT_TRACE_SetPullup()
```

Value:

```
do { WPUBbits.WPUB6 = 1; } while(0)
```

6.103.1.163 INT_TRACE_SetPushPull

```
#define INT_TRACE_SetPushPull()
```

Value:

```
do { ODCONBbits.ODCB6 = 0; } while(0)
```

6.103.1.164 INT_TRACE_Toggle

```
#define INT_TRACE_Toggle()
```

Value:

```
do { LATBbits.LATB6 = ~LATBbits.LATB6; } while(0)
```

6.103.1.165 INT_TRACE_TRIS

```
#define INT_TRACE_TRIS TRISBbits.TRISB6
```

6.103.1.166 INT_TRACE_WPU

```
#define INT_TRACE_WPU WPUBbits.WPUB6
```

6.103.1.167 IO_RB5_ANS

```
#define IO_RB5_ANS ANSELBbits.ANSELB5
```

6.103.1.168 IO_RB5_GetValue

```
#define IO_RB5_GetValue()
```

Value:

```
PORTBbits.RB5
```

6.103.1.169 IO_RB5_LAT

```
#define IO_RB5_LAT LATBbits.LATB5
```

6.103.1.170 IO_RB5_OD

```
#define IO_RB5_OD ODCONBbits.ODCB5
```

6.103.1.171 IO_RB5_PORT

```
#define IO_RB5_PORT PORTBbits.RB5
```

6.103.1.172 IO_RB5_ResetPullup

```
#define IO_RB5_ResetPullup()
```

Value:

```
do { WPUBbits.WPUB5 = 0; } while(0)
```

6.103.1.173 IO_RB5_SetAnalogMode

```
#define IO_RB5_SetAnalogMode()
```

Value:

```
do { ANSELBbits.ANSELB5 = 1; } while(0)
```

6.103.1.174 IO_RB5_SetDigitalInput

```
#define IO_RB5_SetDigitalInput()
```

Value:

```
do { TRISBbits.TRISB5 = 1; } while(0)
```

6.103.1.175 IO_RB5_SetDigitalMode

```
#define IO_RB5_SetDigitalMode()
```

Value:

```
do { ANSELBbits.ANSELB5 = 0; } while(0)
```

6.103.1.176 IO_RB5_SetDigitalOutput

```
#define IO_RB5_SetDigitalOutput()
```

Value:

```
do { TRISBbits.TRISB5 = 0; } while(0)
```

6.103.1.177 IO_RB5_SetHigh

```
#define IO_RB5_SetHigh()
```

Value:

```
do { LATBbits.LATB5 = 1; } while(0)
```

6.103.1.178 IO_RB5_SetLow

```
#define IO_RB5_SetLow()
```

Value:

```
do { LATBbits.LATB5 = 0; } while(0)
```

6.103.1.179 IO_RB5_SetOpenDrain

```
#define IO_RB5_SetOpenDrain()
```

Value:

```
do { ODCONBbits.ODCB5 = 1; } while(0)
```

6.103.1.180 IO_RB5_SetPullup

```
#define IO_RB5_SetPullup()
```

Value:

```
do { WPUBbits.WPUB5 = 1; } while(0)
```

6.103.1.181 IO_RB5_SetPushPull

```
#define IO_RB5_SetPushPull()
```

Value:

```
do { ODCONBbits.ODCB5 = 0; } while(0)
```

6.103.1.182 IO_RB5_Toggle

```
#define IO_RB5_Toggle()
```

Value:

```
do { LATBbits.LATB5 = ~LATBbits.LATB5; } while(0)
```

6.103.1.183 IO_RB5_TRIS

```
#define IO_RB5_TRIS TRISBbits.TRISB5
```

6.103.1.184 IO_RB5_WPU

```
#define IO_RB5_WPU WPUBbits.WPUB5
```

6.103.1.185 IO_RD7_ANS

```
#define IO_RD7_ANS ANSELDbits.ANSELD7
```

6.103.1.186 IO_RD7_GetValue

```
#define IO_RD7_GetValue()
```

Value:

```
PORTDbits.RD7
```

6.103.1.187 IO_RD7_LAT

```
#define IO_RD7_LAT LATDbits.LATD7
```

6.103.1.188 IO_RD7_OD

```
#define IO_RD7_OD ODCONDbits.ODCD7
```

6.103.1.189 IO_RD7_PORT

```
#define IO_RD7_PORT PORTDbits.RD7
```


6.103.1.190 IO_RD7_ResetPullup

```
#define IO_RD7_ResetPullup()
```

Value:

```
do { WPUDbits.WPUD7 = 0; } while(0)
```

6.103.1.191 IO_RD7_SetAnalogMode

```
#define IO_RD7_SetAnalogMode()
```

Value:

```
do { ANSELDbits.ANSELD7 = 1; } while(0)
```

6.103.1.192 IO_RD7_SetDigitalInput

```
#define IO_RD7_SetDigitalInput()
```

Value:

```
do { TRISDbits.TRISD7 = 1; } while(0)
```

6.103.1.193 IO_RD7_SetDigitalMode

```
#define IO_RD7_SetDigitalMode()
```

Value:

```
do { ANSELDbits.ANSELD7 = 0; } while(0)
```

6.103.1.194 IO_RD7_SetDigitalOutput

```
#define IO_RD7_SetDigitalOutput()
```

Value:

```
do { TRISDbits.TRISD7 = 0; } while(0)
```

6.103.1.195 IO_RD7_SetHigh

```
#define IO_RD7_SetHigh()
```

Value:

```
do { LATDbits.LATD7 = 1; } while(0)
```

6.103.1.196 IO_RD7_SetLow

```
#define IO_RD7_SetLow()
```

Value:

```
do { LATDbits.LATD7 = 0; } while(0)
```

6.103.1.197 IO_RD7_SetOpenDrain

```
#define IO_RD7_SetOpenDrain()
```

Value:

```
do { ODCONDbits.ODCD7 = 1; } while(0)
```

6.103.1.198 IO_RD7_SetPullup

```
#define IO_RD7_SetPullup()
```

Value:

```
do { WPUDbits.WPUD7 = 1; } while(0)
```

6.103.1.199 IO_RD7_SetPushPull

```
#define IO_RD7_SetPushPull()
```

Value:

```
do { ODCONDbits.ODCD7 = 0; } while(0)
```

6.103.1.200 IO_RD7_Toggle

```
#define IO_RD7_Toggle()
```

Value:

```
do { LATDbits.LATD7 = ~LATDbits.LATD7; } while(0)
```

6.103.1.201 IO_RD7_TRIS

```
#define IO_RD7_TRIS TRISDbits.TRISD7
```

6.103.1.202 IO_RD7_WPU

```
#define IO_RD7_WPU WPUDbits.WPUD7
```

6.103.1.203 L_SWITCH_ANS

```
#define L_SWITCH_ANS ANSELABits.ANSELA2
```

6.103.1.204 L_SWITCH_GetValue

```
#define L_SWITCH_GetValue()
```

Value:

```
PORTAbits.RA2
```

6.103.1.205 L_SWITCH_LAT

```
#define L_SWITCH_LAT LATAbits.LATA2
```

6.103.1.206 L_SWITCH_OD

```
#define L_SWITCH_OD ODCONAbits.ODCA2
```

6.103.1.207 L_SWITCH_PORT

```
#define L_SWITCH_PORT PORTAbits.RA2
```

6.103.1.208 L_SWITCH_ResetPullup

```
#define L_SWITCH_ResetPullup()
```

Value:

```
do { WPUAbits.WPUA2 = 0; } while(0)
```

6.103.1.209 L_SWITCH_SetAnalogMode

```
#define L_SWITCH_SetAnalogMode()
```

Value:

```
do { ANSELAbits.ANSELA2 = 1; } while(0)
```

6.103.1.210 L_SWITCH_SetDigitalInput

```
#define L_SWITCH_SetDigitalInput()
```

Value:

```
do { TRISAbits.TRISA2 = 1; } while(0)
```

6.103.1.211 L_SWITCH_SetDigitalMode

```
#define L_SWITCH_SetDigitalMode()
```

Value:

```
do { ANSELAbits.ANSELA2 = 0; } while(0)
```

6.103.1.212 L_SWITCH_SetDigitalOutput

```
#define L_SWITCH_SetDigitalOutput()
```

Value:

```
do { TRISAbits.TRISA2 = 0; } while(0)
```

6.103.1.213 L_SWITCH_SetHigh

```
#define L_SWITCH_SetHigh()
```

Value:

```
do { LATABits.LATA2 = 1; } while(0)
```

6.103.1.214 L_SWITCH_SetLow

```
#define L_SWITCH_SetLow()
```

Value:

```
do { LATABits.LATA2 = 0; } while(0)
```

6.103.1.215 L_SWITCH_SetOpenDrain

```
#define L_SWITCH_SetOpenDrain()
```

Value:

```
do { ODCONAbits.ODCA2 = 1; } while(0)
```

6.103.1.216 L_SWITCH_SetPullup

```
#define L_SWITCH_SetPullup()
```

Value:

```
do { WPUAbits.WPUA2 = 1; } while(0)
```

6.103.1.217 L_SWITCH_SetPushPull

```
#define L_SWITCH_SetPushPull()
```

Value:

```
do { ODCONAbits.ODCA2 = 0; } while(0)
```

6.103.1.218 L_SWITCH_Toggle

```
#define L_SWITCH_Toggle()
```

Value:

```
do { LATABits.LATA2 = ~LATABits.LATA2; } while(0)
```

6.103.1.219 L_SWITCH_TRIS

```
#define L_SWITCH_TRIS TRISAbits.TRISA2
```

6.103.1.220 L_SWITCH_WPU

```
#define L_SWITCH_WPU WPUAbits.WPUA2
```

6.103.1.221 LOW

```
#define LOW 0
```

6.103.1.222 M_SWITCH_ANS

```
#define M_SWITCH_ANS ANSELAbits.ANSELA1
```

6.103.1.223 M_SWITCH_GetValue

```
#define M_SWITCH_GetValue()
```

Value:

```
PORTAbits.RA1
```

6.103.1.224 M_SWITCH_LAT

```
#define M_SWITCH_LAT LATAbits.LATA1
```

6.103.1.225 M_SWITCH_OD

```
#define M_SWITCH_OD ODCONAbits.ODCA1
```

6.103.1.226 M_SWITCH_PORT

```
#define M_SWITCH_PORT PORTAbits.RA1
```

6.103.1.227 M_SWITCH_ResetPullup

```
#define M_SWITCH_ResetPullup()
```

Value:

```
do { WPUAbits.WPUA1 = 0; } while(0)
```

6.103.1.228 M_SWITCH_SetAnalogMode

```
#define M_SWITCH_SetAnalogMode()
```

Value:

```
do { ANSELAbits.ANSELA1 = 1; } while(0)
```

6.103.1.229 M_SWITCH_SetDigitalInput

```
#define M_SWITCH_SetDigitalInput()
```

Value:

```
do { TRISAbits.TRISA1 = 1; } while(0)
```

6.103.1.230 M_SWITCH_SetDigitalMode

```
#define M_SWITCH_SetDigitalMode()
```

Value:

```
do { ANSELAbits.ANSELA1 = 0; } while(0)
```

6.103.1.231 M_SWITCH_SetDigitalOutput

```
#define M_SWITCH_SetDigitalOutput()
```

Value:

```
do { TRISAbits.TRISA1 = 0; } while(0)
```

6.103.1.232 M_SWITCH_SetHigh

```
#define M_SWITCH_SetHigh()
```

Value:

```
do { LATAbits.LATA1 = 1; } while(0)
```

6.103.1.233 M_SWITCH_SetLow

```
#define M_SWITCH_SetLow()
```

Value:

```
do { LATAbits.LATA1 = 0; } while(0)
```

6.103.1.234 M_SWITCH_SetOpenDrain

```
#define M_SWITCH_SetOpenDrain()
```

Value:

```
do { ODCONAbits.ODCA1 = 1; } while(0)
```

6.103.1.235 M_SWITCH_SetPullup

```
#define M_SWITCH_SetPullup()
```

Value:

```
do { WPUAbits.WPUA1 = 1; } while(0)
```

6.103.1.236 M_SWITCH_SetPushPull

```
#define M_SWITCH_SetPushPull()
```

Value:

```
do { ODCONAbits.ODCA1 = 0; } while(0)
```

6.103.1.237 M_SWITCH_Toggle

```
#define M_SWITCH_Toggle()
```

Value:

```
do { LATAbits.LATA1 = ~LATAbits.LATA1; } while(0)
```

6.103.1.238 M_SWITCH_TRIS

```
#define M_SWITCH_TRIS TRISAbits.TRISA1
```

6.103.1.239 M_SWITCH_WPU

```
#define M_SWITCH_WPU WPUAbits.WPUA1
```

6.103.1.240 MISC_ANS

```
#define MISC_ANS ANSELEbits.ANSELE1
```

6.103.1.241 MISC_GetValue

```
#define MISC_GetValue()
```

Value:

```
PORTEbits.RE1
```

6.103.1.242 MISC_LAT

```
#define MISC_LAT LATEbits.LATE1
```

6.103.1.243 MISC_OD

```
#define MISC_OD ODCONEbits.ODCE1
```

6.103.1.244 MISC_PORT

```
#define MISC_PORT PORTEbits.RE1
```

6.103.1.245 MISC_ResetPullup

```
#define MISC_ResetPullup()
```

Value:

```
do { WPUEbits.WPUE1 = 0; } while(0)
```

6.103.1.246 MISC_SetAnalogMode

```
#define MISC_SetAnalogMode()
```

Value:

```
do { ANSELEbits.ANSELE1 = 1; } while(0)
```

6.103.1.247 MISC_SetDigitalInput

```
#define MISC_SetDigitalInput()
```

Value:

```
do { TRISEbits.TRISE1 = 1; } while(0)
```

6.103.1.248 MISC_SetDigitalMode

```
#define MISC_SetDigitalMode()
```

Value:

```
do { ANSELEbits.ANSELE1 = 0; } while(0)
```

6.103.1.249 MISC_SetDigitalOutput

```
#define MISC_SetDigitalOutput()
```

Value:

```
do { TRISEbits.TRISE1 = 0; } while(0)
```

6.103.1.250 MISC_SetHigh

```
#define MISC_SetHigh()
```

Value:

```
do { LATEbits.LATE1 = 1; } while(0)
```

6.103.1.251 MISC_SetLow

```
#define MISC_SetLow()
```

Value:

```
do { LATEbits.LATE1 = 0; } while(0)
```


6.103.1.252 MISC_SetOpenDrain

```
#define MISC_SetOpenDrain()
```

Value:

```
do { ODCONEbits.ODCE1 = 1; } while(0)
```

6.103.1.253 MISC_SetPullup

```
#define MISC_SetPullup()
```

Value:

```
do { WPUEbits.WPUE1 = 1; } while(0)
```

6.103.1.254 MISC_SetPushPull

```
#define MISC_SetPushPull()
```

Value:

```
do { ODCONEbits.ODCE1 = 0; } while(0)
```

6.103.1.255 MISC_Toggle

```
#define MISC_Toggle()
```

Value:

```
do { LATEbits.LATE1 = ~LATEbits.LATE1; } while(0)
```

6.103.1.256 MISC_TRIS

```
#define MISC_TRIS TRISEbits.TRISE1
```

6.103.1.257 MISC_WPU

```
#define MISC_WPU WPUEbits.WPUE1
```

6.103.1.258 MLED_ANS

```
#define MLED_ANS ANSELBbits.ANSELB1
```

6.103.1.259 MLED_GetValue

```
#define MLED_GetValue()
```

Value:

```
PORTBbits.RB1
```

6.103.1.260 MLED_LAT

```
#define MLED_LAT LATBbits.LATB1
```

6.103.1.261 MLED_OD

```
#define MLED_OD ODCONBbits.ODCB1
```

6.103.1.262 MLED_PORT

```
#define MLED_PORT PORTBbits.RB1
```

6.103.1.263 MLED_ResetPullup

```
#define MLED_ResetPullup()
```

Value:

```
do { WPUBbits.WPUB1 = 0; } while(0)
```

6.103.1.264 MLED_SetAnalogMode

```
#define MLED_SetAnalogMode()
```

Value:

```
do { ANSELBbits.ANSELB1 = 1; } while(0)
```

6.103.1.265 MLED_SetDigitalInput

```
#define MLED_SetDigitalInput()
```

Value:

```
do { TRISBbits.TRISB1 = 1; } while(0)
```

6.103.1.266 MLED_SetDigitalMode

```
#define MLED_SetDigitalMode()
```

Value:

```
do { ANSELBbits.ANSELB1 = 0; } while(0)
```

6.103.1.267 MLED_SetDigitalOutput

```
#define MLED_SetDigitalOutput()
```

Value:

```
do { TRISBbits.TRISB1 = 0; } while(0)
```

6.103.1.268 MLED_SetHigh

```
#define MLED_SetHigh()
```

Value:

```
do { LATBbits.LATB1 = 1; } while(0)
```

6.103.1.269 MLED_SetLow

```
#define MLED_SetLow()
```

Value:

```
do { LATBbits.LATB1 = 0; } while(0)
```

6.103.1.270 MLED_SetOpenDrain

```
#define MLED_SetOpenDrain()
```

Value:

```
do { ODCONBbits.ODCB1 = 1; } while(0)
```

6.103.1.271 MLED_SetPullup

```
#define MLED_SetPullup()
```

Value:

```
do { WPUBbits.WPUB1 = 1; } while(0)
```

6.103.1.272 MLED_SetPushPull

```
#define MLED_SetPushPull()
```

Value:

```
do { ODCONBbits.ODCB1 = 0; } while(0)
```

6.103.1.273 MLED_Toggle

```
#define MLED_Toggle()
```

Value:

```
do { LATBbits.LATB1 = ~LATBbits.LATB1; } while(0)
```

6.103.1.274 MLED_TRIS

```
#define MLED_TRIS TRISBbits.TRISB1
```

6.103.1.275 MLED_WPU

```
#define MLED_WPU WPUBbits.WPUB1
```

6.103.1.276 OUTPUT

```
#define OUTPUT 0
```

6.103.1.277 PROG_TRACE_ANS

```
#define PROG_TRACE_ANS ANSELDbits.ANSELD5
```

6.103.1.278 PROG_TRACE_GetValue

```
#define PROG_TRACE_GetValue()
```

Value:

```
PORTDbits.RD5
```

6.103.1.279 PROG_TRACE_LAT

```
#define PROG_TRACE_LAT LATDbits.LATD5
```

6.103.1.280 PROG_TRACE_OD

```
#define PROG_TRACE_OD ODCONDbits.ODCD5
```

6.103.1.281 PROG_TRACE_PORT

```
#define PROG_TRACE_PORT PORTDbits.RD5
```

6.103.1.282 PROG_TRACE_ResetPullup

```
#define PROG_TRACE_ResetPullup()
```

Value:

```
do { WPUDbits.WPUD5 = 0; } while(0)
```

6.103.1.283 PROG_TRACE_SetAnalogMode

```
#define PROG_TRACE_SetAnalogMode()
```

Value:

```
do { ANSELDbits.ANSELD5 = 1; } while(0)
```

6.103.1.284 PROG_TRACE_SetDigitalInput

```
#define PROG_TRACE_SetDigitalInput()
```

Value:

```
do { TRISDbits.TRISD5 = 1; } while(0)
```

6.103.1.285 PROG_TRACE_SetDigitalMode

```
#define PROG_TRACE_SetDigitalMode()
```

Value:

```
do { ANSELDbits.ANSEL5 = 0; } while(0)
```

6.103.1.286 PROG_TRACE_SetDigitalOutput

```
#define PROG_TRACE_SetDigitalOutput()
```

Value:

```
do { TRISDbits.TRISD5 = 0; } while(0)
```

6.103.1.287 PROG_TRACE_SetHigh

```
#define PROG_TRACE_SetHigh()
```

Value:

```
do { LATDbits.LATD5 = 1; } while(0)
```

6.103.1.288 PROG_TRACE_SetLow

```
#define PROG_TRACE_SetLow()
```

Value:

```
do { LATDbits.LATD5 = 0; } while(0)
```

6.103.1.289 PROG_TRACE_SetOpenDrain

```
#define PROG_TRACE_SetOpenDrain()
```

Value:

```
do { ODCONDbits.ODCD5 = 1; } while(0)
```

6.103.1.290 PROG_TRACE_SetPullup

```
#define PROG_TRACE_SetPullup()
```

Value:

```
do { WPUDbits.WPUD5 = 1; } while(0)
```

6.103.1.291 PROG_TRACE_SetPushPull

```
#define PROG_TRACE_SetPushPull()
```

Value:

```
do { ODCONDbits.ODCD5 = 0; } while(0)
```

6.103.1.292 PROG_TRACE_Toggle

```
#define PROG_TRACE_Toggle()
```

Value:

```
do { LATDbits.LATD5 = ~LATDbits.LATD5; } while(0)
```

6.103.1.293 PROG_TRACE_TRIS

```
#define PROG_TRACE_TRIS TRISDbits.TRISD5
```

6.103.1.294 PROG_TRACE_WPU

```
#define PROG_TRACE_WPU WPUDbits.WPUD5
```

6.103.1.295 PULL_UP_DISABLED

```
#define PULL_UP_DISABLED 0
```

6.103.1.296 PULL_UP_ENABLED

```
#define PULL_UP_ENABLED 1
```

6.103.1.297 RA0_GetValue

```
#define RA0_GetValue()
```

Value:

```
PORTAbits.RA0
```

6.103.1.298 RA0_ResetPullup

```
#define RA0_ResetPullup()
```

Value:

```
do { WPUAbits.WPUA0 = 0; } while(0)
```

6.103.1.299 RA0_SetAnalogMode

```
#define RA0_SetAnalogMode()
```

Value:

```
do { ANSELAbits.ANSELA0 = 1; } while(0)
```

6.103.1.300 RA0_SetDigitalInput

```
#define RA0_SetDigitalInput()
```

Value:

```
do { TRISAbits.TRISA0 = 1; } while(0)
```

6.103.1.301 RA0_SetDigitalMode

```
#define RA0_SetDigitalMode()
```

Value:

```
do { ANSELAbits.ANSELA0 = 0; } while(0)
```

6.103.1.302 RA0_SetDigitalOutput

```
#define RA0_SetDigitalOutput()
```

Value:

```
do { TRISAbits.TRISA0 = 0; } while(0)
```

6.103.1.303 RA0_SetHigh

```
#define RA0_SetHigh()
```

Value:

```
do { LATAbits.LATA0 = 1; } while(0)
```

6.103.1.304 RA0_SetLow

```
#define RA0_SetLow()
```

Value:

```
do { LATAbits.LATA0 = 0; } while(0)
```

6.103.1.305 RA0_SetPullup

```
#define RA0_SetPullup()
```

Value:

```
do { WPUAbits.WPUA0 = 1; } while(0)
```

6.103.1.306 RA0_Toggle

```
#define RA0_Toggle()
```

Value:

```
do { LATAbits.LATA0 = ~LATAbits.LATA0; } while(0)
```

6.103.1.307 RA3_GetValue

```
#define RA3_GetValue()
```

Value:

```
PORTAbits.RA3
```

6.103.1.308 RA3_ResetPullup

```
#define RA3_ResetPullup()
```

Value:

```
do { WPUAbits.WPUA3 = 0; } while(0)
```

6.103.1.309 RA3_SetAnalogMode

```
#define RA3_SetAnalogMode()
```

Value:

```
do { ANSELAbits.ANSELA3 = 1; } while(0)
```

6.103.1.310 RA3_SetDigitalInput

```
#define RA3_SetDigitalInput()
```

Value:

```
do { TRISAbits.TRISA3 = 1; } while(0)
```

6.103.1.311 RA3_SetDigitalMode

```
#define RA3_SetDigitalMode()
```

Value:

```
do { ANSELAbits.ANSELA3 = 0; } while(0)
```

6.103.1.312 RA3_SetDigitalOutput

```
#define RA3_SetDigitalOutput()
```

Value:

```
do { TRISAbits.TRISA3 = 0; } while(0)
```


6.103.1.313 RA3_SetHigh

```
#define RA3_SetHigh()
```

Value:

```
do { LATAbits.LATA3 = 1; } while(0)
```

6.103.1.314 RA3_SetLow

```
#define RA3_SetLow()
```

Value:

```
do { LATAbits.LATA3 = 0; } while(0)
```

6.103.1.315 RA3_SetPullup

```
#define RA3_SetPullup()
```

Value:

```
do { WPUAbits.WPUA3 = 1; } while(0)
```

6.103.1.316 RA3_Toggle

```
#define RA3_Toggle()
```

Value:

```
do { LATAbits.LATA3 = ~LATAbits.LATA3; } while(0)
```

6.103.1.317 RA4_GetValue

```
#define RA4_GetValue()
```

Value:

```
PORTAbits.RA4
```

6.103.1.318 RA4_ResetPullup

```
#define RA4_ResetPullup()
```

Value:

```
do { WPUAbits.WPUA4 = 0; } while(0)
```

6.103.1.319 RA4_SetAnalogMode

```
#define RA4_SetAnalogMode()
```

Value:

```
do { ANSELAbits.ANSELA4 = 1; } while(0)
```

6.103.1.320 RA4_SetDigitalInput

```
#define RA4_SetDigitalInput()
```

Value:

```
do { TRISAbits.TRISA4 = 1; } while(0)
```

6.103.1.321 RA4_SetDigitalMode

```
#define RA4_SetDigitalMode()
```

Value:

```
do { ANSELAbits.ANSELA4 = 0; } while(0)
```

6.103.1.322 RA4_SetDigitalOutput

```
#define RA4_SetDigitalOutput()
```

Value:

```
do { TRISAbits.TRISA4 = 0; } while(0)
```

6.103.1.323 RA4_SetHigh

```
#define RA4_SetHigh()
```

Value:

```
do { LATAbits.LATA4 = 1; } while(0)
```

6.103.1.324 RA4_SetLow

```
#define RA4_SetLow()
```

Value:

```
do { LATAbits.LATA4 = 0; } while(0)
```

6.103.1.325 RA4_SetPullup

```
#define RA4_SetPullup()
```

Value:

```
do { WPUAbits.WPUA4 = 1; } while(0)
```

6.103.1.326 RA4_Toggle

```
#define RA4_Toggle()
```

Value:

```
do { LATAbits.LATA4 = ~LATAbits.LATA4; } while(0)
```

6.103.1.327 RB0_GetValue

```
#define RB0_GetValue()
```

Value:

```
PORTBbits.RB0
```

6.103.1.328 RB0_ResetPullup

```
#define RB0_ResetPullup()
```

Value:

```
do { WPUBbits.WPUB0 = 0; } while(0)
```

6.103.1.329 RB0_SetAnalogMode

```
#define RB0_SetAnalogMode()
```

Value:

```
do { ANSELBbits.ANSELB0 = 1; } while(0)
```

6.103.1.330 RB0_SetDigitalInput

```
#define RB0_SetDigitalInput()
```

Value:

```
do { TRISBbits.TRISB0 = 1; } while(0)
```

6.103.1.331 RB0_SetDigitalMode

```
#define RB0_SetDigitalMode()
```

Value:

```
do { ANSELBbits.ANSELB0 = 0; } while(0)
```

6.103.1.332 RB0_SetDigitalOutput

```
#define RB0_SetDigitalOutput()
```

Value:

```
do { TRISBbits.TRISB0 = 0; } while(0)
```

6.103.1.333 RB0_SetHigh

```
#define RB0_SetHigh()
```

Value:

```
do { LATBbits.LATB0 = 1; } while(0)
```

6.103.1.334 RB0_SetLow

```
#define RB0_SetLow()
```

Value:

```
do { LATBbits.LATB0 = 0; } while(0)
```

6.103.1.335 RB0_SetPullup

```
#define RB0_SetPullup()
```

Value:

```
do { WPUBbits.WPUB0 = 1; } while(0)
```

6.103.1.336 RB0_Toggle

```
#define RB0_Toggle()
```

Value:

```
do { LATBbits.LATB0 = ~LATBbits.LATB0; } while(0)
```

6.103.1.337 RB4_GetValue

```
#define RB4_GetValue()
```

Value:

```
PORTBbits.RB4
```

6.103.1.338 RB4_ResetPullup

```
#define RB4_ResetPullup()
```

Value:

```
do { WPUBbits.WPUB4 = 0; } while(0)
```

6.103.1.339 RB4_SetAnalogMode

```
#define RB4_SetAnalogMode()
```

Value:

```
do { ANSELBbits.ANSELB4 = 1; } while(0)
```

6.103.1.340 RB4_SetDigitalInput

```
#define RB4_SetDigitalInput()
```

Value:

```
do { TRISBbits.TRISB4 = 1; } while(0)
```

6.103.1.341 RB4_SetDigitalMode

```
#define RB4_SetDigitalMode()
```

Value:

```
do { ANSELBbits.ANSELB4 = 0; } while(0)
```

6.103.1.342 RB4_SetDigitalOutput

```
#define RB4_SetDigitalOutput()
```

Value:

```
do { TRISBbits.TRISB4 = 0; } while(0)
```

6.103.1.343 RB4_SetHigh

```
#define RB4_SetHigh()
```

Value:

```
do { LATBbits.LATB4 = 1; } while(0)
```

6.103.1.344 RB4_SetLow

```
#define RB4_SetLow()
```

Value:

```
do { LATBbits.LATB4 = 0; } while(0)
```

6.103.1.345 RB4_SetPullup

```
#define RB4_SetPullup()
```

Value:

```
do { WPUBbits.WPUB4 = 1; } while(0)
```

6.103.1.346 RB4_Toggle

```
#define RB4_Toggle()
```

Value:

```
do { LATBbits.LATB4 = ~LATBbits.LATB4; } while(0)
```

6.103.1.347 RB7_GetValue

```
#define RB7_GetValue()
```

Value:

```
PORTBbits.RB7
```

6.103.1.348 RB7_ResetPullup

```
#define RB7_ResetPullup()
```

Value:

```
do { WPUBbits.WPUB7 = 0; } while(0)
```

6.103.1.349 RB7_SetAnalogMode

```
#define RB7_SetAnalogMode()
```

Value:

```
do { ANSELBbits.ANSELB7 = 1; } while(0)
```

6.103.1.350 RB7_SetDigitalInput

```
#define RB7_SetDigitalInput()
```

Value:

```
do { TRISBbits.TRISB7 = 1; } while(0)
```

6.103.1.351 RB7_SetDigitalMode

```
#define RB7_SetDigitalMode()
```

Value:

```
do { ANSELBbits.ANSELB7 = 0; } while(0)
```

6.103.1.352 RB7_SetDigitalOutput

```
#define RB7_SetDigitalOutput()
```

Value:

```
do { TRISBbits.TRISB7 = 0; } while(0)
```

6.103.1.353 RB7_SetHigh

```
#define RB7_SetHigh()
```

Value:

```
do { LATBbits.LATB7 = 1; } while(0)
```

6.103.1.354 RB7_SetLow

```
#define RB7_SetLow()
```

Value:

```
do { LATBbits.LATB7 = 0; } while(0)
```

6.103.1.355 RB7_SetPullup

```
#define RB7_SetPullup()
```

Value:

```
do { WPUBbits.WPUB7 = 1; } while(0)
```

6.103.1.356 RB7_Toggle

```
#define RB7_Toggle()
```

Value:

```
do { LATBbits.LATB7 = ~LATBbits.LATB7; } while(0)
```

6.103.1.357 RC0_GetValue

```
#define RC0_GetValue()
```

Value:

```
PORTCbits.RC0
```

6.103.1.358 RC0_ResetPullup

```
#define RC0_ResetPullup()
```

Value:

```
do { WPUCbits.WPUC0 = 0; } while(0)
```

6.103.1.359 RC0_SetAnalogMode

```
#define RC0_SetAnalogMode()
```

Value:

```
do { ANSELbits.ANSEL0 = 1; } while(0)
```

6.103.1.360 RC0_SetDigitalInput

```
#define RC0_SetDigitalInput()
```

Value:

```
do { TRISCbits.TRISC0 = 1; } while(0)
```

6.103.1.361 RC0_SetDigitalMode

```
#define RC0_SetDigitalMode()
```

Value:

```
do { ANSELbits.ANSEL0 = 0; } while(0)
```

6.103.1.362 RC0_SetDigitalOutput

```
#define RC0_SetDigitalOutput()
```

Value:

```
do { TRISCbits.TRISC0 = 0; } while(0)
```

6.103.1.363 RC0_SetHigh

```
#define RC0_SetHigh()
```

Value:

```
do { LATCbits.LATC0 = 1; } while(0)
```

6.103.1.364 RC0_SetLow

```
#define RC0_SetLow()
```

Value:

```
do { LATCbits.LATC0 = 0; } while(0)
```

6.103.1.365 RC0_SetPullup

```
#define RC0_SetPullup()
```

Value:

```
do { WPUCbits.WPUC0 = 1; } while(0)
```

6.103.1.366 RC0_Toggle

```
#define RC0_Toggle()
```

Value:

```
do { LATCbits.LATC0 = ~LATCbits.LATC0; } while(0)
```

6.103.1.367 RC1_GetValue

```
#define RC1_GetValue()
```

Value:

```
PORTCbits.RC1
```

6.103.1.368 RC1_ResetPullup

```
#define RC1_ResetPullup()
```

Value:

```
do { WPUCbits.WPUC1 = 0; } while(0)
```


6.103.1.369 RC1_SetAnalogMode

```
#define RC1_SetAnalogMode()
```

Value:

```
do { ANSELbits.ANSEL1 = 1; } while(0)
```

6.103.1.370 RC1_SetDigitalInput

```
#define RC1_SetDigitalInput()
```

Value:

```
do { TRISCbits.TRISC1 = 1; } while(0)
```

6.103.1.371 RC1_SetDigitalMode

```
#define RC1_SetDigitalMode()
```

Value:

```
do { ANSELbits.ANSEL1 = 0; } while(0)
```

6.103.1.372 RC1_SetDigitalOutput

```
#define RC1_SetDigitalOutput()
```

Value:

```
do { TRISCbits.TRISC1 = 0; } while(0)
```

6.103.1.373 RC1_SetHigh

```
#define RC1_SetHigh()
```

Value:

```
do { LATCbits.LATC1 = 1; } while(0)
```

6.103.1.374 RC1_SetLow

```
#define RC1_SetLow()
```

Value:

```
do { LATCbits.LATC1 = 0; } while(0)
```

6.103.1.375 RC1_SetPullup

```
#define RC1_SetPullup()
```

Value:

```
do { WPUCbits.WPUC1 = 1; } while(0)
```

6.103.1.376 RC1_Toggle

```
#define RC1_Toggle()
```

Value:

```
do { LATCbits.LATC1 = ~LATCbits.LATC1; } while(0)
```

6.103.1.377 RC3_GetValue

```
#define RC3_GetValue()
```

Value:

```
PORTCbits.RC3
```

6.103.1.378 RC3_ResetPullup

```
#define RC3_ResetPullup()
```

Value:

```
do { WPUCbits.WPUC3 = 0; } while(0)
```

6.103.1.379 RC3_SetAnalogMode

```
#define RC3_SetAnalogMode()
```

Value:

```
do { ANSELbits.ANSEL3 = 1; } while(0)
```

6.103.1.380 RC3_SetDigitalInput

```
#define RC3_SetDigitalInput()
```

Value:

```
do { TRISCbits.TRISC3 = 1; } while(0)
```

6.103.1.381 RC3_SetDigitalMode

```
#define RC3_SetDigitalMode()
```

Value:

```
do { ANSELbits.ANSEL3 = 0; } while(0)
```

6.103.1.382 RC3_SetDigitalOutput

```
#define RC3_SetDigitalOutput()
```

Value:

```
do { TRISCbits.TRISC3 = 0; } while(0)
```

6.103.1.383 RC3_SetHigh

```
#define RC3_SetHigh()
```

Value:

```
do { LATCbits.LATC3 = 1; } while(0)
```

6.103.1.384 RC3_SetLow

```
#define RC3_SetLow()
```

Value:

```
do { LATCbits.LATC3 = 0; } while(0)
```

6.103.1.385 RC3_SetPullup

```
#define RC3_SetPullup()
```

Value:

```
do { WPUCbits.WPUC3 = 1; } while(0)
```

6.103.1.386 RC3_Toggle

```
#define RC3_Toggle()
```

Value:

```
do { LATCbits.LATC3 = ~LATCbits.LATC3; } while(0)
```

6.103.1.387 RC4_GetValue

```
#define RC4_GetValue()
```

Value:

```
PORTCbits.RC4
```

6.103.1.388 RC4_ResetPullup

```
#define RC4_ResetPullup()
```

Value:

```
do { WPUCbits.WPUC4 = 0; } while(0)
```

6.103.1.389 RC4_SetAnalogMode

```
#define RC4_SetAnalogMode()
```

Value:

```
do { ANSELbits.ANSEL4 = 1; } while(0)
```

6.103.1.390 RC4_SetDigitalInput

```
#define RC4_SetDigitalInput()
```

Value:

```
do { TRISCbits.TRISC4 = 1; } while(0)
```

6.103.1.391 RC4_SetDigitalMode

```
#define RC4_SetDigitalMode()
```

Value:

```
do { ANSELbits.ANSEL4 = 0; } while(0)
```

6.103.1.392 RC4_SetDigitalOutput

```
#define RC4_SetDigitalOutput()
```

Value:

```
do { TRISCbits.TRISC4 = 0; } while(0)
```

6.103.1.393 RC4_SetHigh

```
#define RC4_SetHigh()
```

Value:

```
do { LATCbits.LATC4 = 1; } while(0)
```

6.103.1.394 RC4_SetLow

```
#define RC4_SetLow()
```

Value:

```
do { LATCbits.LATC4 = 0; } while(0)
```

6.103.1.395 RC4_SetPullup

```
#define RC4_SetPullup()
```

Value:

```
do { WPUCbits.WPUC4 = 1; } while(0)
```

6.103.1.396 RC4_Toggle

```
#define RC4_Toggle()
```

Value:

```
do { LATCbits.LATC4 = ~LATCbits.LATC4; } while(0)
```

6.103.1.397 RC5_GetValue

```
#define RC5_GetValue()
```

Value:

```
PORTCbits.RC5
```

6.103.1.398 RC5_ResetPullup

```
#define RC5_ResetPullup()
```

Value:

```
do { WPUCbits.WPUC5 = 0; } while(0)
```

6.103.1.399 RC5_SetAnalogMode

```
#define RC5_SetAnalogMode()
```

Value:

```
do { ANSELbits.ANSLC5 = 1; } while(0)
```

6.103.1.400 RC5_SetDigitalInput

```
#define RC5_SetDigitalInput()
```

Value:

```
do { TRISCbits.TRISC5 = 1; } while(0)
```

6.103.1.401 RC5_SetDigitalMode

```
#define RC5_SetDigitalMode()
```

Value:

```
do { ANSELbits.ANSLC5 = 0; } while(0)
```

6.103.1.402 RC5_SetDigitalOutput

```
#define RC5_SetDigitalOutput()
```

Value:

```
do { TRISCbits.TRISC5 = 0; } while(0)
```

6.103.1.403 RC5_SetHigh

```
#define RC5_SetHigh()
```

Value:

```
do { LATCbits.LATC5 = 1; } while(0)
```

6.103.1.404 RC5_SetLow

```
#define RC5_SetLow()
```

Value:

```
do { LATCbits.LATC5 = 0; } while(0)
```

6.103.1.405 RC5_SetPullup

```
#define RC5_SetPullup()
```

Value:

```
do { WPUCbits.WPUC5 = 1; } while(0)
```

6.103.1.406 RC5_Toggle

```
#define RC5_Toggle()
```

Value:

```
do { LATCbits.LATC5 = ~LATCbits.LATC5; } while(0)
```

6.103.1.407 RC6_GetValue

```
#define RC6_GetValue()
```

Value:

```
PORTCbits.RC6
```

6.103.1.408 RC6_ResetPullup

```
#define RC6_ResetPullup()
```

Value:

```
do { WPUCbits.WPUC6 = 0; } while(0)
```

6.103.1.409 RC6_SetAnalogMode

```
#define RC6_SetAnalogMode()
```

Value:

```
do { ANSELbits.ANSEL6 = 1; } while(0)
```

6.103.1.410 RC6_SetDigitalInput

```
#define RC6_SetDigitalInput()
```

Value:

```
do { TRISCbits.TRISC6 = 1; } while(0)
```

6.103.1.411 RC6_SetDigitalMode

```
#define RC6_SetDigitalMode()
```

Value:

```
do { ANSELbits.ANSEL6 = 0; } while(0)
```

6.103.1.412 RC6_SetDigitalOutput

```
#define RC6_SetDigitalOutput()
```

Value:

```
do { TRISCbits.TRISC6 = 0; } while(0)
```

6.103.1.413 RC6_SetHigh

```
#define RC6_SetHigh()
```

Value:

```
do { LATCbits.LATC6 = 1; } while(0)
```

6.103.1.414 RC6_SetLow

```
#define RC6_SetLow()
```

Value:

```
do { LATCbits.LATC6 = 0; } while(0)
```

6.103.1.415 RC6_SetPullup

```
#define RC6_SetPullup()
```

Value:

```
do { WPUCbits.WPUC6 = 1; } while(0)
```

6.103.1.416 RC6_Toggle

```
#define RC6_Toggle()
```

Value:

```
do { LATCbits.LATC6 = ~LATCbits.LATC6; } while(0)
```

6.103.1.417 RC7_GetValue

```
#define RC7_GetValue()
```

Value:

```
PORTCbits.RC7
```

6.103.1.418 RC7_ResetPullup

```
#define RC7_ResetPullup()
```

Value:

```
do { WPUCbits.WPUC7 = 0; } while(0)
```

6.103.1.419 RC7_SetAnalogMode

```
#define RC7_SetAnalogMode()
```

Value:

```
do { ANSELbits.ANSEL7 = 1; } while(0)
```

6.103.1.420 RC7_SetDigitalInput

```
#define RC7_SetDigitalInput()
```

Value:

```
do { TRISCbits.TRISC7 = 1; } while(0)
```

6.103.1.421 RC7_SetDigitalMode

```
#define RC7_SetDigitalMode()
```

Value:

```
do { ANSELbits.ANSEL7 = 0; } while(0)
```

6.103.1.422 RC7_SetDigitalOutput

```
#define RC7_SetDigitalOutput()
```

Value:

```
do { TRISCbits.TRISC7 = 0; } while(0)
```

6.103.1.423 RC7_SetHigh

```
#define RC7_SetHigh()
```

Value:

```
do { LATCbits.LATC7 = 1; } while(0)
```

6.103.1.424 RC7_SetLow

```
#define RC7_SetLow()
```

Value:

```
do { LATCbits.LATC7 = 0; } while(0)
```


6.103.1.425 RC7_SetPullup

```
#define RC7_SetPullup()
```

Value:

```
do { WPUCbits.WPUC7 = 1; } while(0)
```

6.103.1.426 RC7_Toggle

```
#define RC7_Toggle()
```

Value:

```
do { LATCbits.LATC7 = ~LATCbits.LATC7; } while(0)
```

6.103.1.427 RD0_GetValue

```
#define RD0_GetValue()
```

Value:

```
PORTDbits.RD0
```

6.103.1.428 RD0_ResetPullup

```
#define RD0_ResetPullup()
```

Value:

```
do { WPUDbits.WPUD0 = 0; } while(0)
```

6.103.1.429 RD0_SetAnalogMode

```
#define RD0_SetAnalogMode()
```

Value:

```
do { ANSELDbits.ANSELD0 = 1; } while(0)
```

6.103.1.430 RD0_SetDigitalInput

```
#define RD0_SetDigitalInput()
```

Value:

```
do { TRISDbits.TRISD0 = 1; } while(0)
```

6.103.1.431 RD0_SetDigitalMode

```
#define RD0_SetDigitalMode()
```

Value:

```
do { ANSELDbits.ANSELD0 = 0; } while(0)
```

6.103.1.432 RD0_SetDigitalOutput

```
#define RD0_SetDigitalOutput()
```

Value:

```
do { TRISDbits.TRISD0 = 0; } while(0)
```

6.103.1.433 RD0_SetHigh

```
#define RD0_SetHigh()
```

Value:

```
do { LATDbits.LATD0 = 1; } while(0)
```

6.103.1.434 RD0_SetLow

```
#define RD0_SetLow()
```

Value:

```
do { LATDbits.LATD0 = 0; } while(0)
```

6.103.1.435 RD0_SetPullup

```
#define RD0_SetPullup()
```

Value:

```
do { WPUDbits.WPUD0 = 1; } while(0)
```

6.103.1.436 RD0_Toggle

```
#define RD0_Toggle()
```

Value:

```
do { LATDbits.LATD0 = ~LATDbits.LATD0; } while(0)
```

6.103.1.437 RD6_GetValue

```
#define RD6_GetValue()
```

Value:

```
PORTDbits.RD6
```

6.103.1.438 RD6_ResetPullup

```
#define RD6_ResetPullup()
```

Value:

```
do { WPUDbits.WPUD6 = 0; } while(0)
```

6.103.1.439 RD6_SetAnalogMode

```
#define RD6_SetAnalogMode()
```

Value:

```
do { ANSELDbits.ANSELD6 = 1; } while(0)
```

6.103.1.440 RD6_SetDigitalInput

```
#define RD6_SetDigitalInput()
```

Value:

```
do { TRISDbits.TRISD6 = 1; } while(0)
```

6.103.1.441 RD6_SetDigitalMode

```
#define RD6_SetDigitalMode()
```

Value:

```
do { ANSELDbits.ANSELD6 = 0; } while(0)
```

6.103.1.442 RD6_SetDigitalOutput

```
#define RD6_SetDigitalOutput()
```

Value:

```
do { TRISDbits.TRISD6 = 0; } while(0)
```

6.103.1.443 RD6_SetHigh

```
#define RD6_SetHigh()
```

Value:

```
do { LATDbits.LATD6 = 1; } while(0)
```

6.103.1.444 RD6_SetLow

```
#define RD6_SetLow()
```

Value:

```
do { LATDbits.LATD6 = 0; } while(0)
```

6.103.1.445 RD6_SetPullup

```
#define RD6_SetPullup()
```

Value:

```
do { WPUDbits.WPUD6 = 1; } while(0)
```

6.103.1.446 RD6_Toggle

```
#define RD6_Toggle()
```

Value:

```
do { LATDbits.LATD6 = ~LATDbits.LATD6; } while(0)
```

6.103.1.447 RELAY_ANS

```
#define RELAY_ANS ANSELEbits.ANSELE0
```

6.103.1.448 RELAY_GetValue

```
#define RELAY_GetValue()
```

Value:

```
PORTEbits.RE0
```

6.103.1.449 RELAY_LAT

```
#define RELAY_LAT LATEbits.LATE0
```

6.103.1.450 RELAY_OD

```
#define RELAY_OD ODCONEbits.ODCE0
```

6.103.1.451 RELAY_PORT

```
#define RELAY_PORT PORTEbits.RE0
```

6.103.1.452 RELAY_ResetPullup

```
#define RELAY_ResetPullup()
```

Value:

```
do { WPUEbits.WPUE0 = 0; } while(0)
```

6.103.1.453 RELAY_SetAnalogMode

```
#define RELAY_SetAnalogMode()
```

Value:

```
do { ANSELEbits.ANSELE0 = 1; } while(0)
```

6.103.1.454 RELAY_SetDigitalInput

```
#define RELAY_SetDigitalInput()
```

Value:

```
do { TRISEbits.TRISE0 = 1; } while(0)
```

6.103.1.455 RELAY_SetDigitalMode

```
#define RELAY_SetDigitalMode()
```

Value:

```
do { ANSELEbits.ANSELE0 = 0; } while(0)
```

6.103.1.456 RELAY_SetDigitalOutput

```
#define RELAY_SetDigitalOutput()
```

Value:

```
do { TRISEbits.TRISE0 = 0; } while(0)
```

6.103.1.457 RELAY_SetHigh

```
#define RELAY_SetHigh()
```

Value:

```
do { LATEbits.LATE0 = 1; } while(0)
```

6.103.1.458 RELAY_SetLow

```
#define RELAY_SetLow()
```

Value:

```
do { LATEbits.LATE0 = 0; } while(0)
```

6.103.1.459 RELAY_SetOpenDrain

```
#define RELAY_SetOpenDrain()
```

Value:

```
do { ODCONEbits.ODCE0 = 1; } while(0)
```

6.103.1.460 RELAY_SetPullup

```
#define RELAY_SetPullup()
```

Value:

```
do { WPUEbits.WPUE0 = 1; } while(0)
```

6.103.1.461 RELAY_SetPushPull

```
#define RELAY_SetPushPull()
```

Value:

```
do { ODCONEbits.ODCE0 = 0; } while(0)
```

6.103.1.462 RELAY_Toggle

```
#define RELAY_Toggle()
```

Value:

```
do { LATEbits.LATE0 = ~LATEbits.LATE0; } while(0)
```

6.103.1.463 RELAY_TRIS

```
#define RELAY_TRIS TRISEbits.TRISE0
```

6.103.1.464 RELAY_WPU

```
#define RELAY_WPU WPUEbits.WPUE0
```

6.103.1.465 RLED_ANS

```
#define RLED_ANS ANSELBbits.ANSELB2
```

6.103.1.466 RLED_GetValue

```
#define RLED_GetValue()
```

Value:

```
PORTBbits.RB2
```

6.103.1.467 RLED_LAT

```
#define RLED_LAT LATBbits.LATB2
```

6.103.1.468 RLED_OD

```
#define RLED_OD ODCONBbits.ODCB2
```

6.103.1.469 RLED_PORT

```
#define RLED_PORT PORTBbits.RB2
```

6.103.1.470 RLED_ResetPullup

```
#define RLED_ResetPullup()
```

Value:

```
do { WPUBbits.WPUB2 = 0; } while(0)
```

6.103.1.471 RLED_SetAnalogMode

```
#define RLED_SetAnalogMode()
```

Value:

```
do { ANSELBbits.ANSELB2 = 1; } while(0)
```

6.103.1.472 RLED_SetDigitalInput

```
#define RLED_SetDigitalInput()
```

Value:

```
do { TRISBbits.TRISB2 = 1; } while(0)
```

6.103.1.473 RLED_SetDigitalMode

```
#define RLED_SetDigitalMode()
```

Value:

```
do { ANSELBbits.ANSELB2 = 0; } while(0)
```

6.103.1.474 RLED_SetDigitalOutput

```
#define RLED_SetDigitalOutput()
```

Value:

```
do { TRISBbits.TRISB2 = 0; } while(0)
```

6.103.1.475 RLED_SetHigh

```
#define RLED_SetHigh()
```

Value:

```
do { LATBbits.LATB2 = 1; } while(0)
```

6.103.1.476 RLED_SetLow

```
#define RLED_SetLow()
```

Value:

```
do { LATBbits.LATB2 = 0; } while(0)
```

6.103.1.477 RLED_SetOpenDrain

```
#define RLED_SetOpenDrain()
```

Value:

```
do { ODCONBbits.ODCB2 = 1; } while(0)
```

6.103.1.478 RLED_SetPullup

```
#define RLED_SetPullup()
```

Value:

```
do { WPUBbits.WPUB2 = 1; } while(0)
```

6.103.1.479 RLED_SetPushPull

```
#define RLED_SetPushPull()
```

Value:

```
do { ODCONBbits.ODCB2 = 0; } while(0)
```

6.103.1.480 RLED_Toggle

```
#define RLED_Toggle()
```

Value:

```
do { LATBbits.LATB2 = ~LATBbits.LATB2; } while(0)
```

6.103.1.481 RLED_TRIS

```
#define RLED_TRIS TRISBbits.TRISB2
```

6.103.1.482 RLED_WPU

```
#define RLED_WPU WPUBbits.WPUB2
```


6.103.2 Function Documentation

6.103.2.1 IOCAF1_DefaultInterruptHandler()

```
void IOCAF1_DefaultInterruptHandler (
    void )
```

@Summary Default Interrupt Handler for IOCAF1 pin

@Description This is a predefined interrupt handler to be used together with the [IOCAF1_SetInterruptHandler\(\)](#) method. This handler is called every time the IOCAF1 ISR is executed.

@Preconditions Pin Manager initializer called

@Returns None.

@Param None.

@Example [PIN_MANAGER_Initialize\(\)](#); [IOCAF1_SetInterruptHandler\(IOCAF1_DefaultInterruptHandler\)](#);

Default interrupt handler for IOCAF1 Here is the caller graph for this function:



6.103.2.2 IOCAF1_ISR()

```
void IOCAF1_ISR (
    void )
```

@Param none @Returns none @Description Interrupt on Change Handler for the IOCAF1 pin functionality @↔ Example [IOCAF1_ISR\(\)](#);

IOCAF1 Interrupt Service Routine Here is the caller graph for this function:



6.103.2.3 IOCAF1_SetInterruptHandler()

```
void IOCAF1_SetInterruptHandler (
    void(* InterruptHandler ) (void))
```

@Summary Interrupt Handler Setter for IOCAF1 pin interrupt-on-change functionality

@Description Allows selecting an interrupt handler for IOCAF1 at application runtime

@Preconditions Pin Manager initializer called

@Returns None.

@Param InterruptHandler function pointer.

@Example [PIN_MANAGER_Initialize\(\)](#); IOCAF1_SetInterruptHandler(MyInterruptHandler);

Allows selecting an interrupt handler for IOCAF1 at application runtime Here is the caller graph for this function:



6.103.2.4 IOCAF2_DefaultInterruptHandler()

```
void IOCAF2_DefaultInterruptHandler (
    void )
```

@Summary Default Interrupt Handler for IOCAF2 pin

@Description This is a predefined interrupt handler to be used together with the [IOCAF2_SetInterruptHandler\(\)](#) method. This handler is called every time the IOCAF2 ISR is executed.

@Preconditions Pin Manager initializer called

@Returns None.

@Param None.

@Example [PIN_MANAGER_Initialize\(\)](#); IOCAF2_SetInterruptHandler(IOCAF2_DefaultInterruptHandler);

Default interrupt handler for IOCAF2 Here is the caller graph for this function:



6.103.2.5 IOCAF2_ISR()

```
void IOCAF2_ISR (
    void )
```

@Param none @Returns none @Description Interrupt on Change Handler for the IOCAF2 pin functionality @↔
Example [IOCAF2_ISR\(\)](#);

IOCAF2 Interrupt Service Routine Here is the caller graph for this function:



6.103.2.6 IOCAF2_SetInterruptHandler()

```
void IOCAF2_SetInterruptHandler (
    void(* InterruptHandler ) (void))
```

@Summary Interrupt Handler Setter for IOCAF2 pin interrupt-on-change functionality

@Description Allows selecting an interrupt handler for IOCAF2 at application runtime

@Preconditions Pin Manager initializer called

@Returns None.

@Param InterruptHandler function pointer.

@Example [PIN_MANAGER_Initialize\(\)](#); [IOCAF2_SetInterruptHandler\(MyInterruptHandler\)](#);

Allows selecting an interrupt handler for IOCAF2 at application runtime Here is the caller graph for this function:



6.103.2.7 IOCAF5_DefaultInterruptHandler()

```
void IOCAF5_DefaultInterruptHandler (
    void )
```

@Summary Default Interrupt Handler for IOCAF5 pin

@Description This is a predefined interrupt handler to be used together with the [IOCAF5_SetInterruptHandler\(\)](#) method. This handler is called every time the IOCAF5 ISR is executed.

@Preconditions Pin Manager initializer called

@Returns None.

@Param None.

@Example [PIN_MANAGER_Initialize\(\)](#); [IOCAF5_SetInterruptHandler\(IOCAF5_DefaultInterruptHandler\)](#);

Default interrupt handler for IOCAF5 Here is the caller graph for this function:



6.103.2.8 IOCAF5_ISR()

```
void IOCAF5_ISR (
    void )
```

@Param none @Returns none @Description Interrupt on Change Handler for the IOCAF5 pin functionality @↔
Example [IOCAF5_ISR\(\)](#);

IOCAF5 Interrupt Service Routine Here is the caller graph for this function:



6.103.2.9 IOCAF5_SetInterruptHandler()

```
void IOCAF5_SetInterruptHandler (
    void(* InterruptHandler ) (void))
```

@Summary Interrupt Handler Setter for IOCAF5 pin interrupt-on-change functionality

@Description Allows selecting an interrupt handler for IOCAF5 at application runtime

@Preconditions Pin Manager initializer called

@Returns None.

@Param InterruptHandler function pointer.

@Example [PIN_MANAGER_Initialize\(\)](#); IOCAF5_SetInterruptHandler(MyInterruptHandler);

Allows selecting an interrupt handler for IOCAF5 at application runtime Here is the caller graph for this function:



6.103.2.10 PIN_MANAGER_Initialize()

```
void PIN_MANAGER_Initialize (
    void )
```

@Param none @Returns none @Description GPIO and peripheral I/O initialization @Example [PIN_MANAGER_Initialize\(\)](#);
LATx registers

TRISx registers

ANSELx registers

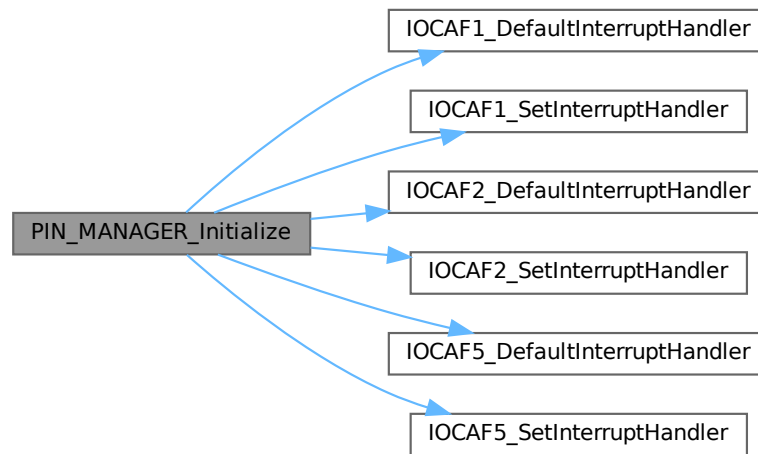
WPUx registers

ODx registers

SLRCONx registers

INLVLx registers

IOCx registersHere is the call graph for this function:



Here is the caller graph for this function:



6.103.3 Variable Documentation

6.103.3.1 IOCAF1_InterruptHandler

```
void(* IOCAF1_InterruptHandler) (void) (
    void ) [extern]
```

@Summary Dynamic Interrupt Handler for IOCAF1 pin

@Description This is a dynamic interrupt handler to be used together with the [IOCAF1_SetInterruptHandler\(\)](#) method. This handler is called every time the IOCAF1 ISR is executed and allows any function to be registered at runtime.

@Preconditions Pin Manager initializer called

@Returns None.

@Param None.

@Example `PIN_MANAGER_Initialize(); IOCAF1_SetInterruptHandler(IOCAF1_InterruptHandler);`

Generated Pin Manager File

Company: Microchip Technology Inc.

File Name: [pin_manager.c](#)

Summary: This is the Pin Manager file generated using PIC10 / PIC12 / PIC16 / PIC18 MCUs

Description: This header file provides implementations for pin APIs for all pins selected in the GUI. Generation Information : Product Revision : PIC10 / PIC12 / PIC16 / PIC18 MCUs - 1.81.8 Device : PIC18F47Q84 Driver Version : 2.11 The generated drivers are tested against the following: Compiler : XC8 2.36 and above MPLAB : MPLAB X 6.00

Copyright (c) 2013 - 2015 released Microchip Technology Inc. All rights reserved.

6.103.3.2 IOCAF2_InterruptHandler

```
void(* IOCAF2_InterruptHandler) (void) (  
    void ) [extern]
```

@Summary Dynamic Interrupt Handler for IOCAF2 pin

@Description This is a dynamic interrupt handler to be used together with the [IOCAF2_SetInterruptHandler\(\)](#) method. This handler is called every time the IOCAF2 ISR is executed and allows any function to be registered at runtime.

@Preconditions Pin Manager initializer called

@Returns None.

@Param None.

@Example [PIN_MANAGER_Initialize\(\)](#); IOCAF2_SetInterruptHandler(IOCAF2_InterruptHandler);

6.103.3.3 IOCAF5_InterruptHandler

```
void(* IOCAF5_InterruptHandler) (void) (  
    void ) [extern]
```

@Summary Dynamic Interrupt Handler for IOCAF5 pin

@Description This is a dynamic interrupt handler to be used together with the [IOCAF5_SetInterruptHandler\(\)](#) method. This handler is called every time the IOCAF5 ISR is executed and allows any function to be registered at runtime.

@Preconditions Pin Manager initializer called

@Returns None.

@Param None.

@Example [PIN_MANAGER_Initialize\(\)](#); IOCAF5_SetInterruptHandler(IOCAF5_InterruptHandler);

6.104 pin_manager.h

[Go to the documentation of this file.](#)

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
party
00029     license terms applicable to your use of third party software (including open source software) that
may accompany Microchip software.
00030
00031     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00032     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00033     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00034     FOR A PARTICULAR PURPOSE.
00035
00036     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00037     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00038     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00039     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00040     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00041     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00042     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00043     SOFTWARE.
00044 */
00045
00046 #ifndef PIN_MANAGER_H
00047 #define PIN_MANAGER_H
00048
00049 #include <xc.h>
00050
00051 #define INPUT    1
00052 #define OUTPUT   0
00053
00054 #define HIGH     1
00055 #define LOW      0
00056
00057 #define ANALOG   1
00058 #define DIGITAL  0
00059
00060 #define PULL_UP_ENABLED      1
00061 #define PULL_UP_DISABLED    0
00062
00063 // get/set RA0 procedures
00064 #define RA0_SetHigh()        do { LATAbits.LATA0 = 1; } while(0)
00065 #define RA0_SetLow()         do { LATAbits.LATA0 = 0; } while(0)
00066 #define RA0_Toggle()         do { LATAbits.LATA0 = ~LATAbits.LATA0; } while(0)
00067 #define RA0_GetValue()       PORTAbits.RA0
00068 #define RA0_SetDigitalInput() do { TRISAbits.TRISA0 = 1; } while(0)
00069 #define RA0_SetDigitalOutput() do { TRISAbits.TRISA0 = 0; } while(0)
00070 #define RA0_SetPullup()      do { WPUAbits.WPUA0 = 1; } while(0)
00071 #define RA0_ResetPullup()    do { WPUAbits.WPUA0 = 0; } while(0)
00072 #define RA0_SetAnalogMode()  do { ANSELAbits.ANSELA0 = 1; } while(0)
00073 #define RA0_SetDigitalMode() do { ANSELAbits.ANSELA0 = 0; } while(0)
00074
00075 // get/set M_SWITCH aliases
00076 #define M_SWITCH_TRIS        TRISAbits.TRISA1
00077 #define M_SWITCH_LAT         LATAbits.LATA1
00078 #define M_SWITCH_PORT        PORTAbits.RA1
00079 #define M_SWITCH_WPU         WPUAbits.WPUA1
00080 #define M_SWITCH_OD          ODCONAbits.ODCA1
00081 #define M_SWITCH_ANS         ANSELAbits.ANSELA1
00082 #define M_SWITCH_SetHigh()   do { LATAbits.LATA1 = 1; } while(0)
00083 #define M_SWITCH_SetLow()    do { LATAbits.LATA1 = 0; } while(0)
00084 #define M_SWITCH_Toggle()    do { LATAbits.LATA1 = ~LATAbits.LATA1; } while(0)
00085 #define M_SWITCH_GetValue()  PORTAbits.RA1
00086 #define M_SWITCH_SetDigitalInput() do { TRISAbits.TRISA1 = 1; } while(0)
00087 #define M_SWITCH_SetDigitalOutput() do { TRISAbits.TRISA1 = 0; } while(0)
00088 #define M_SWITCH_SetPullup()  do { WPUAbits.WPUA1 = 1; } while(0)
00089 #define M_SWITCH_ResetPullup() do { WPUAbits.WPUA1 = 0; } while(0)
00090 #define M_SWITCH_SetPushPull() do { ODCONAbits.ODCA1 = 0; } while(0)
00091 #define M_SWITCH_SetOpenDrain() do { ODCONAbits.ODCA1 = 1; } while(0)
00092 #define M_SWITCH_SetAnalogMode() do { ANSELAbits.ANSELA1 = 1; } while(0)
00093 #define M_SWITCH_SetDigitalMode() do { ANSELAbits.ANSELA1 = 0; } while(0)
00094
00095 // get/set L_SWITCH aliases
00096 #define L_SWITCH_TRIS        TRISAbits.TRISA2
00097 #define L_SWITCH_LAT         LATAbits.LATA2
00098 #define L_SWITCH_PORT        PORTAbits.RA2
00099 #define L_SWITCH_WPU         WPUAbits.WPUA2

```



```

00105 #define L_SWITCH_OD                ODCONAbits.ODCA2
00106 #define L_SWITCH_ANS                ANSELAbits.ANSELA2
00107 #define L_SWITCH_SetHigh()          do { LATAbits.LATA2 = 1; } while(0)
00108 #define L_SWITCH_SetLow()           do { LATAbits.LATA2 = 0; } while(0)
00109 #define L_SWITCH_Toggle()           do { LATAbits.LATA2 = ~LATAbits.LATA2; } while(0)
00110 #define L_SWITCH_GetValue()         PORTAbits.RA2
00111 #define L_SWITCH_SetDigitalInput()  do { TRISAbits.TRISA2 = 1; } while(0)
00112 #define L_SWITCH_SetDigitalOutput() do { TRISAbits.TRISA2 = 0; } while(0)
00113 #define L_SWITCH_SetPullup()        do { WPUAbits.WPUA2 = 1; } while(0)
00114 #define L_SWITCH_ResetPullup()      do { WPUAbits.WPUA2 = 0; } while(0)
00115 #define L_SWITCH_SetPushPull()      do { ODCONAbits.ODCA2 = 0; } while(0)
00116 #define L_SWITCH_SetOpenDrain()     do { ODCONAbits.ODCA2 = 1; } while(0)
00117 #define L_SWITCH_SetAnalogMode()    do { ANSELAbits.ANSELA2 = 1; } while(0)
00118 #define L_SWITCH_SetDigitalMode()   do { ANSELAbits.ANSELA2 = 0; } while(0)
00119
00120 // get/set RA3 procedures
00121 #define RA3_SetHigh()               do { LATAbits.LATA3 = 1; } while(0)
00122 #define RA3_SetLow()                do { LATAbits.LATA3 = 0; } while(0)
00123 #define RA3_Toggle()               do { LATAbits.LATA3 = ~LATAbits.LATA3; } while(0)
00124 #define RA3_GetValue()              PORTAbits.RA3
00125 #define RA3_SetDigitalInput()       do { TRISAbits.TRISA3 = 1; } while(0)
00126 #define RA3_SetDigitalOutput()      do { TRISAbits.TRISA3 = 0; } while(0)
00127 #define RA3_SetPullup()             do { WPUAbits.WPUA3 = 1; } while(0)
00128 #define RA3_ResetPullup()          do { WPUAbits.WPUA3 = 0; } while(0)
00129 #define RA3_SetAnalogMode()         do { ANSELAbits.ANSELA3 = 1; } while(0)
00130 #define RA3_SetDigitalMode()        do { ANSELAbits.ANSELA3 = 0; } while(0)
00131
00132 // get/set RA4 procedures
00133 #define RA4_SetHigh()               do { LATAbits.LATA4 = 1; } while(0)
00134 #define RA4_SetLow()                do { LATAbits.LATA4 = 0; } while(0)
00135 #define RA4_Toggle()               do { LATAbits.LATA4 = ~LATAbits.LATA4; } while(0)
00136 #define RA4_GetValue()              PORTAbits.RA4
00137 #define RA4_SetDigitalInput()       do { TRISAbits.TRISA4 = 1; } while(0)
00138 #define RA4_SetDigitalOutput()      do { TRISAbits.TRISA4 = 0; } while(0)
00139 #define RA4_SetPullup()             do { WPUAbits.WPUA4 = 1; } while(0)
00140 #define RA4_ResetPullup()          do { WPUAbits.WPUA4 = 0; } while(0)
00141 #define RA4_SetAnalogMode()         do { ANSELAbits.ANSELA4 = 1; } while(0)
00142 #define RA4_SetDigitalMode()        do { ANSELAbits.ANSELA4 = 0; } while(0)
00143
00144 // get/set A_SWITCH aliases
00145 #define A_SWITCH_TRIS               TRISAbits.TRISA5
00146 #define A_SWITCH_LAT                LATAbits.LATA5
00147 #define A_SWITCH_PORT               PORTAbits.RA5
00148 #define A_SWITCH_WPU                WPUAbits.WPUA5
00149 #define A_SWITCH_OD                 ODCONAbits.ODCA5
00150 #define A_SWITCH_ANS                ANSELAbits.ANSELA5
00151 #define A_SWITCH_SetHigh()          do { LATAbits.LATA5 = 1; } while(0)
00152 #define A_SWITCH_SetLow()           do { LATAbits.LATA5 = 0; } while(0)
00153 #define A_SWITCH_Toggle()           do { LATAbits.LATA5 = ~LATAbits.LATA5; } while(0)
00154 #define A_SWITCH_GetValue()         PORTAbits.RA5
00155 #define A_SWITCH_SetDigitalInput()  do { TRISAbits.TRISA5 = 1; } while(0)
00156 #define A_SWITCH_SetDigitalOutput() do { TRISAbits.TRISA5 = 0; } while(0)
00157 #define A_SWITCH_SetPullup()        do { WPUAbits.WPUA5 = 1; } while(0)
00158 #define A_SWITCH_ResetPullup()      do { WPUAbits.WPUA5 = 0; } while(0)
00159 #define A_SWITCH_SetPushPull()      do { ODCONAbits.ODCA5 = 0; } while(0)
00160 #define A_SWITCH_SetOpenDrain()     do { ODCONAbits.ODCA5 = 1; } while(0)
00161 #define A_SWITCH_SetAnalogMode()    do { ANSELAbits.ANSELA5 = 1; } while(0)
00162 #define A_SWITCH_SetDigitalMode()   do { ANSELAbits.ANSELA5 = 0; } while(0)
00163
00164 // get/set RB0 procedures
00165 #define RB0_SetHigh()               do { LATBbits.LATB0 = 1; } while(0)
00166 #define RB0_SetLow()                do { LATBbits.LATB0 = 0; } while(0)
00167 #define RB0_Toggle()               do { LATBbits.LATB0 = ~LATBbits.LATB0; } while(0)
00168 #define RB0_GetValue()              PORTBbits.RB0
00169 #define RB0_SetDigitalInput()       do { TRISBbits.TRISB0 = 1; } while(0)
00170 #define RB0_SetDigitalOutput()      do { TRISBbits.TRISB0 = 0; } while(0)
00171 #define RB0_SetPullup()             do { WPUBbits.WPUB0 = 1; } while(0)
00172 #define RB0_ResetPullup()          do { WPUBbits.WPUB0 = 0; } while(0)
00173 #define RB0_SetAnalogMode()         do { ANSELBbits.ANSELB0 = 1; } while(0)
00174 #define RB0_SetDigitalMode()        do { ANSELBbits.ANSELB0 = 0; } while(0)
00175
00176 // get/set MLED aliases
00177 #define MLED_TRIS                   TRISBbits.TRISB1
00178 #define MLED_LAT                    LATBbits.LATB1
00179 #define MLED_PORT                   PORTBbits.RB1
00180 #define MLED_WPU                    WPUBbits.WPUB1
00181 #define MLED_OD                     ODCONBbits.ODCB1
00182 #define MLED_ANS                     ANSELBbits.ANSELB1
00183 #define MLED_SetHigh()              do { LATBbits.LATB1 = 1; } while(0)
00184 #define MLED_SetLow()               do { LATBbits.LATB1 = 0; } while(0)
00185 #define MLED_Toggle()               do { LATBbits.LATB1 = ~LATBbits.LATB1; } while(0)
00186 #define MLED_GetValue()             PORTBbits.RB1
00187 #define MLED_SetDigitalInput()       do { TRISBbits.TRISB1 = 1; } while(0)
00188 #define MLED_SetDigitalOutput()      do { TRISBbits.TRISB1 = 0; } while(0)
00189 #define MLED_SetPullup()            do { WPUBbits.WPUB1 = 1; } while(0)
00190 #define MLED_ResetPullup()          do { WPUBbits.WPUB1 = 0; } while(0)
00191 #define MLED_SetPushPull()          do { ODCONBbits.ODCB1 = 0; } while(0)

```

```

00192 #define MLED_SetOpenDrain() do { ODCONBbits.ODCB1 = 1; } while(0)
00193 #define MLED_SetAnalogMode() do { ANSELBbits.ANSELB1 = 1; } while(0)
00194 #define MLED_SetDigitalMode() do { ANSELBbits.ANSELB1 = 0; } while(0)
00195
00196 // get/set RLED aliases
00197 #define RLED_SetHigh() TRISBbits.TRISB2
00198 #define RLED_LAT LATBbits.LATB2
00199 #define RLED_PORT PORTBbits.RB2
00200 #define RLED_WPU WPUBbits.WPUB2
00201 #define RLED_OD ODCONBbits.ODCB2
00202 #define RLED_ANS ANSELBbits.ANSELB2
00203 #define RLED_SetHigh() do { LATBbits.LATB2 = 1; } while(0)
00204 #define RLED_SetLow() do { LATBbits.LATB2 = 0; } while(0)
00205 #define RLED_Toggle() do { LATBbits.LATB2 = ~LATBbits.LATB2; } while(0)
00206 #define RLED_GetValue() PORTBbits.RB2
00207 #define RLED_SetDigitalInput() do { TRISBbits.TRISB2 = 1; } while(0)
00208 #define RLED_SetDigitalOutput() do { TRISBbits.TRISB2 = 0; } while(0)
00209 #define RLED_SetPullup() do { WPUBbits.WPUB2 = 1; } while(0)
00210 #define RLED_ResetPullup() do { WPUBbits.WPUB2 = 0; } while(0)
00211 #define RLED_SetPushPull() do { ODCONBbits.ODCB2 = 0; } while(0)
00212 #define RLED_SetOpenDrain() do { ODCONBbits.ODCB2 = 1; } while(0)
00213 #define RLED_SetAnalogMode() do { ANSELBbits.ANSELB2 = 1; } while(0)
00214 #define RLED_SetDigitalMode() do { ANSELBbits.ANSELB2 = 0; } while(0)
00215
00216 // get/set DLED1 aliases
00217 #define DLED1_TRIS TRISBbits.TRISB3
00218 #define DLED1_LAT LATBbits.LATB3
00219 #define DLED1_PORT PORTBbits.RB3
00220 #define DLED1_WPU WPUBbits.WPUB3
00221 #define DLED1_OD ODCONBbits.ODCB3
00222 #define DLED1_ANS ANSELBbits.ANSELB3
00223 #define DLED1_SetHigh() do { LATBbits.LATB3 = 1; } while(0)
00224 #define DLED1_SetLow() do { LATBbits.LATB3 = 0; } while(0)
00225 #define DLED1_Toggle() do { LATBbits.LATB3 = ~LATBbits.LATB3; } while(0)
00226 #define DLED1_GetValue() PORTBbits.RB3
00227 #define DLED1_SetDigitalInput() do { TRISBbits.TRISB3 = 1; } while(0)
00228 #define DLED1_SetDigitalOutput() do { TRISBbits.TRISB3 = 0; } while(0)
00229 #define DLED1_SetPullup() do { WPUBbits.WPUB3 = 1; } while(0)
00230 #define DLED1_ResetPullup() do { WPUBbits.WPUB3 = 0; } while(0)
00231 #define DLED1_SetPushPull() do { ODCONBbits.ODCB3 = 0; } while(0)
00232 #define DLED1_SetOpenDrain() do { ODCONBbits.ODCB3 = 1; } while(0)
00233 #define DLED1_SetAnalogMode() do { ANSELBbits.ANSELB3 = 1; } while(0)
00234 #define DLED1_SetDigitalMode() do { ANSELBbits.ANSELB3 = 0; } while(0)
00235
00236 // get/set RB4 procedures
00237 #define RB4_SetHigh() do { LATBbits.LATB4 = 1; } while(0)
00238 #define RB4_SetLow() do { LATBbits.LATB4 = 0; } while(0)
00239 #define RB4_Toggle() do { LATBbits.LATB4 = ~LATBbits.LATB4; } while(0)
00240 #define RB4_GetValue() PORTBbits.RB4
00241 #define RB4_SetDigitalInput() do { TRISBbits.TRISB4 = 1; } while(0)
00242 #define RB4_SetDigitalOutput() do { TRISBbits.TRISB4 = 0; } while(0)
00243 #define RB4_SetPullup() do { WPUBbits.WPUB4 = 1; } while(0)
00244 #define RB4_ResetPullup() do { WPUBbits.WPUB4 = 0; } while(0)
00245 #define RB4_SetAnalogMode() do { ANSELBbits.ANSELB4 = 1; } while(0)
00246 #define RB4_SetDigitalMode() do { ANSELBbits.ANSELB4 = 0; } while(0)
00247
00248 // get/set IO_RB5 aliases
00249 #define IO_RB5_TRIS TRISBbits.TRISB5
00250 #define IO_RB5_LAT LATBbits.LATB5
00251 #define IO_RB5_PORT PORTBbits.RB5
00252 #define IO_RB5_WPU WPUBbits.WPUB5
00253 #define IO_RB5_OD ODCONBbits.ODCB5
00254 #define IO_RB5_ANS ANSELBbits.ANSELB5
00255 #define IO_RB5_SetHigh() do { LATBbits.LATB5 = 1; } while(0)
00256 #define IO_RB5_SetLow() do { LATBbits.LATB5 = 0; } while(0)
00257 #define IO_RB5_Toggle() do { LATBbits.LATB5 = ~LATBbits.LATB5; } while(0)
00258 #define IO_RB5_GetValue() PORTBbits.RB5
00259 #define IO_RB5_SetDigitalInput() do { TRISBbits.TRISB5 = 1; } while(0)
00260 #define IO_RB5_SetDigitalOutput() do { TRISBbits.TRISB5 = 0; } while(0)
00261 #define IO_RB5_SetPullup() do { WPUBbits.WPUB5 = 1; } while(0)
00262 #define IO_RB5_ResetPullup() do { WPUBbits.WPUB5 = 0; } while(0)
00263 #define IO_RB5_SetPushPull() do { ODCONBbits.ODCB5 = 0; } while(0)
00264 #define IO_RB5_SetOpenDrain() do { ODCONBbits.ODCB5 = 1; } while(0)
00265 #define IO_RB5_SetAnalogMode() do { ANSELBbits.ANSELB5 = 1; } while(0)
00266 #define IO_RB5_SetDigitalMode() do { ANSELBbits.ANSELB5 = 0; } while(0)
00267
00268 // get/set INT_TRACE aliases
00269 #define INT_TRACE_TRIS TRISBbits.TRISB6
00270 #define INT_TRACE_LAT LATBbits.LATB6
00271 #define INT_TRACE_PORT PORTBbits.RB6
00272 #define INT_TRACE_WPU WPUBbits.WPUB6
00273 #define INT_TRACE_OD ODCONBbits.ODCB6
00274 #define INT_TRACE_ANS ANSELBbits.ANSELB6
00275 #define INT_TRACE_SetHigh() do { LATBbits.LATB6 = 1; } while(0)
00276 #define INT_TRACE_SetLow() do { LATBbits.LATB6 = 0; } while(0)
00277 #define INT_TRACE_Toggle() do { LATBbits.LATB6 = ~LATBbits.LATB6; } while(0)
00278 #define INT_TRACE_GetValue() PORTBbits.RB6

```

```

00279 #define INT_TRACE_SetDigitalInput() do { TRISBbits.TRISB6 = 1; } while(0)
00280 #define INT_TRACE_SetDigitalOutput() do { TRISBbits.TRISB6 = 0; } while(0)
00281 #define INT_TRACE_SetPullup() do { WPUBbits.WPUB6 = 1; } while(0)
00282 #define INT_TRACE_ResetPullup() do { WPUBbits.WPUB6 = 0; } while(0)
00283 #define INT_TRACE_SetPushPull() do { ODCONBbits.ODCB6 = 0; } while(0)
00284 #define INT_TRACE_SetOpenDrain() do { ODCONBbits.ODCB6 = 1; } while(0)
00285 #define INT_TRACE_SetAnalogMode() do { ANSELBbits.ANSELB6 = 1; } while(0)
00286 #define INT_TRACE_SetDigitalMode() do { ANSELBbits.ANSELB6 = 0; } while(0)
00287
00288 // get/set RB7 procedures
00289 #define RB7_SetHigh() do { LATBbits.LATB7 = 1; } while(0)
00290 #define RB7_SetLow() do { LATBbits.LATB7 = 0; } while(0)
00291 #define RB7_Toggle() do { LATBbits.LATB7 = ~LATBbits.LATB7; } while(0)
00292 #define RB7_GetValue() PORTBbits.RB7
00293 #define RB7_SetDigitalInput() do { TRISBbits.TRISB7 = 1; } while(0)
00294 #define RB7_SetDigitalOutput() do { TRISBbits.TRISB7 = 0; } while(0)
00295 #define RB7_SetPullup() do { WPUBbits.WPUB7 = 1; } while(0)
00296 #define RB7_ResetPullup() do { WPUBbits.WPUB7 = 0; } while(0)
00297 #define RB7_SetAnalogMode() do { ANSELBbits.ANSELB7 = 1; } while(0)
00298 #define RB7_SetDigitalMode() do { ANSELBbits.ANSELB7 = 0; } while(0)
00299
00300 // get/set RC0 procedures
00301 #define RC0_SetHigh() do { LATCbits.LATC0 = 1; } while(0)
00302 #define RC0_SetLow() do { LATCbits.LATC0 = 0; } while(0)
00303 #define RC0_Toggle() do { LATCbits.LATC0 = ~LATCbits.LATC0; } while(0)
00304 #define RC0_GetValue() PORTCbits.RC0
00305 #define RC0_SetDigitalInput() do { TRISCbits.TRISC0 = 1; } while(0)
00306 #define RC0_SetDigitalOutput() do { TRISCbits.TRISC0 = 0; } while(0)
00307 #define RC0_SetPullup() do { WPUCbits.WPUC0 = 1; } while(0)
00308 #define RC0_ResetPullup() do { WPUCbits.WPUC0 = 0; } while(0)
00309 #define RC0_SetAnalogMode() do { ANSELCbits.ANSEL0 = 1; } while(0)
00310 #define RC0_SetDigitalMode() do { ANSELCbits.ANSEL0 = 0; } while(0)
00311
00312 // get/set RC1 procedures
00313 #define RC1_SetHigh() do { LATCbits.LATC1 = 1; } while(0)
00314 #define RC1_SetLow() do { LATCbits.LATC1 = 0; } while(0)
00315 #define RC1_Toggle() do { LATCbits.LATC1 = ~LATCbits.LATC1; } while(0)
00316 #define RC1_GetValue() PORTCbits.RC1
00317 #define RC1_SetDigitalInput() do { TRISCbits.TRISC1 = 1; } while(0)
00318 #define RC1_SetDigitalOutput() do { TRISCbits.TRISC1 = 0; } while(0)
00319 #define RC1_SetPullup() do { WPUCbits.WPUC1 = 1; } while(0)
00320 #define RC1_ResetPullup() do { WPUCbits.WPUC1 = 0; } while(0)
00321 #define RC1_SetAnalogMode() do { ANSELCbits.ANSEL1 = 1; } while(0)
00322 #define RC1_SetDigitalMode() do { ANSELCbits.ANSEL1 = 0; } while(0)
00323
00324 // get/set DERE aliases
00325 #define DERE_TRIS TRISCbits.TRISC2
00326 #define DERE_LAT LATCbits.LATC2
00327 #define DERE_PORT PORTCbits.RC2
00328 #define DERE_WPU WPUCbits.WPUC2
00329 #define DERE_OD ODCONCbits.ODCC2
00330 #define DERE_ANS ANSELCbits.ANSEL2
00331 #define DERE_SetHigh() do { LATCbits.LATC2 = 1; } while(0)
00332 #define DERE_SetLow() do { LATCbits.LATC2 = 0; } while(0)
00333 #define DERE_Toggle() do { LATCbits.LATC2 = ~LATCbits.LATC2; } while(0)
00334 #define DERE_GetValue() PORTCbits.RC2
00335 #define DERE_SetDigitalInput() do { TRISCbits.TRISC2 = 1; } while(0)
00336 #define DERE_SetDigitalOutput() do { TRISCbits.TRISC2 = 0; } while(0)
00337 #define DERE_SetPullup() do { WPUCbits.WPUC2 = 1; } while(0)
00338 #define DERE_ResetPullup() do { WPUCbits.WPUC2 = 0; } while(0)
00339 #define DERE_SetPushPull() do { ODCONCbits.ODCC2 = 0; } while(0)
00340 #define DERE_SetOpenDrain() do { ODCONCbits.ODCC2 = 1; } while(0)
00341 #define DERE_SetAnalogMode() do { ANSELCbits.ANSEL2 = 1; } while(0)
00342 #define DERE_SetDigitalMode() do { ANSELCbits.ANSEL2 = 0; } while(0)
00343
00344 // get/set RC3 procedures
00345 #define RC3_SetHigh() do { LATCbits.LATC3 = 1; } while(0)
00346 #define RC3_SetLow() do { LATCbits.LATC3 = 0; } while(0)
00347 #define RC3_Toggle() do { LATCbits.LATC3 = ~LATCbits.LATC3; } while(0)
00348 #define RC3_GetValue() PORTCbits.RC3
00349 #define RC3_SetDigitalInput() do { TRISCbits.TRISC3 = 1; } while(0)
00350 #define RC3_SetDigitalOutput() do { TRISCbits.TRISC3 = 0; } while(0)
00351 #define RC3_SetPullup() do { WPUCbits.WPUC3 = 1; } while(0)
00352 #define RC3_ResetPullup() do { WPUCbits.WPUC3 = 0; } while(0)
00353 #define RC3_SetAnalogMode() do { ANSELCbits.ANSEL3 = 1; } while(0)
00354 #define RC3_SetDigitalMode() do { ANSELCbits.ANSEL3 = 0; } while(0)
00355
00356 // get/set RC4 procedures
00357 #define RC4_SetHigh() do { LATCbits.LATC4 = 1; } while(0)
00358 #define RC4_SetLow() do { LATCbits.LATC4 = 0; } while(0)
00359 #define RC4_Toggle() do { LATCbits.LATC4 = ~LATCbits.LATC4; } while(0)
00360 #define RC4_GetValue() PORTCbits.RC4
00361 #define RC4_SetDigitalInput() do { TRISCbits.TRISC4 = 1; } while(0)
00362 #define RC4_SetDigitalOutput() do { TRISCbits.TRISC4 = 0; } while(0)
00363 #define RC4_SetPullup() do { WPUCbits.WPUC4 = 1; } while(0)
00364 #define RC4_ResetPullup() do { WPUCbits.WPUC4 = 0; } while(0)
00365 #define RC4_SetAnalogMode() do { ANSELCbits.ANSEL4 = 1; } while(0)

```

```

00366 #define RC4_SetDigitalMode()          do { ANSELbits.ANSEL4 = 0; } while(0)
00367
00368 // get/set RC5 procedures
00369 #define RC5_SetHigh()                  do { LATbits.LATC5 = 1; } while(0)
00370 #define RC5_SetLow()                   do { LATbits.LATC5 = 0; } while(0)
00371 #define RC5_Toggle()                   do { LATbits.LATC5 = ~LATbits.LATC5; } while(0)
00372 #define RC5_GetValue()                 PORTbits.RC5
00373 #define RC5_SetDigitalInput()          do { TRISbits.TRISC5 = 1; } while(0)
00374 #define RC5_SetDigitalOutput()         do { TRISbits.TRISC5 = 0; } while(0)
00375 #define RC5_SetPullup()                do { WPUCbits.WPUC5 = 1; } while(0)
00376 #define RC5_ResetPullup()              do { WPUCbits.WPUC5 = 0; } while(0)
00377 #define RC5_SetAnalogMode()            do { ANSELbits.ANSEL5 = 1; } while(0)
00378 #define RC5_SetDigitalMode()           do { ANSELbits.ANSEL5 = 0; } while(0)
00379
00380 // get/set RC6 procedures
00381 #define RC6_SetHigh()                  do { LATbits.LATC6 = 1; } while(0)
00382 #define RC6_SetLow()                   do { LATbits.LATC6 = 0; } while(0)
00383 #define RC6_Toggle()                   do { LATbits.LATC6 = ~LATbits.LATC6; } while(0)
00384 #define RC6_GetValue()                 PORTbits.RC6
00385 #define RC6_SetDigitalInput()          do { TRISbits.TRISC6 = 1; } while(0)
00386 #define RC6_SetDigitalOutput()         do { TRISbits.TRISC6 = 0; } while(0)
00387 #define RC6_SetPullup()                do { WPUCbits.WPUC6 = 1; } while(0)
00388 #define RC6_ResetPullup()              do { WPUCbits.WPUC6 = 0; } while(0)
00389 #define RC6_SetAnalogMode()            do { ANSELbits.ANSEL6 = 1; } while(0)
00390 #define RC6_SetDigitalMode()           do { ANSELbits.ANSEL6 = 0; } while(0)
00391
00392 // get/set RC7 procedures
00393 #define RC7_SetHigh()                  do { LATbits.LATC7 = 1; } while(0)
00394 #define RC7_SetLow()                   do { LATbits.LATC7 = 0; } while(0)
00395 #define RC7_Toggle()                   do { LATbits.LATC7 = ~LATbits.LATC7; } while(0)
00396 #define RC7_GetValue()                 PORTbits.RC7
00397 #define RC7_SetDigitalInput()          do { TRISbits.TRISC7 = 1; } while(0)
00398 #define RC7_SetDigitalOutput()         do { TRISbits.TRISC7 = 0; } while(0)
00399 #define RC7_SetPullup()                do { WPUCbits.WPUC7 = 1; } while(0)
00400 #define RC7_ResetPullup()              do { WPUCbits.WPUC7 = 0; } while(0)
00401 #define RC7_SetAnalogMode()            do { ANSELbits.ANSEL7 = 1; } while(0)
00402 #define RC7_SetDigitalMode()           do { ANSELbits.ANSEL7 = 0; } while(0)
00403
00404 // get/set RD0 procedures
00405 #define RD0_SetHigh()                  do { LATbits.LATD0 = 1; } while(0)
00406 #define RD0_SetLow()                   do { LATbits.LATD0 = 0; } while(0)
00407 #define RD0_Toggle()                   do { LATbits.LATD0 = ~LATbits.LATD0; } while(0)
00408 #define RD0_GetValue()                 PORTbits.RD0
00409 #define RD0_SetDigitalInput()          do { TRISbits.TRISD0 = 1; } while(0)
00410 #define RD0_SetDigitalOutput()         do { TRISbits.TRISD0 = 0; } while(0)
00411 #define RD0_SetPullup()                do { WPUDbits.WPUD0 = 1; } while(0)
00412 #define RD0_ResetPullup()              do { WPUDbits.WPUD0 = 0; } while(0)
00413 #define RD0_SetAnalogMode()            do { ANSELbits.ANSEL0 = 1; } while(0)
00414 #define RD0_SetDigitalMode()           do { ANSELbits.ANSEL0 = 0; } while(0)
00415
00416 // get/set DB0 aliases
00417 #define DB0_TRIS                       TRISbits.TRISD1
00418 #define DB0_LAT                         LATbits.LATD1
00419 #define DB0_PORT                        PORTbits.RD1
00420 #define DB0_WPU                         WPUDbits.WPUD1
00421 #define DB0_OD                         ODCONbits.ODCD1
00422 #define DB0_ANS                         ANSELbits.ANSEL0
00423 #define DB0_SetHigh()                  do { LATbits.LATD1 = 1; } while(0)
00424 #define DB0_SetLow()                   do { LATbits.LATD1 = 0; } while(0)
00425 #define DB0_Toggle()                   do { LATbits.LATD1 = ~LATbits.LATD1; } while(0)
00426 #define DB0_GetValue()                 PORTbits.RD1
00427 #define DB0_SetDigitalInput()          do { TRISbits.TRISD1 = 1; } while(0)
00428 #define DB0_SetDigitalOutput()         do { TRISbits.TRISD1 = 0; } while(0)
00429 #define DB0_SetPullup()                do { WPUDbits.WPUD1 = 1; } while(0)
00430 #define DB0_ResetPullup()              do { WPUDbits.WPUD1 = 0; } while(0)
00431 #define DB0_SetPushPull()              do { ODCONbits.ODCD1 = 0; } while(0)
00432 #define DB0_SetOpenDrain()             do { ODCONbits.ODCD1 = 1; } while(0)
00433 #define DB0_SetAnalogMode()            do { ANSELbits.ANSEL1 = 1; } while(0)
00434 #define DB0_SetDigitalMode()           do { ANSELbits.ANSEL1 = 0; } while(0)
00435
00436 // get/set DB1 aliases
00437 #define DB1_TRIS                       TRISbits.TRISD2
00438 #define DB1_LAT                         LATbits.LATD2
00439 #define DB1_PORT                        PORTbits.RD2
00440 #define DB1_WPU                         WPUDbits.WPUD2
00441 #define DB1_OD                         ODCONbits.ODCD2
00442 #define DB1_ANS                         ANSELbits.ANSEL2
00443 #define DB1_SetHigh()                  do { LATbits.LATD2 = 1; } while(0)
00444 #define DB1_SetLow()                   do { LATbits.LATD2 = 0; } while(0)
00445 #define DB1_Toggle()                   do { LATbits.LATD2 = ~LATbits.LATD2; } while(0)
00446 #define DB1_GetValue()                 PORTbits.RD2
00447 #define DB1_SetDigitalInput()          do { TRISbits.TRISD2 = 1; } while(0)
00448 #define DB1_SetDigitalOutput()         do { TRISbits.TRISD2 = 0; } while(0)
00449 #define DB1_SetPullup()                do { WPUDbits.WPUD2 = 1; } while(0)
00450 #define DB1_ResetPullup()              do { WPUDbits.WPUD2 = 0; } while(0)
00451 #define DB1_SetPushPull()              do { ODCONbits.ODCD2 = 0; } while(0)
00452 #define DB1_SetOpenDrain()             do { ODCONbits.ODCD2 = 1; } while(0)

```

```

00453 #define DB1_SetAnalogMode()      do { ANSELDbits.ANSELD2 = 1; } while(0)
00454 #define DB1_SetDigitalMode()     do { ANSELDbits.ANSELD2 = 0; } while(0)
00455
00456 // get/set CS aliases
00457 #define CS_TRIS                  TRISDbits.TRISD3
00458 #define CS_LAT                   LATDbits.LATD3
00459 #define CS_PORT                  PORTDbits.RD3
00460 #define CS_WPU                   WPUDbits.WPUD3
00461 #define CS_OD                    ODCONDbits.ODCD3
00462 #define CS_ANS                   ANSELDbits.ANSELD3
00463 #define CS_SetHigh()             do { LATDbits.LATD3 = 1; } while(0)
00464 #define CS_SetLow()              do { LATDbits.LATD3 = 0; } while(0)
00465 #define CS_Toggle()              do { LATDbits.LATD3 = ~LATDbits.LATD3; } while(0)
00466 #define CS_GetValue()            PORTDbits.RD3
00467 #define CS_SetDigitalInput()     do { TRISDbits.TRISD3 = 1; } while(0)
00468 #define CS_SetDigitalOutput()    do { TRISDbits.TRISD3 = 0; } while(0)
00469 #define CS_SetPullup()           do { WPUDbits.WPUD3 = 1; } while(0)
00470 #define CS_ResetPullup()         do { WPUDbits.WPUD3 = 0; } while(0)
00471 #define CS_SetPushPull()         do { ODCONDbits.ODCD3 = 0; } while(0)
00472 #define CS_SetOpenDrain()        do { ODCONDbits.ODCD3 = 1; } while(0)
00473 #define CS_SetAnalogMode()       do { ANSELDbits.ANSELD3 = 1; } while(0)
00474 #define CS_SetDigitalMode()      do { ANSELDbits.ANSELD3 = 0; } while(0)
00475
00476 // get/set DLED aliases
00477 #define DLED_TRIS                TRISDbits.TRISD4
00478 #define DLED_LAT                 LATDbits.LATD4
00479 #define DLED_PORT                PORTDbits.RD4
00480 #define DLED_WPU                 WPUDbits.WPUD4
00481 #define DLED_OD                  ODCONDbits.ODCD4
00482 #define DLED_ANS                 ANSELDbits.ANSELD4
00483 #define DLED_SetHigh()           do { LATDbits.LATD4 = 1; } while(0)
00484 #define DLED_SetLow()            do { LATDbits.LATD4 = 0; } while(0)
00485 #define DLED_Toggle()            do { LATDbits.LATD4 = ~LATDbits.LATD4; } while(0)
00486 #define DLED_GetValue()          PORTDbits.RD4
00487 #define DLED_SetDigitalInput()   do { TRISDbits.TRISD4 = 1; } while(0)
00488 #define DLED_SetDigitalOutput()  do { TRISDbits.TRISD4 = 0; } while(0)
00489 #define DLED_SetPullup()         do { WPUDbits.WPUD4 = 1; } while(0)
00490 #define DLED_ResetPullup()       do { WPUDbits.WPUD4 = 0; } while(0)
00491 #define DLED_SetPushPull()       do { ODCONDbits.ODCD4 = 0; } while(0)
00492 #define DLED_SetOpenDrain()      do { ODCONDbits.ODCD4 = 1; } while(0)
00493 #define DLED_SetAnalogMode()     do { ANSELDbits.ANSELD4 = 1; } while(0)
00494 #define DLED_SetDigitalMode()    do { ANSELDbits.ANSELD4 = 0; } while(0)
00495
00496 // get/set PROG_TRACE aliases
00497 #define PROG_TRACE_TRIS          TRISDbits.TRISD5
00498 #define PROG_TRACE_LAT           LATDbits.LATD5
00499 #define PROG_TRACE_PORT          PORTDbits.RD5
00500 #define PROG_TRACE_WPU           WPUDbits.WPUD5
00501 #define PROG_TRACE_OD            ODCONDbits.ODCD5
00502 #define PROG_TRACE_ANS           ANSELDbits.ANSELD5
00503 #define PROG_TRACE_SetHigh()     do { LATDbits.LATD5 = 1; } while(0)
00504 #define PROG_TRACE_SetLow()      do { LATDbits.LATD5 = 0; } while(0)
00505 #define PROG_TRACE_Toggle()      do { LATDbits.LATD5 = ~LATDbits.LATD5; } while(0)
00506 #define PROG_TRACE_GetValue()    PORTDbits.RD5
00507 #define PROG_TRACE_SetDigitalInput() do { TRISDbits.TRISD5 = 1; } while(0)
00508 #define PROG_TRACE_SetDigitalOutput() do { TRISDbits.TRISD5 = 0; } while(0)
00509 #define PROG_TRACE_SetPullup()   do { WPUDbits.WPUD5 = 1; } while(0)
00510 #define PROG_TRACE_ResetPullup() do { WPUDbits.WPUD5 = 0; } while(0)
00511 #define PROG_TRACE_SetPushPull() do { ODCONDbits.ODCD5 = 0; } while(0)
00512 #define PROG_TRACE_SetOpenDrain() do { ODCONDbits.ODCD5 = 1; } while(0)
00513 #define PROG_TRACE_SetAnalogMode() do { ANSELDbits.ANSELD5 = 1; } while(0)
00514 #define PROG_TRACE_SetDigitalMode() do { ANSELDbits.ANSELD5 = 0; } while(0)
00515
00516 // get/set RD6 procedures
00517 #define RD6_SetHigh()            do { LATDbits.LATD6 = 1; } while(0)
00518 #define RD6_SetLow()             do { LATDbits.LATD6 = 0; } while(0)
00519 #define RD6_Toggle()             do { LATDbits.LATD6 = ~LATDbits.LATD6; } while(0)
00520 #define RD6_GetValue()           PORTDbits.RD6
00521 #define RD6_SetDigitalInput()    do { TRISDbits.TRISD6 = 1; } while(0)
00522 #define RD6_SetDigitalOutput()   do { TRISDbits.TRISD6 = 0; } while(0)
00523 #define RD6_SetPullup()          do { WPUDbits.WPUD6 = 1; } while(0)
00524 #define RD6_ResetPullup()        do { WPUDbits.WPUD6 = 0; } while(0)
00525 #define RD6_SetAnalogMode()      do { ANSELDbits.ANSELD6 = 1; } while(0)
00526 #define RD6_SetDigitalMode()     do { ANSELDbits.ANSELD6 = 0; } while(0)
00527
00528 // get/set IO_RD7 aliases
00529 #define IO_RD7_TRIS              TRISDbits.TRISD7
00530 #define IO_RD7_LAT               LATDbits.LATD7
00531 #define IO_RD7_PORT              PORTDbits.RD7
00532 #define IO_RD7_WPU               WPUDbits.WPUD7
00533 #define IO_RD7_OD                ODCONDbits.ODCD7
00534 #define IO_RD7_ANS               ANSELDbits.ANSELD7
00535 #define IO_RD7_SetHigh()         do { LATDbits.LATD7 = 1; } while(0)
00536 #define IO_RD7_SetLow()          do { LATDbits.LATD7 = 0; } while(0)
00537 #define IO_RD7_Toggle()          do { LATDbits.LATD7 = ~LATDbits.LATD7; } while(0)
00538 #define IO_RD7_GetValue()        PORTDbits.RD7
00539 #define IO_RD7_SetDigitalInput() do { TRISDbits.TRISD7 = 1; } while(0)

```



```

00540 #define IO_RD7_SetDigitalOutput() do { TRISDbits.TRISD7 = 0; } while(0)
00541 #define IO_RD7_SetPullup() do { WPUDbits.WPUD7 = 1; } while(0)
00542 #define IO_RD7_ResetPullup() do { WPUDbits.WPUD7 = 0; } while(0)
00543 #define IO_RD7_SetPushPull() do { ODCONDbits.ODCD7 = 0; } while(0)
00544 #define IO_RD7_SetOpenDrain() do { ODCONDbits.ODCD7 = 1; } while(0)
00545 #define IO_RD7_SetAnalogMode() do { ANSELDbits.ANSELD7 = 1; } while(0)
00546 #define IO_RD7_SetDigitalMode() do { ANSELDbits.ANSELD7 = 0; } while(0)
00547
00548 // get/set RELAY aliases
00549 #define RELAY_TRIS TRISEbits.TRISE0
00550 #define RELAY_LAT LATEbits.LATE0
00551 #define RELAY_PORT PORTEbits.RE0
00552 #define RELAY_WPU WPUEbits.WPUE0
00553 #define RELAY_OD ODCONEbits.ODCE0
00554 #define RELAY_ANS ANSELEbits.ANSELE0
00555 #define RELAY_SetHigh() do { LATEbits.LATE0 = 1; } while(0)
00556 #define RELAY_SetLow() do { LATEbits.LATE0 = 0; } while(0)
00557 #define RELAY_Toggle() do { LATEbits.LATE0 = ~LATEbits.LATE0; } while(0)
00558 #define RELAY_GetValue() PORTEbits.RE0
00559 #define RELAY_SetDigitalInput() do { TRISEbits.TRISE0 = 1; } while(0)
00560 #define RELAY_SetDigitalOutput() do { TRISEbits.TRISE0 = 0; } while(0)
00561 #define RELAY_SetPullup() do { WPUEbits.WPUE0 = 1; } while(0)
00562 #define RELAY_ResetPullup() do { WPUEbits.WPUE0 = 0; } while(0)
00563 #define RELAY_SetPushPull() do { ODCONEbits.ODCE0 = 0; } while(0)
00564 #define RELAY_SetOpenDrain() do { ODCONEbits.ODCE0 = 1; } while(0)
00565 #define RELAY_SetAnalogMode() do { ANSELEbits.ANSELE0 = 1; } while(0)
00566 #define RELAY_SetDigitalMode() do { ANSELEbits.ANSELE0 = 0; } while(0)
00567
00568 // get/set MISC aliases
00569 #define MISC_TRIS TRISEbits.TRISE1
00570 #define MISC_LAT LATEbits.LATE1
00571 #define MISC_PORT PORTEbits.RE1
00572 #define MISC_WPU WPUEbits.WPUE1
00573 #define MISC_OD ODCONEbits.ODCE1
00574 #define MISC_ANS ANSELEbits.ANSELE1
00575 #define MISC_SetHigh() do { LATEbits.LATE1 = 1; } while(0)
00576 #define MISC_SetLow() do { LATEbits.LATE1 = 0; } while(0)
00577 #define MISC_Toggle() do { LATEbits.LATE1 = ~LATEbits.LATE1; } while(0)
00578 #define MISC_GetValue() PORTEbits.RE1
00579 #define MISC_SetDigitalInput() do { TRISEbits.TRISE1 = 1; } while(0)
00580 #define MISC_SetDigitalOutput() do { TRISEbits.TRISE1 = 0; } while(0)
00581 #define MISC_SetPullup() do { WPUEbits.WPUE1 = 1; } while(0)
00582 #define MISC_ResetPullup() do { WPUEbits.WPUE1 = 0; } while(0)
00583 #define MISC_SetPushPull() do { ODCONEbits.ODCE1 = 0; } while(0)
00584 #define MISC_SetOpenDrain() do { ODCONEbits.ODCE1 = 1; } while(0)
00585 #define MISC_SetAnalogMode() do { ANSELEbits.ANSELE1 = 1; } while(0)
00586 #define MISC_SetDigitalMode() do { ANSELEbits.ANSELE1 = 0; } while(0)
00587
00588 // get/set CHARGER aliases
00589 #define CHARGER_TRIS TRISEbits.TRISE2
00590 #define CHARGER_LAT LATEbits.LATE2
00591 #define CHARGER_PORT PORTEbits.RE2
00592 #define CHARGER_WPU WPUEbits.WPUE2
00593 #define CHARGER_OD ODCONEbits.ODCE2
00594 #define CHARGER_ANS ANSELEbits.ANSELE2
00595 #define CHARGER_SetHigh() do { LATEbits.LATE2 = 1; } while(0)
00596 #define CHARGER_SetLow() do { LATEbits.LATE2 = 0; } while(0)
00597 #define CHARGER_Toggle() do { LATEbits.LATE2 = ~LATEbits.LATE2; } while(0)
00598 #define CHARGER_GetValue() PORTEbits.RE2
00599 #define CHARGER_SetDigitalInput() do { TRISEbits.TRISE2 = 1; } while(0)
00600 #define CHARGER_SetDigitalOutput() do { TRISEbits.TRISE2 = 0; } while(0)
00601 #define CHARGER_SetPullup() do { WPUEbits.WPUE2 = 1; } while(0)
00602 #define CHARGER_ResetPullup() do { WPUEbits.WPUE2 = 0; } while(0)
00603 #define CHARGER_SetPushPull() do { ODCONEbits.ODCE2 = 0; } while(0)
00604 #define CHARGER_SetOpenDrain() do { ODCONEbits.ODCE2 = 1; } while(0)
00605 #define CHARGER_SetAnalogMode() do { ANSELEbits.ANSELE2 = 1; } while(0)
00606 #define CHARGER_SetDigitalMode() do { ANSELEbits.ANSELE2 = 0; } while(0)
00607
00618 void PIN_MANAGER_Initialize (void);
00619
00620
00621
00632 void IOCAF1_ISR(void);
00633
00655 void IOCAF1_SetInterruptHandler(void (* InterruptHandler)(void));
00656
00679 extern void (*IOCAF1_InterruptHandler)(void);
00680
00703 void IOCAF1_DefaultInterruptHandler(void);
00704
00705
00716 void IOCAF2_ISR(void);
00717
00739 void IOCAF2_SetInterruptHandler(void (* InterruptHandler)(void));
00740
00763 extern void (*IOCAF2_InterruptHandler)(void);
00764

```

```

00787 void IOCAF2_DefaultInterruptHandler(void);
00788
00789
00800 void IOCAF5_ISR(void);
00801
00823 void IOCAF5_SetInterruptHandler(void (* InterruptHandler)(void));
00824
00847 extern void (*IOCAF5_InterruptHandler)(void);
00848
00871 void IOCAF5_DefaultInterruptHandler(void);
00872
00873
00874
00875 #endif // PIN_MANAGER_H

```

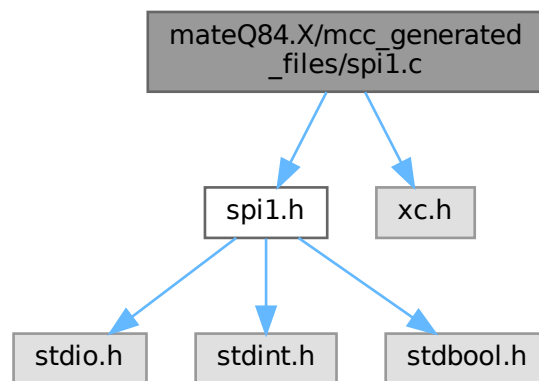
6.105 mateQ84.X/mcc_generated_files/spi1.c File Reference

```

#include "spi1.h"
#include <xc.h>

```

Include dependency graph for spi1.c:



Data Structures

- struct [spi1_configuration_t](#)

Functions

- void [SPI1_Initialize](#) (void)
- bool [SPI1_Open](#) ([spi1_modes_t](#) spi1UniqueConfiguration)
- void [SPI1_Close](#) (void)
- [uint8_t](#) [SPI1_ExchangeByte](#) ([uint8_t](#) data)
- void [SPI1_ExchangeBlock](#) (void *block, [size_t](#) blockSize)
- void [SPI1_WriteBlock](#) (void *block, [size_t](#) blockSize)
- void [SPI1_ReadBlock](#) (void *block, [size_t](#) blockSize)
- void [SPI1_WriteByte](#) ([uint8_t](#) byte)
- [uint8_t](#) [SPI1_ReadByte](#) (void)

6.105.1 Function Documentation

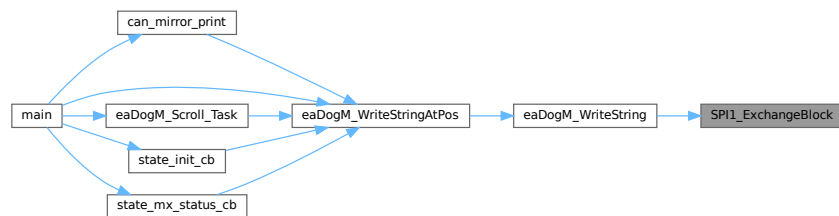
6.105.1.1 SPI1_Close()

```
void SPI1_Close (
    void )
```

6.105.1.2 SPI1_ExchangeBlock()

```
void SPI1_ExchangeBlock (
    void * block,
    size_t blockSize)
```

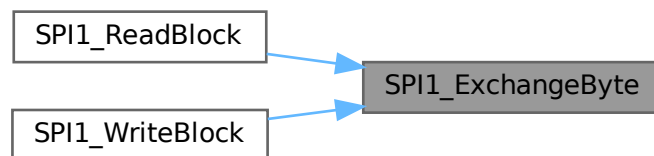
Here is the caller graph for this function:



6.105.1.3 SPI1_ExchangeByte()

```
uint8_t SPI1_ExchangeByte (
    uint8_t data)
```

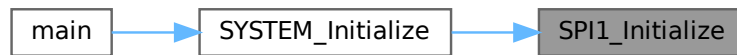
Here is the caller graph for this function:



6.105.1.4 SPI1_Initialize()

```
void SPI1_Initialize (  
    void )
```

Here is the caller graph for this function:



6.105.1.5 SPI1_Open()

```
bool SPI1_Open (  
    spi1_modes_t spi1UniqueConfiguration)
```

6.105.1.6 SPI1_ReadBlock()

```
void SPI1_ReadBlock (  
    void * block,  
    size_t blockSize)
```

Here is the call graph for this function:



6.105.1.7 SPI1_ReadByte()

```
uint8_t SPI1_ReadByte (  
    void )
```

6.105.1.8 SPI1_WriteBlock()

```
void SPI1_WriteBlock (  
    void * block,  
    size_t blockSize)
```

Here is the call graph for this function:

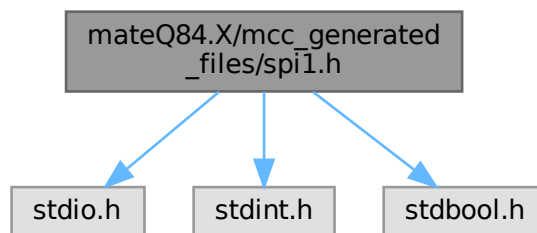


6.105.1.9 SPI1_WriteByte()

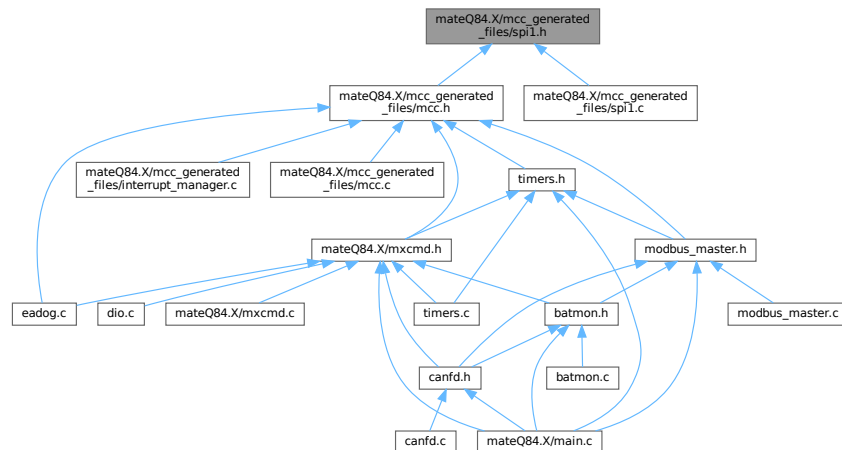
```
void SPI1_WriteByte (  
    uint8_t byte)
```

6.106 mateQ84.X/mcc_generated_files/spi1.h File Reference

```
#include <stdio.h>  
#include <stdint.h>  
#include <stdbool.h>  
Include dependency graph for spi1.h:
```



This graph shows which files directly or indirectly include this file:



Enumerations

- enum [spi1_modes_t](#) { [SPI1_DEFAULT](#) }

Functions

- void [SPI1_Initialize](#) (void)
- bool [SPI1_Open](#) ([spi1_modes_t](#) spi1UniqueConfiguration)
- void [SPI1_Close](#) (void)
- uint8_t [SPI1_ExchangeByte](#) (uint8_t data)
- void [SPI1_ExchangeBlock](#) (void *block, size_t blockSize)
- void [SPI1_WriteBlock](#) (void *block, size_t blockSize)
- void [SPI1_ReadBlock](#) (void *block, size_t blockSize)
- void [SPI1_WriteByte](#) (uint8_t byte)
- uint8_t [SPI1_ReadByte](#) (void)

6.106.1 Enumeration Type Documentation

6.106.1.1 spi1_modes_t

```
enum spi1\_modes\_t
```

SPI1 Generated Driver API Header File

@Company Microchip Technology Inc.

@File Name [spi1.h](#)

@Summary This is the generated header file for the SPI1 driver using PIC10 / PIC12 / PIC16 / PIC18 MCUs

@Description This header file provides APIs for driver for SPI1. Generation Information : Product Revision : PIC10 / PIC12 / PIC16 / PIC18 MCUs - 1.81.8 Device : PIC18F47Q84 Driver Version : 1.0.0 The generated drivers are tested against the following: Compiler : XC8 2.36 and above or later MPLAB : MPLAB X 6.00 Section: Included Files

Enumerator

SPI1_DEFAULT	
--------------	--

6.106.2 Function Documentation

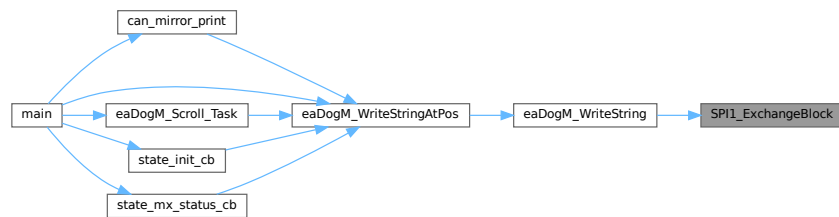
6.106.2.1 SPI1_Close()

```
void SPI1_Close (
    void )
```

6.106.2.2 SPI1_ExchangeBlock()

```
void SPI1_ExchangeBlock (
    void * block,
    size_t blockSize)
```

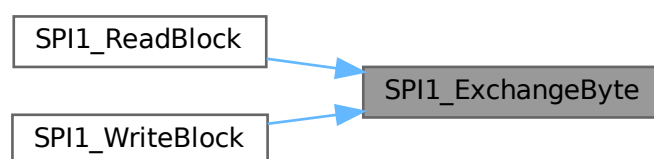
Here is the caller graph for this function:



6.106.2.3 SPI1_ExchangeByte()

```
uint8_t SPI1_ExchangeByte (
    uint8_t data)
```

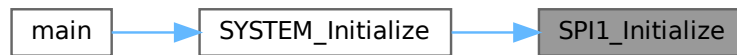
Here is the caller graph for this function:



6.106.2.4 SPI1_Initialize()

```
void SPI1_Initialize (  
    void )
```

Here is the caller graph for this function:



6.106.2.5 SPI1_Open()

```
bool SPI1_Open (  
    spi1_modes_t spi1UniqueConfiguration)
```

6.106.2.6 SPI1_ReadBlock()

```
void SPI1_ReadBlock (  
    void * block,  
    size_t blockSize)
```

Here is the call graph for this function:



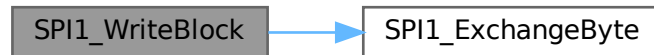
6.106.2.7 SPI1_ReadByte()

```
uint8_t SPI1_ReadByte (  
    void )
```

6.106.2.8 SPI1_WriteBlock()

```
void SPI1_WriteBlock (
    void * block,
    size_t blockSize)
```

Here is the call graph for this function:



6.106.2.9 SPI1_WriteByte()

```
void SPI1_WriteByte (
    uint8_t byte)
```

6.107 spi1.h

[Go to the documentation of this file.](#)

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029     party
00030     license terms applicable to your use of third party software (including open source software) that
00031     may accompany Microchip software.
00032
00033     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00034     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00035     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00036     FOR A PARTICULAR PURPOSE.
00037
00038     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00039     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00040     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00041     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00042     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00043     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00044     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00045     SOFTWARE.
00046 */
00047 #ifndef SPI1_MASTER_H
00048 #define SPI1_MASTER_H
00049
00053
00054 #include <stdio.h>
00055 #include <stdint.h>
00056 #include <stdbool.h>
00057
00058 /* SPI interfaces */
00059 typedef enum {
00060     SPI1_DEFAULT
00061 } spi1_modes_t;
```

```

00062
00063 void SPI1_Initialize(void);
00064 bool SPI1_Open(spi1_modes_t spi1UniqueConfiguration);
00065 void SPI1_Close(void);
00066 uint8_t SPI1_ExchangeByte(uint8_t data);
00067 void SPI1_ExchangeBlock(void *block, size_t blockSize);
00068 void SPI1_WriteBlock(void *block, size_t blockSize);
00069 void SPI1_ReadBlock(void *block, size_t blockSize);
00070 void SPI1_WriteByte(uint8_t byte);
00071 uint8_t SPI1_ReadByte(void);
00072
00073 #endif //SPI1_H

```

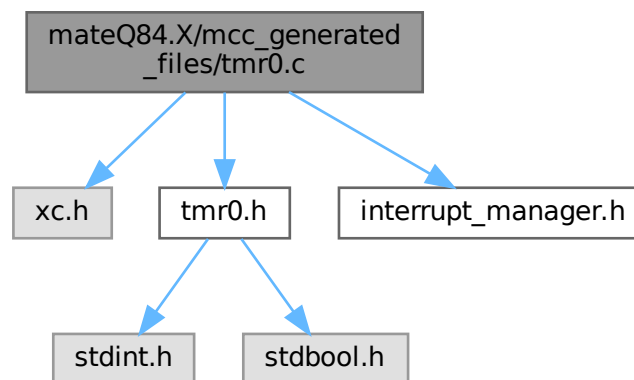
6.108 mateQ84.X/mcc_generated_files/tmr0.c File Reference

```

#include <xc.h>
#include "tmr0.h"
#include "interrupt_manager.h"

```

Include dependency graph for tmr0.c:



Functions

- void `TMR0_Initialize` (void)
- void `TMR0_StartTimer` (void)
- void `TMR0_StopTimer` (void)
- uint8_t `TMR0_ReadTimer` (void)
- void `TMR0_WriteTimer` (uint8_t timerVal)
- void `TMR0_Reload` (uint8_t periodVal)
- void `__interrupt` (irq(TMR0), base(8), low_priority)
- void `TMR0_SetInterruptHandler` (void(*InterruptHandler)(void))
- void `TMR0_DefaultInterruptHandler` (void)

Variables

- void(* `TMR0_InterruptHandler`)(void)

6.108.1 Function Documentation

6.108.1.1 `__interrupt()`

```
void __interrupt (
    irq(TMR0) ,
    base(8) ,
    low_priority )
```

6.108.1.2 `TMR0_DefaultInterruptHandler()`

```
void TMR0_DefaultInterruptHandler (
    void )
```

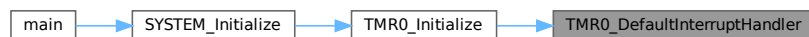
@Summary Default Timer Interrupt Handler

@Description This is the default Interrupt Handler function

@Preconditions Initialize the TMR0 module with interrupt before calling this isr.

@Param None

@Returns None Here is the caller graph for this function:



6.108.1.3 `TMR0_Initialize()`

```
void TMR0_Initialize (
    void )
```

TMR0 Generated Driver API Header File

@Company Microchip Technology Inc.

@File Name [tmr0.h](#)

@Summary This is the generated header file for the TMR0 driver using PIC10 / PIC12 / PIC16 / PIC18 MCUs

@Description This header file provides APIs for TMR0. Generation Information : Product Revision : PIC10 / PIC12 / PIC16 / PIC18 MCUs - 1.81.8 Device : PIC18F47Q84 Driver Version : 3.10 The generated drivers are tested against the following: Compiler : XC8 2.36 and above MPLAB : MPLAB X 6.00 Section: Included Files Section: TMR0 APIs @Summary Initializes the TMR0.

@Description This function initializes the TMR0 Registers. This function must be called before any other TMR0 function is called.

@Preconditions None

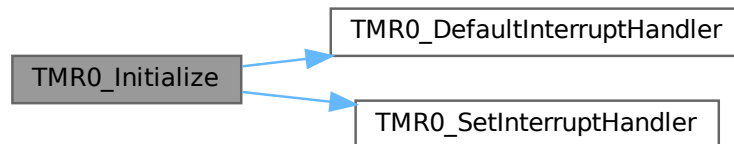
@Param None

@Returns None

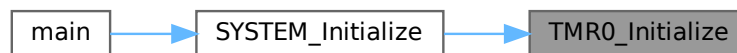
@Comment

@Example `main()` { Initialize TMR0 module `TMR0_Initialize()`;

Do something else... } Here is the call graph for this function:



Here is the caller graph for this function:



6.108.1.4 TMR0_ReadTimer()

```

uint8_t TMR0_ReadTimer (
    void )

```

@Summary Reads the 8 bits TMR0 register value.

@Description This function reads the 8 bits TMR0 register value and return it.

@Preconditions Initialize the TMR0 before calling this function.

@Param None

@Returns This function returns the 8 bits value of TMR0 register.

@Example Initialize TMR0 module

Start TMR0 `TMR0_StartTimer()`;

Read the current value of TMR0 if(0 == `TMR0_ReadTimer()`) { Do something else...

Stop TMR0; `TMR0_StopTimer()`; }

6.108.1.5 TMR0_Reload()

```
void TMR0_Reload (
    uint8_t periodVal)
```

@Summary Load value to Period Register.

@Description This function writes the value to TMR0H register. This function must be called after the initialization of TMR0.

@Preconditions Initialize the TMR0 before calling this function.

@Param periodVal - Value to load into TMR0 register.

@Returns None

@Example while(1) { if(TMR0IF) { Do something else...

clear the TMR0 interrupt flag TMR0IF = 0;

Change the period value of TMR0 TMR0_Reload(0x80); } }

6.108.1.6 TMR0_SetInterruptHandler()

```
void TMR0_SetInterruptHandler (
    void(* InterruptHandler ) (void))
```

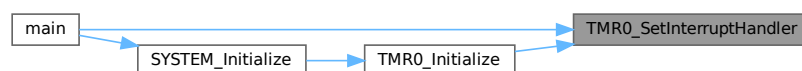
@Summary Set Timer Interrupt Handler

@Description This sets the function to be called during the ISR

@Preconditions Initialize the TMR0 module with interrupt before calling this.

@Param Address of function to be set

@Returns None Here is the caller graph for this function:



6.108.1.7 TMR0_StartTimer()

```
void TMR0_StartTimer (  
    void )
```

@Summary This function starts the TMR0.

@Description This function starts the TMR0 operation. This function must be called after the initialization of TMR0.

@Preconditions Initialize the TMR0 before calling this function.

@Param None

@Returns None

@Example Initialize TMR0 module

```
Start TMR0 TMR0_StartTimer();
```

Do something else... Here is the caller graph for this function:



6.108.1.8 TMR0_StopTimer()

```
void TMR0_StopTimer (  
    void )
```

@Summary This function stops the TMR0.

@Description This function stops the TMR0 operation. This function must be called after the start of TMR0.

@Preconditions Initialize the TMR0 before calling this function.

@Param None

@Returns None

@Example Initialize TMR0 module

```
Start TMR0 TMR0_StartTimer();
```

Do something else...

```
Stop TMR0; TMR0_StopTimer();
```

6.108.1.9 TMR0_WriteTimer()

```
void TMR0_WriteTimer (
    uint8_t timerVal)
```

@Summary Writes the 8 bits value to TMR0 register.

@Description This function writes the 8 bits value to TMR0 register. This function must be called after the initialization of TMR0.

@Preconditions Initialize the TMR0 before calling this function.

@Param timerVal - Value to write into TMR0 register.

@Returns None

@Example #define PERIOD 0x80 #define ZERO 0x00

```
while(1) { Read the TMR0 register if(ZERO == TMR0_ReadTimer()) { Do something
else...
```

```
Write the TMR0 register TMR0_WriteTimer(PERIOD); }
```

```
Do something else... }
```

6.108.2 Variable Documentation

6.108.2.1 TMR0_InterruptHandler

```
void(* TMR0_InterruptHandler) (void) (
    void )
```

TMR0 Generated Driver File

@Company Microchip Technology Inc.

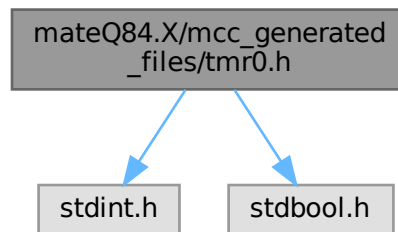
@File Name [tmr0.c](#)

@Summary This is the generated driver implementation file for the TMR0 driver using PIC10 / PIC12 / PIC16 / PIC18 MCUs

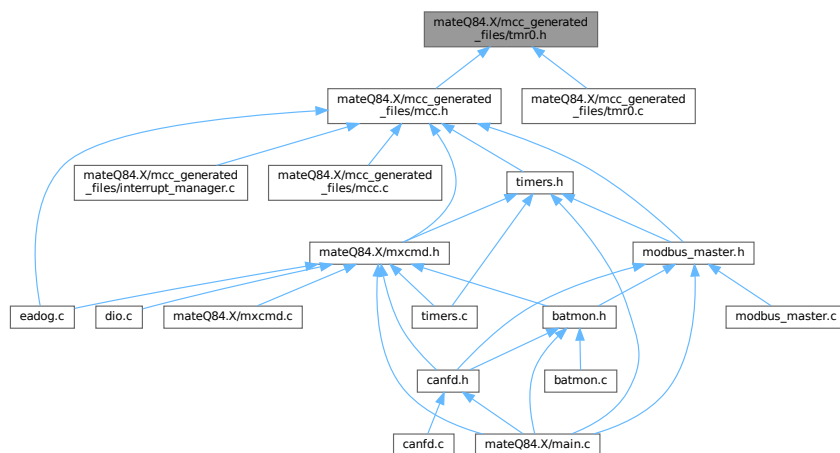
@Description This source file provides APIs for TMR0. Generation Information : Product Revision : PIC10 / PIC12 / PIC16 / PIC18 MCUs - 1.81.8 Device : PIC18F47Q84 Driver Version : 3.10 The generated drivers are tested against the following: Compiler : XC8 2.36 and above MPLAB : MPLAB X 6.00 Section: Included Files Section: TMR0 APIs

6.109 mateQ84.X/mcc_generated_files/tmr0.h File Reference

```
#include <stdint.h>
#include <stdbool.h>
Include dependency graph for tmr0.h:
```



This graph shows which files directly or indirectly include this file:



Functions

- void [TMR0_Initialize](#) (void)
- void [TMR0_StartTimer](#) (void)
- void [TMR0_StopTimer](#) (void)
- uint8_t [TMR0_ReadTimer](#) (void)
- void [TMR0_WriteTimer](#) (uint8_t timerVal)
- void [TMR0_Reload](#) (uint8_t periodVal)
- void [TMR0_SetInterruptHandler](#) (void(*InterruptHandler)(void))
- void [TMR0_DefaultInterruptHandler](#) (void)

Variables

- void(* [TMR0_InterruptHandler](#))(void)

6.109.1 Function Documentation

6.109.1.1 TMR0_DefaultInterruptHandler()

```
void TMR0_DefaultInterruptHandler (
    void )
```

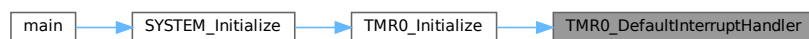
@Summary Default Timer Interrupt Handler

@Description This is the default Interrupt Handler function

@Preconditions Initialize the TMR0 module with interrupt before calling this isr.

@Param None

@Returns None Here is the caller graph for this function:



6.109.1.2 TMR0_Initialize()

```
void TMR0_Initialize (
    void )
```

TMR0 Generated Driver API Header File

@Company Microchip Technology Inc.

@File Name [tmr0.h](#)

@Summary This is the generated header file for the TMR0 driver using PIC10 / PIC12 / PIC16 / PIC18 MCUs

@Description This header file provides APIs for TMR0. Generation Information : Product Revision : PIC10 / PIC12 / PIC16 / PIC18 MCUs - 1.81.8 Device : PIC18F47Q84 Driver Version : 3.10 The generated drivers are tested against the following: Compiler : XC8 2.36 and above MPLAB : MPLAB X 6.00 Section: Included Files Section: TMR0 APIs @Summary Initializes the TMR0.

@Description This function initializes the TMR0 Registers. This function must be called before any other TMR0 function is called.

@Preconditions None

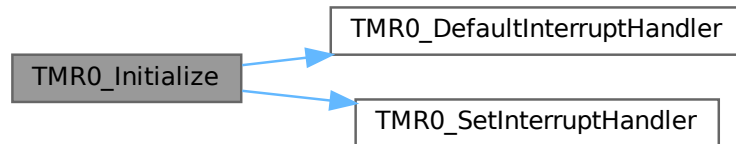
@Param None

@Returns None

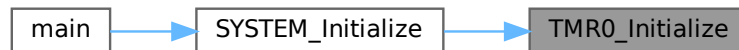
@Comment

@Example `main()` { Initialize TMR0 module `TMR0_Initialize()`;

Do something else... } Here is the call graph for this function:



Here is the caller graph for this function:



6.109.1.3 TMR0_ReadTimer()

```
uint8_t TMR0_ReadTimer (
    void )
```

@Summary Reads the 8 bits TMR0 register value.

@Description This function reads the 8 bits TMR0 register value and return it.

@Preconditions Initialize the TMR0 before calling this function.

@Param None

@Returns This function returns the 8 bits value of TMR0 register.

@Example Initialize TMR0 module

```
Start TMR0 TMR0_StartTimer();
```

```
Read the current value of TMR0 if(0 == TMR0_ReadTimer()) { Do something else...
```

```
Stop TMR0; TMR0_StopTimer(); }
```

6.109.1.4 TMR0_Reload()

```
void TMR0_Reload (
    uint8_t periodVal)
```

@Summary Load value to Period Register.

@Description This function writes the value to TMR0H register. This function must be called after the initialization of TMR0.

@Preconditions Initialize the TMR0 before calling this function.

@Param periodVal - Value to load into TMR0 register.

@Returns None

@Example while(1) { if(TMR0IF) { Do something else...

clear the TMR0 interrupt flag TMR0IF = 0;

Change the period value of TMR0 TMR0_Reload(0x80); } }

6.109.1.5 TMR0_SetInterruptHandler()

```
void TMR0_SetInterruptHandler (
    void(* InterruptHandler ) (void))
```

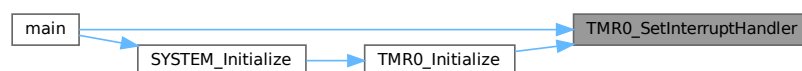
@Summary Set Timer Interrupt Handler

@Description This sets the function to be called during the ISR

@Preconditions Initialize the TMR0 module with interrupt before calling this.

@Param Address of function to be set

@Returns None Here is the caller graph for this function:



6.109.1.6 TMR0_StartTimer()

```
void TMR0_StartTimer (  
    void )
```

@Summary This function starts the TMR0.

@Description This function starts the TMR0 operation. This function must be called after the initialization of TMR0.

@Preconditions Initialize the TMR0 before calling this function.

@Param None

@Returns None

@Example Initialize TMR0 module

```
Start TMR0 TMR0_StartTimer();
```

Do something else... Here is the caller graph for this function:



6.109.1.7 TMR0_StopTimer()

```
void TMR0_StopTimer (  
    void )
```

@Summary This function stops the TMR0.

@Description This function stops the TMR0 operation. This function must be called after the start of TMR0.

@Preconditions Initialize the TMR0 before calling this function.

@Param None

@Returns None

@Example Initialize TMR0 module

```
Start TMR0 TMR0_StartTimer();
```

Do something else...

```
Stop TMR0; TMR0_StopTimer();
```

6.109.1.8 TMR0_WriteTimer()

```
void TMR0_WriteTimer (
    uint8_t timerVal)
```

@Summary Writes the 8 bits value to TMR0 register.

@Description This function writes the 8 bits value to TMR0 register. This function must be called after the initialization of TMR0.

@Preconditions Initialize the TMR0 before calling this function.

@Param timerVal - Value to write into TMR0 register.

@Returns None

@Example #define PERIOD 0x80 #define ZERO 0x00

```
while(1) { Read the TMR0 register if(ZERO == TMR0_ReadTimer()) { Do something
else...
```

```
Write the TMR0 register TMR0_WriteTimer(PERIOD); }
```

```
Do something else... }
```

6.109.2 Variable Documentation

6.109.2.1 TMR0_InterruptHandler

```
void(* TMR0_InterruptHandler) (void) (
    void ) [extern]
```

@Summary Timer Interrupt Handler

@Description This is a function pointer to the function that will be called during the ISR

@Preconditions Initialize the TMR0 module with interrupt before calling this isr.

@Param None

@Returns None

TMR0 Generated Driver File

@Company Microchip Technology Inc.

@File Name [tmr0.c](#)

@Summary This is the generated driver implementation file for the TMR0 driver using PIC10 / PIC12 / PIC16 / PIC18 MCUs

@Description This source file provides APIs for TMR0. Generation Information : Product Revision : PIC10 / PIC12 / PIC16 / PIC18 MCUs - 1.81.8 Device : PIC18F47Q84 Driver Version : 3.10 The generated drivers are tested against the following: Compiler : XC8 2.36 and above MPLAB : MPLAB X 6.00 Section: Included Files Section: TMR0 APIs

6.110 tmr0.h

[Go to the documentation of this file.](#)

```

00001
00023
00024 /*
00025      (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027      Subject to your compliance with these terms, you may use Microchip software and any
00028      derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029      party
00029      license terms applicable to your use of third party software (including open source software) that
00030
00030      may accompany Microchip software.
00031
00032      THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033      EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034      IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035      FOR A PARTICULAR PURPOSE.
00036
00037      IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038      INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039      WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040      HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041      THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042      CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043      OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044      SOFTWARE.
00045 */
00046
00047 #ifndef TMR0_H
00048 #define TMR0_H
00049
00053
00054 #include <stdint.h>
00055 #include <stdbool.h>
00056
00057 #ifdef __cplusplus // Provide C++ Compatibility
00058     extern "C" {
00059
00060
00061 #endif
00062
00063
00064
00068
00100 void TMR0_Initialize(void);
00101
00129 void TMR0_StartTimer(void);
00130
00161 void TMR0_StopTimer(void);
00162
00196 uint8_t TMR0_ReadTimer(void);
00197
00235 void TMR0_WriteTimer(uint8_t timerVal);
00236
00272 void TMR0_Reload(uint8_t periodVal);
00273
00274
00275
00276
00293 void TMR0_SetInterruptHandler(void (* InterruptHandler)(void));
00294
00311 extern void (*TMR0_InterruptHandler)(void);
00312
00329 void TMR0_DefaultInterruptHandler(void);
00330
00331 #ifdef __cplusplus // Provide C++ Compatibility
00332     }
00333
00334
00335 #endif
00336
00337 #endif // TMR0_H

```

6.111 mateQ84.X/mcc_generated_files/tmr2.c File Reference

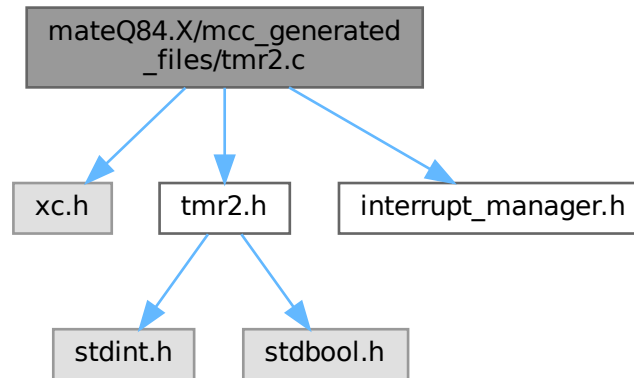
```

#include <xc.h>
#include "tmr2.h"

```

```
#include "interrupt_manager.h"
```

Include dependency graph for tmr2.c:



Functions

- void [TMR2_Initialize](#) (void)
- void [TMR2_ModeSet](#) (TMR2_HLT_MODE mode)
- void [TMR2_ExtResetSourceSet](#) (TMR2_HLT_EXT_RESET_SOURCE reset)
- void [TMR2_Start](#) (void)
- void [TMR2_StartTimer](#) (void)
- void [TMR2_Stop](#) (void)
- void [TMR2_StopTimer](#) (void)
- uint8_t [TMR2_Counter8BitGet](#) (void)
- uint8_t [TMR2_ReadTimer](#) (void)
- void [TMR2_Counter8BitSet](#) (uint8_t timerVal)
- void [TMR2_WriteTimer](#) (uint8_t timerVal)
- void [TMR2_Period8BitSet](#) (uint8_t periodVal)
- void [TMR2_LoadPeriodRegister](#) (uint8_t periodVal)
- void [__interrupt](#) (irq(TMR2), base(8), low_priority)
- void [TMR2_SetInterruptHandler](#) (void(*InterruptHandler)(void))
- void [TMR2_DefaultInterruptHandler](#) (void)

Variables

- void(* [TMR2_InterruptHandler](#))(void)

6.111.1 Function Documentation

6.111.1.1 [__interrupt\(\)](#)

```
void __interrupt (
    irq(TMR2) ,
    base(8) ,
    low_priority )
```

6.111.1.2 TMR2_Counter8BitGet()

```
uint8_t TMR2_Counter8BitGet (
    void )
```

@Summary Reads the TMR2 register.

@Description This function reads the TMR2 register value and return it.

@Preconditions Initialize the TMR2 before calling this function.

@Param None

@Returns This function returns the current value of TMR2 register.

@Example Initialize TMR2 module

```
Start TMR2 TMR2_Start();
```

```
Read the current value of TMR2 if(0 == TMR2_Counter8BitGet()) { Do something
else...
```

```
Reload the TMR value TMR2_Period8BitSet(); } Here is the caller graph for this function:
```

**6.111.1.3 TMR2_Counter8BitSet()**

```
void TMR2_Counter8BitSet (
    uint8_t timerVal)
```

@Summary Writes the TMR2 register.

@Description This function writes the TMR2 register. This function must be called after the initialization of TMR2.

@Preconditions Initialize the TMR2 before calling this function.

@Param timerVal - Value to write into TMR2 register.

@Returns None

@Example #define PERIOD 0x80 #define ZERO 0x00

```
while(1) { Read the TMR2 register if(ZERO == TMR2_Counter8BitGet()) { Do
something else...
```

```
Write the TMR2 register TMR2_Counter8BitSet(PERIOD); }
```

```
Do something else... } Here is the caller graph for this function:
```



6.111.1.4 TMR2_DefaultInterruptHandler()

```
void TMR2_DefaultInterruptHandler (
    void )
```

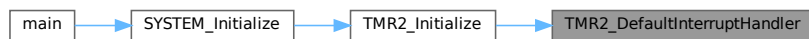
@Summary Default Timer Interrupt Handler

@Description This is the default Interrupt Handler function

@Preconditions Initialize the TMR2 module with interrupt before calling this isr.

@Param None

@Returns None Here is the caller graph for this function:



6.111.1.5 TMR2_ExtResetSourceSet()

```
void TMR2_ExtResetSourceSet (
    TMR2_HLT_EXT_RESET_SOURCE reset)
```

@Summary Configures the HLT external reset source.

@Description Writes the T2RSTbits.RSEL bits.

@Preconditions Initialize the TMR2 before calling this function.

@Param reset - Value to write into T2RSTbits.RSEL bits.

@Returns None

@Example `main()` {

```
TMR2_HLT_EXT_RESET_SOURCE hltresetsrc;
hltresetsrc = T2IN;
```

```
Initialize TMR2 module
```

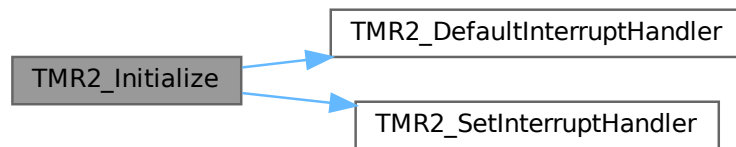
```
Set the HLT mode TMR2_ExtResetSourceSet(hltresetsrc);
```

```
Do something else... }
```

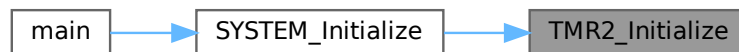
6.111.1.6 TMR2_Initialize()

```
void TMR2_Initialize (
    void )
```

Section: TMR2 APIs Here is the call graph for this function:



Here is the caller graph for this function:



6.111.1.7 TMR2_LoadPeriodRegister()

```
void TMR2_LoadPeriodRegister (
    uint8_t periodVal)
```

@Summary Load value to Period Register.

@Description This function writes the value to PR2 register. This function must be called after the initialization of TMR2.

@Preconditions Initialize the TMR2 before calling this function.

@Param periodVal - Value to load into TMR2 register.

@Returns None

@Example `#define PERIOD1 0x80 #define PERIOD2 0x40 #define ZERO 0x00`

```
while(1) { Read the TMR2 register if(ZERO == TMR2_ReadTimer()) { Do something
else...
```

```
    if(flag)
    {
        flag = 0;
```

```
Load Period 1 value TMR2_LoadPeriodRegister(PERIOD1); } else { flag = 1;

Load Period 2 value TMR2_LoadPeriodRegister(PERIOD2); } }

Do something else... } Here is the call graph for this function:
```



6.111.1.8 TMR2_ModeSet()

```
void TMR2_ModeSet (
    TMR2_HLT_MODE mode)
```

@Summary Configures the Hardware Limit Timer mode.

@Description Writes the T2HLTbits.MODE bits.

@Preconditions Initialize the TMR2 before calling this function.

@Param mode - Value to write into T2HLTbits.MODE bits.

@Returns None

@Example `main()` {

```
TMR2_HLT_MODE hltmode;
hltmode = TMR2_ROP_STARTS_TMRON_EN;
```

```
Initialize TMR2 module TMR2.Initialize();
```

```
Set the HLT mode TMR2_ModeSet (hltmode);
```

```
Do something else... }
```


6.111.1.9 TMR2_Period8BitSet()

```
void TMR2_Period8BitSet (
    uint8_t periodVal)
```

@Summary Load value to Period Register.

@Description This function writes the value to PR2 register. This function must be called after the initialization of TMR2.

@Preconditions Initialize the TMR2 before calling this function.

@Param periodVal - Value to load into TMR2 register.

@Returns None

@Example #define PERIOD1 0x80 #define PERIOD2 0x40 #define ZERO 0x00

```
while(1) { Read the TMR2 register if(ZERO == TMR2_Counter8BitGet()) { Do
something else...
```

```
    if(flag)
    {
        flag = 0;
```

```
Load Period 1 value TMR2_Period8BitSet(PERIOD1); } else { flag = 1;
```

```
Load Period 2 value TMR2_Period8BitSet(PERIOD2); } }
```

```
Do something else... } Here is the caller graph for this function:
```

**6.111.1.10 TMR2_ReadTimer()**

```
uint8_t TMR2_ReadTimer (
    void )
```

@Summary Reads the TMR2 register.

@Description This function reads the TMR2 register value and return it.

@Preconditions Initialize the TMR2 before calling this function.

@Param None

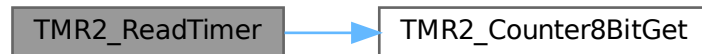
@Returns This function returns the current value of TMR2 register.

@Example Initialize TMR2 module

```
Start TMR2 TMR2_StartTimer();
```

```
Read the current value of TMR2 if(0 == TMR2_ReadTimer()) { Do something
else...
```

```
Reload the TMR value TMR2_LoadPeriodRegister(); } Here is the call graph for this function:
```



6.111.1.11 TMR2_SetInterruptHandler()

```
void TMR2_SetInterruptHandler (
    void(* InterruptHandler ) (void))
```

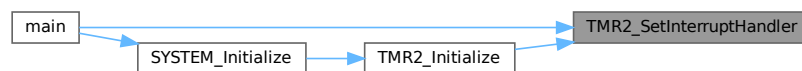
@Summary Set Timer Interrupt Handler

@Description This sets the function to be called during the ISR

@Preconditions Initialize the TMR2 module with interrupt before calling this.

@Param Address of function to be set

@Returns None Here is the caller graph for this function:



6.111.1.12 TMR2_Start()

```
void TMR2_Start (
    void )
```

@Summary This function starts the TMR2.

@Description This function starts the TMR2 operation. This function must be called after the initialization of TMR2.

@Preconditions Initialize the TMR2 before calling this function.

@Param None

@Returns None

@Example Initialize TMR2 module

```
Start TMR2 TMR2_Start();
```

Do something else... Here is the caller graph for this function:



6.111.1.13 TMR2_StartTimer()

```
void TMR2_StartTimer (  
    void )
```

@Summary This function starts the TMR2.

@Description This function starts the TMR2 operation. This function must be called after the initialization of TMR2.

@Preconditions Initialize the TMR2 before calling this function.

@Param None

@Returns None

@Example Initialize TMR2 module

```
Start TMR2 TMR2_StartTimer();
```

Do something else... Here is the call graph for this function:



Here is the caller graph for this function:



6.111.1.14 TMR2_Stop()

```
void TMR2_Stop (
    void )
```

@Summary This function stops the TMR2.

@Description This function stops the TMR2 operation. This function must be called after the start of TMR2.

@Preconditions Initialize the TMR2 before calling this function.

@Param None

@Returns None

@Example Initialize TMR2 module

```
Start TMR2 TMR2_Start();
```

```
Do something else...
```

```
Stop TMR2; TMR2_Stop(); Here is the caller graph for this function:
```

**6.111.1.15 TMR2_StopTimer()**

```
void TMR2_StopTimer (
    void )
```

@Summary This function stops the TMR2.

@Description This function stops the TMR2 operation. This function must be called after the start of TMR2.

@Preconditions Initialize the TMR2 before calling this function.

@Param None

@Returns None

@Example Initialize TMR2 module

```
Start TMR2 TMR2_StartTimer();
```

```
Do something else...
```

```
Stop TMR2; TMR2_StopTimer(); Here is the call graph for this function:
```



6.111.1.16 TMR2_WriteTimer()

```
void TMR2_WriteTimer (
    uint8_t timerVal)
```

@Summary Writes the TMR2 register.

@Description This function writes the TMR2 register. This function must be called after the initialization of TMR2.

@Preconditions Initialize the TMR2 before calling this function.

@Param timerVal - Value to write into TMR2 register.

@Returns None

@Example #define PERIOD 0x80 #define ZERO 0x00

```
while(1) { Read the TMR2 register if(ZERO == TMR2_ReadTimer()) { Do something
else...
```

```
Write the TMR2 register TMR2_WriteTimer(PERIOD); }
```

```
Do something else... } Here is the call graph for this function:
```

**6.111.2 Variable Documentation****6.111.2.1 TMR2_InterruptHandler**

```
void(* TMR2_InterruptHandler) (void) (
    void )
```

TMR2 Generated Driver File

@Company Microchip Technology Inc.

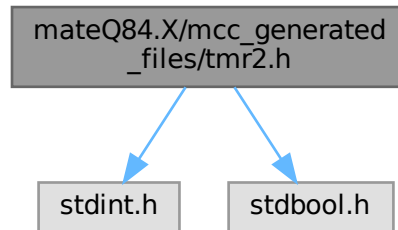
@File Name [tmr2.c](#)

@Summary This is the generated driver implementation file for the TMR2 driver using PIC10 / PIC12 / PIC16 / PIC18 MCUs

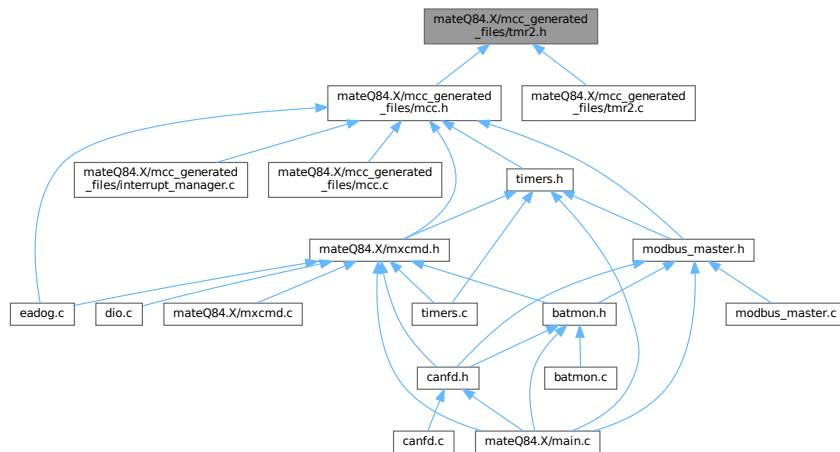
@Description This source file provides APIs for TMR2. Generation Information : Product Revision : PIC10 / PIC12 / PIC16 / PIC18 MCUs - 1.81.8 Device : PIC18F47Q84 Driver Version : 2.11 The generated drivers are tested against the following: Compiler : XC8 2.36 and above MPLAB : MPLAB X 6.00 Section: Included Files Section: Global Variables Definitions

6.112 mateQ84.X/mcc_generated_files/tmr2.h File Reference

```
#include <stdint.h>
#include <stdbool.h>
Include dependency graph for tmr2.h:
```



This graph shows which files directly or indirectly include this file:



Enumerations

- enum `TMR2_HLT_MODE` {
 `TMR2_ROP_STARTS_TMRON`, `TMR2_ROP_STARTS_TMRON_ERSHIGH`, `TMR2_ROP_STARTS_TMRON_ERSLOW`,
 `TMR2_ROP_RESETS_ERBOTHEDGE`,
 `TMR2_ROP_RESETS_ERSRISINGEDGE`, `TMR2_ROP_RESETS_ERSFALLINGEDGE`, `TMR2_ROP_RESETS_ERSLOW`,
 `TMR2_ROP_RESETS_ERSHIGH`,
 `TMR2_OS_STARTS_TMRON`, `TMR2_OS_STARTS_ERSRISINGEDGE`, `TMR2_OS_STARTS_ERSFALLINGEDGE`,
 `TMR2_OS_STARTS_ERBOTHEDGE`,
 `TMR2_OS_STARTS_ERSFIRSTRISINGEDGE`, `TMR2_OS_STARTS_ERSFIRSTFALLINGEDGE`,
 `TMR2_OS_STARTS_ERSRISINGEDGEDETECT`, `TMR2_OS_STARTS_ERSFALLINGEDGEDETECT`,
 `TMR2_OS_STARTS_TMRON_ERSHIGH` = 0x16, `TMR2_OS_STARTS_TMRON_ERSLOW` = 0x17,
 `TMR2_MS_STARTS_TMRON_ERSRISINGEDGEDETECT` = 0x11, `TMR2_MS_STARTS_TMRON_ERSFALLINGEDGEDETECT` = 0x12,
 `TMR2_MS_STARTS_TMRON_ERBOTHEDGE` = 0x13 }

```

• enum TMR2_HLT_EXT_RESET_SOURCE {
    TMR2_T2INPPS , TMR2_RESERVED , TMR2_T4POSTSCALED , TMR2_T6POSTSCALED ,
    TMR2_CCP1_OUT , TMR2_CCP2_OUT , TMR2_CCP3_OUT , TMR2_PWM1S1P1_OUT ,
    TMR2_PWM1S1P2_OUT , TMR2_PWM2S1P1_OUT , TMR2_PWM2S1P2_OUT , TMR2_PWM3S1P1_OUT
    ,
    TMR2_PWM3S1P2_OUT , TMR2_RESERVED_2 , TMR2_RESERVED_3 , TMR2_CMP1_OUT ,
    TMR2_CMP2_OUT , TMR2_ZCD_OUTPUT , TMR2_CLC1_OUT , TMR2_CLC2_OUT ,
    TMR2_CLC3_OUT , TMR2_CLC4_OUT , TMR2_CLC5_OUT , TMR2_CLC6_OUT ,
    TMR2_CLC7_OUT , TMR2_CLC8_OUT , TMR2_UART1_RX_EDGE , TMR2_UART1_TX_EDGE ,
    TMR2_UART2_RX_EDGE , TMR2_UART2_TX_EDGE , TMR2_UART3_RX_EDGE , TMR2_UART3_TX_EDGE
    ,
    TMR2_UART4_RX_EDGE , TMR2_UART4_TX_EDGE , TMR2_UART5_RX_EDGE , TMR2_UART5_TX_EDGE
    ,
    TMR2_RESERVED_4 }

```

Functions

```

• void TMR2_Initialize (void)
• void TMR2_ModeSet (TMR2_HLT_MODE mode)
• void TMR2_ExtResetSourceSet (TMR2_HLT_EXT_RESET_SOURCE reset)
• void TMR2_Start (void)
• void TMR2_StartTimer (void)
• void TMR2_Stop (void)
• void TMR2_StopTimer (void)
• uint8_t TMR2_Counter8BitGet (void)
• uint8_t TMR2_ReadTimer (void)
• void TMR2_Counter8BitSet (uint8_t timerVal)
• void TMR2_WriteTimer (uint8_t timerVal)
• void TMR2_Period8BitSet (uint8_t periodVal)
• void TMR2_LoadPeriodRegister (uint8_t periodVal)
• void TMR2_SetInterruptHandler (void(*InterruptHandler)(void))
• void TMR2_DefaultInterruptHandler (void)

```

Variables

```

• void(* TMR2_InterruptHandler )(void)

```

6.112.1 Enumeration Type Documentation

6.112.1.1 TMR2_HLT_EXT_RESET_SOURCE

```
enum TMR2_HLT_EXT_RESET_SOURCE
```

HLT Reset Source Enumeration

@Summary Defines the different reset source of the HLT.

@Description This source can control starting and stopping of the timer, as well as resetting the timer, depending on which mode the timer is in. The mode of the timer is controlled by the HLT_MODE

Enumerator

TMR2_T2INPPS	
TMR2_RESERVED	
TMR2_T4POSTSCALED	
TMR2_T6POSTSCALED	
TMR2_CCP1_OUT	
TMR2_CCP2_OUT	
TMR2_CCP3_OUT	
TMR2_PWM1S1P1_OUT	
TMR2_PWM1S1P2_OUT	
TMR2_PWM2S1P1_OUT	
TMR2_PWM2S1P2_OUT	
TMR2_PWM3S1P1_OUT	
TMR2_PWM3S1P2_OUT	
TMR2_RESERVED_2	
TMR2_RESERVED_3	
TMR2_CMP1_OUT	
TMR2_CMP2_OUT	
TMR2_ZCD_OUTPUT	
TMR2_CLC1_OUT	
TMR2_CLC2_OUT	
TMR2_CLC3_OUT	
TMR2_CLC4_OUT	
TMR2_CLC5_OUT	
TMR2_CLC6_OUT	
TMR2_CLC7_OUT	
TMR2_CLC8_OUT	
TMR2_UART1_RX_EDGE	
TMR2_UART1_TX_EDGE	
TMR2_UART2_RX_EDGE	
TMR2_UART2_TX_EDGE	
TMR2_UART3_RX_EDGE	
TMR2_UART3_TX_EDGE	
TMR2_UART4_RX_EDGE	
TMR2_UART4_TX_EDGE	
TMR2_UART5_RX_EDGE	
TMR2_UART5_TX_EDGE	
TMR2_RESERVED_4	

6.112.1.2 TMR2_HLT_MODEenum [TMR2_HLT_MODE](#)

TMR2 Generated Driver API Header File

@Company Microchip Technology Inc.

@File Name [tmr2.h](#)

@Summary This is the generated header file for the TMR2 driver using PIC10 / PIC12 / PIC16 / PIC18 MCUs

@Description This header file provides APIs for driver for TMR2. Generation Information : Product Revision : PIC10 / PIC12 / PIC16 / PIC18 MCUs - 1.81.8 Device : PIC18F47Q84 Driver Version : 2.11 The generated drivers are tested against the following: Compiler : XC8 2.36 and above MPLAB : MPLAB X 6.00 Section: Included Files Section: Data Type Definitions HLT Mode Setting Enumeration

@Summary Defines the different modes of the HLT.

@Description This defines the several modes of operation of the Timer with HLT extension. The modes can be set in a control register associated with the timer

Enumerator

TMR2_ROP_STARTS_TMRON	
TMR2_ROP_STARTS_TMRON_ERSHIGH	
TMR2_ROP_STARTS_TMRON_ERSLOW	
TMR2_ROP_RESETS_ERSBOTHEDGE	
TMR2_ROP_RESETS_ERSRISINGEDGE	
TMR2_ROP_RESETS_ERSFALLINGEDGE	
TMR2_ROP_RESETS_ERSLOW	
TMR2_ROP_RESETS_ERSHIGH	
TMR2_OS_STARTS_TMRON	
TMR2_OS_STARTS_ERSRISINGEDGE	
TMR2_OS_STARTS_ERSFALLINGEDGE	
TMR2_OS_STARTS_ERSBOTHEDGE	
TMR2_OS_STARTS_ERSFIRSTRISINGEDGE	
TMR2_OS_STARTS_ERSFIRSTFALLINGEDGE	
TMR2_OS_STARTS_ERSRISINGEDGEDETECT	
TMR2_OS_STARTS_ERSFALLINGEDGEDETECT	
TMR2_OS_STARTS_TMRON_ERSHIGH	
TMR2_OS_STARTS_TMRON_ERSLOW	
TMR2_MS_STARTS_TMRON_ERSRISINGEDGEDETECT	
TMR2_MS_STARTS_TMRON_ERSFALLINGEDGEDETECT	
TMR2_MS_STARTS_TMRON_ERSBOTHEDGE	

6.112.2 Function Documentation

6.112.2.1 TMR2_Counter8BitGet()

```
uint8_t TMR2_Counter8BitGet (
    void )
```

@Summary Reads the TMR2 register.

@Description This function reads the TMR2 register value and return it.

@Preconditions Initialize the TMR2 before calling this function.

@Param None

@Returns This function returns the current value of TMR2 register.

@Example Initialize TMR2 module

```
Start TMR2 TMR2_Start();
```

```
Read the current value of TMR2 if(0 == TMR2_Counter8BitGet()) { Do something else...
```

```
Reload the TMR value TMR2_Period8BitSet(); } Here is the caller graph for this function:
```



6.112.2.2 TMR2_Counter8BitSet()

```
void TMR2_Counter8BitSet (
    uint8_t timerVal)
```

@Summary Writes the TMR2 register.

@Description This function writes the TMR2 register. This function must be called after the initialization of TMR2.

@Preconditions Initialize the TMR2 before calling this function.

@Param timerVal - Value to write into TMR2 register.

@Returns None

@Example `#define PERIOD 0x80 #define ZERO 0x00`

```
while(1) { Read the TMR2 register if(ZERO == TMR2_Counter8BitGet()) { Do something else...
```

```
Write the TMR2 register TMR2_Counter8BitSet(PERIOD); }
```

```
Do something else... } Here is the caller graph for this function:
```



6.112.2.3 TMR2_DefaultInterruptHandler()

```
void TMR2_DefaultInterruptHandler (
    void )
```

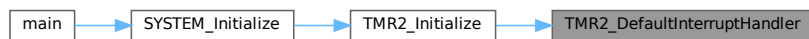
@Summary Default Timer Interrupt Handler

@Description This is the default Interrupt Handler function

@Preconditions Initialize the TMR2 module with interrupt before calling this isr.

@Param None

@Returns None Here is the caller graph for this function:



6.112.2.4 TMR2_ExtResetSourceSet()

```
void TMR2_ExtResetSourceSet (
    TMR2_HLT_EXT_RESET_SOURCE reset)
```

@Summary Configures the HLT external reset source.

@Description Writes the T2RSTbits.RSEL bits.

@Preconditions Initialize the TMR2 before calling this function.

@Param reset - Value to write into T2RSTbits.RSEL bits.

@Returns None

@Example `main()` {

```
TMR2_HLT_EXT_RESET_SOURCE hltresetsrc;
hltresetsrc = T2IN;
```

```
Initialize TMR2 module
```

```
Set the HLT mode TMR2_ExtResetSourceSet(hltresetsrc);
```

```
Do something else... }
```

6.112.2.5 TMR2_Initialize()

```
void TMR2_Initialize (  
    void )
```

Section: Macro Declarations Section: TMR2 APIs @Summary Initializes the TMR2 module.

@Description This function initializes the TMR2 Registers. This function must be called before any other TMR2 function is called.

@Preconditions None

@Param None

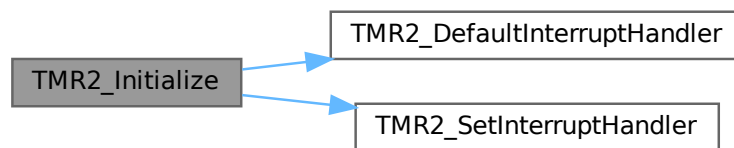
@Returns None

@Comment

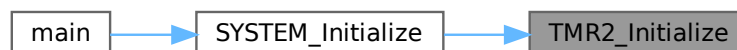
@Example `main()` { Initialize TMR2 module `TMR2_Initialize()`;

Do something else... }

Section: TMR2 APIs Here is the call graph for this function:



Here is the caller graph for this function:



6.112.2.6 TMR2_LoadPeriodRegister()

```
void TMR2_LoadPeriodRegister (
    uint8_t periodVal)
```

@Summary Load value to Period Register.

@Description This function writes the value to PR2 register. This function must be called after the initialization of TMR2.

@Preconditions Initialize the TMR2 before calling this function.

@Param periodVal - Value to load into TMR2 register.

@Returns None

@Example #define PERIOD1 0x80 #define PERIOD2 0x40 #define ZERO 0x00

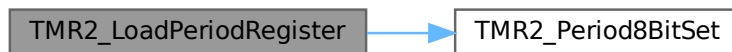
```
while(1) { Read the TMR2 register if(ZERO == TMR2_ReadTimer()) { Do something
else...
```

```
    if(flag)
    {
        flag = 0;
```

```
Load Period 1 value TMR2_LoadPeriodRegister(PERIOD1); } else { flag = 1;
```

```
Load Period 2 value TMR2_LoadPeriodRegister(PERIOD2); } }
```

```
Do something else... } Here is the call graph for this function:
```

**6.112.2.7 TMR2_ModeSet()**

```
void TMR2_ModeSet (
    TMR2_HLT_MODE mode)
```

@Summary Configures the Hardware Limit Timer mode.

@Description Writes the T2HLTbits.MODE bits.

@Preconditions Initialize the TMR2 before calling this function.

@Param mode - Value to write into T2HLTbits.MODE bits.

@Returns None

@Example main() {

```
TMR2_HLT_MODE hltmode;
hltmode = TMR2_ROP_STARTS_TMRON_EN;
```

```
Initialize TMR2 module TMR2.Initialize();
```

```
Set the HLT mode TMR2_ModeSet (hltmode);
```

```
Do something else... }
```

6.112.2.8 TMR2_Period8BitSet()

```
void TMR2_Period8BitSet (
    uint8_t periodVal)
```

@Summary Load value to Period Register.

@Description This function writes the value to PR2 register. This function must be called after the initialization of TMR2.

@Preconditions Initialize the TMR2 before calling this function.

@Param periodVal - Value to load into TMR2 register.

@Returns None

@Example #define PERIOD1 0x80 #define PERIOD2 0x40 #define ZERO 0x00

```
while(1) { Read the TMR2 register if(ZERO == TMR2_Counter8BitGet()) { Do
something else...
```

```
    if(flag)
    {
        flag = 0;
```

```
Load Period 1 value TMR2_Period8BitSet(PERIOD1); } else { flag = 1;
```

```
Load Period 2 value TMR2_Period8BitSet(PERIOD2); } }
```

```
Do something else... } Here is the caller graph for this function:
```



6.112.2.9 TMR2_ReadTimer()

```
uint8_t TMR2_ReadTimer (
    void )
```

@Summary Reads the TMR2 register.

@Description This function reads the TMR2 register value and return it.

@Preconditions Initialize the TMR2 before calling this function.

@Param None

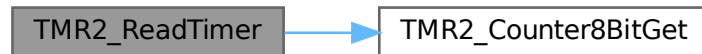
@Returns This function returns the current value of TMR2 register.

@Example Initialize TMR2 module

```
Start TMR2 TMR2_StartTimer();
```

```
Read the current value of TMR2 if(0 == TMR2_ReadTimer()) { Do something
else...
```

```
Reload the TMR value TMR2_LoadPeriodRegister(); } Here is the call graph for this function:
```



6.112.2.10 TMR2_SetInterruptHandler()

```
void TMR2_SetInterruptHandler (
    void(* InterruptHandler ) (void))
```

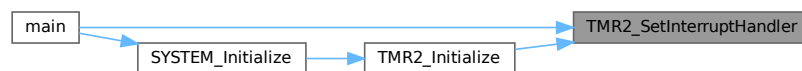
@Summary Set Timer Interrupt Handler

@Description This sets the function to be called during the ISR

@Preconditions Initialize the TMR2 module with interrupt before calling this.

@Param Address of function to be set

@Returns None Here is the caller graph for this function:



6.112.2.11 TMR2_Start()

```
void TMR2_Start (
    void )
```

@Summary This function starts the TMR2.

@Description This function starts the TMR2 operation. This function must be called after the initialization of TMR2.

@Preconditions Initialize the TMR2 before calling this function.

@Param None

@Returns None

@Example Initialize TMR2 module

Start TMR2 `TMR2_Start()`;

Do something else... Here is the caller graph for this function:



6.112.2.12 TMR2_StartTimer()

```
void TMR2_StartTimer (
    void )
```

@Summary This function starts the TMR2.

@Description This function starts the TMR2 operation. This function must be called after the initialization of TMR2.

@Preconditions Initialize the TMR2 before calling this function.

@Param None

@Returns None

@Example Initialize TMR2 module

Start TMR2 `TMR2_StartTimer()`;

Do something else... Here is the call graph for this function:



Here is the caller graph for this function:



6.112.2.13 TMR2_Stop()

```
void TMR2_Stop (  
    void )
```

@Summary This function stops the TMR2.

@Description This function stops the TMR2 operation. This function must be called after the start of TMR2.

@Preconditions Initialize the TMR2 before calling this function.

@Param None

@Returns None

@Example Initialize TMR2 module

```
Start TMR2 TMR2_Start();
```

```
Do something else...
```

```
Stop TMR2; TMR2_Stop(); Here is the caller graph for this function:
```



6.112.2.14 TMR2_StopTimer()

```
void TMR2_StopTimer (  
    void )
```

@Summary This function stops the TMR2.

@Description This function stops the TMR2 operation. This function must be called after the start of TMR2.

@Preconditions Initialize the TMR2 before calling this function.

@Param None

@Returns None

@Example Initialize TMR2 module

```
Start TMR2 TMR2_StartTimer();
```

```
Do something else...
```

```
Stop TMR2; TMR2_StopTimer(); Here is the call graph for this function:
```



6.112.2.15 TMR2_WriteTimer()

```
void TMR2_WriteTimer (
    uint8_t timerVal)
```

@Summary Writes the TMR2 register.

@Description This function writes the TMR2 register. This function must be called after the initialization of TMR2.

@Preconditions Initialize the TMR2 before calling this function.

@Param timerVal - Value to write into TMR2 register.

@Returns None

@Example #define PERIOD 0x80 #define ZERO 0x00

```
while(1) { Read the TMR2 register if(ZERO == TMR2_ReadTimer()) { Do something
else...
```

```
Write the TMR2 register TMR2_WriteTimer(PERIOD); }
```

```
Do something else... } Here is the call graph for this function:
```



6.112.3 Variable Documentation

6.112.3.1 TMR2_InterruptHandler

```
void(* TMR2_InterruptHandler) (void) (
    void ) [extern]
```

@Summary Timer Interrupt Handler

@Description This is a function pointer to the function that will be called during the ISR

@Preconditions Initialize the TMR2 module with interrupt before calling this isr.

@Param None

@Returns None

TMR2 Generated Driver File

@Company Microchip Technology Inc.

@File Name [tmr2.c](#)

@Summary This is the generated driver implementation file for the TMR2 driver using PIC10 / PIC12 / PIC16 / PIC18 MCUs

@Description This source file provides APIs for TMR2. Generation Information : Product Revision : PIC10 / PIC12 / PIC16 / PIC18 MCUs - 1.81.8 Device : PIC18F47Q84 Driver Version : 2.11 The generated drivers are tested against the following: Compiler : XC8 2.36 and above MPLAB : MPLAB X 6.00 Section: Included Files Section: Global Variables Definitions

6.113 tmr2.h

[Go to the documentation of this file.](#)

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029     party
00030     license terms applicable to your use of third party software (including open source software) that
00031     may accompany Microchip software.
00032
00033     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00034     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00035     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00036     FOR A PARTICULAR PURPOSE.
00037
00038     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00039     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00040     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00041     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00042     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00043     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00044     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00045     SOFTWARE.
00046 */
00047 #ifndef TMR2_H
00048 #define TMR2_H
00049
00053
00054 #include <stdint.h>
00055 #include <stdbool.h>
00056
00057 #ifdef __cplusplus // Provide C++ Compatibility
00058     extern "C" {
00059
00060
00061 #endif
00062
00066
00078
00079 typedef enum
00080 {
00081
00082     /* Roll-over Pulse mode clears the TMRx upon TMRx = PRx, then continue running.
00083     ON bit must be set and is not affected by Resets
00084     */
00085
00086     /* Roll-over Pulse mode indicates that Timer starts
00087     immediately upon ON = 1 (Software Control)
00088     */
00089     TMR2_ROP_STARTS_TMRON,
00090
00091     /* Roll-over Pulse mode indicates that the Timer starts
00092     when ON = 1 and TMRx_ers = 1. Stops when TMRx_ers = 0
00093     */
00094     TMR2_ROP_STARTS_TMRON_ERSHIGH,
00095
00096     /* Roll-over Pulse mode indicates that the Timer starts
00097     when ON = 1 and TMRx_ers = 0. Stops when TMRx_ers = 1
00098     */
00099     TMR2_ROP_STARTS_TMRON_ERSLOW,
00100
00101     /* Roll-over Pulse mode indicates that the Timer resets
00102     upon rising or falling edge of TMRx_ers
00103     */
00104     TMR2_ROP_RESETS_ERSBOTHEDGE,
00105
00106     /* Roll-over Pulse mode indicates that the Timer resets
00107     upon rising edge of TMRx_ers
00108     */
00109     TMR2_ROP_RESETS_ERSRISINGEDGE,
00110
00111     /* Roll-over Pulse mode indicates that the Timer resets
00112     upon falling edge of TMRx_ers
00113     */
00114     TMR2_ROP_RESETS_ERSFALLINGEDGE,
00115
00116     /* Roll-over Pulse mode indicates that the Timer resets
00117     upon TMRx_ers = 0
00118     */

```

```

00119     TMR2_ROP_RESETS_ERLOW,
00120
00121     /* Roll-over Pulse mode indicates that the Timer resets
00122     upon TMRx_ers = 1
00123     */
00124     TMR2_ROP_RESETS_ERHIGH,
00125
00126     /*In all One-Shot mode the timer resets and the ON bit is
00127     cleared when the timer value matches the PRx period
00128     value. The ON bit must be set by software to start
00129     another timer cycle.
00130     */
00131
00132     /* One shot mode indicates that the Timer starts
00133     immediately upon ON = 1 (Software Control)
00134     */
00135     TMR2_OS_STARTS_TMRON,
00136
00137     /* One shot mode indicates that the Timer starts
00138     when a rising edge is detected on the TMRx_ers
00139     */
00140     TMR2_OS_STARTS_ERSRISINGEDGE ,
00141
00142     /* One shot mode indicates that the Timer starts
00143     when a falling edge is detected on the TMRx_ers
00144     */
00145     TMR2_OS_STARTS_ERFALLINGEDGE ,
00146
00147     /* One shot mode indicates that the Timer starts
00148     when either a rising or falling edge is detected on TMRx_ers
00149     */
00150     TMR2_OS_STARTS_ERSBOTHEDGE,
00151
00152     /* One shot mode indicates that the Timer starts
00153     upon first TMRx_ers rising edge and resets on all
00154     subsequent TMRx_ers rising edges
00155     */
00156     TMR2_OS_STARTS_ERFIRSTRISINGEDGE,
00157
00158     /* One shot mode indicates that the Timer starts
00159     upon first TMRx_ers falling edge and restarts on all
00160     subsequent TMRx_ers falling edges
00161     */
00162     TMR2_OS_STARTS_ERFIRSTFALLINGEDGE,
00163
00164     /* One shot mode indicates that the Timer starts
00165     when a rising edge is detected on the TMRx_ers,
00166     resets upon TMRx_ers = 0
00167     */
00168     TMR2_OS_STARTS_ERSRISINGEDGEDETECT,
00169     /* One shot mode indicates that the Timer starts
00170     when a falling edge is detected on the TMRx_ers,
00171     resets upon TMRx_ers = 1
00172     */
00173     TMR2_OS_STARTS_ERFALLINGEDGEDETECT,
00174
00175     /* One shot mode indicates that the Timer starts
00176     when a TMRx_ers = 1,ON =1 and resets upon TMRx_ers =0
00177     */
00178     TMR2_OS_STARTS_TMRON_ERHIGH = 0x16,
00179
00180     /* One shot mode indicates that the Timer starts
00181     when a TMRx_ers = 0,ON = 1 and resets upon TMRx_ers =1
00182     */
00183     TMR2_OS_STARTS_TMRON_ERLOW = 0x17,
00184
00185     /*In all Mono-Stable mode the ON bit must be initially set,but
00186     not cleared upon the TMRx = PRx, and the timer will start upon
00187     an TMRx_ers start event following TMRx = PRx.*/
00188
00189     /* Mono Stable mode indicates that the Timer starts
00190     when a rising edge is detected on the TMRx_ers and ON = 1
00191     */
00192     TMR2_MS_STARTS_TMRON_ERSRISINGEDGEDETECT = 0x11,
00193
00194     /* Mono Stable mode indicates that the Timer starts
00195     when a falling edge is detected on the TMRx_ers and ON = 1
00196     */
00197     TMR2_MS_STARTS_TMRON_ERFALLINGEDGEDETECT = 0x12,
00198
00199     /* Mono Stable mode indicates that the Timer starts
00200     when either a rising or falling edge is detected on TMRx_ers
00201     and ON = 1
00202     */
00203     TMR2_MS_STARTS_TMRON_ERSBOTHEDGE = 0x13
00204
00205 } TMR2_HLT_MODE;

```

```
00206
00219
00220 typedef enum
00221 {
00222     /* T2INPPS is the Timer external reset source
00223     */
00224     TMR2_T2INPPS,
00225
00226     /* Reserved enum cannot be used
00227     */
00228     TMR2_RESERVED,
00229
00230     /* Timer4 Postscale is the Timer external reset source
00231     */
00232     TMR2_T4POSTSCALED,
00233
00234     /* Timer6 Postscale is the Timer external reset source
00235     */
00236     TMR2_T6POSTSCALED,
00237
00238     /* CCP1_OUT is the Timer external reset source
00239     */
00240     TMR2_CCP1_OUT,
00241
00242     /* CCP2_OUT is the Timer external reset source
00243     */
00244     TMR2_CCP2_OUT,
00245
00246     /* CCP3_OUT is the Timer external reset source
00247     */
00248     TMR2_CCP3_OUT,
00249
00250     /* PWM1S1P1_out is the Timer external reset source
00251     */
00252     TMR2_PWM1S1P1_OUT,
00253
00254     /* PWM1S1P2_out is the Timer external reset source
00255     */
00256     TMR2_PWM1S1P2_OUT,
00257
00258     /* PWM2S1P1_out is the Timer external reset source
00259     */
00260     TMR2_PWM2S1P1_OUT,
00261
00262     /* PWM2S1P2_out is the Timer external reset source
00263     */
00264     TMR2_PWM2S1P2_OUT,
00265
00266     /* PWM3S1P1_out is the Timer external reset source
00267     */
00268     TMR2_PWM3S1P1_OUT,
00269
00270     /* PWM3S1P2_out is the Timer external reset source
00271     */
00272     TMR2_PWM3S1P2_OUT,
00273
00274     /* Reserved enum cannot be used
00275     */
00276     TMR2_RESERVED_2,
00277
00278     /* Reserved enum cannot be used
00279     */
00280     TMR2_RESERVED_3,
00281
00282     /* CMP1_OUT is the Timer external reset source
00283     */
00284     TMR2_CMP1_OUT,
00285
00286     /* CMP2_OUT is the Timer external reset source
00287     */
00288     TMR2_CMP2_OUT,
00289
00290     /* ZCD_Output is the Timer external reset source
00291     */
00292     TMR2_ZCD_OUTPUT,
00293
00294     /* CLC1_out is the Timer external reset source
00295     */
00296     TMR2_CLC1_OUT,
00297
00298     /* CLC2_out is the Timer external reset source
00299     */
00300     TMR2_CLC2_OUT,
00301
00302     /* CLC3_out is the Timer external reset source
00303     */
00304     TMR2_CLC3_OUT,
```

```
00305
00306     /* CLC4_out is the Timer external reset source
00307     */
00308     TMR2_CLC4_OUT,
00309
00310     /* CLC5_out is the Timer external reset source
00311     */
00312     TMR2_CLC5_OUT,
00313
00314     /* CLC6_out is the Timer external reset source
00315     */
00316     TMR2_CLC6_OUT,
00317
00318     /* CLC7_out is the Timer external reset source
00319     */
00320     TMR2_CLC7_OUT,
00321
00322     /* CLC8_out is the Timer external reset source
00323     */
00324     TMR2_CLC8_OUT,
00325
00326     /* UART1_rx_edge is the Timer external reset source
00327     */
00328     TMR2_UART1_RX_EDGE,
00329
00330     /* UART1_tx_edge is the Timer external reset source
00331     */
00332     TMR2_UART1_TX_EDGE,
00333
00334     /* UART2_rx_edge is the Timer external reset source
00335     */
00336     TMR2_UART2_RX_EDGE,
00337
00338     /* UART2_tx_edge is the Timer external reset source
00339     */
00340     TMR2_UART2_TX_EDGE,
00341
00342     /* UART3_rx_edge is the Timer external reset source
00343     */
00344     TMR2_UART3_RX_EDGE,
00345
00346     /* UART3_tx_edge is the Timer external reset source
00347     */
00348     TMR2_UART3_TX_EDGE,
00349
00350     /* UART4_rx_edge is the Timer external reset source
00351     */
00352     TMR2_UART4_RX_EDGE,
00353
00354     /* UART4_tx_edge is the Timer external reset source
00355     */
00356     TMR2_UART4_TX_EDGE,
00357
00358     /* UART5_rx_edge is the Timer external reset source
00359     */
00360     TMR2_UART5_RX_EDGE,
00361
00362     /* UART5_tx_edge is the Timer external reset source
00363     */
00364     TMR2_UART5_TX_EDGE,
00365
00366     /* Reserved enum cannot be used
00367     */
00368     TMR2_RESERVED_4
00369
00370
00371 } TMR2_HLT_EXT_RESET_SOURCE;
00372
00373
00377
00381
00413 void TMR2_Initialize(void);
00414
00449 void TMR2_ModeSet(TMR2_HLT_MODE mode);
00450
00484 void TMR2_ExtResetSourceSet(TMR2_HLT_EXT_RESET_SOURCE reset);
00485
00513 void TMR2_Start(void);
00514
00542 void TMR2_StartTimer(void);
00543
00574 void TMR2_Stop(void);
00575
00606 void TMR2_StopTimer(void);
00607
00641 uint8_t TMR2_Counter8BitGet(void);
00642
```

```

00676 uint8_t TMR2_ReadTimer(void);
00677
00715 void TMR2_Counter8BitSet(uint8_t timerVal);
00716
00754 void TMR2_WriteTimer(uint8_t timerVal);
00755
00806 void TMR2_Period8BitSet(uint8_t periodVal);
00807
00858 void TMR2_LoadPeriodRegister(uint8_t periodVal);
00859
00860
00877 void TMR2_SetInterruptHandler(void (* InterruptHandler)(void));
00878
00895 extern void (*TMR2_InterruptHandler)(void);
00896
00913 void TMR2_DefaultInterruptHandler(void);
00914
00915
00916 #ifdef __cplusplus // Provide C++ Compatibility
00917     }
00918 }
00919
00920 #endif
00921
00922 #endif // TMR2_H

```

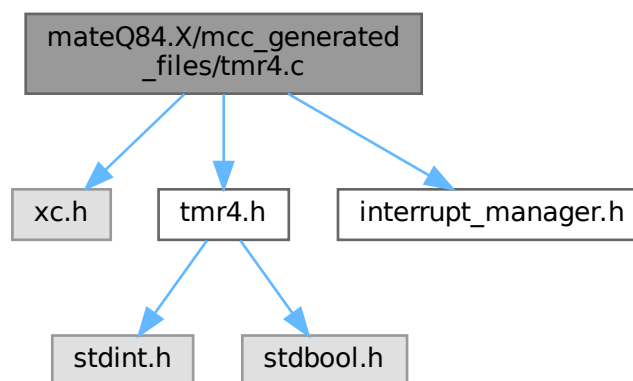
6.114 mateQ84.X/mcc_generated_files/tmr4.c File Reference

```

#include <xc.h>
#include "tmr4.h"
#include "interrupt_manager.h"

```

Include dependency graph for tmr4.c:



Functions

- void `TMR4_Initialize` (void)
- void `TMR4_ModeSet` (TMR4_HLT_MODE mode)
- void `TMR4_ExtResetSourceSet` (TMR4_HLT_EXT_RESET_SOURCE reset)
- void `TMR4_Start` (void)
- void `TMR4_StartTimer` (void)
- void `TMR4_Stop` (void)
- void `TMR4_StopTimer` (void)

- `uint8_t TMR4_Counter8BitGet` (void)
- `uint8_t TMR4_ReadTimer` (void)
- `void TMR4_Counter8BitSet` (uint8_t timerVal)
- `void TMR4_WriteTimer` (uint8_t timerVal)
- `void TMR4_Period8BitSet` (uint8_t periodVal)
- `void TMR4_LoadPeriodRegister` (uint8_t periodVal)
- `void __interrupt` (irq(TMR4), base(8))
- `void TMR4_SetInterruptHandler` (void(*InterruptHandler)(void))
- `void TMR4_DefaultInterruptHandler` (void)

Variables

- `void(* TMR4_InterruptHandler)` (void)

6.114.1 Function Documentation

6.114.1.1 __interrupt()

```
void __interrupt (
    irq(TMR4) ,
    base(8) )
```

6.114.1.2 TMR4_Counter8BitGet()

```
uint8_t TMR4_Counter8BitGet (
    void )
```

@Summary Reads the TMR4 register.

@Description This function reads the TMR4 register value and return it.

@Preconditions Initialize the TMR4 before calling this function.

@Param None

@Returns This function returns the current value of TMR4 register.

@Example Initialize TMR4 module

```
Start TMR4 TMR4_Start();
```

```
Read the current value of TMR4 if(0 == TMR4_Counter8BitGet()) { Do something
else...
```

```
Reload the TMR value TMR4_Period8BitSet(); } Here is the caller graph for this function:
```



6.114.1.3 TMR4_Counter8BitSet()

```
void TMR4_Counter8BitSet (
    uint8_t timerVal)
```

@Summary Writes the TMR4 register.

@Description This function writes the TMR4 register. This function must be called after the initialization of TMR4.

@Preconditions Initialize the TMR4 before calling this function.

@Param timerVal - Value to write into TMR4 register.

@Returns None

@Example #define PERIOD 0x80 #define ZERO 0x00

```
while(1) { Read the TMR4 register if(ZERO == TMR4_Counter8BitGet()) { Do
something else...
```

```
Write the TMR4 register TMR4_Counter8BitSet(PERIOD); }
```

```
Do something else... } Here is the caller graph for this function:
```

**6.114.1.4 TMR4_DefaultInterruptHandler()**

```
void TMR4_DefaultInterruptHandler (
    void )
```

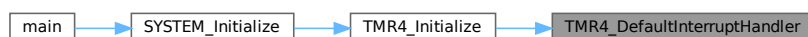
@Summary Default Timer Interrupt Handler

@Description This is the default Interrupt Handler function

@Preconditions Initialize the TMR4 module with interrupt before calling this isr.

@Param None

@Returns None Here is the caller graph for this function:



6.114.1.5 TMR4_ExtResetSourceSet()

```
void TMR4_ExtResetSourceSet (
    TMR4_HLT_EXT_RESET_SOURCE reset)
```

@Summary Configures the HLT external reset source.

@Description Writes the T4RSTbits.RSEL bits.

@Preconditions Initialize the TMR4 before calling this function.

@Param reset - Value to write into T4RSTbits.RSEL bits.

@Returns None

@Example `main()` {

```
TMR4_HLT_EXT_RESET_SOURCE hltresetsrc;
hltresetsrc = T2IN;
```

```
Initialize TMR4 module
```

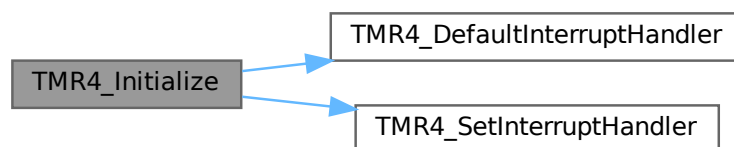
```
Set the HLT mode TMR4_ExtResetSourceSet(hltresetsrc);
```

```
Do something else... }
```

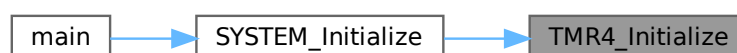
6.114.1.6 TMR4_Initialize()

```
void TMR4_Initialize (
    void )
```

Section: TMR4 APIs Here is the call graph for this function:



Here is the caller graph for this function:



6.114.1.7 TMR4_LoadPeriodRegister()

```
void TMR4_LoadPeriodRegister (
    uint8_t periodVal)
```

@Summary Load value to Period Register.

@Description This function writes the value to PR4 register. This function must be called after the initialization of TMR4.

@Preconditions Initialize the TMR4 before calling this function.

@Param periodVal - Value to load into TMR4 register.

@Returns None

@Example #define PERIOD1 0x80 #define PERIOD2 0x40 #define ZERO 0x00

```
while(1) { Read the TMR4 register if(ZERO == TMR4_ReadTimer()) { Do something
else...
```

```
    if(flag)
    {
        flag = 0;
```

```
Load Period 1 value TMR4_LoadPeriodRegister(PERIOD1); } else { flag = 1;
```

```
Load Period 2 value TMR4_LoadPeriodRegister(PERIOD2); } }
```

```
Do something else... } Here is the call graph for this function:
```

**6.114.1.8 TMR4_ModeSet()**

```
void TMR4_ModeSet (
    TMR4_HLT_MODE mode)
```

@Summary Configures the Hardware Limit Timer mode.

@Description Writes the T4HLTbits.MODE bits.

@Preconditions Initialize the TMR4 before calling this function.

@Param mode - Value to write into T4HLTbits.MODE bits.

@Returns None

@Example main() {

```
TMR4_HLT_MODE hltmode;
hltmode = TMR4_ROP_STARTS_TMRON_EN;
```

```
Initialize TMR4 module TMR4.Initialize();
```

```
Set the HLT mode TMR4_ModeSet (hltmode);
```

```
Do something else... }
```

6.114.1.9 TMR4_Period8BitSet()

```
void TMR4_Period8BitSet (
    uint8_t periodVal)
```

@Summary Load value to Period Register.

@Description This function writes the value to PR4 register. This function must be called after the initialization of TMR4.

@Preconditions Initialize the TMR4 before calling this function.

@Param periodVal - Value to load into TMR4 register.

@Returns None

@Example #define PERIOD1 0x80 #define PERIOD2 0x40 #define ZERO 0x00

```
while(1) { Read the TMR4 register if(ZERO == TMR4_Counter8BitGet()) { Do
something else...
```

```
    if(flag)
    {
        flag = 0;
```

```
Load Period 1 value TMR4_Period8BitSet(PERIOD1); } else { flag = 1;
```

```
Load Period 2 value TMR4_Period8BitSet(PERIOD2); } }
```

```
Do something else... } Here is the caller graph for this function:
```



6.114.1.10 TMR4_ReadTimer()

```
uint8_t TMR4_ReadTimer (
    void )
```

@Summary Reads the TMR4 register.

@Description This function reads the TMR4 register value and return it.

@Preconditions Initialize the TMR4 before calling this function.

@Param None

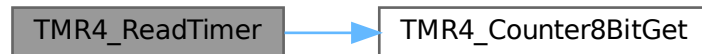
@Returns This function returns the current value of TMR4 register.

@Example Initialize TMR4 module

```
Start TMR4 TMR4_StartTimer();
```

```
Read the current value of TMR4 if(0 == TMR4_ReadTimer()) { Do something
else...
```

```
Reload the TMR value TMR4_LoadPeriodRegister(); } Here is the call graph for this function:
```



6.114.1.11 TMR4_SetInterruptHandler()

```
void TMR4_SetInterruptHandler (
    void(* InterruptHandler ) (void))
```

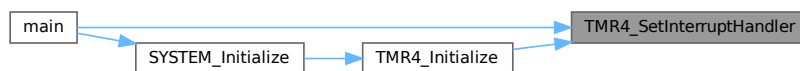
@Summary Set Timer Interrupt Handler

@Description This sets the function to be called during the ISR

@Preconditions Initialize the TMR4 module with interrupt before calling this.

@Param Address of function to be set

@Returns None Here is the caller graph for this function:



6.114.1.12 TMR4_Start()

```
void TMR4_Start (
    void )
```

@Summary This function starts the TMR4.

@Description This function starts the TMR4 operation. This function must be called after the initialization of TMR4.

@Preconditions Initialize the TMR4 before calling this function.

@Param None

@Returns None

@Example Initialize TMR4 module

Start TMR4 `TMR4_Start()`;

Do something else... Here is the caller graph for this function:



6.114.1.13 TMR4_StartTimer()

```
void TMR4_StartTimer (
    void )
```

@Summary This function starts the TMR4.

@Description This function starts the TMR4 operation. This function must be called after the initialization of TMR4.

@Preconditions Initialize the TMR4 before calling this function.

@Param None

@Returns None

@Example Initialize TMR4 module

Start TMR4 `TMR4_StartTimer()`;

Do something else... Here is the call graph for this function:



Here is the caller graph for this function:



6.114.1.14 TMR4_Stop()

```
void TMR4_Stop (  
    void )
```

@Summary This function stops the TMR4.

@Description This function stops the TMR4 operation. This function must be called after the start of TMR4.

@Preconditions Initialize the TMR4 before calling this function.

@Param None

@Returns None

@Example Initialize TMR4 module

```
Start TMR4 TMR4_Start();
```

Do something else...

Stop TMR4; TMR4_Stop(); Here is the caller graph for this function:



6.114.1.15 TMR4_StopTimer()

```
void TMR4_StopTimer (  
    void )
```

@Summary This function stops the TMR4.

@Description This function stops the TMR4 operation. This function must be called after the start of TMR4.

@Preconditions Initialize the TMR4 before calling this function.

@Param None

@Returns None

@Example Initialize TMR4 module

```
Start TMR4 TMR4_StartTimer();
```

Do something else...

Stop TMR4; TMR4_StopTimer(); Here is the call graph for this function:



6.114.1.16 TMR4_WriteTimer()

```
void TMR4_WriteTimer (
    uint8_t timerVal)
```

@Summary Writes the TMR4 register.

@Description This function writes the TMR4 register. This function must be called after the initialization of TMR4.

@Preconditions Initialize the TMR4 before calling this function.

@Param timerVal - Value to write into TMR4 register.

@Returns None

@Example #define PERIOD 0x80 #define ZERO 0x00

```
while(1) { Read the TMR4 register if(ZERO == TMR4_ReadTimer()) { Do something
else...
```

```
Write the TMR4 register TMR4_WriteTimer(PERIOD); }
```

```
Do something else... } Here is the call graph for this function:
```



6.114.2 Variable Documentation

6.114.2.1 TMR4_InterruptHandler

```
void(* TMR4_InterruptHandler) (void) (
    void )
```

TMR4 Generated Driver File

@Company Microchip Technology Inc.

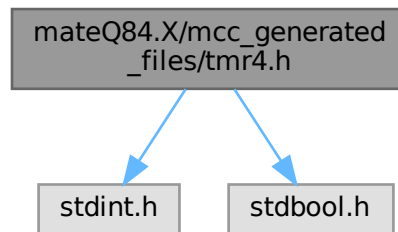
@File Name [tmr4.c](#)

@Summary This is the generated driver implementation file for the TMR4 driver using PIC10 / PIC12 / PIC16 / PIC18 MCUs

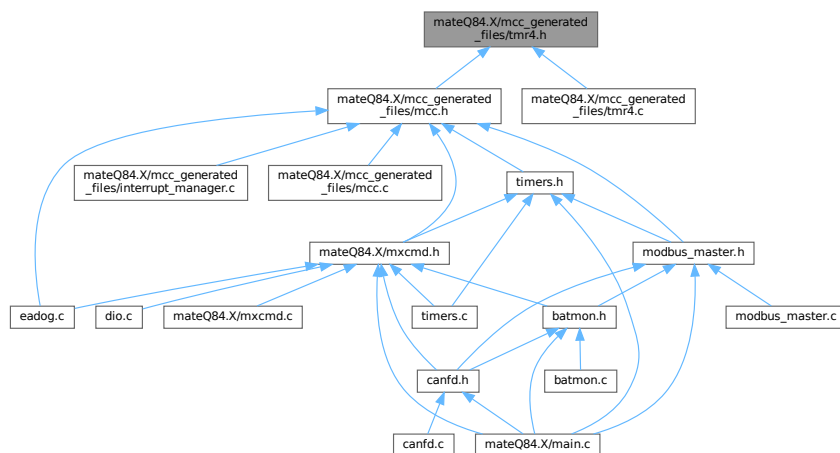
@Description This source file provides APIs for TMR4. Generation Information : Product Revision : PIC10 / PIC12 / PIC16 / PIC18 MCUs - 1.81.8 Device : PIC18F47Q84 Driver Version : 2.11 The generated drivers are tested against the following: Compiler : XC8 2.36 and above MPLAB : MPLAB X 6.00 Section: Included Files Section: Global Variables Definitions

6.115 mateQ84.X/mcc_generated_files/tmr4.h File Reference

```
#include <stdint.h>
#include <stdbool.h>
Include dependency graph for tmr4.h:
```



This graph shows which files directly or indirectly include this file:



Enumerations

- enum `TMR4_HLT_MODE` {
 - `TMR4_ROP_STARTS_TMRON`, `TMR4_ROP_STARTS_TMRON_ERSHIGH`, `TMR4_ROP_STARTS_TMRON_ERSLOW`
 - `TMR4_ROP_RESETS_ERSBOTHEDGE`,
 - `TMR4_ROP_RESETS_ERSRISINGEDGE`, `TMR4_ROP_RESETS_ERSFALLINGEDGE`, `TMR4_ROP_RESETS_ERSLOW`
 - `TMR4_ROP_RESETS_ERSHIGH`,
 - `TMR4_OS_STARTS_TMRON`, `TMR4_OS_STARTS_ERSRISINGEDGE`, `TMR4_OS_STARTS_ERSFALLINGEDGE`
 - `TMR4_OS_STARTS_ERSBOTHEDGE`,
 - `TMR4_OS_STARTS_ERSFIRSTRISINGEDGE`, `TMR4_OS_STARTS_ERSFIRSTFALLINGEDGE`,
 - `TMR4_OS_STARTS_ERSRISINGEDGEDETECT`, `TMR4_OS_STARTS_ERSFALLINGEDGEDETECT`,
 - `TMR4_OS_STARTS_TMRON_ERSHIGH` = 0x16, `TMR4_OS_STARTS_TMRON_ERSLOW` = 0x17,
 - `TMR4_MS_STARTS_TMRON_ERSRISINGEDGEDETECT` = 0x11, `TMR4_MS_STARTS_TMRON_ERSFALLINGEDGEDETECT` = 0x12,
 - `TMR4_MS_STARTS_TMRON_ERSBOTHEDGE` = 0x13 }

```

• enum TMR4_HLT_EXT_RESET_SOURCE {
    TMR4_T4INPPS , TMR4_T2POSTSCALED , TMR4_RESERVED , TMR4_T6POSTSCALED ,
    TMR4_CCP1_OUT , TMR4_CCP2_OUT , TMR4_CCP3_OUT , TMR4_PWM1S1P1_OUT ,
    TMR4_PWM1S1P2_OUT , TMR4_PWM2S1P1_OUT , TMR4_PWM2S1P2_OUT , TMR4_PWM3S1P1_OUT
    ,
    TMR4_PWM3S1P2_OUT , TMR4_RESERVED_2 , TMR4_RESERVED_3 , TMR4_CMP1_OUT ,
    TMR4_CMP2_OUT , TMR4_ZCD_OUTPUT , TMR4_CLC1_OUT , TMR4_CLC2_OUT ,
    TMR4_CLC3_OUT , TMR4_CLC4_OUT , TMR4_CLC5_OUT , TMR4_CLC6_OUT ,
    TMR4_CLC7_OUT , TMR4_CLC8_OUT , TMR4_UART1_RX_EDGE , TMR4_UART1_TX_EDGE ,
    TMR4_UART2_RX_EDGE , TMR4_UART2_TX_EDGE , TMR4_UART3_RX_EDGE , TMR4_UART3_TX_EDGE
    ,
    TMR4_UART4_RX_EDGE , TMR4_UART4_TX_EDGE , TMR4_UART5_RX_EDGE , TMR4_UART5_TX_EDGE
    ,
    TMR4_RESERVED_4 }

```

Functions

- void [TMR4_Initialize](#) (void)
- void [TMR4_ModeSet](#) (TMR4_HLT_MODE mode)
- void [TMR4_ExtResetSourceSet](#) (TMR4_HLT_EXT_RESET_SOURCE reset)
- void [TMR4_Start](#) (void)
- void [TMR4_StartTimer](#) (void)
- void [TMR4_Stop](#) (void)
- void [TMR4_StopTimer](#) (void)
- uint8_t [TMR4_Counter8BitGet](#) (void)
- uint8_t [TMR4_ReadTimer](#) (void)
- void [TMR4_Counter8BitSet](#) (uint8_t timerVal)
- void [TMR4_WriteTimer](#) (uint8_t timerVal)
- void [TMR4_Period8BitSet](#) (uint8_t periodVal)
- void [TMR4_LoadPeriodRegister](#) (uint8_t periodVal)
- void [TMR4_SetInterruptHandler](#) (void(*InterruptHandler)(void))
- void [TMR4_DefaultInterruptHandler](#) (void)

Variables

- void(* [TMR4_InterruptHandler](#))(void)

6.115.1 Enumeration Type Documentation

6.115.1.1 TMR4_HLT_EXT_RESET_SOURCE

```
enum TMR4\_HLT\_EXT\_RESET\_SOURCE
```

HLT Reset Source Enumeration

@Summary Defines the different reset source of the HLT.

@Description This source can control starting and stopping of the timer, as well as resetting the timer, depending on which mode the timer is in. The mode of the timer is controlled by the HLT_MODE

Enumerator

TMR4_T4INPPS	
TMR4_T2POSTSCALED	
TMR4_RESERVED	
TMR4_T6POSTSCALED	
TMR4_CCP1_OUT	
TMR4_CCP2_OUT	
TMR4_CCP3_OUT	
TMR4_PWM1S1P1_OUT	
TMR4_PWM1S1P2_OUT	
TMR4_PWM2S1P1_OUT	
TMR4_PWM2S1P2_OUT	
TMR4_PWM3S1P1_OUT	
TMR4_PWM3S1P2_OUT	
TMR4_RESERVED_2	
TMR4_RESERVED_3	
TMR4_CMP1_OUT	
TMR4_CMP2_OUT	
TMR4_ZCD_OUTPUT	
TMR4_CLC1_OUT	
TMR4_CLC2_OUT	
TMR4_CLC3_OUT	
TMR4_CLC4_OUT	
TMR4_CLC5_OUT	
TMR4_CLC6_OUT	
TMR4_CLC7_OUT	
TMR4_CLC8_OUT	
TMR4_UART1_RX_EDGE	
TMR4_UART1_TX_EDGE	
TMR4_UART2_RX_EDGE	
TMR4_UART2_TX_EDGE	
TMR4_UART3_RX_EDGE	
TMR4_UART3_TX_EDGE	
TMR4_UART4_RX_EDGE	
TMR4_UART4_TX_EDGE	
TMR4_UART5_RX_EDGE	
TMR4_UART5_TX_EDGE	
TMR4_RESERVED_4	

6.115.1.2 TMR4_HLT_MODE

```
enum TMR4_HLT_MODE
```

TMR4 Generated Driver API Header File

@Company Microchip Technology Inc.

@File Name [tmr4.h](#)

@Summary This is the generated header file for the TMR4 driver using PIC10 / PIC12 / PIC16 / PIC18 MCUs

@Description This header file provides APIs for driver for TMR4. Generation Information : Product Revision : PIC10 / PIC12 / PIC16 / PIC18 MCUs - 1.81.8 Device : PIC18F47Q84 Driver Version : 2.11 The generated drivers are tested against the following: Compiler : XC8 2.36 and above MPLAB : MPLAB X 6.00 Section: Included Files Section: Data Type Definitions HLT Mode Setting Enumeration

@Summary Defines the different modes of the HLT.

@Description This defines the several modes of operation of the Timer with HLT extension. The modes can be set in a control register associated with the timer

Enumerator

TMR4_ROP_STARTS_TMRON	
TMR4_ROP_STARTS_TMRON_ERSHIGH	
TMR4_ROP_STARTS_TMRON_ERSLOW	
TMR4_ROP_RESETS_ERSBOTHEDGE	
TMR4_ROP_RESETS_ERSRISINGEDGE	
TMR4_ROP_RESETS_ERSFALLINGEDGE	
TMR4_ROP_RESETS_ERSLOW	
TMR4_ROP_RESETS_ERSHIGH	
TMR4_OS_STARTS_TMRON	
TMR4_OS_STARTS_ERSRISINGEDGE	
TMR4_OS_STARTS_ERSFALLINGEDGE	
TMR4_OS_STARTS_ERSBOTHEDGE	
TMR4_OS_STARTS_ERSFIRSTRISINGEDGE	
TMR4_OS_STARTS_ERSFIRSTFALLINGEDGE	
TMR4_OS_STARTS_ERSRISINGEDGEDETECT	
TMR4_OS_STARTS_ERSFALLINGEDGEDETECT	
TMR4_OS_STARTS_TMRON_ERSHIGH	
TMR4_OS_STARTS_TMRON_ERSLOW	
TMR4_MS_STARTS_TMRON_ERSRISINGEDGEDETECT	
TMR4_MS_STARTS_TMRON_ERSFALLINGEDGEDETECT	
TMR4_MS_STARTS_TMRON_ERSBOTHEDGE	

6.115.2 Function Documentation

6.115.2.1 TMR4_Counter8BitGet()

```
uint8_t TMR4_Counter8BitGet (
    void )
```

@Summary Reads the TMR4 register.

@Description This function reads the TMR4 register value and return it.

@Preconditions Initialize the TMR4 before calling this function.

@Param None

@Returns This function returns the current value of TMR4 register.

@Example Initialize TMR4 module

```
Start TMR4 TMR4_Start();
```

Read the current value of TMR4 if(0 == TMR4_Counter8BitGet()) { Do something else...

Reload the TMR value TMR4_Period8BitSet(); } Here is the caller graph for this function:



6.115.2.2 TMR4_Counter8BitSet()

```
void TMR4_Counter8BitSet (  
    uint8_t timerVal)
```

@Summary Writes the TMR4 register.

@Description This function writes the TMR4 register. This function must be called after the initialization of TMR4.

@Preconditions Initialize the TMR4 before calling this function.

@Param timerVal - Value to write into TMR4 register.

@Returns None

@Example #define PERIOD 0x80 #define ZERO 0x00

```
while(1) { Read the TMR4 register if(ZERO == TMR4_Counter8BitGet()) { Do  
something else...
```

```
Write the TMR4 register TMR4_Counter8BitSet(PERIOD); }
```

```
Do something else... } Here is the caller graph for this function:
```



6.115.2.3 TMR4_DefaultInterruptHandler()

```
void TMR4_DefaultInterruptHandler (
    void )
```

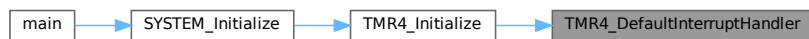
@Summary Default Timer Interrupt Handler

@Description This is the default Interrupt Handler function

@Preconditions Initialize the TMR4 module with interrupt before calling this isr.

@Param None

@Returns None Here is the caller graph for this function:



6.115.2.4 TMR4_ExtResetSourceSet()

```
void TMR4_ExtResetSourceSet (
    TMR4_HLT_EXT_RESET_SOURCE reset)
```

@Summary Configures the HLT external reset source.

@Description Writes the T4RSTbits.RSEL bits.

@Preconditions Initialize the TMR4 before calling this function.

@Param reset - Value to write into T4RSTbits.RSEL bits.

@Returns None

@Example `main()` {

```
TMR4_HLT_EXT_RESET_SOURCE hltresetsrc;
hltresetsrc = T2IN;
```

```
Initialize TMR4 module
```

```
Set the HLT mode TMR4_ExtResetSourceSet(hltresetsrc);
```

```
Do something else... }
```

6.115.2.5 TMR4_Initialize()

```
void TMR4_Initialize (  
    void )
```

Section: Macro Declarations Section: TMR4 APIs @Summary Initializes the TMR4 module.

@Description This function initializes the TMR4 Registers. This function must be called before any other TMR4 function is called.

@Preconditions None

@Param None

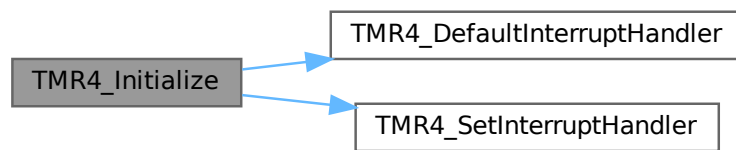
@Returns None

@Comment

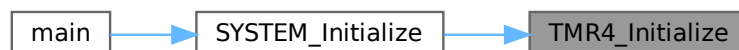
@Example `main()` { Initialize TMR4 module `TMR4_Initialize()`;

Do something else... }

Section: TMR4 APIs Here is the call graph for this function:



Here is the caller graph for this function:



6.115.2.6 TMR4_LoadPeriodRegister()

```
void TMR4_LoadPeriodRegister (
    uint8_t periodVal)
```

@Summary Load value to Period Register.

@Description This function writes the value to PR4 register. This function must be called after the initialization of TMR4.

@Preconditions Initialize the TMR4 before calling this function.

@Param periodVal - Value to load into TMR4 register.

@Returns None

@Example #define PERIOD1 0x80 #define PERIOD2 0x40 #define ZERO 0x00

```
while(1) { Read the TMR4 register if(ZERO == TMR4_ReadTimer()) { Do something
else...
```

```
    if(flag)
    {
        flag = 0;
```

```
Load Period 1 value TMR4_LoadPeriodRegister(PERIOD1); } else { flag = 1;
```

```
Load Period 2 value TMR4_LoadPeriodRegister(PERIOD2); } }
```

```
Do something else... } Here is the call graph for this function:
```



6.115.2.7 TMR4_ModeSet()

```
void TMR4_ModeSet (
    TMR4_HLT_MODE mode)
```

@Summary Configures the Hardware Limit Timer mode.

@Description Writes the T4HLTbits.MODE bits.

@Preconditions Initialize the TMR4 before calling this function.

@Param mode - Value to write into T4HLTbits.MODE bits.

@Returns None

@Example main() {

```
TMR4_HLT_MODE hltmode;
hltmode = TMR4_ROP_STARTS_TMRON_EN;
```

```
Initialize TMR4 module TMR4.Initialize();
```

```
Set the HLT mode TMR4_ModeSet (hltmode);
```

```
Do something else... }
```


6.115.2.8 TMR4_Period8BitSet()

```
void TMR4_Period8BitSet (
    uint8_t periodVal)
```

@Summary Load value to Period Register.

@Description This function writes the value to PR4 register. This function must be called after the initialization of TMR4.

@Preconditions Initialize the TMR4 before calling this function.

@Param periodVal - Value to load into TMR4 register.

@Returns None

@Example #define PERIOD1 0x80 #define PERIOD2 0x40 #define ZERO 0x00

```
while(1) { Read the TMR4 register if(ZERO == TMR4_Counter8BitGet()) { Do
something else...
```

```
    if(flag)
    {
        flag = 0;
```

```
Load Period 1 value TMR4_Period8BitSet(PERIOD1); } else { flag = 1;
```

```
Load Period 2 value TMR4_Period8BitSet(PERIOD2); } }
```

```
Do something else... } Here is the caller graph for this function:
```



6.115.2.9 TMR4_ReadTimer()

```
uint8_t TMR4_ReadTimer (
    void )
```

@Summary Reads the TMR4 register.

@Description This function reads the TMR4 register value and return it.

@Preconditions Initialize the TMR4 before calling this function.

@Param None

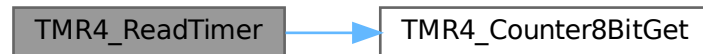
@Returns This function returns the current value of TMR4 register.

@Example Initialize TMR4 module

```
Start TMR4 TMR4_StartTimer();
```

```
Read the current value of TMR4 if(0 == TMR4_ReadTimer()) { Do something
else...
```

```
Reload the TMR value TMR4_LoadPeriodRegister(); } Here is the call graph for this function:
```



6.115.2.10 TMR4_SetInterruptHandler()

```
void TMR4_SetInterruptHandler (
    void(* InterruptHandler ) (void))
```

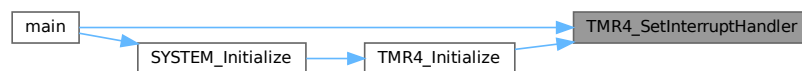
@Summary Set Timer Interrupt Handler

@Description This sets the function to be called during the ISR

@Preconditions Initialize the TMR4 module with interrupt before calling this.

@Param Address of function to be set

@Returns None Here is the caller graph for this function:



6.115.2.11 TMR4_Start()

```
void TMR4_Start (
    void )
```

@Summary This function starts the TMR4.

@Description This function starts the TMR4 operation. This function must be called after the initialization of TMR4.

@Preconditions Initialize the TMR4 before calling this function.

@Param None

@Returns None

@Example Initialize TMR4 module

```
Start TMR4 TMR4_Start();
```

Do something else... Here is the caller graph for this function:



6.115.2.12 TMR4_StartTimer()

```
void TMR4_StartTimer (  
    void )
```

@Summary This function starts the TMR4.

@Description This function starts the TMR4 operation. This function must be called after the initialization of TMR4.

@Preconditions Initialize the TMR4 before calling this function.

@Param None

@Returns None

@Example Initialize TMR4 module

```
Start TMR4 TMR4_StartTimer();
```

Do something else... Here is the call graph for this function:



Here is the caller graph for this function:



6.115.2.13 TMR4_Stop()

```
void TMR4_Stop (
    void )
```

@Summary This function stops the TMR4.

@Description This function stops the TMR4 operation. This function must be called after the start of TMR4.

@Preconditions Initialize the TMR4 before calling this function.

@Param None

@Returns None

@Example Initialize TMR4 module

```
Start TMR4 TMR4_Start();
```

```
Do something else...
```

```
Stop TMR4; TMR4_Stop(); Here is the caller graph for this function:
```

**6.115.2.14 TMR4_StopTimer()**

```
void TMR4_StopTimer (
    void )
```

@Summary This function stops the TMR4.

@Description This function stops the TMR4 operation. This function must be called after the start of TMR4.

@Preconditions Initialize the TMR4 before calling this function.

@Param None

@Returns None

@Example Initialize TMR4 module

```
Start TMR4 TMR4_StartTimer();
```

```
Do something else...
```

```
Stop TMR4; TMR4_StopTimer(); Here is the call graph for this function:
```



6.115.2.15 TMR4_WriteTimer()

```
void TMR4_WriteTimer (
    uint8_t timerVal)
```

@Summary Writes the TMR4 register.

@Description This function writes the TMR4 register. This function must be called after the initialization of TMR4.

@Preconditions Initialize the TMR4 before calling this function.

@Param timerVal - Value to write into TMR4 register.

@Returns None

@Example #define PERIOD 0x80 #define ZERO 0x00

```
while(1) { Read the TMR4 register if(ZERO == TMR4_ReadTimer()) { Do something
else...
```

```
Write the TMR4 register TMR4_WriteTimer(PERIOD); }
```

```
Do something else... } Here is the call graph for this function:
```



6.115.3 Variable Documentation

6.115.3.1 TMR4_InterruptHandler

```
void(* TMR4_InterruptHandler) (void) (
    void ) [extern]
```

@Summary Timer Interrupt Handler

@Description This is a function pointer to the function that will be called during the ISR

@Preconditions Initialize the TMR4 module with interrupt before calling this isr.

@Param None

@Returns None

TMR4 Generated Driver File

@Company Microchip Technology Inc.

@File Name [tmr4.c](#)

@Summary This is the generated driver implementation file for the TMR4 driver using PIC10 / PIC12 / PIC16 / PIC18 MCUs

@Description This source file provides APIs for TMR4. Generation Information : Product Revision : PIC10 / PIC12 / PIC16 / PIC18 MCUs - 1.81.8 Device : PIC18F47Q84 Driver Version : 2.11 The generated drivers are tested against the following: Compiler : XC8 2.36 and above MPLAB : MPLAB X 6.00 Section: Included Files Section: Global Variables Definitions

6.116 tmr4.h

[Go to the documentation of this file.](#)

```

00001
00023
00024 /*
00025      (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027      Subject to your compliance with these terms, you may use Microchip software and any
00028      derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029      party
00030      license terms applicable to your use of third party software (including open source software) that
00031      may accompany Microchip software.
00032
00033      THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00034      EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00035      IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00036      FOR A PARTICULAR PURPOSE.
00037
00038      IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00039      INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00040      WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00041      HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00042      THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00043      CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00044      OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00045      SOFTWARE.
00046 */
00047 #ifndef TMR4_H
00048 #define TMR4_H
00049
00053
00054 #include <stdint.h>
00055 #include <stdbool.h>
00056
00057 #ifdef __cplusplus // Provide C++ Compatibility
00058     extern "C" {
00059
00060
00061 #endif
00062
00066
00078
00079 typedef enum
00080 {
00081
00082     /* Roll-over Pulse mode clears the TMRx upon TMRx = PRx, then continue running.
00083     ON bit must be set and is not affected by Resets
00084     */
00085
00086     /* Roll-over Pulse mode indicates that Timer starts
00087     immediately upon ON = 1 (Software Control)
00088     */
00089     TMR4_ROP_STARTS_TMRON,
00090
00091     /* Roll-over Pulse mode indicates that the Timer starts
00092     when ON = 1 and TMRx_ers = 1. Stops when TMRx_ers = 0
00093     */
00094     TMR4_ROP_STARTS_TMRON_ERSHIGH,
00095
00096     /* Roll-over Pulse mode indicates that the Timer starts
00097     when ON = 1 and TMRx_ers = 0. Stops when TMRx_ers = 1
00098     */
00099     TMR4_ROP_STARTS_TMRON_ERSLOW,
00100
00101     /* Roll-over Pulse mode indicates that the Timer resets
00102     upon rising or falling edge of TMRx_ers
00103     */
00104     TMR4_ROP_RESETS_ERSEBOTHEDGE,
00105
00106     /* Roll-over Pulse mode indicates that the Timer resets
00107     upon rising edge of TMRx_ers
00108     */
00109     TMR4_ROP_RESETS_ERSRISINGEDGE,
00110
00111     /* Roll-over Pulse mode indicates that the Timer resets
00112     upon falling edge of TMRx_ers
00113     */
00114     TMR4_ROP_RESETS_ERSFALLINGEDGE,
00115
00116     /* Roll-over Pulse mode indicates that the Timer resets
00117     upon TMRx_ers = 0
00118     */

```

```

00119     TMR4_ROP_RESETS_ERLOW,
00120
00121     /* Roll-over Pulse mode indicates that the Timer resets
00122     upon TMRx_ers = 1
00123     */
00124     TMR4_ROP_RESETS_ERHIGH,
00125
00126     /*In all One-Shot mode the timer resets and the ON bit is
00127     cleared when the timer value matches the PRx period
00128     value. The ON bit must be set by software to start
00129     another timer cycle.
00130     */
00131
00132     /* One shot mode indicates that the Timer starts
00133     immediately upon ON = 1 (Software Control)
00134     */
00135     TMR4_OS_STARTS_TMRON,
00136
00137     /* One shot mode indicates that the Timer starts
00138     when a rising edge is detected on the TMRx_ers
00139     */
00140     TMR4_OS_STARTS_ERSRISINGEDGE ,
00141
00142     /* One shot mode indicates that the Timer starts
00143     when a falling edge is detected on the TMRx_ers
00144     */
00145     TMR4_OS_STARTS_ERFALLINGEDGE ,
00146
00147     /* One shot mode indicates that the Timer starts
00148     when either a rising or falling edge is detected on TMRx_ers
00149     */
00150     TMR4_OS_STARTS_ERSBOTHEDGE,
00151
00152     /* One shot mode indicates that the Timer starts
00153     upon first TMRx_ers rising edge and resets on all
00154     subsequent TMRx_ers rising edges
00155     */
00156     TMR4_OS_STARTS_ERFIRSTRISINGEDGE,
00157
00158     /* One shot mode indicates that the Timer starts
00159     upon first TMRx_ers falling edge and restarts on all
00160     subsequent TMRx_ers falling edges
00161     */
00162     TMR4_OS_STARTS_ERFIRSTFALLINGEDGE,
00163
00164     /* One shot mode indicates that the Timer starts
00165     when a rising edge is detected on the TMRx_ers,
00166     resets upon TMRx_ers = 0
00167     */
00168     TMR4_OS_STARTS_ERSRISINGEDGEDETECT,
00169     /* One shot mode indicates that the Timer starts
00170     when a falling edge is detected on the TMRx_ers,
00171     resets upon TMRx_ers = 1
00172     */
00173     TMR4_OS_STARTS_ERFALLINGEDGEDETECT,
00174
00175     /* One shot mode indicates that the Timer starts
00176     when a TMRx_ers = 1, ON =1 and resets upon TMRx_ers =0
00177     */
00178     TMR4_OS_STARTS_TMRON_ERHIGH = 0x16,
00179
00180     /* One shot mode indicates that the Timer starts
00181     when a TMRx_ers = 0, ON = 1 and resets upon TMRx_ers =1
00182     */
00183     TMR4_OS_STARTS_TMRON_ERLOW = 0x17,
00184
00185     /*In all Mono-Stable mode the ON bit must be initially set, but
00186     not cleared upon the TMRx = PRx, and the timer will start upon
00187     an TMRx_ers start event following TMRx = PRx.*/
00188
00189     /* Mono Stable mode indicates that the Timer starts
00190     when a rising edge is detected on the TMRx_ers and ON = 1
00191     */
00192     TMR4_MS_STARTS_TMRON_ERSRISINGEDGEDETECT = 0x11,
00193
00194     /* Mono Stable mode indicates that the Timer starts
00195     when a falling edge is detected on the TMRx_ers and ON = 1
00196     */
00197     TMR4_MS_STARTS_TMRON_ERFALLINGEDGEDETECT = 0x12,
00198
00199     /* Mono Stable mode indicates that the Timer starts
00200     when either a rising or falling edge is detected on TMRx_ers
00201     and ON = 1
00202     */
00203     TMR4_MS_STARTS_TMRON_ERSBOTHEDGE = 0x13
00204
00205 } TMR4_HLT_MODE;

```

```
00206
00219
00220 typedef enum
00221 {
00222     /* T4INPPS is the Timer external reset source
00223     */
00224     TMR4_T4INPPS,
00225
00226     /* Timer2 Postscale is the Timer external reset source
00227     */
00228     TMR4_T2POSTSCALED,
00229
00230     /* Reserved enum cannot be used
00231     */
00232     TMR4_RESERVED,
00233
00234     /* Timer6 Postscale is the Timer external reset source
00235     */
00236     TMR4_T6POSTSCALED,
00237
00238     /* CCP1_OUT is the Timer external reset source
00239     */
00240     TMR4_CCP1_OUT,
00241
00242     /* CCP2_OUT is the Timer external reset source
00243     */
00244     TMR4_CCP2_OUT,
00245
00246     /* CCP3_OUT is the Timer external reset source
00247     */
00248     TMR4_CCP3_OUT,
00249
00250     /* PWM1S1P1_out is the Timer external reset source
00251     */
00252     TMR4_PWM1S1P1_OUT,
00253
00254     /* PWM1S1P2_out is the Timer external reset source
00255     */
00256     TMR4_PWM1S1P2_OUT,
00257
00258     /* PWM2S1P1_out is the Timer external reset source
00259     */
00260     TMR4_PWM2S1P1_OUT,
00261
00262     /* PWM2S1P2_out is the Timer external reset source
00263     */
00264     TMR4_PWM2S1P2_OUT,
00265
00266     /* PWM3S1P1_out is the Timer external reset source
00267     */
00268     TMR4_PWM3S1P1_OUT,
00269
00270     /* PWM3S1P2_out is the Timer external reset source
00271     */
00272     TMR4_PWM3S1P2_OUT,
00273
00274     /* Reserved enum cannot be used
00275     */
00276     TMR4_RESERVED_2,
00277
00278     /* Reserved enum cannot be used
00279     */
00280     TMR4_RESERVED_3,
00281
00282     /* CMP1_OUT is the Timer external reset source
00283     */
00284     TMR4_CMP1_OUT,
00285
00286     /* CMP2_OUT is the Timer external reset source
00287     */
00288     TMR4_CMP2_OUT,
00289
00290     /* ZCD_Output is the Timer external reset source
00291     */
00292     TMR4_ZCD_OUTPUT,
00293
00294     /* CLC1_out is the Timer external reset source
00295     */
00296     TMR4_CLC1_OUT,
00297
00298     /* CLC2_out is the Timer external reset source
00299     */
00300     TMR4_CLC2_OUT,
00301
00302     /* CLC3_out is the Timer external reset source
00303     */
00304     TMR4_CLC3_OUT,
```



```
00305
00306     /* CLC4_out is the Timer external reset source
00307     */
00308     TMR4_CLC4_OUT,
00309
00310     /* CLC5_out is the Timer external reset source
00311     */
00312     TMR4_CLC5_OUT,
00313
00314     /* CLC6_out is the Timer external reset source
00315     */
00316     TMR4_CLC6_OUT,
00317
00318     /* CLC7_out is the Timer external reset source
00319     */
00320     TMR4_CLC7_OUT,
00321
00322     /* CLC8_out is the Timer external reset source
00323     */
00324     TMR4_CLC8_OUT,
00325
00326     /* UART1_rx_edge is the Timer external reset source
00327     */
00328     TMR4_UART1_RX_EDGE,
00329
00330     /* UART1_tx_edge is the Timer external reset source
00331     */
00332     TMR4_UART1_TX_EDGE,
00333
00334     /* UART2_rx_edge is the Timer external reset source
00335     */
00336     TMR4_UART2_RX_EDGE,
00337
00338     /* UART2_tx_edge is the Timer external reset source
00339     */
00340     TMR4_UART2_TX_EDGE,
00341
00342     /* UART3_rx_edge is the Timer external reset source
00343     */
00344     TMR4_UART3_RX_EDGE,
00345
00346     /* UART3_tx_edge is the Timer external reset source
00347     */
00348     TMR4_UART3_TX_EDGE,
00349
00350     /* UART4_rx_edge is the Timer external reset source
00351     */
00352     TMR4_UART4_RX_EDGE,
00353
00354     /* UART4_tx_edge is the Timer external reset source
00355     */
00356     TMR4_UART4_TX_EDGE,
00357
00358     /* UART5_rx_edge is the Timer external reset source
00359     */
00360     TMR4_UART5_RX_EDGE,
00361
00362     /* UART5_tx_edge is the Timer external reset source
00363     */
00364     TMR4_UART5_TX_EDGE,
00365
00366     /* Reserved enum cannot be used
00367     */
00368     TMR4_RESERVED_4
00369
00370
00371 } TMR4_HLT_EXT_RESET_SOURCE;
00372
00373
00377
00381
00413 void TMR4_Initialize(void);
00414
00449 void TMR4_ModeSet(TMR4_HLT_MODE mode);
00450
00484 void TMR4_ExtResetSourceSet(TMR4_HLT_EXT_RESET_SOURCE reset);
00485
00513 void TMR4_Start(void);
00514
00542 void TMR4_StartTimer(void);
00543
00574 void TMR4_Stop(void);
00575
00606 void TMR4_StopTimer(void);
00607
00641 uint8_t TMR4_Counter8BitGet(void);
00642
```

```

00676 uint8_t TMR4_ReadTimer(void);
00677
00715 void TMR4_Counter8BitSet(uint8_t timerVal);
00716
00754 void TMR4_WriteTimer(uint8_t timerVal);
00755
00806 void TMR4_Period8BitSet(uint8_t periodVal);
00807
00858 void TMR4_LoadPeriodRegister(uint8_t periodVal);
00859
00860
00877 void TMR4_SetInterruptHandler(void (* InterruptHandler)(void));
00878
00895 extern void (*TMR4_InterruptHandler)(void);
00896
00913 void TMR4_DefaultInterruptHandler(void);
00914
00915
00916 #ifdef __cplusplus // Provide C++ Compatibility
00917     }
00918 }
00919
00920 #endif
00921
00922 #endif // TMR4_H

```

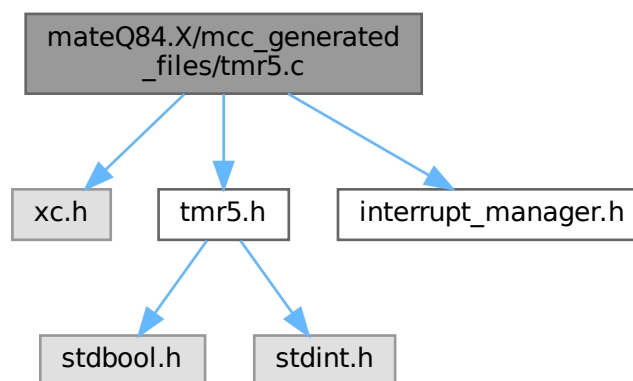
6.117 mateQ84.X/mcc_generated_files/tmr5.c File Reference

```

#include <xc.h>
#include "tmr5.h"
#include "interrupt_manager.h"

```

Include dependency graph for tmr5.c:



Functions

- void `TMR5_Initialize` (void)
- void `TMR5_StartTimer` (void)
- void `TMR5_StopTimer` (void)
- uint16_t `TMR5_ReadTimer` (void)
- void `TMR5_WriteTimer` (uint16_t timerVal)
- void `TMR5_Reload` (void)
- void `TMR5_StartSinglePulseAcquisition` (void)

- uint8_t [TMR5_CheckGateValueStatus](#) (void)
- void [__interrupt](#) (irq(TMR5), base(8), low_priority)
- void [TMR5_SetInterruptHandler](#) (void(*InterruptHandler)(void))
- void [TMR5_DefaultInterruptHandler](#) (void)

Variables

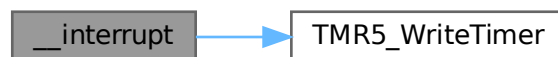
- volatile uint16_t [timer5ReloadVal](#)
- void(* [TMR5_InterruptHandler](#))(void)

6.117.1 Function Documentation

6.117.1.1 [__interrupt\(\)](#)

```
void __interrupt (
    irq(TMR5) ,
    base(8) ,
    low_priority )
```

Here is the call graph for this function:



6.117.1.2 [TMR5_CheckGateValueStatus\(\)](#)

```
uint8_t TMR5_CheckGateValueStatus (
    void )
```

@Summary Check the current state of Timer1 gate.

@Description This function reads the TMR5 gate value and return it. This function must be used when the TMR5 gate is enabled.

@Preconditions Initialize the TMR5 with gate enable before calling this function.

@Param None

@Returns None

@Example uint16_t xVal; uint16_t yVal;

```
enable TMR5 singlepulse mode TMR5_StartSinglePulseAcquistion();
```

```
check TMR5 gate status if(TMR5_CheckGateValueStatus() == 0) xVal = TMR5_ReadTimer();
```

```
wait untill gate interrupt occured while(TMR5IF == 0) { }
```

```
yVal = TMR5_ReadTimer();
```

6.117.1.3 TMR5_DefaultInterruptHandler()

```
void TMR5_DefaultInterruptHandler (
    void )
```

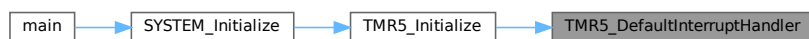
@Summary Default Timer Interrupt Handler

@Description This is the default Interrupt Handler function

@Preconditions Initialize the TMR5 module with interrupt before calling this isr.

@Param None

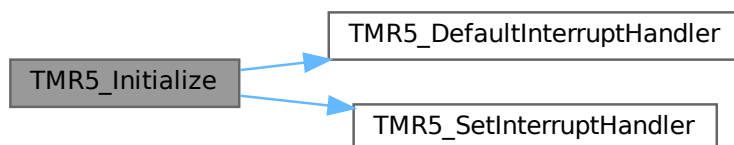
@Returns None Here is the caller graph for this function:



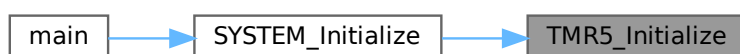
6.117.1.4 TMR5_Initialize()

```
void TMR5_Initialize (
    void )
```

Section: TMR5 APIs Here is the call graph for this function:



Here is the caller graph for this function:



6.117.1.5 TMR5_ReadTimer()

```
uint16_t TMR5_ReadTimer (
    void )
```

@Summary Reads the TMR5 register.

@Description This function reads the TMR5 register value and return it.

@Preconditions Initialize the TMR5 before calling this function.

@Param None

@Returns This function returns the current value of TMR5 register.

@Example Initialize TMR5 module

```
Start TMR5 TMR5_StartTimer();
```

```
Read the current value of TMR5 if(0 == TMR5_ReadTimer()) { Do something
else...
```

```
Reload the TMR value TMR5_Reload(); }
```

6.117.1.6 TMR5_Reload()

```
void TMR5_Reload (
    void )
```

@Summary Reload the TMR5 register.

@Description This function reloads the TMR5 register. This function must be called to write initial value into TMR5 register.

@Preconditions Initialize the TMR5 before calling this function.

@Param None

@Returns None

@Example while(1) { if(TMR5IF) { Do something else...

```
clear the TMR5 interrupt flag TMR5IF = 0;
```

```
Reload the initial value of TMR5 TMR5_Reload(); } } Here is the call graph for this
function:
```



6.117.1.7 TMR5_SetInterruptHandler()

```
void TMR5_SetInterruptHandler (
    void(* InterruptHandler ) (void))
```

@Summary Set Timer Interrupt Handler

@Description This sets the function to be called during the ISR

@Preconditions Initialize the TMR5 module with interrupt before calling this.

@Param Address of function to be set

@Returns None Here is the caller graph for this function:



6.117.1.8 TMR5_StartSinglePulseAcquisition()

```
void TMR5_StartSinglePulseAcquisition (
    void )
```

@Summary Starts the single pulse acquisition in TMR5 gate operation.

@Description This function starts the single pulse acquisition in TMR5 gate operation. This function must be used when the TMR5 gate is enabled.

@Preconditions Initialize the TMR5 with gate enable before calling this function.

@Param None

@Returns None

@Example `uint16_t xVal; uint16_t yVal;`

```
enable TMR5 singlepulse mode TMR5_StartSinglePulseAcquistion();
```

```
check TMR5 gate status if(TMR5_CheckGateValueStatus()== 0) xVal = TMR5_ReadTimer();
```

```
wait untill gate interrupt occured while(TMR5GIF == 0) { }
```

```
yVal = TMR5_ReadTimer();
```

6.117.1.9 TMR5_StartTimer()

```
void TMR5_StartTimer (  
    void )
```

@Summary This function starts the TMR5.

@Description This function starts the TMR5 operation. This function must be called after the initialization of TMR5.

@Preconditions Initialize the TMR5 before calling this function.

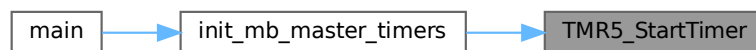
@Param None

@Returns None

@Example Initialize TMR5 module

```
Start TMR5 TMR5_StartTimer();
```

Do something else... Here is the caller graph for this function:



6.117.1.10 TMR5_StopTimer()

```
void TMR5_StopTimer (  
    void )
```

@Summary This function stops the TMR5.

@Description This function stops the TMR5 operation. This function must be called after the start of TMR5.

@Preconditions Initialize the TMR5 before calling this function.

@Param None

@Returns None

@Example Initialize TMR5 module

```
Start TMR5 TMR5_StartTimer();
```

```
Do something else...
```

```
Stop TMR5; TMR5_StopTimer();
```

6.117.1.11 TMR5_WriteTimer()

```
void TMR5_WriteTimer (
    uint16_t timerVal)
```

@Summary Writes the TMR5 register.

@Description This function writes the TMR5 register. This function must be called after the initialization of TMR5.

@Preconditions Initialize the TMR5 before calling this function.

@Param timerVal - Value to write into TMR5 register.

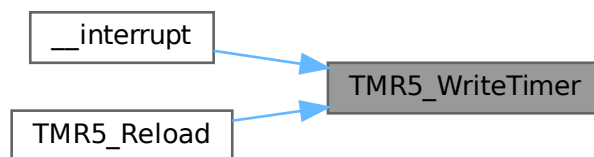
@Returns None

@Example #define PERIOD 0x80 #define ZERO 0x00

```
while(1) { Read the TMR5 register if(ZERO == TMR5_ReadTimer()) { Do something
else...
```

```
Write the TMR5 register TMR5_WriteTimer(PERIOD); }
```

```
Do something else... } Here is the caller graph for this function:
```



6.117.2 Variable Documentation

6.117.2.1 timer5ReloadVal

```
volatile uint16_t timer5ReloadVal
```

TMR5 Generated Driver File

@Company Microchip Technology Inc.

@File Name [tmr5.c](#)

@Summary This is the generated driver implementation file for the TMR5 driver using PIC10 / PIC12 / PIC16 / PIC18 MCUs

@Description This source file provides APIs for TMR5. Generation Information : Product Revision : PIC10 / PIC12 / PIC16 / PIC18 MCUs - 1.81.8 Device : PIC18F47Q84 Driver Version : 2.11 The generated drivers are tested against the following: Compiler : XC8 2.36 and above MPLAB : MPLAB X 6.00 Section: Included Files Section: Global Variables Definitions

6.117.2.2 TMR5_InterruptHandler

```
void(* TMR5_InterruptHandler) (void) (
    void )
```

@Summary Timer Interrupt Handler

@Description This is a function pointer to the function that will be called during the ISR

@Preconditions Initialize the TMR5 module with interrupt before calling this isr.

@Param None

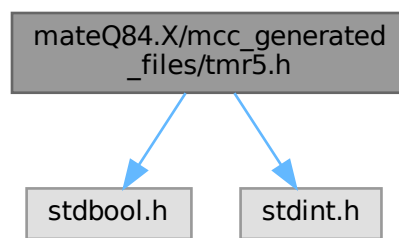
@Returns None

6.118 mateQ84.X/mcc_generated_files/tmr5.h File Reference

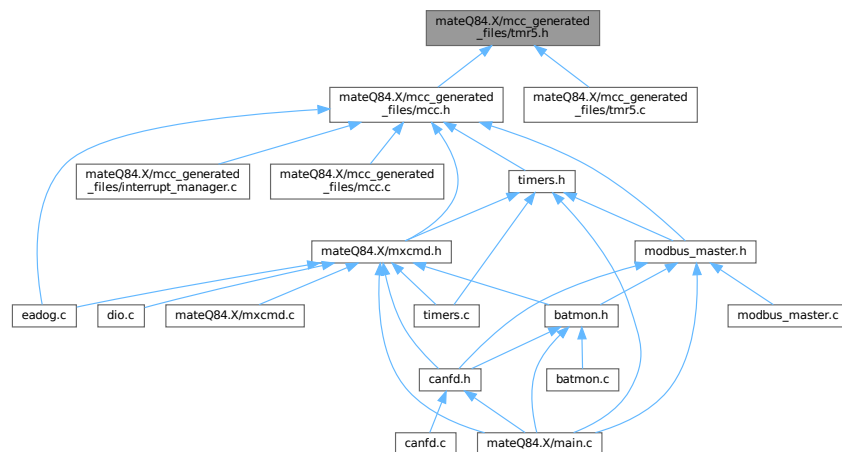
```
#include <stdbool.h>
```

```
#include <stdint.h>
```

Include dependency graph for tmr5.h:



This graph shows which files directly or indirectly include this file:



Functions

- void [TMR5_Initialize](#) (void)
- void [TMR5_StartTimer](#) (void)
- void [TMR5_StopTimer](#) (void)
- uint16_t [TMR5_ReadTimer](#) (void)
- void [TMR5_WriteTimer](#) (uint16_t timerVal)
- void [TMR5_Reload](#) (void)
- void [TMR5_StartSinglePulseAcquisition](#) (void)
- uint8_t [TMR5_CheckGateValueStatus](#) (void)
- void [TMR5_SetInterruptHandler](#) (void(*InterruptHandler)(void))
- void [TMR5_DefaultInterruptHandler](#) (void)

Variables

- void(* [TMR5_InterruptHandler](#))(void)

6.118.1 Function Documentation

6.118.1.1 TMR5_CheckGateValueStatus()

```
uint8_t TMR5_CheckGateValueStatus (
    void )
```

@Summary Check the current state of Timer1 gate.

@Description This function reads the TMR5 gate value and return it. This function must be used when the TMR5 gate is enabled.

@Preconditions Initialize the TMR5 with gate enable before calling this function.

@Param None

@Returns None

@Example `uint16_t xVal; uint16_t yVal;`

```
enable TMR5 singlepulse mode TMR5_StartSinglePulseAcquistion();
```

```
check TMR5 gate status if(TMR5_CheckGateValueStatus() == 0) xVal = TMR5_ReadTimer();
```

```
wait untill gate interrupt occured while(TMR5IF == 0) { }
```

```
yVal = TMR5_ReadTimer();
```

6.118.1.2 TMR5_DefaultInterruptHandler()

```
void TMR5_DefaultInterruptHandler (
    void )
```

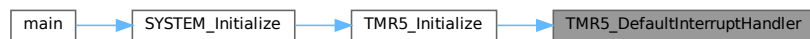
@Summary Default Timer Interrupt Handler

@Description This is the default Interrupt Handler function

@Preconditions Initialize the TMR5 module with interrupt before calling this isr.

@Param None

@Returns None Here is the caller graph for this function:



6.118.1.3 TMR5_Initialize()

```
void TMR5_Initialize (
    void )
```

TMR5 Generated Driver API Header File

@Company Microchip Technology Inc.

@File Name [tmr5.h](#)

@Summary This is the generated header file for the TMR5 driver using PIC10 / PIC12 / PIC16 / PIC18 MCUs

@Description This header file provides APIs for driver for TMR5. Generation Information : Product Revision : PIC10 / PIC12 / PIC16 / PIC18 MCUs - 1.81.8 Device : PIC18F47Q84 Driver Version : 2.11 The generated drivers are tested against the following: Compiler : XC8 2.36 and above MPLAB : MPLAB X 6.00 Section: Included Files Section: TMR5 APIs @Summary Initializes the TMR5

@Description This routine initializes the TMR5. This routine must be called before any other TMR5 routine is called. This routine should only be called once during system initialization.

@Preconditions None

@Param None

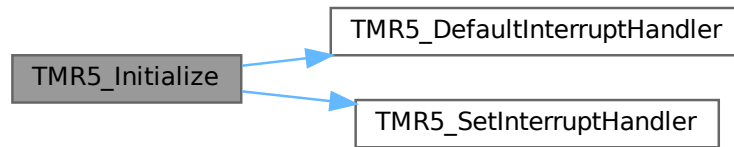
@Returns None

@Comment

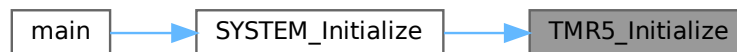
@Example `main()` { Initialize TMR5 module `TMR5_Initialize()` ;

Do something else... }

Section: TMR5 APIs Here is the call graph for this function:



Here is the caller graph for this function:



6.118.1.4 TMR5_ReadTimer()

```
uint16_t TMR5_ReadTimer (
    void )
```

@Summary Reads the TMR5 register.

@Description This function reads the TMR5 register value and return it.

@Preconditions Initialize the TMR5 before calling this function.

@Param None

@Returns This function returns the current value of TMR5 register.

@Example Initialize TMR5 module

```
Start TMR5 TMR5_StartTimer();
```

```
Read the current value of TMR5 if(0 == TMR5_ReadTimer()) { Do something
else...
```

```
Reload the TMR value TMR5_Reload(); }
```

6.118.1.5 TMR5_Reload()

```
void TMR5_Reload (
    void )
```

@Summary Reload the TMR5 register.

@Description This function reloads the TMR5 register. This function must be called to write initial value into TMR5 register.

@Preconditions Initialize the TMR5 before calling this function.

@Param None

@Returns None

@Example while(1) { if(TMR5IF) { Do something else...

clear the TMR5 interrupt flag TMR5IF = 0;

Reload the initial value of TMR5 `TMR5_Reload();` } } Here is the call graph for this function:

**6.118.1.6 TMR5_SetInterruptHandler()**

```
void TMR5_SetInterruptHandler (
    void(* InterruptHandler ) (void))
```

@Summary Set Timer Interrupt Handler

@Description This sets the function to be called during the ISR

@Preconditions Initialize the TMR5 module with interrupt before calling this.

@Param Address of function to be set

@Returns None Here is the caller graph for this function:



6.118.1.7 TMR5_StartSinglePulseAcquisition()

```
void TMR5_StartSinglePulseAcquisition (
    void )
```

@Summary Starts the single pulse acquisition in TMR5 gate operation.

@Description This function starts the single pulse acquisition in TMR5 gate operation. This function must be used when the TMR5 gate is enabled.

@Preconditions Initialize the TMR5 with gate enable before calling this function.

@Param None

@Returns None

@Example uint16_t xVal; uint16_t yVal;

```
enable TMR5 singlepulse mode TMR5_StartSinglePulseAcquisition();
```

```
check TMR5 gate status if(TMR5_CheckGateValueStatus()== 0) xVal = TMR5_ReadTimer();
```

```
wait untill gate interrupt occured while(TMR5GIF == 0) { }
```

```
yVal = TMR5_ReadTimer();
```

6.118.1.8 TMR5_StartTimer()

```
void TMR5_StartTimer (
    void )
```

@Summary This function starts the TMR5.

@Description This function starts the TMR5 operation. This function must be called after the initialization of TMR5.

@Preconditions Initialize the TMR5 before calling this function.

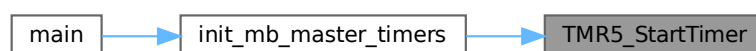
@Param None

@Returns None

@Example Initialize TMR5 module

```
Start TMR5 TMR5_StartTimer();
```

Do something else... Here is the caller graph for this function:



6.118.1.9 TMR5_StopTimer()

```
void TMR5_StopTimer (
    void )
```

@Summary This function stops the TMR5.

@Description This function stops the TMR5 operation. This function must be called after the start of TMR5.

@Preconditions Initialize the TMR5 before calling this function.

@Param None

@Returns None

@Example Initialize TMR5 module

```
Start TMR5 TMR5_StartTimer();
```

```
Do something else...
```

```
Stop TMR5; TMR5_StopTimer();
```

6.118.1.10 TMR5_WriteTimer()

```
void TMR5_WriteTimer (
    uint16_t timerVal)
```

@Summary Writes the TMR5 register.

@Description This function writes the TMR5 register. This function must be called after the initialization of TMR5.

@Preconditions Initialize the TMR5 before calling this function.

@Param timerVal - Value to write into TMR5 register.

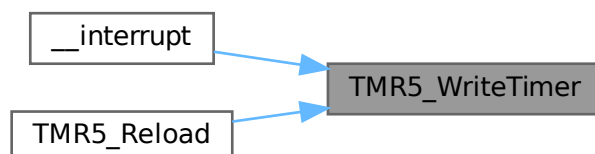
@Returns None

@Example #define PERIOD 0x80 #define ZERO 0x00

```
while(1) { Read the TMR5 register if(ZERO == TMR5_ReadTimer()) { Do something
else...
```

```
Write the TMR5 register TMR5_WriteTimer(PERIOD); }
```

```
Do something else... } Here is the caller graph for this function:
```



6.118.2 Variable Documentation

6.118.2.1 TMR5_InterruptHandler

```
void(* TMR5_InterruptHandler) (void) (
    void ) [extern]
```

@Summary Timer Interrupt Handler

@Description This is a function pointer to the function that will be called during the ISR

@Preconditions Initialize the TMR5 module with interrupt before calling this isr.

@Param None

@Returns None

6.119 tmr5.h

[Go to the documentation of this file.](#)

```
00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029     party
00029     license terms applicable to your use of third party software (including open source software) that
00030
00030     may accompany Microchip software.
00031
00032     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035     FOR A PARTICULAR PURPOSE.
00036
00037     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044     SOFTWARE.
00045 */
00046
00047 #ifndef TMR5_H
00048 #define TMR5_H
00049
00053
00054 #include <stdbool.h>
00055 #include <stdint.h>
00056
00057 #ifdef __cplusplus // Provide C++ Compatibility
00058
00059     extern "C" {
00060
00061 #endif
00062
00063
00067
00100 void TMR5_Initialize(void);
00101
00129 void TMR5_StartTimer(void);
00130
00161 void TMR5_StopTimer(void);
00162
00196 uint16_t TMR5_ReadTimer(void);
00197
00235 void TMR5_WriteTimer(uint16_t timerVal);
00236
```



```

00271 void TMR5_Reload(void);
00272
00310 void TMR5_StartSinglePulseAcquisition(void);
00311
00349 uint8_t TMR5_CheckGateValueStatus(void);
00350
00351
00368 void TMR5_SetInterruptHandler(void (* InterruptHandler)(void));
00369
00386 extern void (*TMR5_InterruptHandler)(void);
00387
00404 void TMR5_DefaultInterruptHandler(void);
00405
00406 #ifdef __cplusplus // Provide C++ Compatibility
00407
00408 }
00409
00410 #endif
00411
00412 #endif // TMR5_H

```

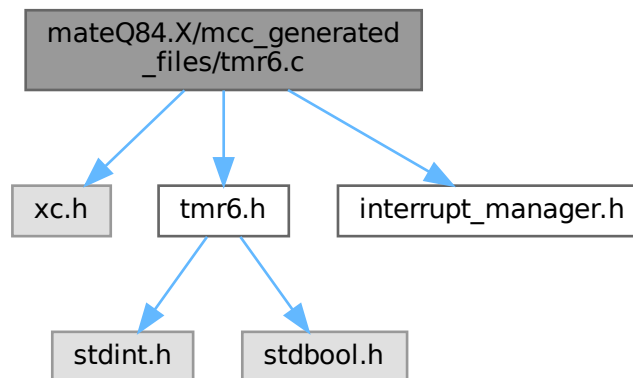
6.120 mateQ84.X/mcc_generated_files/tmr6.c File Reference

```

#include <xc.h>
#include "tmr6.h"
#include "interrupt_manager.h"

```

Include dependency graph for tmr6.c:



Functions

- void [TMR6_Initialize](#) (void)
- void [TMR6_ModeSet](#) (TMR6_HLT_MODE mode)
- void [TMR6_ExtResetSourceSet](#) (TMR6_HLT_EXT_RESET_SOURCE reset)
- void [TMR6_Start](#) (void)
- void [TMR6_StartTimer](#) (void)
- void [TMR6_Stop](#) (void)
- void [TMR6_StopTimer](#) (void)
- uint8_t [TMR6_Counter8BitGet](#) (void)
- uint8_t [TMR6_ReadTimer](#) (void)
- void [TMR6_Counter8BitSet](#) (uint8_t timerVal)

- void [TMR6_WriteTimer](#) (uint8_t timerVal)
- void [TMR6_Period8BitSet](#) (uint8_t periodVal)
- void [TMR6_LoadPeriodRegister](#) (uint8_t periodVal)
- void [__interrupt](#) (irq(TMR6), base(8), low_priority)
- void [TMR6_SetInterruptHandler](#) (void(*InterruptHandler)(void))
- void [TMR6_DefaultInterruptHandler](#) (void)

Variables

- void(* [TMR6_InterruptHandler](#))(void)

6.120.1 Function Documentation

6.120.1.1 [__interrupt\(\)](#)

```
void __interrupt (
    irq(TMR6) ,
    base(8) ,
    low_priority )
```

6.120.1.2 [TMR6_Counter8BitGet\(\)](#)

```
uint8_t TMR6_Counter8BitGet (
    void )
```

@Summary Reads the TMR6 register.

@Description This function reads the TMR6 register value and return it.

@Preconditions Initialize the TMR6 before calling this function.

@Param None

@Returns This function returns the current value of TMR6 register.

@Example Initialize TMR6 module

```
Start TMR6 TMR6\_Start\(\) ;
```

```
Read the current value of TMR6 if(0 == TMR6\_Counter8BitGet\(\)) { Do something else...
```

```
Reload the TMR value TMR6\_Period8BitSet\(\) ; } Here is the caller graph for this function:
```



6.120.1.3 TMR6_Counter8BitSet()

```
void TMR6_Counter8BitSet (
    uint8_t timerVal)
```

@Summary Writes the TMR6 register.

@Description This function writes the TMR6 register. This function must be called after the initialization of TMR6.

@Preconditions Initialize the TMR6 before calling this function.

@Param timerVal - Value to write into TMR6 register.

@Returns None

@Example #define PERIOD 0x80 #define ZERO 0x00

```
while(1) { Read the TMR6 register if(ZERO == TMR6_Counter8BitGet()) { Do
something else...
```

```
Write the TMR6 register TMR6_Counter8BitSet(PERIOD); }
```

```
Do something else... } Here is the caller graph for this function:
```

**6.120.1.4 TMR6_DefaultInterruptHandler()**

```
void TMR6_DefaultInterruptHandler (
    void )
```

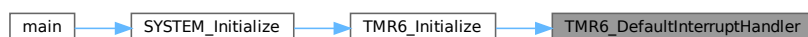
@Summary Default Timer Interrupt Handler

@Description This is the default Interrupt Handler function

@Preconditions Initialize the TMR6 module with interrupt before calling this isr.

@Param None

@Returns None Here is the caller graph for this function:



6.120.1.5 TMR6_ExtResetSourceSet()

```
void TMR6_ExtResetSourceSet (
    TMR6_HLT_EXT_RESET_SOURCE reset)
```

@Summary Configures the HLT external reset source.

@Description Writes the T6RSTbits.RSEL bits.

@Preconditions Initialize the TMR6 before calling this function.

@Param reset - Value to write into T6RSTbits.RSEL bits.

@Returns None

@Example `main()` {

```
TMR6_HLT_EXT_RESET_SOURCE hltresetsrc;
hltresetsrc = T2IN;
```

```
Initialize TMR6 module
```

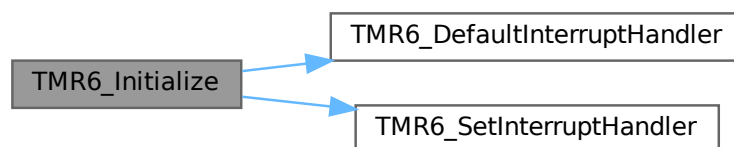
```
Set the HLT mode TMR6_ExtResetSourceSet(hltresetsrc);
```

```
Do something else... }
```

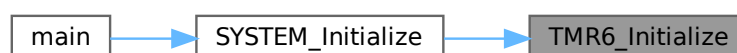
6.120.1.6 TMR6_Initialize()

```
void TMR6_Initialize (
    void )
```

Section: TMR6 APIs Here is the call graph for this function:



Here is the caller graph for this function:



6.120.1.7 TMR6_LoadPeriodRegister()

```
void TMR6_LoadPeriodRegister (
    uint8_t periodVal)
```

@Summary Load value to Period Register.

@Description This function writes the value to PR6 register. This function must be called after the initialization of TMR6.

@Preconditions Initialize the TMR6 before calling this function.

@Param periodVal - Value to load into TMR6 register.

@Returns None

@Example #define PERIOD1 0x80 #define PERIOD2 0x40 #define ZERO 0x00

```
while(1) { Read the TMR6 register if(ZERO == TMR6_ReadTimer()) { Do something
else...
```

```
    if(flag)
    {
        flag = 0;
```

```
Load Period 1 value TMR6_LoadPeriodRegister(PERIOD1); } else { flag = 1;
```

```
Load Period 2 value TMR6_LoadPeriodRegister(PERIOD2); } }
```

```
Do something else... } Here is the call graph for this function:
```

**6.120.1.8 TMR6_ModeSet()**

```
void TMR6_ModeSet (
    TMR6_HLT_MODE mode)
```

@Summary Configures the Hardware Limit Timer mode.

@Description Writes the T6HLTbits.MODE bits.

@Preconditions Initialize the TMR6 before calling this function.

@Param mode - Value to write into T6HLTbits.MODE bits.

@Returns None

@Example main() {

```
TMR6_HLT_MODE hltmode;
hltmode = TMR6_ROM_STARTS_TMRON_EN;
```

```
Initialize TMR6 module TMR6.Initialize();
```

```
Set the HLT mode TMR6_ModeSet (hltmode);
```

```
Do something else... }
```

6.120.1.9 TMR6_Period8BitSet()

```
void TMR6_Period8BitSet (
    uint8_t periodVal)
```

@Summary Load value to Period Register.

@Description This function writes the value to PR6 register. This function must be called after the initialization of TMR6.

@Preconditions Initialize the TMR6 before calling this function.

@Param periodVal - Value to load into TMR6 register.

@Returns None

@Example #define PERIOD1 0x80 #define PERIOD2 0x40 #define ZERO 0x00

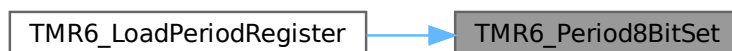
```
while(1) { Read the TMR6 register if(ZERO == TMR6_Counter8BitGet()) { Do
something else...
```

```
    if(flag)
    {
        flag = 0;
```

```
Load Period 1 value TMR6_Period8BitSet(PERIOD1); } else { flag = 1;
```

```
Load Period 2 value TMR6_Period8BitSet(PERIOD2); } }
```

```
Do something else... } Here is the caller graph for this function:
```



6.120.1.10 TMR6_ReadTimer()

```
uint8_t TMR6_ReadTimer (
    void )
```

@Summary Reads the TMR6 register.

@Description This function reads the TMR6 register value and return it.

@Preconditions Initialize the TMR6 before calling this function.

@Param None

@Returns This function returns the current value of TMR6 register.

@Example Initialize TMR6 module

```
Start TMR6 TMR6_StartTimer();
```

```
Read the current value of TMR6 if(0 == TMR6_ReadTimer()) { Do something
else...
```

```
Reload the TMR value TMR6_LoadPeriodRegister(); } Here is the call graph for this function:
```



6.120.1.11 TMR6_SetInterruptHandler()

```
void TMR6_SetInterruptHandler (
    void(* InterruptHandler ) (void))
```

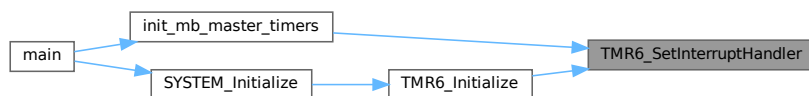
@Summary Set Timer Interrupt Handler

@Description This sets the function to be called during the ISR

@Preconditions Initialize the TMR6 module with interrupt before calling this.

@Param Address of function to be set

@Returns None Here is the caller graph for this function:



6.120.1.12 TMR6_Start()

```
void TMR6_Start (
    void )
```

@Summary This function starts the TMR6.

@Description This function starts the TMR6 operation. This function must be called after the initialization of TMR6.

@Preconditions Initialize the TMR6 before calling this function.

@Param None

@Returns None

@Example Initialize TMR6 module

Start TMR6 `TMR6_Start()`;

Do something else... Here is the caller graph for this function:



6.120.1.13 TMR6_StartTimer()

```
void TMR6_StartTimer (
    void )
```

@Summary This function starts the TMR6.

@Description This function starts the TMR6 operation. This function must be called after the initialization of TMR6.

@Preconditions Initialize the TMR6 before calling this function.

@Param None

@Returns None

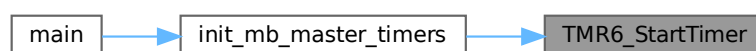
@Example Initialize TMR6 module

Start TMR6 `TMR6_StartTimer()`;

Do something else... Here is the call graph for this function:



Here is the caller graph for this function:



6.120.1.14 TMR6_Stop()

```
void TMR6_Stop (  
    void )
```

@Summary This function stops the TMR6.

@Description This function stops the TMR6 operation. This function must be called after the start of TMR6.

@Preconditions Initialize the TMR6 before calling this function.

@Param None

@Returns None

@Example Initialize TMR6 module

```
Start TMR6 TMR6_Start();
```

Do something else...

Stop TMR6; TMR6_Stop(); Here is the caller graph for this function:



6.120.1.15 TMR6_StopTimer()

```
void TMR6_StopTimer (  
    void )
```

@Summary This function stops the TMR6.

@Description This function stops the TMR6 operation. This function must be called after the start of TMR6.

@Preconditions Initialize the TMR6 before calling this function.

@Param None

@Returns None

@Example Initialize TMR6 module

```
Start TMR6 TMR6_StartTimer();
```

Do something else...

Stop TMR6; TMR6_StopTimer(); Here is the call graph for this function:



6.120.1.16 TMR6_WriteTimer()

```
void TMR6_WriteTimer (
    uint8_t timerVal)
```

@Summary Writes the TMR6 register.

@Description This function writes the TMR6 register. This function must be called after the initialization of TMR6.

@Preconditions Initialize the TMR6 before calling this function.

@Param timerVal - Value to write into TMR6 register.

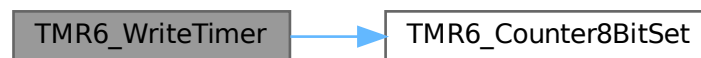
@Returns None

@Example #define PERIOD 0x80 #define ZERO 0x00

```
while(1) { Read the TMR6 register if(ZERO == TMR6_ReadTimer()) { Do something
else...
```

```
Write the TMR6 register TMR6_WriteTimer(PERIOD); }
```

```
Do something else... } Here is the call graph for this function:
```



6.120.2 Variable Documentation

6.120.2.1 TMR6_InterruptHandler

```
void(* TMR6_InterruptHandler) (void) (
    void )
```

TMR6 Generated Driver File

@Company Microchip Technology Inc.

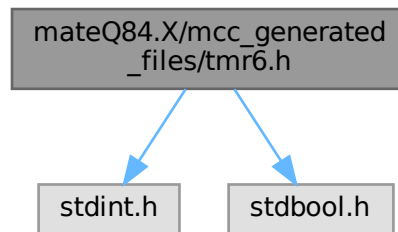
@File Name [tmr6.c](#)

@Summary This is the generated driver implementation file for the TMR6 driver using PIC10 / PIC12 / PIC16 / PIC18 MCUs

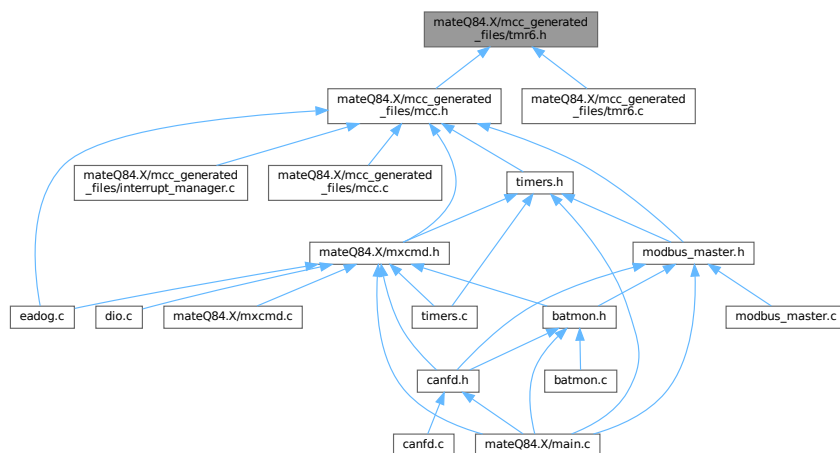
@Description This source file provides APIs for TMR6. Generation Information : Product Revision : PIC10 / PIC12 / PIC16 / PIC18 MCUs - 1.81.8 Device : PIC18F47Q84 Driver Version : 2.11 The generated drivers are tested against the following: Compiler : XC8 2.36 and above MPLAB : MPLAB X 6.00 Section: Included Files Section: Global Variables Definitions

6.121 mateQ84.X/mcc_generated_files/tmr6.h File Reference

```
#include <stdint.h>
#include <stdbool.h>
Include dependency graph for tmr6.h:
```



This graph shows which files directly or indirectly include this file:



Enumerations

- enum `TMR6_HLT_MODE` {
 `TMR6_ROP_STARTS_TMRON` , `TMR6_ROP_STARTS_TMRON_ERSHIGH` , `TMR6_ROP_STARTS_TMRON_ERSLOW` , `TMR6_ROP_RESETS_ERBOTHEDGE` , `TMR6_ROP_RESETS_ERSRISINGEDGE` , `TMR6_ROP_RESETS_ERSFALLINGEDGE` , `TMR6_ROP_RESETS_ERSLOW` , `TMR6_ROP_RESETS_ERSHIGH` , `TMR6_OS_STARTS_TMRON` , `TMR6_OS_STARTS_ERSRISINGEDGE` , `TMR6_OS_STARTS_ERSFALLINGEDGE` , `TMR6_OS_STARTS_ERBOTHEDGE` , `TMR6_OS_STARTS_ERSFIRSTRISINGEDGE` , `TMR6_OS_STARTS_ERSFIRSTFALLINGEDGE` , `TMR6_OS_STARTS_ERSRISINGEDGEDETECT` , `TMR6_OS_STARTS_ERSFALLINGEDGEDETECT` , `TMR6_OS_STARTS_TMRON_ERSHIGH` = 0x16 , `TMR6_OS_STARTS_TMRON_ERSLOW` = 0x17 , `TMR6_MS_STARTS_TMRON_ERSRISINGEDGEDETECT` = 0x11 , `TMR6_MS_STARTS_TMRON_ERSFALLINGEDGEDETECT` = 0x12 , `TMR6_MS_STARTS_TMRON_ERBOTHEDGE` = 0x13 }

```

• enum TMR6_HLT_EXT_RESET_SOURCE {
    TMR6_T6INPPS , TMR6_T2POSTSCALED , TMR6_T4POSTSCALED , TMR6_RESERVED ,
    TMR6_CCP1_OUT , TMR6_CCP2_OUT , TMR6_CCP3_OUT , TMR6_PWM1S1P1_OUT ,
    TMR6_PWM1S1P2_OUT , TMR6_PWM2S1P1_OUT , TMR6_PWM2S1P2_OUT , TMR6_PWM3S1P1_OUT
    ,
    TMR6_PWM3S1P2_OUT , TMR6_RESERVED_2 , TMR6_RESERVED_3 , TMR6_CMP1_OUT ,
    TMR6_CMP2_OUT , TMR6_ZCD_OUTPUT , TMR6_CLC1_OUT , TMR6_CLC2_OUT ,
    TMR6_CLC3_OUT , TMR6_CLC4_OUT , TMR6_CLC5_OUT , TMR6_CLC6_OUT ,
    TMR6_CLC7_OUT , TMR6_CLC8_OUT , TMR6_UART1_RX_EDGE , TMR6_UART1_TX_EDGE ,
    TMR6_UART2_RX_EDGE , TMR6_UART2_TX_EDGE , TMR6_UART3_RX_EDGE , TMR6_UART3_TX_EDGE
    ,
    TMR6_UART4_RX_EDGE , TMR6_UART4_TX_EDGE , TMR6_UART5_RX_EDGE , TMR6_UART5_TX_EDGE
    ,
    TMR6_RESERVED_4 }

```

Functions

- void `TMR6_Initialize` (void)
- void `TMR6_ModeSet` (TMR6_HLT_MODE mode)
- void `TMR6_ExtResetSourceSet` (TMR6_HLT_EXT_RESET_SOURCE reset)
- void `TMR6_Start` (void)
- void `TMR6_StartTimer` (void)
- void `TMR6_Stop` (void)
- void `TMR6_StopTimer` (void)
- uint8_t `TMR6_Counter8BitGet` (void)
- uint8_t `TMR6_ReadTimer` (void)
- void `TMR6_Counter8BitSet` (uint8_t timerVal)
- void `TMR6_WriteTimer` (uint8_t timerVal)
- void `TMR6_Period8BitSet` (uint8_t periodVal)
- void `TMR6_LoadPeriodRegister` (uint8_t periodVal)
- void `TMR6_SetInterruptHandler` (void(*InterruptHandler)(void))
- void `TMR6_DefaultInterruptHandler` (void)

Variables

- void(* `TMR6_InterruptHandler`)(void)

6.121.1 Enumeration Type Documentation

6.121.1.1 TMR6_HLT_EXT_RESET_SOURCE

```
enum TMR6_HLT_EXT_RESET_SOURCE
```

HLT Reset Source Enumeration

@Summary Defines the different reset source of the HLT.

@Description This source can control starting and stopping of the timer, as well as resetting the timer, depending on which mode the timer is in. The mode of the timer is controlled by the HLT_MODE

Enumerator

TMR6_T6INPPS	
TMR6_T2POSTSCALED	
TMR6_T4POSTSCALED	
TMR6_RESERVED	
TMR6_CCP1_OUT	
TMR6_CCP2_OUT	
TMR6_CCP3_OUT	
TMR6_PWM1S1P1_OUT	
TMR6_PWM1S1P2_OUT	
TMR6_PWM2S1P1_OUT	
TMR6_PWM2S1P2_OUT	
TMR6_PWM3S1P1_OUT	
TMR6_PWM3S1P2_OUT	
TMR6_RESERVED_2	
TMR6_RESERVED_3	
TMR6_CMP1_OUT	
TMR6_CMP2_OUT	
TMR6_ZCD_OUTPUT	
TMR6_CLC1_OUT	
TMR6_CLC2_OUT	
TMR6_CLC3_OUT	
TMR6_CLC4_OUT	
TMR6_CLC5_OUT	
TMR6_CLC6_OUT	
TMR6_CLC7_OUT	
TMR6_CLC8_OUT	
TMR6_UART1_RX_EDGE	
TMR6_UART1_TX_EDGE	
TMR6_UART2_RX_EDGE	
TMR6_UART2_TX_EDGE	
TMR6_UART3_RX_EDGE	
TMR6_UART3_TX_EDGE	
TMR6_UART4_RX_EDGE	
TMR6_UART4_TX_EDGE	
TMR6_UART5_RX_EDGE	
TMR6_UART5_TX_EDGE	
TMR6_RESERVED_4	

6.121.1.2 TMR6_HLT_MODE

```
enum TMR6_HLT_MODE
```

TMR6 Generated Driver API Header File

@Company Microchip Technology Inc.

@File Name [tmr6.h](#)

@Summary This is the generated header file for the TMR6 driver using PIC10 / PIC12 / PIC16 / PIC18 MCUs

@Description This header file provides APIs for driver for TMR6. Generation Information : Product Revision : PIC10 / PIC12 / PIC16 / PIC18 MCUs - 1.81.8 Device : PIC18F47Q84 Driver Version : 2.11 The generated drivers are tested against the following: Compiler : XC8 2.36 and above MPLAB : MPLAB X 6.00 Section: Included Files Section: Data Type Definitions HLT Mode Setting Enumeration

@Summary Defines the different modes of the HLT.

@Description This defines the several modes of operation of the Timer with HLT extension. The modes can be set in a control register associated with the timer

Enumerator

TMR6_ROP_STARTS_TMRON	
TMR6_ROP_STARTS_TMRON_ERSHIGH	
TMR6_ROP_STARTS_TMRON_ERSLOW	
TMR6_ROP_RESETS_ERSBOTHEDGE	
TMR6_ROP_RESETS_ERSRISINGEDGE	
TMR6_ROP_RESETS_ERSFALLINGEDGE	
TMR6_ROP_RESETS_ERSLOW	
TMR6_ROP_RESETS_ERSHIGH	
TMR6_OS_STARTS_TMRON	
TMR6_OS_STARTS_ERSRISINGEDGE	
TMR6_OS_STARTS_ERSFALLINGEDGE	
TMR6_OS_STARTS_ERSBOTHEDGE	
TMR6_OS_STARTS_ERSFIRSTRISINGEDGE	
TMR6_OS_STARTS_ERSFIRSTFALLINGEDGE	
TMR6_OS_STARTS_ERSRISINGEDGEDETECT	
TMR6_OS_STARTS_ERSFALLINGEDGEDETECT	
TMR6_OS_STARTS_TMRON_ERSHIGH	
TMR6_OS_STARTS_TMRON_ERSLOW	
TMR6_MS_STARTS_TMRON_ERSRISINGEDGEDETECT	
TMR6_MS_STARTS_TMRON_ERSFALLINGEDGEDETECT	
TMR6_MS_STARTS_TMRON_ERSBOTHEDGE	

6.121.2 Function Documentation

6.121.2.1 TMR6_Counter8BitGet()

```
uint8_t TMR6_Counter8BitGet (
    void )
```

@Summary Reads the TMR6 register.

@Description This function reads the TMR6 register value and return it.

@Preconditions Initialize the TMR6 before calling this function.

@Param None

@Returns This function returns the current value of TMR6 register.

@Example Initialize TMR6 module

Start TMR6 `TMR6_Start();`

Read the current value of TMR6 `if(0 == TMR6_Counter8BitGet()) { Do something else...`

Reload the TMR value `TMR6_Period8BitSet();` } Here is the caller graph for this function:



6.121.2.2 TMR6_Counter8BitSet()

```
void TMR6_Counter8BitSet (
    uint8_t timerVal)
```

@Summary Writes the TMR6 register.

@Description This function writes the TMR6 register. This function must be called after the initialization of TMR6.

@Preconditions Initialize the TMR6 before calling this function.

@Param timerVal - Value to write into TMR6 register.

@Returns None

@Example `#define PERIOD 0x80 #define ZERO 0x00`

```
while(1) { Read the TMR6 register if(ZERO == TMR6_Counter8BitGet()) { Do
something else...
```

```
Write the TMR6 register TMR6_Counter8BitSet(PERIOD); }
```

```
Do something else... } Here is the caller graph for this function:
```



6.121.2.3 TMR6_DefaultInterruptHandler()

```
void TMR6_DefaultInterruptHandler (
    void )
```

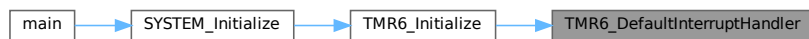
@Summary Default Timer Interrupt Handler

@Description This is the default Interrupt Handler function

@Preconditions Initialize the TMR6 module with interrupt before calling this isr.

@Param None

@Returns None Here is the caller graph for this function:



6.121.2.4 TMR6_ExtResetSourceSet()

```
void TMR6_ExtResetSourceSet (
    TMR6_HLT_EXT_RESET_SOURCE reset)
```

@Summary Configures the HLT external reset source.

@Description Writes the T6RSTbits.RSEL bits.

@Preconditions Initialize the TMR6 before calling this function.

@Param reset - Value to write into T6RSTbits.RSEL bits.

@Returns None

@Example `main()` {

```
TMR6_HLT_EXT_RESET_SOURCE hltresetsrc;
hltresetsrc = T2IN;
```

```
Initialize TMR6 module
```

```
Set the HLT mode TMR6_ExtResetSourceSet(hltresetsrc);
```

```
Do something else... }
```


6.121.2.5 TMR6_Initialize()

```
void TMR6_Initialize (  
    void )
```

Section: Macro Declarations Section: TMR6 APIs @Summary Initializes the TMR6 module.

@Description This function initializes the TMR6 Registers. This function must be called before any other TMR6 function is called.

@Preconditions None

@Param None

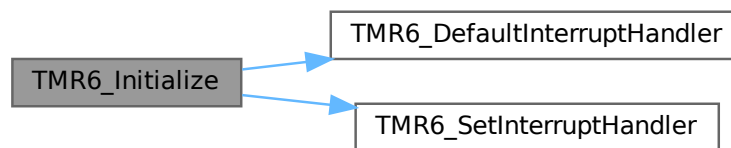
@Returns None

@Comment

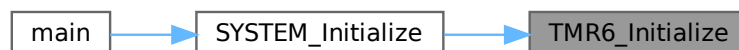
@Example `main()` { Initialize TMR6 module `TMR6_Initialize()`;

Do something else... }

Section: TMR6 APIs Here is the call graph for this function:



Here is the caller graph for this function:



6.121.2.6 TMR6_LoadPeriodRegister()

```
void TMR6_LoadPeriodRegister (
    uint8_t periodVal)
```

@Summary Load value to Period Register.

@Description This function writes the value to PR6 register. This function must be called after the initialization of TMR6.

@Preconditions Initialize the TMR6 before calling this function.

@Param periodVal - Value to load into TMR6 register.

@Returns None

@Example #define PERIOD1 0x80 #define PERIOD2 0x40 #define ZERO 0x00

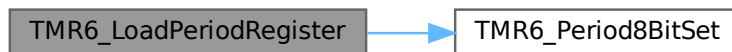
```
while(1) { Read the TMR6 register if(ZERO == TMR6_ReadTimer()) { Do something
else...
```

```
    if(flag)
    {
        flag = 0;
```

```
Load Period 1 value TMR6_LoadPeriodRegister(PERIOD1); } else { flag = 1;
```

```
Load Period 2 value TMR6_LoadPeriodRegister(PERIOD2); } }
```

```
Do something else... } Here is the call graph for this function:
```



6.121.2.7 TMR6_ModeSet()

```
void TMR6_ModeSet (
    TMR6_HLT_MODE mode)
```

@Summary Configures the Hardware Limit Timer mode.

@Description Writes the T6HLTbits.MODE bits.

@Preconditions Initialize the TMR6 before calling this function.

@Param mode - Value to write into T6HLTbits.MODE bits.

@Returns None

@Example main() {

```
TMR6_HLT_MODE hltmode;
hltmode = TMR6_ROM_STARTS_TMRON_EN;
```

```
Initialize TMR6 module TMR6.Initialize();
```

```
Set the HLT mode TMR6_ModeSet (hltmode);
```

```
Do something else... }
```

6.121.2.8 TMR6_Period8BitSet()

```
void TMR6_Period8BitSet (
    uint8_t periodVal)
```

@Summary Load value to Period Register.

@Description This function writes the value to PR6 register. This function must be called after the initialization of TMR6.

@Preconditions Initialize the TMR6 before calling this function.

@Param periodVal - Value to load into TMR6 register.

@Returns None

@Example #define PERIOD1 0x80 #define PERIOD2 0x40 #define ZERO 0x00

```
while(1) { Read the TMR6 register if(ZERO == TMR6_Counter8BitGet()) { Do
something else...
```

```
    if(flag)
    {
        flag = 0;
```

```
Load Period 1 value TMR6_Period8BitSet(PERIOD1); } else { flag = 1;
```

```
Load Period 2 value TMR6_Period8BitSet(PERIOD2); } }
```

```
Do something else... } Here is the caller graph for this function:
```

**6.121.2.9 TMR6_ReadTimer()**

```
uint8_t TMR6_ReadTimer (
    void )
```

@Summary Reads the TMR6 register.

@Description This function reads the TMR6 register value and return it.

@Preconditions Initialize the TMR6 before calling this function.

@Param None

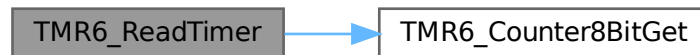
@Returns This function returns the current value of TMR6 register.

@Example Initialize TMR6 module

```
Start TMR6 TMR6_StartTimer();
```

```
Read the current value of TMR6 if(0 == TMR6_ReadTimer()) { Do something
else...
```

```
Reload the TMR value TMR6_LoadPeriodRegister(); } Here is the call graph for this function:
```



6.121.2.10 TMR6_SetInterruptHandler()

```
void TMR6_SetInterruptHandler (
    void(* InterruptHandler ) (void))
```

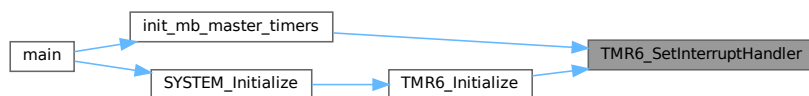
@Summary Set Timer Interrupt Handler

@Description This sets the function to be called during the ISR

@Preconditions Initialize the TMR6 module with interrupt before calling this.

@Param Address of function to be set

@Returns None Here is the caller graph for this function:



6.121.2.11 TMR6_Start()

```
void TMR6_Start (
    void )
```

@Summary This function starts the TMR6.

@Description This function starts the TMR6 operation. This function must be called after the initialization of TMR6.

@Preconditions Initialize the TMR6 before calling this function.

@Param None

@Returns None

@Example Initialize TMR6 module

Start TMR6 `TMR6_Start()`;

Do something else... Here is the caller graph for this function:



6.121.2.12 TMR6_StartTimer()

```
void TMR6_StartTimer (
    void )
```

@Summary This function starts the TMR6.

@Description This function starts the TMR6 operation. This function must be called after the initialization of TMR6.

@Preconditions Initialize the TMR6 before calling this function.

@Param None

@Returns None

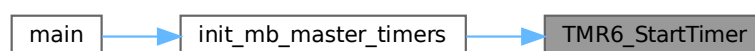
@Example Initialize TMR6 module

Start TMR6 `TMR6_StartTimer()`;

Do something else... Here is the call graph for this function:



Here is the caller graph for this function:



6.121.2.13 TMR6_Stop()

```
void TMR6_Stop (
    void )
```

@Summary This function stops the TMR6.

@Description This function stops the TMR6 operation. This function must be called after the start of TMR6.

@Preconditions Initialize the TMR6 before calling this function.

@Param None

@Returns None

@Example Initialize TMR6 module

```
Start TMR6 TMR6_Start();
```

```
Do something else...
```

```
Stop TMR6; TMR6_Stop(); Here is the caller graph for this function:
```

**6.121.2.14 TMR6_StopTimer()**

```
void TMR6_StopTimer (
    void )
```

@Summary This function stops the TMR6.

@Description This function stops the TMR6 operation. This function must be called after the start of TMR6.

@Preconditions Initialize the TMR6 before calling this function.

@Param None

@Returns None

@Example Initialize TMR6 module

```
Start TMR6 TMR6_StartTimer();
```

```
Do something else...
```

```
Stop TMR6; TMR6_StopTimer(); Here is the call graph for this function:
```



6.121.2.15 TMR6_WriteTimer()

```
void TMR6_WriteTimer (
    uint8_t timerVal)
```

@Summary Writes the TMR6 register.

@Description This function writes the TMR6 register. This function must be called after the initialization of TMR6.

@Preconditions Initialize the TMR6 before calling this function.

@Param timerVal - Value to write into TMR6 register.

@Returns None

@Example #define PERIOD 0x80 #define ZERO 0x00

```
while(1) { Read the TMR6 register if(ZERO == TMR6_ReadTimer()) { Do something
else...
```

```
Write the TMR6 register TMR6_WriteTimer(PERIOD); }
```

```
Do something else... } Here is the call graph for this function:
```



6.121.3 Variable Documentation

6.121.3.1 TMR6_InterruptHandler

```
void(* TMR6_InterruptHandler) (void) (
    void ) [extern]
```

@Summary Timer Interrupt Handler

@Description This is a function pointer to the function that will be called during the ISR

@Preconditions Initialize the TMR6 module with interrupt before calling this isr.

@Param None

@Returns None

TMR6 Generated Driver File

@Company Microchip Technology Inc.

@File Name [tmr6.c](#)

@Summary This is the generated driver implementation file for the TMR6 driver using PIC10 / PIC12 / PIC16 / PIC18 MCUs

@Description This source file provides APIs for TMR6. Generation Information : Product Revision : PIC10 / PIC12 / PIC16 / PIC18 MCUs - 1.81.8 Device : PIC18F47Q84 Driver Version : 2.11 The generated drivers are tested against the following: Compiler : XC8 2.36 and above MPLAB : MPLAB X 6.00 Section: Included Files Section: Global Variables Definitions

6.122 tmr6.h

[Go to the documentation of this file.](#)

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
party
00029     license terms applicable to your use of third party software (including open source software) that
may accompany Microchip software.
00030
00031     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00032     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00033     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00034     FOR A PARTICULAR PURPOSE.
00035
00036     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00037     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00038     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00039     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00040     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00041     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00042     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00043     SOFTWARE.
00044 */
00045 */
00046
00047 #ifndef TMR6_H
00048 #define TMR6_H
00049
00053
00054 #include <stdint.h>
00055 #include <stdbool.h>
00056
00057 #ifdef __cplusplus // Provide C++ Compatibility
00058     extern "C" {
00059
00060
00061 #endif
00062
00066
00078
00079 typedef enum
00080 {
00081
00082     /* Roll-over Pulse mode clears the TMRx upon TMRx = PRx, then continue running.
00083     ON bit must be set and is not affected by Resets
00084     */
00085
00086     /* Roll-over Pulse mode indicates that Timer starts
00087     immediately upon ON = 1 (Software Control)
00088     */
00089     TMR6_ROP_STARTS_TMRON,
00090
00091     /* Roll-over Pulse mode indicates that the Timer starts
00092     when ON = 1 and TMRx_ers = 1. Stops when TMRx_ers = 0
00093     */
00094     TMR6_ROP_STARTS_TMRON_ERSHIGH,
00095
00096     /* Roll-over Pulse mode indicates that the Timer starts
00097     when ON = 1 and TMRx_ers = 0. Stops when TMRx_ers = 1
00098     */
00099     TMR6_ROP_STARTS_TMRON_ERSLOW,
00100
00101     /* Roll-over Pulse mode indicates that the Timer resets
00102     upon rising or falling edge of TMRx_ers
00103     */
00104     TMR6_ROP_RESETS_ERSETHEDGE,
00105
00106     /* Roll-over Pulse mode indicates that the Timer resets
00107     upon rising edge of TMRx_ers
00108     */
00109     TMR6_ROP_RESETS_ERSRISINGEDGE,
00110
00111     /* Roll-over Pulse mode indicates that the Timer resets
00112     upon falling edge of TMRx_ers
00113     */
00114     TMR6_ROP_RESETS_ERSFALLINGEDGE,
00115
00116     /* Roll-over Pulse mode indicates that the Timer resets
00117     upon TMRx_ers = 0
00118     */

```



```

00119     TMR6_ROP_RESETS_ERLOW,
00120
00121     /* Roll-over Pulse mode indicates that the Timer resets
00122     upon TMRx_ers = 1
00123     */
00124     TMR6_ROP_RESETS_ERHIGH,
00125
00126     /*In all One-Shot mode the timer resets and the ON bit is
00127     cleared when the timer value matches the PRx period
00128     value. The ON bit must be set by software to start
00129     another timer cycle.
00130     */
00131
00132     /* One shot mode indicates that the Timer starts
00133     immediately upon ON = 1 (Software Control)
00134     */
00135     TMR6_OS_STARTS_TMRON,
00136
00137     /* One shot mode indicates that the Timer starts
00138     when a rising edge is detected on the TMRx_ers
00139     */
00140     TMR6_OS_STARTS_ERSRISINGEDGE ,
00141
00142     /* One shot mode indicates that the Timer starts
00143     when a falling edge is detected on the TMRx_ers
00144     */
00145     TMR6_OS_STARTS_ERFALLINGEDGE ,
00146
00147     /* One shot mode indicates that the Timer starts
00148     when either a rising or falling edge is detected on TMRx_ers
00149     */
00150     TMR6_OS_STARTS_ERSBOTHEDGE,
00151
00152     /* One shot mode indicates that the Timer starts
00153     upon first TMRx_ers rising edge and resets on all
00154     subsequent TMRx_ers rising edges
00155     */
00156     TMR6_OS_STARTS_ERFIRSTRISINGEDGE,
00157
00158     /* One shot mode indicates that the Timer starts
00159     upon first TMRx_ers falling edge and restarts on all
00160     subsequent TMRx_ers falling edges
00161     */
00162     TMR6_OS_STARTS_ERFIRSTFALLINGEDGE,
00163
00164     /* One shot mode indicates that the Timer starts
00165     when a rising edge is detected on the TMRx_ers,
00166     resets upon TMRx_ers = 0
00167     */
00168     TMR6_OS_STARTS_ERSRISINGEDGEDETECT,
00169     /* One shot mode indicates that the Timer starts
00170     when a falling edge is detected on the TMRx_ers,
00171     resets upon TMRx_ers = 1
00172     */
00173     TMR6_OS_STARTS_ERFALLINGEDGEDETECT,
00174
00175     /* One shot mode indicates that the Timer starts
00176     when a TMRx_ers = 1, ON =1 and resets upon TMRx_ers =0
00177     */
00178     TMR6_OS_STARTS_TMRON_ERHIGH = 0x16,
00179
00180     /* One shot mode indicates that the Timer starts
00181     when a TMRx_ers = 0, ON = 1 and resets upon TMRx_ers =1
00182     */
00183     TMR6_OS_STARTS_TMRON_ERLOW = 0x17,
00184
00185     /*In all Mono-Stable mode the ON bit must be initially set, but
00186     not cleared upon the TMRx = PRx, and the timer will start upon
00187     an TMRx_ers start event following TMRx = PRx.*/
00188
00189     /* Mono Stable mode indicates that the Timer starts
00190     when a rising edge is detected on the TMRx_ers and ON = 1
00191     */
00192     TMR6_MS_STARTS_TMRON_ERSRISINGEDGEDETECT = 0x11,
00193
00194     /* Mono Stable mode indicates that the Timer starts
00195     when a falling edge is detected on the TMRx_ers and ON = 1
00196     */
00197     TMR6_MS_STARTS_TMRON_ERFALLINGEDGEDETECT = 0x12,
00198
00199     /* Mono Stable mode indicates that the Timer starts
00200     when either a rising or falling edge is detected on TMRx_ers
00201     and ON = 1
00202     */
00203     TMR6_MS_STARTS_TMRON_ERSBOTHEDGE = 0x13
00204
00205 } TMR6_HLT_MODE;

```

```
00206
00219
00220 typedef enum
00221 {
00222     /* T6INPPS is the Timer external reset source
00223     */
00224     TMR6_T6INPPS,
00225
00226     /* Timer2 Postscale is the Timer external reset source
00227     */
00228     TMR6_T2POSTSCALED,
00229
00230     /* Timer4 Postscale is the Timer external reset source
00231     */
00232     TMR6_T4POSTSCALED,
00233
00234     /* Reserved enum cannot be used
00235     */
00236     TMR6_RESERVED,
00237
00238     /* CCP1_OUT is the Timer external reset source
00239     */
00240     TMR6_CCP1_OUT,
00241
00242     /* CCP2_OUT is the Timer external reset source
00243     */
00244     TMR6_CCP2_OUT,
00245
00246     /* CCP3_OUT is the Timer external reset source
00247     */
00248     TMR6_CCP3_OUT,
00249
00250     /* PWM1S1P1_out is the Timer external reset source
00251     */
00252     TMR6_PWM1S1P1_OUT,
00253
00254     /* PWM1S1P2_out is the Timer external reset source
00255     */
00256     TMR6_PWM1S1P2_OUT,
00257
00258     /* PWM2S1P1_out is the Timer external reset source
00259     */
00260     TMR6_PWM2S1P1_OUT,
00261
00262     /* PWM2S1P2_out is the Timer external reset source
00263     */
00264     TMR6_PWM2S1P2_OUT,
00265
00266     /* PWM3S1P1_out is the Timer external reset source
00267     */
00268     TMR6_PWM3S1P1_OUT,
00269
00270     /* PWM3S1P2_out is the Timer external reset source
00271     */
00272     TMR6_PWM3S1P2_OUT,
00273
00274     /* Reserved enum cannot be used
00275     */
00276     TMR6_RESERVED_2,
00277
00278     /* Reserved enum cannot be used
00279     */
00280     TMR6_RESERVED_3,
00281
00282     /* CMP1_OUT is the Timer external reset source
00283     */
00284     TMR6_CMP1_OUT,
00285
00286     /* CMP2_OUT is the Timer external reset source
00287     */
00288     TMR6_CMP2_OUT,
00289
00290     /* ZCD_Output is the Timer external reset source
00291     */
00292     TMR6_ZCD_OUTPUT,
00293
00294     /* CLC1_out is the Timer external reset source
00295     */
00296     TMR6_CLC1_OUT,
00297
00298     /* CLC2_out is the Timer external reset source
00299     */
00300     TMR6_CLC2_OUT,
00301
00302     /* CLC3_out is the Timer external reset source
00303     */
00304     TMR6_CLC3_OUT,
```

```
00305
00306     /* CLC4_out is the Timer external reset source
00307     */
00308     TMR6_CLC4_OUT,
00309
00310     /* CLC5_out is the Timer external reset source
00311     */
00312     TMR6_CLC5_OUT,
00313
00314     /* CLC6_out is the Timer external reset source
00315     */
00316     TMR6_CLC6_OUT,
00317
00318     /* CLC7_out is the Timer external reset source
00319     */
00320     TMR6_CLC7_OUT,
00321
00322     /* CLC8_out is the Timer external reset source
00323     */
00324     TMR6_CLC8_OUT,
00325
00326     /* UART1_rx_edge is the Timer external reset source
00327     */
00328     TMR6_UART1_RX_EDGE,
00329
00330     /* UART1_tx_edge is the Timer external reset source
00331     */
00332     TMR6_UART1_TX_EDGE,
00333
00334     /* UART2_rx_edge is the Timer external reset source
00335     */
00336     TMR6_UART2_RX_EDGE,
00337
00338     /* UART2_tx_edge is the Timer external reset source
00339     */
00340     TMR6_UART2_TX_EDGE,
00341
00342     /* UART3_rx_edge is the Timer external reset source
00343     */
00344     TMR6_UART3_RX_EDGE,
00345
00346     /* UART3_tx_edge is the Timer external reset source
00347     */
00348     TMR6_UART3_TX_EDGE,
00349
00350     /* UART4_rx_edge is the Timer external reset source
00351     */
00352     TMR6_UART4_RX_EDGE,
00353
00354     /* UART4_tx_edge is the Timer external reset source
00355     */
00356     TMR6_UART4_TX_EDGE,
00357
00358     /* UART5_rx_edge is the Timer external reset source
00359     */
00360     TMR6_UART5_RX_EDGE,
00361
00362     /* UART5_tx_edge is the Timer external reset source
00363     */
00364     TMR6_UART5_TX_EDGE,
00365
00366     /* Reserved enum cannot be used
00367     */
00368     TMR6_RESERVED_4
00369
00370
00371 } TMR6_HLT_EXT_RESET_SOURCE;
00372
00373
00377
00381
00413 void TMR6_Initialize(void);
00414
00449 void TMR6_ModeSet(TMR6_HLT_MODE mode);
00450
00484 void TMR6_ExtResetSourceSet(TMR6_HLT_EXT_RESET_SOURCE reset);
00485
00513 void TMR6_Start(void);
00514
00542 void TMR6_StartTimer(void);
00543
00574 void TMR6_Stop(void);
00575
00606 void TMR6_StopTimer(void);
00607
00641 uint8_t TMR6_Counter8BitGet(void);
00642
```

```

00676 uint8_t TMR6_ReadTimer(void);
00677
00715 void TMR6_Counter8BitSet(uint8_t timerVal);
00716
00754 void TMR6_WriteTimer(uint8_t timerVal);
00755
00806 void TMR6_Period8BitSet(uint8_t periodVal);
00807
00858 void TMR6_LoadPeriodRegister(uint8_t periodVal);
00859
00860
00877 void TMR6_SetInterruptHandler(void (* InterruptHandler)(void));
00878
00895 extern void (*TMR6_InterruptHandler)(void);
00896
00913 void TMR6_DefaultInterruptHandler(void);
00914
00915
00916 #ifdef __cplusplus // Provide C++ Compatibility
00917     }
00918 }
00919
00920 #endif
00921
00922 #endif // TMR6_H

```

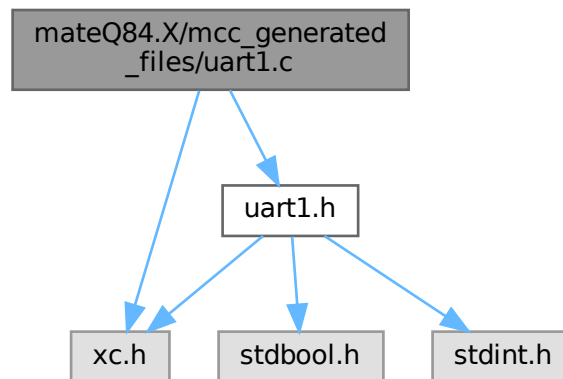
6.123 mateQ84.X/mcc_generated_files/uart1.c File Reference

```

#include <xc.h>
#include "uart1.h"

```

Include dependency graph for uart1.c:



Functions

- void [UART1_DefaultFramingErrorHandler](#)(void)
- void [UART1_DefaultOverrunErrorHandler](#)(void)
- void [UART1_DefaultErrorHandler](#)(void)
- void [UART1_Initialize](#)(void)
- bool [UART1_is_rx_ready](#)(void)
- bool [UART1_is_tx_ready](#)(void)
- bool [UART1_is_tx_done](#)(void)
- [uart1_status_t](#) [UART1_get_last_status](#)(void)

- `uint8_t UART1_Read (void)`
- `void UART1_Write (uint8_t txData)`
- `void UART1_SetAddressToTransmit (uint8_t txAddress)`
- `void UART1_SetFramingErrorHandler (void(*interruptHandler)(void))`
- `void UART1_SetOverrunErrorHandler (void(*interruptHandler)(void))`
- `void UART1_SetErrorHandler (void(*interruptHandler)(void))`

Variables

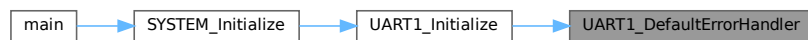
- `void(* UART1_FramingErrorHandler )(void)`
- `void(* UART1_OverrunErrorHandler )(void)`
- `void(* UART1_ErrorHandler )(void)`

6.123.1 Function Documentation

6.123.1.1 UART1_DefaultErrorHandler()

```
void UART1_DefaultErrorHandler (  
    void )
```

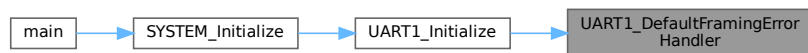
Here is the caller graph for this function:



6.123.1.2 UART1_DefaultFramingErrorHandler()

```
void UART1_DefaultFramingErrorHandler (  
    void )
```

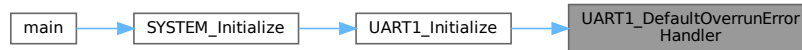
Here is the caller graph for this function:



6.123.1.3 UART1_DefaultOverrunErrorHandler()

```
void UART1_DefaultOverrunErrorHandler (
    void )
```

Here is the caller graph for this function:



6.123.1.4 UART1_get_last_status()

```
uart1_status_t UART1_get_last_status (
    void )
```

@Summary Gets the error status of the last read byte.

@Description This routine gets the error status of the last read byte.

@Preconditions [UART1_Initialize\(\)](#) function should have been called before calling this function. The returned value is only updated after a read is called.

@Param None

@Returns the status of the last read byte

@Example

```
void main(void) { volatile uint8_t rxData; volatile uart1_status_t rxStatus;
```

Initialize the device `SYSTEM_Initialize();`

Enable the Global Interrupts `INTERRUPT_GlobalInterruptEnable();`

```
while(1) { Logic to echo received data if(UART1_is_rx_ready()) { rxData =
UART1_Read(); rxStatus = UART1_get_last_status(); if(rxStatus.ferr){ LED_↔
0_SetHigh(); } } }
```

6.123.1.5 UART1_Initialize()

```
void UART1_Initialize (  
    void )
```

Section: UART1 APIs @Summary Initialization routine that takes inputs from the UART1 GUI.

@Description This routine initializes the UART1 driver. This routine must be called before any other UART1 routine is called.

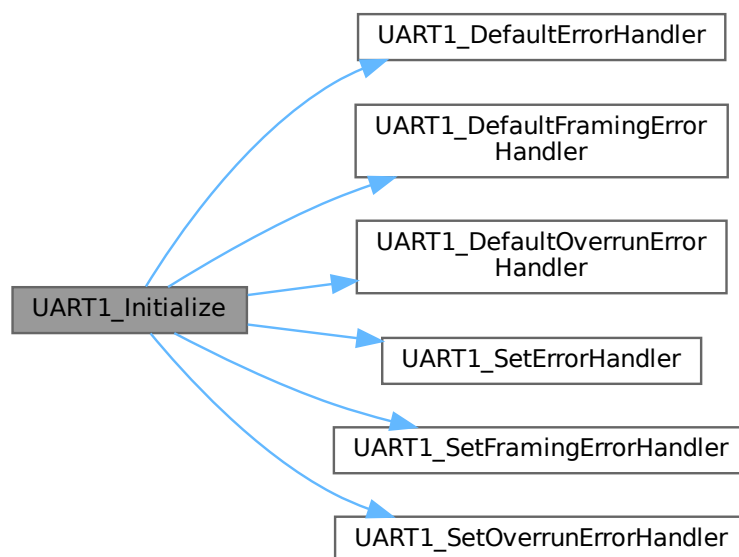
@Preconditions None

@Param None

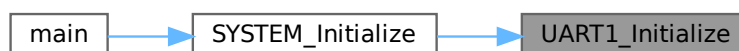
@Returns None

@Comment

@Example Here is the call graph for this function:



Here is the caller graph for this function:



6.123.1.6 UART1_is_rx_ready()

```
bool UART1_is_rx_ready (
    void )
```

@Summary Checks if the UART1 receiver ready for reading

@Description This routine checks if UART1 receiver has received data and ready to be read

@Preconditions [UART1_Initialize\(\)](#) function should be called before calling this function UART1 receiver should be enabled before calling this function

@Param None

@Returns Status of UART1 receiver TRUE: UART1 receiver is ready for reading FALSE: UART1 receiver is not ready for reading

@Example void [main\(void\)](#) { volatile uint8_t rxData;

Initialize the device [SYSTEM_Initialize\(\)](#);

```
while(1) { Logic to echo received data if(UART1_is_rx_ready()) { rxData =
UART1\_Read\(\); if(UART1_is_tx_ready()) { UART1\_Write(rxData); } } }
```

6.123.1.7 UART1_is_tx_done()

```
bool UART1_is_tx_done (
    void )
```

@Summary Checks if UART1 data is transmitted

@Description This function return the status of transmit shift register

@Preconditions [UART1_Initialize\(\)](#) function should be called before calling this function UART1 transmitter should be enabled and [UART1_Write](#) should be called before calling this function

@Param None

@Returns Status of UART1 transmit shift register TRUE: Data completely shifted out if the UART shift register FALSE: Data is not completely shifted out of the shift register

@Example void [main\(void\)](#) { volatile uint8_t rxData;

Initialize the device [SYSTEM_Initialize\(\)](#);

```
while(1) { if(UART1_is_tx_ready()) { LED\_0\_SetHigh\(\); UART1Write(rxData); }
if(UART1\_is\_tx\_done\(\)) { LED\_0\_SetLow\(\); } }
```


6.123.1.8 UART1_is_tx_ready()

```
bool UART1_is_tx_ready (
    void )
```

@Summary Checks if the UART1 transmitter is ready to transmit data

@Description This routine checks if UART1 transmitter is ready to accept and transmit data byte

@Preconditions [UART1_Initialize\(\)](#) function should have been called before calling this function. UART1 transmitter should be enabled before calling this function

@Param None

@Returns Status of UART1 transmitter TRUE: UART1 transmitter is ready FALSE: UART1 transmitter is not ready

@Example void [main\(void\)](#) { volatile uint8_t rxData;

Initialize the device [SYSTEM_Initialize\(\)](#);

```
while(1) { Logic to echo received data if(UART1_is_rx_ready()) { rxData =
UART1\_Read\(\); if(UART1_is_tx_ready()) { UART1\_Write(rxData); } } }
```

6.123.1.9 UART1_Read()

```
uint8_t UART1_Read (
    void )
```

@Summary Read a byte of data from the UART1.

@Description This routine reads a byte of data from the UART1.

@Preconditions [UART1_Initialize\(\)](#) function should have been called before calling this function. The transfer status should be checked to see if the receiver is not empty before calling this function.

UART1_DataReady is a macro which checks if any byte is received. Call this macro before using this function.

@Param None

@Returns A data byte received by the driver.

@Example void [main\(void\)](#) { initialize the device [SYSTEM_Initialize\(\)](#); uint8_t data;

Enable the Global Interrupts [INTERRUPT_GlobalInterruptEnable\(\)](#);

Enable the Peripheral Interrupts [INTERRUPT_PeripheralInterruptEnable\(\)](#);

```
printf("\t\tTEST CODE\n\r"); //Enable redirect STDIO to USART before using
printf statements printf("\t\t---- ----\n\r"); printf("\t\tECHO TEST\n\r");
printf("\t\t---- ----\n\r"); printf("Enter any string: "); do{ data =
UART1\_Read\(\); // Read data received UART1\_Write(data); // Echo back the
data received }while(!UART1_DataReady); //check if any data is received

}
```

6.123.1.10 UART1_SetAddressToTransmit()

```
void UART1_SetAddressToTransmit (
    uint8_t txAddress)
```

@Summary Sets the driver's receiver address and mask

@Description This routine is sets the register UxP2L and UxP3L for Address receive mode

@Preconditions [UART1_Initialize\(\)](#) function should have been called for the ISR to execute correctly with $MODE<3:0> = 0100$

@Param None

@Returns None

6.123.1.11 UART1_SetErrorHandler()

```
void UART1_SetErrorHandler (
    void(* interruptHandler ) (void))
```

@Summary Set UART1 Error Handler

@Description This API sets the function to be called upon UART1 error

@Preconditions Initialize the UART1 module before calling this API

@Param Address of function to be set as error handler

@Returns None Here is the caller graph for this function:



6.123.1.12 UART1_SetFramingErrorHandler()

```
void UART1_SetFramingErrorHandler (
    void(* interruptHandler ) (void))
```

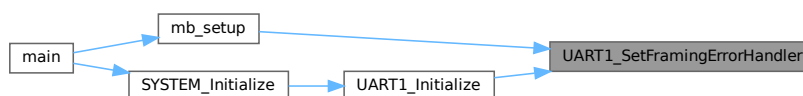
@Summary Set UART1 Framing Error Handler

@Description This API sets the function to be called upon UART1 framing error

@Preconditions Initialize the UART1 before calling this API

@Param Address of function to be set as framing error handler

@Returns None Here is the caller graph for this function:



6.123.1.13 UART1_SetOverrunErrorHandler()

```
void UART1_SetOverrunErrorHandler (
    void(* interruptHandler ) (void))
```

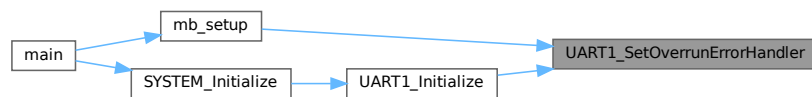
@Summary Set UART1 Overrun Error Handler

@Description This API sets the function to be called upon UART1 overrun error

@Preconditions Initialize the UART1 module before calling this API

@Param Address of function to be set as overrun error handler

@Returns None Here is the caller graph for this function:

**6.123.1.14 UART1_Write()**

```
void UART1_Write (
    uint8_t txData)
```

@Summary Writes a byte of data to the UART1.

@Description This routine writes a byte of data to the UART1.

@Preconditions [UART1_Initialize\(\)](#) function should have been called before calling this function. The transfer status should be checked to see if transmitter is not busy before calling this function.

@Param txData - Data byte to write to the UART1

@Returns None

@Example Refer to [UART1_Read\(\)](#) for an example
Here is the caller graph for this function:



6.123.2 Variable Documentation

6.123.2.1 UART1_ErrorHandler

```
void(* UART1_ErrorHandler) (void) (  
    void )
```

6.123.2.2 UART1_FramingErrorHandler

```
void(* UART1_FramingErrorHandler) (void) (  
    void )
```

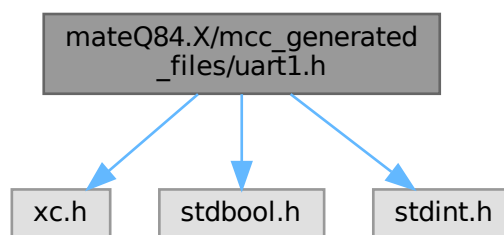
Section: UART1 APIs

6.123.2.3 UART1_OverrunErrorHandler

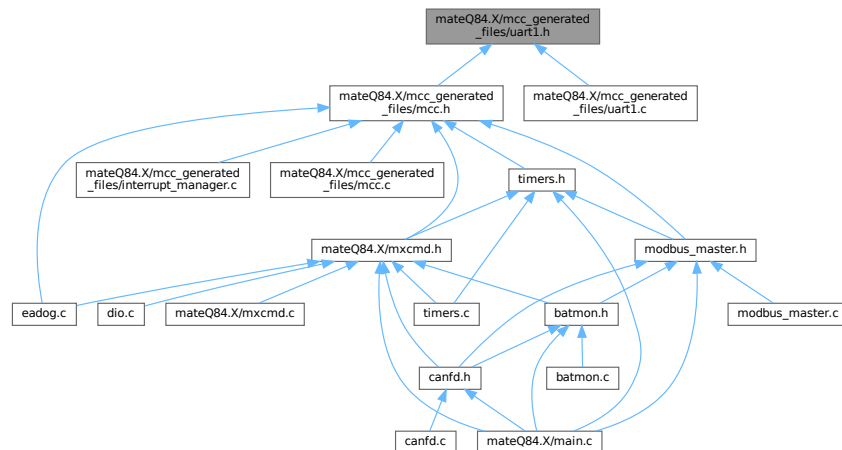
```
void(* UART1_OverrunErrorHandler) (void) (  
    void )
```

6.124 mateQ84.X/mcc_generated_files/uart1.h File Reference

```
#include <xc.h>  
#include <stdbool.h>  
#include <stdint.h>  
Include dependency graph for uart1.h:
```



This graph shows which files directly or indirectly include this file:



Data Structures

- union [uart1_status_t](#)

Macros

- `#define UART1\_DataReady (UART1\_is\_rx\_ready())`

Functions

- void [UART1_Initialize](#) (void)
- bool [UART1_is_rx_ready](#) (void)
- bool [UART1_is_tx_ready](#) (void)
- bool [UART1_is_tx_done](#) (void)
- [uart1_status_t](#) [UART1_get_last_status](#) (void)
- uint8_t [UART1_Read](#) (void)
- void [UART1_Write](#) (uint8_t txData)
- void [UART1_SetAddressToTransmit](#) (uint8_t txAddress)
- void [UART1_SetFramingErrorHandler](#) (void(*interruptHandler)(void))
- void [UART1_SetOverrunErrorHandler](#) (void(*interruptHandler)(void))
- void [UART1_SetErrorHandler](#) (void(*interruptHandler)(void))

6.124.1 Macro Definition Documentation

6.124.1.1 [UART1_DataReady](#)

```
#define UART1\_DataReady (UART1\_is\_rx\_ready())
```

UART1 Generated Driver API Header File

@Company Microchip Technology Inc.

@File Name [uart1.h](#)

@Summary This is the generated header file for the UART1 driver using PIC10 / PIC12 / PIC16 / PIC18 MCUs

@Description This header file provides APIs for driver for UART1. Generation Information : Product Revision : PIC10 / PIC12 / PIC16 / PIC18 MCUs - 1.81.8 Device : PIC18F47Q84 Driver Version : 2.4.1 The generated drivers are tested against the following: Compiler : XC8 2.36 and above MPLAB : MPLAB X 6.00 Section: Included Files Section: Macro Declarations

6.124.2 Function Documentation

6.124.2.1 UART1_get_last_status()

```
uart1_status_t UART1_get_last_status (
    void )
```

@Summary Gets the error status of the last read byte.

@Description This routine gets the error status of the last read byte.

@Preconditions [UART1_Initialize\(\)](#) function should have been called before calling this function. The returned value is only updated after a read is called.

@Param None

@Returns the status of the last read byte

@Example

```
void main(void) { volatile uint8_t rxData; volatile uart1_status_t rxStatus;
```

```
Initialize the device SYSTEM_Initialize();
```

```
Enable the Global Interrupts INTERRUPT_GlobalInterruptEnable();
```

```
while(1) { Logic to echo received data if(UART1_is_rx_ready()) { rxData =
UART1_Read(); rxStatus = UART1_get_last_status(); if(rxStatus.ferr){ LED_↔
0_SetHigh(); } } }
```

6.124.2.2 UART1_Initialize()

```
void UART1_Initialize (
    void )
```

Section: UART1 APIs @Summary Initialization routine that takes inputs from the UART1 GUI.

@Description This routine initializes the UART1 driver. This routine must be called before any other UART1 routine is called.

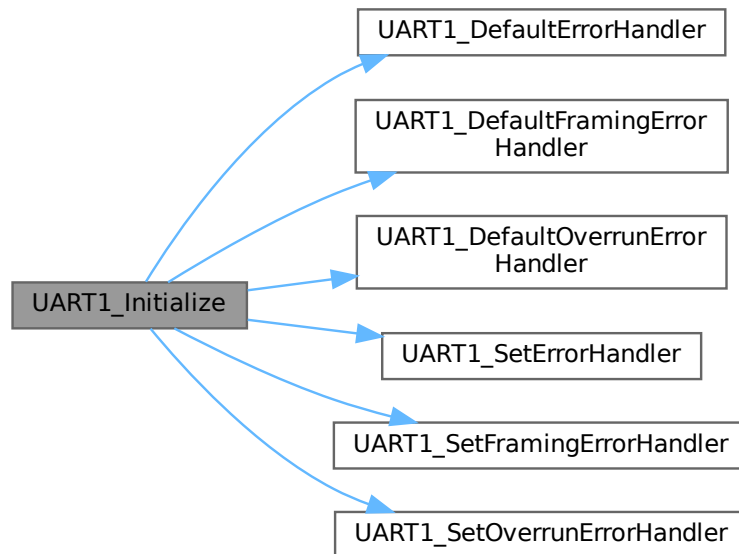
@Preconditions None

@Param None

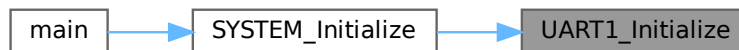
@Returns None

@Comment

@Example Here is the call graph for this function:



Here is the caller graph for this function:



6.124.2.3 UART1_is_rx_ready()

```
bool UART1_is_rx_ready (
    void )
```

@Summary Checks if the UART1 receiver ready for reading

@Description This routine checks if UART1 receiver has received data and ready to be read

@Preconditions `UART1_Initialize()` function should be called before calling this function UART1 receiver should be enabled before calling this function

@Param None

@Returns Status of UART1 receiver TRUE: UART1 receiver is ready for reading FALSE: UART1 receiver is not ready for reading

@Example void `main(void)` { volatile uint8_t rxData;

Initialize the device `SYSTEM_Initialize()`;

```
while(1) { Logic to echo received data if(UART1_is_rx_ready()) { rxData =
UART1_Read(); if(UART1_is_tx_ready()) { UART1_Write(rxData); } } }
```

6.124.2.4 UART1_is_tx_done()

```
bool UART1_is_tx_done (
    void )
```

@Summary Checks if UART1 data is transmitted

@Description This function return the status of transmit shift register

@Preconditions [UART1_Initialize\(\)](#) function should be called before calling this function UART1 transmitter should be enabled and UART1_Write should be called before calling this function

@Param None

@Returns Status of UART1 transmit shift register TRUE: Data completely shifted out if the UART shift register
FALSE: Data is not completely shifted out of the shift register

@Example void [main\(void\)](#) { volatile uint8_t rxData;

Initialize the device [SYSTEM_Initialize\(\)](#);

```
while(1) { if(UART1_is_tx_ready()) { LED_0_SetHigh(); UART1Write(rxData); }
if(UART1\_is\_tx\_done\(\) { LED_0_SetLow(); } } }
```

6.124.2.5 UART1_is_tx_ready()

```
bool UART1_is_tx_ready (
    void )
```

@Summary Checks if the UART1 transmitter is ready to transmit data

@Description This routine checks if UART1 transmitter is ready to accept and transmit data byte

@Preconditions [UART1_Initialize\(\)](#) function should have been called before calling this function. UART1 transmitter should be enabled before calling this function

@Param None

@Returns Status of UART1 transmitter TRUE: UART1 transmitter is ready FALSE: UART1 transmitter is not ready

@Example void [main\(void\)](#) { volatile uint8_t rxData;

Initialize the device [SYSTEM_Initialize\(\)](#);

```
while(1) { Logic to echo received data if(UART1_is_rx_ready()) { rxData =
UART1\_Read\(\); if(UART1_is_tx_ready()) { UART1\_Write\(rxData\); } } } }
```


6.124.2.6 UART1_Read()

```
uint8_t UART1_Read (
    void )
```

@Summary Read a byte of data from the UART1.

@Description This routine reads a byte of data from the UART1.

@Preconditions [UART1_Initialize\(\)](#) function should have been called before calling this function. The transfer status should be checked to see if the receiver is not empty before calling this function.

UART1_DataReady is a macro which checks if any byte is received. Call this macro before using this function.

@Param None

@Returns A data byte received by the driver.

@Example

```
void main(void) { initialize the device SYSTEM\_Initialize\(\); uint8_t data;
```

```
Enable the Global Interrupts INTERRUPT_GlobalInterruptEnable();
```

```
Enable the Peripheral Interrupts INTERRUPT_PeripheralInterruptEnable();
```

```
printf("\t\tTEST CODE\n\r"); //Enable redirect STDIO to USART before using
printf statements printf("\t\t---- ----\n\r"); printf("\t\tECHO TEST\n\r");
printf("\t\t---- ----\n\r"); printf("Enter any string: "); do{ data =
UART1\_Read\(\); // Read data received UART1\_Write\(data\); // Echo back the
data received }while(!UART1_DataReady); //check if any data is received

}
```

6.124.2.7 UART1_SetAddressstoTransmit()

```
void UART1_SetAddressstoTransmit (
    uint8_t txAddress)
```

@Summary Sets the driver's receiver address and mask

@Description This routine is sets the register UxP2L and UxP3L for Address receive mode

@Preconditions [UART1_Initialize\(\)](#) function should have been called for the ISR to execute correctly with MODE<3:0> = 0100

@Param None

@Returns None

6.124.2.8 UART1_SetErrorHandler()

```
void UART1_SetErrorHandler (
    void(* interruptHandler ) (void))
```

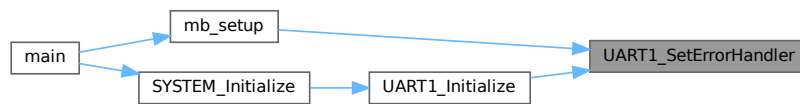
@Summary Set UART1 Error Handler

@Description This API sets the function to be called upon UART1 error

@Preconditions Initialize the UART1 module before calling this API

@Param Address of function to be set as error handler

@Returns None Here is the caller graph for this function:



6.124.2.9 UART1_SetFramingErrorHandler()

```
void UART1_SetFramingErrorHandler (
    void(* interruptHandler ) (void))
```

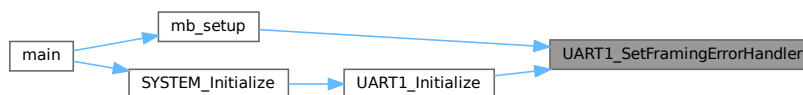
@Summary Set UART1 Framing Error Handler

@Description This API sets the function to be called upon UART1 framing error

@Preconditions Initialize the UART1 before calling this API

@Param Address of function to be set as framing error handler

@Returns None Here is the caller graph for this function:



6.124.2.10 UART1_SetOverrunErrorHandler()

```
void UART1_SetOverrunErrorHandler (
    void(* interruptHandler ) (void))
```

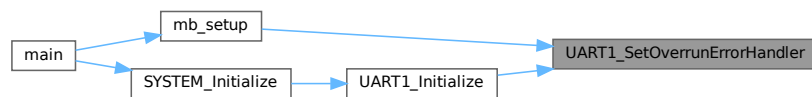
@Summary Set UART1 Overrun Error Handler

@Description This API sets the function to be called upon UART1 overrun error

@Preconditions Initialize the UART1 module before calling this API

@Param Address of function to be set as overrun error handler

@Returns None Here is the caller graph for this function:

**6.124.2.11 UART1_Write()**

```
void UART1_Write (
    uint8_t txData)
```

@Summary Writes a byte of data to the UART1.

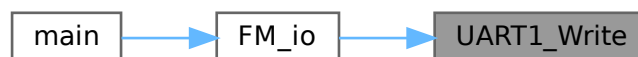
@Description This routine writes a byte of data to the UART1.

@Preconditions [UART1_Initialize\(\)](#) function should have been called before calling this function. The transfer status should be checked to see if transmitter is not busy before calling this function.

@Param txData - Data byte to write to the UART1

@Returns None

@Example Refer to [UART1_Read\(\)](#) for an example
Here is the caller graph for this function:



6.125 uart1.h

[Go to the documentation of this file.](#)

```

00001
00023
00024 /*
00025      (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027      Subject to your compliance with these terms, you may use Microchip software and any
00028      derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029      party
00029      license terms applicable to your use of third party software (including open source software) that
00030
00030      may accompany Microchip software.
00031
00032      THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033      EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034      IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035      FOR A PARTICULAR PURPOSE.
00036
00037      IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038      INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039      WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040      HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041      THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042      CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043      OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044      SOFTWARE.
00045 */
00046
00047 #ifndef UART1_H
00048 #define UART1_H
00049
00053
00054 #include <xc.h>
00055 #include <stdbool.h>
00056 #include <stdint.h>
00057
00058 #ifdef __cplusplus // Provide C++ Compatibility
00059
00060     extern "C" {
00061
00062 #endif
00063
00067
00068 #define UART1_DataReady (UART1_is_rx_ready())
00069
00073
00074 typedef union {
00075     struct {
00076         unsigned perr : 1;
00077         unsigned ferr : 1;
00078         unsigned oerr : 1;
00079         unsigned reserved : 5;
00080     };
00081     uint8_t status;
00082 }uart1_status_t;
00083
00084
00088
00110 void UART1_Initialize(void);
00111
00158 bool UART1_is_rx_ready(void);
00159
00206 bool UART1_is_tx_ready(void);
00207
00253 bool UART1_is_tx_done(void);
00254
00301 uart1_status_t UART1_get_last_status(void);
00302
00350 uint8_t UART1_Read(void);
00351
00375 void UART1_Write(uint8_t txData);
00376
00394 void UART1_SetAddressToTransmit(uint8_t txAddress);
00395
00396
00397
00414 void UART1_SetFramingErrorHandler(void (* interruptHandler)(void));
00415
00432 void UART1_SetOverrunErrorHandler(void (* interruptHandler)(void));
00433
00450 void UART1_SetErrorHandler(void (* interruptHandler)(void));
00451
00452

```

```

00453
00454
00455
00456
00457
00458
00459
00460
00461
00462 #ifdef __cplusplus // Provide C++ Compatibility
00463     }
00464 }
00465 #endif
00466 #endif
00467
00468 #endif // UART1_H

```

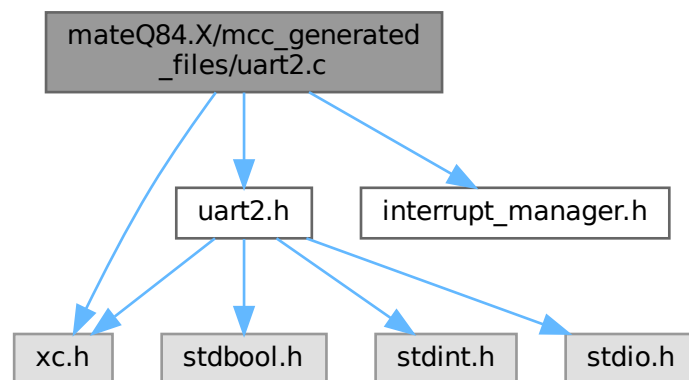
6.126 mateQ84.X/mcc_generated_files/uart2.c File Reference

```

#include <xc.h>
#include "uart2.h"
#include "interrupt_manager.h"

```

Include dependency graph for uart2.c:



Macros

- `#define IO_RB6_Toggle()`
- `#define TRACE`
- `#define INT_TRACE IO_RB6_Toggle()`
- `#define UART2_TX_BUFFER_SIZE 64`
- `#define UART2_RX_BUFFER_SIZE 64`

Functions

- `void UART2_DefaultFramingErrorHandler (void)`
- `void UART2_DefaultOverrunErrorHandler (void)`
- `void UART2_DefaultErrorHandler (void)`

- void [UART2_Initialize](#) (void)
- bool [UART2_is_rx_ready](#) (void)
- bool [UART2_is_tx_ready](#) (void)
- bool [UART2_is_tx_done](#) (void)
- [uart2_status_t](#) [UART2_get_last_status](#) (void)
- [uint8_t](#) [UART2_Read](#) (void)
- void [UART2_Write](#) ([uint8_t](#) txData)
- int [getch](#) (void)
- void [putch](#) (char txData)
- void [__interrupt](#) (irq(U2TX), base(8))
- void [__interrupt](#) (irq(U2RX), base(8))
- void [UART2_Transmit_ISR](#) (void)
- void [UART2_Receive_ISR](#) (void)
- void [UART2_RxDataHandler](#) (void)
- void [UART2_SetFramingErrorHandler](#) (void(*interruptHandler)(void))
- void [UART2_SetOverrunErrorHandler](#) (void(*interruptHandler)(void))
- void [UART2_SetErrorHandler](#) (void(*interruptHandler)(void))
- void [UART2_SetRxInterruptHandler](#) (void(*InterruptHandler)(void))
- void [UART2_SetTxInterruptHandler](#) (void(*InterruptHandler)(void))

Variables

- volatile [uint8_t](#) [uart2TxBufferRemaining](#)
- volatile [uint8_t](#) [uart2RxCount](#)
- void(* [UART2_FramingErrorHandler](#))(void)
- void(* [UART2_OverrunErrorHandler](#))(void)
- void(* [UART2_ErrorHandler](#))(void)

6.126.1 Macro Definition Documentation

6.126.1.1 INT_TRACE

```
#define INT_TRACE IO\_RB6\_Toggle()
```

6.126.1.2 IO_RB6_Toggle

```
#define IO\_RB6\_Toggle()
```

Value:

```
do { LATBbits.LATB6 = ~LATBbits.LATB6; } while(0)
```

UART2 Generated Driver File

@Company Microchip Technology Inc.

@File Name [uart2.c](#)

@Summary This is the generated driver implementation file for the UART2 driver using PIC10 / PIC12 / PIC16 / PIC18 MCUs

@Description This source file provides APIs for UART2. Generation Information : Product Revision : PIC10 / PIC12 / PIC16 / PIC18 MCUs - 1.81.8 Device : PIC18F47Q84 Driver Version : 2.4.1 The generated drivers are tested against the following: Compiler : XC8 2.36 and above MPLAB : MPLAB X 6.00 Section: Included Files

6.126.1.3 TRACE

```
#define TRACE
```

6.126.1.4 UART2_RX_BUFFER_SIZE

```
#define UART2_RX_BUFFER_SIZE 64
```

6.126.1.5 UART2_TX_BUFFER_SIZE

```
#define UART2_TX_BUFFER_SIZE 64
```

Section: Macro Declarations

6.126.2 Function Documentation

6.126.2.1 __interrupt() [1/2]

```
void __interrupt (
    irq(U2RX) ,
    base(8) )
```

6.126.2.2 __interrupt() [2/2]

```
void __interrupt (
    irq(U2TX) ,
    base(8) )
```

6.126.2.3 getch()

```
int getch (
    void )
```

Here is the call graph for this function:



6.126.2.4 `putc()`

```
void putc (  
    char txData)
```

Here is the call graph for this function:



6.126.2.5 `UART2_DefaultErrorHandler()`

```
void UART2_DefaultErrorHandler (  
    void )
```

Here is the call graph for this function:



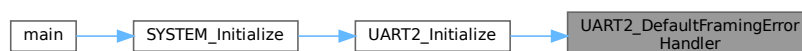
Here is the caller graph for this function:



6.126.2.6 `UART2_DefaultFramingErrorHandler()`

```
void UART2_DefaultFramingErrorHandler (  
    void )
```

Here is the caller graph for this function:



6.126.2.7 UART2_DefaultOverrunErrorHandler()

```
void UART2_DefaultOverrunErrorHandler (
    void )
```

Here is the caller graph for this function:



6.126.2.8 UART2_get_last_status()

```
uart2_status_t UART2_get_last_status (
    void )
```

@Summary Gets the error status of the last read byte.

@Description This routine gets the error status of the last read byte.

@Preconditions [UART2_Initialize\(\)](#) function should have been called before calling this function. The returned value is only updated after a read is called.

@Param None

@Returns the status of the last read byte

@Example

```
void main(void) { volatile uint8_t rxData; volatile uart2_status_t rxStatus;
```

Initialize the device `SYSTEM_Initialize();`

Enable the Global Interrupts `INTERRUPT_GlobalInterruptEnable();`

```
while(1) { Logic to echo received data if(UART2_is_rx_ready()) { rxData =
UART2_Read(); rxStatus = UART2_get_last_status(); if(rxStatus.ferr){ LED_↔
0_SetHigh(); } } }
```

6.126.2.9 UART2_Initialize()

```
void UART2_Initialize (
    void )
```

Section: UART2 APIs @Summary Initialization routine that takes inputs from the UART2 GUI.

@Description This routine initializes the UART2 driver. This routine must be called before any other UART2 routine is called.

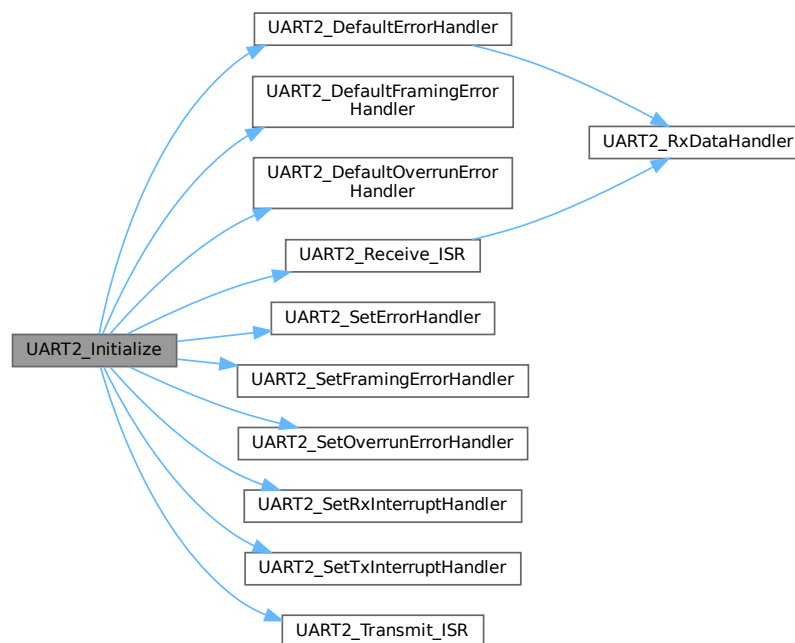
@Preconditions None

@Param None

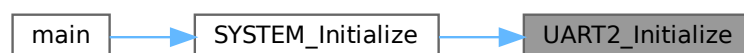
@Returns None

@Comment

@Example Here is the call graph for this function:



Here is the caller graph for this function:



6.126.2.10 UART2_is_rx_ready()

```
bool UART2_is_rx_ready (
    void )
```

@Summary Checks if the UART2 receiver ready for reading

@Description This routine checks if UART2 receiver has received data and ready to be read

@Preconditions [UART2_Initialize\(\)](#) function should be called before calling this function UART2 receiver should be enabled before calling this function

@Param None

@Returns Status of UART2 receiver TRUE: UART2 receiver is ready for reading FALSE: UART2 receiver is not ready for reading

@Example void [main\(void\)](#) { volatile uint8_t rxData;

Initialize the device [SYSTEM_Initialize\(\)](#);

while(1) { Logic to echo received data if(UART2_is_rx_ready()) { rxData = [UART2_Read\(\)](#); if(UART2_is_tx_ready()) { [UART2_Write\(rxData\)](#); } } } } Here is the caller graph for this function:

**6.126.2.11 UART2_is_tx_done()**

```
bool UART2_is_tx_done (
    void )
```

@Summary Checks if UART2 data is transmitted

@Description This function return the status of transmit shift register

@Preconditions [UART2_Initialize\(\)](#) function should be called before calling this function UART2 transmitter should be enabled and [UART2_Write](#) should be called before calling this function

@Param None

@Returns Status of UART2 transmit shift register TRUE: Data completely shifted out if the UART shift register FALSE: Data is not completely shifted out of the shift register

@Example void [main\(void\)](#) { volatile uint8_t rxData;

Initialize the device [SYSTEM_Initialize\(\)](#);

```
while(1) { if(UART2_is_tx_ready()) { LED_0_SetHigh(); UART2Write\(rxData\); }
if(UART2\_is\_tx\_done\(\)) { LED_0_SetLow(); } } }
```

6.126.2.12 UART2_is_tx_ready()

```
bool UART2_is_tx_ready (
    void )
```

@Summary Checks if the UART2 transmitter is ready to transmit data

@Description This routine checks if UART2 transmitter is ready to accept and transmit data byte

@Preconditions [UART2_Initialize\(\)](#) function should have been called before calling this function. UART2 transmitter should be enabled before calling this function

@Param None

@Returns Status of UART2 transmitter TRUE: UART2 transmitter is ready FALSE: UART2 transmitter is not ready

@Example void [main\(void\)](#) { volatile uint8_t rxData;

Initialize the device [SYSTEM_Initialize\(\)](#);

```
while(1) { Logic to echo received data if(UART2_is_rx_ready()) { rxData =
UART2\_Read\(\); if(UART2_is_tx_ready()) { UART2\_Write(rxData); } } }
```

6.126.2.13 UART2_Read()

```
uint8_t UART2_Read (
    void )
```

@Summary Read a byte of data from the UART2.

@Description This routine reads a byte of data from the UART2.

@Preconditions [UART2_Initialize\(\)](#) function should have been called before calling this function. The transfer status should be checked to see if the receiver is not empty before calling this function.

UART2_DataReady is a macro which checks if any byte is received. Call this macro before using this function.

@Param None

@Returns A data byte received by the driver.

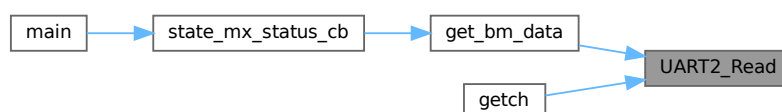
@Example void [main\(void\)](#) { initialize the device [SYSTEM_Initialize\(\)](#); uint8↵
_t data;

Enable the Global Interrupts [INTERRUPT_GlobalInterruptEnable\(\)](#);

Enable the Peripheral Interrupts [INTERRUPT_PeripheralInterruptEnable\(\)](#);

```
printf("\t\tTEST CODE\n\r"); //Enable redirect STDIO to USART before using
printf statements printf("\t\t---- ----\n\r"); printf("\t\tECHO TEST\n\r");
printf("\t\t---- ----\n\r"); printf("Enter any string: "); do{ data =
UART2\_Read\(\); // Read data received UART2\_Write(data); // Echo back the
data received }while(!UART2\_DataReady); //check if any data is received
```

} Here is the caller graph for this function:



6.126.2.14 UART2_Receive_ISR()

```
void UART2_Receive_ISR (
    void )
```

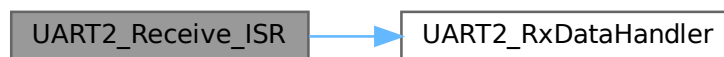
@Summary Maintains the driver's receiver state machine and implements its ISR

@Description This routine is used to maintain the driver's internal receiver state machine. This interrupt service routine is called when the state of the receiver needs to be maintained in a non polled manner.

@Preconditions [UART2_Initialize\(\)](#) function should have been called for the ISR to execute correctly.

@Param None

@Returns None Here is the call graph for this function:



Here is the caller graph for this function:

**6.126.2.15 UART2_RxDataHandler()**

```
void UART2_RxDataHandler (
    void )
```

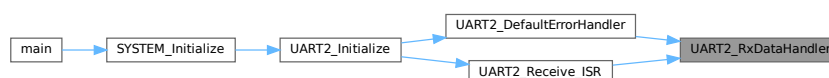
@Summary Maintains the driver's receiver state machine

@Description This routine is called by the receive state routine and is used to maintain the driver's internal receiver state machine. It should be called by a custom ISR to maintain normal behavior

@Preconditions [UART2_Initialize\(\)](#) function should have been called for the ISR to execute correctly.

@Param None

@Returns None Here is the caller graph for this function:



6.126.2.16 UART2_SetErrorHandler()

```
void UART2_SetErrorHandler (
    void(* interruptHandler ) (void))
```

@Summary Set UART2 Error Handler

@Description This API sets the function to be called upon UART2 error

@Preconditions Initialize the UART2 module before calling this API

@Param Address of function to be set as error handler

@Returns None Here is the caller graph for this function:



6.126.2.17 UART2_SetFramingErrorHandler()

```
void UART2_SetFramingErrorHandler (
    void(* interruptHandler ) (void))
```

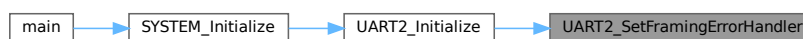
@Summary Set UART2 Framing Error Handler

@Description This API sets the function to be called upon UART2 framing error

@Preconditions Initialize the UART2 before calling this API

@Param Address of function to be set as framing error handler

@Returns None Here is the caller graph for this function:



6.126.2.18 UART2_SetOverrunErrorHandler()

```
void UART2_SetOverrunErrorHandler (
    void(* interruptHandler ) (void))
```

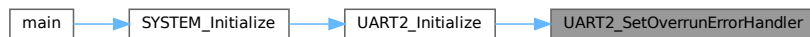
@Summary Set UART2 Overrun Error Handler

@Description This API sets the function to be called upon UART2 overrun error

@Preconditions Initialize the UART2 module before calling this API

@Param Address of function to be set as overrun error handler

@Returns None Here is the caller graph for this function:



6.126.2.19 UART2_SetRxInterruptHandler()

```
void UART2_SetRxInterruptHandler (
    void(* InterruptHandler ) (void))
```

@Summary Set UART2 Receive Interrupt Handler

@Description This API sets the function to be called upon UART2 receive interrupt

@Preconditions Initialize the UART2 module with receive interrupt enabled before calling this API

@Param Address of function to be set as receive interrupt handler

@Returns None Here is the caller graph for this function:



6.126.2.20 UART2_SetTxInterruptHandler()

```
void UART2_SetTxInterruptHandler (
    void(* InterruptHandler ) (void))
```

@Summary Set UART2 Transmit Interrupt Handler

@Description This API sets the function to be called upon UART2 transmit interrupt

@Preconditions Initialize the UART2 module with transmit interrupt enabled before calling this API

@Param Address of function to be set as transmit interrupt handler

@Returns None Here is the caller graph for this function:



6.126.2.21 UART2_Transmit_ISR()

```
void UART2_Transmit_ISR (  
    void )
```

@Summary Maintains the driver's transmitter state machine and implements its ISR.

@Description This routine is used to maintain the driver's internal transmitter state machine. This interrupt service routine is called when the state of the transmitter needs to be maintained in a non polled manner.

@Preconditions [UART2_Initialize\(\)](#) function should have been called for the ISR to execute correctly.

@Param None

@Returns None Here is the caller graph for this function:



6.126.2.22 UART2_Write()

```
void UART2_Write (  
    uint8_t txData)
```

@Summary Writes a byte of data to the UART2.

@Description This routine writes a byte of data to the UART2.

@Preconditions [UART2_Initialize\(\)](#) function should have been called before calling this function. The transfer status should be checked to see if transmitter is not busy before calling this function.

@Param txData - Data byte to write to the UART2

@Returns None

@Example Refer to [UART2_Read\(\)](#) for an example
Here is the caller graph for this function:



6.126.3 Variable Documentation

6.126.3.1 UART2_ErrorHandler

```
void(* UART2_ErrorHandler) (void) (  
    void )
```

6.126.3.2 UART2_FramingErrorHandler

```
void(* UART2_FramingErrorHandler) (void) (  
    void )
```

Section: UART2 APIs

6.126.3.3 UART2_OverrunErrorHandler

```
void(* UART2_OverrunErrorHandler) (void) (  
    void )
```

6.126.3.4 uart2RxCount

```
volatile uint8_t uart2RxCount
```

6.126.3.5 uart2TxBufferRemaining

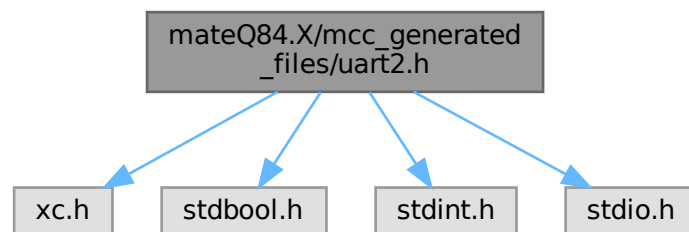
```
volatile uint8_t uart2TxBufferRemaining
```

Section: Global variables

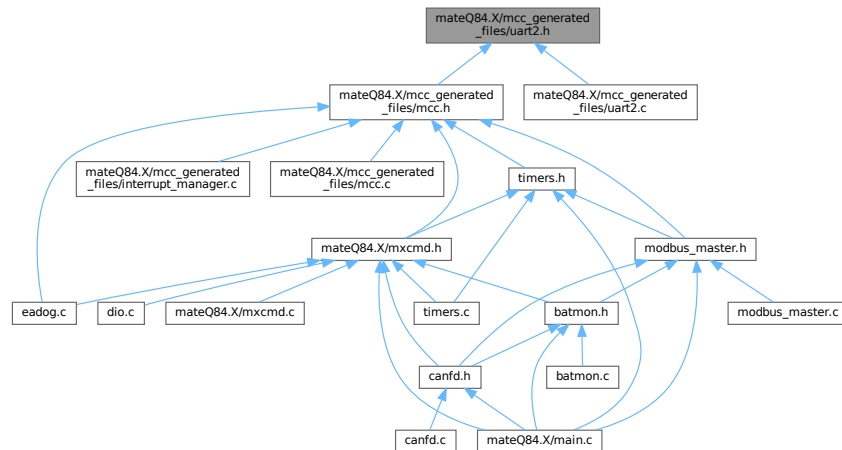
6.127 mateQ84.X/mcc_generated_files/uart2.h File Reference

```
#include <xc.h>  
#include <stdbool.h>  
#include <stdint.h>  
#include <stdio.h>
```

Include dependency graph for uart2.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- union [uart2_status_t](#)

Macros

- `#define` [UART2_DataReady](#) ([UART2_is_rx_ready](#)())

Functions

- void [UART2_Initialize](#) (void)
- bool [UART2_is_rx_ready](#) (void)
- bool [UART2_is_tx_ready](#) (void)
- bool [UART2_is_tx_done](#) (void)
- [uart2_status_t](#) [UART2_get_last_status](#) (void)
- uint8_t [UART2_Read](#) (void)
- void [UART2_Write](#) (uint8_t txData)
- void [UART2_Transmit_ISR](#) (void)
- void [UART2_Receive_ISR](#) (void)
- void [UART2_RxDataHandler](#) (void)
- void [UART2_SetFramingErrorHandler](#) (void(*interruptHandler)(void))
- void [UART2_SetOverrunErrorHandler](#) (void(*interruptHandler)(void))
- void [UART2_SetErrorHandler](#) (void(*interruptHandler)(void))
- void [UART2_SetRxInterruptHandler](#) (void(*InterruptHandler)(void))
- void [UART2_SetTxInterruptHandler](#) (void(*InterruptHandler)(void))

Variables

- volatile uint8_t [uart2TxBufferRemaining](#)
- volatile uint8_t [uart2RxCount](#)
- void(* [UART2_RxInterruptHandler](#))(void)
- void(* [UART2_TxInterruptHandler](#))(void)

6.127.1 Macro Definition Documentation

6.127.1.1 UART2_DataReady

```
#define UART2_DataReady (UART2_is_rx_ready())
```

UART2 Generated Driver API Header File

@Company Microchip Technology Inc.

@File Name [uart2.h](#)

@Summary This is the generated header file for the UART2 driver using PIC10 / PIC12 / PIC16 / PIC18 MCUs

@Description This header file provides APIs for driver for UART2. Generation Information : Product Revision : PIC10 / PIC12 / PIC16 / PIC18 MCUs - 1.81.8 Device : PIC18F47Q84 Driver Version : 2.4.1 The generated drivers are tested against the following: Compiler : XC8 2.36 and above MPLAB : MPLAB X 6.00 Section: Included Files Section: Macro Declarations

6.127.2 Function Documentation

6.127.2.1 UART2_get_last_status()

```
uart2_status_t UART2_get_last_status (
    void )
```

@Summary Gets the error status of the last read byte.

@Description This routine gets the error status of the last read byte.

@Preconditions [UART2_Initialize\(\)](#) function should have been called before calling this function. The returned value is only updated after a read is called.

@Param None

@Returns the status of the last read byte

@Example

```
void main(void) { volatile uint8_t rxData; volatile uart2_status_t rxStatus;
```

Initialize the device [SYSTEM_Initialize\(\)](#);

Enable the Global Interrupts [INTERRUPT_GlobalInterruptEnable\(\)](#);

```
while(1) { Logic to echo received data if(UART2_is_rx_ready()) { rxData =
UART2_Read(); rxStatus = UART2_get_last_status(); if(rxStatus.ferr){ LED_↔
0_SetHigh(); } } }
```

6.127.2.2 UART2_Initialize()

```
void UART2_Initialize (
    void )
```

Section: UART2 APIs @Summary Initialization routine that takes inputs from the UART2 GUI.

@Description This routine initializes the UART2 driver. This routine must be called before any other UART2 routine is called.

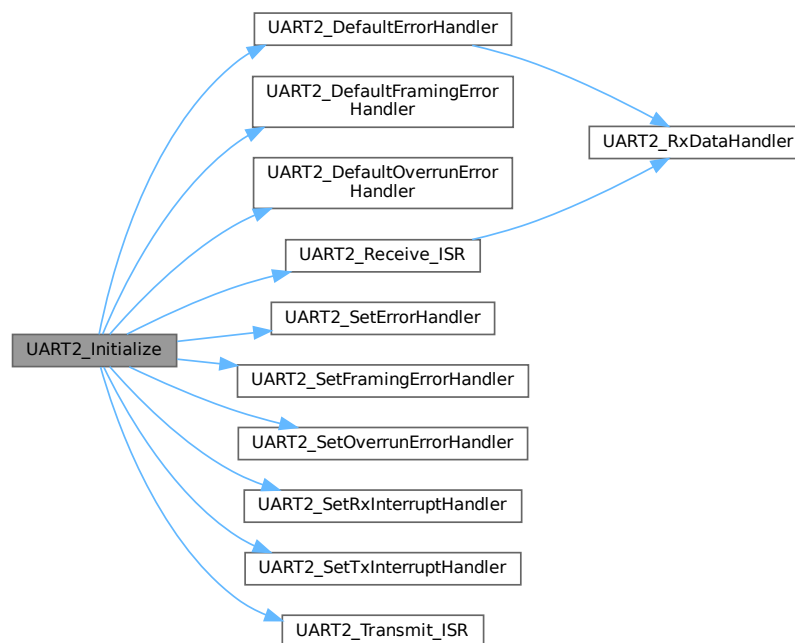
@Preconditions None

@Param None

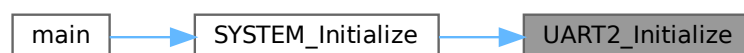
@Returns None

@Comment

@Example Here is the call graph for this function:



Here is the caller graph for this function:



6.127.2.3 UART2_is_rx_ready()

```
bool UART2_is_rx_ready (
    void )
```

@Summary Checks if the UART2 receiver ready for reading

@Description This routine checks if UART2 receiver has received data and ready to be read

@Preconditions [UART2_Initialize\(\)](#) function should be called before calling this function UART2 receiver should be enabled before calling this function

@Param None

@Returns Status of UART2 receiver TRUE: UART2 receiver is ready for reading FALSE: UART2 receiver is not ready for reading

@Example void [main\(void\)](#) { volatile uint8_t rxData;

Initialize the device [SYSTEM_Initialize\(\)](#);

while(1) { Logic to echo received data if(UART2_is_rx_ready()) { rxData = [UART2_Read\(\)](#); if(UART2_is_tx_ready()) { [UART2_Write\(rxData\)](#); } } } } Here is the caller graph for this function:



6.127.2.4 UART2_is_tx_done()

```
bool UART2_is_tx_done (
    void )
```

@Summary Checks if UART2 data is transmitted

@Description This function return the status of transmit shift register

@Preconditions [UART2_Initialize\(\)](#) function should be called before calling this function UART2 transmitter should be enabled and [UART2_Write](#) should be called before calling this function

@Param None

@Returns Status of UART2 transmit shift register TRUE: Data completely shifted out if the UART shift register FALSE: Data is not completely shifted out of the shift register

@Example void [main\(void\)](#) { volatile uint8_t rxData;

Initialize the device [SYSTEM_Initialize\(\)](#);

```
while(1) { if(UART2_is_tx_ready()) { LED_0_SetHigh(); UART2Write\(rxData\); }
if(UART2\_is\_tx\_done\(\)) { LED_0_SetLow(); } } }
```

6.127.2.5 UART2_is_tx_ready()

```
bool UART2_is_tx_ready (
    void )
```

@Summary Checks if the UART2 transmitter is ready to transmit data

@Description This routine checks if UART2 transmitter is ready to accept and transmit data byte

@Preconditions [UART2_Initialize\(\)](#) function should have been called before calling this function. UART2 transmitter should be enabled before calling this function

@Param None

@Returns Status of UART2 transmitter TRUE: UART2 transmitter is ready FALSE: UART2 transmitter is not ready

@Example void [main\(void\)](#) { volatile uint8_t rxData;

Initialize the device [SYSTEM_Initialize\(\)](#);

```
while(1) { Logic to echo received data if(UART2_is_rx_ready()) { rxData =
UART2\_Read\(\); if(UART2_is_tx_ready()) { UART2\_Write(rxData); } } }
```

6.127.2.6 UART2_Read()

```
uint8_t UART2_Read (
    void )
```

@Summary Read a byte of data from the UART2.

@Description This routine reads a byte of data from the UART2.

@Preconditions [UART2_Initialize\(\)](#) function should have been called before calling this function. The transfer status should be checked to see if the receiver is not empty before calling this function.

UART2_DataReady is a macro which checks if any byte is received. Call this macro before using this function.

@Param None

@Returns A data byte received by the driver.

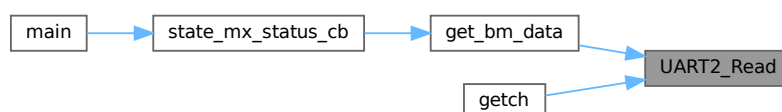
@Example void [main\(void\)](#) { initialize the device [SYSTEM_Initialize\(\)](#); uint8↵
_t data;

Enable the Global Interrupts [INTERRUPT_GlobalInterruptEnable\(\)](#);

Enable the Peripheral Interrupts [INTERRUPT_PeripheralInterruptEnable\(\)](#);

```
printf("\t\tTEST CODE\n\r"); //Enable redirect STDIO to USART before using
printf statements printf("\t\t---- ----\n\r"); printf("\t\tECHO TEST\n\r");
printf("\t\t---- ----\n\r"); printf("Enter any string: "); do{ data =
UART2\_Read\(\); // Read data received UART2\_Write(data); // Echo back the
data received }while(!UART2\_DataReady); //check if any data is received
```

} Here is the caller graph for this function:



6.127.2.7 UART2_Receive_ISR()

```
void UART2_Receive_ISR (
    void )
```

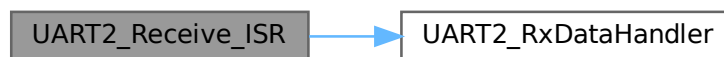
@Summary Maintains the driver's receiver state machine and implements its ISR

@Description This routine is used to maintain the driver's internal receiver state machine. This interrupt service routine is called when the state of the receiver needs to be maintained in a non polled manner.

@Preconditions [UART2_Initialize\(\)](#) function should have been called for the ISR to execute correctly.

@Param None

@Returns None Here is the call graph for this function:



Here is the caller graph for this function:

**6.127.2.8 UART2_RxDataHandler()**

```
void UART2_RxDataHandler (
    void )
```

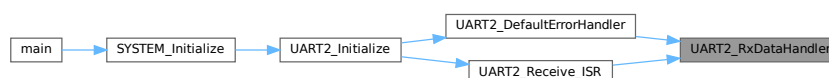
@Summary Maintains the driver's receiver state machine

@Description This routine is called by the receive state routine and is used to maintain the driver's internal receiver state machine. It should be called by a custom ISR to maintain normal behavior

@Preconditions [UART2_Initialize\(\)](#) function should have been called for the ISR to execute correctly.

@Param None

@Returns None Here is the caller graph for this function:



6.127.2.9 UART2_SetErrorHandler()

```
void UART2_SetErrorHandler (
    void(* interruptHandler ) (void))
```

@Summary Set UART2 Error Handler

@Description This API sets the function to be called upon UART2 error

@Preconditions Initialize the UART2 module before calling this API

@Param Address of function to be set as error handler

@Returns None Here is the caller graph for this function:



6.127.2.10 UART2_SetFramingErrorHandler()

```
void UART2_SetFramingErrorHandler (
    void(* interruptHandler ) (void))
```

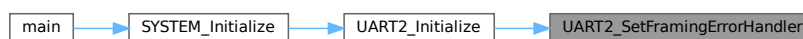
@Summary Set UART2 Framing Error Handler

@Description This API sets the function to be called upon UART2 framing error

@Preconditions Initialize the UART2 before calling this API

@Param Address of function to be set as framing error handler

@Returns None Here is the caller graph for this function:



6.127.2.11 UART2_SetOverrunErrorHandler()

```
void UART2_SetOverrunErrorHandler (
    void(* interruptHandler ) (void))
```

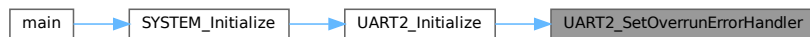
@Summary Set UART2 Overrun Error Handler

@Description This API sets the function to be called upon UART2 overrun error

@Preconditions Initialize the UART2 module before calling this API

@Param Address of function to be set as overrun error handler

@Returns None Here is the caller graph for this function:



6.127.2.12 UART2_SetRxInterruptHandler()

```
void UART2_SetRxInterruptHandler (
    void(* InterruptHandler ) (void))
```

@Summary Set UART2 Receive Interrupt Handler

@Description This API sets the function to be called upon UART2 receive interrupt

@Preconditions Initialize the UART2 module with receive interrupt enabled before calling this API

@Param Address of function to be set as receive interrupt handler

@Returns None Here is the caller graph for this function:



6.127.2.13 UART2_SetTxInterruptHandler()

```
void UART2_SetTxInterruptHandler (
    void(* InterruptHandler ) (void))
```

@Summary Set UART2 Transmit Interrupt Handler

@Description This API sets the function to be called upon UART2 transmit interrupt

@Preconditions Initialize the UART2 module with transmit interrupt enabled before calling this API

@Param Address of function to be set as transmit interrupt handler

@Returns None Here is the caller graph for this function:



6.127.2.14 UART2_Transmit_ISR()

```
void UART2_Transmit_ISR (  
    void )
```

@Summary Maintains the driver's transmitter state machine and implements its ISR.

@Description This routine is used to maintain the driver's internal transmitter state machine. This interrupt service routine is called when the state of the transmitter needs to be maintained in a non polled manner.

@Preconditions [UART2_Initialize\(\)](#) function should have been called for the ISR to execute correctly.

@Param None

@Returns None Here is the caller graph for this function:



6.127.2.15 UART2_Write()

```
void UART2_Write (  
    uint8_t txData)
```

@Summary Writes a byte of data to the UART2.

@Description This routine writes a byte of data to the UART2.

@Preconditions [UART2_Initialize\(\)](#) function should have been called before calling this function. The transfer status should be checked to see if transmitter is not busy before calling this function.

@Param txData - Data byte to write to the UART2

@Returns None

@Example Refer to [UART2_Read\(\)](#) for an example
Here is the caller graph for this function:



6.127.3 Variable Documentation

6.127.3.1 UART2_RxInterruptHandler

```
void(* UART2_RxInterruptHandler) (void) (  
    void )
```

@Summary UART2 Receive Interrupt Handler

@Description This is a pointer to the function that will be called upon UART2 receive interrupt

@Preconditions Initialize the UART2 module with receive interrupt enabled

@Param None

@Returns None

6.127.3.2 UART2_TxInterruptHandler

```
void(* UART2_TxInterruptHandler) (void) (  
    void )
```

@Summary UART2 Transmit Interrupt Handler

@Description This is a pointer to the function that will be called upon UART2 transmit interrupt

@Preconditions Initialize the UART2 module with transmit interrupt enabled

@Param None

@Returns None

6.127.3.3 uart2RxCount

```
volatile uint8_t uart2RxCount [extern]
```

6.127.3.4 uart2TxBufferRemaining

```
volatile uint8_t uart2TxBufferRemaining [extern]
```

Section: Global variables

6.128 uart2.h

[Go to the documentation of this file.](#)

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029     party
00029     license terms applicable to your use of third party software (including open source software) that
00030
00030     may accompany Microchip software.
00031
00032     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035     FOR A PARTICULAR PURPOSE.
00036
00037     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044     SOFTWARE.
00045 */
00046
00047 #ifndef UART2_H
00048 #define UART2_H
00049
00053
00054 #include <xc.h>
00055 #include <stdbool.h>
00056 #include <stdint.h>
00057 #include <stdio.h>
00058
00059 #ifdef __cplusplus // Provide C++ Compatibility
00060
00061     extern "C" {
00062
00063 #endif
00064
00068
00069 #define UART2_DataReady (UART2_is_rx_ready())
00070
00074
00075 typedef union {
00076     struct {
00077         unsigned perr : 1;
00078         unsigned ferr : 1;
00079         unsigned oerr : 1;
00080         unsigned reserved : 5;
00081     };
00082     uint8_t status;
00083 }uart2_status_t;
00084
00088 extern volatile uint8_t uart2TxBufferRemaining;
00089 extern volatile uint8_t uart2RxCount;
00090
00094
00116 void UART2_Initialize(void);
00117
00164 bool UART2_is_rx_ready(void);
00165
00212 bool UART2_is_tx_ready(void);
00213
00259 bool UART2_is_tx_done(void);
00260
00307 uart2_status_t UART2_get_last_status(void);
00308
00356 uint8_t UART2_Read(void);
00357
00381 void UART2_Write(uint8_t txData);
00382
00402 void UART2_Transmit_ISR(void);
00403
00423 void UART2_Receive_ISR(void);
00424
00444 void UART2_RxDataHandler(void);
00445
00462 void UART2_SetFramingErrorHandler(void (* interruptHandler)(void));
00463

```

```

00480 void UART2_SetOverrunErrorHandler(void (* interruptHandler)(void));
00481
00498 void UART2_SetErrorHandler(void (* interruptHandler)(void));
00499
00500
00501
00518 void (*UART2_RxInterruptHandler)(void);
00519
00536 void (*UART2_TxInterruptHandler)(void);
00537
00538
00539
00556 void UART2_SetRxInterruptHandler(void (* InterruptHandler)(void));
00557
00574 void UART2_SetTxInterruptHandler(void (* InterruptHandler)(void));
00575
00576
00577
00578 #ifdef __cplusplus // Provide C++ Compatibility
00579     }
00580 }
00581
00582 #endif
00583
00584 #endif // UART2_H

```

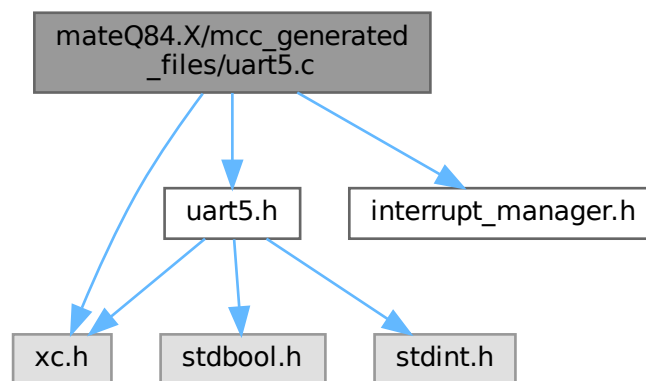
6.129 mateQ84.X/mcc_generated_files/uart5.c File Reference

```

#include <xc.h>
#include "uart5.h"
#include "interrupt_manager.h"

```

Include dependency graph for uart5.c:



Macros

- `#define IO_RB6_Toggle()`
- `#define TRACE`
- `#define INT_TRACE IO_RB6_Toggle()`
- `#define UART5_TX_BUFFER_SIZE 64`
- `#define UART5_RX_BUFFER_SIZE 8`

Functions

- void [UART5_DefaultFramingErrorHandler](#) (void)
- void [UART5_DefaultOverrunErrorHandler](#) (void)
- void [UART5_DefaultErrorHandler](#) (void)
- void [UART5_Initialize](#) (void)
- bool [UART5_is_rx_ready](#) (void)
- bool [UART5_is_tx_ready](#) (void)
- bool [UART5_is_tx_done](#) (void)
- [uart5_status_t](#) [UART5_get_last_status](#) (void)
- [uint8_t](#) [UART5_Read](#) (void)
- void [UART5_Write](#) ([uint8_t](#) txData)
- void [__interrupt](#) (irq(U5TX), base(8))
- void [__interrupt](#) (irq(U5RX), base(8))
- void [UART5_Transmit_ISR](#) (void)
- void [UART5_Receive_ISR](#) (void)
- void [UART5_RxDataHandler](#) (void)
- void [UART5_SetFramingErrorHandler](#) (void(*interruptHandler)(void))
- void [UART5_SetOverrunErrorHandler](#) (void(*interruptHandler)(void))
- void [UART5_SetErrorHandler](#) (void(*interruptHandler)(void))
- void [UART5_SetRxInterruptHandler](#) (void(*InterruptHandler)(void))
- void [UART5_SetTxInterruptHandler](#) (void(*InterruptHandler)(void))

Variables

- volatile [uint8_t](#) [uart5TxBufferRemaining](#)
- volatile [uint8_t](#) [uart5RxCount](#)
- void(* [UART5_FramingErrorHandler](#))(void)
- void(* [UART5_OverrunErrorHandler](#))(void)
- void(* [UART5_ErrorHandler](#))(void)

6.129.1 Macro Definition Documentation

6.129.1.1 INT_TRACE

```
#define INT_TRACE IO\_RB6\_Toggle()
```

6.129.1.2 IO_RB6_Toggle

```
#define IO\_RB6\_Toggle()
```

Value:

```
do { LATBbits.LATB6 = ~LATBbits.LATB6; } while (0)
```

UART5 Generated Driver File

@Company Microchip Technology Inc.

@File Name [uart5.c](#)

@Summary This is the generated driver implementation file for the UART5 driver using PIC10 / PIC12 / PIC16 / PIC18 MCUs

@Description This source file provides APIs for UART5. Generation Information : Product Revision : PIC10 / PIC12 / PIC16 / PIC18 MCUs - 1.81.8 Device : PIC18F47Q84 Driver Version : 2.4.1 The generated drivers are tested against the following: Compiler : XC8 2.36 and above MPLAB : MPLAB X 6.00 Section: Included Files

6.129.1.3 TRACE

```
#define TRACE
```

6.129.1.4 UART5_RX_BUFFER_SIZE

```
#define UART5_RX_BUFFER_SIZE 8
```

6.129.1.5 UART5_TX_BUFFER_SIZE

```
#define UART5_TX_BUFFER_SIZE 64
```

Section: Macro Declarations

6.129.2 Function Documentation

6.129.2.1 __interrupt() [1/2]

```
void __interrupt (
    irq(U5RX) ,
    base(8) )
```

6.129.2.2 __interrupt() [2/2]

```
void __interrupt (
    irq(U5TX) ,
    base(8) )
```

6.129.2.3 UART5_DefaultErrorHandler()

```
void UART5_DefaultErrorHandler (
    void )
```

Here is the call graph for this function:



Here is the caller graph for this function:



6.129.2.4 UART5_DefaultFramingErrorHandler()

```
void UART5_DefaultFramingErrorHandler (
    void )
```

Here is the caller graph for this function:



6.129.2.5 UART5_DefaultOverrunErrorHandler()

```
void UART5_DefaultOverrunErrorHandler (
    void )
```

Here is the caller graph for this function:



6.129.2.6 UART5_get_last_status()

```
uart5_status_t UART5_get_last_status (
    void )
```

@Summary Gets the error status of the last read byte.

@Description This routine gets the error status of the last read byte.

@Preconditions [UART5_Initialize\(\)](#) function should have been called before calling this function. The returned value is only updated after a read is called.

@Param None

@Returns the status of the last read byte

@Example void [main\(void\)](#) { volatile uint8_t rxData; volatile [uart5_status_t](#) rxStatus;

Initialize the device [SYSTEM_Initialize\(\)](#);

Enable the Global Interrupts [INTERRUPT_GlobalInterruptEnable\(\)](#);

```
while(1) { Logic to echo received data if(UART5_is_rx_ready()) { rxData =
UART5\_Read\(\); rxStatus = UART5\_get\_last\_status\(\); if(rxStatus.ferr){ LED_↔
0_SetHigh(); } } }
```


6.129.2.7 UART5_Initialize()

```
void UART5_Initialize (
    void )
```

Section: UART5 APIs @Summary Initialization routine that takes inputs from the UART5 GUI.

@Description This routine initializes the UART5 driver. This routine must be called before any other UART5 routine is called.

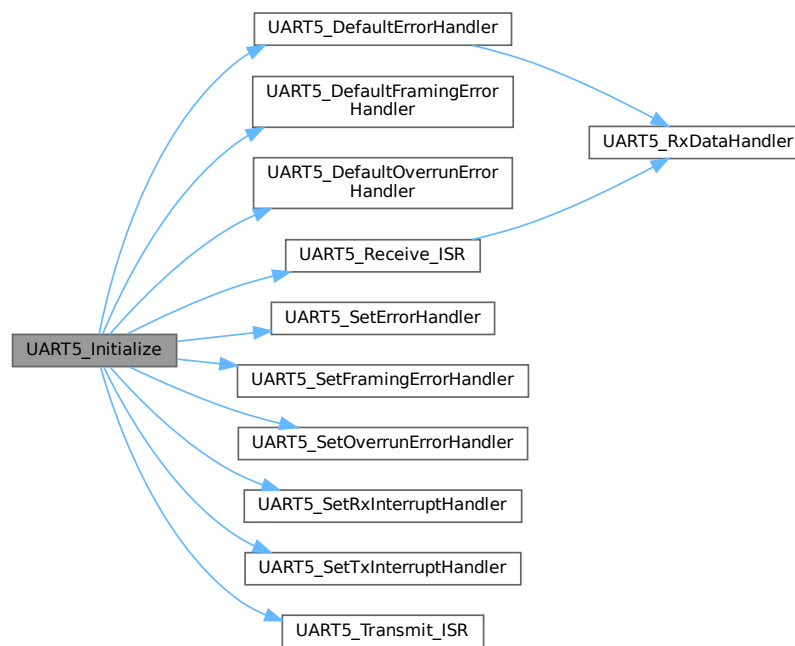
@Preconditions None

@Param None

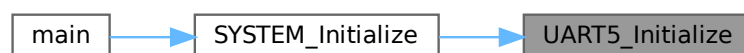
@Returns None

@Comment

@Example Here is the call graph for this function:



Here is the caller graph for this function:



6.129.2.8 UART5_is_rx_ready()

```
bool UART5_is_rx_ready (
    void )
```

@Summary Checks if the UART5 receiver ready for reading

@Description This routine checks if UART5 receiver has received data and ready to be read

@Preconditions [UART5_Initialize\(\)](#) function should be called before calling this function UART5 receiver should be enabled before calling this function

@Param None

@Returns Status of UART5 receiver TRUE: UART5 receiver is ready for reading FALSE: UART5 receiver is not ready for reading

@Example void [main\(void\)](#) { volatile uint8_t rxData;

Initialize the device [SYSTEM_Initialize\(\)](#);

```
while(1) { Logic to echo received data if(UART5_is_rx_ready()) { rxData =
UART5\_Read\(\); if(UART5_is_tx_ready()) { UART5\_Write\(rxData\); } } }
```

6.129.2.9 UART5_is_tx_done()

```
bool UART5_is_tx_done (
    void )
```

@Summary Checks if UART5 data is transmitted

@Description This function return the status of transmit shift register

@Preconditions [UART5_Initialize\(\)](#) function should be called before calling this function UART5 transmitter should be enabled and [UART5_Write](#) should be called before calling this function

@Param None

@Returns Status of UART5 transmit shift register TRUE: Data completely shifted out if the UART shift register FALSE: Data is not completely shifted out of the shift register

@Example void [main\(void\)](#) { volatile uint8_t rxData;

Initialize the device [SYSTEM_Initialize\(\)](#);

```
while(1) { if(UART5_is_tx_ready()) { LED\_0\_SetHigh\(\); UART5Write\(rxData\); }
if(UART5\_is\_tx\_done\(\)) { LED\_0\_SetLow\(\); } }
```

6.129.2.10 UART5_is_tx_ready()

```
bool UART5_is_tx_ready (
    void )
```

@Summary Checks if the UART5 transmitter is ready to transmit data

@Description This routine checks if UART5 transmitter is ready to accept and transmit data byte

@Preconditions [UART5_Initialize\(\)](#) function should have been called before calling this function. UART5 transmitter should be enabled before calling this function

@Param None

@Returns Status of UART5 transmitter TRUE: UART5 transmitter is ready FALSE: UART5 transmitter is not ready

@Example void [main\(void\)](#) { volatile uint8_t rxData;

Initialize the device [SYSTEM_Initialize\(\)](#);

while(1) { Logic to echo received data if(UART5_is_rx_ready()) { rxData = [UART5_Read\(\)](#); if(UART5_is_tx_ready()) { [UART5_Write\(rxData\)](#); } } } } Here is the caller graph for this function:

**6.129.2.11 UART5_Read()**

```
uint8_t UART5_Read (
    void )
```

@Summary Read a byte of data from the UART5.

@Description This routine reads a byte of data from the UART5.

@Preconditions [UART5_Initialize\(\)](#) function should have been called before calling this function. The transfer status should be checked to see if the receiver is not empty before calling this function.

UART5_DataReady is a macro which checks if any byte is received. Call this macro before using this function.

@Param None

@Returns A data byte received by the driver.

@Example void [main\(void\)](#) { initialize the device [SYSTEM_Initialize\(\)](#); uint8_t data;

Enable the Global Interrupts [INTERRUPT_GlobalInterruptEnable\(\)](#);

Enable the Peripheral Interrupts [INTERRUPT_PeripheralInterruptEnable\(\)](#);

```
printf("\t\tTEST CODE\n\r"); //Enable redirect STDIO to USART before using
printf statements printf("\t\t---- ----\n\r"); printf("\t\tECHO TEST\n\r");
printf("\t\t---- ----\n\r"); printf("Enter any string: "); do{ data =
UART5\_Read\(\); // Read data received UART5\_Write\(data\); // Echo back the
data received }while(!UART5_DataReady); //check if any data is received
}
```

6.129.2.12 UART5_Receive_ISR()

```
void UART5_Receive_ISR (
    void )
```

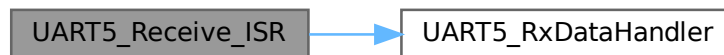
@Summary Maintains the driver's receiver state machine and implements its ISR

@Description This routine is used to maintain the driver's internal receiver state machine. This interrupt service routine is called when the state of the receiver needs to be maintained in a non polled manner.

@Preconditions [UART5_Initialize\(\)](#) function should have been called for the ISR to execute correctly.

@Param None

@Returns None Here is the call graph for this function:



Here is the caller graph for this function:



6.129.2.13 UART5_RxDataHandler()

```
void UART5_RxDataHandler (
    void )
```

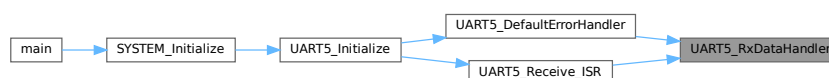
@Summary Maintains the driver's receiver state machine

@Description This routine is called by the receive state routine and is used to maintain the driver's internal receiver state machine. It should be called by a custom ISR to maintain normal behavior

@Preconditions [UART5_Initialize\(\)](#) function should have been called for the ISR to execute correctly.

@Param None

@Returns None Here is the caller graph for this function:



6.129.2.14 UART5_SetErrorHandler()

```
void UART5_SetErrorHandler (
    void(* interruptHandler ) (void))
```

@Summary Set UART5 Error Handler

@Description This API sets the function to be called upon UART5 error

@Preconditions Initialize the UART5 module before calling this API

@Param Address of function to be set as error handler

@Returns None Here is the caller graph for this function:



6.129.2.15 UART5_SetFramingErrorHandler()

```
void UART5_SetFramingErrorHandler (
    void(* interruptHandler ) (void))
```

@Summary Set UART5 Framing Error Handler

@Description This API sets the function to be called upon UART5 framing error

@Preconditions Initialize the UART5 before calling this API

@Param Address of function to be set as framing error handler

@Returns None Here is the caller graph for this function:



6.129.2.16 UART5_SetOverrunErrorHandler()

```
void UART5_SetOverrunErrorHandler (
    void(* interruptHandler ) (void))
```

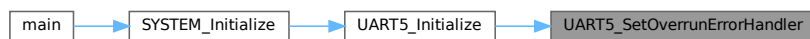
@Summary Set UART5 Overrun Error Handler

@Description This API sets the function to be called upon UART5 overrun error

@Preconditions Initialize the UART5 module before calling this API

@Param Address of function to be set as overrun error handler

@Returns None Here is the caller graph for this function:



6.129.2.17 UART5_SetRxInterruptHandler()

```
void UART5_SetRxInterruptHandler (
    void(* InterruptHandler ) (void))
```

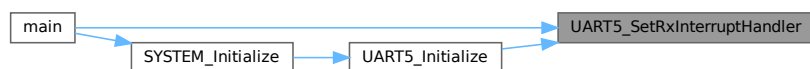
@Summary Set UART5 Receive Interrupt Handler

@Description This API sets the function to be called upon UART5 receive interrupt

@Preconditions Initialize the UART5 module with receive interrupt enabled before calling this API

@Param Address of function to be set as receive interrupt handler

@Returns None Here is the caller graph for this function:



6.129.2.18 UART5_SetTxInterruptHandler()

```
void UART5_SetTxInterruptHandler (
    void(* InterruptHandler ) (void))
```

@Summary Set UART5 Transmit Interrupt Handler

@Description This API sets the function to be called upon UART5 transmit interrupt

@Preconditions Initialize the UART5 module with transmit interrupt enabled before calling this API

@Param Address of function to be set as transmit interrupt handler

@Returns None Here is the caller graph for this function:

**6.129.2.19 UART5_Transmit_ISR()**

```
void UART5_Transmit_ISR (
    void )
```

@Summary Maintains the driver's transmitter state machine and implements its ISR.

@Description This routine is used to maintain the driver's internal transmitter state machine. This interrupt service routine is called when the state of the transmitter needs to be maintained in a non polled manner.

@Preconditions [UART5_Initialize\(\)](#) function should have been called for the ISR to execute correctly.

@Param None

@Returns None Here is the caller graph for this function:

**6.129.2.20 UART5_Write()**

```
void UART5_Write (
    uint8_t txData)
```

@Summary Writes a byte of data to the UART5.

@Description This routine writes a byte of data to the UART5.

@Preconditions [UART5_Initialize\(\)](#) function should have been called before calling this function. The transfer status should be checked to see if transmitter is not busy before calling this function.

@Param txData - Data byte to write to the UART5

@Returns None

@Example Refer to [UART5_Read\(\)](#) for an example

6.129.3 Variable Documentation

6.129.3.1 UART5_ErrorHandler

```
void(* UART5_ErrorHandler) (void) (  
    void )
```

6.129.3.2 UART5_FramingErrorHandler

```
void(* UART5_FramingErrorHandler) (void) (  
    void )
```

Section: UART5 APIs

6.129.3.3 UART5_OverrunErrorHandler

```
void(* UART5_OverrunErrorHandler) (void) (  
    void )
```

6.129.3.4 uart5RxCount

```
volatile uint8_t uart5RxCount
```

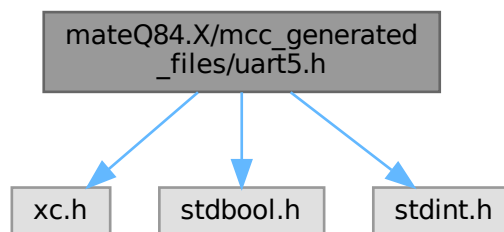
6.129.3.5 uart5TxBufferRemaining

```
volatile uint8_t uart5TxBufferRemaining
```

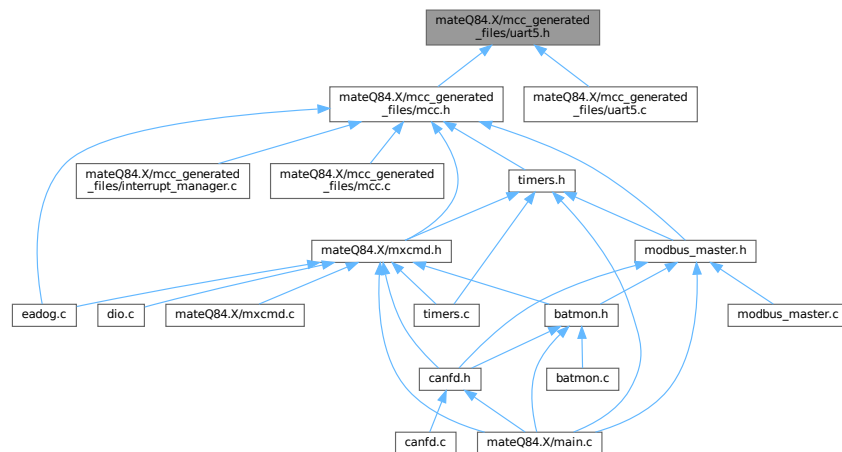
Section: Global variables

6.130 mateQ84.X/mcc_generated_files/uart5.h File Reference

```
#include <xc.h>  
#include <stdbool.h>  
#include <stdint.h>  
Include dependency graph for uart5.h:
```



This graph shows which files directly or indirectly include this file:



Data Structures

- union [uart5_status_t](#)

Macros

- `#define UART5_DataReady (UART5_is_rx_ready())`

Functions

- void [UART5_Initialize](#) (void)
- bool [UART5_is_rx_ready](#) (void)
- bool [UART5_is_tx_ready](#) (void)
- bool [UART5_is_tx_done](#) (void)
- [uart5_status_t](#) [UART5_get_last_status](#) (void)
- uint8_t [UART5_Read](#) (void)
- void [UART5_Write](#) (uint8_t txData)
- void [UART5_Transmit_ISR](#) (void)
- void [UART5_Receive_ISR](#) (void)
- void [UART5_RxDataHandler](#) (void)
- void [UART5_SetFramingErrorHandler](#) (void(*interruptHandler)(void))
- void [UART5_SetOverrunErrorHandler](#) (void(*interruptHandler)(void))
- void [UART5_SetErrorHandler](#) (void(*interruptHandler)(void))
- void [UART5_SetRxInterruptHandler](#) (void(*InterruptHandler)(void))
- void [UART5_SetTxInterruptHandler](#) (void(*InterruptHandler)(void))

Variables

- volatile uint8_t [uart5TxBufferRemaining](#)
- volatile uint8_t [uart5RxCount](#)
- void(* [UART5_RxInterruptHandler](#))(void)
- void(* [UART5_TxInterruptHandler](#))(void)

6.130.1 Macro Definition Documentation

6.130.1.1 UART5_DataReady

```
#define UART5_DataReady (UART5_is_rx_ready())
```

UART5 Generated Driver API Header File

@Company Microchip Technology Inc.

@File Name [uart5.h](#)

@Summary This is the generated header file for the UART5 driver using PIC10 / PIC12 / PIC16 / PIC18 MCUs

@Description This header file provides APIs for driver for UART5. Generation Information : Product Revision : PIC10 / PIC12 / PIC16 / PIC18 MCUs - 1.81.8 Device : PIC18F47Q84 Driver Version : 2.4.1 The generated drivers are tested against the following: Compiler : XC8 2.36 and above MPLAB : MPLAB X 6.00 Section: Included Files Section: Macro Declarations

6.130.2 Function Documentation

6.130.2.1 UART5_get_last_status()

```
uart5_status_t UART5_get_last_status (
    void )
```

@Summary Gets the error status of the last read byte.

@Description This routine gets the error status of the last read byte.

@Preconditions [UART5_Initialize\(\)](#) function should have been called before calling this function. The returned value is only updated after a read is called.

@Param None

@Returns the status of the last read byte

@Example

```
void main(void) { volatile uint8_t rxData; volatile uart5_status_t rxStatus;
```

Initialize the device [SYSTEM_Initialize\(\)](#);

Enable the Global Interrupts [INTERRUPT_GlobalInterruptEnable\(\)](#);

```
while(1) { Logic to echo received data if(UART5_is_rx_ready()) { rxData =
UART5_Read(); rxStatus = UART5_get_last_status(); if(rxStatus.ferr){ LED_↔
0_SetHigh(); } } }
```

6.130.2.2 UART5_Initialize()

```
void UART5_Initialize (
    void )
```

Section: UART5 APIs @Summary Initialization routine that takes inputs from the UART5 GUI.

@Description This routine initializes the UART5 driver. This routine must be called before any other UART5 routine is called.

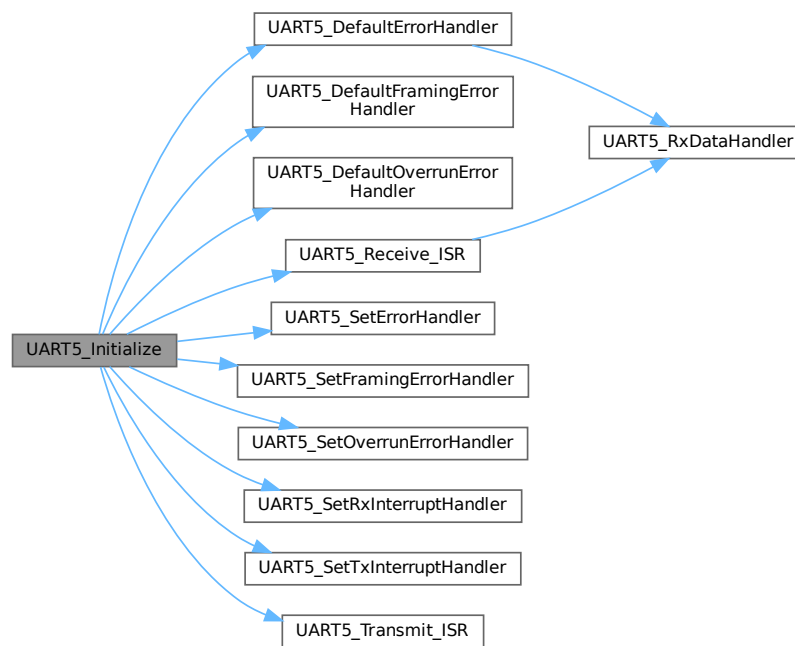
@Preconditions None

@Param None

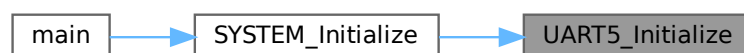
@Returns None

@Comment

@Example Here is the call graph for this function:



Here is the caller graph for this function:



6.130.2.3 UART5_is_rx_ready()

```
bool UART5_is_rx_ready (
    void )
```

@Summary Checks if the UART5 receiver ready for reading

@Description This routine checks if UART5 receiver has received data and ready to be read

@Preconditions [UART5_Initialize\(\)](#) function should be called before calling this function UART5 receiver should be enabled before calling this function

@Param None

@Returns Status of UART5 receiver TRUE: UART5 receiver is ready for reading FALSE: UART5 receiver is not ready for reading

@Example void [main\(void\)](#) { volatile uint8_t rxData;

Initialize the device [SYSTEM_Initialize\(\)](#);

```
while(1) { Logic to echo received data if(UART5_is_rx_ready()) { rxData =
UART5\_Read\(\); if(UART5_is_tx_ready()) { UART5\_Write\(rxData\); } } }
```

6.130.2.4 UART5_is_tx_done()

```
bool UART5_is_tx_done (
    void )
```

@Summary Checks if UART5 data is transmitted

@Description This function return the status of transmit shift register

@Preconditions [UART5_Initialize\(\)](#) function should be called before calling this function UART5 transmitter should be enabled and [UART5_Write](#) should be called before calling this function

@Param None

@Returns Status of UART5 transmit shift register TRUE: Data completely shifted out if the UART shift register FALSE: Data is not completely shifted out of the shift register

@Example void [main\(void\)](#) { volatile uint8_t rxData;

Initialize the device [SYSTEM_Initialize\(\)](#);

```
while(1) { if(UART5_is_tx_ready()) { LED\_0\_SetHigh\(\); UART5Write\(rxData\); }
if(UART5\_is\_tx\_done\(\)) { LED\_0\_SetLow\(\); } }
```

6.130.2.5 UART5_is_tx_ready()

```
bool UART5_is_tx_ready (
    void )
```

@Summary Checks if the UART5 transmitter is ready to transmit data

@Description This routine checks if UART5 transmitter is ready to accept and transmit data byte

@Preconditions [UART5_Initialize\(\)](#) function should have been called before calling this function. UART5 transmitter should be enabled before calling this function

@Param None

@Returns Status of UART5 transmitter TRUE: UART5 transmitter is ready FALSE: UART5 transmitter is not ready

@Example void [main\(void\)](#) { volatile uint8_t rxData;

Initialize the device [SYSTEM_Initialize\(\)](#);

while(1) { Logic to echo received data if(UART5_is_rx_ready()) { rxData = [UART5_Read\(\)](#); if(UART5_is_tx_ready()) { [UART5_Write\(rxData\)](#); } } } } Here is the caller graph for this function:



6.130.2.6 UART5_Read()

```
uint8_t UART5_Read (
    void )
```

@Summary Read a byte of data from the UART5.

@Description This routine reads a byte of data from the UART5.

@Preconditions [UART5_Initialize\(\)](#) function should have been called before calling this function. The transfer status should be checked to see if the receiver is not empty before calling this function.

UART5_DataReady is a macro which checks if any byte is received. Call this macro before using this function.

@Param None

@Returns A data byte received by the driver.

@Example void [main\(void\)](#) { initialize the device [SYSTEM_Initialize\(\)](#); uint8_t data;

Enable the Global Interrupts [INTERRUPT_GlobalInterruptEnable\(\)](#);

Enable the Peripheral Interrupts [INTERRUPT_PeripheralInterruptEnable\(\)](#);

```
printf("\t\tTEST CODE\n\r"); //Enable redirect STDIO to USART before using
printf statements printf("\t\t---- ----\n\r"); printf("\t\tECHO TEST\n\r");
printf("\t\t---- ----\n\r"); printf("Enter any string: "); do{ data =
UART5\_Read\(\); // Read data received UART5\_Write\(data\); // Echo back the
data received }while(!UART5\_DataReady\(\)); //check if any data is received
}
```

6.130.2.7 UART5_Receive_ISR()

```
void UART5_Receive_ISR (
    void )
```

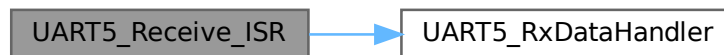
@Summary Maintains the driver's receiver state machine and implements its ISR

@Description This routine is used to maintain the driver's internal receiver state machine. This interrupt service routine is called when the state of the receiver needs to be maintained in a non polled manner.

@Preconditions [UART5_Initialize\(\)](#) function should have been called for the ISR to execute correctly.

@Param None

@Returns None Here is the call graph for this function:



Here is the caller graph for this function:



6.130.2.8 UART5_RxDataHandler()

```
void UART5_RxDataHandler (
    void )
```

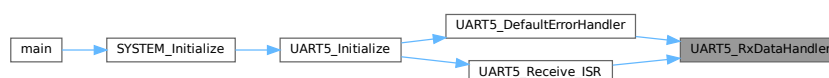
@Summary Maintains the driver's receiver state machine

@Description This routine is called by the receive state routine and is used to maintain the driver's internal receiver state machine. It should be called by a custom ISR to maintain normal behavior

@Preconditions [UART5_Initialize\(\)](#) function should have been called for the ISR to execute correctly.

@Param None

@Returns None Here is the caller graph for this function:



6.130.2.9 UART5_SetErrorHandler()

```
void UART5_SetErrorHandler (
    void(* interruptHandler ) (void))
```

@Summary Set UART5 Error Handler

@Description This API sets the function to be called upon UART5 error

@Preconditions Initialize the UART5 module before calling this API

@Param Address of function to be set as error handler

@Returns None Here is the caller graph for this function:



6.130.2.10 UART5_SetFramingErrorHandler()

```
void UART5_SetFramingErrorHandler (
    void(* interruptHandler ) (void))
```

@Summary Set UART5 Framing Error Handler

@Description This API sets the function to be called upon UART5 framing error

@Preconditions Initialize the UART5 before calling this API

@Param Address of function to be set as framing error handler

@Returns None Here is the caller graph for this function:



6.130.2.11 UART5_SetOverrunErrorHandler()

```
void UART5_SetOverrunErrorHandler (
    void(* interruptHandler ) (void))
```

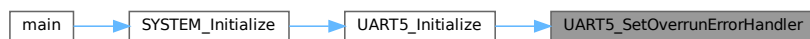
@Summary Set UART5 Overrun Error Handler

@Description This API sets the function to be called upon UART5 overrun error

@Preconditions Initialize the UART5 module before calling this API

@Param Address of function to be set as overrun error handler

@Returns None Here is the caller graph for this function:



6.130.2.12 UART5_SetRxInterruptHandler()

```
void UART5_SetRxInterruptHandler (
    void(* InterruptHandler ) (void))
```

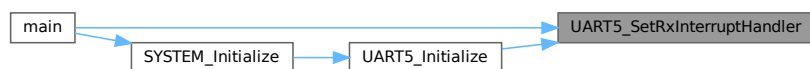
@Summary Set UART5 Receive Interrupt Handler

@Description This API sets the function to be called upon UART5 receive interrupt

@Preconditions Initialize the UART5 module with receive interrupt enabled before calling this API

@Param Address of function to be set as receive interrupt handler

@Returns None Here is the caller graph for this function:



6.130.2.13 UART5_SetTxInterruptHandler()

```
void UART5_SetTxInterruptHandler (
    void(* InterruptHandler ) (void))
```

@Summary Set UART5 Transmit Interrupt Handler

@Description This API sets the function to be called upon UART5 transmit interrupt

@Preconditions Initialize the UART5 module with transmit interrupt enabled before calling this API

@Param Address of function to be set as transmit interrupt handler

@Returns None Here is the caller graph for this function:

**6.130.2.14 UART5_Transmit_ISR()**

```
void UART5_Transmit_ISR (
    void )
```

@Summary Maintains the driver's transmitter state machine and implements its ISR.

@Description This routine is used to maintain the driver's internal transmitter state machine. This interrupt service routine is called when the state of the transmitter needs to be maintained in a non polled manner.

@Preconditions [UART5_Initialize\(\)](#) function should have been called for the ISR to execute correctly.

@Param None

@Returns None Here is the caller graph for this function:

**6.130.2.15 UART5_Write()**

```
void UART5_Write (
    uint8_t txData)
```

@Summary Writes a byte of data to the UART5.

@Description This routine writes a byte of data to the UART5.

@Preconditions [UART5_Initialize\(\)](#) function should have been called before calling this function. The transfer status should be checked to see if transmitter is not busy before calling this function.

@Param txData - Data byte to write to the UART5

@Returns None

@Example Refer to [UART5_Read\(\)](#) for an example

6.130.3 Variable Documentation

6.130.3.1 UART5_RxInterruptHandler

```
void(* UART5_RxInterruptHandler) (void) (  
    void )
```

@Summary UART5 Receive Interrupt Handler

@Description This is a pointer to the function that will be called upon UART5 receive interrupt

@Preconditions Initialize the UART5 module with receive interrupt enabled

@Param None

@Returns None

6.130.3.2 UART5_TxInterruptHandler

```
void(* UART5_TxInterruptHandler) (void) (  
    void )
```

@Summary UART5 Transmit Interrupt Handler

@Description This is a pointer to the function that will be called upon UART5 transmit interrupt

@Preconditions Initialize the UART5 module with transmit interrupt enabled

@Param None

@Returns None

6.130.3.3 uart5RxCount

```
volatile uint8_t uart5RxCount [extern]
```

6.130.3.4 uart5TxBufferRemaining

```
volatile uint8_t uart5TxBufferRemaining [extern]
```

Section: Global variables

6.131 uart5.h

[Go to the documentation of this file.](#)

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029     party
00029     license terms applicable to your use of third party software (including open source software) that
00030
00030     may accompany Microchip software.
00031
00032     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035     FOR A PARTICULAR PURPOSE.
00036
00037     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044     SOFTWARE.
00045 */
00046
00047 #ifndef UART5_H
00048 #define UART5_H
00049
00053
00054 #include <xc.h>
00055 #include <stdbool.h>
00056 #include <stdint.h>
00057
00058 #ifdef __cplusplus // Provide C++ Compatibility
00059
00060     extern "C" {
00061
00062 #endif
00063
00067
00068 #define UART5_DataReady (UART5_is_rx_ready())
00069
00073
00074 typedef union {
00075     struct {
00076         unsigned perr : 1;
00077         unsigned ferr : 1;
00078         unsigned oerr : 1;
00079         unsigned reserved : 5;
00080     };
00081     uint8_t status;
00082 }uart5_status_t;
00083
00087 extern volatile uint8_t uart5TxBufferRemaining;
00088 extern volatile uint8_t uart5RxCount;
00089
00093
00115 void UART5_Initialize(void);
00116
00163 bool UART5_is_rx_ready(void);
00164
00211 bool UART5_is_tx_ready(void);
00212
00258 bool UART5_is_tx_done(void);
00259
00306 uart5_status_t UART5_get_last_status(void);
00307
00355 uint8_t UART5_Read(void);
00356
00380 void UART5_Write(uint8_t txData);
00381
00401 void UART5_Transmit_ISR(void);
00402
00422 void UART5_Receive_ISR(void);
00423
00443 void UART5_RxDataHandler(void);
00444
00461 void UART5_SetFramingErrorHandler(void (* interruptHandler)(void));
00462
00479 void UART5_SetOverrunErrorHandler(void (* interruptHandler)(void));

```

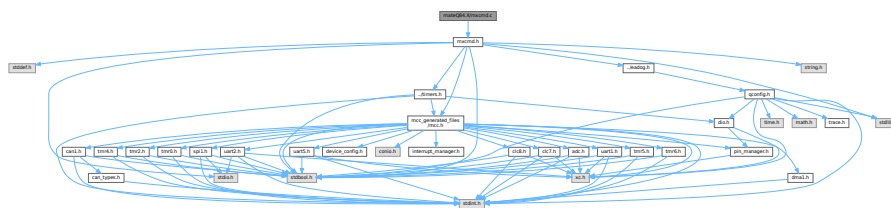
```

00480
00497 void UART5_SetErrorHandler(void (* interruptHandler)(void));
00498
00499
00500
00517 void (*UART5_RxInterruptHandler)(void);
00518
00535 void (*UART5_TxInterruptHandler)(void);
00536
00537
00538
00555 void UART5_SetRxInterruptHandler(void (* InterruptHandler)(void));
00556
00573 void UART5_SetTxInterruptHandler(void (* InterruptHandler)(void));
00574
00575
00576
00577 #ifdef __cplusplus // Provide C++ Compatibility
00578     }
00579 }
00580
00581 #endif
00582
00583 #endif // UART5_H

```

6.132 mateQ84.X/mxcmd.c File Reference

#include "mxcmd.h"
 Include dependency graph for mxcmd.c:



Functions

- void [FM_restart](#)(void)
- bool [FM_tx_empty](#)(void)
- uint8_t [FM_tx](#)(const uint16_t *data, const uint8_t count)
- void [FM_io](#)(void)
- uint8_t [FM_rx](#)(uint16_t *data)
- bool [FM_rx_ready](#)(void)
- uint8_t [FM_rx_count](#)(void)
- void [oneseq_io](#)(void)
- void [tenseq_io](#)(void)
- float [lp_filter](#)(const float new, const uint8_t bn, const int8_t slow)
- uint16_t [calc_checksum](#)(uint8_t *data, const uint8_t len)

6.132.1 Function Documentation

6.132.1.1 [calc_checksum\(\)](#)

```

uint16_t calc_checksum (
    uint8_t * data,
    const uint8_t len)

```

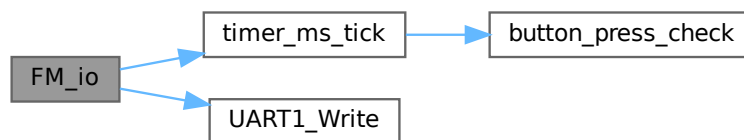
Here is the caller graph for this function:



6.132.1.2 FM_io()

```
void FM_io (  
    void )
```

Here is the call graph for this function:



Here is the caller graph for this function:



6.132.1.3 FM_restart()

```
void FM_restart (  
    void )
```

6.132.1.4 FM_rx()

```
uint8_t FM_rx (  
    uint16_t * data)
```

6.132.1.5 FM_rx_count()

```
uint8_t FM_rx_count (  
    void )
```

6.132.1.6 FM_rx_ready()

```
bool FM_rx_ready (  
    void )
```

6.132.1.7 FM_tx()

```
uint8_t FM_tx (  
    const uint16_t * data,  
    const uint8_t count)
```

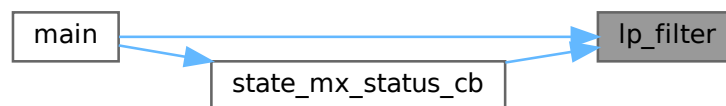
6.132.1.8 FM_tx_empty()

```
bool FM_tx_empty (  
    void )
```

6.132.1.9 lp_filter()

```
float lp_filter (  
    const float new,  
    const uint8_t bn,  
    const int8_t slow)
```

Here is the caller graph for this function:



6.132.1.10 onesecc_io()

```
void onesecc_io (
    void )
```

Here is the caller graph for this function:

**6.132.1.11 tensescc_io()**

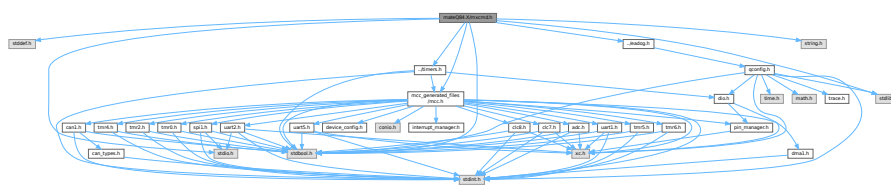
```
void tensescc_io (
    void )
```

Here is the caller graph for this function:

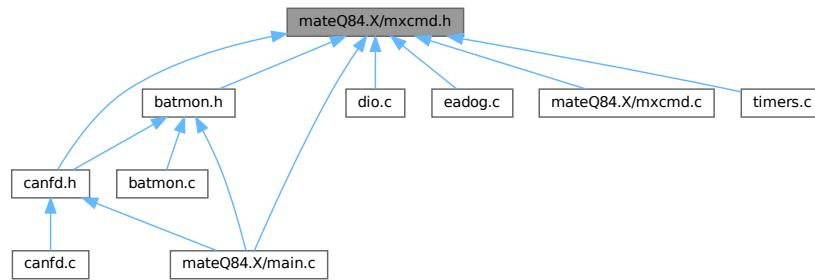
**6.133 mateQ84.X/mxcmd.h File Reference**

```
#include <stddef.h>
#include <stdbool.h>
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include "mcc_generated_files/mcc.h"
#include "../eadog.h"
#include "../timers.h"
```

Include dependency graph for mxcmd.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct [mx_status_packed_t](#)
- struct [mx_log_packed_t](#)
- struct [mx_logpage_t](#)
- struct [B_type](#)

Macros

- `#define VER 1`
- `#define MAX_B_BUF 512`
- `#define MAX_C_BUF CANFD_BYTES*3`
- `#define IO_TEST`
- `#define FM_BUFFER 32`
- `#define BUFFER_SPACING 4`
- `#define SPINNER_SPEED 200`
- `#define LP_BUFFER_SIZE 9`
- `#define ONLINE_TIMEOUT 30000`
- `#define FM80_ID 0x03`
- `#define FMxx_STATE abuf[2]`
- `#define AMP_WHOLE_ZERO 0`
- `#define CMD_CRC_LEN 10`
- `#define DTG_LEN 3`

Typedefs

- `typedef struct B_type B\_type`

Enumerations

- `enum status_type {
 STATUS_SLEEPING = 0 , STATUS_FLOATING = 1 , STATUS_BULK = 2 , STATUS_ABSORB = 3 ,
 STATUS_EQUALIZE = 4 , STATUS_LAST }`

Functions

- void [onesecc_io](#) (void)
- void [tensecc_io](#) (void)
- void [FM_io](#) (void)
- uint8_t [FM_tx](#) (const uint16_t *, const uint8_t)
- bool [FM_tx_empty](#) (void)
- uint8_t [FM_rx](#) (uint16_t *)
- bool [FM_rx_ready](#) (void)
- uint8_t [FM_rx_count](#) (void)
- void [FM_restart](#) (void)
- void [wdtdelay](#) (const uint32_t)
- float [lp_filter](#) (const float, const uint8_t, const int8_t)
- uint16_t [calc_checksum](#) (uint8_t *, const uint8_t)

Variables

- const char [build_version](#) [] = "V2.03 FM80 Q84"
- const uint16_t [cmd_id](#) [] = {0x100, 0x02, 0x00, 0x00, 0x00, 0x00, 0x00, 0x02}
- const uint16_t [cmd_status](#) [] = {0x100, 0x02, 0x01, 0xc8, 0x00, 0x00, 0x00, 0xcb}
- const uint16_t [cmd_mx_status](#) [] = {0x100, 0x04, 0x00, 0x01, 0x00, 0x00, 0x00, 0x05}
- uint16_t [cmd_mx_log](#) [] = {0x100, 0x16, 0x00, 0x00, 0x00, 0x01, 0x00, 0x17}
- const uint16_t [cmd_panelv](#) [] = {0x100, 0x02, 0x01, 0xc6, 0x00, 0x00, 0x00, 0xc9}
- const uint16_t [cmd_batteryv](#) [] = {0x100, 0x02, 0x00, 0x08, 0x00, 0x00, 0x00, 0x0a}
- const uint16_t [cmd_batterya](#) [] = {0x100, 0x02, 0x01, 0xc7, 0x00, 0x00, 0x00, 0xca}
- const uint16_t [cmd_watts](#) [] = {0x100, 0x02, 0x01, 0x6a, 0x00, 0x00, 0x00, 0x6d}
- const uint16_t [cmd_misc](#) [] = {0x100, 0x02, 0x00, 0x00, 0x00, 0x00, 0x00, 0x02}
- const uint16_t [cmd_fwreva](#) [] = {0x100, 0x02, 0x00, 0x02, 0x00, 0x00, 0x00, 0x04}
- const uint16_t [cmd_fwrevb](#) [] = {0x100, 0x02, 0x00, 0x03, 0x00, 0x00, 0x00, 0x05}
- const uint16_t [cmd_fwrevc](#) [] = {0x100, 0x02, 0x00, 0x04, 0x00, 0x00, 0x00, 0x06}
- uint16_t [cmd_time](#) [] = {0x100, 0x03, 0x40, 0x04, 0x00, 0x00, 0x00, 0x00}
- uint16_t [cmd_date](#) [] = {0x100, 0x03, 0x40, 0x05, 0x00, 0x00, 0x00, 0x00}
- const char [state_name](#) [][12]
- const char [FM80_name](#) [][12]
- const char [canbus_name](#) [][12]
- const char [modbus_name](#) [][12]
- [B_type B](#)

6.133.1 Macro Definition Documentation

6.133.1.1 AMP_WHOLE_ZERO

```
#define AMP_WHOLE_ZERO 0
```

6.133.1.2 BUFFER_SPACING

```
#define BUFFER_SPACING 4
```

6.133.1.3 CMD_CRC_LEN

```
#define CMD_CRC_LEN 10
```

6.133.1.4 DTG_LEN

```
#define DTG_LEN 3
```

6.133.1.5 FM80_ID

```
#define FM80_ID 0x03
```

6.133.1.6 FM_BUFFER

```
#define FM_BUFFER 32
```

6.133.1.7 FMxx_STATE

```
#define FMxx_STATE abuf[2]
```

6.133.1.8 IO_TEST

```
#define IO_TEST
```

6.133.1.9 LP_BUFFER_SIZE

```
#define LP_BUFFER_SIZE 9
```

6.133.1.10 MAX_B_BUF

```
#define MAX_B_BUF 512
```

6.133.1.11 MAX_C_BUF

```
#define MAX_C_BUF CANFD_BYTES*3
```

6.133.1.12 ONLINE_TIMEOUT

```
#define ONLINE_TIMEOUT 30000
```

6.133.1.13 SPINNER_SPEED

```
#define SPINNER_SPEED 200
```

6.133.1.14 VER

```
#define VER 1
```

6.133.2 Typedef Documentation

6.133.2.1 B_type

```
typedef struct B_type B_type
```

6.133.3 Enumeration Type Documentation

6.133.3.1 status_type

```
enum status_type
```

Enumerator

STATUS_SLEEPING	
STATUS_FLOATING	
STATUS_BULK	
STATUS_ABSORB	
STATUS_EQUALIZE	
STATUS_LAST	

6.133.4 Function Documentation

6.133.4.1 calc_checksum()

```
uint16_t calc_checksum (  
    uint8_t * data,  
    const uint8_t len) [extern]
```

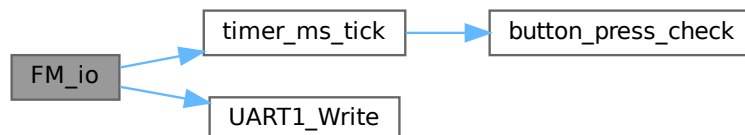
Here is the caller graph for this function:



6.133.4.2 FM_io()

```
void FM_io (  
    void ) [extern]
```

Here is the call graph for this function:



Here is the caller graph for this function:



6.133.4.3 FM_restart()

```
void FM_restart (  
    void ) [extern]
```

6.133.4.4 FM_rx()

```
uint8_t FM_rx (  
    uint16_t * data) [extern]
```

6.133.4.5 FM_rx_count()

```
uint8_t FM_rx_count (  
    void ) [extern]
```

6.133.4.6 FM_rx_ready()

```
bool FM_rx_ready (  
    void ) [extern]
```

6.133.4.7 FM_tx()

```
uint8_t FM_tx (  
    const uint16_t * data,  
    const uint8_t count) [extern]
```

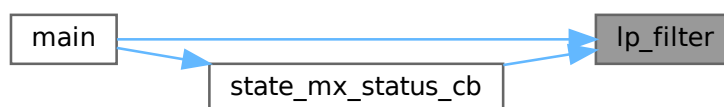
6.133.4.8 FM_tx_empty()

```
bool FM_tx_empty (  
    void ) [extern]
```

6.133.4.9 lp_filter()

```
float lp_filter (  
    const float new,  
    const uint8_t bn,  
    const int8_t slow) [extern]
```

Here is the caller graph for this function:



6.133.4.10 onesecc_io()

```
void onesecc_io (  
    void ) [extern]
```

Here is the caller graph for this function:



6.133.4.11 tensesec_io()

```
void tensesec_io (
    void ) [extern]
```

Here is the caller graph for this function:



6.133.4.12 wtdtdelay()

```
void wtdtdelay (
    const uint32_t delay) [extern]
```

Here is the caller graph for this function:



6.133.5 Variable Documentation

6.133.5.1 B

```
B_type B [extern]
```

6.133.5.2 build_version

```
const char build_version[] = "V2.03 FM80 Q84"
```

6.133.5.3 canbus_name

```
const char canbus_name[][12]
```

Initial value:

```
= {
    " Offline",
    "CANBUS ",
}
```

6.133.5.4 cmd_batterya

```
const uint16_t cmd_batterya[] = {0x100, 0x02, 0x01, 0xc7, 0x00, 0x00, 0x00, 0xca}
```

6.133.5.5 cmd_batteryv

```
const uint16_t cmd_batteryv[] = {0x100, 0x02, 0x00, 0x08, 0x00, 0x00, 0x00, 0x0a}
```

6.133.5.6 cmd_date

```
uint16_t cmd_date[] = {0x100, 0x03, 0x40, 0x05, 0x00, 0x00, 0x00, 0x00}
```

6.133.5.7 cmd_fwreva

```
const uint16_t cmd_fwreva[] = {0x100, 0x02, 0x00, 0x02, 0x00, 0x00, 0x00, 0x04}
```

6.133.5.8 cmd_fwrevb

```
const uint16_t cmd_fwrevb[] = {0x100, 0x02, 0x00, 0x03, 0x00, 0x00, 0x00, 0x05}
```

6.133.5.9 cmd_fwrevc

```
const uint16_t cmd_fwrevc[] = {0x100, 0x02, 0x00, 0x04, 0x00, 0x00, 0x00, 0x06}
```

6.133.5.10 cmd_id

```
const uint16_t cmd_id[] = {0x100, 0x02, 0x00, 0x00, 0x00, 0x00, 0x00, 0x02}
```

6.133.5.11 cmd_misc

```
const uint16_t cmd_misc[] = {0x100, 0x02, 0x00, 0x00, 0x00, 0x00, 0x00, 0x02}
```

6.133.5.12 cmd_mx_log

```
uint16_t cmd_mx_log[] = {0x100, 0x16, 0x00, 0x00, 0x00, 0x01, 0x00, 0x17}
```

6.133.5.13 cmd_mx_status

```
const uint16_t cmd_mx_status[] = {0x100, 0x04, 0x00, 0x01, 0x00, 0x00, 0x00, 0x05}
```

6.133.5.14 cmd_panelv

```
const uint16_t cmd_panelv[] = {0x100, 0x02, 0x01, 0xc6, 0x00, 0x00, 0x00, 0xc9}
```

6.133.5.15 cmd_status

```
const uint16_t cmd_status[] = {0x100, 0x02, 0x01, 0xc8, 0x00, 0x00, 0x00, 0xcb}
```

6.133.5.16 cmd_time

```
uint16_t cmd_time[] = {0x100, 0x03, 0x40, 0x04, 0x00, 0x00, 0x00, 0x00}
```

6.133.5.17 cmd_watts

```
const uint16_t cmd_watts[] = {0x100, 0x02, 0x01, 0x6a, 0x00, 0x00, 0x00, 0x6d}
```

6.133.5.18 FM80_name

```
const char FM80_name[][12]
```

Initial value:

```
= {  
    " Offline",  
    "FM80    ",  
}
```

6.133.5.19 modbus_name

```
const char modbus_name[][12]
```

Initial value:

```
= {  
    " Offline",  
    "MODBUS  ",  
}
```

6.133.5.20 state_name

```
const char state_name[][12]
```

Initial value:

```
= {  
    "Sleeping",  
    "Floating",  
    "Bulk",  
    "Absorb",  
    "Equalize",  
    " Last",  
}
```


6.134 mxcmd.h

[Go to the documentation of this file.](#)

```

00001 /*
00002  * File:    mxcmd.h
00003  * Author:  root
00004  *
00005  * Created on June 2, 2023, 6:16 AM
00006  */
00007
00008 #ifndef MXCMD_H
00009 #define MXCMD_H
00010
00011 #ifdef __cplusplus
00012 extern "C" {
00013 #endif
00014
00015 #include <stddef.h>           // Defines NULL
00016 #include <stdbool.h>         // Defines true
00017 #include <stdlib.h>          // Defines EXIT_FAILURE
00018 #include <stdio.h>
00019 #include <string.h>
00020 #include "mcc_generated_files/mcc.h"
00021 #include "../eadog.h"
00022 #include "../timers.h"
00023
00024 #define VER 1
00025 const char build_version[] = "V2.03 FM80 Q84";
00026 /*
00027  * code changes
00028  * 1.55 remove critical section interrupt disables for FM80 serial
00029  * 1.56 add battery voltage to SoC for boot setting of battery energy
00030  * 1.57 fix FM80 memcpy length bug in FM_rx and FM_tx
00031  * 1.58 add filter and masking to the canbus setup function so we can
00032  * properly receive canbus packets
00033  * 1.59 CAN receive packet buffering and display
00034  * 1.60 misc code cleanups
00035  * 1.61 add FIFO for xmit
00036  * 1.62 add modules firmware/ID capture and display
00037  * 1.63 FM80 FW fixes
00038  * 1.65 add FM80 log data reporting for the previous day
00039  * 1.70 add date and time updates via CAN to FM80 from the network server
00040  * 1.71 time server cleanup and display codes
00041  * 1.72 add day log functions from the serial port
00042  * 1.73 add LCD data mirror data via CANBUS
00043  * 1.74 clean up error indicator and data routines
00044  * 1.75 lockout CANBUS during FM80 serial receive functions
00045  * 1.80 fix LCD DMA transfers using SPI, still needs a bit of work during startup
00046  * 1.81 fix scroll buffer junk and add running PV Wh energy total for the day
00047  * 1.82 day/night switching glitches
00048  * 1.85 run time accounting and logging
00049  * 1.86 fix more logging buffer length issues
00050  * 1.87 use CLC for heartbeat timer and I/O signal
00051  * 1.88 XC8 must use V2.41 as V2.45 seems to be buggy
00052  * 1.90 Remote DMA LCD mode fixes
00053  * 1.91 LCD command timing fixes
00054  * 1.92 reorg trace signal names and add LATE status to canbus info packet to sync remotes
00055  * 1.93 send/receive blob binary data can data packets
00056  * 1.94, 1.95 add button functions and debouncing
00057  * 1.96 smooth run-time data point
00058  * 1.97 expand logging buffer for canbus data
00059  * 1.98 main display dimming
00060  * 1.99 Float fixes
00061  * 2.00 add set display to 1 mode canbus command
00062  * 2.01 minor code cleanups
00063  * 2.02 LCD can rx/tx logging display
00064  * 2.03 button causing display switches while dimmed fixed, mxlog_ptr having issues with xc8 3.00
00065  */
00066
00067 #define MAX_B_BUF    512
00068 #define MAX_C_BUF    CANFD_BYTES*3
00069 #define IO_TEST
00070
00071 #define FM_BUFFER    32
00072 #define BUFFER_SPACING  4
00073 #define SPINNER_SPEED  200
00074 #define LP_BUFFER_SIZE  9
00075 #define ONLINE_TIMEOUT 30000
00076
00077 #define FM80_ID      0x03
00078 #define FMxx_STATE   abuf[2]
00079
00080 #define AMP_WHOLE_ZERO 0
00081
00082 #define CMD_CRC_LEN 10

```

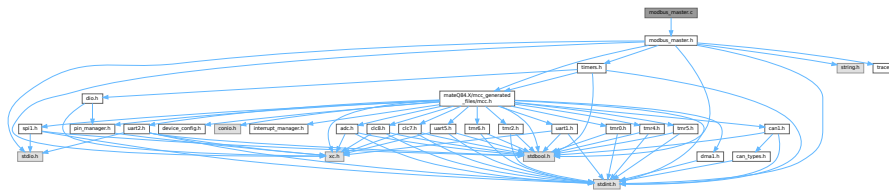
```

00083
00084 #define DTG_LEN      3 // normal size is 26 but we must save buffer space
00085
00086     const uint16_t cmd_id[] = {0x100, 0x02, 0x00, 0x00, 0x00, 0x00, 0x00, 0x02};
00087     const uint16_t cmd_status[] = {0x100, 0x02, 0x01, 0xc8, 0x00, 0x00, 0x00, 0xcb};
00088     const uint16_t cmd_mx_status[] = {0x100, 0x04, 0x00, 0x01, 0x00, 0x00, 0x00, 0x05};
00089     uint16_t cmd_mx_log[] = {0x100, 0x16, 0x00, 0x00, 0x00, 0x01, 0x00, 0x17}; // get logs, start from
day 1
00090     const uint16_t cmd_panelv[] = {0x100, 0x02, 0x01, 0xc6, 0x00, 0x00, 0x00, 0xc9};
00091     const uint16_t cmd_batteryv[] = {0x100, 0x02, 0x00, 0x08, 0x00, 0x00, 0x00, 0x0a};
00092     const uint16_t cmd_batterya[] = {0x100, 0x02, 0x01, 0xc7, 0x00, 0x00, 0x00, 0xca};
00093     const uint16_t cmd_watts[] = {0x100, 0x02, 0x01, 0x6a, 0x00, 0x00, 0x00, 0x6d};
00094     const uint16_t cmd_misc[] = {0x100, 0x02, 0x00, 0x00, 0x00, 0x00, 0x00, 0x02}; // example FM80
command ID request
00095     const uint16_t cmd_fwreva[] = {0x100, 0x02, 0x00, 0x02, 0x00, 0x00, 0x00, 0x04};
00096     const uint16_t cmd_fwrevb[] = {0x100, 0x02, 0x00, 0x03, 0x00, 0x00, 0x00, 0x05};
00097     const uint16_t cmd_fwrevc[] = {0x100, 0x02, 0x00, 0x04, 0x00, 0x00, 0x00, 0x06};
00098     uint16_t cmd_time[] = {0x100, 0x03, 0x40, 0x04, 0x00, 0x00, 0x00, 0x00};
00099     uint16_t cmd_date[] = {0x100, 0x03, 0x40, 0x05, 0x00, 0x00, 0x00, 0x00};
00100
00101     enum status_type {
00102         STATUS_SLEEPING = 0,
00103         STATUS_FLOATING = 1,
00104         STATUS_BULK = 2,
00105         STATUS_ABSORB = 3,
00106         STATUS_EQUALIZE = 4,
00107         STATUS_LAST,
00108     };
00109
00110     const char state_name[][12] = {
00111         "Sleeping",
00112         "Floating",
00113         "Bulk",
00114         "Absorb",
00115         "Equalize",
00116         "Last",
00117     };
00118
00119     const char FM80_name[][12] = {
00120         "Offline",
00121         "FM80",
00122     };
00123
00124     const char canbus_name[][12] = {
00125         "Offline",
00126         "CANBUS",
00127     };
00128
00129     const char modbus_name[][12] = {
00130         "Offline",
00131         "MODBUS",
00132     };
00133
00134     typedef struct {
00135         uint8_t a[16]; // raw_ah(part2)
00136     } mx_status_packed_t;
00137
00138     typedef struct {
00139         uint8_t a[16];
00140     } mx_log_packed_t;
00141
00142     typedef struct {
00143         uint8_t type;
00144         int16_t day;
00145         int16_t amp_hours;
00146         int16_t kilowatt_hours;
00147         int16_t volts_peak;
00148         int16_t amps_peak;
00149         int16_t kilowatts_peak;
00150         int16_t bat_min;
00151         int16_t bat_max;
00152         int16_t absorb_time;
00153         int16_t float_time;
00154         uint8_t select;
00155     } mx_logpage_t;
00156
00157     typedef struct B_type {
00158         volatile bool ten_sec_flag, one_sec_flag, FM80_charged, pv_high, pv_update, once,
a_switch[D_SW_COUNT], a_trigger[D_SW_COUNT], a_type[D_SW_COUNT];
00159         volatile uint16_t pacing, rx_count, flush, pv_prev, day_check, node_id, dim_delay;
00160         volatile bool FM80_online, FM80_io, LOG, display_dim, display_update, display_on;
00161         volatile uint8_t canbus_online, modbus_online, alt_display, a_pin[D_SW_COUNT];
00162         float run_time, net_balance;
00163         uint16_t mui[10];
00164         uint16_t fwrev[3];
00165         mx_logpage_t log;
00166     } B_type;

```

6.135 modbus_master.c File Reference

Include dependency graph for modbus_master.c:



- #define ON 1
- #define OFF 0

- uint16_t [modbus_rtu_send_msg](#) (void *cc_buffer, const void *modbus_cc_mode, uint16_t req_length)
- uint16_t [crc16](#) (volatile uint8_t *buffer, uint16_t buffer_length)
- void [my_modbus_rx_32](#) (void)
- uint8_t [init_stream_params](#) (void)
- void [init_mb_master_timers](#) (void)
- int32_t [mb32_swap](#) (const int32_t value)
- int16_t [mb16_swap](#) (const int16_t value)
- int8_t [master_controller_work](#) (C_data *client)
- void [clear_2hz](#) (void)
- void [clear_500ahz](#) (void)
- void [clear_500hz](#) (void)
- uint32_t [get_2hz](#) (const uint8_t mode)
- uint32_t [get_500ahz](#) (const uint8_t mode)
- uint32_t [get_500hz](#) (const uint8_t mode)
- void [timer_500ms_tick](#) (void)
- void [timer_2ms_tick](#) (void)
- void [mb_tx_test](#) (C_data *client)
- void [mb_setup](#) (void)

Variables

- volatile uint8_t [cc_stream_file](#)
- volatile uint8_t [cc_buffer](#) [MAX_DATA]
- volatile uint8_t [cc_buffer_tx](#) [MAX_DATA]
- volatile [M_data](#) M
- volatile [M_time_data](#) MT
- [C_data](#) C
- volatile struct [V_type](#) V
- const uint8_t [modbus_em_id](#) [] = {MADDR, READ_HOLDING_REGISTERS, 0x00, 0x0b, 0x00, 0x01}
- const uint8_t [modbus_em_version](#) [] = {MADDR, READ_HOLDING_REGISTERS, 0x03, 0x02, 0x00, 0x01}
- const uint8_t [modbus_em_data1](#) [] = {MADDR, READ_HOLDING_REGISTERS, 0x00, 0x00, 0x00, [EM_DATA_LEN1](#)}
- const uint8_t [modbus_em_data2](#) [] = {MADDR, READ_HOLDING_REGISTERS, 0x05, 0x00, 0x00, [EM_DATA_LEN2](#)}
- const uint8_t [modbus_em_serial](#) [] = {MADDR, READ_HOLDING_REGISTERS, 0x50, 0x00, 0x00, [SERIAL_DATA_LEN](#)}
- const uint8_t [modbus_em_config](#) [] = {MADDR, WRITE_SINGLE_REGISTER, 0x10, 0x02, 0x00, 0x02}
- const uint8_t [modbus_em_passwd](#) [] = {MADDR, WRITE_SINGLE_REGISTER, 0x10, 0x00, 0x00, 0x00}
- const uint8_t [modbus_em_light](#) [] = {MADDR, WRITE_SINGLE_REGISTER, 0x16, 0x04, 0x00, 0x01}
- const uint8_t [em_id](#) [] = {MADDR, READ_HOLDING_REGISTERS, 0x00, 0x00, 0x00, 0x00, 0x00}
- const uint8_t [em_version](#) [] = {MADDR, READ_HOLDING_REGISTERS, 0x00, 0x00, 0x00, 0x00, 0x00}
- const uint8_t [em_data1](#) [(EM_DATA_LEN1 *2)+5] = {MADDR, READ_HOLDING_REGISTERS, 0x00}
- const uint8_t [em_data2](#) [(EM_DATA_LEN2 *2)+5] = {MADDR, READ_HOLDING_REGISTERS, 0x00}
- const uint8_t [em_serial](#) [(SERIAL_DATA_LEN *2)+5] = {MADDR, READ_HOLDING_REGISTERS, 0x00}
- const uint8_t [em_config](#) [] = {MADDR, WRITE_SINGLE_REGISTER, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00}
- const uint8_t [em_passwd](#) [] = {MADDR, WRITE_SINGLE_REGISTER, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00}
- const uint8_t [em_light](#) [] = {MADDR, WRITE_SINGLE_REGISTER, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00}
- [EM_data1](#) em
- [EM_data2](#) emt
- [EM_serial](#) ems
- [EM_version](#) emv

6.135.1 Macro Definition Documentation

6.135.1.1 OFF

```
#define OFF 0
```

6.135.1.2 ON

```
#define ON 1
```

6.135.2 Function Documentation

6.135.2.1 clear_2hz()

```
void clear_2hz (  
    void )
```

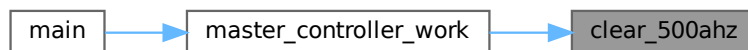
Here is the caller graph for this function:



6.135.2.2 clear_500ahz()

```
void clear_500ahz (  
    void )
```

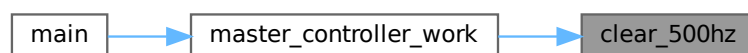
Here is the caller graph for this function:



6.135.2.3 clear_500hz()

```
void clear_500hz (  
    void )
```

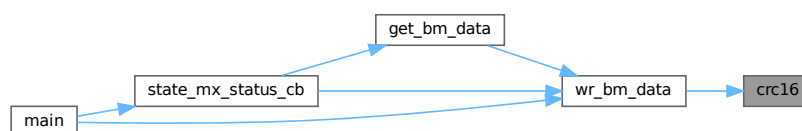
Here is the caller graph for this function:



6.135.2.4 crc16()

```
uint16_t crc16 (  
    volatile uint8_t * buffer,  
    uint16_t buffer_length)
```

Here is the caller graph for this function:



6.135.2.5 get_2hz()

```
uint32_t get_2hz (  
    const uint8_t mode)
```

Here is the caller graph for this function:



6.135.2.6 get_500ahz()

```
uint32_t get_500ahz (  
    const uint8_t mode)
```

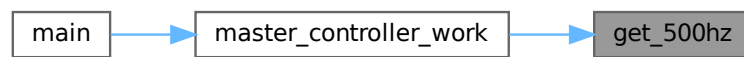
Here is the caller graph for this function:



6.135.2.7 get_500hz()

```
uint32_t get_500hz (  
    const uint8_t mode)
```

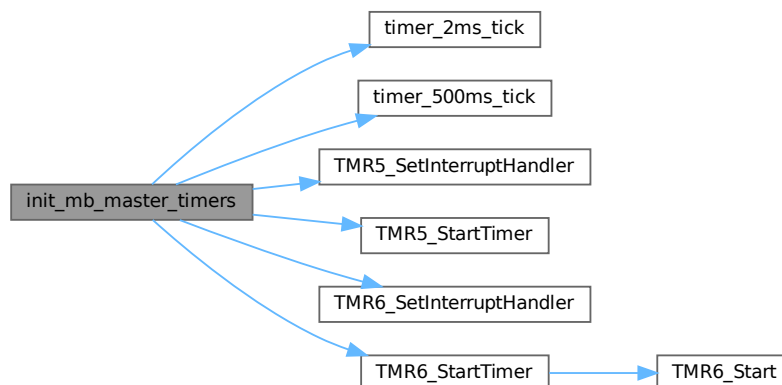
Here is the caller graph for this function:



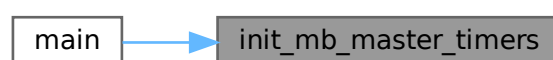
6.135.2.8 init_mb_master_timers()

```
void init_mb_master_timers (  
    void )
```

Here is the call graph for this function:



Here is the caller graph for this function:



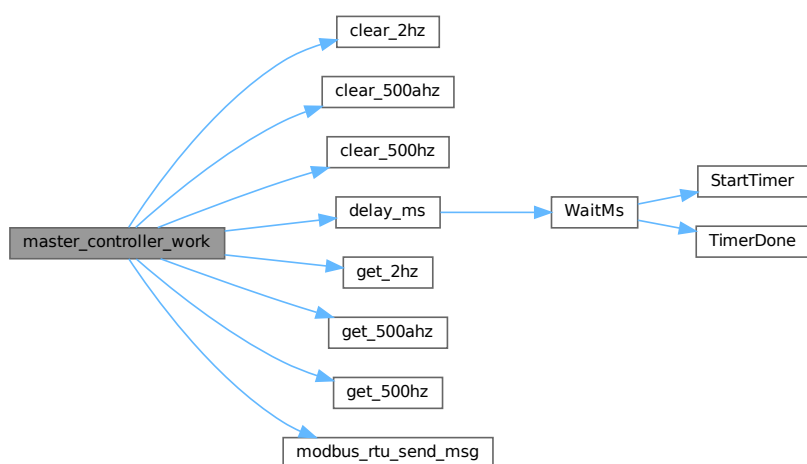
6.135.2.9 init_stream_params()

```
uint8_t init_stream_params (
    void )
```

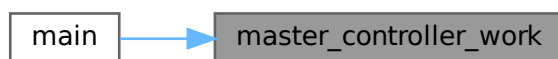
6.135.2.10 master_controller_work()

```
int8_t master_controller_work (
    C_data * client)
```

Here is the call graph for this function:



Here is the caller graph for this function:



6.135.2.11 mb16_swap()

```
int16_t mb16_swap (
    const int16_t value)
```

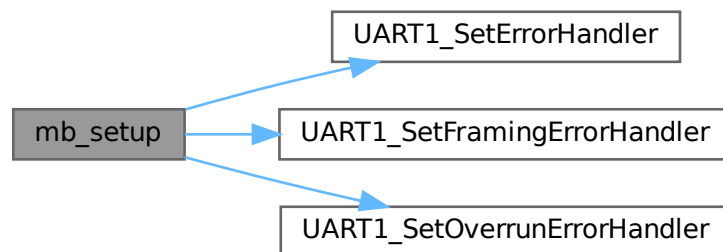

6.135.2.12 mb32_swap()

```
int32_t mb32_swap (  
    const int32_t value)
```

6.135.2.13 mb_setup()

```
void mb_setup (  
    void )
```

Here is the call graph for this function:



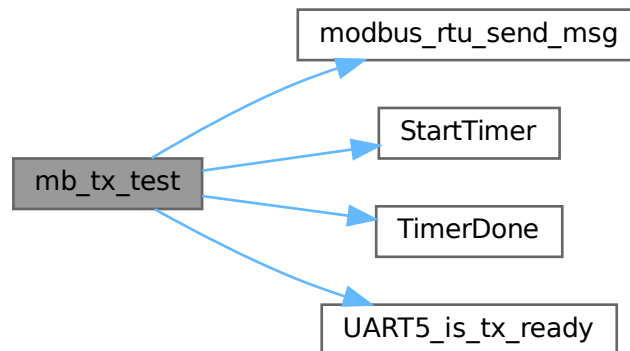
Here is the caller graph for this function:



6.135.2.14 mb_tx_test()

```
void mb_tx_test (  
    C_data * client)
```

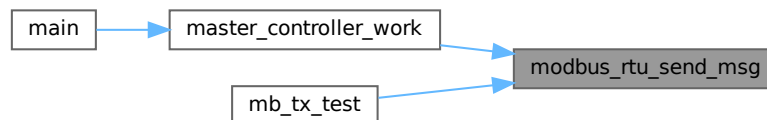
Here is the call graph for this function:



6.135.2.15 modbus_rtu_send_msg()

```
uint16_t modbus_rtu_send_msg (  
    void * cc_buffer,  
    const void * modbus_cc_mode,  
    uint16_t req_length)
```

Here is the caller graph for this function:



6.135.2.16 my_modbus_rx_32()

```
void my_modbus_rx_32 (  
    void )
```

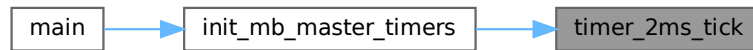
Here is the caller graph for this function:



6.135.2.17 timer_2ms_tick()

```
void timer_2ms_tick (  
    void )
```

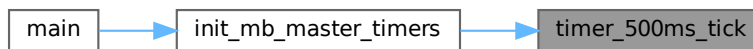
Here is the caller graph for this function:



6.135.2.18 timer_500ms_tick()

```
void timer_500ms_tick (  
    void )
```

Here is the caller graph for this function:



6.135.3 Variable Documentation

6.135.3.1 C

`C_data` C

Initial value:

```
= {  
    .mcmd = G_ID,  
    .cstate = CLEAR,  
    .modbus_command = G_ID,  
    .req_length = 0,  
    .trace = 0,  
    .config_ok = false,  
    .id_ok = false,  
    .passwd_ok = false,  
    .light_ok = false,  
    .M.blink_lock = false,  
    .M.power_on = true,  
    .tm_ok = false,  
}
```

6.135.3.2 cc_buffer

```
volatile uint8_t cc_buffer[MAX_DATA]
```

6.135.3.3 cc_buffer_tx

```
volatile uint8_t cc_buffer_tx[MAX_DATA]
```

6.135.3.4 cc_stream_file

```
volatile uint8_t cc_stream_file
```

6.135.3.5 em

```
EM_data1 em
```

6.135.3.6 em_config

```
const uint8_t em_config[] = {MADDR, WRITE_SINGLE_REGISTER, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00}
```

6.135.3.7 em_data1

```
const uint8_t em_data1[(EM_DATA_LEN1 *2)+5] = {MADDR, READ_HOLDING_REGISTERS, 0x00}
```

6.135.3.8 em_data2

```
const uint8_t em_data2[(EM_DATA_LEN2 *2)+5] = {MADDR, READ_HOLDING_REGISTERS, 0x00}
```

6.135.3.9 em_id

```
const uint8_t em_id[] = {MADDR, READ_HOLDING_REGISTERS, 0x00, 0x00, 0x00, 0x00, 0x00}
```

6.135.3.10 em_light

```
const uint8_t em_light[] = {MADDR, WRITE_SINGLE_REGISTER, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00}
```

6.135.3.11 em_passwd

```
const uint8_t em_passwd[] = {MADDR, WRITE_SINGLE_REGISTER, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00}
```

6.135.3.12 em_serial

```
const uint8_t em_serial[(SERIAL_DATA_LEN *2)+5] = {MADDR, READ_HOLDING_REGISTERS, 0x00}
```

6.135.3.13 em_version

```
const uint8_t em_version[] = {MADDR, READ_HOLDING_REGISTERS, 0x00, 0x00, 0x00, 0x00, 0x00}
```

6.135.3.14 ems

```
EM_serial ems
```

6.135.3.15 emt

```
EM_data2 emt
```

6.135.3.16 emv

```
EM_version emv
```

6.135.3.17 M

```
volatile M_data M
```

Initial value:

```
= {  
    .blink_lock = false,  
    .power_on = true,  
}
```

6.135.3.18 modbus_em_config

```
const uint8_t modbus_em_config[] = {MADDR, WRITE_SINGLE_REGISTER, 0x10, 0x02, 0x00, 0x02}
```

6.135.3.19 modbus_em_data1

```
const uint8_t modbus_em_data1[] = {MADDR, READ_HOLDING_REGISTERS, 0x00, 0x00, 0x00, EM_DATA_LEN1}
```

6.135.3.20 modbus_em_data2

```
const uint8_t modbus_em_data2[] = {MADDR, READ_HOLDING_REGISTERS, 0x05, 0x00, 0x00, EM_DATA_LEN2}
```

6.135.3.21 modbus_em_id

```
const uint8_t modbus_em_id[] = {MADDR, READ_HOLDING_REGISTERS, 0x00, 0x0b, 0x00, 0x01}
```

6.135.3.22 modbus_em_light

```
const uint8_t modbus_em_light[] = {MADDR, WRITE_SINGLE_REGISTER, 0x16, 0x04, 0x00, 0x01}
```

6.135.3.23 modbus_em_passwd

```
const uint8_t modbus_em_passwd[] = {MADDR, WRITE_SINGLE_REGISTER, 0x10, 0x00, 0x00, 0x00}
```

6.135.3.24 modbus_em_serial

```
const uint8_t modbus_em_serial[] = {MADDR, READ_HOLDING_REGISTERS, 0x50, 0x00, 0x00, SERIAL_DATA_LEN}
```

6.135.3.25 modbus_em_version

```
const uint8_t modbus_em_version[] = {MADDR, READ_HOLDING_REGISTERS, 0x03, 0x02, 0x00, 0x01}
```

6.135.3.26 MT

```
volatile M_time_data MT
```

Initial value:

```
= {  
    .clock_500ahz = 0,  
    .clock_2hz = 0,  
    .clock_500hz = 0,  
}
```

6.135.3.27 V

```
volatile struct V_type V
```

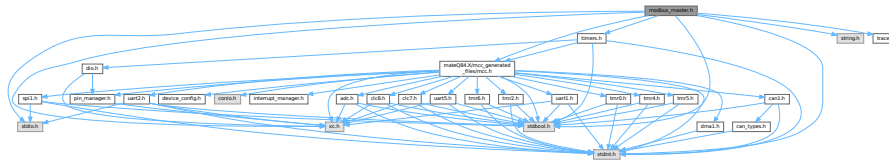
Initial value:

```
= {  
    .StartTime = 1,  
    .TimeUsed = 1,  
    .pacing = 1,  
    .pwm_update = true,  
    .pwm_stop = true,  
    .fault_active = false,  
    .fault_count = 0,  
    .dmt_sosc_flag = false,  
}
```

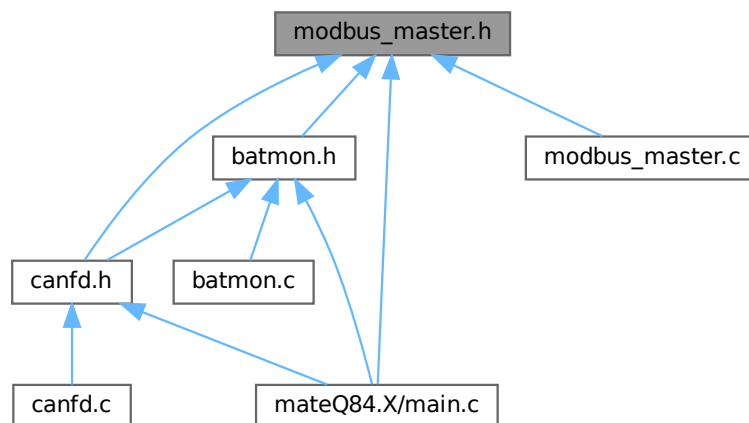
6.136 modbus_master.h File Reference

```
#include <xc.h>
#include <stdint.h>
#include <stdbool.h>
#include <stdio.h>
#include <string.h>
#include "mateQ84.X/mcc_generated_files/mcc.h"
#include "timers.h"
#include "trace.h"
```

Include dependency graph for modbus_master.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct [V_type](#)
- union [PWMDC](#)
- union [MREG](#)
- union [MREG32](#)
- union [MREG64](#)
- struct [M_time_data](#)
- struct [M_data](#)
- struct [C_data](#)
- struct [EM_data1](#)
- struct [EM_data2](#)
- struct [EM_serial](#)
- struct [EM_version](#)

Macros

- `#define MB_MASTER`
- `#define MB_EM540`
- `#define MB_EM540_ID_H 0x06`
- `#define MB_EM540_ID_L 0xE0`
- `#define SWMBMVER 0X0033`
- `#define MADDR 0x01`
- `#define EM_DATA_LEN1 52`
- `#define EM_DATA_LEN2 64`
- `#define SERIAL_DATA_LEN 8`
- `#define DEBUG_R`
- `#define INT_TRACE INT_TRACE_Toggle()`
- `#define MAX_DATA 240`
- `#define FASTQ`
- `#define TDELAY 2`
- `#define TEDELAY 1`
- `#define RDELAY 200`
- `#define CDELAY 40`
- `#define QDELAY 2`
- `#define TODELAY 4`
- `#define SPACING 40`
- `#define DUPL_DELAY 2`
- `#define Swrite UART5_Write`
- `#define Strmt UART5_is_tx_done`
- `#define Serror UART5_get_last_status`
- `#define Sread UART5_Read`
- `#define Srbuffer U5RXB`
- `#define READ_COILS 1`
- `#define READ_DISCRETE_INPUTS 2`
- `#define READ_HOLDING_REGISTERS 3`
- `#define READ_INPUT_REGISTERS 4`
- `#define WRITE_SINGLE_COIL 5`
- `#define WRITE_SINGLE_REGISTER 6`
- `#define WRITE_MULTIPLE_COILS 15`
- `#define WRITE_MULTIPLE_REGISTERS 16`
- `#define BOFF 0`
- `#define BON 255`
- `#define MM_ERROR_S MLED_SetHigh()`
- `#define MM_ERROR_C MLED_SetLow()`

Typedefs

- `typedef enum comm_type comm_type`
- `typedef enum cmd_type cmd_type`
- `typedef enum trace_type trace_type`
- `typedef struct M_time_data M_time_data`
- `typedef struct M_data M_data`
- `typedef struct C_data C_data`
- `typedef __pack struct EM_data1 EM_data1`
- `typedef __pack struct EM_data2 EM_data2`
- `typedef __pack struct EM_serial EM_serial`
- `typedef __pack struct EM_version EM_version`
- `typedef enum filter_type filter_type`

Enumerations

- enum `comm_type` { `CLEAR` = 0 , `INIT` , `SEND` , `RECV` }
- enum `cmd_type` {
`G_ID` = 0 , `G_DATA1` , `G_DATA2` , `G_CONFIG` ,
`G_PASSWD` , `G_LIGHT` , `G_VERSION` , `G_SERIAL` ,
`G_LAST` }
- enum `trace_type` {
`T_begin` = 1 , `T_clear` , `T_passwd` , `T_config` ,
`T_data` , `T_id` , `T_serial` , `T_version` ,
`T_init` , `T_init_d` , `T_send` , `T_send_d` ,
`T_recv` , `T_recv_r` , `T_misc1` , `T_misc2` ,
`T_misc3` , `T_spacing` , `T_light` }
- enum `filter_type` {
`F_ac` = 0 , `F_wac` = 1 , `F_wva` = 2 , `F_var` = 3 ,
`F_run` = 4 , `F_5` = 5 , `F_6` = 6 , `F_7` = 7 ,
`F_8` = 8 }

Functions

- `uint16_t crc16` (volatile `uint8_t *`, `uint16_t`)
- `uint16_t modbus_rtu_send_msg` (void *, const void *, `uint16_t`)
- void `my_modbus_rx_32` (void)
- `uint8_t init_stream_params` (void)
- void `init_mb_master_timers` (void)
- `int8_t master_controller_work` (`C_data *`)
- `int32_t mb32_swap` (const `int32_t`)
- `int16_t mb16_swap` (const `int16_t`)
- void `clear_2hz` (void)
- void `clear_500ahz` (void)
- void `clear_500hz` (void)
- `uint32_t get_2hz` (const `uint8_t`)
- `uint32_t get_500ahz` (const `uint8_t`)
- `uint32_t get_500hz` (const `uint8_t`)
- void `timer_500ms_tick` (void)
- void `timer_2ms_tick` (void)
- void `mb_tx_test` (`C_data *`)
- void `mb_setup` (void)

Variables

- volatile struct `V_type` `V`
- `C_data` `C`
- volatile `M_data` `M`
- volatile `M_time_data` `MT`
- `EM_data1` `em`
- `EM_data2` `emt`
- `EM_serial` `ems`
- `EM_version` `emv`

6.136.1 Macro Definition Documentation

6.136.1.1 BOFF

```
#define BOFF 0
```

6.136.1.2 BON

```
#define BON 255
```

6.136.1.3 CDELAY

```
#define CDELAY 40
```

6.136.1.4 DBUG_R

```
#define DBUG_R
```

6.136.1.5 DUPL_DELAY

```
#define DUPL_DELAY 2
```

6.136.1.6 EM_DATA_LEN1

```
#define EM_DATA_LEN1 52
```

6.136.1.7 EM_DATA_LEN2

```
#define EM_DATA_LEN2 64
```

6.136.1.8 FASTQ

```
#define FASTQ
```

6.136.1.9 INT_TRACE

```
#define INT_TRACE INT\_TRACE\_Toggle\(\)
```

6.136.1.10 MADDR

```
#define MADDR 0x01
```

6.136.1.11 MAX_DATA

```
#define MAX_DATA 240
```

6.136.1.12 MB_EM540

```
#define MB_EM540
```

6.136.1.13 MB_EM540_ID_H

```
#define MB_EM540_ID_H 0x06
```

6.136.1.14 MB_EM540_ID_L

```
#define MB_EM540_ID_L 0xE0
```

6.136.1.15 MB_MASTER

```
#define MB_MASTER
```

6.136.1.16 MM_ERROR_C

```
#define MM_ERROR_C MLED_SetLow()
```

6.136.1.17 MM_ERROR_S

```
#define MM_ERROR_S MLED_SetHigh()
```

6.136.1.18 QDELAY

```
#define QDELAY 2
```

6.136.1.19 RDELAY

```
#define RDELAY 200
```

6.136.1.20 READ_COILS

```
#define READ_COILS 1
```

6.136.1.21 READ_DISCRETE_INPUTS

```
#define READ_DISCRETE_INPUTS 2
```

6.136.1.22 READ_HOLDING_REGISTERS

```
#define READ_HOLDING_REGISTERS 3
```

6.136.1.23 READ_INPUT_REGISTERS

```
#define READ_INPUT_REGISTERS 4
```

6.136.1.24 SERIAL_DATA_LEN

```
#define SERIAL_DATA_LEN 8
```

6.136.1.25 Serror

```
#define Serror UART5\_get\_last\_status
```

6.136.1.26 SPACING

```
#define SPACING 40
```

6.136.1.27 Srbuffer

```
#define Srbuffer U5RXB
```

6.136.1.28 Sread

```
#define Sread UART5\_Read
```

6.136.1.29 Strmt

```
#define Strmt UART5\_is\_tx\_done
```

6.136.1.30 SWMBMVER

```
#define SWMBMVER 0X0033
```

6.136.1.31 Swrite

```
#define Swrite UART5_Write
```

6.136.1.32 TDELAY

```
#define TDELAY 2
```

6.136.1.33 TEDELAY

```
#define TEDELAY 1
```

6.136.1.34 TODELAY

```
#define TODELAY 4
```

6.136.1.35 WRITE_MULTIPLE_COILS

```
#define WRITE_MULTIPLE_COILS 15
```

6.136.1.36 WRITE_MULTIPLE_REGISTERS

```
#define WRITE_MULTIPLE_REGISTERS 16
```

6.136.1.37 WRITE_SINGLE_COIL

```
#define WRITE_SINGLE_COIL 5
```

6.136.1.38 WRITE_SINGLE_REGISTER

```
#define WRITE_SINGLE_REGISTER 6
```

6.136.2 Typedef Documentation

6.136.2.1 C_data

```
typedef struct C_data C_data
```

6.136.2.2 cmd_type

```
typedef enum cmd_type cmd_type
```

6.136.2.3 comm_type

```
typedef enum comm_type comm_type
```

6.136.2.4 EM_data1

```
typedef __pack struct EM_data1 EM_data1
```

6.136.2.5 EM_data2

```
typedef __pack struct EM_data2 EM_data2
```

6.136.2.6 EM_serial

```
typedef __pack struct EM_serial EM_serial
```

6.136.2.7 EM_version

```
typedef __pack struct EM_version EM_version
```

6.136.2.8 filter_type

```
typedef enum filter_type filter_type
```

6.136.2.9 M_data

```
typedef struct M_data M_data
```

6.136.2.10 M_time_data

```
typedef struct M_time_data M_time_data
```

6.136.2.11 trace_type

```
typedef enum trace_type trace_type
```

6.136.3 Enumeration Type Documentation

6.136.3.1 cmd_type

```
enum cmd_type
```

Enumerator

G_ID	
G_DATA1	
G_DATA2	
G_CONFIG	
G_PASSWD	
G_LIGHT	
G_VERSION	
G_SERIAL	
G_LAST	

6.136.3.2 comm_type

enum [comm_type](#)

Enumerator

CLEAR	
INIT	
SEND	
RECV	

6.136.3.3 filter_type

enum [filter_type](#)

Enumerator

F_ac	
F_wac	
F_wva	
F_var	
F_run	
F_5	
F_6	
F_7	
F_8	

6.136.3.4 trace_type

enum [trace_type](#)

Enumerator

T_begin	
T_clear	

Enumerator

T_passwd	
T_config	
T_data	
T_id	
T_serial	
T_version	
T_init	
T_init_d	
T_send	
T_send_d	
T_rcv	
T_rcv_r	
T_misc1	
T_misc2	
T_misc3	
T_spacing	
T_light	

6.136.4 Function Documentation

6.136.4.1 clear_2hz()

```
void clear_2hz (
    void )
```

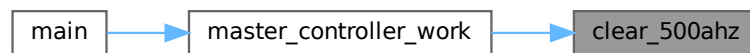
Here is the caller graph for this function:



6.136.4.2 clear_500ahz()

```
void clear_500ahz (
    void )
```

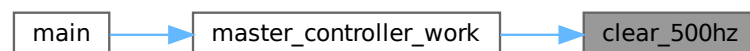

Here is the caller graph for this function:



6.136.4.3 clear_500hz()

```
void clear_500hz (  
    void )
```

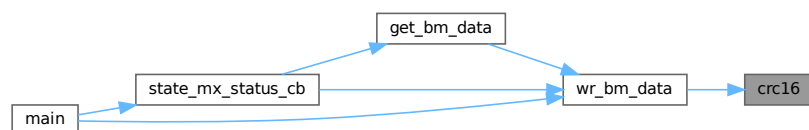
Here is the caller graph for this function:



6.136.4.4 crc16()

```
uint16_t crc16 (  
    volatile uint8_t * buffer,  
    uint16_t buffer_length)
```

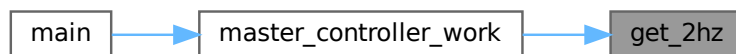
Here is the caller graph for this function:



6.136.4.5 get_2hz()

```
uint32_t get_2hz (  
    const uint8_t mode)
```

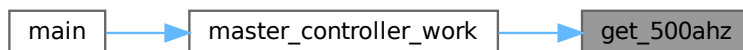
Here is the caller graph for this function:



6.136.4.6 get_500ahz()

```
uint32_t get_500ahz (  
    const uint8_t mode)
```

Here is the caller graph for this function:



6.136.4.7 get_500hz()

```
uint32_t get_500hz (  
    const uint8_t mode)
```

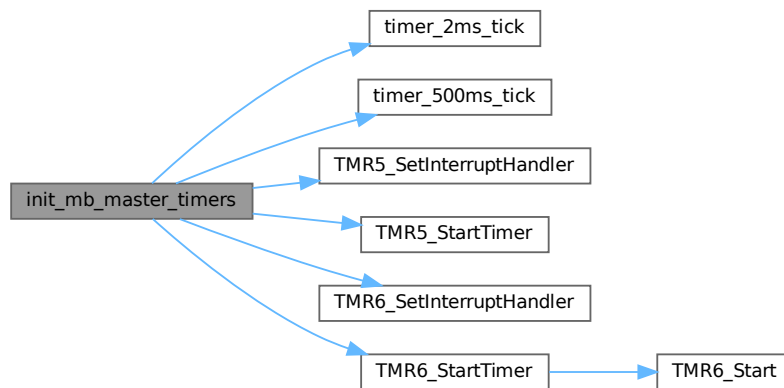
Here is the caller graph for this function:



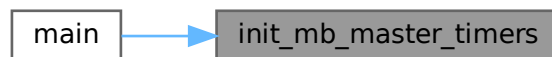
6.136.4.8 init_mb_master_timers()

```
void init_mb_master_timers (  
    void )
```

Here is the call graph for this function:



Here is the caller graph for this function:



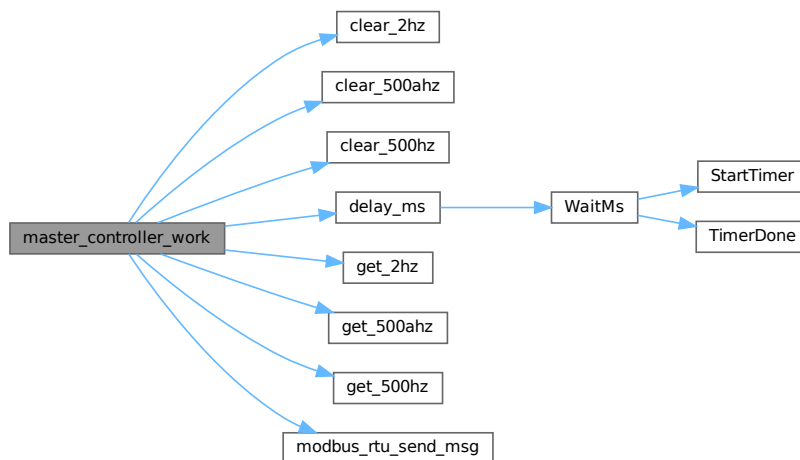
6.136.4.9 init_stream_params()

```
uint8_t init_stream_params (  
    void )
```

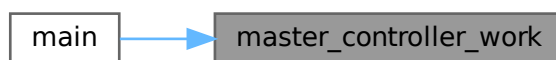
6.136.4.10 master_controller_work()

```
int8_t master_controller_work (  
    C_data * client)
```

Here is the call graph for this function:



Here is the caller graph for this function:



6.136.4.11 mb16_swap()

```
int16_t mb16_swap (
    const int16_t value)
```

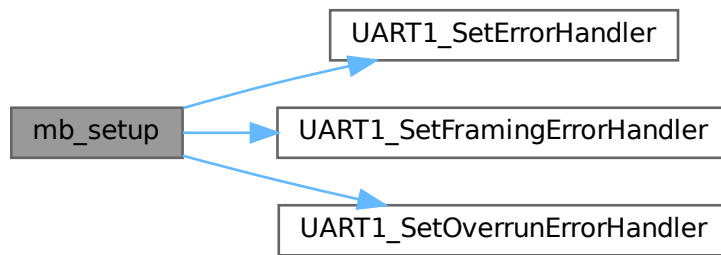
6.136.4.12 mb32_swap()

```
int32_t mb32_swap (
    const int32_t value)
```

6.136.4.13 mb_setup()

```
void mb_setup (
    void )
```

Here is the call graph for this function:



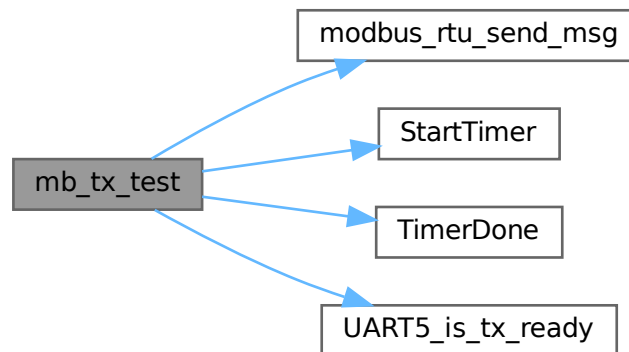
Here is the caller graph for this function:



6.136.4.14 mb_tx_test()

```
void mb_tx_test (  
    C_data * client)
```

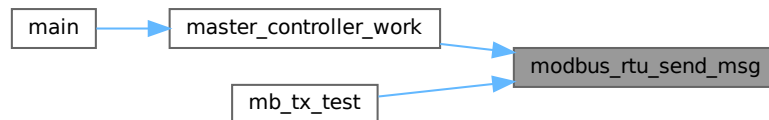
Here is the call graph for this function:



6.136.4.15 modbus_rtu_send_msg()

```
uint16_t modbus_rtu_send_msg (  
    void * cc_buffer,  
    const void * modbus_cc_mode,  
    uint16_t req_length)
```

Here is the caller graph for this function:



6.136.4.16 my_modbus_rx_32()

```
void my_modbus_rx_32 (  
    void )
```

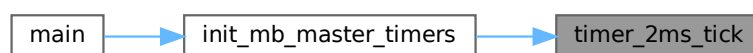
Here is the caller graph for this function:



6.136.4.17 timer_2ms_tick()

```
void timer_2ms_tick (  
    void )
```

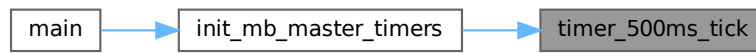
Here is the caller graph for this function:



6.136.4.18 timer_500ms_tick()

```
void timer_500ms_tick (  
    void )
```

Here is the caller graph for this function:



6.136.5 Variable Documentation

6.136.5.1 C

`C_data` C [extern]

6.136.5.2 em

`EM_data1` em [extern]

6.136.5.3 ems

`EM_serial` ems [extern]

6.136.5.4 emt

`EM_data2` emt [extern]

6.136.5.5 emv

`EM_version` emv [extern]

6.136.5.6 M

volatile `M_data` M [extern]

6.136.5.7 MT

volatile `M_time_data` MT [extern]

6.136.5.8 V

volatile struct [V_type](#) V [extern]

6.137 modbus_master.h

[Go to the documentation of this file.](#)

```

00001 /*
00002  * File:   modbus_master.h
00003  * Author: root
00004  *
00005  * Created on April 27, 2022, 4:02 PM
00006  */
00007
00008 /*
00009  * Simple MODBUS data polling master for using MCC
00010  * the USART port uses interrupt driven buffered I/O
00011  * hard coded for data collection from a EM540 3-phase power monitor
00012  * https://www.gavazzionline.com/pdf/EM540_DS_ENG.pdf
00013  */
00014
00015
00016 #ifndef MODBUS_MASTER_H
00017 #define MODBUS_MASTER_H
00018
00019 #ifdef __cplusplus
00020 extern "C" {
00021 #endif
00022
00023 #define MB_MASTER
00024 #define MB_EM540
00025 #define MB_EM540_ID_H 0x06
00026 #define MB_EM540_ID_L 0xE0
00027 #define SWBMVER 0X0033 // master SW version
00028 #define MADDR 0x01 // modbus client address
00029 #define EM_DATA_LEN1 52 // 16-bit words returned
00030 #define EM_DATA_LEN2 64 // 16-bit words returned
00031 #define SERIAL_DATA_LEN 8
00032 /*
00033  * setup options on the EM540 from the factor defaults
00034  * 115200 baud, measurement mode C for bidirectional values
00035  */
00036
00037 /*
00038  * fake good received bytes from client
00039  * a null string when not debugging
00040  */
00041 #define DEBUG_R // true ||
00042 /*
00043  * Display MODBUS debugging info screen
00044  */
00045 // #define SHOW_MODBUS_DEBUG
00046
00047 #include <xc.h>
00048 #include <stdint.h>
00049 #include <stdbool.h>
00050 #include <stdio.h>
00051 #include <string.h>
00052 #include "mateQ84.X/mcc_generated_files/mcc.h"
00053 #include "timers.h"
00054 #include "trace.h"
00055
00056 #ifdef TRACE
00057 #define INT_TRACE INT_TRACE_Toggle() // EXT_IO pin 5
00058 #else
00059 #define INT_TRACE ""
00060 #endif
00061 /*
00062  * 5Hz data updates in FASTQ mode
00063  */
00064 #define MAX_DATA 240
00065 // #define LOCAL_ECHO 1
00066 #define FASTQ // MODBUS query speed, define for faster sampling rates
00067 #define TDELAY 2 // half-duplex delay
00068 #define TEDELAY 1 // half-duplex delay
00069 #define RDELAY 200 // receive timeout
00070 #define CDELAY 40 // fast query delay 100ms
00071 #define QDELAY 2 // slow query delay 1s
00072 #define TODELAY 4 // misc delay

```



```

00073 #define SPACING      40 // control loop cpu usage factor
00074 #define DUPL_DELAY    2 // extra duplex delay mode
00075
00076 /*
00077  * RS485 port defines
00078  * port
00079  */
00080 #define Swrite          UART5_Write
00081 #define Strmt           UART5_is_tx_done
00082 #define Serror          UART5_get_last_status
00083 #define Sread           UART5_Read
00084 #define Srbuffer        U5RXB
00085
00086 /*
00087  * serial communications states
00088  */
00089 typedef enum comm_type {
00090     CLEAR = 0,
00091     INIT,
00092     SEND,
00093     RECV,
00094 } comm_type;
00095
00096 /*
00097  * EM540 command states
00098  */
00099 typedef enum cmd_type {
00100     G_ID = 0,
00101     G_DATA1,
00102     G_DATA2,
00103     G_CONFIG, // keep sequence
00104     G_PASSWD, // keep sequence
00105     G_LIGHT,
00106     G_VERSION,
00107     G_SERIAL,
00108     G_LAST,
00109 } cmd_type;
00110
00111 typedef enum trace_type {
00112     T_begin = 1,
00113     T_clear,
00114     T_passwd,
00115     T_config,
00116     T_data,
00117     T_id,
00118     T_serial,
00119     T_version,
00120     T_init,
00121     T_init_d,
00122     T_send,
00123     T_send_d,
00124     T_recv,
00125     T_recv_r,
00126     T_miscl,
00127     T_misc2,
00128     T_misc3,
00129     T_spacing,
00130     T_light,
00131 } trace_type;
00132
00133 struct V_type {
00134     uint32_t StartTime, TimeUsed;
00135     volatile uint32_t pacing, pwm_update, pwm_stop, fault_count, fault_ticks, fault_source,
modbus_rx, modbus_tx;
00136     int32_t motor_speed;
00137     volatile bool fault_active, dmt_sosc_flag;
00138 };
00139
00140 union PWMDC {
00141     uint16_t lpwm;
00142     uint8_t bpwm[2];
00143 };
00144
00145 union MREG {
00146     int16_t value;
00147     char bytes[2];
00148 };
00149
00150 union MREG32 {
00151     int32_t value;
00152     char bytes[4];
00153 };
00154
00155 union MREG64 {
00156     int64_t value;
00157     char bytes[8];
00158 };

```

```

00159
00160 typedef struct M_time_data { // ISR used, mainly for non-atomic mod problems
00161     uint32_t clock_500hz;
00162     uint32_t clock_500ahz;
00163     uint32_t clock_2hz;
00164 } M_time_data;
00165
00166 typedef struct M_data { // ISR used, mainly for non-atomic mod problems
00167     uint8_t blink_lock : 1;
00168     uint8_t config : 1;
00169     uint8_t stable : 1;
00170     uint8_t boot_code : 1;
00171     uint8_t power_on : 1;
00172     uint8_t send_count, recv_count, pwm_volts;
00173     uint16_t error, crc_data, crc_calc;
00174     uint32_t crc_error;
00175     uint32_t to_error;
00176     uint32_t sends;
00177     volatile bool rx;
00178 } M_data;
00179
00180 typedef struct C_data { // client state machine data
00181     uint8_t mcmd;
00182     comm_type cstate;
00183     cmd_type modbus_command;
00184     uint16_t req_length;
00185     int8_t trace;
00186     bool id_ok, passwd_ok, config_ok, data_ok, light_ok, serial_ok, version_ok, tm_ok;
00187     uint32_t data_count, data_prev;
00188     volatile M_data M;
00189 } C_data;
00190
00191 /*
00192  * maps the EM540 modbus registers to int32_t and uint16_t values
00193  */
00194 typedef __pack struct EM_data1 {
00195     volatile int32_t
00196         vl1n, vl2n, vl3n,
00197         vl1l2, vl2l3, vl3l1,
00198         al1, al2, al3,
00199         wl1, wl2, wl3,
00200         val1, val2, val3,
00201         varl1, varl2, varl3, // extra stuff
00202         vlmsys, vlmsys, wsys, vasys, varsys;
00203     volatile int16_t
00204         pfl1, pfl2, pfl3, pfsys,
00205         phaseseq, hz;
00206 } EM_data1;
00207
00208 typedef __pack struct EM_data2 {
00209     volatile int64_t
00210         kwhpt, kvarhpt, kwhpp, kvarhpp,
00211         kwhpl1, kwhpl2, kwhpl3,
00212         kwhnt, kvarhnt, kwhnp, kvarhnp,
00213         kvaht, kvahp;
00214     volatile int32_t
00215         rhm, rhmk, rhmp, rhmkp,
00216         hz, rhlc;
00217 } EM_data2;
00218
00219 typedef __pack struct EM_serial {
00220     volatile char serial[14];
00221     volatile int16_t year;
00222 } EM_serial;
00223
00224 typedef __pack struct EM_version {
00225     volatile int16_t firmware;
00226 } EM_version;
00227
00228 typedef enum filter_type {
00229     F_ac = 0,
00230     F_wac = 1,
00231     F_wva = 2,
00232     F_var = 3,
00233     F_run = 4,
00234     F_5 = 5,
00235     F_6 = 6,
00236     F_7 = 7,
00237     F_8 = 8,
00238 } filter_type;
00239
00240 // code from libmodbus:
00241 https://raw.githubusercontent.com/stephane/libmodbus/master/src/modbus-rtu.c
00242
00243 /* Table of CRC values for high-order byte */
00244 static const uint8_t table_crc_hi[] = {
00245     0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x01, 0xC0,

```

```
00245     0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41,
00246     0x00, 0xC1, 0x81, 0x40, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0,
00247     0x80, 0x41, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40,
00248     0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1,
00249     0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x01, 0xC0, 0x80, 0x41,
00250     0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1,
00251     0x81, 0x40, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41,
00252     0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x01, 0xC0,
00253     0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x00, 0xC1, 0x81, 0x40,
00254     0x01, 0xC0, 0x80, 0x41, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1,
00255     0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40,
00256     0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x01, 0xC0,
00257     0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x00, 0xC1, 0x81, 0x40,
00258     0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0,
00259     0x80, 0x41, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40,
00260     0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x01, 0xC0,
00261     0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41,
00262     0x00, 0xC1, 0x81, 0x40, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0,
00263     0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41,
00264     0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0,
00265     0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x00, 0xC1, 0x81, 0x40,
00266     0x01, 0xC0, 0x80, 0x41, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1,
00267     0x81, 0x40, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41,
00268     0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x01, 0xC0,
00269     0x80, 0x41, 0x00, 0xC1, 0x81, 0x40
00270 };
00271
00272 /* Table of CRC values for low-order byte */
00273 static const uint8_t table_crc_lo[] = {
00274     0x00, 0xC0, 0xC1, 0x01, 0xC3, 0x03, 0x02, 0xC2, 0xC6, 0x06,
00275     0x07, 0xC7, 0x05, 0xC5, 0xC4, 0x04, 0xCC, 0x0C, 0x0D, 0xCD,
00276     0x0F, 0xCF, 0xCE, 0x0E, 0x0A, 0xCA, 0xCB, 0x0B, 0xC9, 0x09,
00277     0x08, 0xC8, 0xD8, 0x18, 0x19, 0xD9, 0x1B, 0xDB, 0xDA, 0x1A,
00278     0x1E, 0xDE, 0xDF, 0x1F, 0xDD, 0x1D, 0x1C, 0xDC, 0x14, 0xD4,
00279     0xD5, 0x15, 0xD7, 0x17, 0x16, 0xD6, 0xD2, 0x12, 0x13, 0xD3,
00280     0x11, 0xD1, 0xD0, 0x10, 0xF0, 0x30, 0x31, 0xF1, 0x33, 0xF3,
00281     0xF2, 0x32, 0x36, 0xF6, 0xF7, 0x37, 0xF5, 0x35, 0x34, 0xF4,
00282     0x3C, 0xFC, 0xFD, 0x3D, 0xFF, 0x3F, 0x3E, 0xFE, 0xFA, 0x3A,
00283     0x3B, 0xFB, 0x39, 0xF9, 0xF8, 0x38, 0x28, 0xE8, 0xE9, 0x29,
00284     0xEB, 0x2B, 0x2A, 0xEA, 0xEE, 0x2E, 0x2F, 0xEF, 0x2D, 0xED,
00285     0xEC, 0x2C, 0xE4, 0x24, 0x25, 0xE5, 0x27, 0xE7, 0xE6, 0x26,
00286     0x22, 0xE2, 0xE3, 0x23, 0xE1, 0x21, 0x20, 0xE0, 0xA0, 0x60,
00287     0x61, 0xA1, 0x63, 0xA3, 0xA2, 0x62, 0x66, 0xA6, 0xA7, 0x67,
00288     0xA5, 0x65, 0x64, 0xA4, 0x6C, 0xAC, 0xAD, 0x6D, 0xAF, 0x6F,
00289     0x6E, 0xAE, 0xAA, 0x6A, 0x6B, 0xAB, 0x69, 0xA9, 0xA8, 0x68,
00290     0x78, 0xB8, 0xB9, 0x79, 0xBB, 0x7B, 0x7A, 0xBA, 0xBE, 0x7E,
00291     0x7F, 0xBF, 0x7D, 0xBD, 0xBC, 0x7C, 0xB4, 0x74, 0x75, 0xB5,
00292     0x77, 0xB7, 0xB6, 0x76, 0x72, 0xB2, 0xB3, 0x73, 0xB1, 0x71,
00293     0x70, 0xB0, 0x50, 0x90, 0x91, 0x51, 0x93, 0x53, 0x52, 0x92,
00294     0x96, 0x56, 0x57, 0x97, 0x55, 0x95, 0x94, 0x54, 0x9C, 0x5C,
00295     0x5D, 0x9D, 0x5F, 0x9F, 0x9E, 0x5E, 0x5A, 0x9A, 0x9B, 0x5B,
00296     0x99, 0x59, 0x58, 0x98, 0x88, 0x48, 0x49, 0x89, 0x4B, 0x8B,
00297     0x8A, 0x4A, 0x4E, 0x8E, 0x8F, 0x4F, 0x8D, 0x4D, 0x4C, 0x8C,
00298     0x44, 0x84, 0x85, 0x45, 0x87, 0x47, 0x46, 0x86, 0x82, 0x42,
00299     0x43, 0x83, 0x41, 0x81, 0x80, 0x40
00300 };
00301
00302
00303 /******ModBus Functions******/
00304 #define READ_COILS 1
00305 #define READ_DISCRETE_INPUTS 2
00306 #define READ_HOLDING_REGISTERS 3
00307 #define READ_INPUT_REGISTERS 4
00308 #define WRITE_SINGLE_COIL 5
00309 #define WRITE_SINGLE_REGISTER 6
00310 #define WRITE_MULTIPLE_COILS 15
00311 #define WRITE_MULTIPLE_REGISTERS 16
00312
00313 #define BOFF 0
00314 #define BON 255
00315
00316 #define MM_ERROR_S MLED_SetHigh() // RED LED
00317 #define MM_ERROR_C MLED_SetLow() // RED LED
00318
00319 uint16_t crc16(volatile uint8_t *, uint16_t);
00320 uint16_t modbus_rtu_send_msg(void *, const void *, uint16_t);
00321
00322 void my_modbus_rx_32(void);
00323 uint8_t init_stream_params(void);
00324 void init_mb_master_timers(void);
00325 int8_t master_controller_work(C_data *);
00326 int32_t mb32_swap(const int32_t);
00327 int16_t mb16_swap(const int16_t);
00328
00329 void clear_2hz(void);
00330 void clear_500ahz(void);
00331 void clear_500hz(void);
```

```

00332     uint32_t get_2hz(const uint8_t);
00333     uint32_t get_500ahz(const uint8_t);
00334     uint32_t get_500hz(const uint8_t);
00335
00336     void timer_500ms_tick(void);
00337     void timer_2ms_tick(void);
00338
00339     void mb_tx_test(C_data *);
00340
00341     void mb_setup(void);
00342
00343     extern volatile struct V_type V;
00344     extern C_data C; // MODBUS client state data
00345     extern volatile M_data M; // MODBUS hardware state data
00346     extern volatile M_time_data MT; // MODBUS sequence timers
00347     extern EM_data1 em; // converted results data
00348     extern EM_data2 emt; // converted results data
00349     extern EM_serial ems; // converted results data
00350     extern EM_version emv; // converted results data
00351
00352 #ifdef __cplusplus
00353 }
00354 #endif
00355
00356 #endif /* MODBUS_MASTER_H */
00357

```

6.138 PIC18F47Q84-E_P/PADS/DIP1543W58P254L5175H635Q40N.d File Reference

6.139 PIC18F47Q84-E_P/PADS/PIC18F47Q84-E_P.c File Reference

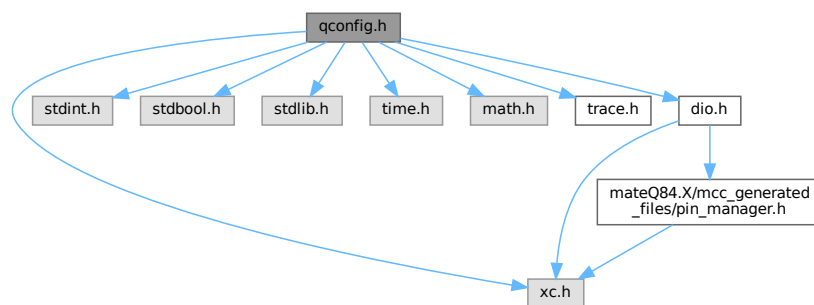
6.140 qconfig.h File Reference

```

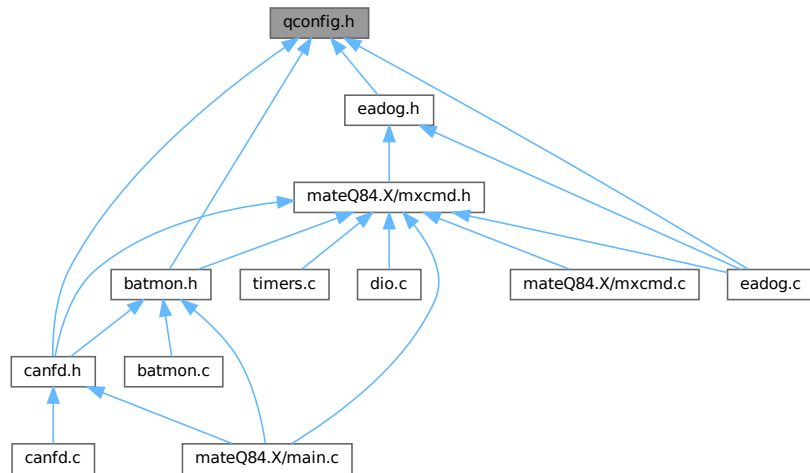
#include <xc.h>
#include <stdint.h>
#include <stdbool.h>
#include <stdlib.h>
#include <time.h>
#include <math.h>
#include "trace.h"
#include "dio.h"

```

Include dependency graph for qconfig.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct [spi_link_type](#)

Macros

- `#define INT_TRACE INT_TRACE_Toggle()`
- `#define NHD`
- `#define MAX_SHAPES 6`
- `#define SPIN_VAL_UPDATE 5`

Functions

- char [spinners](#) (uint8_t, const uint8_t)

Variables

- const char [spin](#) [MAX_SHAPES][20]
- volatile uint16_t [cc_mode](#)
- uint16_t [panel_watts](#)
- uint16_t [volt_whole](#)
- uint16_t [bat_amp_whole](#)
- uint16_t [volt_fract](#)
- uint16_t [vw](#)

6.140.1 Macro Definition Documentation

6.140.1.1 INT_TRACE

```
#define INT_TRACE INT_TRACE_Toggle()
```

6.140.1.2 MAX_SHAPES

```
#define MAX_SHAPES 6
```

6.140.1.3 NHD

```
#define NHD
```

6.140.1.4 SPIN_VAL_UPDATE

```
#define SPIN_VAL_UPDATE 5
```

6.140.2 Function Documentation

6.140.2.1 spinners()

```
char spinners (  
    uint8_t shape,  
    const uint8_t reset) [extern]
```

Here is the caller graph for this function:



6.140.3 Variable Documentation

6.140.3.1 bat_amp_whole

```
uint16_t bat_amp_whole
```

6.140.3.2 cc_mode

```
volatile uint16_t cc_mode [extern]
```

6.140.3.3 panel_watts

```
uint16_t panel_watts [extern]
```

6.140.3.4 spin

```
const char spin[MAX_SHAPES][20]
```

Initial value:

```
= {
    "| |//--",
    "| |//--\\\\\\",
    "000000--",
    "vv<<^>>",
    ". .**x#x#XX| |--",
    ". .oo00oo"
}
```

6.140.3.5 volt_fract

```
uint16_t volt_fract
```

6.140.3.6 volt_whole

```
uint16_t volt_whole
```

6.140.3.7 vw

```
uint16_t vw
```

6.141 qconfig.h

[Go to the documentation of this file.](#)

```
00001 /* Microchip Technology Inc. and its subsidiaries. You may use this software
00002  * and any derivatives exclusively with Microchip products.
00003  *
00004  * THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00005  * EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY IMPLIED
00006  * WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS FOR A
00007  * PARTICULAR PURPOSE, OR ITS INTERACTION WITH MICROCHIP PRODUCTS, COMBINATION
00008  * WITH ANY OTHER PRODUCTS, OR USE IN ANY APPLICATION.
00009  *
00010  * IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00011  * INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00012  * WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP HAS
00013  * BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO THE
00014  * FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL CLAIMS
00015  * IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT OF FEES, IF
00016  * ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS SOFTWARE.
00017  *
00018  * MICROCHIP PROVIDES THIS SOFTWARE CONDITIONALLY UPON YOUR ACCEPTANCE OF THESE
00019  * TERMS.
00020  */
00021
00022 /*
00023  * File:
00024  * Author:
00025  * Comments:
00026  * Revision history:
00027  */
00028
00029 // This is a guard condition so that contents of this file are not included
00030 // more than once.
00031 #ifndef QCONFIG_H
00032 #define QCONFIG_H
00033
00034 #include <xc.h> // include processor files - each processor file is guarded.
```

```

00035 #include <stdint.h>
00036 #include <stdbool.h>
00037 #include <stdlib.h>
00038 #include <time.h>
00039 #include <math.h>
00040 #include "trace.h"
00041 #include "dio.h"
00042
00043 // #define CAN_REMOTE // can received data on LCD for remote mate board
00044 // #define CAN_REMOTE_NODMA // don't use DMA in remote mode
00045
00046 #ifdef TRACE
00047 #define INT_TRACE INT_TRACE_Toggle() // Ext_IO PIN 5
00048 #else
00049 #define INT_TRACE ""
00050 #endif
00051
00052 // #define DATA_DEBUG // show CANBUS registers
00053 // #define SDEBUG // FM80 date/time DIAG scrolling text debug messages
00054 #define NHD // 4x20 SPI display
00055
00056 /*
00057  * timers used
00058  * 0 one second task
00059  * 2 ten second task
00060  * 4 500us FM80 I/O polling, sw timers and button press task
00061  * 5 500ms modbus task
00062  * 6 2ms modbus task
00063  */
00064
00065 /* spinner defines */
00066 #define MAX_SHAPES 6
00067 const char spin[MAX_SHAPES][20] = {
00068     "||/--", // classic LCD version with no \ character
00069     "||/--\\", // classic
00070     "OOOOO--", // eye blink
00071     "vv<^>", // point spinner
00072     "..*x#x#XX|--", // warp portal
00073     "..ooOoo" // ball bouncer
00074 };
00075
00076 #define SPIN_VAL_UPDATE 5
00077
00078 struct spi_link_type { // internal SPI state table
00079     uint8_t SPI_LCD : 1;
00080     uint8_t SPI_AUX : 1;
00081     uint8_t LCD_TIMER : 1;
00082     volatile uint8_t LCD_DATA : 1;
00083     uint16_t delay;
00084     uint8_t config;
00085     uint8_t * txbuf;
00086     volatile int32_t int_count;
00087 };
00088
00089 extern volatile uint16_t cc_mode;
00090 extern uint16_t panel_watts, volt_whole, bat_amp_whole, volt_fract, vw;
00091 extern char spinners(uint8_t, const uint8_t);
00092
00093 #ifdef __cplusplus
00094 extern "C" {
00095 #endif /* __cplusplus */
00096
00097     // TODO If C++ is being used, regular C code needs function names to have C
00098     // linkage so the functions can be used by the c code.
00099
00100 #ifdef __cplusplus
00101 }
00102 #endif /* __cplusplus */
00103
00104 #endif /* XC_HEADER_TEMPLATE_H */

```

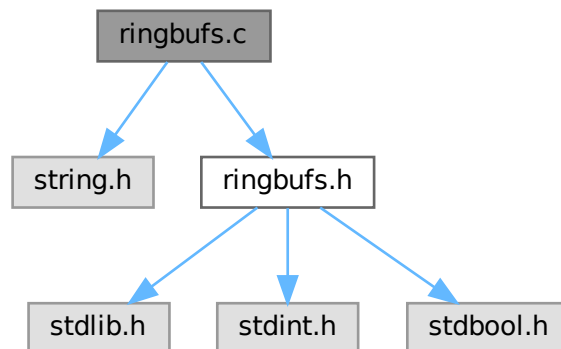
6.142 ringbufs.c File Reference

```

#include <string.h>
#include "ringbufs.h"

```


Include dependency graph for ringbufs.c:



Functions

- `uint8_t modulo_inc` (const `uint8_t` value, const `uint8_t` modulus)
- `uint8_t modulo_dec` (const `uint8_t` value, const `uint8_t` modulus)
- void * `ringBufS_init` (volatile `ringBufS_t` *_this)
- `int8_t ringBufS_empty` (`ringBufS_t` *_this)
- `int8_t ringBufS_full` (`ringBufS_t` *_this)
- `uint8_t ringBufS_get` (`ringBufS_t` *_this)
- void `ringBufS_put` (`ringBufS_t` *_this, const `uint8_t` c)
- void `ringBufS_put_dma` (`ringBufS_t` *_this, const `uint8_t` c)
- void `ringBufS_put_dma_cpy` (`ringBufS_t` *_this, const char *ptr, const `uint8_t` len)
- void * `ringBufS_flush` (`ringBufS_t` *_this, const bool clearBuffer)

6.142.1 Function Documentation

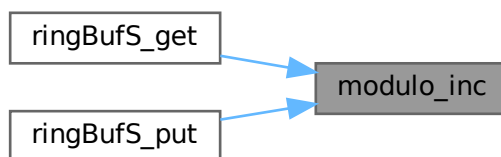
6.142.1.1 modulo_dec()

```
uint8_t modulo_dec (  
    const uint8_t value,  
    const uint8_t modulus)
```

6.142.1.2 modulo_inc()

```
uint8_t modulo_inc (  
    const uint8_t value,  
    const uint8_t modulus)
```

Here is the caller graph for this function:



6.142.1.3 ringBufS_empty()

```
int8_t ringBufS_empty (  
    ringBufS_t * _this)
```

6.142.1.4 ringBufS_flush()

```
void * ringBufS_flush (  
    ringBufS_t * _this,  
    const bool clearBuffer)
```

6.142.1.5 ringBufS_full()

```
int8_t ringBufS_full (  
    ringBufS_t * _this)
```

6.142.1.6 ringBufS_get()

```
uint8_t ringBufS_get (  
    ringBufS_t * _this)
```

Here is the call graph for this function:



6.142.1.7 ringBufS_init()

```
void * ringBufS_init (
    volatile ringBufS_t * _this)
```

6.142.1.8 ringBufS_put()

```
void ringBufS_put (
    ringBufS_t * _this,
    const uint8_t c)
```

Here is the call graph for this function:



6.142.1.9 ringBufS_put_dma()

```
void ringBufS_put_dma (
    ringBufS_t * _this,
    const uint8_t c)
```

6.142.1.10 ringBufS_put_dma_cpy()

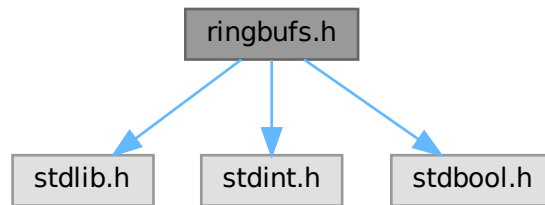
```
void ringBufS_put_dma_cpy (
    ringBufS_t * _this,
    const char * ptr,
    const uint8_t len)
```

6.143 ringbufs.h File Reference

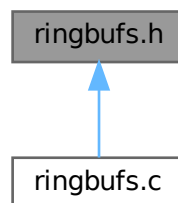
```
#include <stdlib.h>
#include <stdint.h>
```

```
#include <stdbool.h>
```

Include dependency graph for ringbufs.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct [ringBufS_t](#)

Macros

- `#define` [RBUF_SIZE](#) 32

Typedefs

- typedef struct ringBufS_t [ringBufS_t](#)

Functions

- void * [ringBufS_init](#) (volatile [ringBufS_t](#) * _this)
- int8_t [ringBufS_empty](#) ([ringBufS_t](#) * _this)
- int8_t [ringBufS_full](#) ([ringBufS_t](#) * _this)
- uint8_t [ringBufS_get](#) ([ringBufS_t](#) * _this)
- void [ringBufS_put](#) ([ringBufS_t](#) * _this, const uint8_t c)
- void [ringBufS_put_dma](#) ([ringBufS_t](#) * _this, const uint8_t c)
- void [ringBufS_put_dma_cpy](#) ([ringBufS_t](#) *, const char *, const uint8_t)
- void * [ringBufS_flush](#) ([ringBufS_t](#) * _this, const bool clearBuffer)

6.143.1 Macro Definition Documentation

6.143.1.1 RBUF_SIZE

```
#define RBUF_SIZE 32
```

6.143.2 Typedef Documentation

6.143.2.1 ringBufS_t

```
typedef struct ringBufS_t ringBufS_t
```

6.143.3 Function Documentation

6.143.3.1 ringBufS_empty()

```
int8_t ringBufS_empty (  
    ringBufS_t * _this)
```

6.143.3.2 ringBufS_flush()

```
void * ringBufS_flush (  
    ringBufS_t * _this,  
    const bool clearBuffer)
```

6.143.3.3 ringBufS_full()

```
int8_t ringBufS_full (  
    ringBufS_t * _this)
```

6.143.3.4 ringBufS_get()

```
uint8_t ringBufS_get (  
    ringBufS_t * _this)
```

Here is the call graph for this function:



6.143.3.5 ringBufS_init()

```
void * ringBufS_init (
    volatile ringBufS_t * _this)
```

6.143.3.6 ringBufS_put()

```
void ringBufS_put (
    ringBufS_t * _this,
    const uint8_t c)
```

Here is the call graph for this function:



6.143.3.7 ringBufS_put_dma()

```
void ringBufS_put_dma (
    ringBufS_t * _this,
    const uint8_t c)
```

6.143.3.8 ringBufS_put_dma_cpy()

```
void ringBufS_put_dma_cpy (
    ringBufS_t * _this,
    const char * ptr,
    const uint8_t len)
```

6.144 ringbufs.h

[Go to the documentation of this file.](#)

```
00001 /*
00002  * File:   ringbufs.h
00003  * Author: rootswitcDC circuit breakerh debounce icdebounce icfuse type T
00004  *
00005  * Created on July 25, 2015, 2:03 PM
00006  */
00007
00008 #ifndef RINGBUFS_H
00009 #define RINGBUFS_H
00010
00011 #ifdef __cplusplus
00012 extern "C" {
00013 #endif
00014 #include <stdlib.h>
```

```

00015 #include <stdint.h>
00016 #include <stdbool.h>
00017 #define RBUF_SIZE 32
00018
00019 typedef struct ringBufS_t {
00020     uint8_t buf[RBUF_SIZE];
00021     uint8_t head;
00022     uint8_t tail;
00023     uint8_t count;
00024 } ringBufS_t;
00025
00026 void *ringBufS_init(volatile ringBufS_t *_this);
00027 int8_t ringBufS_empty(ringBufS_t *_this);
00028 int8_t ringBufS_full(ringBufS_t *_this);
00029 uint8_t ringBufS_get(ringBufS_t *_this);
00030 void ringBufS_put(ringBufS_t *_this, const uint8_t c);
00031 void ringBufS_put_dma(ringBufS_t *_this, const uint8_t c);
00032 void ringBufS_put_dma_cpy(ringBufS_t *, const char *, const uint8_t);
00033 void *ringBufS_flush(ringBufS_t *_this, const bool clearBuffer);
00034
00035 #ifdef __cplusplus
00036 }
00037 #endif
00038 #endif /* RINGBUFS_H */
00041

```

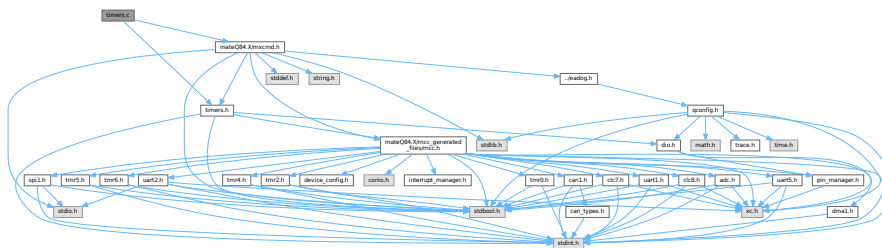
6.145 timers.c File Reference

```

#include "timers.h"
#include "mateQ84.X/mxcmd.h"

```

Include dependency graph for timers.c:



Functions

- void [StartTimer](#) (const uint8_t timer, const uint16_t count)
- bool [TimerDone](#) (const uint8_t timer)
- void [WaitMs](#) (const uint16_t numMilliseconds)
- void [timer_ms_tick](#) (const uint32_t status, const uintptr_t context)
- void [delay_us](#) (uint32_t us)
- void [delay_ms](#) (const uint16_t ms)

Variables

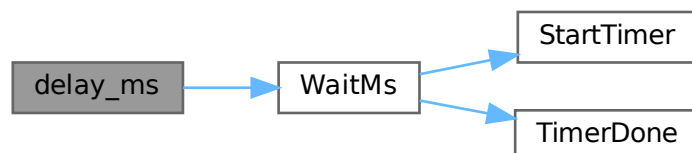
- volatile uint16_t [tickCount](#) [[TMR_COUNT](#)]

6.145.1 Function Documentation

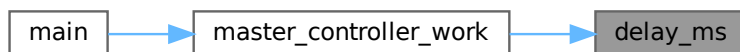
6.145.1.1 delay_ms()

```
void delay_ms (
    const uint16_t ms)
```

Here is the call graph for this function:



Here is the caller graph for this function:



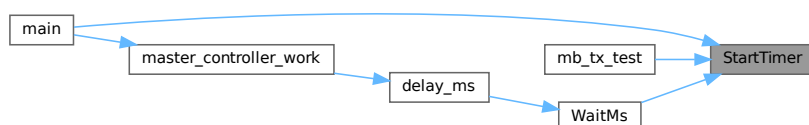
6.145.1.2 delay_us()

```
void delay_us (
    uint32_t us)
```

6.145.1.3 StartTimer()

```
void StartTimer (
    const uint8_t timer,
    const uint16_t count)
```

Here is the caller graph for this function:



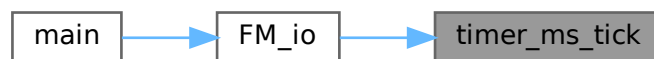
6.145.1.4 timer_ms_tick()

```
void timer_ms_tick (  
    const uint32_t status,  
    const uintptr_t context)
```

Here is the call graph for this function:



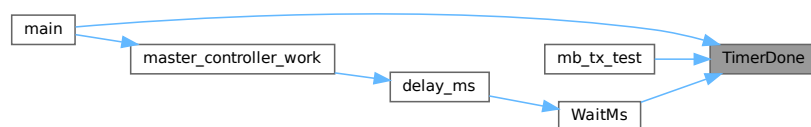
Here is the caller graph for this function:



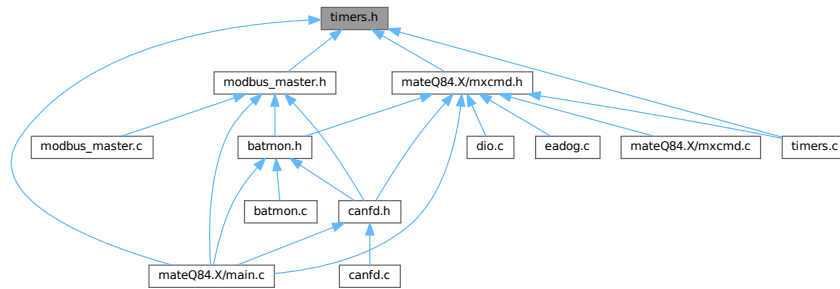
6.145.1.5 TimerDone()

```
bool TimerDone (  
    const uint8_t timer)
```

Here is the caller graph for this function:



This graph shows which files directly or indirectly include this file:



Enumerations

- enum [APP_TIMERS](#) {
[TMR_INTERNAL](#) = 0 , [TMR_MBMASTER](#) , [TMR_MBTEST](#) , [TMR_SPIN](#) ,
[TMR_DERE](#) , [TMR_COUNT](#) }

Functions

- void [StartTimer](#) (const uint8_t timer, const uint16_t count)
- bool [TimerDone](#) (const uint8_t timer)
- void [WaitMs](#) (const uint16_t numMilliseconds)
- void [timer_ms_tick](#) (const uint32_t, const uintptr_t)
- void [delay_ms](#) (const uint16_t)

6.146.1 Enumeration Type Documentation

6.146.1.1 APP_TIMERS

enum [APP_TIMERS](#)

Enumerator

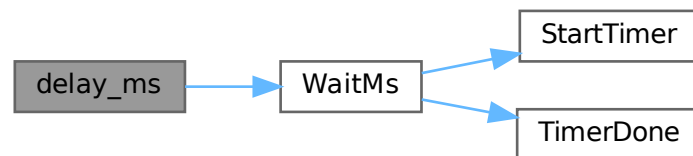
TMR_INTERNAL	
TMR_MBMASTER	
TMR_MBTEST	
TMR_SPIN	
TMR_DERE	
TMR_COUNT	

6.146.2 Function Documentation

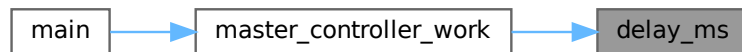
6.146.2.1 delay_ms()

```
void delay_ms (  
    const uint16_t ms)
```

Here is the call graph for this function:



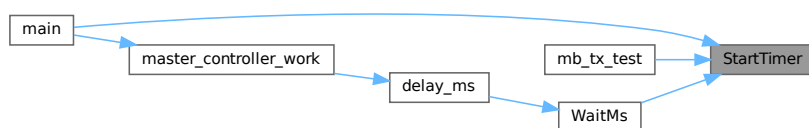
Here is the caller graph for this function:



6.146.2.2 StartTimer()

```
void StartTimer (  
    const uint8_t timer,  
    const uint16_t count)
```

Here is the caller graph for this function:



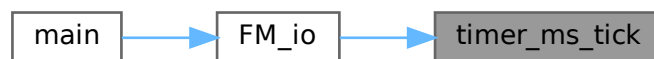
6.146.2.3 timer_ms_tick()

```
void timer_ms_tick (  
    const uint32_t status,  
    const uintptr_t context)
```

Here is the call graph for this function:



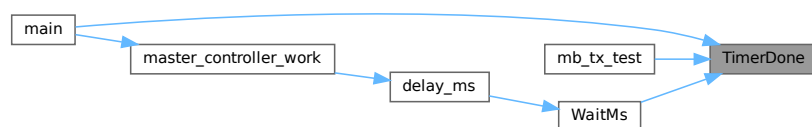
Here is the caller graph for this function:



6.146.2.4 TimerDone()

```
bool TimerDone (  
    const uint8_t timer)
```

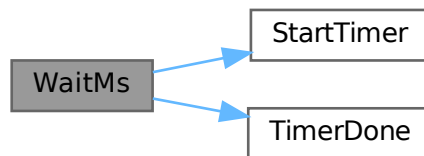
Here is the caller graph for this function:



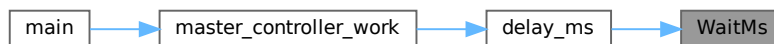
6.146.2.5 WaitMs()

```
void WaitMs (
    const uint16_t numMilliseconds)
```

Here is the call graph for this function:



Here is the caller graph for this function:



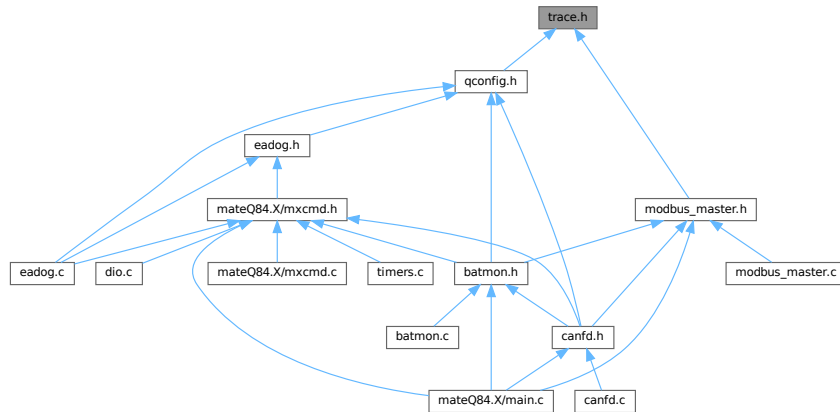
6.147 timers.h

[Go to the documentation of this file.](#)

```
00001 #ifndef TIMERS_H
00002 #define TIMERS_H
00003
00004 #include <stdint.h>
00005 #include <stdbool.h>
00006 #include "mateQ84.X/mcc_generated_files/mcc.h"
00007 #include "dio.h"
00008
00009 //Software timers - use these to refer to timers rather than
00010 //integers. Add more as needed.
00011 //NOTE: 32767 ms max delay
00012
00013 enum APP_TIMERS {
00014     TMR_INTERNAL = 0, //Used in timers.c - do not remove or use elsewhere
00015     TMR_MBMASTER,
00016     TMR_MBTEST,
00017     TMR_SPIN,
00018     TMR_DERE,
00019     //
00020     //(Add timers here as needed)
00021     //
00022     TMR_COUNT //number of timers - always keep at end of enum!
00023 };
00024
00025 void StartTimer(const uint8_t timer, const uint16_t count);
00026 bool TimerDone(const uint8_t timer);
00027 void WaitMs(const uint16_t numMilliseconds);
00028 void timer_ms_tick(const uint32_t, const uintptr_t);
00029
00030 void delay_ms(const uint16_t);
00031
00032 #endif //TIMERS_H
```

6.148 trace.h File Reference

This graph shows which files directly or indirectly include this file:



Macros

- `#define TRACE`

6.148.1 Macro Definition Documentation

6.148.1.1 TRACE

```
#define TRACE
```

6.149 trace.h

[Go to the documentation of this file.](#)

```

00001 /*
00002  * File:   trace.h
00003  * Author: root
00004  *
00005  * Created on October 11, 2023, 5:33 PM
00006  *
00007  * enable/disable various GPIO pins for program timing and debugging with logic analyzers
00008  * and o-scopes
00009  *
00010  * adds small program delay and latencies when defined
00011  */
00012
00013 #ifndef TRACE_H
00014 #define TRACE_H
00015
00016 #ifdef __cplusplus
00017 extern "C" {
00018 #endif
00019
00020 #define TRACE
00021 // EXT_IO pin 4, MISC_IO pin 6, SPI DMA driver toggles to RB5
00022 // EXT_IO PIN 5, interrupt toggles to RB6 PGC
00023 // EXT_IO pin 6, composite I/O from CLC7 to RB7 PGD
00024 // EXT_IO pin 2, MISC_IO pin 5, main program trace high to RD5

```

```
00025 // EXT_IO pin 3, MISC_IO pin 7, RD7
00026
00027
00028 #ifdef __cplusplus
00029 }
00030 #endif
00031
00032 #endif /* TRACE_H */
00033
```


Chapter 7

Examples

7.1 simple_publisher.c

A simple program to that publishes the current time whenever ENTER is pressed.

Usage:

```
./bin/simple_publisher [address [port [topic]]]
```

Where `address` is the address of the MQTT broker, `port` is the port number the MQTT broker is running on, and `topic` is the name of the topic to publish with. Note that all these arguments are optional and the defaults are `address = "test.mosquitto.org"`, `port = "1883"`, and `topic = "datetime"`.

7.2 simple_subscriber.c

A simple program that subscribes to a single topic and prints all updates that are received.

Usage:

```
./bin/simple_subscriber [address [port [topic]]]
```

Where `address` is the address of the MQTT broker, `port` is the port number the MQTT broker is running on, and `topic` is the name of the topic subscribe to. Note that all these arguments are optional and the defaults are `address = "test.mosquitto.org"`, `port = "1883"`, and `topic = "datetime"`.

7.3 reconnect_subscriber.c

Same program as [simple_subscriber.c](#), but using the automatic reconnect functionality.

7.4 bio_publisher.c

Same program as [simple_publisher.c](#), but uses a unencrypted BIO socket.

7.5 openssl_publisher.c

Same program as [simple_publisher.c](#), but over an encrypted connection using OpenSSL.

Usage:

```
./bin/openssl_publisher ca_file [address [port [topic]]]
```


Index

- `__DEFAULT_SOURCE`
 - `raw.githubusercontent.com_linux-can_can-utils_master_can-utils.c`, 258
 - `__XTAL_FREQ`
 - `device_config.h`, 353
 - `__ALL_MQTT_ERRORS`
 - `mqtt.h`, 215
 - `__interrupt`
 - `adc.c`, 287
 - `can1.c`, 296, 297
 - `dma1.c`, 355
 - `interrupt_manager.c`, 375
 - `pin_manager.c`, 387
 - `tmr0.c`, 490
 - `tmr2.c`, 502
 - `tmr4.c`, 530
 - `tmr5.c`, 557
 - `tmr6.c`, 572
 - `uart2.c`, 617
 - `uart5.c`, 641
 - `__mqtt_next_pid`
 - Utilities, 35
 - `__mqtt_pack_str`
 - `mqtt.h`, 216
 - `__mqtt_pack_uint16`
 - `mqtt.h`, 217
 - `__mqtt_packed_cstrlen`
 - `mqtt.h`, 215
 - `__mqtt_ping`
 - `mqtt.h`, 218
 - `__mqtt_puback`
 - Utilities, 35
 - `__mqtt_pubcomp`
 - Utilities, 35
 - `__mqtt_pubrec`
 - Utilities, 36
 - `__mqtt_pubrel`
 - Utilities, 36
 - `__mqtt_recv`
 - Utilities, 36
 - `__mqtt_send`
 - Utilities, 37
 - `__mqtt_unpack_uint16`
 - `mqtt.h`, 218
- a
 - `mx_log_packed_t`, 120
 - `mx_status_packed_t`, 122
- a_pin
 - B_type, 68
- a_switch
 - B_type, 68
- A_SWITCH_ANS
 - `pin_manager.h`, 402
- A_SWITCH_GetValue
 - `pin_manager.h`, 402
- A_SWITCH_LAT
 - `pin_manager.h`, 402
- A_SWITCH_OD
 - `pin_manager.h`, 402
- A_SWITCH_PORT
 - `pin_manager.h`, 403
- A_SWITCH_ResetPullup
 - `pin_manager.h`, 403
- A_SWITCH_SetAnalogMode
 - `pin_manager.h`, 403
- A_SWITCH_SetDigitalInput
 - `pin_manager.h`, 403
- A_SWITCH_SetDigitalMode
 - `pin_manager.h`, 403
- A_SWITCH_SetDigitalOutput
 - `pin_manager.h`, 403
- A_SWITCH_SetHigh
 - `pin_manager.h`, 403
- A_SWITCH_SetLow
 - `pin_manager.h`, 404
- A_SWITCH_SetOpenDrain
 - `pin_manager.h`, 404
- A_SWITCH_SetPullup
 - `pin_manager.h`, 404
- A_SWITCH_SetPushPull
 - `pin_manager.h`, 404
- A_SWITCH_Toggle
 - `pin_manager.h`, 404
- A_SWITCH_TRIS
 - `pin_manager.h`, 404
- A_SWITCH_WPU
 - `pin_manager.h`, 404
- a_trigger
 - B_type, 68
- a_type
 - B_type, 68
- absorb_time
 - `mx_logpage_t`, 120
- ac_Wh_daily
 - `batmon.c`, 138
 - `batmon.h`, 149
- ac_Wh_daily_prev
 - `batmon.c`, 138

- batmon.h, [149](#)
- acamps
 - raw.githubusercontent.com_linux-can_can-utils_master_canutils_264
- acenergy
 - raw.githubusercontent.com_linux-can_can-utils_master_canutils_264
- acvolts
 - raw.githubusercontent.com_linux-can_can-utils_master_canutils_265
- adc.c
 - __interrupt, [287](#)
- ADC3: Analog-to-Digital Converter with Context, [42](#)
 - ADC_channel_t, [45](#)
 - ADC_ClearAccumulator, [45](#)
 - ADC_context_t, [45](#)
 - ADC_DefineSetPoint, [45](#)
 - ADC_DisableChannelScan, [46](#)
 - ADC_DisableChannelSequencer, [46](#)
 - ADC_DisableChargePump, [47](#)
 - ADC_DisableContinuousConversion, [47](#)
 - ADC_DischargeSampleCapacitor, [48](#)
 - ADC_EnableChannelScan, [48](#)
 - ADC_EnableChannelSequencer, [48](#)
 - ADC_EnableChargePump, [49](#)
 - ADC_EnableContinuousConversion, [49](#)
 - ADC_EnableDoubleSampling, [50](#)
 - ADC_GetAccumulatorValue, [50](#)
 - ADC_GetConversionResult, [51](#)
 - ADC_GetConversionStageStatus, [51](#)
 - ADC_GetCurrentCountofConversions, [52](#)
 - ADC_GetErrorCalculation, [52](#)
 - ADC_GetFilterValue, [52](#)
 - ADC_GetPreviousResult, [53](#)
 - ADC_GetSingleConversion, [53](#)
 - ADC_HasAccumulatorOverflowed, [54](#)
 - ADC_HasErrorCrossedLowerThreshold, [54](#)
 - ADC_HasErrorCrossedUpperThreshold, [55](#)
 - ADC_Initialize, [55](#)
 - ADC_IsConversionDone, [56](#)
 - ADC_LoadAcquisitionRegister, [57](#)
 - adc_result_t, [44](#)
 - ADC_SelectContext, [57](#)
 - ADC_SetActiveClockTuningInterruptHandler, [57](#)
 - ADC_SetADInterruptHandler, [58](#)
 - ADC_SetContext1ThresholdInterruptHandler, [59](#)
 - ADC_SetContext2ThresholdInterruptHandler, [60](#)
 - ADC_SetContext3ThresholdInterruptHandler, [60](#)
 - ADC_SetContext4ThresholdInterruptHandler, [61](#)
 - ADC_SetLowerThreshold, [61](#)
 - ADC_SetPrechargeTime, [62](#)
 - ADC_SetRepeatCount, [62](#)
 - ADC_SetStopOnInterrupt, [62](#)
 - ADC_SetUpperThreshold, [63](#)
 - ADC_StartChannelSequencer, [63](#)
 - ADC_StartConversion, [64](#)
 - ADC_StopConversion, [64](#)
 - channel_ANA0, [45](#)
 - channel_ANA4, [45](#)
 - channel_DAC1, [45](#)
 - channel_FVR_Buffer1, [45](#)
 - channel_FVR_Buffer2, [45](#)
 - channel_Temp, [45](#)
 - channel_VSS, [45](#)
 - mTouchContext, [45](#)
 - ADC_channel_t
 - ADC3: Analog-to-Digital Converter with Context, [45](#)
 - ADC_ClearAccumulator
 - ADC3: Analog-to-Digital Converter with Context, [45](#)
 - ADC_context_t
 - ADC3: Analog-to-Digital Converter with Context, [45](#)
 - ADC_DefineSetPoint
 - ADC3: Analog-to-Digital Converter with Context, [45](#)
 - ADC_DisableChannelScan
 - ADC3: Analog-to-Digital Converter with Context, [46](#)
 - ADC_DisableChannelSequencer
 - ADC3: Analog-to-Digital Converter with Context, [46](#)
 - ADC_DisableChargePump
 - ADC3: Analog-to-Digital Converter with Context, [47](#)
 - ADC_DisableContinuousConversion
 - ADC3: Analog-to-Digital Converter with Context, [47](#)
 - ADC_DischargeSampleCapacitor
 - ADC3: Analog-to-Digital Converter with Context, [48](#)
 - ADC_EnableChannelScan
 - ADC3: Analog-to-Digital Converter with Context, [48](#)
 - ADC_EnableChannelSequencer
 - ADC3: Analog-to-Digital Converter with Context, [48](#)
 - ADC_EnableChargePump
 - ADC3: Analog-to-Digital Converter with Context, [49](#)
 - ADC_EnableContinuousConversion
 - ADC3: Analog-to-Digital Converter with Context, [49](#)
 - ADC_EnableDoubleSampling
 - ADC3: Analog-to-Digital Converter with Context, [50](#)
 - ADC_GetAccumulatorValue
 - ADC3: Analog-to-Digital Converter with Context, [50](#)
 - ADC_GetConversionResult
 - ADC3: Analog-to-Digital Converter with Context, [51](#)
 - ADC_GetConversionStageStatus
 - ADC3: Analog-to-Digital Converter with Context, [51](#)
 - ADC_GetCurrentCountofConversions
 - ADC3: Analog-to-Digital Converter with Context, [52](#)
 - ADC_GetErrorCalculation
 - ADC3: Analog-to-Digital Converter with Context, [52](#)
 - ADC_GetFilterValue
 - ADC3: Analog-to-Digital Converter with Context, [52](#)
 - ADC_GetPreviousResult
 - ADC3: Analog-to-Digital Converter with Context, [53](#)
 - ADC_GetSingleConversion
 - ADC3: Analog-to-Digital Converter with Context, [53](#)
 - ADC_HasAccumulatorOverflowed
 - ADC3: Analog-to-Digital Converter with Context, [54](#)
 - ADC_HasErrorCrossedLowerThreshold
 - ADC3: Analog-to-Digital Converter with Context, [54](#)
 - ADC_HasErrorCrossedUpperThreshold
 - ADC3: Analog-to-Digital Converter with Context, [55](#)
 - ADC_Initialize

- ADC3: Analog-to-Digital Converter with Context, [55](#)
- ADC_IsConversionDone
 - ADC3: Analog-to-Digital Converter with Context, [56](#)
- ADC_LoadAcquisitionRegister
 - ADC3: Analog-to-Digital Converter with Context, [57](#)
- adc_result_t
 - ADC3: Analog-to-Digital Converter with Context, [44](#)
- ADC_SelectContext
 - ADC3: Analog-to-Digital Converter with Context, [57](#)
- ADC_SetActiveClockTuningInterruptHandler
 - ADC3: Analog-to-Digital Converter with Context, [57](#)
- ADC_SetADIIInterruptHandler
 - ADC3: Analog-to-Digital Converter with Context, [58](#)
- ADC_SetContext1ThresholdInterruptHandler
 - ADC3: Analog-to-Digital Converter with Context, [59](#)
- ADC_SetContext2ThresholdInterruptHandler
 - ADC3: Analog-to-Digital Converter with Context, [60](#)
- ADC_SetContext3ThresholdInterruptHandler
 - ADC3: Analog-to-Digital Converter with Context, [60](#)
- ADC_SetContext4ThresholdInterruptHandler
 - ADC3: Analog-to-Digital Converter with Context, [61](#)
- ADC_SetLowerThreshold
 - ADC3: Analog-to-Digital Converter with Context, [61](#)
- ADC_SetPrechargeTime
 - ADC3: Analog-to-Digital Converter with Context, [62](#)
- ADC_SetRepeatCount
 - ADC3: Analog-to-Digital Converter with Context, [62](#)
- ADC_SetStopOnInterrupt
 - ADC3: Analog-to-Digital Converter with Context, [62](#)
- ADC_SetUpperThreshold
 - ADC3: Analog-to-Digital Converter with Context, [63](#)
- ADC_StartChannelSequencer
 - ADC3: Analog-to-Digital Converter with Context, [63](#)
- ADC_StartConversion
 - ADC3: Analog-to-Digital Converter with Context, [64](#)
- ADC_StopConversion
 - ADC3: Analog-to-Digital Converter with Context, [64](#)
- addr
 - mqtt_pub.c, [232](#)
- ADDR_MQTT
 - mqtt_pub.h, [234](#)
- ADDRESS
 - raw.githubusercontent.com_linux-can_can-utils_master_canutils.c, [258](#)
- al1
 - EM_data1, [84](#)
- al2
 - EM_data1, [84](#)
- al3
 - EM_data1, [84](#)
- all_relays_off
 - dio.c, [171](#)
 - dio.h, [175](#)
- alt_display
 - B_type, [68](#)
 - EB_data, [81](#)
- amp_hours
 - mx_logpage_t, [120](#)
- AMP_WHOLE_ZERO
 - mxcmd.h, [667](#)
- amps_peak
 - mx_logpage_t, [121](#)
- ANALOG
 - pin_manager.h, [405](#)
- API, [9](#)
 - mqtt_connect, [11](#)
 - mqtt_disconnect, [12](#)
 - mqtt_error_str, [12](#)
 - MQTT_ERROR_UNKNOWN, [10](#)
 - mqtt_init, [13](#)
 - mqtt_init_reconnect, [15](#)
 - MQTT_OK, [10](#)
 - mqtt_ping, [15](#)
 - mqtt_publish, [16](#)
 - mqtt_reconnect, [17](#)
 - mqtt_reinit, [17](#)
 - mqtt_subscribe, [18](#)
 - mqtt_sync, [19](#)
 - mqtt_unsubscribe, [19](#)
 - MQTTErrors, [10](#)
- APP_TIMERS
 - timers.h, [725](#)
- application_message
 - mqtt_response_publish, [112](#)
- application_message_size
 - mqtt_response_publish, [112](#)
- B
 - main.c, [280](#)
 - mxcmd.h, [672](#)
- B_type, [67](#)
 - a_pin, [68](#)
 - a_switch, [68](#)
 - a_trigger, [68](#)
 - a_type, [68](#)
 - alt_display, [68](#)
 - canbus_online, [68](#)
 - day_check, [68](#)
 - dim_delay, [69](#)
 - display_dim, [69](#)
 - display_on, [69](#)
 - display_update, [69](#)
 - flush, [69](#)
 - FM80_charged, [69](#)
 - FM80_io, [69](#)
 - FM80_online, [69](#)
 - fwrev, [69](#)
 - LOG, [69](#)
 - log, [70](#)
 - modbus_online, [70](#)
 - mui, [70](#)
 - mxcmd.h, [669](#)
 - net_balance, [70](#)
 - node_id, [70](#)
 - once, [70](#)
 - one_sec_flag, [70](#)
 - pacing, [70](#)

- pv_high, 70
- pv_prev, 70
- pv_update, 71
- run_time, 71
- rx_count, 71
- ten_sec_flag, 71
- bamps
 - raw.githubusercontent.com_linux-can_can-utils_master_canutils.c, 265
- BAT_AH
 - batmon.h, 141
- bat_amp_whole
 - EB_data, 81
 - main.c, 280
 - qconfig.h, 712
- BAT_CHARGED_V
 - batmon.h, 141
- BAT_CHARGED_W
 - batmon.h, 141
- BAT_CYCLES
 - batmon.h, 141
- bat_cycles
 - EB_data, 81
- BAT_DAY_COUNT
 - batmon.h, 141
- BAT_EFF_VAL
 - batmon.h, 141
- BAT_ENERGY
 - batmon.h, 141
- bat_energy
 - EB_data, 81
- bat_energy_kw
 - raw.githubusercontent.com_linux-can_can-utils_master_canutils.c, 265
- BAT_ENERGY_LOW
 - batmon.h, 141
- bat_energy_scaled
 - raw.githubusercontent.com_linux-can_can-utils_master_canutils.c, 265
- BAT_LOW_DISC
 - batmon.h, 142
- BAT_LOW_RECON
 - batmon.h, 142
- bat_max
 - mx_logpage_t, 121
- bat_min
 - mx_logpage_t, 121
- bat_mode
 - EB_data, 81
- BAT_NIGHT_COUNT
 - batmon.h, 142
- BAT_OVER_RECON
 - batmon.h, 142
- BAT_OVER_VOLT
 - batmon.h, 142
- BAT_RUN_MAX
 - batmon.h, 142
- bat_time
 - EB_data, 81
- BAT_UNDER_VOLT
 - batmon.h, 142
- BAT_VOLT
 - batmon.h, 142
- batmon.c, 133
 - ac_Wh_daily, 138
 - ac_Wh_daily_prev, 138
 - compute_bm_data, 134
 - DATAEE_ReadByte, 134
 - DATAEE_WriteByte, 134
 - DeviceID_Read, 135
 - EBD, 138
 - EBD_update, 138
 - get_bm_data, 135
 - get_EEPROM, 136
 - initbm_data, 136
 - pv_Wh_daily, 139
 - pv_Wh_daily_prev, 139
 - update_time, 137
 - Volts_to_SOC, 137
 - wr_bm_data, 137
- batmon.h, 139
 - ac_Wh_daily, 149
 - ac_Wh_daily_prev, 149
 - BAT_AH, 141
 - BAT_CHARGED_V, 141
 - BAT_CHARGED_W, 141
 - BAT_CYCLES, 141
 - BAT_DAY_COUNT, 141
 - BAT_EFF_VAL, 141
 - BAT_ENERGY, 141
 - BAT_ENERGY_LOW, 141
 - BAT_LOW_DISC, 142
 - BAT_LOW_RECON, 142
 - BAT_NIGHT_COUNT, 142
 - BAT_OVER_RECON, 142
 - BAT_OVER_VOLT, 142
 - BAT_RUN_MAX, 142
 - BAT_UNDER_VOLT, 142
 - BAT_VOLT, 142
 - BM_CM, 142
 - BM_EEPROM_VER, 142
 - BM_UPDATE, 143
 - BM_UPDATE_RUN, 143
 - BVSOC_SLOTS, 143
 - BVSOC_TABLE, 150
 - can_newtime, 150
 - CHK_DAY_TIME, 143
 - compute_bm_data, 144
 - DATAEE_ReadByte, 144
 - DATAEE_WriteByte, 145
 - device_id_address_t, 144
 - device_id_data_t, 144
 - DeviceID_Read, 145
 - EB_data, 144
 - EBD, 150
 - EBD_update, 150

- get_bm_data, 145
- get_EEPROM, 146
- IDLE_DRAIN, 143
- initbm_data, 146
- INV_EFF_VAL, 143
- log_format, 150
- LOG_VARS, 143
- MAX_ALT_DIS, 143
- pv_Wh_daily, 150
- pv_Wh_daily_prev, 150
- run_day_to_night, 147
- run_night_to_day, 147
- TEN_SEC_HOUR, 143
- update_time, 148
- Volts_to_SOC, 148
- wr_bm_data, 149
- baud
 - spi1_configuration_t, 124
- bearssl_sockets.h
 - close_socket, 244
 - open_nb_socket, 244
- benergy
 - raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c, 265
- bio_sockets.h
 - open_nb_socket, 246
- blink_lock
 - M_data, 91
- blob
 - blob_log, 72
 - blob_type, 72
- blob_log, 71
 - blob, 72
 - log, 72
- blob_type, 72
 - blob, 72
 - rx_flag, 72
 - tx_flag, 72
- BM_CM
 - batmon.h, 142
- BM_EEPROM_VER
 - batmon.h, 142
- BM_UPDATE
 - batmon.h, 143
- BM_UPDATE_RUN
 - batmon.h, 143
- BOFF
 - modbus_master.h, 692
- BON
 - modbus_master.h, 692
- boot_code
 - M_data, 91
- bpwm
 - PWMDC, 122
- brs
 - CAN_MSG_FIELD, 78
- BRS_POSN
 - can1.c, 294
- buf
 - ringBufS_t, 123
- buffer
 - main.c, 280
- BUFFER_SPACING
 - mxcmd.h, 667
- build_date
 - main.c, 281
- build_time
 - main.c, 281
- build_version
 - mxcmd.h, 672
- button_press_check
 - dio.c, 171
 - dio.h, 175
- bvolts
 - raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c, 265
- BVSOC_SLOTS
 - batmon.h, 143
- BVSOC_TABLE
 - batmon.h, 150
- Byte1
 - canfd.c, 159
- Byte2
 - canfd.c, 159
- bytes
 - MREG, 118
 - MREG32, 119
 - MREG64, 119
- C
 - modbus_master.c, 685
 - modbus_master.h, 705
- C_data, 73
 - config_ok, 73
 - cstate, 73
 - data_count, 73
 - data_ok, 74
 - data_prev, 74
 - id_ok, 74
 - light_ok, 74
 - M, 74
 - mcmd, 74
 - modbus_command, 74
 - modbus_master.h, 695
 - passwd_ok, 74
 - req_length, 74
 - serial_ok, 74
 - tm_ok, 75
 - trace, 75
 - version_ok, 75
- ca_crt
 - mbedtls_context, 94
- calc_checksum
 - mxcmd.c, 662
 - mxcmd.h, 669
- CAN-Examples/canfilter.c, 152
- CAN-Examples/canreceive.c, 153

- CAN-Examples/cantransmit.c, [154](#)
- CAN-Examples/README.md, [154](#)
- can1.c
 - [__interrupt](#), [296](#), [297](#)
 - [BRS_POSN](#), [294](#)
 - [CAN1_Initialize](#), [297](#)
 - [CAN1_IsBusOff](#), [297](#)
 - [CAN1_IsRxErrorActive](#), [298](#)
 - [CAN1_IsRxErrorPassive](#), [299](#)
 - [CAN1_IsRxErrorWarning](#), [300](#)
 - [CAN1_IsTxErrorActive](#), [300](#)
 - [CAN1_IsTxErrorPassive](#), [301](#)
 - [CAN1_IsTxErrorWarning](#), [302](#)
 - [CAN1_OperationModeGet](#), [302](#)
 - [CAN1_OperationModeSet](#), [303](#)
 - [CAN1_Receive](#), [304](#)
 - [CAN1_ReceivedMessageCountGet](#), [304](#)
 - [CAN1_ReceiveFrom](#), [305](#)
 - [CAN1_RX_FIFO_ResetInfo](#), [305](#)
 - [CAN1_SetBusErrorInterruptHandler](#), [306](#)
 - [CAN1_SetBusWakeUpActivityInterruptHandler](#), [306](#)
 - [CAN1_SetFIFO1NotEmptyHandler](#), [307](#)
 - [CAN1_SetInvalidMessageInterruptHandler](#), [307](#)
 - [CAN1_SetModeChangeInterruptHandler](#), [308](#)
 - [CAN1_SetRxBufferOverflowInterruptHandler](#), [308](#)
 - [CAN1_SetSystemErrorInterruptHandler](#), [309](#)
 - [CAN1_SetTxAttemptInterruptHandler](#), [310](#)
 - [CAN1_SetTXQNotFullHandler](#), [310](#)
 - [CAN1_Sleep](#), [311](#)
 - [CAN1_Transmit](#), [312](#)
 - [CAN1_TransmitFIFOStatusGet](#), [313](#)
 - [CAN_RX_FIFO_STATUS](#), [296](#)
 - [CAN_RX_MSG_AVAILABLE](#), [296](#)
 - [CAN_RX_MSG_NOT_AVAILABLE](#), [296](#)
 - [CAN_RX_MSG_OVERFLOW](#), [296](#)
 - [DLCToPayloadBytes](#), [294](#)
 - [EID_HIGH_MASK](#), [294](#)
 - [EID_HIGH_WIDTH](#), [294](#)
 - [EID_LOW_MASK](#), [295](#)
 - [EID_LOW_POSN](#), [295](#)
 - [EID_LOW_WIDTH](#), [295](#)
 - [EID_MID_WIDTH](#), [295](#)
 - [FDF_POSN](#), [295](#)
 - [IDE_POSN](#), [295](#)
 - [NUM_OF_RX_FIFO](#), [295](#)
 - [PLSIZEToPayloadBytes](#), [295](#)
 - [RTR_POSN](#), [295](#)
 - [RX_FIFO_MSG_DATA](#), [296](#)
 - [SID_HIGH_MASK](#), [296](#)
 - [SID_LOW_WIDTH](#), [296](#)
- can1.h
 - [CAN1_Initialize](#), [317](#)
 - [CAN1_IsBusOff](#), [317](#)
 - [CAN1_IsRxErrorActive](#), [318](#)
 - [CAN1_IsRxErrorPassive](#), [319](#)
 - [CAN1_IsRxErrorWarning](#), [319](#)
 - [CAN1_IsTxErrorActive](#), [320](#)
- [CAN1_IsTxErrorPassive](#), [320](#)
- [CAN1_IsTxErrorWarning](#), [321](#)
- [CAN1_OperationModeGet](#), [322](#)
- [CAN1_OperationModeSet](#), [322](#)
- [CAN1_Receive](#), [323](#)
- [CAN1_ReceivedMessageCountGet](#), [324](#)
- [CAN1_ReceiveFrom](#), [324](#)
- [CAN1_RX_FIFO_CHANNELS](#), [316](#)
- [CAN1_SetBusErrorInterruptHandler](#), [325](#)
- [CAN1_SetBusWakeUpActivityInterruptHandler](#), [326](#)
- [CAN1_SetFIFO1NotEmptyHandler](#), [326](#)
- [CAN1_SetFIFO2nullHandler](#), [327](#)
- [CAN1_SetFIFO3nullHandler](#), [327](#)
- [CAN1_SetInvalidMessageInterruptHandler](#), [328](#)
- [CAN1_SetModeChangeInterruptHandler](#), [328](#)
- [CAN1_SetRxBufferOverflowInterruptHandler](#), [329](#)
- [CAN1_SetSystemErrorInterruptHandler](#), [329](#)
- [CAN1_SetTxAttemptInterruptHandler](#), [330](#)
- [CAN1_SetTXQNotFullHandler](#), [331](#)
- [CAN1_Sleep](#), [332](#)
- [CAN1_Transmit](#), [333](#)
- [CAN1_TransmitFIFOStatusGet](#), [334](#)
- [CAN1_TX_FIFO2](#), [315](#)
- [CAN1_TX_FIFO3](#), [315](#)
- [CAN1_TX_FIFO_CHANNELS](#), [316](#)
- [CAN1_TX_TXQ](#), [315](#)
- [FIFO1](#), [316](#)
- [FIFO2](#), [316](#)
- [FIFO3](#), [316](#)
- [TXQ](#), [316](#)
- [CAN1_Initialize](#)
 - [can1.c](#), [297](#)
 - [can1.h](#), [317](#)
- [CAN1_IsBusOff](#)
 - [can1.c](#), [297](#)
 - [can1.h](#), [317](#)
- [CAN1_IsRxErrorActive](#)
 - [can1.c](#), [298](#)
 - [can1.h](#), [318](#)
- [CAN1_IsRxErrorPassive](#)
 - [can1.c](#), [299](#)
 - [can1.h](#), [319](#)
- [CAN1_IsRxErrorWarning](#)
 - [can1.c](#), [300](#)
 - [can1.h](#), [319](#)
- [CAN1_IsTxErrorActive](#)
 - [can1.c](#), [300](#)
 - [can1.h](#), [320](#)
- [CAN1_IsTxErrorPassive](#)
 - [can1.c](#), [301](#)
 - [can1.h](#), [320](#)
- [CAN1_IsTxErrorWarning](#)
 - [can1.c](#), [302](#)
 - [can1.h](#), [321](#)
- [CAN1_OperationModeGet](#)
 - [can1.c](#), [302](#)
 - [can1.h](#), [322](#)

CAN1_OperationModeSet
 can1.c, [303](#)
 can1.h, [322](#)
 CAN1_Receive
 can1.c, [304](#)
 can1.h, [323](#)
 CAN1_ReceivedMessageCountGet
 can1.c, [304](#)
 can1.h, [324](#)
 CAN1_ReceiveFrom
 can1.c, [305](#)
 can1.h, [324](#)
 CAN1_RX_FIFO, [75](#)
 channel, [75](#)
 fifoHead, [75](#)
 CAN1_RX_FIFO_CHANNELS
 can1.h, [316](#)
 CAN1_RX_FIFO_ResetInfo
 can1.c, [305](#)
 CAN1_SetBusErrorInterruptHandler
 can1.c, [306](#)
 can1.h, [325](#)
 CAN1_SetBusWakeUpActivityInterruptHandler
 can1.c, [306](#)
 can1.h, [326](#)
 CAN1_SetFIFO1NotEmptyHandler
 can1.c, [307](#)
 can1.h, [326](#)
 CAN1_SetFIFO2nullHandler
 can1.h, [327](#)
 CAN1_SetFIFO3nullHandler
 can1.h, [327](#)
 CAN1_SetInvalidMessageInterruptHandler
 can1.c, [307](#)
 can1.h, [328](#)
 CAN1_SetModeChangeInterruptHandler
 can1.c, [308](#)
 can1.h, [328](#)
 CAN1_SetRxBufferOverFlowInterruptHandler
 can1.c, [308](#)
 can1.h, [329](#)
 CAN1_SetSystemErrorInterruptHandler
 can1.c, [309](#)
 can1.h, [329](#)
 CAN1_SetTxAttemptInterruptHandler
 can1.c, [310](#)
 can1.h, [330](#)
 CAN1_SetTXQNotFullHandler
 can1.c, [310](#)
 can1.h, [331](#)
 CAN1_Sleep
 can1.c, [311](#)
 can1.h, [332](#)
 CAN1_Transmit
 can1.c, [312](#)
 can1.h, [333](#)
 CAN1_TransmitFIFOStatusGet
 can1.c, [313](#)
 can1.h, [334](#)
 CAN1_TX_FIFO2
 can1.h, [315](#)
 CAN1_TX_FIFO3
 can1.h, [315](#)
 CAN1_TX_FIFO_CHANNELS
 can1.h, [316](#)
 CAN1_TX_TXQ
 can1.h, [315](#)
 Can1FIFO1NotEmptyHandler
 canfd.c, [155](#)
 canfd.h, [165](#)
 CAN_2_0_FORMAT
 can_types.h, [339](#)
 can_blob_get
 canfd.c, [156](#)
 canfd.h, [165](#)
 can_blob_set
 canfd.c, [156](#)
 canfd.h, [165](#)
 CAN_BRS_MODE
 can_types.h, [338](#)
 CAN_CONFIGURATION_MODE
 can_types.h, [341](#)
 CAN_DISABLE_MODE
 can_types.h, [341](#)
 CAN_DLC
 can_types.h, [338](#)
 CAN_ERROR_BUF
 canfd.h, [162](#)
 CAN_EXTERNAL_LOOPBACK_MODE
 can_types.h, [341](#)
 CAN_FD_FORMAT
 can_types.h, [339](#)
 can_fd_lcd_mirror
 canfd.c, [156](#)
 canfd.h, [165](#)
 eadog.h, [196](#)
 can_fd_tx
 canfd.c, [156](#)
 canfd.h, [165](#)
 CAN_FIFOREG, [76](#)
 CONH, [76](#)
 CONL, [76](#)
 CONT, [76](#)
 CONU, [76](#)
 STAH, [76](#)
 STAL, [76](#)
 STAT, [76](#)
 STAU, [76](#)
 UA, [77](#)
 CAN_FRAME_DATA
 can_types.h, [339](#)
 CAN_FRAME_EXT
 can_types.h, [339](#)
 CAN_FRAME_RTR
 can_types.h, [339](#)
 CAN_FRAME_STD

- can_types.h, 339
- can_frames_t, 77
 - rec_count, 77
 - tx_count, 77
- CAN_FULL_BUFFER
 - raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c, 258
- CAN_HIGH_BUF
 - canfd.h, 162
- CAN_INFO_BUF
 - canfd.h, 162
- CAN_INTERNAL_LOOPBACK_MODE
 - can_types.h, 341
- CAN_LISTEN_ONLY_MODE
 - can_types.h, 341
- CAN_LOW_BUF
 - canfd.h, 162
- can_mirror_print
 - canfd.c, 157
 - canfd.h, 166
- CAN_MIRRORS
 - canfd.h, 162
- CAN_MSG_COUNT
 - raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c, 258
- CAN_MSG_FIELD, 77
 - brs, 78
 - dlc, 78
 - formatType, 78
 - frameType, 78
 - idType, 79
 - msgfields, 79
- CAN_MSG_ID_PING
 - raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c, 258
- CAN_MSG_ID_PING_X
 - raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c, 258
- CAN_MSG_ID_PONG
 - raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c, 258
- CAN_MSG_LEN
 - raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c, 258
- CAN_MSG_OBJ, 79
 - data, 79
 - field, 79
 - msgId, 80
- CAN_MSG_OBJ_BRS_MODE
 - can_types.h, 338
- CAN_MSG_OBJ_FRAME_TYPE
 - can_types.h, 338
- CAN_MSG_OBJ_ID_TYPE
 - can_types.h, 339
- CAN_MSG_OBJ_TYPE
 - can_types.h, 339
- CAN_MSG_WAIT
 - raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c, 259
- can_newtime
 - batmon.h, 150
 - canfd.h, 168
 - main.c, 281
- CAN_NON_BRS_MODE
 - can_types.h, 338
- CAN_NORMAL_2_0_MODE
 - can_types.h, 341
- CAN_NORMAL_FD_MODE
 - can_types.h, 341
- CAN_OP_MODE_REQUEST_FAIL
 - can_types.h, 339
- CAN_OP_MODE_REQUEST_SUCCESS
 - can_types.h, 339
- CAN_OP_MODE_STATUS
 - can_types.h, 339
- CAN_OP_MODE_SYS_ERROR_OCCURED
 - can_types.h, 339
- CAN_OP_MODES
 - can_types.h, 339
- can_packets
 - canfd.c, 159
 - canfd.h, 168
- CAN_REC_BUFFERS
 - canfd.h, 162
- can_rec_count
 - canfd.c, 159
 - canfd.h, 168
- can_rec_count_t, 80
 - rec_count, 80
 - rec_flag, 80
- CAN_RESTRICTED_OPERATION_MODE
 - can_types.h, 341
- CAN_RX_FIFO_STATUS
 - canfd.h, 162
- CAN_RX_MSG_AVAILABLE
 - can1.c, 296
- CAN_RX_MSG_NOT_AVAILABLE
 - can1.c, 296
- CAN_RX_MSG_OVERFLOW
 - canfd.h, 163
- CAN_RX_TRIES
 - canfd.h, 163
- can_setup
 - canfd.c, 158
 - canfd.h, 167
- can_timer
 - canfd.h, 168
 - main.c, 281
- CAN_TM_TIME
 - raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c, 259
- CAN_TX_FIFO_AVAILABLE
 - can_types.h, 341
- CAN_TX_FIFO_FULL
 - can_types.h, 341

CAN_TX_FIFO_STATUS
 can_types.h, 341

CAN_TX_MSG_REQUEST_BRS_ERROR
 can_types.h, 341

CAN_TX_MSG_REQUEST_DLC_EXCEED_ERROR
 can_types.h, 341

CAN_TX_MSG_REQUEST_FIFO_FULL
 can_types.h, 341

CAN_TX_MSG_REQUEST_STATUS
 can_types.h, 341

CAN_TX_MSG_REQUEST_SUCCESS
 can_types.h, 341

can_types.h
 CAN_2_0_FORMAT, 339
 CAN_BRS_MODE, 338
 CAN_CONFIGURATION_MODE, 341
 CAN_DISABLE_MODE, 341
 CAN_DLC, 338
 CAN_EXTERNAL_LOOPBACK_MODE, 341
 CAN_FD_FORMAT, 339
 CAN_FRAME_DATA, 339
 CAN_FRAME_EXT, 339
 CAN_FRAME_RTR, 339
 CAN_FRAME_STD, 339
 CAN_INTERNAL_LOOPBACK_MODE, 341
 CAN_LISTEN_ONLY_MODE, 341
 CAN_MSG_OBJ_BRS_MODE, 338
 CAN_MSG_OBJ_FRAME_TYPE, 338
 CAN_MSG_OBJ_ID_TYPE, 339
 CAN_MSG_OBJ_TYPE, 339
 CAN_NON_BRS_MODE, 338
 CAN_NORMAL_2_0_MODE, 341
 CAN_NORMAL_FD_MODE, 341
 CAN_OP_MODE_REQUEST_FAIL, 339
 CAN_OP_MODE_REQUEST_SUCCESS, 339
 CAN_OP_MODE_STATUS, 339
 CAN_OP_MODE_SYS_ERROR_OCCURED, 339
 CAN_OP_MODES, 339
 CAN_RESTRICTED_OPERATION_MODE, 341
 CAN_TX_FIFO_AVAILABLE, 341
 CAN_TX_FIFO_FULL, 341
 CAN_TX_FIFO_STATUS, 341
 CAN_TX_MSG_REQUEST_BRS_ERROR, 341
 CAN_TX_MSG_REQUEST_DLC_EXCEED_ERROR, 341
 CAN_TX_MSG_REQUEST_FIFO_FULL, 341
 CAN_TX_MSG_REQUEST_STATUS, 341
 CAN_TX_MSG_REQUEST_SUCCESS, 341
 DLC_0, 338
 DLC_1, 338
 DLC_12, 338
 DLC_16, 338
 DLC_2, 338
 DLC_20, 338
 DLC_24, 338
 DLC_3, 338
 DLC_32, 338
 DLC_4, 338
 DLC_48, 338
 DLC_5, 338
 DLC_6, 338
 DLC_64, 338
 DLC_7, 338
 DLC_8, 338

canbus_name
 mxcmd.h, 672

canbus_online
 B_type, 68

canfd.c, 154
 Byte1, 159
 Byte2, 159
 Can1FIFO1NotEmptyHandler, 155
 can_blob_get, 156
 can_blob_set, 156
 can_fd_lcd_mirror, 156
 can_fd_tx, 156
 can_mirror_print, 157
 can_packets, 159
 can_rec_count, 159
 can_setup, 158
 get_blob_rx, 158
 get_blob_tx, 158
 msg, 159
 myVar, 160
 rxMsgData, 160
 structBytes, 160
 TXQNotFullHandler, 159
 Word, 160

canfd.h, 160
 Can1FIFO1NotEmptyHandler, 165
 can_blob_get, 165
 can_blob_set, 165
 CAN_ERROR_BUF, 162
 can_fd_lcd_mirror, 165
 can_fd_tx, 165
 CAN_HIGH_BUF, 162
 CAN_INFO_BUF, 162
 CAN_LOW_BUF, 162
 can_mirror_print, 166
 CAN_MIRRORS, 162
 can_newtime, 168
 can_packets, 168
 CAN_REC_BUFFERS, 162
 can_rec_count, 168
 CAN_RX_TRIES, 163
 can_setup, 167
 can_timer, 168
 CANFD_BYTES, 163
 CANFD_NBUF, 163
 DATA_DEBUG, 163
 EMON_CO, 163
 EMON_DA, 163
 EMON_DM, 163
 EMON_DS, 163
 EMON_ER, 163
 EMON_M, 163

- EMON_MR, [164](#)
- EMON_SL, [164](#)
- EMON_SU, [164](#)
- EMON_TM, [164](#)
- get_blob_rx, [167](#)
- get_blob_tx, [167](#)
- info_buffer, [168](#)
- LCD_BUF_SIZ, [164](#)
- log_buffer, [168](#)
- MIRROR_BUF, [164](#)
- msg, [169](#)
- NO_NODE_ID, [164](#)
- NODE_ID_SHIFT, [164](#)
- rxMsgData, [169](#)
- TXQNotFullHandler, [168](#)
- USE_FD, [164](#)
- CANFD_BYTES
 - canfd.h, [163](#)
- CANFD_NBUF
 - canfd.h, [163](#)
- canfilter.c
 - main, [153](#)
- canreceive.c
 - main, [153](#)
- cantransmit.c
 - main, [154](#)
- cc_buffer
 - modbus_master.c, [685](#)
- cc_buffer_tx
 - modbus_master.c, [685](#)
- cc_mode
 - EB_data, [82](#)
 - main.c, [281](#)
 - qconfig.h, [712](#)
- cc_stream_file
 - modbus_master.c, [686](#)
- ccmode
 - raw.githubusercontent.com_linux-can_can-utils_master/CHANGELOG, [265](#)
- CDELAY
 - modbus_master.h, [692](#)
- cert_verify_failed
 - mbedtls_sockets.h, [248](#)
- channel
 - CAN1_RX_FIFO, [75](#)
- channel_ANA0
 - ADC3: Analog-to-Digital Converter with Context, [45](#)
- channel_ANA4
 - ADC3: Analog-to-Digital Converter with Context, [45](#)
- channel_DAC1
 - ADC3: Analog-to-Digital Converter with Context, [45](#)
- channel_FVR_Buffer1
 - ADC3: Analog-to-Digital Converter with Context, [45](#)
- channel_FVR_Buffer2
 - ADC3: Analog-to-Digital Converter with Context, [45](#)
- channel_Temp
 - ADC3: Analog-to-Digital Converter with Context, [45](#)
- channel_VSS
 - ADC3: Analog-to-Digital Converter with Context, [45](#)
- CHARGER_ANS
 - pin_manager.h, [405](#)
- CHARGER_GetValue
 - pin_manager.h, [405](#)
- CHARGER_LAT
 - pin_manager.h, [405](#)
- CHARGER_OD
 - pin_manager.h, [405](#)
- CHARGER_PORT
 - pin_manager.h, [405](#)
- CHARGER_RELAY_OFF
 - dio.h, [173](#)
- CHARGER_RELAY_ON
 - dio.h, [173](#)
- CHARGER_ResetPullup
 - pin_manager.h, [405](#)
- CHARGER_SetAnalogMode
 - pin_manager.h, [405](#)
- CHARGER_SetDigitalInput
 - pin_manager.h, [405](#)
- CHARGER_SetDigitalMode
 - pin_manager.h, [406](#)
- CHARGER_SetDigitalOutput
 - pin_manager.h, [406](#)
- CHARGER_SetHigh
 - pin_manager.h, [406](#)
- CHARGER_SetLow
 - pin_manager.h, [406](#)
- CHARGER_SetOpenDrain
 - pin_manager.h, [406](#)
- CHARGER_SetPullup
 - pin_manager.h, [406](#)
- CHARGER_SetPushPull
 - pin_manager.h, [406](#)
- CHARGER_Toggle
 - pin_manager.h, [407](#)
- CHARGER_TRIS
 - pin_manager.h, [407](#)
- CHARGER_WPU
 - pin_manager.h, [407](#)
- check_lcd_dim
 - eadog.c, [179](#)
 - eadog.h, [197](#)
- checkmark
 - EB_data, [82](#)
- CHK_DAY_TIME
 - batmon.h, [143](#)
- clc7.c
 - CLC7_Initialize, [344](#)
 - CLC7_OutputStatusGet, [344](#)
- clc7.h
 - CLC7_Initialize, [346](#)
 - CLC7_OutputStatusGet, [346](#)
- CLC7_Initialize
 - clc7.c, [344](#)
 - clc7.h, [346](#)
- CLC7_OutputStatusGet

- clc7.c, [344](#)
- clc7.h, [346](#)
- clc8.c
 - CLC8_Initialize, [348](#)
 - CLC8_OutputStatusGet, [349](#)
- clc8.h
 - CLC8_Initialize, [350](#)
 - CLC8_OutputStatusGet, [351](#)
- CLC8_Initialize
 - clc8.c, [348](#)
 - clc8.h, [350](#)
- CLC8_OutputStatusGet
 - clc8.c, [349](#)
 - clc8.h, [351](#)
- CLEAR
 - modbus_master.h, [697](#)
- clear_2hz
 - modbus_master.c, [679](#)
 - modbus_master.h, [698](#)
- clear_500ahz
 - modbus_master.c, [679](#)
 - modbus_master.h, [698](#)
- clear_500hz
 - modbus_master.c, [679](#)
 - modbus_master.h, [699](#)
- clear_lcd_done
 - eadog.c, [179](#)
 - eadog.h, [197](#)
- client
 - mqtt_pub.c, [232](#)
 - raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c, [265](#)
- client_daemon
 - mqtt_pub.c, [232](#)
- client_id
 - mqtt_pub.c, [232](#)
- client_refresher
 - main.c, [269](#)
 - mqtt_pub.c, [229](#)
 - mqtt_pub.h, [234](#)
- CLIENTID
 - raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c, [259](#)
- clock_2hz
 - M_time_data, [93](#)
- clock_500ahz
 - M_time_data, [93](#)
- clock_500hz
 - M_time_data, [93](#)
- close_socket
 - bearssl_sockets.h, [244](#)
- cmd_battery
 - mxcmd.h, [672](#)
- cmd_batteryv
 - mxcmd.h, [673](#)
- CMD_CRC_LEN
 - mxcmd.h, [667](#)
- cmd_date
 - mxcmd.h, [673](#)
- cmd_fwrev
 - mxcmd.h, [673](#)
- cmd_fwrevb
 - mxcmd.h, [673](#)
- cmd_fwrevc
 - mxcmd.h, [673](#)
- cmd_id
 - mxcmd.h, [673](#)
- CMD_LEN
 - main.c, [272](#)
- cmd_misc
 - mxcmd.h, [673](#)
- cmd_mx_log
 - mxcmd.h, [673](#)
- cmd_mx_status
 - mxcmd.h, [673](#)
- cmd_panelv
 - mxcmd.h, [673](#)
- cmd_status
 - mxcmd.h, [674](#)
- cmd_time
 - mxcmd.h, [674](#)
- cmd_type
 - modbus_master.h, [695](#), [696](#)
- cmd_watts
 - mxcmd.h, [674](#)
- comm_type
 - modbus_master.h, [695](#), [697](#)
- compute_bm_data
 - batmon.h, [144](#)
- con0
 - spi1_configuration_t, [124](#)
- con1
 - spi1_configuration_t, [124](#)
- con2
 - spi1_configuration_t, [124](#)
- config
 - M_data, [91](#)
 - spi_link_type, [125](#)
- config_data
 - C_data, [73](#)
- CONH
 - CAN_FIFOREG, [76](#)
- CONL
 - CAN_FIFOREG, [76](#)
- conn_opts
 - raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c, [265](#)
- connack
 - mqtt_response, [107](#)
- connect_flags
 - mqtt_pub.c, [232](#)
- connlost
 - raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c, [262](#)
- CONT

- CAN_FIFOREG, 76
- Control Packet Deserialization, 27
 - MQTT_CONNACK_ACCEPTED, 29
 - MQTT_CONNACK_REFUSED_BAD_USER_NAME_OR_PASSWORD, 29
 - MQTT_CONNACK_REFUSED_IDENTIFIER_REJECTED, 29
 - MQTT_CONNACK_REFUSED_NOT_AUTHORIZED, 29
 - MQTT_CONNACK_REFUSED_PROTOCOL_VERSION, 29
 - MQTT_CONNACK_REFUSED_SERVER_UNAVAILABLE, 29
 - MQTT_CONTROL_CONNACK, 30
 - MQTT_CONTROL_CONNECT, 30
 - MQTT_CONTROL_DISCONNECT, 30
 - MQTT_CONTROL_PINGREQ, 30
 - MQTT_CONTROL_PINGRESP, 30
 - MQTT_CONTROL_PUBACK, 30
 - MQTT_CONTROL_PUBCOMP, 30
 - MQTT_CONTROL_PUBLISH, 30
 - MQTT_CONTROL_PUBREC, 30
 - MQTT_CONTROL_PUBREL, 30
 - MQTT_CONTROL_SUBACK, 30
 - MQTT_CONTROL_SUBSCRIBE, 30
 - MQTT_CONTROL_UNSUBACK, 30
 - MQTT_CONTROL_UNSUBSCRIBE, 30
 - MQTT_SUBACK_FAILURE, 30
 - MQTT_SUBACK_SUCCESS_MAX_QOS_0, 30
 - MQTT_SUBACK_SUCCESS_MAX_QOS_1, 30
 - MQTT_SUBACK_SUCCESS_MAX_QOS_2, 30
 - mqtt_unpack_connack_response, 30
 - mqtt_unpack_fixed_header, 31
 - mqtt_unpack_publish_response, 31
 - mqtt_unpack_pubxxx_response, 32
 - mqtt_unpack_response, 32
 - MQTTConnackReturnCode, 29
 - MQTTControlPacketType, 29
 - MQTTSubackReturnCodes, 30
- Control Packet Serialization, 20
 - MQTT_CONNECT_CLEAN_SESSION, 22
 - MQTT_CONNECT_PASSWORD, 22
 - MQTT_CONNECT_RESERVED, 22
 - MQTT_CONNECT_USER_NAME, 22
 - MQTT_CONNECT_WILL_FLAG, 22
 - MQTT_CONNECT_WILL_QOS_0, 22
 - MQTT_CONNECT_WILL_QOS_1, 22
 - MQTT_CONNECT_WILL_QOS_2, 22
 - MQTT_CONNECT_WILL_RETAIN, 22
 - mqtt_pack_connection_request, 23
 - mqtt_pack_disconnect, 23
 - mqtt_pack_fixed_header, 24
 - mqtt_pack_ping_request, 24
 - mqtt_pack_publish_request, 25
 - mqtt_pack_pubxxx_request, 25
 - mqtt_pack_subscribe_request, 26
 - mqtt_pack_unsubscribe_request, 27
 - MQTT_PROTOCOL_LEVEL, 21
 - MQTT_PUBLISH_DUP, 22
 - MQTT_PUBLISH_QOS_0, 22
 - MQTT_PUBLISH_QOS_1, 22
 - MQTT_PUBLISH_QOS_2, 22
 - MQTT_PUBLISH_QOS_MASK, 22
 - MQTT_PUBLISH_RETAIN, 22
 - MQTT_SUBSCRIBE_REQUEST_MAX_NUM_TOPICS, 21
 - MQTT_UNSUBSCRIBE_REQUEST_MAX_NUM_TOPICS, 21
 - MQTTConnectFlags, 22
 - MQTTPublishFlags, 22
 - control_flags
 - mqtt_fixed_header, 101
 - control_type
 - mqtt_fixed_header, 101
 - mqtt_queued_message, 104
- CONU
 - CAN_FIFOREG, 76
- count
 - ringBufS_t, 123
- crc
 - EB_data, 82
- crc16
 - modbus_master.c, 679
 - modbus_master.h, 699
- crc_calc
 - M_data, 91
- crc_data
 - M_data, 91
- crc_error
 - M_data, 91
- CS_ANS
 - pin_manager.h, 407
- CS_GetValue
 - pin_manager.h, 407
- CS_LAT
 - pin_manager.h, 407
- CS_OD
 - pin_manager.h, 407
- CS_PORT
 - pin_manager.h, 407
- CS_ResetPullup
 - pin_manager.h, 407
- CS_SetAnalogMode
 - pin_manager.h, 408
- CS_SetDigitalInput
 - pin_manager.h, 408
- CS_SetDigitalMode
 - pin_manager.h, 408
- CS_SetDigitalOutput
 - pin_manager.h, 408
- CS_SetHigh
 - pin_manager.h, 408
- CS_SetLow
 - pin_manager.h, 408
- CS_SetOpenDrain
 - pin_manager.h, 408

- CS_SetPullup
 - pin_manager.h, [409](#)
- CS_SetPushPull
 - pin_manager.h, [409](#)
- CS_Toggle
 - pin_manager.h, [409](#)
- CS_TRIS
 - pin_manager.h, [409](#)
- CS_WPU
 - pin_manager.h, [409](#)
- cstate
 - C_data, [73](#)
- ctr_drbg
 - MBEDTLS_CONTEXT, [94](#)
- curr
 - mqtt_client, [96](#)
 - mqtt_message_queue, [103](#)
- curr_sz
 - mqtt_client, [96](#)
 - mqtt_message_queue, [103](#)
- current_timestamp
 - raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c, [262](#)
- D_SW
 - dio.h, [174](#)
- D_SW_A
 - dio.h, [174](#)
- D_SW_A_PIN
 - dio.h, [173](#)
- D_SW_COUNT
 - dio.h, [174](#)
- D_SW_L
 - dio.h, [174](#)
- D_SW_L_PIN
 - dio.h, [173](#)
- D_SW_M
 - dio.h, [174](#)
- D_SW_M_PIN
 - dio.h, [173](#)
- data
 - CAN_MSG_OBJ, [79](#)
- data_buffer
 - raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c, [265](#)
- data_count
 - C_data, [73](#)
- DATA_DEBUG
 - canfd.h, [163](#)
- DATA_MQTT_SOLAR
 - mqtt_pub.h, [234](#)
- data_ok
 - C_data, [74](#)
- data_prev
 - C_data, [74](#)
- DATAEE_ReadByte
 - batmon.c, [134](#)
 - batmon.h, [144](#)
- DATAEE_WriteByte
 - batmon.c, [134](#)
 - batmon.h, [145](#)
- date
 - EB_data, [82](#)
- day
 - mx_logpage_t, [121](#)
- day_check
 - B_type, [68](#)
- DAY_RELAY_OFF
 - dio.h, [174](#)
- DAY_RELAY_ON
 - dio.h, [174](#)
- DAY_SEC
 - raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c, [259](#)
- day_time
 - raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c, [266](#)
- DB0_ANS
 - pin_manager.h, [409](#)
- DB0_GetValue
 - pin_manager.h, [409](#)
- DB0_LAT
 - pin_manager.h, [409](#)
- DB0_OD
 - pin_manager.h, [410](#)
- DB0_PORT
 - pin_manager.h, [410](#)
- DB0_ResetPullup
 - pin_manager.h, [410](#)
- DB0_SetAnalogMode
 - pin_manager.h, [410](#)
- DB0_SetDigitalInput
 - pin_manager.h, [410](#)
- DB0_SetDigitalMode
 - pin_manager.h, [410](#)
- DB0_SetDigitalOutput
 - pin_manager.h, [410](#)
- DB0_SetHigh
 - pin_manager.h, [410](#)
- DB0_SetLow
 - pin_manager.h, [411](#)
- DB0_SetOpenDrain
 - pin_manager.h, [411](#)
- DB0_SetPullup
 - pin_manager.h, [411](#)
- DB0_SetPushPull
 - pin_manager.h, [411](#)
- DB0_Toggle
 - pin_manager.h, [411](#)
- DB0_TRIS
 - pin_manager.h, [411](#)
- DB0_WPU
 - pin_manager.h, [411](#)
- DB1_ANS
 - pin_manager.h, [412](#)
- DB1_GetValue
 - pin_manager.h, [412](#)

- DB1_LAT
 - pin_manager.h, [412](#)
- DB1_OD
 - pin_manager.h, [412](#)
- DB1_PORT
 - pin_manager.h, [412](#)
- DB1_ResetPullup
 - pin_manager.h, [412](#)
- DB1_SetAnalogMode
 - pin_manager.h, [412](#)
- DB1_SetDigitalInput
 - pin_manager.h, [412](#)
- DB1_SetDigitalMode
 - pin_manager.h, [413](#)
- DB1_SetDigitalOutput
 - pin_manager.h, [413](#)
- DB1_SetHigh
 - pin_manager.h, [413](#)
- DB1_SetLow
 - pin_manager.h, [413](#)
- DB1_SetOpenDrain
 - pin_manager.h, [413](#)
- DB1_SetPullup
 - pin_manager.h, [413](#)
- DB1_SetPushPull
 - pin_manager.h, [413](#)
- DB1_Toggle
 - pin_manager.h, [414](#)
- DB1_TRIS
 - pin_manager.h, [414](#)
- DB1_WPU
 - pin_manager.h, [414](#)
- DEBUG_R
 - modbus_master.h, [692](#)
- debounce_time
 - dio.h, [174](#)
- decoded
 - mqtt_response, [107](#)
- delay
 - spi_link_type, [125](#)
- delay_ms
 - timers.c, [722](#)
 - timers.h, [726](#)
- delay_us
 - timers.c, [722](#)
- delivered
 - raw.githubusercontent.com_linux-can_can-utils_master_canutils, [262](#)
- deliveredtoken
 - raw.githubusercontent.com_linux-can_can-utils_master_canutils, [266](#)
- DERE_ANS
 - pin_manager.h, [414](#)
- DERE_GetValue
 - pin_manager.h, [414](#)
- DERE_LAT
 - pin_manager.h, [414](#)
- DERE_OD
 - pin_manager.h, [414](#)
- DERE_PORT
 - pin_manager.h, [414](#)
- DERE_ResetPullup
 - pin_manager.h, [414](#)
- DERE_SetAnalogMode
 - pin_manager.h, [415](#)
- DERE_SetDigitalInput
 - pin_manager.h, [415](#)
- DERE_SetDigitalMode
 - pin_manager.h, [415](#)
- DERE_SetDigitalOutput
 - pin_manager.h, [415](#)
- DERE_SetHigh
 - pin_manager.h, [415](#)
- DERE_SetLow
 - pin_manager.h, [415](#)
- DERE_SetOpenDrain
 - pin_manager.h, [415](#)
- DERE_SetPullup
 - pin_manager.h, [416](#)
- DERE_SetPushPull
 - pin_manager.h, [416](#)
- DERE_Toggle
 - pin_manager.h, [416](#)
- DERE_TRIS
 - pin_manager.h, [416](#)
- DERE_WPU
 - pin_manager.h, [416](#)
- device_config.h
 - _XTAL_FREQ, [353](#)
- device_id_address_t
 - batmon.h, [144](#)
- device_id_data_t
 - batmon.h, [144](#)
- DeviceID_Read
 - batmon.c, [135](#)
 - batmon.h, [145](#)
- DIGITAL
 - pin_manager.h, [416](#)
- DIM_DELAY
 - eadog.h, [191](#)
- dim_delay
 - B_type, [69](#)
- dio.c, [170](#)
 - all_relays_off, [171](#)
 - button_press_check, [171](#)
 - init_all_switch, [171](#)
- dio.h, [172](#)
 - all_relays_off, [175](#)
 - button_press_check, [175](#)
- CHARGER_RELAY_OFF, [173](#)
- CHARGER_RELAY_ON, [173](#)
- D_SW, [174](#)
- D_SW_A, [174](#)
- D_SW_A_PIN, [173](#)
- D_SW_COUNT, [174](#)
- D_SW_L, [174](#)

- D_SW_L_PIN, 173
- D_SW_M, 174
- D_SW_M_PIN, 173
- DAY_RELAY_OFF, 174
- DAY_RELAY_ON, 174
- debounce_time, 174
- init_all_switch, 175
- NIGHT_RELAY_OFF, 174
- NIGHT_RELAY_ON, 174
- sw_contact_types, 176
- SW_NC, 174
- SW_NO, 174
- display_dim
 - B_type, 69
- display_on
 - B_type, 69
- display_update
 - B_type, 69
- dlc
 - CAN_MSG_FIELD, 78
- DLC_0
 - can_types.h, 338
- DLC_1
 - can_types.h, 338
- DLC_12
 - can_types.h, 338
- DLC_16
 - can_types.h, 338
- DLC_2
 - can_types.h, 338
- DLC_20
 - can_types.h, 338
- DLC_24
 - can_types.h, 338
- DLC_3
 - can_types.h, 338
- DLC_32
 - can_types.h, 338
- DLC_4
 - can_types.h, 338
- DLC_48
 - can_types.h, 338
- DLC_5
 - can_types.h, 338
- DLC_6
 - can_types.h, 338
- DLC_64
 - can_types.h, 338
- DLC_7
 - can_types.h, 338
- DLC_8
 - can_types.h, 338
- DLCToPayloadBytes
 - can1.c, 294
- DLED1_ANS
 - pin_manager.h, 416
- DLED1_GetValue
 - pin_manager.h, 416
- DLED1_LAT
 - pin_manager.h, 417
- DLED1_OD
 - pin_manager.h, 417
- DLED1_PORT
 - pin_manager.h, 417
- DLED1_ResetPullup
 - pin_manager.h, 417
- DLED1_SetAnalogMode
 - pin_manager.h, 417
- DLED1_SetDigitalInput
 - pin_manager.h, 417
- DLED1_SetDigitalMode
 - pin_manager.h, 417
- DLED1_SetDigitalOutput
 - pin_manager.h, 417
- DLED1_SetHigh
 - pin_manager.h, 418
- DLED1_SetLow
 - pin_manager.h, 418
- DLED1_SetOpenDrain
 - pin_manager.h, 418
- DLED1_SetPullup
 - pin_manager.h, 418
- DLED1_SetPushPull
 - pin_manager.h, 418
- DLED1_Toggle
 - pin_manager.h, 418
- DLED1_TRIS
 - pin_manager.h, 418
- DLED1_WPU
 - pin_manager.h, 419
- DLED_ANS
 - pin_manager.h, 419
- DLED_GetValue
 - pin_manager.h, 419
- DLED_LAT
 - pin_manager.h, 419
- DLED_OD
 - pin_manager.h, 419
- DLED_PORT
 - pin_manager.h, 419
- DLED_ResetPullup
 - pin_manager.h, 419
- DLED_SetAnalogMode
 - pin_manager.h, 419
- DLED_SetDigitalInput
 - pin_manager.h, 419
- DLED_SetDigitalMode
 - pin_manager.h, 420
- DLED_SetDigitalOutput
 - pin_manager.h, 420
- DLED_SetHigh
 - pin_manager.h, 420
- DLED_SetLow
 - pin_manager.h, 420
- DLED_SetOpenDrain
 - pin_manager.h, 420

DLED_SetPullup
 pin_manager.h, [420](#)
 DLED_SetPushPull
 pin_manager.h, [420](#)
 DLED_Toggle
 pin_manager.h, [421](#)
 DLED_TRIS
 pin_manager.h, [421](#)
 DLED_WPU
 pin_manager.h, [421](#)
 dma1.c
 __interrupt, [355](#)
 DMA1_AI_IRQHandler, [363](#)
 DMA1_DefaultInterruptHandler, [355](#)
 DMA1_GetDestinationPointer, [356](#)
 DMA1_GetSourcePointer, [356](#)
 DMA1_Initialize, [356](#)
 DMA1_ORI_IRQHandler, [363](#)
 DMA1_SCNTI_IRQHandler, [363](#)
 DMA1_SelectSourceRegion, [357](#)
 DMA1_SetAbortTrigger, [357](#)
 DMA1_SetAllInterruptHandler, [358](#)
 DMA1_SetDestinationAddress, [358](#)
 DMA1_SetDestinationSize, [359](#)
 DMA1_SetDMAPriority, [359](#)
 DMA1_SetORI_IRQHandler, [360](#)
 DMA1_SetSCNTI_IRQHandler, [360](#)
 DMA1_SetSourceAddress, [361](#)
 DMA1_SetSourceSize, [361](#)
 DMA1_SetStartTrigger, [361](#)
 DMA1_StartTransfer, [362](#)
 DMA1_StartTransferWithTrigger, [362](#)
 DMA1_StopTransfer, [362](#)
 dma1.h
 DMA1_DefaultInterruptHandler, [365](#)
 DMA1_GetDestinationPointer, [365](#)
 DMA1_GetSourcePointer, [366](#)
 DMA1_Initialize, [366](#)
 DMA1_SelectSourceRegion, [367](#)
 DMA1_SetAbortTrigger, [367](#)
 DMA1_SetAllInterruptHandler, [368](#)
 DMA1_SetDestinationAddress, [368](#)
 DMA1_SetDestinationSize, [369](#)
 DMA1_SetDMAPriority, [369](#)
 DMA1_SetORI_IRQHandler, [370](#)
 DMA1_SetSCNTI_IRQHandler, [370](#)
 DMA1_SetSourceAddress, [371](#)
 DMA1_SetSourceSize, [371](#)
 DMA1_SetStartTrigger, [371](#)
 DMA1_StartTransfer, [372](#)
 DMA1_StartTransferWithTrigger, [372](#)
 DMA1_StopTransfer, [372](#)
 lcd_dma_buf, [373](#)
 DMA1_AI_IRQHandler
 dma1.c, [363](#)
 DMA1_DefaultInterruptHandler
 dma1.c, [355](#)
 dma1.h, [365](#)
 DMA1_GetDestinationPointer
 dma1.c, [356](#)
 dma1.h, [365](#)
 DMA1_GetSourcePointer
 dma1.c, [356](#)
 dma1.h, [366](#)
 DMA1_Initialize
 dma1.c, [356](#)
 dma1.h, [366](#)
 DMA1_ORI_IRQHandler
 dma1.c, [363](#)
 DMA1_SCNTI_IRQHandler
 dma1.c, [363](#)
 DMA1_SelectSourceRegion
 dma1.c, [357](#)
 dma1.h, [367](#)
 DMA1_SetAbortTrigger
 dma1.c, [357](#)
 dma1.h, [367](#)
 DMA1_SetAllInterruptHandler
 dma1.c, [358](#)
 dma1.h, [368](#)
 DMA1_SetDestinationAddress
 dma1.c, [358](#)
 dma1.h, [368](#)
 DMA1_SetDestinationSize
 dma1.c, [359](#)
 dma1.h, [369](#)
 DMA1_SetDMAPriority
 dma1.c, [359](#)
 dma1.h, [369](#)
 DMA1_SetORI_IRQHandler
 dma1.c, [360](#)
 dma1.h, [370](#)
 DMA1_SetSCNTI_IRQHandler
 dma1.c, [360](#)
 dma1.h, [370](#)
 DMA1_SetSourceAddress
 dma1.c, [361](#)
 dma1.h, [371](#)
 DMA1_SetSourceSize
 dma1.c, [361](#)
 dma1.h, [371](#)
 DMA1_SetStartTrigger
 dma1.c, [361](#)
 dma1.h, [371](#)
 DMA1_StartTransfer
 dma1.c, [362](#)
 dma1.h, [372](#)
 DMA1_StartTransferWithTrigger
 dma1.c, [362](#)
 dma1.h, [372](#)
 DMA1_StopTransfer
 dma1.c, [362](#)
 dma1.h, [372](#)
 dmt_sosc_flag
 V_type, [130](#)
 DONE_DELAY

- eadog.c, 178
- DTG_LEN
 - mxcmd.h, 668
- dummy
 - mqtt_response_pingresp, 110
- dup_flag
 - mqtt_response_publish, 113
- DUPL_DELAY
 - modbus_master.h, 692
- E_DAYS
 - pge.h, 242
 - raw.githubusercontent.com_linux-can_can-utils_master_confdesktop, 259
- E_MONTH
 - pge.h, 242
 - raw.githubusercontent.com_linux-can_can-utils_master_confdesktop, 259
- E_PER_DAY
 - pge.h, 242
 - raw.githubusercontent.com_linux-can_can-utils_master_confdesktop, 259
- E_PER_HOUR
 - pge.h, 242
 - raw.githubusercontent.com_linux-can_can-utils_master_confdesktop, 259
- E_SAVE
 - main.c, 272
- E_TRACE
 - eadog.c, 178
- E_UPDATE
 - main.c, 272
- eadog.c, 177
 - check_lcd_dim, 179
 - clear_lcd_done, 179
 - DONE_DELAY, 178
 - E_TRACE, 178
 - eaDogM_ClearRow, 179
 - eaDogM_Scroll_String, 179
 - eaDogM_Scroll_Task, 180
 - eaDogM_SetPos, 180
 - eaDogM_WriteByteToCGRAM, 181
 - eaDogM_WriteChr, 181
 - eaDogM_WriteCommand, 181
 - eaDogM_WriteIntAtPos, 181
 - eaDogM_WriteString, 182
 - eaDogM_WriteStringAtPos, 182
 - init_display, 183
 - no_dma_set_lcd, 184
 - send_lcd_cmd_dma, 184
 - send_lcd_data_dma, 185
 - send_lcd_pos_dma, 186
 - set_lcd_dim, 186
 - spi_link, 189
 - spi_rec_done, 187
 - start_lcd, 187
 - wait_lcd_check, 188
 - wait_lcd_done, 188
 - wait_lcd_set, 188
- eadog.h, 189
 - can_fd_lcd_mirror, 196
 - check_lcd_dim, 197
 - clear_lcd_done, 197
 - DIM_DELAY, 191
 - eaDogM_ClearRow, 198
 - eaDogM_Cls, 191
 - EADOGM_CMD_CGRAM_ADDR, 192
 - EADOGM_CMD_CLR, 192
 - EADOGM_CMD_CURSOR_OFF, 192
 - EADOGM_CMD_CURSOR_ON, 192
 - EADOGM_CMD_DDRAM_ADDR, 192
 - EADOGM_CMD_DISPLAY_OFF, 192
 - EADOGM_CMD_DISPLAY_ON, 192
 - EADOGM_CMD_SELECT_R0, 192
 - EADOGM_CMD_SELECT_R1, 192
 - EADOGM_CMD_SET_TABLE2, 192
 - EADOGM_COLSPAN, 193
 - eaDogM_CursorOff, 193
 - eaDogM_CursorOn, 193
 - eaDogM_DisplayOff, 193
 - eaDogM_DisplayOn, 193
 - eaDogM_Scroll_String, 198
 - eaDogM_Scroll_Task, 198
 - eaDogM_SetPos, 199
 - eaDogM_WriteByteToCGRAM, 199
 - eaDogM_WriteChr, 199
 - eaDogM_WriteCommand, 199
 - eaDogM_WriteIntAtPos, 200
 - eaDogM_WriteString, 200
 - eaDogM_WriteStringAtPos, 201
 - init_display, 202
 - LCD0, 193
 - LCD1, 193
 - LCD2, 193
 - LCD3, 194
 - LCD_CLEAR_HOME, 194
 - LCD_CMD_BRI, 194
 - LCD_CMD_CLR, 194
 - LCD_CMD_CONT, 194
 - LCD_CMD_HOME, 194
 - LCD_CMD_MASK, 194
 - LCD_CMD_OFF, 194
 - LCD_CMD_ON, 194
 - LCD_CMD_SET, 194
 - LCD_PWR_DELAY, 195
 - LSB, 195
 - max_strlen, 195
 - NHD_BL_HIGH, 195
 - NHD_BL_LOW, 195
 - NHD_BL_MED, 195
 - NHD_BL_OFF, 195
 - NHD_CMD, 195
 - NHD_CONT, 195
 - NHD_L_DELAY, 195
 - NHD_POS, 196
 - NHD_S_DELAY, 196
 - NHD_T_DELAY, 196

- no_dma_set_lcd, 202
- NSB, 196
- send_lcd_cmd_dma, 203
- send_lcd_data_dma, 204
- send_lcd_pos_dma, 204
- set_lcd_dim, 205
- spi_rec_done, 206
- start_lcd, 206
- USE_LCD_DMA, 196
- USEMCC_SPI, 196
- wait_lcd_check, 206
- wait_lcd_done, 206
- wait_lcd_set, 207
- wtdelay, 207
- eaDogM_ClearRow
 - eadog.c, 179
 - eadog.h, 198
- eaDogM_Cls
 - eadog.h, 191
- EADOGM_CMD_CGRAM_ADDR
 - eadog.h, 192
- EADOGM_CMD_CLR
 - eadog.h, 192
- EADOGM_CMD_CURSOR_OFF
 - eadog.h, 192
- EADOGM_CMD_CURSOR_ON
 - eadog.h, 192
- EADOGM_CMD_DDRAM_ADDR
 - eadog.h, 192
- EADOGM_CMD_DISPLAY_OFF
 - eadog.h, 192
- EADOGM_CMD_DISPLAY_ON
 - eadog.h, 192
- EADOGM_CMD_SELECT_R0
 - eadog.h, 192
- EADOGM_CMD_SELECT_R1
 - eadog.h, 192
- EADOGM_CMD_SET_TABLE2
 - eadog.h, 192
- EADOGM_COLSPAN
 - eadog.h, 193
- eaDogM_CursorOff
 - eadog.h, 193
- eaDogM_CursorOn
 - eadog.h, 193
- eaDogM_DisplayOff
 - eadog.h, 193
- eaDogM_DisplayOn
 - eadog.h, 193
- eaDogM_Scroll_String
 - eadog.c, 179
 - eadog.h, 198
- eaDogM_Scroll_Task
 - eadog.c, 180
 - eadog.h, 198
- eaDogM_SetPos
 - eadog.c, 180
 - eadog.h, 199
- eaDogM_WriteByteToCGRAM
 - eadog.c, 181
 - eadog.h, 199
- eaDogM_WriteChr
 - eadog.c, 181
 - eadog.h, 199
- eaDogM_WriteCommand
 - eadog.c, 181
 - eadog.h, 199
- eaDogM_WriteIntAtPos
 - eadog.c, 181
 - eadog.h, 200
- eaDogM_WriteString
 - eadog.c, 182
 - eadog.h, 200
- eaDogM_WriteStringAtPos
 - eadog.c, 182
 - eadog.h, 201
- EB_data, 80
 - alt_display, 81
 - bat_amp_whole, 81
 - bat_cycles, 81
 - bat_energy, 81
 - bat_mode, 81
 - bat_time, 81
 - batmon.h, 144
 - cc_mode, 82
 - checkmark, 82
 - crc, 82
 - date, 82
 - ENac, 82
 - ENva, 82
 - ENvar, 82
 - ENw, 82
 - fm80_time, 82
 - FMa, 82
 - FMbv, 83
 - FMpv, 83
 - FMw, 83
 - loaded, 83
 - q84_sequence, 83
 - time, 83
 - version, 83
 - volt_whole, 83
- EBD
 - batmon.c, 138
 - batmon.h, 150
- EBD_update
 - batmon.c, 138
 - batmon.h, 150
- EID_HIGH_MASK
 - can1.c, 294
- EID_HIGH_WIDTH
 - can1.c, 294
- EID_LOW_MASK
 - can1.c, 295
- EID_LOW_POSN
 - can1.c, 295

- EID_LOW_WIDTH
 - can1.c, [295](#)
- EID_MID_WIDTH
 - can1.c, [295](#)
- em
 - modbus_master.c, [686](#)
 - modbus_master.h, [705](#)
- em_config
 - modbus_master.c, [686](#)
- EM_data1, [84](#)
 - al1, [84](#)
 - al2, [84](#)
 - al3, [84](#)
 - hz, [84](#)
 - modbus_master.h, [696](#)
 - pfl1, [85](#)
 - pfl2, [85](#)
 - pfl3, [85](#)
 - pfsys, [85](#)
 - phaseseq, [85](#)
 - val1, [85](#)
 - val2, [85](#)
 - val3, [85](#)
 - varl1, [85](#)
 - varl2, [85](#)
 - varl3, [86](#)
 - varsys, [86](#)
 - vasys, [86](#)
 - vl1l2, [86](#)
 - vl1n, [86](#)
 - vl2l3, [86](#)
 - vl2n, [86](#)
 - vl3l1, [86](#)
 - vl3n, [86](#)
 - vllsys, [86](#)
 - vlnsys, [87](#)
 - wl1, [87](#)
 - wl2, [87](#)
 - wl3, [87](#)
 - wsys, [87](#)
- em_data1
 - modbus_master.c, [686](#)
- EM_data2, [87](#)
 - hz, [88](#)
 - kvahp, [88](#)
 - kvaht, [88](#)
 - kvarhnp, [88](#)
 - kvarhnt, [88](#)
 - kvarhpp, [88](#)
 - kvarhpt, [88](#)
 - kwhnp, [88](#)
 - kwhnt, [88](#)
 - kwhpl1, [88](#)
 - kwhpl2, [88](#)
 - kwhpl3, [89](#)
 - kwhpp, [89](#)
 - kwhpt, [89](#)
 - modbus_master.h, [696](#)
 - rhlc, [89](#)
 - rhm, [89](#)
 - rhmk, [89](#)
 - rhmkp, [89](#)
 - rhmp, [89](#)
- em_data2
 - modbus_master.c, [686](#)
- EM_DATA_LEN1
 - modbus_master.h, [692](#)
- EM_DATA_LEN2
 - modbus_master.h, [692](#)
- em_id
 - modbus_master.c, [686](#)
- em_light
 - modbus_master.c, [686](#)
- em_passwd
 - modbus_master.c, [686](#)
- EM_serial, [90](#)
 - modbus_master.h, [696](#)
 - serial, [90](#)
 - year, [90](#)
- em_serial
 - modbus_master.c, [686](#)
- EM_version, [90](#)
 - firmware, [90](#)
 - modbus_master.h, [696](#)
- em_version
 - modbus_master.c, [686](#)
- EMON_CO
 - canfd.h, [163](#)
 - raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c, [259](#)
- EMON_DA
 - canfd.h, [163](#)
 - raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c, [259](#)
- EMON_DM
 - canfd.h, [163](#)
- EMON_DS
 - canfd.h, [163](#)
- EMON_ER
 - canfd.h, [163](#)
 - raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c, [260](#)
- EMON_M
 - canfd.h, [163](#)
- EMON_MR
 - canfd.h, [164](#)
- EMON_SH
 - raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c, [260](#)
- EMON_SL
 - canfd.h, [164](#)
 - raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c, [260](#)
- EMON_SU
 - canfd.h, [164](#)

- raw.githubusercontent.com_linux-can_can-utils_master_can1.c, 260
- EMON_TM
 - canfd.h, 164
 - raw.githubusercontent.com_linux-can_can-utils_master_can1.c, 260
- ems
 - modbus_master.c, 687
 - modbus_master.h, 705
- emt
 - modbus_master.c, 687
 - modbus_master.h, 705
- emv
 - modbus_master.c, 687
 - modbus_master.h, 705
- ENac
 - EB_data, 82
- entropy
 - mbedtls_context, 94
- ENva
 - EB_data, 82
- ENvar
 - EB_data, 82
- ENw
 - EB_data, 82
- error
 - M_data, 92
 - mqtt_client, 96
- exit_example
 - main.c, 269
 - mqtt_pub.c, 229
 - mqtt_pub.h, 235
- F_5
 - modbus_master.h, 697
- F_6
 - modbus_master.h, 697
- F_7
 - modbus_master.h, 697
- F_8
 - modbus_master.h, 697
- F_ac
 - modbus_master.h, 697
- F_run
 - modbus_master.h, 697
- F_var
 - modbus_master.h, 697
- F_wac
 - modbus_master.h, 697
- F_wva
 - modbus_master.h, 697
- failed
 - mbedtls_sockets.h, 248
- FASTQ
 - modbus_master.h, 692
- fault_active
 - V_type, 130
- fault_count
 - V_type, 130
- faultless
 - V_type, 130
- fault_ticks
 - V_type, 130
- FDcanFDcan
 - can1.c, 295
- ferr
 - uart1_status_t, 126
 - uart2_status_t, 128
 - uart5_status_t, 129
- field
 - CAN_MSG_OBJ, 79
- FIFO1
 - can1.h, 316
- FIFO2
 - can1.h, 316
- FIFO3
 - can1.h, 316
- fifoHead
 - CAN1_RX_FIFO, 75
- filter_type
 - modbus_master.h, 696, 697
- firmware
 - EM_version, 90
- fixed_header
 - mqtt_response, 107
- float_time
 - mx_logpage_t, 121
- flush
 - B_type, 69
- FM80_charged
 - B_type, 69
- FM80_ID
 - mxcmd.h, 668
- FM80_io
 - B_type, 69
- FM80_name
 - mxcmd.h, 674
- FM80_online
 - B_type, 69
- fm80_time
 - EB_data, 82
- FM_BUFFER
 - mxcmd.h, 668
- FM_io
 - mxcmd.c, 663
 - mxcmd.h, 669
- FM_restart
 - mxcmd.c, 663
 - mxcmd.h, 670
- FM_rx
 - mxcmd.c, 663
 - mxcmd.h, 670
- FM_rx_count
 - mxcmd.c, 663
 - mxcmd.h, 670
- FM_rx_ready
 - mxcmd.c, 664

- mxcmd.h, 670
- FM_tx
 - mxcmd.c, 664
 - mxcmd.h, 670
- FM_tx_empty
 - mxcmd.c, 664
 - mxcmd.h, 671
- FMA
 - EB_data, 82
- FMbv
 - EB_data, 83
- FMpv
 - EB_data, 83
- FMw
 - EB_data, 83
- FMxx_STATE
 - mxcmd.h, 668
- formatType
 - CAN_MSG_FIELD, 78
- fout
 - raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c, 266
- frameType
 - CAN_MSG_FIELD, 78
- full_buffer
 - raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c, 266
- fw_state
 - main.c, 281
- fwrev
 - B_type, 69
- G_CONFIG
 - modbus_master.h, 697
- G_DATA1
 - modbus_master.h, 697
- G_DATA2
 - modbus_master.h, 697
- G_ID
 - modbus_master.h, 697
- G_LAST
 - modbus_master.h, 697
- G_LIGHT
 - modbus_master.h, 697
- G_MONTH
 - pge.h, 242
 - raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c, 260
- G_PASSWD
 - modbus_master.h, 697
- G_PER_DAY
 - pge.h, 242
 - raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c, 260
- G_PER_HOUR
 - pge.h, 242
 - raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c, 260
- G_SERIAL
 - modbus_master.h, 697
- G_VERSION
 - modbus_master.h, 697
- gasenergy
 - raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c, 266
- GENERATE_ENUM
 - mqtt.h, 215
- GENERATE_STRING
 - mqtt.h, 216
- get_2hz
 - modbus_master.c, 680
 - modbus_master.h, 699
- get_500ahz
 - modbus_master.c, 680
 - modbus_master.h, 700
- get_500hz
 - modbus_master.c, 680
 - modbus_master.h, 700
- get_blob_rx
 - canfdtest.c, 158
 - canfd.h, 167
- get_blob_tx
 - canfd.c, 158
 - canfd.h, 167
- get_can_data
 - batmon.c, 135
 - batmon.h, 145
- get_EEPROM
 - batmon.c, 136
 - batmon.h, 146
- getch
 - uart2.c, 617
- gridin
 - raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c, 266
- gridout
 - raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c, 266
- head
 - ringBufS_t, 123
- HIGH
 - pin_manager.h, 421
- hour_time
 - raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c, 266
- HR_SEC
 - raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c, 260
- hz
 - EM_data1, 84
 - EM_data2, 88
- id_ok
 - C_data, 74
 - DE_PRINTF.c, 295
- IDLE_DRAIN

- batmon.h, [143](#)
- idType
 - CAN_MSG_FIELD, [79](#)
- info_buffer
 - canfd.h, [168](#)
 - main.c, [281](#)
- INIT
 - modbus_master.h, [697](#)
- init_all_switch
 - dio.c, [171](#)
 - dio.h, [175](#)
- init_display
 - eadog.c, [183](#)
 - eadog.h, [202](#)
- init_mb_master_timers
 - modbus_master.c, [681](#)
 - modbus_master.h, [700](#)
- init_stream_params
 - modbus_master.c, [681](#)
 - modbus_master.h, [701](#)
- initbm_data
 - batmon.c, [136](#)
 - batmon.h, [146](#)
- INPUT
 - pin_manager.h, [421](#)
- inspector_callback
 - mqtt_client, [97](#)
- int_count
 - spi_link_type, [125](#)
- INT_TRACE
 - modbus_master.h, [692](#)
 - qconfig.h, [711](#)
 - uart2.c, [616](#)
 - uart5.c, [640](#)
- INT_TRACE_ANS
 - pin_manager.h, [421](#)
- INT_TRACE_GetValue
 - pin_manager.h, [421](#)
- INT_TRACE_LAT
 - pin_manager.h, [422](#)
- INT_TRACE_OD
 - pin_manager.h, [422](#)
- INT_TRACE_PORT
 - pin_manager.h, [422](#)
- INT_TRACE_ResetPullup
 - pin_manager.h, [422](#)
- INT_TRACE_SetAnalogMode
 - pin_manager.h, [422](#)
- INT_TRACE_SetDigitalInput
 - pin_manager.h, [422](#)
- INT_TRACE_SetDigitalMode
 - pin_manager.h, [422](#)
- INT_TRACE_SetDigitalOutput
 - pin_manager.h, [422](#)
- INT_TRACE_SetHigh
 - pin_manager.h, [423](#)
- INT_TRACE_SetLow
 - pin_manager.h, [423](#)
- INT_TRACE_SetOpenDrain
 - pin_manager.h, [423](#)
- INT_TRACE_SetPullup
 - pin_manager.h, [423](#)
- INT_TRACE_SetPushPull
 - pin_manager.h, [423](#)
- INT_TRACE_Toggle
 - pin_manager.h, [423](#)
- INT_TRACE_TRIS
 - pin_manager.h, [423](#)
- INT_TRACE_WPU
 - pin_manager.h, [424](#)
- INTERRUPT_GlobalInterruptHighDisable
 - interrupt_manager.h, [376](#)
- INTERRUPT_GlobalInterruptHighEnable
 - interrupt_manager.h, [376](#)
- INTERRUPT_GlobalInterruptLowDisable
 - interrupt_manager.h, [376](#)
- INTERRUPT_GlobalInterruptLowEnable
 - interrupt_manager.h, [377](#)
- INTERRUPT_Initialize
 - interrupt_manager.c, [375](#)
 - interrupt_manager.h, [377](#)
- interrupt_manager.c
 - __interrupt, [375](#)
 - INTERRUPT_Initialize, [375](#)
- interrupt_manager.h
 - INTERRUPT_GlobalInterruptHighDisable, [376](#)
 - INTERRUPT_GlobalInterruptHighEnable, [376](#)
 - INTERRUPT_GlobalInterruptLowDisable, [376](#)
 - INTERRUPT_GlobalInterruptLowEnable, [377](#)
 - INTERRUPT_Initialize, [377](#)
- INV_EFF_VAL
 - batmon.h, [143](#)
- IO_RB5_ANS
 - pin_manager.h, [424](#)
- IO_RB5_GetValue
 - pin_manager.h, [424](#)
- IO_RB5_LAT
 - pin_manager.h, [424](#)
- IO_RB5_OD
 - pin_manager.h, [424](#)
- IO_RB5_PORT
 - pin_manager.h, [424](#)
- IO_RB5_ResetPullup
 - pin_manager.h, [424](#)
- IO_RB5_SetAnalogMode
 - pin_manager.h, [424](#)
- IO_RB5_SetDigitalInput
 - pin_manager.h, [424](#)
- IO_RB5_SetDigitalMode
 - pin_manager.h, [425](#)
- IO_RB5_SetDigitalOutput
 - pin_manager.h, [425](#)
- IO_RB5_SetHigh
 - pin_manager.h, [425](#)
- IO_RB5_SetLow
 - pin_manager.h, [425](#)

- IO_RB5_SetOpenDrain
 - pin_manager.h, [425](#)
- IO_RB5_SetPullup
 - pin_manager.h, [425](#)
- IO_RB5_SetPushPull
 - pin_manager.h, [425](#)
- IO_RB5_Toggle
 - pin_manager.h, [426](#)
- IO_RB5_TRIS
 - pin_manager.h, [426](#)
- IO_RB5_WPU
 - pin_manager.h, [426](#)
- IO_RB6_Toggle
 - uart2.c, [616](#)
 - uart5.c, [640](#)
- IO_RD7_ANS
 - pin_manager.h, [426](#)
- IO_RD7_GetValue
 - pin_manager.h, [426](#)
- IO_RD7_LAT
 - pin_manager.h, [426](#)
- IO_RD7_OD
 - pin_manager.h, [426](#)
- IO_RD7_PORT
 - pin_manager.h, [426](#)
- IO_RD7_ResetPullup
 - pin_manager.h, [426](#)
- IO_RD7_SetAnalogMode
 - pin_manager.h, [427](#)
- IO_RD7_SetDigitalInput
 - pin_manager.h, [427](#)
- IO_RD7_SetDigitalMode
 - pin_manager.h, [427](#)
- IO_RD7_SetDigitalOutput
 - pin_manager.h, [427](#)
- IO_RD7_SetHigh
 - pin_manager.h, [427](#)
- IO_RD7_SetLow
 - pin_manager.h, [427](#)
- IO_RD7_SetOpenDrain
 - pin_manager.h, [427](#)
- IO_RD7_SetPullup
 - pin_manager.h, [428](#)
- IO_RD7_SetPushPull
 - pin_manager.h, [428](#)
- IO_RD7_Toggle
 - pin_manager.h, [428](#)
- IO_RD7_TRIS
 - pin_manager.h, [428](#)
- IO_RD7_WPU
 - pin_manager.h, [428](#)
- IO_TEST
 - mxcmd.h, [668](#)
- IOCAF1_DefaultInterruptHandler
 - pin_manager.c, [387](#)
 - pin_manager.h, [467](#)
- IOCAF1_InterruptHandler
 - pin_manager.c, [391](#)
- pin_manager.h, [472](#)
- IOCAF1_ISR
 - pin_manager.c, [387](#)
 - pin_manager.h, [467](#)
- IOCAF1_SetInterruptHandler
 - pin_manager.c, [388](#)
 - pin_manager.h, [467](#)
- IOCAF2_DefaultInterruptHandler
 - pin_manager.c, [388](#)
 - pin_manager.h, [468](#)
- IOCAF2_InterruptHandler
 - pin_manager.c, [391](#)
 - pin_manager.h, [473](#)
- IOCAF2_ISR
 - pin_manager.c, [388](#)
 - pin_manager.h, [468](#)
- IOCAF2_SetInterruptHandler
 - pin_manager.c, [389](#)
 - pin_manager.h, [469](#)
- IOCAF5_DefaultInterruptHandler
 - pin_manager.c, [389](#)
 - pin_manager.h, [469](#)
- IOCAF5_InterruptHandler
 - pin_manager.c, [392](#)
 - pin_manager.h, [473](#)
- IOCAF5_ISR
 - pin_manager.c, [389](#)
 - pin_manager.h, [470](#)
- IOCAF5_SetInterruptHandler
 - pin_manager.c, [390](#)
 - pin_manager.h, [470](#)
- json
 - raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c,
[266](#)
- keep_alive
 - mqtt_client, [97](#)
- kilowatt_hours
 - mx_logpage_t, [121](#)
- kilowatts_peak
 - mx_logpage_t, [121](#)
- kvahp
 - EM_data2, [88](#)
- kvaht
 - EM_data2, [88](#)
- kvarhnp
 - EM_data2, [88](#)
- kvarhnt
 - EM_data2, [88](#)
- kvarhpp
 - EM_data2, [88](#)
- kvarhpt
 - EM_data2, [88](#)
- kwhnp
 - EM_data2, [88](#)
- kwhnt
 - EM_data2, [88](#)
- kwhpl1

EM_data2, [88](#)
 kwhpl2
 EM_data2, [88](#)
 kwhpl3
 EM_data2, [89](#)
 kwhpp
 EM_data2, [89](#)
 kwhpt
 EM_data2, [89](#)
 L_SWITCH_ANS
 pin_manager.h, [428](#)
 L_SWITCH_GetValue
 pin_manager.h, [428](#)
 L_SWITCH_LAT
 pin_manager.h, [428](#)
 L_SWITCH_OD
 pin_manager.h, [429](#)
 L_SWITCH_PORT
 pin_manager.h, [429](#)
 L_SWITCH_ResetPullup
 pin_manager.h, [429](#)
 L_SWITCH_SetAnalogMode
 pin_manager.h, [429](#)
 L_SWITCH_SetDigitalInput
 pin_manager.h, [429](#)
 L_SWITCH_SetDigitalMode
 pin_manager.h, [429](#)
 L_SWITCH_SetDigitalOutput
 pin_manager.h, [429](#)
 L_SWITCH_SetHigh
 pin_manager.h, [429](#)
 L_SWITCH_SetLow
 pin_manager.h, [430](#)
 L_SWITCH_SetOpenDrain
 pin_manager.h, [430](#)
 L_SWITCH_SetPullup
 pin_manager.h, [430](#)
 L_SWITCH_SetPushPull
 pin_manager.h, [430](#)
 L_SWITCH_Toggle
 pin_manager.h, [430](#)
 L_SWITCH_TRIS
 pin_manager.h, [430](#)
 L_SWITCH_WPU
 pin_manager.h, [430](#)
 LCD0
 eadog.h, [193](#)
 LCD1
 eadog.h, [193](#)
 LCD2
 eadog.h, [193](#)
 LCD3
 eadog.h, [194](#)
 LCD_BUF_SIZ
 canfd.h, [164](#)
 LCD_CLEAR_HOME
 eadog.h, [194](#)
 LCD_CMD_BRI

 eadog.h, [194](#)
 LCD_CMD_CLR
 eadog.h, [194](#)
 LCD_CMD_CONT
 eadog.h, [194](#)
 LCD_CMD_HOME
 eadog.h, [194](#)
 LCD_CMD_MASK
 eadog.h, [194](#)
 LCD_CMD_OFF
 eadog.h, [194](#)
 LCD_CMD_ON
 eadog.h, [194](#)
 LCD_CMD_SET
 eadog.h, [194](#)
 LCD_DATA
 spi_link_type, [125](#)
 lcd_dma_buf
 dma1.h, [373](#)
 LCD_PWR_DELAY
 eadog.h, [195](#)
 LCD_TIMER
 spi_link_type, [125](#)
 light_ok
 C_data, [74](#)
 linux/matesocketcan/.dep.inc, [210](#)
 linux/matesocketcan/build/Debug/GNU-Linux/_ext/5c0/raw.githubusercontent.com
 can_can-utils_master_canfdtest.o.d, [210](#)
 linux/matesocketcan/build/Release/GNU-Linux/_ext/5c0/raw.githubusercontent.com
 can_can-utils_master_canfdtest.o.d, [210](#)
 linux/matesocketcan/mqtt.c, [210](#)
 linux/matesocketcan/mqtt.h, [211](#), [219](#)
 linux/matesocketcan/mqtt_pal.c, [225](#)
 linux/matesocketcan/mqtt_pal.h, [225](#), [226](#)
 linux/matesocketcan/mqtt_pub.c, [228](#)
 linux/matesocketcan/mqtt_pub.h, [233](#), [238](#)
 linux/matesocketcan/nbproject/private/c_standard_headers_indexer.c,
 [238](#)
 linux/matesocketcan/nbproject/private/cpp_standard_headers_indexer.cpp,
 [239](#)
 linux/matesocketcan/pge.h, [241](#), [243](#)
 linux/matesocketcan/templates/bearssl_sockets.h, [243](#),
 [244](#)
 linux/matesocketcan/templates/bio_sockets.h, [246](#), [247](#)
 linux/matesocketcan/templates/mbedtls_sockets.h, [247](#),
 [249](#)
 linux/matesocketcan/templates/openssl_sockets.h, [251](#),
 [252](#)
 linux/matesocketcan/templates/posix_sockets.h, [253](#),
 [255](#)
 linux/mqtt_cl/main.c, [268](#)
 linux/mqtt_cl/nbproject/private/c_standard_headers_indexer.c,
 [239](#)
 linux/mqtt_cl/nbproject/private/cpp_standard_headers_indexer.cpp,
 [240](#)
 linux/raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c,
 [256](#)
 load

- raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c, 266
- loaded
 - EB_data, 83
- LOG
 - B_type, 69
- log
 - B_type, 70
 - blob_log, 72
- log_buffer
 - canfd.h, 168
 - main.c, 281
- log_format
 - batmon.h, 150
- log_time
 - raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c, 262
- LOG_TO_FILE
 - raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c, 260
- LOG_VARS
 - batmon.h, 143
- LOG_VERSION
 - raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c, 261
- LOW
 - pin_manager.h, 431
- LP_BUFFER_SIZE
 - mxcmd.h, 668
- lp_filter
 - mxcmd.c, 664
 - mxcmd.h, 671
- lpwm
 - PWMDC, 122
- LSB
 - eadog.h, 195
- M
 - C_data, 74
 - modbus_master.c, 687
 - modbus_master.h, 705
- M_data, 91
 - blink_lock, 91
 - boot_code, 91
 - config, 91
 - crc_calc, 91
 - crc_data, 91
 - crc_error, 91
 - error, 92
 - modbus_master.h, 696
 - power_on, 92
 - pwm_volts, 92
 - recv_count, 92
 - rx, 92
 - send_count, 92
 - sends, 92
 - stable, 92
 - to_error, 92
- M_SWITCH_ANS
 - pin_manager.h, 431
 - M_SWITCH_GetValue
 - pin_manager.h, 431
 - M_SWITCH_LAT
 - pin_manager.h, 431
 - M_SWITCH_OD
 - pin_manager.h, 431
 - M_SWITCH_PORT
 - pin_manager.h, 431
 - M_SWITCH_ResetPullup
 - pin_manager.h, 431
 - M_SWITCH_SetAnalogMode
 - pin_manager.h, 431
 - M_SWITCH_SetDigitalInput
 - pin_manager.h, 431
 - M_SWITCH_SetDigitalMode
 - pin_manager.h, 432
 - M_SWITCH_SetDigitalOutput
 - pin_manager.h, 432
 - M_SWITCH_SetHigh
 - pin_manager.h, 432
 - M_SWITCH_SetLow
 - pin_manager.h, 432
 - M_SWITCH_SetOpenDrain
 - pin_manager.h, 432
 - M_SWITCH_SetPullup
 - pin_manager.h, 432
 - M_SWITCH_SetPushPull
 - pin_manager.h, 432
 - M_SWITCH_Toggle
 - pin_manager.h, 433
 - M_SWITCH_TRIS
 - pin_manager.h, 433
 - M_SWITCH_WPU
 - pin_manager.h, 433
 - M_time_data, 93
 - clock_2hz, 93
 - clock_500ahz, 93
 - clock_500hz, 93
 - modbus_master.h, 696
- MADDR
 - modbus_master.h, 692
- main
 - canfilter.c, 153
 - canreceive.c, 153
 - cantransmit.c, 154
 - main.c, 269, 271, 274
 - raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c, 263
- main.c, 271
 - B, 280
 - bat_amp_whole, 280
 - buffer, 280
 - build_date, 281
 - build_time, 281
 - can_newtime, 281
 - can_timer, 281
 - cc_mode, 281

- client_refresher, [269](#)
- CMD_LEN, [272](#)
- E_SAVE, [272](#)
- E_UPDATE, [272](#)
- exit_example, [269](#)
- fw_state, [281](#)
- info_buffer, [281](#)
- log_buffer, [281](#)
- main, [269](#), [271](#), [274](#)
- mx_code, [281](#)
- PACE, [273](#)
- panel_watts, [281](#)
- publish_callback, [270](#)
- REC_LEN, [273](#)
- REC_LOG_LEN, [273](#)
- REC_STATUS_LEN, [273](#)
- run_day_to_night, [274](#)
- run_night_to_day, [275](#)
- show_can, [282](#)
- spinners, [275](#)
- state, [282](#)
- state_battery, [273](#)
- state_battery_cb, [276](#)
- state_batteryv, [273](#)
- state_batteryv_cb, [276](#)
- state_date, [273](#)
- state_fwrev, [273](#)
- state_init, [273](#)
- state_init_cb, [276](#)
- state_last, [273](#)
- state_misc, [273](#)
- state_misc_cb, [277](#)
- state_mx_log, [273](#)
- state_mx_log_cb, [277](#)
- state_mx_status, [273](#)
- state_mx_status_cb, [278](#)
- state_panel, [273](#)
- state_panelv_cb, [278](#)
- state_status, [273](#)
- state_status_cb, [279](#)
- state_time, [273](#)
- state_type, [273](#)
- state_watts, [273](#)
- state_watts_cb, [279](#)
- tickCount, [282](#)
- time_show_can, [282](#)
- vf, [282](#)
- volt_fract, [282](#)
- volt_whole, [282](#)
- vw, [282](#)
- wdtdelay, [279](#)
- master_controller_work
 - modbus_master.c, [682](#)
 - modbus_master.h, [701](#)
- mateQ84.X/build/default/production/_ext/1472/batmon.p1.d, [284](#)
- mateQ84.X/build/default/production/_ext/1472/canfd.p1.d, [284](#)
- mateQ84.X/build/default/production/_ext/1472/dio.p1.d, [284](#)
- mateQ84.X/build/default/production/_ext/1472/eadog.p1.d, [284](#)
- mateQ84.X/build/default/production/_ext/1472/modbus_master.p1.d, [284](#)
- mateQ84.X/build/default/production/_ext/1472/timers.p1.d, [284](#)
- mateQ84.X/build/default/production/main.p1.d, [284](#)
- mateQ84.X/build/default/production/mcc_generated_files/adc.p1.d, [284](#)
- mateQ84.X/build/default/production/mcc_generated_files/can1.p1.d, [284](#)
- mateQ84.X/build/default/production/mcc_generated_files/clc7.p1.d, [284](#)
- mateQ84.X/build/default/production/mcc_generated_files/clc8.p1.d, [284](#)
- mateQ84.X/build/default/production/mcc_generated_files/device_config.p1.d, [284](#)
- mateQ84.X/build/default/production/mcc_generated_files/dma1.p1.d, [284](#)
- mateQ84.X/build/default/production/mcc_generated_files/interrupt_manage.p1.d, [284](#)
- mateQ84.X/build/default/production/mcc_generated_files/mcc.p1.d, [284](#)
- mateQ84.X/build/default/production/mcc_generated_files/pin_manager.p1.d, [284](#)
- mateQ84.X/build/default/production/mcc_generated_files/spi1.p1.d, [284](#)
- mateQ84.X/build/default/production/mcc_generated_files/tmr0.p1.d, [284](#)
- mateQ84.X/build/default/production/mcc_generated_files/tmr2.p1.d, [284](#)
- mateQ84.X/build/default/production/mcc_generated_files/tmr4.p1.d, [284](#)
- mateQ84.X/build/default/production/mcc_generated_files/tmr5.p1.d, [284](#)
- mateQ84.X/build/default/production/mcc_generated_files/tmr6.p1.d, [284](#)
- mateQ84.X/build/default/production/mcc_generated_files/uart1.p1.d, [284](#)
- mateQ84.X/build/default/production/mcc_generated_files/uart2.p1.d, [284](#)
- mateQ84.X/build/default/production/mcc_generated_files/uart5.p1.d, [284](#)
- mateQ84.X/build/default/production/mxcmd.p1.d, [284](#)
- mateQ84.X/dist/default/production/doprnt.d, [284](#)
- mateQ84.X/main.c, [271](#)
- mateQ84.X/mcc_generated_files/adc.c, [284](#)
- mateQ84.X/mcc_generated_files/adc.h, [288](#), [291](#)
- mateQ84.X/mcc_generated_files/can1.c, [292](#)
- mateQ84.X/mcc_generated_files/can1.h, [314](#), [335](#)
- mateQ84.X/mcc_generated_files/can_types.h, [336](#), [342](#)
- mateQ84.X/mcc_generated_files/clc7.c, [343](#)
- mateQ84.X/mcc_generated_files/clc7.h, [345](#), [347](#)
- mateQ84.X/mcc_generated_files/clc8.c, [348](#)
- mateQ84.X/mcc_generated_files/clc8.h, [349](#), [351](#)
- mateQ84.X/mcc_generated_files/device_config.c, [352](#)

- mateQ84.X/mcc_generated_files/device_config.h, [352](#), [353](#)
- mateQ84.X/mcc_generated_files/dma1.c, [354](#)
- mateQ84.X/mcc_generated_files/dma1.h, [364](#), [374](#)
- mateQ84.X/mcc_generated_files/interrupt_manager.c, [374](#)
- mateQ84.X/mcc_generated_files/interrupt_manager.h, [376](#), [378](#)
- mateQ84.X/mcc_generated_files/mcc.c, [378](#)
- mateQ84.X/mcc_generated_files/mcc.h, [381](#), [385](#)
- mateQ84.X/mcc_generated_files/pin_manager.c, [386](#)
- mateQ84.X/mcc_generated_files/pin_manager.h, [392](#), [474](#)
- mateQ84.X/mcc_generated_files/spi1.c, [481](#)
- mateQ84.X/mcc_generated_files/spi1.h, [484](#), [488](#)
- mateQ84.X/mcc_generated_files/tmr0.c, [489](#)
- mateQ84.X/mcc_generated_files/tmr0.h, [495](#), [501](#)
- mateQ84.X/mcc_generated_files/tmr2.c, [501](#)
- mateQ84.X/mcc_generated_files/tmr2.h, [512](#), [525](#)
- mateQ84.X/mcc_generated_files/tmr4.c, [529](#)
- mateQ84.X/mcc_generated_files/tmr4.h, [539](#), [552](#)
- mateQ84.X/mcc_generated_files/tmr5.c, [556](#)
- mateQ84.X/mcc_generated_files/tmr5.h, [563](#), [570](#)
- mateQ84.X/mcc_generated_files/tmr6.c, [571](#)
- mateQ84.X/mcc_generated_files/tmr6.h, [581](#), [594](#)
- mateQ84.X/mcc_generated_files/uart1.c, [598](#)
- mateQ84.X/mcc_generated_files/uart1.h, [606](#), [614](#)
- mateQ84.X/mcc_generated_files/uart2.c, [615](#)
- mateQ84.X/mcc_generated_files/uart2.h, [627](#), [638](#)
- mateQ84.X/mcc_generated_files/uart5.c, [639](#)
- mateQ84.X/mcc_generated_files/uart5.h, [650](#), [661](#)
- mateQ84.X/mxcmd.c, [662](#)
- mateQ84.X/mxcmd.h, [665](#), [675](#)
- MAX_ALT_DIS
 - batmon.h, [143](#)
- MAX_B_BUF
 - mxcmd.h, [668](#)
- MAX_C_BUF
 - mxcmd.h, [668](#)
- MAX_DATA
 - modbus_master.h, [692](#)
- MAX_SHAPES
 - qconfig.h, [711](#)
- max_strlen
 - eadog.h, [195](#)
- mb16_swap
 - modbus_master.c, [682](#)
 - modbus_master.h, [702](#)
- mb32_swap
 - modbus_master.c, [682](#)
 - modbus_master.h, [702](#)
- MB_EM540
 - modbus_master.h, [693](#)
- MB_EM540_ID_H
 - modbus_master.h, [693](#)
- MB_EM540_ID_L
 - modbus_master.h, [693](#)
- MB_MASTER
 - modbus_master.h, [693](#)
- mb_setup
 - modbus_master.c, [683](#)
 - modbus_master.h, [702](#)
- mb_tx_test
 - modbus_master.c, [683](#)
 - modbus_master.h, [703](#)
- mbedtls_context, [93](#)
 - ca_crt, [94](#)
 - ctr_drbg, [94](#)
 - entropy, [94](#)
 - net_ctx, [94](#)
 - ssl_conf, [94](#)
 - ssl_ctx, [94](#)
- mbedtls_net_poll
 - mbedtls_sockets.h, [248](#)
- MBEDTLS_NET_POLL_READ
 - mbedtls_sockets.h, [248](#)
- MBEDTLS_NET_POLL_WRITE
 - mbedtls_sockets.h, [248](#)
- mbedtls_sockets.h
 - cert_verify_failed, [248](#)
 - failed, [248](#)
 - mbedtls_net_poll, [248](#)
 - MBEDTLS_NET_POLL_READ, [248](#)
 - MBEDTLS_NET_POLL_WRITE, [248](#)
 - open_nb_socket, [249](#)
- mcc.c
 - OSCILLATOR_Initialize, [379](#)
 - PMD_Initialize, [379](#)
 - SYSTEM_Initialize, [379](#)
 - SystemArbiter_Initialize, [381](#)
- mcc.h
 - OSCILLATOR_Initialize, [382](#)
 - PMD_Initialize, [382](#)
 - SYSTEM_Initialize, [383](#)
 - SystemArbiter_Initialize, [385](#)
- mcmd
 - C_data, [74](#)
- mem_end
 - mqtt_message_queue, [103](#)
- mem_size
 - mqtt_client, [97](#)
- mem_start
 - mqtt_client, [97](#)
 - mqtt_message_queue, [103](#)
- MIRROR_BUF
 - canfd.h, [164](#)
- MISC_ANS
 - pin_manager.h, [433](#)
- MISC_GetValue
 - pin_manager.h, [433](#)
- MISC_LAT
 - pin_manager.h, [433](#)
- MISC_OD
 - pin_manager.h, [433](#)
- MISC_PORT
 - pin_manager.h, [433](#)

MISC_ResetPullup
 pin_manager.h, [433](#)
MISC_SetAnalogMode
 pin_manager.h, [434](#)
MISC_SetDigitalInput
 pin_manager.h, [434](#)
MISC_SetDigitalMode
 pin_manager.h, [434](#)
MISC_SetDigitalOutput
 pin_manager.h, [434](#)
MISC_SetHigh
 pin_manager.h, [434](#)
MISC_SetLow
 pin_manager.h, [434](#)
MISC_SetOpenDrain
 pin_manager.h, [434](#)
MISC_SetPullup
 pin_manager.h, [435](#)
MISC_SetPushPull
 pin_manager.h, [435](#)
MISC_Toggle
 pin_manager.h, [435](#)
MISC_TRIS
 pin_manager.h, [435](#)
MISC_WPU
 pin_manager.h, [435](#)
MLED_ANS
 pin_manager.h, [435](#)
MLED_GetValue
 pin_manager.h, [435](#)
MLED_LAT
 pin_manager.h, [435](#)
MLED_OD
 pin_manager.h, [436](#)
MLED_PORT
 pin_manager.h, [436](#)
MLED_ResetPullup
 pin_manager.h, [436](#)
MLED_SetAnalogMode
 pin_manager.h, [436](#)
MLED_SetDigitalInput
 pin_manager.h, [436](#)
MLED_SetDigitalMode
 pin_manager.h, [436](#)
MLED_SetDigitalOutput
 pin_manager.h, [436](#)
MLED_SetHigh
 pin_manager.h, [436](#)
MLED_SetLow
 pin_manager.h, [437](#)
MLED_SetOpenDrain
 pin_manager.h, [437](#)
MLED_SetPullup
 pin_manager.h, [437](#)
MLED_SetPushPull
 pin_manager.h, [437](#)
MLED_Toggle
 pin_manager.h, [437](#)
MLED_TRIS
 pin_manager.h, [437](#)
MLED_WPU
 pin_manager.h, [437](#)
modbus_command
 C_data, [74](#)
modbus_em_config
 modbus_master.c, [687](#)
modbus_em_data1
 modbus_master.c, [687](#)
modbus_em_data2
 modbus_master.c, [687](#)
modbus_em_id
 modbus_master.c, [687](#)
modbus_em_light
 modbus_master.c, [687](#)
modbus_em_passwd
 modbus_master.c, [688](#)
modbus_em_serial
 modbus_master.c, [688](#)
modbus_em_version
 modbus_master.c, [688](#)
modbus_master.c, [677](#)
 C, [685](#)
 cc_buffer, [685](#)
 cc_buffer_tx, [685](#)
 cc_stream_file, [686](#)
 clear_2hz, [679](#)
 clear_500ahz, [679](#)
 clear_500hz, [679](#)
 crc16, [679](#)
 em, [686](#)
 em_config, [686](#)
 em_data1, [686](#)
 em_data2, [686](#)
 em_id, [686](#)
 em_light, [686](#)
 em_passwd, [686](#)
 em_serial, [686](#)
 em_version, [686](#)
 ems, [687](#)
 emt, [687](#)
 emv, [687](#)
 get_2hz, [680](#)
 get_500ahz, [680](#)
 get_500hz, [680](#)
 init_mb_master_timers, [681](#)
 init_stream_params, [681](#)
 M, [687](#)
 master_controller_work, [682](#)
 mb16_swap, [682](#)
 mb32_swap, [682](#)
 mb_setup, [683](#)
 mb_tx_test, [683](#)

- modbus_em_config, 687
- modbus_em_data1, 687
- modbus_em_data2, 687
- modbus_em_id, 687
- modbus_em_light, 687
- modbus_em_passwd, 688
- modbus_em_serial, 688
- modbus_em_version, 688
- modbus_rtu_send_msg, 684
- MT, 688
- my_modbus_rx_32, 684
- OFF, 678
- ON, 678
- timer_2ms_tick, 685
- timer_500ms_tick, 685
- V, 688
- modbus_master.h, 689
 - BOFF, 692
 - BON, 692
 - C, 705
 - C_data, 695
 - CDELAY, 692
 - CLEAR, 697
 - clear_2hz, 698
 - clear_500ahz, 698
 - clear_500hz, 699
 - cmd_type, 695, 696
 - comm_type, 695, 697
 - crc16, 699
 - DEBUG_R, 692
 - DUPL_DELAY, 692
 - em, 705
 - EM_data1, 696
 - EM_data2, 696
 - EM_DATA_LEN1, 692
 - EM_DATA_LEN2, 692
 - EM_serial, 696
 - EM_version, 696
 - ems, 705
 - emt, 705
 - emv, 705
 - F_5, 697
 - F_6, 697
 - F_7, 697
 - F_8, 697
 - F_ac, 697
 - F_run, 697
 - F_var, 697
 - F_wac, 697
 - F_wva, 697
 - FASTQ, 692
 - filter_type, 696, 697
 - G_CONFIG, 697
 - G_DATA1, 697
 - G_DATA2, 697
 - G_ID, 697
 - G_LAST, 697
 - G_LIGHT, 697
 - G_PASSWD, 697
 - G_SERIAL, 697
 - G_VERSION, 697
 - get_2hz, 699
 - get_500ahz, 700
 - get_500hz, 700
 - INIT, 697
 - init_mb_master_timers, 700
 - init_stream_params, 701
 - INT_TRACE, 692
 - M, 705
 - M_data, 696
 - M_time_data, 696
 - MADDR, 692
 - master_controller_work, 701
 - MAX_DATA, 692
 - mb16_swap, 702
 - mb32_swap, 702
 - MB_EM540, 693
 - MB_EM540_ID_H, 693
 - MB_EM540_ID_L, 693
 - MB_MASTER, 693
 - mb_setup, 702
 - mb_tx_test, 703
 - MM_ERROR_C, 693
 - MM_ERROR_S, 693
 - modbus_rtu_send_msg, 703
 - MT, 705
 - my_modbus_rx_32, 704
 - QDELAY, 693
 - RDELAY, 693
 - READ_COILS, 693
 - READ_DISCRETE_INPUTS, 693
 - READ_HOLDING_REGISTERS, 694
 - READ_INPUT_REGISTERS, 694
 - RECV, 697
 - SEND, 697
 - SERIAL_DATA_LEN, 694
 - Serror, 694
 - SPACING, 694
 - Srbuffer, 694
 - Sread, 694
 - Strmt, 694
 - SWMBMVER, 694
 - Swrite, 694
 - T_begin, 697
 - T_clear, 697
 - T_config, 698
 - T_data, 698
 - T_id, 698
 - T_init, 698
 - T_init_d, 698
 - T_light, 698
 - T_misc1, 698
 - T_misc2, 698
 - T_misc3, 698
 - T_passwd, 698
 - T_recv, 698

- T_recv_r, 698
- T_send, 698
- T_send_d, 698
- T_serial, 698
- T_spacing, 698
- T_version, 698
- TDELAY, 695
- TEDELAY, 695
- timer_2ms_tick, 704
- timer_500ms_tick, 704
- TODELAY, 695
- trace_type, 696, 697
- V, 705
- WRITE_MULTIPLE_COILS, 695
- WRITE_MULTIPLE_REGISTERS, 695
- WRITE_SINGLE_COIL, 695
- WRITE_SINGLE_REGISTER, 695
- modbus_name
 - mxcmd.h, 674
- modbus_online
 - B_type, 70
- modbus_rtu_send_msg
 - modbus_master.c, 684
 - modbus_master.h, 703
- modbus_rx
 - V_type, 130
- modbus_tx
 - V_type, 130
- modulo_dec
 - ringbufs.c, 715
- modulo_inc
 - ringbufs.c, 715
- motor_speed
 - V_type, 130
- mq
 - mqtt_client, 97
- mqtt.h
 - __ALL_MQTT_ERRORS, 215
 - __mqtt_pack_str, 216
 - __mqtt_pack_uint16, 217
 - __mqtt_packed_cstrlen, 215
 - __mqtt_ping, 218
 - __mqtt_unpack_uint16, 218
 - GENERATE_ENUM, 215
 - GENERATE_STRING, 216
- mqtt_check
 - mqtt_pub.c, 230
 - mqtt_pub.h, 235
- mqtt_client, 95
 - curr, 96
 - curr_sz, 96
 - error, 96
 - inspector_callback, 97
 - keep_alive, 97
 - mem_size, 97
 - mem_start, 97
 - mq, 97
 - mutex, 97
 - number_of_keep_alives, 98
 - number_of_timeouts, 98
 - pid_lfsr, 98
 - publish_response_callback, 98
 - publish_response_callback_state, 98
 - reconnect_callback, 99
 - reconnect_state, 99
 - recv_buffer, 99
 - response_timeout, 99
 - send_offset, 99
 - socketfd, 100
 - time_of_last_send, 100
 - typical_response_time, 100
- MQTT_CONNACK_ACCEPTED
 - Control Packet Deserialization, 29
- MQTT_CONNACK_REFUSED_BAD_USER_NAME_OR_PASSWORD
 - Control Packet Deserialization, 29
- MQTT_CONNACK_REFUSED_IDENTIFIER_REJECTED
 - Control Packet Deserialization, 29
- MQTT_CONNACK_REFUSED_NOT_AUTHORIZED
 - Control Packet Deserialization, 29
- MQTT_CONNACK_REFUSED_PROTOCOL_VERSION
 - Control Packet Deserialization, 29
- MQTT_CONNACK_REFUSED_SERVER_UNAVAILABLE
 - Control Packet Deserialization, 29
- mqtt_connect
 - API, 11
- MQTT_CONNECT_CLEAN_SESSION
 - Control Packet Serialization, 22
- MQTT_CONNECT_PASSWORD
 - Control Packet Serialization, 22
- MQTT_CONNECT_RESERVED
 - Control Packet Serialization, 22
- MQTT_CONNECT_USER_NAME
 - Control Packet Serialization, 22
- MQTT_CONNECT_WILL_FLAG
 - Control Packet Serialization, 22
- MQTT_CONNECT_WILL_QOS_0
 - Control Packet Serialization, 22
- MQTT_CONNECT_WILL_QOS_1
 - Control Packet Serialization, 22
- MQTT_CONNECT_WILL_QOS_2
 - Control Packet Serialization, 22
- MQTT_CONNECT_WILL_RETAIN
 - Control Packet Serialization, 22
- MQTT_CONTROL_CONNACK
 - Control Packet Deserialization, 30
- MQTT_CONTROL_CONNECT
 - Control Packet Deserialization, 30
- MQTT_CONTROL_DISCONNECT
 - Control Packet Deserialization, 30
- MQTT_CONTROL_PINGREQ
 - Control Packet Deserialization, 30
- MQTT_CONTROL_PINGRESP
 - Control Packet Deserialization, 30
- MQTT_CONTROL_PUBACK
 - Control Packet Deserialization, 30
- MQTT_CONTROL_PUBCOMP

- Control Packet Deserialization, [30](#)
- MQTT_CONTROL_PUBLISH
 - Control Packet Deserialization, [30](#)
- MQTT_CONTROL_PUBREC
 - Control Packet Deserialization, [30](#)
- MQTT_CONTROL_PUBREL
 - Control Packet Deserialization, [30](#)
- MQTT_CONTROL_SUBACK
 - Control Packet Deserialization, [30](#)
- MQTT_CONTROL_SUBSCRIBE
 - Control Packet Deserialization, [30](#)
- MQTT_CONTROL_UNSUBACK
 - Control Packet Deserialization, [30](#)
- MQTT_CONTROL_UNSUBSCRIBE
 - Control Packet Deserialization, [30](#)
- mqtt_disconnect
 - API, [12](#)
- mqtt_error_str
 - API, [12](#)
- MQTT_ERROR_UNKNOWN
 - API, [10](#)
- mqtt_exit
 - mqtt_pub.c, [230](#)
 - mqtt_pub.h, [236](#)
- mqtt_fixed_header, [100](#)
 - control_flags, [101](#)
 - control_type, [101](#)
 - remaining_length, [101](#)
- mqtt_init
 - API, [13](#)
- mqtt_init_reconnect
 - API, [15](#)
- mqtt_message_queue, [102](#)
 - curr, [103](#)
 - curr_sz, [103](#)
 - mem_end, [103](#)
 - mem_start, [103](#)
 - queue_tail, [103](#)
- mqtt_mq_clean
 - Utilities, [37](#)
- mqtt_mq_currSZ
 - Utilities, [34](#)
- mqtt_mq_find
 - Utilities, [37](#)
- mqtt_mq_get
 - Utilities, [34](#)
- mqtt_mq_init
 - Utilities, [38](#)
- mqtt_mq_length
 - Utilities, [34](#)
- mqtt_mq_register
 - Utilities, [38](#)
- MQTT_OK
 - API, [10](#)
- mqtt_pack_connection_request
 - Control Packet Serialization, [23](#)
- mqtt_pack_disconnect
 - Control Packet Serialization, [23](#)
- mqtt_pack_fixed_header
 - Control Packet Serialization, [24](#)
- mqtt_pack_ping_request
 - Control Packet Serialization, [24](#)
- mqtt_pack_publish_request
 - Control Packet Serialization, [25](#)
- mqtt_pack_pubxxx_request
 - Control Packet Serialization, [25](#)
- mqtt_pack_subscribe_request
 - Control Packet Serialization, [26](#)
- mqtt_pack_unsubscribe_request
 - Control Packet Serialization, [27](#)
- mqtt_pal_rcvll
 - Platform abstraction layer, [40](#)
- mqtt_pal_sendall
 - Platform abstraction layer, [41](#)
- mqtt_ping
 - API, [15](#)
- MQTT_PROTOCOL_LEVEL
 - Control Packet Serialization, [21](#)
- mqtt_pub.c
 - addr, [232](#)
 - client, [232](#)
 - client_daemon, [232](#)
 - client_id, [232](#)
 - client_refresher, [229](#)
 - connect_flags, [232](#)
 - exit_example, [229](#)
 - mqtt_check, [230](#)
 - mqtt_exit, [230](#)
 - mqtt_socket, [230](#)
 - port, [232](#)
 - publish_callback, [231](#)
 - recvbuf, [232](#)
 - sendbuf, [232](#)
 - sockfd, [232](#)
 - topic, [232](#)
- mqtt_pub.h
 - ADDR_MQTT, [234](#)
 - client_refresher, [234](#)
 - DATA_MQTT_SOLAR, [234](#)
 - exit_example, [235](#)
 - mqtt_check, [235](#)
 - mqtt_exit, [236](#)
 - mqtt_socket, [236](#)
 - MQTT_VERSION, [234](#)
 - publish_callback, [237](#)
- mqtt_publish
 - API, [16](#)
- MQTT_PUBLISH_DUP
 - Control Packet Serialization, [22](#)
- MQTT_PUBLISH_QOS_0
 - Control Packet Serialization, [22](#)
- MQTT_PUBLISH_QOS_1
 - Control Packet Serialization, [22](#)
- MQTT_PUBLISH_QOS_2
 - Control Packet Serialization, [22](#)
- MQTT_PUBLISH_QOS_MASK

- Control Packet Serialization, 22
- MQTT_PUBLISH_RETAIN
 - Control Packet Serialization, 22
- MQTT_QUEUED_AWAITING_ACK
 - Utilities, 35
- MQTT_QUEUED_COMPLETE
 - Utilities, 35
- mqtt_queued_message, 104
 - control_type, 104
 - packet_id, 104
 - size, 105
 - start, 105
 - state, 105
 - time_sent, 105
- MQTT_QUEUED_UNSENT
 - Utilities, 35
- mqtt_reconnect
 - API, 17
- mqtt_reinit
 - API, 17
- mqtt_response, 106
 - connack, 107
 - decoded, 107
 - fixed_header, 107
 - pingresp, 107
 - puback, 107
 - pubcomp, 107
 - publish, 108
 - pubrec, 108
 - pubrel, 108
 - suback, 108
 - unsuback, 108
- mqtt_response_connack, 108
 - return_code, 109
 - session_present_flag, 109
- mqtt_response_pingresp, 109
 - dummy, 110
- mqtt_response_puback, 110
 - packet_id, 111
- mqtt_response_pubcomp, 111
 - packet_id, 111
- mqtt_response_publish, 112
 - application_message, 112
 - application_message_size, 112
 - dup_flag, 113
 - packet_id, 113
 - qos_level, 113
 - retain_flag, 113
 - topic_name, 113
 - topic_name_size, 113
- mqtt_response_pubrec, 114
 - packet_id, 114
- mqtt_response_pubrel, 114
 - packet_id, 115
- mqtt_response_suback, 115
 - mqtt_unpack_suback_response, 116
 - num_return_codes, 116
 - packet_id, 116
 - return_codes, 116
- mqtt_response_unsuback, 117
 - mqtt_unpack_unsuback_response, 117
 - packet_id, 118
- mqtt_socket
 - mqtt_pub.c, 230
 - mqtt_pub.h, 236
- MQTT_SUBACK_FAILURE
 - Control Packet Deserialization, 30
- MQTT_SUBACK_SUCCESS_MAX_QOS_0
 - Control Packet Deserialization, 30
- MQTT_SUBACK_SUCCESS_MAX_QOS_1
 - Control Packet Deserialization, 30
- MQTT_SUBACK_SUCCESS_MAX_QOS_2
 - Control Packet Deserialization, 30
- mqtt_subscribe
 - API, 18
- MQTT_SUBSCRIBE_REQUEST_MAX_NUM_TOPICS
 - Control Packet Serialization, 21
- mqtt_sync
 - API, 19
- MQTT_TIMEOUT
 - raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c, 261
- mqtt_unpack_connack_response
 - Control Packet Deserialization, 30
- mqtt_unpack_fixed_header
 - Control Packet Deserialization, 31
- mqtt_unpack_publish_response
 - Control Packet Deserialization, 31
- mqtt_unpack_pubxxx_response
 - Control Packet Deserialization, 32
- mqtt_unpack_response
 - Control Packet Deserialization, 32
- mqtt_unpack_suback_response
 - mqtt_response_suback, 116
- mqtt_unpack_unsuback_response
 - mqtt_response_unsuback, 117
- mqtt_unsubscribe
 - API, 19
- MQTT_UNSUBSCRIBE_REQUEST_MAX_NUM_TOPICS
 - Control Packet Serialization, 21
- MQTT_VERSION
 - mqtt_pub.h, 234
 - raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c, 261
- MQTTConnackReturnCode
 - Control Packet Deserialization, 29
- MQTTConnectFlags
 - Control Packet Serialization, 22
- MQTTControlPacketType
 - Control Packet Deserialization, 29
- MQTTErrors
 - API, 10
- MQTTPublishFlags
 - Control Packet Serialization, 22
- MQTTQueuedMessageState
 - Utilities, 34

- MQTTSubackReturnCodes
 - Control Packet Deserialization, 30
- MREG, 118
 - bytes, 118
 - value, 118
- MREG32, 119
 - bytes, 119
 - value, 119
- MREG64, 119
 - bytes, 119
 - value, 119
- msg
 - canfd.c, 159
 - canfd.h, 169
- msgarrvd
 - raw.githubusercontent.com_linux-can_can-utils_master_canutils-263
- msgfields
 - CAN_MSG_FIELD, 79
- msgld
 - CAN_MSG_OBJ, 80
- MT
 - modbus_master.c, 688
 - modbus_master.h, 705
- mtoken
 - raw.githubusercontent.com_linux-can_can-utils_master_canutils-267
- mTouchContext
 - ADC3: Analog-to-Digital Converter with Context, 45
- mui
 - B_type, 70
- mutex
 - mqtt_client, 97
- mx_code
 - main.c, 281
- mx_log_packed_t, 120
 - a, 120
- mx_logpage_t, 120
 - absorb_time, 120
 - amp_hours, 120
 - amps_peak, 121
 - bat_max, 121
 - bat_min, 121
 - day, 121
 - float_time, 121
 - kilowatt_hours, 121
 - kilowatts_peak, 121
 - select, 121
 - type, 121
 - volts_peak, 121
- mx_status_packed_t, 122
 - a, 122
- mxcmd.c
 - calc_checksum, 662
 - FM_io, 663
 - FM_restart, 663
 - FM_rx, 663
 - FM_rx_count, 663
 - FM_rx_ready, 664
 - FM_tx, 664
 - FM_tx_empty, 664
 - lp_filter, 664
 - oneseq_io, 664
 - tenseq_io, 665
- mxcmd.h
 - AMP_WHOLE_ZERO, 667
 - B, 672
 - B_type, 669
 - BUFFER_SPACING, 667
 - build_version, 672
 - calc_checksum, 669
 - canbus_name, 672
 - cmd_battery, 672
 - cmd_batteryv, 673
 - CMD_CRC_LEN, 667
 - cmd_date, 673
 - cmd_fwrev, 673
 - cmd_fwrevb, 673
 - cmd_fwrevc, 673
 - cmd_id, 673
 - cmd_misc, 673
 - cmd_mx_log, 673
 - cmd_mx_status, 673
 - cmd_status, 673
 - cmd_statuselv, 673
 - cmd_status, 674
 - cmd_time, 674
 - cmd_watts, 674
 - DTG_LEN, 668
 - FM80_ID, 668
 - FM80_name, 674
 - FM_BUFFER, 668
 - FM_io, 669
 - FM_restart, 670
 - FM_rx, 670
 - FM_rx_count, 670
 - FM_rx_ready, 670
 - FM_tx, 670
 - FM_tx_empty, 671
 - FMxx_STATE, 668
 - IO_TEST, 668
 - LP_BUFFER_SIZE, 668
 - lp_filter, 671
 - MAX_B_BUF, 668
 - MAX_C_BUF, 668
 - modbus_name, 674
 - oneseq_io, 671
 - ONLINE_TIMEOUT, 668
 - SPINNER_SPEED, 668
 - state_name, 674
 - STATUS_ABSORB, 669
 - STATUS_BULK, 669
 - STATUS_EQUALIZE, 669
 - STATUS_FLOATING, 669
 - STATUS_LAST, 669
 - STATUS_SLEEPING, 669
 - status_type, 669

- tensec_io, 671
- VER, 669
- wtdelay, 672
- my_modbus_rx_32
 - modbus_master.c, 684
 - modbus_master.h, 704
- myVar
 - canfd.c, 160
- net_balance
 - B_type, 70
- net_ctx
 - mbdtdls_context, 94
- NHD
 - qconfig.h, 712
- NHD_BL_HIGH
 - eadog.h, 195
- NHD_BL_LOW
 - eadog.h, 195
- NHD_BL_MED
 - eadog.h, 195
- NHD_BL_OFF
 - eadog.h, 195
- NHD_CMD
 - eadog.h, 195
- NHD_CONT
 - eadog.h, 195
- NHD_L_DELAY
 - eadog.h, 195
- NHD_POS
 - eadog.h, 196
- NHD_S_DELAY
 - eadog.h, 196
- NHD_T_DELAY
 - eadog.h, 196
- NIGHT_RELAY_OFF
 - dio.h, 174
- NIGHT_RELAY_ON
 - dio.h, 174
- no_dma_set_lcd
 - eadog.c, 184
 - eadog.h, 202
- NO_NODE_ID
 - canfd.h, 164
- node_id
 - B_type, 70
- NODE_ID_SHIFT
 - canfd.h, 164
- NSB
 - eadog.h, 196
- NUM_OF_RX_FIFO
 - can1.c, 295
- num_return_codes
 - mqtt_response_suback, 116
- number_of_keep_alives
 - mqtt_client, 98
- number_of_timeouts
 - mqtt_client, 98

- oerr
 - uart1_status_t, 127
 - uart2_status_t, 128
 - uart5_status_t, 129
- OFF
 - modbus_master.c, 678
- ON
 - modbus_master.c, 678
- once
 - B_type, 70
- one_sec_flag
 - B_type, 70
- onesec_io
 - mxcmd.c, 664
 - mxcmd.h, 671
- ONLINE_TIMEOUT
 - mxcmd.h, 668
- open_nb_socket
 - bearssl_sockets.h, 244
 - bio_sockets.h, 246
 - mbdtdls_sockets.h, 249
 - openssl_sockets.h, 252
- open_nb_socket_mqtt
 - posix_sockets.h, 254
- openssl_sockets.h
 - open_nb_socket, 252
- operation
 - spi1_configuration_t, 124
- OSCILLATOR_Initialize
 - mcc.c, 379
 - mcc.h, 382
- OUTPUT
 - pin_manager.h, 438
- PACE
 - main.c, 273
- pacing
 - B_type, 70
 - V_type, 131
- packet_id
 - mqtt_queued_message, 104
 - mqtt_response_puback, 111
 - mqtt_response_pubcomp, 111
 - mqtt_response_publish, 113
 - mqtt_response_pubrec, 114
 - mqtt_response_pubrel, 115
 - mqtt_response_suback, 116
 - mqtt_response_unsuback, 118
- pamps
 - raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c, 267
- panel_watts
 - main.c, 281
 - qconfig.h, 712
- passwd_ok
 - C_data, 74
- perr
 - uart1_status_t, 127
 - uart2_status_t, 128

- uart5_status_t, [129](#)
- pfl1
 - EM_data1, [85](#)
- pfl2
 - EM_data1, [85](#)
- pfl3
 - EM_data1, [85](#)
- pfsys
 - EM_data1, [85](#)
- pge.h
 - E_DAYS, [242](#)
 - E_MONTH, [242](#)
 - E_PER_DAY, [242](#)
 - E_PER_HOUR, [242](#)
 - G_MONTH, [242](#)
 - G_PER_DAY, [242](#)
 - G_PER_HOUR, [242](#)
 - PGE_ZERO, [242](#)
- PGE_ZERO
 - pge.h, [242](#)
 - raw.githubusercontent.com_linux-can_can-utils_master_canutils-[261](#)
- phaseseq
 - EM_data1, [85](#)
- PIC18F47Q84-E_P/PADS/DIP1543W58P254L5175H635Q40N.6S, [710](#)
- PIC18F47Q84-E_P/PADS/PIC18F47Q84-E_P.c, [710](#)
- pid_lfsr
 - mqtt_client, [98](#)
- pin_manager.c
 - __interrupt, [387](#)
 - IOCAF1_DefaultInterruptHandler, [387](#)
 - IOCAF1_InterruptHandler, [391](#)
 - IOCAF1_ISR, [387](#)
 - IOCAF1_SetInterruptHandler, [388](#)
 - IOCAF2_DefaultInterruptHandler, [388](#)
 - IOCAF2_InterruptHandler, [391](#)
 - IOCAF2_ISR, [388](#)
 - IOCAF2_SetInterruptHandler, [389](#)
 - IOCAF5_DefaultInterruptHandler, [389](#)
 - IOCAF5_InterruptHandler, [392](#)
 - IOCAF5_ISR, [389](#)
 - IOCAF5_SetInterruptHandler, [390](#)
 - PIN_MANAGER_Initialize, [390](#)
- pin_manager.h
 - A_SWITCH_ANS, [402](#)
 - A_SWITCH_GetValue, [402](#)
 - A_SWITCH_LAT, [402](#)
 - A_SWITCH_OD, [402](#)
 - A_SWITCH_PORT, [403](#)
 - A_SWITCH_ResetPullup, [403](#)
 - A_SWITCH_SetAnalogMode, [403](#)
 - A_SWITCH_SetDigitalInput, [403](#)
 - A_SWITCH_SetDigitalMode, [403](#)
 - A_SWITCH_SetDigitalOutput, [403](#)
 - A_SWITCH_SetHigh, [403](#)
 - A_SWITCH_SetLow, [404](#)
 - A_SWITCH_SetOpenDrain, [404](#)
 - A_SWITCH_SetPullup, [404](#)
 - A_SWITCH_SetPushPull, [404](#)
 - A_SWITCH_Toggle, [404](#)
 - A_SWITCH_TRIS, [404](#)
 - A_SWITCH_WPU, [404](#)
 - ANALOG, [405](#)
 - CHARGER_ANS, [405](#)
 - CHARGER_GetValue, [405](#)
 - CHARGER_LAT, [405](#)
 - CHARGER_OD, [405](#)
 - CHARGER_PORT, [405](#)
 - CHARGER_ResetPullup, [405](#)
 - CHARGER_SetAnalogMode, [405](#)
 - CHARGER_SetDigitalInput, [405](#)
 - CHARGER_SetDigitalMode, [406](#)
 - CHARGER_SetDigitalOutput, [406](#)
 - CHARGER_SetHigh, [406](#)
 - CHARGER_SetLow, [406](#)
 - CHARGER_SetOpenDrain, [406](#)
 - CHARGER_SetPullup, [406](#)
 - CHARGER_SetPushPull, [406](#)
 - CHARGER_Toggle, [407](#)
 - CHARGER_TRIS, [407](#)
 - CHARGER_WPU, [407](#)
 - CS_ANS, [407](#)
 - CS_GetValue, [407](#)
 - CS_LAT, [407](#)
 - CS_OD, [407](#)
 - CS_PORT, [407](#)
 - CS_ResetPullup, [407](#)
 - CS_SetAnalogMode, [408](#)
 - CS_SetDigitalInput, [408](#)
 - CS_SetDigitalMode, [408](#)
 - CS_SetDigitalOutput, [408](#)
 - CS_SetHigh, [408](#)
 - CS_SetLow, [408](#)
 - CS_SetOpenDrain, [408](#)
 - CS_SetPullup, [409](#)
 - CS_SetPushPull, [409](#)
 - CS_Toggle, [409](#)
 - CS_TRIS, [409](#)
 - CS_WPU, [409](#)
 - DB0_ANS, [409](#)
 - DB0_GetValue, [409](#)
 - DB0_LAT, [409](#)
 - DB0_OD, [410](#)
 - DB0_PORT, [410](#)
 - DB0_ResetPullup, [410](#)
 - DB0_SetAnalogMode, [410](#)
 - DB0_SetDigitalInput, [410](#)
 - DB0_SetDigitalMode, [410](#)
 - DB0_SetDigitalOutput, [410](#)
 - DB0_SetHigh, [410](#)
 - DB0_SetLow, [411](#)
 - DB0_SetOpenDrain, [411](#)
 - DB0_SetPullup, [411](#)
 - DB0_SetPushPull, [411](#)
 - DB0_Toggle, [411](#)

DB0_TRIS, [411](#)
DB0_WPU, [411](#)
DB1_ANS, [412](#)
DB1_GetValue, [412](#)
DB1_LAT, [412](#)
DB1_OD, [412](#)
DB1_PORT, [412](#)
DB1_ResetPullup, [412](#)
DB1_SetAnalogMode, [412](#)
DB1_SetDigitalInput, [412](#)
DB1_SetDigitalMode, [413](#)
DB1_SetDigitalOutput, [413](#)
DB1_SetHigh, [413](#)
DB1_SetLow, [413](#)
DB1_SetOpenDrain, [413](#)
DB1_SetPullup, [413](#)
DB1_SetPushPull, [413](#)
DB1_Toggle, [414](#)
DB1_TRIS, [414](#)
DB1_WPU, [414](#)
DERE_ANS, [414](#)
DERE_GetValue, [414](#)
DERE_LAT, [414](#)
DERE_OD, [414](#)
DERE_PORT, [414](#)
DERE_ResetPullup, [414](#)
DERE_SetAnalogMode, [415](#)
DERE_SetDigitalInput, [415](#)
DERE_SetDigitalMode, [415](#)
DERE_SetDigitalOutput, [415](#)
DERE_SetHigh, [415](#)
DERE_SetLow, [415](#)
DERE_SetOpenDrain, [415](#)
DERE_SetPullup, [416](#)
DERE_SetPushPull, [416](#)
DERE_Toggle, [416](#)
DERE_TRIS, [416](#)
DERE_WPU, [416](#)
DIGITAL, [416](#)
DLED1_ANS, [416](#)
DLED1_GetValue, [416](#)
DLED1_LAT, [417](#)
DLED1_OD, [417](#)
DLED1_PORT, [417](#)
DLED1_ResetPullup, [417](#)
DLED1_SetAnalogMode, [417](#)
DLED1_SetDigitalInput, [417](#)
DLED1_SetDigitalMode, [417](#)
DLED1_SetDigitalOutput, [417](#)
DLED1_SetHigh, [418](#)
DLED1_SetLow, [418](#)
DLED1_SetOpenDrain, [418](#)
DLED1_SetPullup, [418](#)
DLED1_SetPushPull, [418](#)
DLED1_Toggle, [418](#)
DLED1_TRIS, [418](#)
DLED1_WPU, [419](#)
DLED_ANS, [419](#)
DLED_GetValue, [419](#)
DLED_LAT, [419](#)
DLED_OD, [419](#)
DLED_PORT, [419](#)
DLED_ResetPullup, [419](#)
DLED_SetAnalogMode, [419](#)
DLED_SetDigitalInput, [419](#)
DLED_SetDigitalMode, [420](#)
DLED_SetDigitalOutput, [420](#)
DLED_SetHigh, [420](#)
DLED_SetLow, [420](#)
DLED_SetOpenDrain, [420](#)
DLED_SetPullup, [420](#)
DLED_SetPushPull, [420](#)
DLED_Toggle, [421](#)
DLED_TRIS, [421](#)
DLED_WPU, [421](#)
HIGH, [421](#)
INPUT, [421](#)
INT_TRACE_ANS, [421](#)
INT_TRACE_GetValue, [421](#)
INT_TRACE_LAT, [422](#)
INT_TRACE_OD, [422](#)
INT_TRACE_PORT, [422](#)
INT_TRACE_ResetPullup, [422](#)
INT_TRACE_SetAnalogMode, [422](#)
INT_TRACE_SetDigitalInput, [422](#)
INT_TRACE_SetDigitalMode, [422](#)
INT_TRACE_SetDigitalOutput, [422](#)
INT_TRACE_SetHigh, [423](#)
INT_TRACE_SetLow, [423](#)
INT_TRACE_SetOpenDrain, [423](#)
INT_TRACE_SetPullup, [423](#)
INT_TRACE_SetPushPull, [423](#)
INT_TRACE_Toggle, [423](#)
INT_TRACE_TRIS, [423](#)
INT_TRACE_WPU, [424](#)
IO_RB5_ANS, [424](#)
IO_RB5_GetValue, [424](#)
IO_RB5_LAT, [424](#)
IO_RB5_OD, [424](#)
IO_RB5_PORT, [424](#)
IO_RB5_ResetPullup, [424](#)
IO_RB5_SetAnalogMode, [424](#)
IO_RB5_SetDigitalInput, [424](#)
IO_RB5_SetDigitalMode, [425](#)
IO_RB5_SetDigitalOutput, [425](#)
IO_RB5_SetHigh, [425](#)
IO_RB5_SetLow, [425](#)
IO_RB5_SetOpenDrain, [425](#)
IO_RB5_SetPullup, [425](#)
IO_RB5_SetPushPull, [425](#)
IO_RB5_Toggle, [426](#)
IO_RB5_TRIS, [426](#)
IO_RB5_WPU, [426](#)
IO_RD7_ANS, [426](#)
IO_RD7_GetValue, [426](#)
IO_RD7_LAT, [426](#)

IO_RD7_OD, [426](#)
IO_RD7_PORT, [426](#)
IO_RD7_ResetPullup, [426](#)
IO_RD7_SetAnalogMode, [427](#)
IO_RD7_SetDigitalInput, [427](#)
IO_RD7_SetDigitalMode, [427](#)
IO_RD7_SetDigitalOutput, [427](#)
IO_RD7_SetHigh, [427](#)
IO_RD7_SetLow, [427](#)
IO_RD7_SetOpenDrain, [427](#)
IO_RD7_SetPullup, [428](#)
IO_RD7_SetPushPull, [428](#)
IO_RD7_Toggle, [428](#)
IO_RD7_TRIS, [428](#)
IO_RD7_WPU, [428](#)
IOCAF1_DefaultInterruptHandler, [467](#)
IOCAF1_InterruptHandler, [472](#)
IOCAF1_ISR, [467](#)
IOCAF1_SetInterruptHandler, [467](#)
IOCAF2_DefaultInterruptHandler, [468](#)
IOCAF2_InterruptHandler, [473](#)
IOCAF2_ISR, [468](#)
IOCAF2_SetInterruptHandler, [469](#)
IOCAF5_DefaultInterruptHandler, [469](#)
IOCAF5_InterruptHandler, [473](#)
IOCAF5_ISR, [470](#)
IOCAF5_SetInterruptHandler, [470](#)
L_SWITCH_ANS, [428](#)
L_SWITCH_GetValue, [428](#)
L_SWITCH_LAT, [428](#)
L_SWITCH_OD, [429](#)
L_SWITCH_PORT, [429](#)
L_SWITCH_ResetPullup, [429](#)
L_SWITCH_SetAnalogMode, [429](#)
L_SWITCH_SetDigitalInput, [429](#)
L_SWITCH_SetDigitalMode, [429](#)
L_SWITCH_SetDigitalOutput, [429](#)
L_SWITCH_SetHigh, [429](#)
L_SWITCH_SetLow, [430](#)
L_SWITCH_SetOpenDrain, [430](#)
L_SWITCH_SetPullup, [430](#)
L_SWITCH_SetPushPull, [430](#)
L_SWITCH_Toggle, [430](#)
L_SWITCH_TRIS, [430](#)
L_SWITCH_WPU, [430](#)
LOW, [431](#)
M_SWITCH_ANS, [431](#)
M_SWITCH_GetValue, [431](#)
M_SWITCH_LAT, [431](#)
M_SWITCH_OD, [431](#)
M_SWITCH_PORT, [431](#)
M_SWITCH_ResetPullup, [431](#)
M_SWITCH_SetAnalogMode, [431](#)
M_SWITCH_SetDigitalInput, [431](#)
M_SWITCH_SetDigitalMode, [432](#)
M_SWITCH_SetDigitalOutput, [432](#)
M_SWITCH_SetHigh, [432](#)
M_SWITCH_SetLow, [432](#)
M_SWITCH_SetOpenDrain, [432](#)
M_SWITCH_SetPullup, [432](#)
M_SWITCH_SetPushPull, [432](#)
M_SWITCH_Toggle, [433](#)
M_SWITCH_TRIS, [433](#)
M_SWITCH_WPU, [433](#)
MISC_ANS, [433](#)
MISC_GetValue, [433](#)
MISC_LAT, [433](#)
MISC_OD, [433](#)
MISC_PORT, [433](#)
MISC_ResetPullup, [433](#)
MISC_SetAnalogMode, [434](#)
MISC_SetDigitalInput, [434](#)
MISC_SetDigitalMode, [434](#)
MISC_SetDigitalOutput, [434](#)
MISC_SetHigh, [434](#)
MISC_SetLow, [434](#)
MISC_SetOpenDrain, [434](#)
MISC_SetPullup, [435](#)
MISC_SetPushPull, [435](#)
MISC_Toggle, [435](#)
MISC_TRIS, [435](#)
MISC_WPU, [435](#)
MLED_ANS, [435](#)
MLED_GetValue, [435](#)
MLED_LAT, [435](#)
MLED_OD, [436](#)
MLED_PORT, [436](#)
MLED_ResetPullup, [436](#)
MLED_SetAnalogMode, [436](#)
MLED_SetDigitalInput, [436](#)
MLED_SetDigitalMode, [436](#)
MLED_SetDigitalOutput, [436](#)
MLED_SetHigh, [436](#)
MLED_SetLow, [437](#)
MLED_SetOpenDrain, [437](#)
MLED_SetPullup, [437](#)
MLED_SetPushPull, [437](#)
MLED_Toggle, [437](#)
MLED_TRIS, [437](#)
MLED_WPU, [437](#)
OUTPUT, [438](#)
PIN_MANAGER_Initialize, [471](#)
PROG_TRACE_ANS, [438](#)
PROG_TRACE_GetValue, [438](#)
PROG_TRACE_LAT, [438](#)
PROG_TRACE_OD, [438](#)
PROG_TRACE_PORT, [438](#)
PROG_TRACE_ResetPullup, [438](#)
PROG_TRACE_SetAnalogMode, [438](#)
PROG_TRACE_SetDigitalInput, [438](#)
PROG_TRACE_SetDigitalMode, [439](#)
PROG_TRACE_SetDigitalOutput, [439](#)
PROG_TRACE_SetHigh, [439](#)
PROG_TRACE_SetLow, [439](#)
PROG_TRACE_SetOpenDrain, [439](#)
PROG_TRACE_SetPullup, [439](#)

PROG_TRACE_SetPushPull, [439](#)
PROG_TRACE_Toggle, [440](#)
PROG_TRACE_TRIS, [440](#)
PROG_TRACE_WPU, [440](#)
PULL_UP_DISABLED, [440](#)
PULL_UP_ENABLED, [440](#)
RA0_GetValue, [440](#)
RA0_ResetPullup, [440](#)
RA0_SetAnalogMode, [440](#)
RA0_SetDigitalInput, [441](#)
RA0_SetDigitalMode, [441](#)
RA0_SetDigitalOutput, [441](#)
RA0_SetHigh, [441](#)
RA0_SetLow, [441](#)
RA0_SetPullup, [441](#)
RA0_Toggle, [441](#)
RA3_GetValue, [442](#)
RA3_ResetPullup, [442](#)
RA3_SetAnalogMode, [442](#)
RA3_SetDigitalInput, [442](#)
RA3_SetDigitalMode, [442](#)
RA3_SetDigitalOutput, [442](#)
RA3_SetHigh, [442](#)
RA3_SetLow, [443](#)
RA3_SetPullup, [443](#)
RA3_Toggle, [443](#)
RA4_GetValue, [443](#)
RA4_ResetPullup, [443](#)
RA4_SetAnalogMode, [443](#)
RA4_SetDigitalInput, [443](#)
RA4_SetDigitalMode, [444](#)
RA4_SetDigitalOutput, [444](#)
RA4_SetHigh, [444](#)
RA4_SetLow, [444](#)
RA4_SetPullup, [444](#)
RA4_Toggle, [444](#)
RB0_GetValue, [444](#)
RB0_ResetPullup, [445](#)
RB0_SetAnalogMode, [445](#)
RB0_SetDigitalInput, [445](#)
RB0_SetDigitalMode, [445](#)
RB0_SetDigitalOutput, [445](#)
RB0_SetHigh, [445](#)
RB0_SetLow, [445](#)
RB0_SetPullup, [446](#)
RB0_Toggle, [446](#)
RB4_GetValue, [446](#)
RB4_ResetPullup, [446](#)
RB4_SetAnalogMode, [446](#)
RB4_SetDigitalInput, [446](#)
RB4_SetDigitalMode, [446](#)
RB4_SetDigitalOutput, [447](#)
RB4_SetHigh, [447](#)
RB4_SetLow, [447](#)
RB4_SetPullup, [447](#)
RB4_Toggle, [447](#)
RB7_GetValue, [447](#)
RB7_ResetPullup, [447](#)
RB7_SetAnalogMode, [448](#)
RB7_SetDigitalInput, [448](#)
RB7_SetDigitalMode, [448](#)
RB7_SetDigitalOutput, [448](#)
RB7_SetHigh, [448](#)
RB7_SetLow, [448](#)
RB7_SetPullup, [448](#)
RB7_Toggle, [449](#)
RC0_GetValue, [449](#)
RC0_ResetPullup, [449](#)
RC0_SetAnalogMode, [449](#)
RC0_SetDigitalInput, [449](#)
RC0_SetDigitalMode, [449](#)
RC0_SetDigitalOutput, [449](#)
RC0_SetHigh, [450](#)
RC0_SetLow, [450](#)
RC0_SetPullup, [450](#)
RC0_Toggle, [450](#)
RC1_GetValue, [450](#)
RC1_ResetPullup, [450](#)
RC1_SetAnalogMode, [450](#)
RC1_SetDigitalInput, [451](#)
RC1_SetDigitalMode, [451](#)
RC1_SetDigitalOutput, [451](#)
RC1_SetHigh, [451](#)
RC1_SetLow, [451](#)
RC1_SetPullup, [451](#)
RC1_Toggle, [451](#)
RC3_GetValue, [452](#)
RC3_ResetPullup, [452](#)
RC3_SetAnalogMode, [452](#)
RC3_SetDigitalInput, [452](#)
RC3_SetDigitalMode, [452](#)
RC3_SetDigitalOutput, [452](#)
RC3_SetHigh, [452](#)
RC3_SetLow, [453](#)
RC3_SetPullup, [453](#)
RC3_Toggle, [453](#)
RC4_GetValue, [453](#)
RC4_ResetPullup, [453](#)
RC4_SetAnalogMode, [453](#)
RC4_SetDigitalInput, [453](#)
RC4_SetDigitalMode, [454](#)
RC4_SetDigitalOutput, [454](#)
RC4_SetHigh, [454](#)
RC4_SetLow, [454](#)
RC4_SetPullup, [454](#)
RC4_Toggle, [454](#)
RC5_GetValue, [454](#)
RC5_ResetPullup, [455](#)
RC5_SetAnalogMode, [455](#)
RC5_SetDigitalInput, [455](#)
RC5_SetDigitalMode, [455](#)
RC5_SetDigitalOutput, [455](#)
RC5_SetHigh, [455](#)
RC5_SetLow, [455](#)
RC5_SetPullup, [456](#)
RC5_Toggle, [456](#)

- RC6_GetValue, [456](#)
- RC6_ResetPullup, [456](#)
- RC6_SetAnalogMode, [456](#)
- RC6_SetDigitalInput, [456](#)
- RC6_SetDigitalMode, [456](#)
- RC6_SetDigitalOutput, [457](#)
- RC6_SetHigh, [457](#)
- RC6_SetLow, [457](#)
- RC6_SetPullup, [457](#)
- RC6_Toggle, [457](#)
- RC7_GetValue, [457](#)
- RC7_ResetPullup, [457](#)
- RC7_SetAnalogMode, [458](#)
- RC7_SetDigitalInput, [458](#)
- RC7_SetDigitalMode, [458](#)
- RC7_SetDigitalOutput, [458](#)
- RC7_SetHigh, [458](#)
- RC7_SetLow, [458](#)
- RC7_SetPullup, [458](#)
- RC7_Toggle, [459](#)
- RD0_GetValue, [459](#)
- RD0_ResetPullup, [459](#)
- RD0_SetAnalogMode, [459](#)
- RD0_SetDigitalInput, [459](#)
- RD0_SetDigitalMode, [459](#)
- RD0_SetDigitalOutput, [459](#)
- RD0_SetHigh, [460](#)
- RD0_SetLow, [460](#)
- RD0_SetPullup, [460](#)
- RD0_Toggle, [460](#)
- RD6_GetValue, [460](#)
- RD6_ResetPullup, [460](#)
- RD6_SetAnalogMode, [460](#)
- RD6_SetDigitalInput, [461](#)
- RD6_SetDigitalMode, [461](#)
- RD6_SetDigitalOutput, [461](#)
- RD6_SetHigh, [461](#)
- RD6_SetLow, [461](#)
- RD6_SetPullup, [461](#)
- RD6_Toggle, [461](#)
- RELAY_ANS, [462](#)
- RELAY_GetValue, [462](#)
- RELAY_LAT, [462](#)
- RELAY_OD, [462](#)
- RELAY_PORT, [462](#)
- RELAY_ResetPullup, [462](#)
- RELAY_SetAnalogMode, [462](#)
- RELAY_SetDigitalInput, [462](#)
- RELAY_SetDigitalMode, [463](#)
- RELAY_SetDigitalOutput, [463](#)
- RELAY_SetHigh, [463](#)
- RELAY_SetLow, [463](#)
- RELAY_SetOpenDrain, [463](#)
- RELAY_SetPullup, [463](#)
- RELAY_SetPushPull, [463](#)
- RELAY_Toggle, [464](#)
- RELAY_TRIS, [464](#)
- RELAY_WPU, [464](#)
- RLED_ANS, [464](#)
- RLED_GetValue, [464](#)
- RLED_LAT, [464](#)
- RLED_OD, [464](#)
- RLED_PORT, [464](#)
- RLED_ResetPullup, [464](#)
- RLED_SetAnalogMode, [465](#)
- RLED_SetDigitalInput, [465](#)
- RLED_SetDigitalMode, [465](#)
- RLED_SetDigitalOutput, [465](#)
- RLED_SetHigh, [465](#)
- RLED_SetLow, [465](#)
- RLED_SetOpenDrain, [465](#)
- RLED_SetPullup, [466](#)
- RLED_SetPushPull, [466](#)
- RLED_Toggle, [466](#)
- RLED_TRIS, [466](#)
- RLED_WPU, [466](#)
- PIN_MANAGER_Initialize
 - pin_manager.c, [390](#)
 - pin_manager.h, [471](#)
- pingresp
 - mqtt_response, [107](#)
- Platform abstraction layer, [39](#)
 - mqtt_pal_recvall, [40](#)
 - mqtt_pal_sendall, [41](#)
- PLSIZEToPayloadBytes
 - can1.c, [295](#)
- PMD_Initialize
 - mcc.c, [379](#)
 - mcc.h, [382](#)
- port
 - mqtt_pub.c, [232](#)
- posix_sockets.h
 - open_nb_socket_mqtt, [254](#)
- power_on
 - M_data, [92](#)
- PROG_TRACE_ANS
 - pin_manager.h, [438](#)
- PROG_TRACE_GetValue
 - pin_manager.h, [438](#)
- PROG_TRACE_LAT
 - pin_manager.h, [438](#)
- PROG_TRACE_OD
 - pin_manager.h, [438](#)
- PROG_TRACE_PORT
 - pin_manager.h, [438](#)
- PROG_TRACE_ResetPullup
 - pin_manager.h, [438](#)
- PROG_TRACE_SetAnalogMode
 - pin_manager.h, [438](#)
- PROG_TRACE_SetDigitalInput
 - pin_manager.h, [438](#)
- PROG_TRACE_SetDigitalMode
 - pin_manager.h, [439](#)
- PROG_TRACE_SetDigitalOutput
 - pin_manager.h, [439](#)
- PROG_TRACE_SetHigh

- pin_manager.h, [439](#)
- PROG_TRACE_SetLow
 - pin_manager.h, [439](#)
- PROG_TRACE_SetOpenDrain
 - pin_manager.h, [439](#)
- PROG_TRACE_SetPullup
 - pin_manager.h, [439](#)
- PROG_TRACE_SetPushPull
 - pin_manager.h, [439](#)
- PROG_TRACE_Toggle
 - pin_manager.h, [440](#)
- PROG_TRACE_TRIS
 - pin_manager.h, [440](#)
- PROG_TRACE_WPU
 - pin_manager.h, [440](#)
- puback
 - mqtt_response, [107](#)
- pubcomp
 - mqtt_response, [107](#)
- publish
 - mqtt_response, [108](#)
- publish_callback
 - main.c, [270](#)
 - mqtt_pub.c, [231](#)
 - mqtt_pub.h, [237](#)
- publish_response_callback
 - mqtt_client, [98](#)
- publish_response_callback_state
 - mqtt_client, [98](#)
- pubmsg
 - raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c, [267](#)
- pubrec
 - mqtt_response, [108](#)
- pubrel
 - mqtt_response, [108](#)
- PULL_UP_DISABLED
 - pin_manager.h, [440](#)
- PULL_UP_ENABLED
 - pin_manager.h, [440](#)
- putch
 - uart2.c, [617](#)
- pv_high
 - B_type, [70](#)
- pv_prev
 - B_type, [70](#)
- pv_update
 - B_type, [71](#)
- pv_Wh_daily
 - batmon.c, [139](#)
 - batmon.h, [150](#)
- pv_Wh_daily_prev
 - batmon.c, [139](#)
 - batmon.h, [150](#)
- pvolts
 - raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c, [267](#)
- pwatts
 - raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c, [267](#)
- pwm_stop
 - V_type, [131](#)
- pwm_update
 - V_type, [131](#)
- pwm_volts
 - M_data, [92](#)
- PWMDC, [122](#)
 - bpwm, [122](#)
 - lpwm, [122](#)
- q84_sequence
 - EB_data, [83](#)
- qconfig.h, [710](#)
 - bat_amp_whole, [712](#)
 - cc_mode, [712](#)
 - INT_TRACE, [711](#)
 - MAX_SHAPES, [711](#)
 - NHD, [712](#)
 - panel_watts, [712](#)
 - spin, [712](#)
 - SPIN_VAL_UPDATE, [712](#)
 - spinners, [712](#)
 - volt_fract, [713](#)
 - volt_whole, [713](#)
 - vw, [713](#)
- QDELAY
 - modbus_master.h, [693](#)
- QOS
 - raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c, [261](#)
- qos_level
 - mqtt_response_publish, [113](#)
- queue_tail
 - mqtt_message_queue, [103](#)
- RA0_GetValue
 - pin_manager.h, [440](#)
- RA0_ResetPullup
 - pin_manager.h, [440](#)
- RA0_SetAnalogMode
 - pin_manager.h, [440](#)
- RA0_SetDigitalInput
 - pin_manager.h, [441](#)
- RA0_SetDigitalMode
 - pin_manager.h, [441](#)
- RA0_SetDigitalOutput
 - pin_manager.h, [441](#)
- RA0_SetHigh
 - pin_manager.h, [441](#)
- RA0_SetLow
 - pin_manager.h, [441](#)
- RA0_SetPullup
 - pin_manager.h, [441](#)
- RA0_Toggle
 - pin_manager.h, [441](#)
- RA3_GetValue
 - pin_manager.h, [442](#)

RA3_ResetPullup
 pin_manager.h, 442
 RA3_SetAnalogMode
 pin_manager.h, 442
 RA3_SetDigitalInput
 pin_manager.h, 442
 RA3_SetDigitalMode
 pin_manager.h, 442
 RA3_SetDigitalOutput
 pin_manager.h, 442
 RA3_SetHigh
 pin_manager.h, 442
 RA3_SetLow
 pin_manager.h, 443
 RA3_SetPullup
 pin_manager.h, 443
 RA3_Toggle
 pin_manager.h, 443
 RA4_GetValue
 pin_manager.h, 443
 RA4_ResetPullup
 pin_manager.h, 443
 RA4_SetAnalogMode
 pin_manager.h, 443
 RA4_SetDigitalInput
 pin_manager.h, 443
 RA4_SetDigitalMode
 pin_manager.h, 444
 RA4_SetDigitalOutput
 pin_manager.h, 444
 RA4_SetHigh
 pin_manager.h, 444
 RA4_SetLow
 pin_manager.h, 444
 RA4_SetPullup
 pin_manager.h, 444
 RA4_Toggle
 pin_manager.h, 444
 raw.githubusercontent.com_linux-can_can-utils_master_canfdtest
 _DEFAULT_SOURCE, 258
 acamps, 264
 acenergy, 264
 acvolts, 265
 ADDRESS, 258
 bamps, 265
 bat_energy_kw, 265
 bat_energy_scaled, 265
 benergy, 265
 bvols, 265
 CAN_FULL_BUFFER, 258
 CAN_MSG_COUNT, 258
 CAN_MSG_ID_PING, 258
 CAN_MSG_ID_PING_X, 258
 CAN_MSG_ID_PONG, 258
 CAN_MSG_LEN, 258
 CAN_MSG_WAIT, 259
 CAN_TM_TIME, 259
 ccmode, 265
 client, 265
 CLIENTID, 259
 conn_opts, 265
 connlost, 262
 current_timestamp, 262
 data_buffer, 265
 DAY_SEC, 259
 day_time, 266
 delivered, 262
 deliveredtoken, 266
 E_DAYS, 259
 E_MONTH, 259
 E_PER_DAY, 259
 E_PER_HOUR, 259
 EMON_CO, 259
 EMON_DA, 259
 EMON_ER, 260
 EMON_SH, 260
 EMON_SL, 260
 EMON_SU, 260
 EMON_TM, 260
 fout, 266
 full_buffer, 266
 G_MONTH, 260
 G_PER_DAY, 260
 G_PER_HOUR, 260
 gasenergy, 266
 gridin, 266
 gridout, 266
 hour_time, 266
 HR_SEC, 260
 json, 266
 load, 266
 log_time, 262
 LOG_TO_FILE, 260
 LOG_VERSION, 261
 main, 263
 MQTT_TIMEOUT, 261
 MQTT_VERSION, 261
 msgarrvd, 263
 mtoken, 267
 pamps, 267
 PGE_ZERO, 261
 pubmsg, 267
 pvols, 267
 pwatts, 267
 QOS, 261
 receivedtoken, 267
 runner, 267
 runtime, 267
 sec_30, 267
 sequence, 267
 solar, 268
 SPACING_USEC, 261
 start_time, 268
 TIMEOUT, 261
 timer_callback, 264
 token, 268

TOPIC_P, [261](#)
 TOPIC_S, [261](#)
 watrgal, [268](#)
 RB0_GetValue
 pin_manager.h, [444](#)
 RB0_ResetPullup
 pin_manager.h, [445](#)
 RB0_SetAnalogMode
 pin_manager.h, [445](#)
 RB0_SetDigitalInput
 pin_manager.h, [445](#)
 RB0_SetDigitalMode
 pin_manager.h, [445](#)
 RB0_SetDigitalOutput
 pin_manager.h, [445](#)
 RB0_SetHigh
 pin_manager.h, [445](#)
 RB0_SetLow
 pin_manager.h, [445](#)
 RB0_SetPullup
 pin_manager.h, [446](#)
 RB0_Toggle
 pin_manager.h, [446](#)
 RB4_GetValue
 pin_manager.h, [446](#)
 RB4_ResetPullup
 pin_manager.h, [446](#)
 RB4_SetAnalogMode
 pin_manager.h, [446](#)
 RB4_SetDigitalInput
 pin_manager.h, [446](#)
 RB4_SetDigitalMode
 pin_manager.h, [446](#)
 RB4_SetDigitalOutput
 pin_manager.h, [447](#)
 RB4_SetHigh
 pin_manager.h, [447](#)
 RB4_SetLow
 pin_manager.h, [447](#)
 RB4_SetPullup
 pin_manager.h, [447](#)
 RB4_Toggle
 pin_manager.h, [447](#)
 RB7_GetValue
 pin_manager.h, [447](#)
 RB7_ResetPullup
 pin_manager.h, [447](#)
 RB7_SetAnalogMode
 pin_manager.h, [448](#)
 RB7_SetDigitalInput
 pin_manager.h, [448](#)
 RB7_SetDigitalMode
 pin_manager.h, [448](#)
 RB7_SetDigitalOutput
 pin_manager.h, [448](#)
 RB7_SetHigh
 pin_manager.h, [448](#)
 RB7_SetLow
 pin_manager.h, [448](#)
 RB7_SetPullup
 pin_manager.h, [448](#)
 RB7_Toggle
 pin_manager.h, [449](#)
 RBUF_SIZE
 ringbufs.h, [719](#)
 RC0_GetValue
 pin_manager.h, [449](#)
 RC0_ResetPullup
 pin_manager.h, [449](#)
 RC0_SetAnalogMode
 pin_manager.h, [449](#)
 RC0_SetDigitalInput
 pin_manager.h, [449](#)
 RC0_SetDigitalMode
 pin_manager.h, [449](#)
 RC0_SetDigitalOutput
 pin_manager.h, [449](#)
 RC0_SetHigh
 pin_manager.h, [450](#)
 RC0_SetLow
 pin_manager.h, [450](#)
 RC0_SetPullup
 pin_manager.h, [450](#)
 RC0_Toggle
 pin_manager.h, [450](#)
 RC1_GetValue
 pin_manager.h, [450](#)
 RC1_ResetPullup
 pin_manager.h, [450](#)
 RC1_SetAnalogMode
 pin_manager.h, [450](#)
 RC1_SetDigitalInput
 pin_manager.h, [451](#)
 RC1_SetDigitalMode
 pin_manager.h, [451](#)
 RC1_SetDigitalOutput
 pin_manager.h, [451](#)
 RC1_SetHigh
 pin_manager.h, [451](#)
 RC1_SetLow
 pin_manager.h, [451](#)
 RC1_SetPullup
 pin_manager.h, [451](#)
 RC1_Toggle
 pin_manager.h, [451](#)
 RC3_GetValue
 pin_manager.h, [452](#)
 RC3_ResetPullup
 pin_manager.h, [452](#)
 RC3_SetAnalogMode
 pin_manager.h, [452](#)
 RC3_SetDigitalInput
 pin_manager.h, [452](#)
 RC3_SetDigitalMode
 pin_manager.h, [452](#)
 RC3_SetDigitalOutput

pin_manager.h, [452](#)
RC3_SetHigh
pin_manager.h, [452](#)
RC3_SetLow
pin_manager.h, [453](#)
RC3_SetPullup
pin_manager.h, [453](#)
RC3_Toggle
pin_manager.h, [453](#)
RC4_GetValue
pin_manager.h, [453](#)
RC4_ResetPullup
pin_manager.h, [453](#)
RC4_SetAnalogMode
pin_manager.h, [453](#)
RC4_SetDigitalInput
pin_manager.h, [453](#)
RC4_SetDigitalMode
pin_manager.h, [454](#)
RC4_SetDigitalOutput
pin_manager.h, [454](#)
RC4_SetHigh
pin_manager.h, [454](#)
RC4_SetLow
pin_manager.h, [454](#)
RC4_SetPullup
pin_manager.h, [454](#)
RC4_Toggle
pin_manager.h, [454](#)
RC5_GetValue
pin_manager.h, [454](#)
RC5_ResetPullup
pin_manager.h, [455](#)
RC5_SetAnalogMode
pin_manager.h, [455](#)
RC5_SetDigitalInput
pin_manager.h, [455](#)
RC5_SetDigitalMode
pin_manager.h, [455](#)
RC5_SetDigitalOutput
pin_manager.h, [455](#)
RC5_SetHigh
pin_manager.h, [455](#)
RC5_SetLow
pin_manager.h, [455](#)
RC5_SetPullup
pin_manager.h, [456](#)
RC5_Toggle
pin_manager.h, [456](#)
RC6_GetValue
pin_manager.h, [456](#)
RC6_ResetPullup
pin_manager.h, [456](#)
RC6_SetAnalogMode
pin_manager.h, [456](#)
RC6_SetDigitalInput
pin_manager.h, [456](#)
RC6_SetDigitalMode
pin_manager.h, [456](#)
RC6_SetDigitalOutput
pin_manager.h, [457](#)
RC6_SetHigh
pin_manager.h, [457](#)
RC6_SetLow
pin_manager.h, [457](#)
RC6_SetPullup
pin_manager.h, [457](#)
RC6_Toggle
pin_manager.h, [457](#)
RC7_GetValue
pin_manager.h, [457](#)
RC7_ResetPullup
pin_manager.h, [457](#)
RC7_SetAnalogMode
pin_manager.h, [458](#)
RC7_SetDigitalInput
pin_manager.h, [458](#)
RC7_SetDigitalMode
pin_manager.h, [458](#)
RC7_SetDigitalOutput
pin_manager.h, [458](#)
RC7_SetHigh
pin_manager.h, [458](#)
RC7_SetLow
pin_manager.h, [458](#)
RC7_SetPullup
pin_manager.h, [458](#)
RC7_Toggle
pin_manager.h, [459](#)
RD0_GetValue
pin_manager.h, [459](#)
RD0_ResetPullup
pin_manager.h, [459](#)
RD0_SetAnalogMode
pin_manager.h, [459](#)
RD0_SetDigitalInput
pin_manager.h, [459](#)
RD0_SetDigitalMode
pin_manager.h, [459](#)
RD0_SetDigitalOutput
pin_manager.h, [459](#)
RD0_SetHigh
pin_manager.h, [460](#)
RD0_SetLow
pin_manager.h, [460](#)
RD0_SetPullup
pin_manager.h, [460](#)
RD0_Toggle
pin_manager.h, [460](#)
RD6_GetValue
pin_manager.h, [460](#)
RD6_ResetPullup
pin_manager.h, [460](#)
RD6_SetAnalogMode
pin_manager.h, [460](#)
RD6_SetDigitalInput

- pin_manager.h, 461
- RD6_SetDigitalMode
 - pin_manager.h, 461
- RD6_SetDigitalOutput
 - pin_manager.h, 461
- RD6_SetHigh
 - pin_manager.h, 461
- RD6_SetLow
 - pin_manager.h, 461
- RD6_SetPullup
 - pin_manager.h, 461
- RD6_Toggle
 - pin_manager.h, 461
- RDELAY
 - modbus_master.h, 693
- READ_COILS
 - modbus_master.h, 693
- READ_DISCRETE_INPUTS
 - modbus_master.h, 693
- READ_HOLDING_REGISTERS
 - modbus_master.h, 694
- READ_INPUT_REGISTERS
 - modbus_master.h, 694
- rec_count
 - can_frames_t, 77
 - can_rec_count_t, 80
- rec_flag
 - can_rec_count_t, 80
- REC_LEN
 - main.c, 273
- REC_LOG_LEN
 - main.c, 273
- REC_STATUS_LEN
 - main.c, 273
- receivedtoken
 - raw.githubusercontent.com/linux-can/can-utils/master/compat/linuxcan.h, 267
- reconnect_callback
 - mqtt_client, 99
- reconnect_state
 - mqtt_client, 99
- RECV
 - modbus_master.h, 697
- recv_buffer
 - mqtt_client, 99
- recv_count
 - M_data, 92
- recvbuf
 - mqtt_pub.c, 232
- RELAY_ANS
 - pin_manager.h, 462
- RELAY_GetValue
 - pin_manager.h, 462
- RELAY_LAT
 - pin_manager.h, 462
- RELAY_OD
 - pin_manager.h, 462
- RELAY_PORT
 - pin_manager.h, 462
- RELAY_ResetPullup
 - pin_manager.h, 462
- RELAY_SetAnalogMode
 - pin_manager.h, 462
- RELAY_SetDigitalInput
 - pin_manager.h, 462
- RELAY_SetDigitalMode
 - pin_manager.h, 463
- RELAY_SetDigitalOutput
 - pin_manager.h, 463
- RELAY_SetHigh
 - pin_manager.h, 463
- RELAY_SetLow
 - pin_manager.h, 463
- RELAY_SetOpenDrain
 - pin_manager.h, 463
- RELAY_SetPullup
 - pin_manager.h, 463
- RELAY_SetPushPull
 - pin_manager.h, 463
- RELAY_Toggle
 - pin_manager.h, 464
- RELAY_TRIS
 - pin_manager.h, 464
- RELAY_WPU
 - pin_manager.h, 464
- remaining_length
 - mqtt_fixed_header, 101
- req_length
 - C_data, 74
- reserved
 - uart1_status_t, 127
 - uart2_status_t, 128
 - uart5_status_t, 129
- response
 - mqtt_client, 99
- retain_flag
 - mqtt_response_publish, 113
- return_code
 - mqtt_response_connack, 109
- return_codes
 - mqtt_response_suback, 116
- rhlc
 - EM_data2, 89
- rhm
 - EM_data2, 89
- rhmk
 - EM_data2, 89
- rhmkp
 - EM_data2, 89
- rhmp
 - EM_data2, 89
- ringbufs.c, 714
 - modulo_dec, 715
 - modulo_inc, 715
 - ringBufS_empty, 716
 - ringBufS_flush, 716

- ringBufS_full, [716](#)
- ringBufS_get, [716](#)
- ringBufS_init, [716](#)
- ringBufS_put, [717](#)
- ringBufS_put_dma, [717](#)
- ringBufS_put_dma_cpy, [717](#)
- ringbufs.h, [717](#)
 - RBUF_SIZE, [719](#)
 - ringBufS_empty, [719](#)
 - ringBufS_flush, [719](#)
 - ringBufS_full, [719](#)
 - ringBufS_get, [719](#)
 - ringBufS_init, [719](#)
 - ringBufS_put, [720](#)
 - ringBufS_put_dma, [720](#)
 - ringBufS_put_dma_cpy, [720](#)
 - ringBufS_t, [719](#)
- ringBufS_empty
 - ringbufs.c, [716](#)
 - ringbufs.h, [719](#)
- ringBufS_flush
 - ringbufs.c, [716](#)
 - ringbufs.h, [719](#)
- ringBufS_full
 - ringbufs.c, [716](#)
 - ringbufs.h, [719](#)
- ringBufS_get
 - ringbufs.c, [716](#)
 - ringbufs.h, [719](#)
- ringBufS_init
 - ringbufs.c, [716](#)
 - ringbufs.h, [719](#)
- ringBufS_put
 - ringbufs.c, [717](#)
 - ringbufs.h, [720](#)
- ringBufS_put_dma
 - ringbufs.c, [717](#)
 - ringbufs.h, [720](#)
- ringBufS_put_dma_cpy
 - ringbufs.c, [717](#)
 - ringbufs.h, [720](#)
- ringBufS_t, [123](#)
 - buf, [123](#)
 - count, [123](#)
 - head, [123](#)
 - ringbufs.h, [719](#)
 - tail, [123](#)
- RLED_ANS
 - pin_manager.h, [464](#)
- RLED_GetValue
 - pin_manager.h, [464](#)
- RLED_LAT
 - pin_manager.h, [464](#)
- RLED_OD
 - pin_manager.h, [464](#)
- RLED_PORT
 - pin_manager.h, [464](#)
- RLED_ResetPullup
 - pin_manager.h, [464](#)
- RLED_SetAnalogMode
 - pin_manager.h, [465](#)
- RLED_SetDigitalInput
 - pin_manager.h, [465](#)
- RLED_SetDigitalMode
 - pin_manager.h, [465](#)
- RLED_SetDigitalOutput
 - pin_manager.h, [465](#)
- RLED_SetHigh
 - pin_manager.h, [465](#)
- RLED_SetLow
 - pin_manager.h, [465](#)
- RLED_SetOpenDrain
 - pin_manager.h, [465](#)
- RLED_SetPullup
 - pin_manager.h, [466](#)
- RLED_SetPushPull
 - pin_manager.h, [466](#)
- RLED_Toggle
 - pin_manager.h, [466](#)
- RLED_TRIS
 - pin_manager.h, [466](#)
- RLED_WPU
 - pin_manager.h, [466](#)
- RTR_POSN
 - can1.c, [295](#)
- run_day_to_night
 - batmon.h, [147](#)
 - main.c, [274](#)
- run_night_to_day
 - batmon.h, [147](#)
 - main.c, [275](#)
- run_time
 - B_type, [71](#)
- runner
 - raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c, [267](#)
- runtime
 - raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c, [267](#)
- rx
 - M_data, [92](#)
- rx_count
 - B_type, [71](#)
- RX_FIFO_MSG_DATA
 - can1.c, [296](#)
- rx_flag
 - blob_type, [72](#)
- rxMsgData
 - canfd.c, [160](#)
 - canfd.h, [169](#)
- sec_30
 - raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c, [267](#)
- select
 - mx_logpage_t, [121](#)
- SEND

- modbus_master.h, 697
- send_count
 - M_data, 92
- send_lcd_cmd_dma
 - eadog.c, 184
 - eadog.h, 203
- send_lcd_data_dma
 - eadog.c, 185
 - eadog.h, 204
- send_lcd_pos_dma
 - eadog.c, 186
 - eadog.h, 204
- send_offset
 - mqtt_client, 99
- sendbuf
 - mqtt_pub.c, 232
- sends
 - M_data, 92
- sequence
 - raw.githubusercontent.com_linux-can_can-utils_master_caspiotes, 482
 - 267
- serial
 - EM_serial, 90
- SERIAL_DATA_LEN
 - modbus_master.h, 694
- serial_ok
 - C_data, 74
- Serror
 - modbus_master.h, 694
- session_present_flag
 - mqtt_response_connack, 109
- set_lcd_dim
 - eadog.c, 186
 - eadog.h, 205
- show_can
 - main.c, 282
- SID_HIGH_MASK
 - can1.c, 296
- SID_LOW_WIDTH
 - can1.c, 296
- size
 - mqtt_queued_message, 105
- socketfd
 - mqtt_client, 100
- sockfd
 - mqtt_pub.c, 232
- solar
 - raw.githubusercontent.com_linux-can_can-utils_master_caspiotes, 483
 - 268
- SPACING
 - modbus_master.h, 694
- SPACING_USEC
 - raw.githubusercontent.com_linux-can_can-utils_master_caspiotes, 484
 - 261
- spi1.c
 - SPI1_Close, 482
 - SPI1_ExchangeBlock, 482
 - SPI1_ExchangeByte, 482
- SPI1_Initialize, 482
 - SPI1_Open, 483
 - SPI1_ReadBlock, 483
 - SPI1_ReadByte, 483
 - SPI1_WriteBlock, 483
 - SPI1_WriteByte, 484
- spi1.h
 - SPI1_Close, 486
 - SPI1_DEFAULT, 486
 - SPI1_ExchangeBlock, 486
 - SPI1_ExchangeByte, 486
 - SPI1_Initialize, 486
 - spi1_modes_t, 485
 - SPI1_Open, 487
 - SPI1_ReadBlock, 487
 - SPI1_ReadByte, 487
 - SPI1_WriteBlock, 487
 - SPI1_WriteByte, 488
- SPI1_Close
 - spi1.c, 482
 - spi1.h, 486
- spi1_configuration_t, 124
 - baud, 124
 - con0, 124
 - con1, 124
 - con2, 124
 - operation, 124
- SPI1_DEFAULT
 - spi1.h, 486
- SPI1_ExchangeBlock
 - spi1.c, 482
 - spi1.h, 486
- SPI1_ExchangeByte
 - spi1.c, 482
 - spi1.h, 486
- SPI1_Initialize
 - spi1.c, 482
 - spi1.h, 486
- spi1_modes_t
 - spi1.h, 485
- SPI1_Open
 - spi1.c, 483
 - spi1.h, 487
- SPI1_ReadBlock
 - spi1.c, 483
 - spi1.h, 487
- SPI1_ReadByte
 - spi1.c, 483
 - spi1.h, 487
- SPI1_WriteBlock
 - spi1.c, 483
 - spi1.h, 487
- SPI1_WriteByte
 - spi1.c, 484
 - spi1.h, 488
- SPI_AUX
 - spi_link_type, 126
- SPI_LCD

- spi_link_type, 126
- spi_link
 - eadog.c, 189
- spi_link_type, 125
 - config, 125
 - delay, 125
 - int_count, 125
 - LCD_DATA, 125
 - LCD_TIMER, 125
 - SPI_AUX, 126
 - SPI_LCD, 126
 - txbuf, 126
- spi_rec_done
 - eadog.c, 187
 - eadog.h, 206
- spin
 - qconfig.h, 712
- SPIN_VAL_UPDATE
 - qconfig.h, 712
- SPINNER_SPEED
 - mxcmd.h, 668
- spinners
 - main.c, 275
 - qconfig.h, 712
- Srbuffer
 - modbus_master.h, 694
- Sread
 - modbus_master.h, 694
- ssl_conf
 - mbedtls_context, 94
- ssl_ctx
 - mbedtls_context, 94
- stable
 - M_data, 92
- STAH
 - CAN_FIFOREG, 76
- STAL
 - CAN_FIFOREG, 76
- start
 - mqtt_queued_message, 105
- start_lcd
 - eadog.c, 187
 - eadog.h, 206
- start_time
 - raw.githubusercontent.com_linux-can_can-utils_master_canutils.c, 268
- StartTime
 - V_type, 131
- StartTimer
 - timers.c, 722
 - timers.h, 726
- STAT
 - CAN_FIFOREG, 76
- state
 - main.c, 282
 - mqtt_queued_message, 105
- state_battery
 - main.c, 273
- state_battery_cb
 - main.c, 276
- state_batteryv
 - main.c, 273
- state_batteryv_cb
 - main.c, 276
- state_date
 - main.c, 273
- state_fwrev
 - main.c, 273
- state_init
 - main.c, 273
- state_init_cb
 - main.c, 276
- state_last
 - main.c, 273
- state_misc
 - main.c, 273
- state_misc_cb
 - main.c, 277
- state_mx_log
 - main.c, 273
- state_mx_log_cb
 - main.c, 277
- state_mx_status
 - main.c, 273
- state_mx_status_cb
 - main.c, 278
- state_name
 - mxcmd.h, 674
- state_panel
 - main.c, 273
- state_panelv_cb
 - main.c, 278
- state_status
 - main.c, 273
- state_status_cb
 - main.c, 279
- state_time
 - main.c, 273
- state_type
 - main.c, 273
- state_watts
 - main.c, 273
- state_watts_cb
 - main.c, 279
- status
 - uart1_status_t, 127
 - uart2_status_t, 128
 - uart5_status_t, 129
- STATUS_ABSORB
 - mxcmd.h, 669
- STATUS_BULK
 - mxcmd.h, 669
- STATUS_EQUALIZE
 - mxcmd.h, 669
- STATUS_FLOATING
 - mxcmd.h, 669

- STATUS_LAST
 - mxcmd.h, [669](#)
- STATUS_SLEEPING
 - mxcmd.h, [669](#)
- status_type
 - mxcmd.h, [669](#)
- STAU
 - CAN_FIFOREG, [76](#)
- Strmt
 - modbus_master.h, [694](#)
- structBytes
 - canfd.c, [160](#)
- suback
 - mqtt_response, [108](#)
- sw_contact_types
 - dio.h, [176](#)
- SW_NC
 - dio.h, [174](#)
- SW_NO
 - dio.h, [174](#)
- SWBMVER
 - modbus_master.h, [694](#)
- Swrite
 - modbus_master.h, [694](#)
- SYSTEM_Initialize
 - mcc.c, [379](#)
 - mcc.h, [383](#)
- SystemArbiter_Initialize
 - mcc.c, [381](#)
 - mcc.h, [385](#)
- T_begin
 - modbus_master.h, [697](#)
- T_clear
 - modbus_master.h, [697](#)
- T_config
 - modbus_master.h, [698](#)
- T_data
 - modbus_master.h, [698](#)
- T_id
 - modbus_master.h, [698](#)
- T_init
 - modbus_master.h, [698](#)
- T_init_d
 - modbus_master.h, [698](#)
- T_light
 - modbus_master.h, [698](#)
- T_misc1
 - modbus_master.h, [698](#)
- T_misc2
 - modbus_master.h, [698](#)
- T_misc3
 - modbus_master.h, [698](#)
- T_passwd
 - modbus_master.h, [698](#)
- T_rcv
 - modbus_master.h, [698](#)
- T_rcv_r
 - modbus_master.h, [698](#)
- T_send
 - modbus_master.h, [698](#)
- T_send_d
 - modbus_master.h, [698](#)
- T_serial
 - modbus_master.h, [698](#)
- T_spacing
 - modbus_master.h, [698](#)
- T_version
 - modbus_master.h, [698](#)
- tail
 - ringBufS_t, [123](#)
- TDELAY
 - modbus_master.h, [695](#)
- TEDELAY
 - modbus_master.h, [695](#)
- ten_sec_flag
 - B_type, [71](#)
- TEN_SEC_HOUR
 - batmon.h, [143](#)
- tensec_io
 - mxcmd.c, [665](#)
 - mxcmd.h, [671](#)
- tickCount
 - main.c, [282](#)
 - timers.c, [724](#)
- time
 - EB_data, [83](#)
- time_of_last_send
 - mqtt_client, [100](#)
- time_sent
 - mqtt_queued_message, [105](#)
- time_show_can
 - main.c, [282](#)
- TIMEOUT
 - raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c, [261](#)
- timer5ReloadVal
 - tmr5.c, [562](#)
- timer_2ms_tick
 - modbus_master.c, [685](#)
 - modbus_master.h, [704](#)
- timer_500ms_tick
 - modbus_master.c, [685](#)
 - modbus_master.h, [704](#)
- timer_callback
 - raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c, [264](#)
- timer_ms_tick
 - timers.c, [722](#)
 - timers.h, [726](#)
- TimerDone
 - timers.c, [723](#)
 - timers.h, [727](#)
- timers.c, [721](#)
 - delay_ms, [722](#)
 - delay_us, [722](#)
 - StartTimer, [722](#)

- tickCount, [724](#)
- timer_ms_tick, [722](#)
- TimerDone, [723](#)
- WaitMs, [723](#)
- timers.h, [724](#)
 - APP_TIMERS, [725](#)
 - delay_ms, [726](#)
 - StartTimer, [726](#)
 - timer_ms_tick, [726](#)
 - TimerDone, [727](#)
 - TMR_COUNT, [725](#)
 - TMR_DERE, [725](#)
 - TMR_INTERNAL, [725](#)
 - TMR_MBMASTER, [725](#)
 - TMR_MBTEST, [725](#)
 - TMR_SPIN, [725](#)
 - WaitMs, [727](#)
- TimeUsed
 - V_type, [131](#)
- tm_ok
 - C_data, [75](#)
- tmr0.c
 - __interrupt, [490](#)
 - TMR0_DefaultInterruptHandler, [490](#)
 - TMR0_Initialize, [490](#)
 - TMR0_InterruptHandler, [494](#)
 - TMR0_ReadTimer, [491](#)
 - TMR0_Reload, [491](#)
 - TMR0_SetInterruptHandler, [492](#)
 - TMR0_StartTimer, [492](#)
 - TMR0_StopTimer, [493](#)
 - TMR0_WriteTimer, [493](#)
- tmr0.h
 - TMR0_DefaultInterruptHandler, [496](#)
 - TMR0_Initialize, [496](#)
 - TMR0_InterruptHandler, [500](#)
 - TMR0_ReadTimer, [497](#)
 - TMR0_Reload, [497](#)
 - TMR0_SetInterruptHandler, [498](#)
 - TMR0_StartTimer, [498](#)
 - TMR0_StopTimer, [499](#)
 - TMR0_WriteTimer, [499](#)
- TMR0_DefaultInterruptHandler
 - tmr0.c, [490](#)
 - tmr0.h, [496](#)
- TMR0_Initialize
 - tmr0.c, [490](#)
 - tmr0.h, [496](#)
- TMR0_InterruptHandler
 - tmr0.c, [494](#)
 - tmr0.h, [500](#)
- TMR0_ReadTimer
 - tmr0.c, [491](#)
 - tmr0.h, [497](#)
- TMR0_Reload
 - tmr0.c, [491](#)
 - tmr0.h, [497](#)
- TMR0_SetInterruptHandler
 - tmr0.c, [490](#)
 - tmr0.h, [498](#)
- TMR0_StartTimer
 - tmr0.c, [492](#)
 - tmr0.h, [498](#)
- TMR0_StopTimer
 - tmr0.c, [493](#)
 - tmr0.h, [499](#)
- TMR0_WriteTimer
 - tmr0.c, [493](#)
 - tmr0.h, [499](#)
- tmr2.c
 - __interrupt, [502](#)
 - TMR2_Counter8BitGet, [502](#)
 - TMR2_Counter8BitSet, [503](#)
 - TMR2_DefaultInterruptHandler, [503](#)
 - TMR2_ExtResetSourceSet, [504](#)
 - TMR2_Initialize, [504](#)
 - TMR2_InterruptHandler, [511](#)
 - TMR2_LoadPeriodRegister, [505](#)
 - TMR2_ModeSet, [506](#)
 - TMR2_Period8BitSet, [506](#)
 - TMR2_ReadTimer, [507](#)
 - TMR2_SetInterruptHandler, [508](#)
 - TMR2_Start, [508](#)
 - TMR2_StartTimer, [509](#)
 - TMR2_Stop, [509](#)
 - TMR2_StopTimer, [510](#)
 - TMR2_WriteTimer, [510](#)
- tmr2.h
 - TMR2_CCP1_OUT, [514](#)
 - TMR2_CCP2_OUT, [514](#)
 - TMR2_CCP3_OUT, [514](#)
 - TMR2_CLC1_OUT, [514](#)
 - TMR2_CLC2_OUT, [514](#)
 - TMR2_CLC3_OUT, [514](#)
 - TMR2_CLC4_OUT, [514](#)
 - TMR2_CLC5_OUT, [514](#)
 - TMR2_CLC6_OUT, [514](#)
 - TMR2_CLC7_OUT, [514](#)
 - TMR2_CLC8_OUT, [514](#)
 - TMR2_CMP1_OUT, [514](#)
 - TMR2_CMP2_OUT, [514](#)
 - TMR2_Counter8BitGet, [515](#)
 - TMR2_Counter8BitSet, [516](#)
 - TMR2_DefaultInterruptHandler, [516](#)
 - TMR2_ExtResetSourceSet, [517](#)
 - TMR2_HLT_EXT_RESET_SOURCE, [513](#)
 - TMR2_HLT_MODE, [514](#)
 - TMR2_Initialize, [517](#)
 - TMR2_InterruptHandler, [524](#)
 - TMR2_LoadPeriodRegister, [518](#)
 - TMR2_ModeSet, [519](#)
 - TMR2_MS_STARTS_TMRON_ERSEBOTHEDGE, [515](#)
 - TMR2_MS_STARTS_TMRON_ERSFALLINGEDGEDETECT, [515](#)

TMR2_MS_STARTS_TMRON_ERSRISINGEDGEDETECT, 515
 TMR2_OS_STARTS_ERSBOTHEDGE, 515
 TMR2_OS_STARTS_ERSFALLINGEDGE, 515
 TMR2_OS_STARTS_ERSFALLINGEDGEDETECT, 515
 TMR2_OS_STARTS_ERSFIRSTFALLINGEDGE, 515
 TMR2_OS_STARTS_ERSFIRSTRISINGEDGE, 515
 TMR2_OS_STARTS_ERSRISINGEDGE, 515
 TMR2_OS_STARTS_ERSRISINGEDGEDETECT, 515
 TMR2_OS_STARTS_TMRON, 515
 TMR2_OS_STARTS_TMRON_ERSHIGH, 515
 TMR2_OS_STARTS_TMRON_ERSLOW, 515
 TMR2_Period8BitSet, 519
 TMR2_PWM1S1P1_OUT, 514
 TMR2_PWM1S1P2_OUT, 514
 TMR2_PWM2S1P1_OUT, 514
 TMR2_PWM2S1P2_OUT, 514
 TMR2_PWM3S1P1_OUT, 514
 TMR2_PWM3S1P2_OUT, 514
 TMR2_ReadTimer, 520
 TMR2_RESERVED, 514
 TMR2_RESERVED_2, 514
 TMR2_RESERVED_3, 514
 TMR2_RESERVED_4, 514
 TMR2_ROP_RESETS_ERSBOTHEDGE, 515
 TMR2_ROP_RESETS_ERSFALLINGEDGE, 515
 TMR2_ROP_RESETS_ERSHIGH, 515
 TMR2_ROP_RESETS_ERSLOW, 515
 TMR2_ROP_RESETS_ERSRISINGEDGE, 515
 TMR2_ROP_STARTS_TMRON, 515
 TMR2_ROP_STARTS_TMRON_ERSHIGH, 515
 TMR2_ROP_STARTS_TMRON_ERSLOW, 515
 TMR2_SetInterruptHandler, 521
 TMR2_Start, 521
 TMR2_StartTimer, 522
 TMR2_Stop, 522
 TMR2_StopTimer, 523
 TMR2_T2INPPS, 514
 TMR2_T4POSTSCALED, 514
 TMR2_T6POSTSCALED, 514
 TMR2_UART1_RX_EDGE, 514
 TMR2_UART1_TX_EDGE, 514
 TMR2_UART2_RX_EDGE, 514
 TMR2_UART2_TX_EDGE, 514
 TMR2_UART3_RX_EDGE, 514
 TMR2_UART3_TX_EDGE, 514
 TMR2_UART4_RX_EDGE, 514
 TMR2_UART4_TX_EDGE, 514
 TMR2_UART5_RX_EDGE, 514
 TMR2_UART5_TX_EDGE, 514
 TMR2_WriteTimer, 523
 TMR2_ZCD_OUTPUT, 514
 TMR2_CCP1_OUT
 tmr2.h, 514
 TMR2_CCP2_OUT
 tmr2.h, 514
 TMR2_CCP3_OUT
 tmr2.h, 514
 TMR2_CLC1_OUT
 tmr2.h, 514
 TMR2_CLC2_OUT
 tmr2.h, 514
 TMR2_CLC3_OUT
 tmr2.h, 514
 TMR2_CLC4_OUT
 tmr2.h, 514
 TMR2_CLC5_OUT
 tmr2.h, 514
 TMR2_CLC6_OUT
 tmr2.h, 514
 TMR2_CLC7_OUT
 tmr2.h, 514
 TMR2_CLC8_OUT
 tmr2.h, 514
 TMR2_CMP1_OUT
 tmr2.h, 514
 TMR2_CMP2_OUT
 tmr2.h, 514
 TMR2_Counter8BitGet
 tmr2.c, 502
 tmr2.h, 515
 TMR2_Counter8BitSet
 tmr2.c, 503
 tmr2.h, 516
 TMR2_DefaultInterruptHandler
 tmr2.c, 503
 tmr2.h, 516
 TMR2_ExtResetSourceSet
 tmr2.c, 504
 tmr2.h, 517
 TMR2_HLT_EXT_RESET_SOURCE
 tmr2.h, 513
 TMR2_HLT_MODE
 tmr2.h, 514
 TMR2_Initialize
 tmr2.c, 504
 tmr2.h, 517
 TMR2_InterruptHandler
 tmr2.c, 511
 tmr2.h, 524
 TMR2_LoadPeriodRegister
 tmr2.c, 505
 tmr2.h, 518
 TMR2_ModeSet
 tmr2.c, 506
 tmr2.h, 519
 TMR2_MS_STARTS_TMRON_ERSBOTHEDGE
 tmr2.h, 515
 TMR2_MS_STARTS_TMRON_ERSFALLINGEDGEDETECT
 tmr2.h, 515
 TMR2_MS_STARTS_TMRON_ERSRISINGEDGEDETECT
 tmr2.h, 515

- TMR2_OS_STARTS_ERSBOTHEDGE
 - tmr2.h, [515](#)
- TMR2_OS_STARTS_ERSFALLINGEDGE
 - tmr2.h, [515](#)
- TMR2_OS_STARTS_ERSFALLINGEDGEDETECT
 - tmr2.h, [515](#)
- TMR2_OS_STARTS_ERSFIRSTFALLINGEDGE
 - tmr2.h, [515](#)
- TMR2_OS_STARTS_ERSFIRSTRISINGEDGE
 - tmr2.h, [515](#)
- TMR2_OS_STARTS_ERSRISINGEDGE
 - tmr2.h, [515](#)
- TMR2_OS_STARTS_ERSRISINGEDGEDETECT
 - tmr2.h, [515](#)
- TMR2_OS_STARTS_TMRON
 - tmr2.h, [515](#)
- TMR2_OS_STARTS_TMRON_ERSHIGH
 - tmr2.h, [515](#)
- TMR2_OS_STARTS_TMRON_ERSLOW
 - tmr2.h, [515](#)
- TMR2_Period8BitSet
 - tmr2.c, [506](#)
 - tmr2.h, [519](#)
- TMR2_PWM1S1P1_OUT
 - tmr2.h, [514](#)
- TMR2_PWM1S1P2_OUT
 - tmr2.h, [514](#)
- TMR2_PWM2S1P1_OUT
 - tmr2.h, [514](#)
- TMR2_PWM2S1P2_OUT
 - tmr2.h, [514](#)
- TMR2_PWM3S1P1_OUT
 - tmr2.h, [514](#)
- TMR2_PWM3S1P2_OUT
 - tmr2.h, [514](#)
- TMR2_ReadTimer
 - tmr2.c, [507](#)
 - tmr2.h, [520](#)
- TMR2_RESERVED
 - tmr2.h, [514](#)
- TMR2_RESERVED_2
 - tmr2.h, [514](#)
- TMR2_RESERVED_3
 - tmr2.h, [514](#)
- TMR2_RESERVED_4
 - tmr2.h, [514](#)
- TMR2_ROP_RESETS_ERSBOTHEDGE
 - tmr2.h, [515](#)
- TMR2_ROP_RESETS_ERSFALLINGEDGE
 - tmr2.h, [515](#)
- TMR2_ROP_RESETS_ERSHIGH
 - tmr2.h, [515](#)
- TMR2_ROP_RESETS_ERSLOW
 - tmr2.h, [515](#)
- TMR2_ROP_RESETS_ERSRISINGEDGE
 - tmr2.h, [515](#)
- TMR2_ROP_STARTS_TMRON
 - tmr2.h, [515](#)
- TMR2_ROP_STARTS_TMRON_ERSHIGH
 - tmr2.h, [515](#)
- TMR2_ROP_STARTS_TMRON_ERSLOW
 - tmr2.h, [515](#)
- TMR2_SetInterruptHandler
 - tmr2.c, [508](#)
 - tmr2.h, [521](#)
- TMR2_Start
 - tmr2.c, [508](#)
 - tmr2.h, [521](#)
- TMR2_StartTimer
 - tmr2.c, [509](#)
 - tmr2.h, [522](#)
- TMR2_Stop
 - tmr2.c, [509](#)
 - tmr2.h, [522](#)
- TMR2_StopTimer
 - tmr2.c, [510](#)
 - tmr2.h, [523](#)
- TMR2_T2INPPS
 - tmr2.h, [514](#)
- TMR2_T4POSTSCALED
 - tmr2.h, [514](#)
- TMR2_T6POSTSCALED
 - tmr2.h, [514](#)
- TMR2_UART1_RX_EDGE
 - tmr2.h, [514](#)
- TMR2_UART1_TX_EDGE
 - tmr2.h, [514](#)
- TMR2_UART2_RX_EDGE
 - tmr2.h, [514](#)
- TMR2_UART2_TX_EDGE
 - tmr2.h, [514](#)
- TMR2_UART3_RX_EDGE
 - tmr2.h, [514](#)
- TMR2_UART3_TX_EDGE
 - tmr2.h, [514](#)
- TMR2_UART4_RX_EDGE
 - tmr2.h, [514](#)
- TMR2_UART4_TX_EDGE
 - tmr2.h, [514](#)
- TMR2_UART5_RX_EDGE
 - tmr2.h, [514](#)
- TMR2_UART5_TX_EDGE
 - tmr2.h, [514](#)
- TMR2_WriteTimer
 - tmr2.c, [510](#)
 - tmr2.h, [523](#)
- TMR2_ZCD_OUTPUT
 - tmr2.h, [514](#)
- tmr4.c
 - __interrupt, [530](#)
 - TMR4_Counter8BitGet, [530](#)
 - TMR4_Counter8BitSet, [530](#)
 - TMR4_DefaultInterruptHandler, [531](#)
 - TMR4_ExtResetSourceSet, [531](#)
 - TMR4_Initialize, [532](#)
 - TMR4_InterruptHandler, [538](#)

- TMR4_LoadPeriodRegister, [532](#)
- TMR4_ModeSet, [533](#)
- TMR4_Period8BitSet, [533](#)
- TMR4_ReadTimer, [534](#)
- TMR4_SetInterruptHandler, [535](#)
- TMR4_Start, [535](#)
- TMR4_StartTimer, [536](#)
- TMR4_Stop, [536](#)
- TMR4_StopTimer, [537](#)
- TMR4_WriteTimer, [537](#)
- tmr4.h
 - TMR4_CCP1_OUT, [541](#)
 - TMR4_CCP2_OUT, [541](#)
 - TMR4_CCP3_OUT, [541](#)
 - TMR4_CLC1_OUT, [541](#)
 - TMR4_CLC2_OUT, [541](#)
 - TMR4_CLC3_OUT, [541](#)
 - TMR4_CLC4_OUT, [541](#)
 - TMR4_CLC5_OUT, [541](#)
 - TMR4_CLC6_OUT, [541](#)
 - TMR4_CLC7_OUT, [541](#)
 - TMR4_CLC8_OUT, [541](#)
 - TMR4_CMP1_OUT, [541](#)
 - TMR4_CMP2_OUT, [541](#)
 - TMR4_Counter8BitGet, [542](#)
 - TMR4_Counter8BitSet, [543](#)
 - TMR4_DefaultInterruptHandler, [543](#)
 - TMR4_ExtResetSourceSet, [544](#)
 - TMR4_HLT_EXT_RESET_SOURCE, [540](#)
 - TMR4_HLT_MODE, [541](#)
 - TMR4_Initialize, [544](#)
 - TMR4_InterruptHandler, [551](#)
 - TMR4_LoadPeriodRegister, [545](#)
 - TMR4_ModeSet, [546](#)
 - TMR4_MS_STARTS_TMRON_ERSBOTHEDGE, [542](#)
 - TMR4_MS_STARTS_TMRON_ERSFALLINGEDGEDETECT, [542](#)
 - TMR4_MS_STARTS_TMRON_ERSRISINGEDGEDETECT, [542](#)
 - TMR4_OS_STARTS_ERSBOTHEDGE, [542](#)
 - TMR4_OS_STARTS_ERSFALLINGEDGE, [542](#)
 - TMR4_OS_STARTS_ERSFALLINGEDGEDETECT, [542](#)
 - TMR4_OS_STARTS_ERSFIRSTFALLINGEDGE, [542](#)
 - TMR4_OS_STARTS_ERSFIRSTRISINGEDGE, [542](#)
 - TMR4_OS_STARTS_ERSRISINGEDGE, [542](#)
 - TMR4_OS_STARTS_ERSRISINGEDGEDETECT, [542](#)
 - TMR4_OS_STARTS_TMRON, [542](#)
 - TMR4_OS_STARTS_TMRON_ERSHIGH, [542](#)
 - TMR4_OS_STARTS_TMRON_ERSLOW, [542](#)
 - TMR4_Period8BitSet, [546](#)
 - TMR4_PWM1S1P1_OUT, [541](#)
 - TMR4_PWM1S1P2_OUT, [541](#)
 - TMR4_PWM2S1P1_OUT, [541](#)
 - TMR4_PWM2S1P2_OUT, [541](#)
 - TMR4_PWM3S1P1_OUT, [541](#)
 - TMR4_PWM3S1P2_OUT, [541](#)
 - TMR4_ReadTimer, [547](#)
 - TMR4_RESERVED, [541](#)
 - TMR4_RESERVED_2, [541](#)
 - TMR4_RESERVED_3, [541](#)
 - TMR4_RESERVED_4, [541](#)
 - TMR4_ROP_RESETS_ERSBOTHEDGE, [542](#)
 - TMR4_ROP_RESETS_ERSFALLINGEDGE, [542](#)
 - TMR4_ROP_RESETS_ERSHIGH, [542](#)
 - TMR4_ROP_RESETS_ERSLOW, [542](#)
 - TMR4_ROP_RESETS_ERSRISINGEDGE, [542](#)
 - TMR4_ROP_STARTS_TMRON, [542](#)
 - TMR4_ROP_STARTS_TMRON_ERSHIGH, [542](#)
 - TMR4_ROP_STARTS_TMRON_ERSLOW, [542](#)
 - TMR4_SetInterruptHandler, [548](#)
 - TMR4_Start, [548](#)
 - TMR4_StartTimer, [549](#)
 - TMR4_Stop, [549](#)
 - TMR4_StopTimer, [550](#)
 - TMR4_T2POSTSCALED, [541](#)
 - TMR4_T4INPPS, [541](#)
 - TMR4_T6POSTSCALED, [541](#)
 - TMR4_UART1_RX_EDGE, [541](#)
 - TMR4_UART1_TX_EDGE, [541](#)
 - TMR4_UART2_RX_EDGE, [541](#)
 - TMR4_UART2_TX_EDGE, [541](#)
 - TMR4_UART3_RX_EDGE, [541](#)
 - TMR4_UART3_TX_EDGE, [541](#)
 - TMR4_UART4_RX_EDGE, [541](#)
 - TMR4_UART4_TX_EDGE, [541](#)
 - TMR4_UART5_RX_EDGE, [541](#)
 - TMR4_UART5_TX_EDGE, [541](#)
 - TMR4_WriteTimer, [550](#)
 - TMR4_ZCD_OUTPUT, [541](#)

TMR4_CMP1_OUT
 tmr4.h, [541](#)
 TMR4_CMP2_OUT
 tmr4.h, [541](#)
 TMR4_Counter8BitGet
 tmr4.c, [530](#)
 tmr4.h, [542](#)
 TMR4_Counter8BitSet
 tmr4.c, [530](#)
 tmr4.h, [543](#)
 TMR4_DefaultInterruptHandler
 tmr4.c, [531](#)
 tmr4.h, [543](#)
 TMR4_ExtResetSourceSet
 tmr4.c, [531](#)
 tmr4.h, [544](#)
 TMR4_HLT_EXT_RESET_SOURCE
 tmr4.h, [540](#)
 TMR4_HLT_MODE
 tmr4.h, [541](#)
 TMR4_Initialize
 tmr4.c, [532](#)
 tmr4.h, [544](#)
 TMR4_InterruptHandler
 tmr4.c, [538](#)
 tmr4.h, [551](#)
 TMR4_LoadPeriodRegister
 tmr4.c, [532](#)
 tmr4.h, [545](#)
 TMR4_ModeSet
 tmr4.c, [533](#)
 tmr4.h, [546](#)
 TMR4_MS_STARTS_TMRON_ERSBOTHEDGE
 tmr4.h, [542](#)
 TMR4_MS_STARTS_TMRON_ERSFALLINGEDGEDETECT
 tmr4.h, [542](#)
 TMR4_MS_STARTS_TMRON_ERSRISINGEDGEDETECT
 tmr4.h, [542](#)
 TMR4_OS_STARTS_ERSBOTHEDGE
 tmr4.h, [542](#)
 TMR4_OS_STARTS_ERSFALLINGEDGE
 tmr4.h, [542](#)
 TMR4_OS_STARTS_ERSFALLINGEDGEDETECT
 tmr4.h, [542](#)
 TMR4_OS_STARTS_ERSFIRSTFALLINGEDGE
 tmr4.h, [542](#)
 TMR4_OS_STARTS_ERSFIRSTRISINGEDGE
 tmr4.h, [542](#)
 TMR4_OS_STARTS_ERSRISINGEDGE
 tmr4.h, [542](#)
 TMR4_OS_STARTS_ERSRISINGEDGEDETECT
 tmr4.h, [542](#)
 TMR4_OS_STARTS_TMRON
 tmr4.h, [542](#)
 TMR4_OS_STARTS_TMRON_ERSHIGH
 tmr4.h, [542](#)
 TMR4_OS_STARTS_TMRON_ERSLow
 tmr4.h, [542](#)
 TMR4_Period8BitSet
 tmr4.c, [533](#)
 tmr4.h, [546](#)
 TMR4_PWM1S1P1_OUT
 tmr4.h, [541](#)
 TMR4_PWM1S1P2_OUT
 tmr4.h, [541](#)
 TMR4_PWM2S1P1_OUT
 tmr4.h, [541](#)
 TMR4_PWM2S1P2_OUT
 tmr4.h, [541](#)
 TMR4_PWM3S1P1_OUT
 tmr4.h, [541](#)
 TMR4_PWM3S1P2_OUT
 tmr4.h, [541](#)
 TMR4_ReadTimer
 tmr4.c, [534](#)
 tmr4.h, [547](#)
 TMR4_RESERVED
 tmr4.h, [541](#)
 TMR4_RESERVED_2
 tmr4.h, [541](#)
 TMR4_RESERVED_3
 tmr4.h, [541](#)
 TMR4_RESERVED_4
 tmr4.h, [541](#)
 TMR4_ROP_RESETS_ERSBOTHEDGE
 tmr4.h, [542](#)
 TMR4_ROP_RESETS_ERSFALLINGEDGE
 tmr4.h, [542](#)
 TMR4_ROP_RESETS_ERSHIGH
 tmr4.h, [542](#)
 TMR4_ROP_RESETS_ERSLow
 tmr4.h, [542](#)
 TMR4_ROP_RESETS_ERSRISINGEDGE
 tmr4.h, [542](#)
 TMR4_ROP_STARTS_TMRON
 tmr4.h, [542](#)
 TMR4_ROP_STARTS_TMRON_ERSHIGH
 tmr4.h, [542](#)
 TMR4_ROP_STARTS_TMRON_ERSLow
 tmr4.h, [542](#)
 TMR4_SetInterruptHandler
 tmr4.c, [535](#)
 tmr4.h, [548](#)
 TMR4_Start
 tmr4.c, [535](#)
 tmr4.h, [548](#)
 TMR4_StartTimer
 tmr4.c, [536](#)
 tmr4.h, [549](#)
 TMR4_Stop
 tmr4.c, [536](#)
 tmr4.h, [549](#)
 TMR4_StopTimer
 tmr4.c, [537](#)
 tmr4.h, [550](#)
 TMR4_T2POSTSCALED

- tmr4.h, [541](#)
- TMR4_T4INPPS
 - tmr4.h, [541](#)
- TMR4_T6POSTSCALED
 - tmr4.h, [541](#)
- TMR4_UART1_RX_EDGE
 - tmr4.h, [541](#)
- TMR4_UART1_TX_EDGE
 - tmr4.h, [541](#)
- TMR4_UART2_RX_EDGE
 - tmr4.h, [541](#)
- TMR4_UART2_TX_EDGE
 - tmr4.h, [541](#)
- TMR4_UART3_RX_EDGE
 - tmr4.h, [541](#)
- TMR4_UART3_TX_EDGE
 - tmr4.h, [541](#)
- TMR4_UART4_RX_EDGE
 - tmr4.h, [541](#)
- TMR4_UART4_TX_EDGE
 - tmr4.h, [541](#)
- TMR4_UART5_RX_EDGE
 - tmr4.h, [541](#)
- TMR4_UART5_TX_EDGE
 - tmr4.h, [541](#)
- TMR4_WriteTimer
 - tmr4.c, [537](#)
 - tmr4.h, [550](#)
- TMR4_ZCD_OUTPUT
 - tmr4.h, [541](#)
- tmr5.c
 - __interrupt, [557](#)
 - timer5ReloadVal, [562](#)
 - TMR5_CheckGateValueStatus, [557](#)
 - TMR5_DefaultInterruptHandler, [557](#)
 - TMR5_Initialize, [558](#)
 - TMR5_InterruptHandler, [562](#)
 - TMR5_ReadTimer, [558](#)
 - TMR5_Reload, [559](#)
 - TMR5_SetInterruptHandler, [559](#)
 - TMR5_StartSinglePulseAcquisition, [560](#)
 - TMR5_StartTimer, [560](#)
 - TMR5_StopTimer, [561](#)
 - TMR5_WriteTimer, [561](#)
- tmr5.h
 - TMR5_CheckGateValueStatus, [564](#)
 - TMR5_DefaultInterruptHandler, [564](#)
 - TMR5_Initialize, [565](#)
 - TMR5_InterruptHandler, [570](#)
 - TMR5_ReadTimer, [566](#)
 - TMR5_Reload, [566](#)
 - TMR5_SetInterruptHandler, [567](#)
 - TMR5_StartSinglePulseAcquisition, [567](#)
 - TMR5_StartTimer, [568](#)
 - TMR5_StopTimer, [568](#)
 - TMR5_WriteTimer, [569](#)
- TMR5_CheckGateValueStatus
 - tmr5.c, [557](#)
- tmr5.h, [564](#)
- TMR5_DefaultInterruptHandler
 - tmr5.c, [557](#)
 - tmr5.h, [564](#)
- TMR5_Initialize
 - tmr5.c, [558](#)
 - tmr5.h, [565](#)
- TMR5_InterruptHandler
 - tmr5.c, [562](#)
 - tmr5.h, [570](#)
- TMR5_ReadTimer
 - tmr5.c, [558](#)
 - tmr5.h, [566](#)
- TMR5_Reload
 - tmr5.c, [559](#)
 - tmr5.h, [566](#)
- TMR5_SetInterruptHandler
 - tmr5.c, [559](#)
 - tmr5.h, [567](#)
- TMR5_StartSinglePulseAcquisition
 - tmr5.c, [560](#)
 - tmr5.h, [567](#)
- TMR5_StartTimer
 - tmr5.c, [560](#)
 - tmr5.h, [568](#)
- TMR5_StopTimer
 - tmr5.c, [561](#)
 - tmr5.h, [568](#)
- TMR5_WriteTimer
 - tmr5.c, [561](#)
 - tmr5.h, [569](#)
- tmr6.c
 - __interrupt, [572](#)
 - TMR6_Counter8BitGet, [572](#)
 - TMR6_Counter8BitSet, [572](#)
 - TMR6_DefaultInterruptHandler, [573](#)
 - TMR6_ExtResetSourceSet, [573](#)
 - TMR6_Initialize, [574](#)
 - TMR6_InterruptHandler, [580](#)
 - TMR6_LoadPeriodRegister, [574](#)
 - TMR6_ModeSet, [575](#)
 - TMR6_Period8BitSet, [575](#)
 - TMR6_ReadTimer, [576](#)
 - TMR6_SetInterruptHandler, [577](#)
 - TMR6_Start, [577](#)
 - TMR6_StartTimer, [578](#)
 - TMR6_Stop, [578](#)
 - TMR6_StopTimer, [579](#)
 - TMR6_WriteTimer, [579](#)
- tmr6.h
 - TMR6_CCP1_OUT, [583](#)
 - TMR6_CCP2_OUT, [583](#)
 - TMR6_CCP3_OUT, [583](#)
 - TMR6_CLC1_OUT, [583](#)
 - TMR6_CLC2_OUT, [583](#)
 - TMR6_CLC3_OUT, [583](#)
 - TMR6_CLC4_OUT, [583](#)
 - TMR6_CLC5_OUT, [583](#)

TMR6_CLC6_OUT, [583](#)
 TMR6_CLC7_OUT, [583](#)
 TMR6_CLC8_OUT, [583](#)
 TMR6_CMP1_OUT, [583](#)
 TMR6_CMP2_OUT, [583](#)
 TMR6_Counter8BitGet, [584](#)
 TMR6_Counter8BitSet, [585](#)
 TMR6_DefaultInterruptHandler, [585](#)
 TMR6_ExtResetSourceSet, [586](#)
 TMR6_HLT_EXT_RESET_SOURCE, [582](#)
 TMR6_HLT_MODE, [583](#)
 TMR6_Initialize, [586](#)
 TMR6_InterruptHandler, [593](#)
 TMR6_LoadPeriodRegister, [587](#)
 TMR6_ModeSet, [588](#)
 TMR6_MS_STARTS_TMRON_ERBOTHEDGE, [584](#)
 TMR6_MS_STARTS_TMRON_ERFALLINGEDGEDETECT, [584](#)
 TMR6_MS_STARTS_TMRON_ERSRISINGEDGEDETECT, [584](#)
 TMR6_OS_STARTS_ERBOTHEDGE, [584](#)
 TMR6_OS_STARTS_ERFALLINGEDGE, [584](#)
 TMR6_OS_STARTS_ERFALLINGEDGEDETECT, [584](#)
 TMR6_OS_STARTS_ERFIRSTFALLINGEDGE, [584](#)
 TMR6_OS_STARTS_ERFIRSTRISINGEDGE, [584](#)
 TMR6_OS_STARTS_ERSRISINGEDGE, [584](#)
 TMR6_OS_STARTS_ERSRISINGEDGEDETECT, [584](#)
 TMR6_OS_STARTS_TMRON, [584](#)
 TMR6_OS_STARTS_TMRON_ERHIGH, [584](#)
 TMR6_OS_STARTS_TMRON_ERLOW, [584](#)
 TMR6_Period8BitSet, [588](#)
 TMR6_PWM1S1P1_OUT, [583](#)
 TMR6_PWM1S1P2_OUT, [583](#)
 TMR6_PWM2S1P1_OUT, [583](#)
 TMR6_PWM2S1P2_OUT, [583](#)
 TMR6_PWM3S1P1_OUT, [583](#)
 TMR6_PWM3S1P2_OUT, [583](#)
 TMR6_ReadTimer, [589](#)
 TMR6_RESERVED, [583](#)
 TMR6_RESERVED_2, [583](#)
 TMR6_RESERVED_3, [583](#)
 TMR6_RESERVED_4, [583](#)
 TMR6_ROP_RESETS_ERBOTHEDGE, [584](#)
 TMR6_ROP_RESETS_ERFALLINGEDGE, [584](#)
 TMR6_ROP_RESETS_ERHIGH, [584](#)
 TMR6_ROP_RESETS_ERLOW, [584](#)
 TMR6_ROP_RESETS_ERSRISINGEDGE, [584](#)
 TMR6_ROP_STARTS_TMRON, [584](#)
 TMR6_ROP_STARTS_TMRON_ERHIGH, [584](#)
 TMR6_ROP_STARTS_TMRON_ERLOW, [584](#)
 TMR6_SetInterruptHandler, [590](#)
 TMR6_Start, [590](#)
 TMR6_StartTimer, [591](#)
 TMR6_Stop, [591](#)
 TMR6_StopTimer, [592](#)
 TMR6_T2POSTSCALED, [583](#)
 TMR6_T4POSTSCALED, [583](#)
 TMR6_T6INPPS, [583](#)
 TMR6_UART1_RX_EDGE, [583](#)
 TMR6_UART1_TX_EDGE, [583](#)
 TMR6_UART2_RX_EDGE, [583](#)
 TMR6_UART2_TX_EDGE, [583](#)
 TMR6_UART3_RX_EDGE, [583](#)
 TMR6_UART3_TX_EDGE, [583](#)
 TMR6_UART4_RX_EDGE, [583](#)
 TMR6_UART4_TX_EDGE, [583](#)
 TMR6_UART5_RX_EDGE, [583](#)
 TMR6_UART5_TX_EDGE, [583](#)
 TMR6_WriteTimer, [592](#)
 TMR6_ZCD_OUTPUT, [583](#)
 TMR6_CCP1_OUT, [583](#)
 TMR6_CCP2_OUT, [583](#)
 TMR6_CCP3_OUT, [583](#)
 TMR6_CLC1_OUT, [583](#)
 TMR6_CLC2_OUT, [583](#)
 TMR6_CLC3_OUT, [583](#)
 TMR6_CLC4_OUT, [583](#)
 TMR6_CLC5_OUT, [583](#)
 TMR6_CLC6_OUT, [583](#)
 TMR6_CLC7_OUT, [583](#)
 TMR6_CLC8_OUT, [583](#)
 TMR6_CMP1_OUT, [583](#)
 TMR6_CMP2_OUT, [583](#)
 TMR6_Counter8BitGet, [572](#)
 TMR6_Counter8BitSet, [572](#)
 TMR6_DefaultInterruptHandler, [573](#)
 TMR6_ExtResetSourceSet, [573](#)
 TMR6_HLT_EXT_RESET_SOURCE, [582](#)
 TMR6_HLT_MODE, [583](#)

- tmr6.h, [583](#)
- TMR6_Initialize
 - tmr6.c, [574](#)
 - tmr6.h, [586](#)
- TMR6_InterruptHandler
 - tmr6.c, [580](#)
 - tmr6.h, [593](#)
- TMR6_LoadPeriodRegister
 - tmr6.c, [574](#)
 - tmr6.h, [587](#)
- TMR6_ModeSet
 - tmr6.c, [575](#)
 - tmr6.h, [588](#)
- TMR6_MS_STARTS_TMRON_ERSBOTHEDGE
 - tmr6.h, [584](#)
- TMR6_MS_STARTS_TMRON_ERSFALLINGEDGEDETECT
 - tmr6.h, [584](#)
- TMR6_MS_STARTS_TMRON_ERSRISINGEDGEDETECT
 - tmr6.h, [584](#)
- TMR6_OS_STARTS_ERSBOTHEDGE
 - tmr6.h, [584](#)
- TMR6_OS_STARTS_ERSFALLINGEDGE
 - tmr6.h, [584](#)
- TMR6_OS_STARTS_ERSFALLINGEDGEDETECT
 - tmr6.h, [584](#)
- TMR6_OS_STARTS_ERSFIRSTFALLINGEDGE
 - tmr6.h, [584](#)
- TMR6_OS_STARTS_ERSFIRSTRISINGEDGE
 - tmr6.h, [584](#)
- TMR6_OS_STARTS_ERSRISINGEDGE
 - tmr6.h, [584](#)
- TMR6_OS_STARTS_ERSRISINGEDGEDETECT
 - tmr6.h, [584](#)
- TMR6_OS_STARTS_TMRON
 - tmr6.h, [584](#)
- TMR6_OS_STARTS_TMRON_ERSHIGH
 - tmr6.h, [584](#)
- TMR6_OS_STARTS_TMRON_ERSLow
 - tmr6.h, [584](#)
- TMR6_Period8BitSet
 - tmr6.c, [575](#)
 - tmr6.h, [588](#)
- TMR6_PWM1S1P1_OUT
 - tmr6.h, [583](#)
- TMR6_PWM1S1P2_OUT
 - tmr6.h, [583](#)
- TMR6_PWM2S1P1_OUT
 - tmr6.h, [583](#)
- TMR6_PWM2S1P2_OUT
 - tmr6.h, [583](#)
- TMR6_PWM3S1P1_OUT
 - tmr6.h, [583](#)
- TMR6_PWM3S1P2_OUT
 - tmr6.h, [583](#)
- TMR6_ReadTimer
 - tmr6.c, [576](#)
 - tmr6.h, [589](#)
- TMR6_RESERVED
 - tmr6.h, [583](#)
- TMR6_RESERVED_2
 - tmr6.h, [583](#)
- TMR6_RESERVED_3
 - tmr6.h, [583](#)
- TMR6_RESERVED_4
 - tmr6.h, [583](#)
- TMR6_ROP_RESETS_ERSBOTHEDGE
 - tmr6.h, [584](#)
- TMR6_ROP_RESETS_ERSFALLINGEDGE
 - tmr6.h, [584](#)
- TMR6_ROP_RESETS_ERSHIGH
 - tmr6.h, [584](#)
- TMR6_ROP_RESETS_ERSLow
 - tmr6.h, [584](#)
- TMR6_ROP_RESETS_ERSRISINGEDGE
 - tmr6.h, [584](#)
- TMR6_ROP_STARTS_TMRON
 - tmr6.h, [584](#)
- TMR6_ROP_STARTS_TMRON_ERSHIGH
 - tmr6.h, [584](#)
- TMR6_ROP_STARTS_TMRON_ERSLow
 - tmr6.h, [584](#)
- TMR6_SetInterruptHandler
 - tmr6.c, [577](#)
 - tmr6.h, [590](#)
- TMR6_Start
 - tmr6.c, [577](#)
 - tmr6.h, [590](#)
- TMR6_StartTimer
 - tmr6.c, [578](#)
 - tmr6.h, [591](#)
- TMR6_Stop
 - tmr6.c, [578](#)
 - tmr6.h, [591](#)
- TMR6_StopTimer
 - tmr6.c, [579](#)
 - tmr6.h, [592](#)
- TMR6_T2POSTSCALED
 - tmr6.h, [583](#)
- TMR6_T4POSTSCALED
 - tmr6.h, [583](#)
- TMR6_T6INPPS
 - tmr6.h, [583](#)
- TMR6_UART1_RX_EDGE
 - tmr6.h, [583](#)
- TMR6_UART1_TX_EDGE
 - tmr6.h, [583](#)
- TMR6_UART2_RX_EDGE
 - tmr6.h, [583](#)
- TMR6_UART2_TX_EDGE
 - tmr6.h, [583](#)
- TMR6_UART3_RX_EDGE
 - tmr6.h, [583](#)
- TMR6_UART3_TX_EDGE
 - tmr6.h, [583](#)
- TMR6_UART4_RX_EDGE
 - tmr6.h, [583](#)

- TMR6_UART4_TX_EDGE
 - tmr6.h, [583](#)
- TMR6_UART5_RX_EDGE
 - tmr6.h, [583](#)
- TMR6_UART5_TX_EDGE
 - tmr6.h, [583](#)
- TMR6_WriteTimer
 - tmr6.c, [579](#)
 - tmr6.h, [592](#)
- TMR6_ZCD_OUTPUT
 - tmr6.h, [583](#)
- TMR_COUNT
 - timers.h, [725](#)
- TMR_DERE
 - timers.h, [725](#)
- TMR_INTERNAL
 - timers.h, [725](#)
- TMR_MBMASTER
 - timers.h, [725](#)
- TMR_MBTEST
 - timers.h, [725](#)
- TMR_SPIN
 - timers.h, [725](#)
- to_error
 - M_data, [92](#)
- TODELAY
 - modbus_master.h, [695](#)
- token
 - raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c, [268](#)
- topic
 - mqtt_pub.c, [232](#)
- topic_name
 - mqtt_response_publish, [113](#)
- topic_name_size
 - mqtt_response_publish, [113](#)
- TOPIC_P
 - raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c, [261](#)
- TOPIC_S
 - raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c, [261](#)
- TRACE
 - trace.h, [729](#)
 - uart2.c, [616](#)
 - uart5.c, [640](#)
- trace
 - C_data, [75](#)
- trace.h, [729](#)
 - TRACE, [729](#)
- trace_type
 - modbus_master.h, [696](#), [697](#)
- tx_count
 - can_frames_t, [77](#)
- tx_flag
 - blob_type, [72](#)
- txbuf
 - spi_link_type, [126](#)
- TXQ
 - can1.h, [316](#)
- TXQNotFullHandler
 - canfd.c, [159](#)
 - canfd.h, [168](#)
- type
 - mx_logpage_t, [121](#)
- typical_response_time
 - mqtt_client, [100](#)
- UA
 - CAN_FIFOREG, [77](#)
- uart1.c
 - UART1_DefaultErrorHandler, [599](#)
 - UART1_DefaultFramingErrorHandler, [599](#)
 - UART1_DefaultOverrunErrorHandler, [599](#)
 - UART1_ErrorHandler, [606](#)
 - UART1_FramingErrorHandler, [606](#)
 - UART1_get_last_status, [600](#)
 - UART1_Initialize, [600](#)
 - UART1_is_rx_ready, [601](#)
 - UART1_is_tx_done, [602](#)
 - UART1_is_tx_ready, [602](#)
 - UART1_OverrunErrorHandler, [606](#)
 - UART1_Read, [603](#)
 - UART1_SetAddressToTransmit, [603](#)
 - UART1_SetErrorHandler, [604](#)
 - UART1_SetFramingErrorHandler, [604](#)
 - UART1_SetOverrunErrorHandler, [604](#)
 - UART1_Write, [605](#)
- uart1.h
 - UART1_DataReady, [607](#)
 - UART1_get_last_status, [608](#)
 - UART1_Initialize, [608](#)
 - UART1_is_rx_ready, [609](#)
 - UART1_is_tx_done, [609](#)
 - UART1_is_tx_ready, [610](#)
 - UART1_Read, [610](#)
 - UART1_SetAddressToTransmit, [611](#)
 - UART1_SetErrorHandler, [611](#)
 - UART1_SetFramingErrorHandler, [612](#)
 - UART1_SetOverrunErrorHandler, [612](#)
 - UART1_Write, [613](#)
- UART1_DataReady
 - uart1.h, [607](#)
- UART1_DefaultErrorHandler
 - uart1.c, [599](#)
- UART1_DefaultFramingErrorHandler
 - uart1.c, [599](#)
- UART1_DefaultOverrunErrorHandler
 - uart1.c, [599](#)
- UART1_ErrorHandler
 - uart1.c, [606](#)
- UART1_FramingErrorHandler
 - uart1.c, [606](#)
- UART1_get_last_status
 - uart1.c, [600](#)
 - uart1.h, [608](#)
- UART1_Initialize

- uart1.c, [600](#)
- uart1.h, [608](#)
- UART1_is_rx_ready
 - uart1.c, [601](#)
 - uart1.h, [609](#)
- UART1_is_tx_done
 - uart1.c, [602](#)
 - uart1.h, [609](#)
- UART1_is_tx_ready
 - uart1.c, [602](#)
 - uart1.h, [610](#)
- UART1_OverrunErrorHandler
 - uart1.c, [606](#)
- UART1_Read
 - uart1.c, [603](#)
 - uart1.h, [610](#)
- UART1_SetAddressToTransmit
 - uart1.c, [603](#)
 - uart1.h, [611](#)
- UART1_SetErrorHandler
 - uart1.c, [604](#)
 - uart1.h, [611](#)
- UART1_SetFramingErrorHandler
 - uart1.c, [604](#)
 - uart1.h, [612](#)
- UART1_SetOverrunErrorHandler
 - uart1.c, [604](#)
 - uart1.h, [612](#)
- uart1_status_t, [126](#)
 - ferr, [126](#)
 - oerr, [127](#)
 - perr, [127](#)
 - reserved, [127](#)
 - status, [127](#)
- UART1_Write
 - uart1.c, [605](#)
 - uart1.h, [613](#)
- uart2.c
 - __interrupt, [617](#)
 - getch, [617](#)
 - INT_TRACE, [616](#)
 - IO_RB6_Toggle, [616](#)
 - putch, [617](#)
 - TRACE, [616](#)
 - UART2_DefaultErrorHandler, [618](#)
 - UART2_DefaultFramingErrorHandler, [618](#)
 - UART2_DefaultOverrunErrorHandler, [618](#)
 - UART2_ErrorHandler, [627](#)
 - UART2_FramingErrorHandler, [627](#)
 - UART2_get_last_status, [619](#)
 - UART2_Initialize, [619](#)
 - UART2_is_rx_ready, [620](#)
 - UART2_is_tx_done, [621](#)
 - UART2_is_tx_ready, [621](#)
 - UART2_OverrunErrorHandler, [627](#)
 - UART2_Read, [622](#)
 - UART2_Receive_ISR, [622](#)
 - UART2_RX_BUFFER_SIZE, [617](#)
 - UART2_RxDataHandler, [623](#)
 - UART2_SetErrorHandler, [623](#)
 - UART2_SetFramingErrorHandler, [624](#)
 - UART2_SetOverrunErrorHandler, [624](#)
 - UART2_SetRxInterruptHandler, [625](#)
 - UART2_SetTxInterruptHandler, [625](#)
 - UART2_Transmit_ISR, [625](#)
 - UART2_TX_BUFFER_SIZE, [617](#)
 - UART2_Write, [626](#)
 - uart2RxCount, [627](#)
 - uart2TxBufferRemaining, [627](#)
- uart2.h
 - UART2_DataReady, [629](#)
 - UART2_get_last_status, [629](#)
 - UART2_Initialize, [629](#)
 - UART2_is_rx_ready, [630](#)
 - UART2_is_tx_done, [631](#)
 - UART2_is_tx_ready, [631](#)
 - UART2_Read, [632](#)
 - UART2_Receive_ISR, [632](#)
 - UART2_RxDataHandler, [633](#)
 - UART2_RxInterruptHandler, [637](#)
 - UART2_SetErrorHandler, [633](#)
 - UART2_SetFramingErrorHandler, [634](#)
 - UART2_SetOverrunErrorHandler, [634](#)
 - UART2_SetRxInterruptHandler, [635](#)
 - UART2_SetTxInterruptHandler, [635](#)
 - UART2_Transmit_ISR, [635](#)
 - UART2_TxInterruptHandler, [637](#)
 - UART2_Write, [636](#)
 - uart2RxCount, [637](#)
 - uart2TxBufferRemaining, [637](#)
- UART2_DataReady
 - uart2.h, [629](#)
- UART2_DefaultErrorHandler
 - uart2.c, [618](#)
- UART2_DefaultFramingErrorHandler
 - uart2.c, [618](#)
- UART2_DefaultOverrunErrorHandler
 - uart2.c, [618](#)
- UART2_ErrorHandler
 - uart2.c, [627](#)
- UART2_FramingErrorHandler
 - uart2.c, [627](#)
- UART2_get_last_status
 - uart2.c, [619](#)
 - uart2.h, [629](#)
- UART2_Initialize
 - uart2.c, [619](#)
 - uart2.h, [629](#)
- UART2_is_rx_ready
 - uart2.c, [620](#)
 - uart2.h, [630](#)
- UART2_is_tx_done
 - uart2.c, [621](#)
 - uart2.h, [631](#)
- UART2_is_tx_ready
 - uart2.c, [621](#)

- uart2.h, [631](#)
- UART2_OverrunErrorHandler
 - uart2.c, [627](#)
- UART2_Read
 - uart2.c, [622](#)
 - uart2.h, [632](#)
- UART2_Receive_ISR
 - uart2.c, [622](#)
 - uart2.h, [632](#)
- UART2_RX_BUFFER_SIZE
 - uart2.c, [617](#)
- UART2_RxDataHandler
 - uart2.c, [623](#)
 - uart2.h, [633](#)
- UART2_RxInterruptHandler
 - uart2.h, [637](#)
- UART2_SetErrorHandler
 - uart2.c, [623](#)
 - uart2.h, [633](#)
- UART2_SetFramingErrorHandler
 - uart2.c, [624](#)
 - uart2.h, [634](#)
- UART2_SetOverrunErrorHandler
 - uart2.c, [624](#)
 - uart2.h, [634](#)
- UART2_SetRxInterruptHandler
 - uart2.c, [625](#)
 - uart2.h, [635](#)
- UART2_SetTxInterruptHandler
 - uart2.c, [625](#)
 - uart2.h, [635](#)
- uart2_status_t, [127](#)
 - ferr, [128](#)
 - oerr, [128](#)
 - perr, [128](#)
 - reserved, [128](#)
 - status, [128](#)
- UART2_Transmit_ISR
 - uart2.c, [625](#)
 - uart2.h, [635](#)
- UART2_TX_BUFFER_SIZE
 - uart2.c, [617](#)
- UART2_TxInterruptHandler
 - uart2.h, [637](#)
- UART2_Write
 - uart2.c, [626](#)
 - uart2.h, [636](#)
- uart2RxCount
 - uart2.c, [627](#)
 - uart2.h, [637](#)
- uart2TxBufferRemaining
 - uart2.c, [627](#)
 - uart2.h, [637](#)
- uart5.c
 - __interrupt, [641](#)
 - INT_TRACE, [640](#)
 - IO_RB6_Toggle, [640](#)
 - TRACE, [640](#)
- UART5_DefaultErrorHandler, [641](#)
- UART5_DefaultFramingErrorHandler, [641](#)
- UART5_DefaultOverrunErrorHandler, [642](#)
- UART5_ErrorHandler, [650](#)
- UART5_FramingErrorHandler, [650](#)
- UART5_get_last_status, [642](#)
- UART5_Initialize, [642](#)
- UART5_is_rx_ready, [643](#)
- UART5_is_tx_done, [644](#)
- UART5_is_tx_ready, [644](#)
- UART5_OverrunErrorHandler, [650](#)
- UART5_Read, [645](#)
- UART5_Receive_ISR, [645](#)
- UART5_RX_BUFFER_SIZE, [641](#)
- UART5_RxDataHandler, [646](#)
- UART5_SetErrorHandler, [646](#)
- UART5_SetFramingErrorHandler, [647](#)
- UART5_SetOverrunErrorHandler, [647](#)
- UART5_SetRxInterruptHandler, [648](#)
- UART5_SetTxInterruptHandler, [648](#)
- UART5_Transmit_ISR, [649](#)
- UART5_TX_BUFFER_SIZE, [641](#)
- UART5_Write, [649](#)
- uart5RxCount, [650](#)
- uart5TxBufferRemaining, [650](#)
- uart5.h
 - UART5_DataReady, [652](#)
 - UART5_get_last_status, [652](#)
 - UART5_Initialize, [652](#)
 - UART5_is_rx_ready, [653](#)
 - UART5_is_tx_done, [654](#)
 - UART5_is_tx_ready, [654](#)
 - UART5_Read, [655](#)
 - UART5_Receive_ISR, [655](#)
 - UART5_RxDataHandler, [656](#)
 - UART5_RxInterruptHandler, [660](#)
 - UART5_SetErrorHandler, [656](#)
 - UART5_SetFramingErrorHandler, [657](#)
 - UART5_SetOverrunErrorHandler, [657](#)
 - UART5_SetRxInterruptHandler, [658](#)
 - UART5_SetTxInterruptHandler, [658](#)
 - UART5_Transmit_ISR, [659](#)
 - UART5_TxInterruptHandler, [660](#)
 - UART5_Write, [659](#)
 - uart5RxCount, [660](#)
 - uart5TxBufferRemaining, [660](#)
- UART5_DataReady
 - uart5.h, [652](#)
- UART5_DefaultErrorHandler
 - uart5.c, [641](#)
- UART5_DefaultFramingErrorHandler
 - uart5.c, [641](#)
- UART5_DefaultOverrunErrorHandler
 - uart5.c, [642](#)
- UART5_ErrorHandler
 - uart5.c, [650](#)
- UART5_FramingErrorHandler
 - uart5.c, [650](#)

- UART5_get_last_status
 - uart5.c, [642](#)
 - uart5.h, [652](#)
- UART5_Initialize
 - uart5.c, [642](#)
 - uart5.h, [652](#)
- UART5_is_rx_ready
 - uart5.c, [643](#)
 - uart5.h, [653](#)
- UART5_is_tx_done
 - uart5.c, [644](#)
 - uart5.h, [654](#)
- UART5_is_tx_ready
 - uart5.c, [644](#)
 - uart5.h, [654](#)
- UART5_OverrunErrorHandler
 - uart5.c, [650](#)
- UART5_Read
 - uart5.c, [645](#)
 - uart5.h, [655](#)
- UART5_Receive_ISR
 - uart5.c, [645](#)
 - uart5.h, [655](#)
- UART5_RX_BUFFER_SIZE
 - uart5.c, [641](#)
- UART5_RxDataHandler
 - uart5.c, [646](#)
 - uart5.h, [656](#)
- UART5_RxInterruptHandler
 - uart5.h, [660](#)
- UART5_SetErrorHandler
 - uart5.c, [646](#)
 - uart5.h, [656](#)
- UART5_SetFramingErrorHandler
 - uart5.c, [647](#)
 - uart5.h, [657](#)
- UART5_SetOverrunErrorHandler
 - uart5.c, [647](#)
 - uart5.h, [657](#)
- UART5_SetRxInterruptHandler
 - uart5.c, [648](#)
 - uart5.h, [658](#)
- UART5_SetTxInterruptHandler
 - uart5.c, [648](#)
 - uart5.h, [658](#)
- uart5_status_t, [128](#)
 - ferr, [129](#)
 - oerr, [129](#)
 - perr, [129](#)
 - reserved, [129](#)
 - status, [129](#)
- UART5_Transmit_ISR
 - uart5.c, [649](#)
 - uart5.h, [659](#)
- UART5_TX_BUFFER_SIZE
 - uart5.c, [641](#)
- UART5_TxInterruptHandler
 - uart5.h, [660](#)
- UART5_Write
 - uart5.c, [649](#)
 - uart5.h, [659](#)
- uart5RxCount
 - uart5.c, [650](#)
 - uart5.h, [660](#)
- uart5TxBufferRemaining
 - uart5.c, [650](#)
 - uart5.h, [660](#)
- unsuback
 - mqtt_response, [108](#)
- update_time
 - batmon.c, [137](#)
 - batmon.h, [148](#)
- USE_FD
 - canfd.h, [164](#)
- USE_LCD_DMA
 - eadog.h, [196](#)
- USEMCC_SPI
 - eadog.h, [196](#)
- Utilities, [33](#)
 - __mqtt_next_pid, [35](#)
 - __mqtt_puback, [35](#)
 - __mqtt_pubcomp, [35](#)
 - __mqtt_pubrec, [36](#)
 - __mqtt_pubrel, [36](#)
 - __mqtt_recv, [36](#)
 - __mqtt_send, [37](#)
 - mqtt_mq_clean, [37](#)
 - mqtt_mq_currsh, [34](#)
 - mqtt_mq_find, [37](#)
 - mqtt_mq_get, [34](#)
 - mqtt_mq_init, [38](#)
 - mqtt_mq_length, [34](#)
 - mqtt_mq_register, [38](#)
 - MQTT_QUEUED_AWAITING_ACK, [35](#)
 - MQTT_QUEUED_COMPLETE, [35](#)
 - MQTT_QUEUED_UNSENT, [35](#)
 - MQTTQueuedMessageState, [34](#)
- V
 - modbus_master.c, [688](#)
 - modbus_master.h, [705](#)
- V_type, [129](#)
 - dmt_sosc_flag, [130](#)
 - fault_active, [130](#)
 - fault_count, [130](#)
 - fault_source, [130](#)
 - fault_ticks, [130](#)
 - modbus_rx, [130](#)
 - modbus_tx, [130](#)
 - motor_speed, [130](#)
 - pacing, [131](#)
 - pwm_stop, [131](#)
 - pwm_update, [131](#)
 - StartTime, [131](#)
 - TimeUsed, [131](#)
- val1
 - EM_data1, [85](#)

- val2
 - EM_data1, [85](#)
- val3
 - EM_data1, [85](#)
- value
 - MREG, [118](#)
 - MREG32, [119](#)
 - MREG64, [119](#)
- var1
 - EM_data1, [85](#)
- var12
 - EM_data1, [85](#)
- var13
 - EM_data1, [86](#)
- varsys
 - EM_data1, [86](#)
- vasys
 - EM_data1, [86](#)
- VER
 - mxcmd.h, [669](#)
- version
 - EB_data, [83](#)
- version_ok
 - C_data, [75](#)
- vf
 - main.c, [282](#)
- vl1l2
 - EM_data1, [86](#)
- vl1n
 - EM_data1, [86](#)
- vl2l3
 - EM_data1, [86](#)
- vl2n
 - EM_data1, [86](#)
- vl3l1
 - EM_data1, [86](#)
- vl3n
 - EM_data1, [86](#)
- vllsys
 - EM_data1, [86](#)
- vlnsys
 - EM_data1, [87](#)
- volt_fract
 - main.c, [282](#)
 - qconfig.h, [713](#)
- volt_whole
 - EB_data, [83](#)
 - main.c, [282](#)
 - qconfig.h, [713](#)
- volts_peak
 - mx_logpage_t, [121](#)
- Volts_to_SOC
 - batmon.c, [137](#)
 - batmon.h, [148](#)
- vw
 - main.c, [282](#)
 - qconfig.h, [713](#)
- wait_lcd_check
 - eadog.c, [188](#)
 - eadog.h, [206](#)
- wait_lcd_done
 - eadog.c, [188](#)
 - eadog.h, [206](#)
- wait_lcd_set
 - eadog.c, [188](#)
 - eadog.h, [207](#)
- WaitMs
 - timers.c, [723](#)
 - timers.h, [727](#)
- waterral
 - raw.githubusercontent.com_linux-can_can-utils_master_canfdtest.c, [268](#)
- wtddelay
 - eadog.h, [207](#)
 - main.c, [279](#)
 - mxcmd.h, [672](#)
- wl1
 - EM_data1, [87](#)
- wl2
 - EM_data1, [87](#)
- wl3
 - EM_data1, [87](#)
- Word
 - canfd.c, [160](#)
- wr_bm_data
 - batmon.c, [137](#)
 - batmon.h, [149](#)
- WRITE_MULTIPLE_COILS
 - modbus_master.h, [695](#)
- WRITE_MULTIPLE_REGISTERS
 - modbus_master.h, [695](#)
- WRITE_SINGLE_COIL
 - modbus_master.h, [695](#)
- WRITE_SINGLE_REGISTER
 - modbus_master.h, [695](#)
- wsys
 - EM_data1, [87](#)
- year
 - EM_serial, [90](#)